

计算机组成原理

第3版

唐朔飞



高等教育出版社

高等教育电子音像出版社

为配合由高等教育出版社出版的面向21世纪课程教材《计算机组成原理》教学和自学的需要，随书出版了《计算机组成原理》配套课件。为了配合该教材的第3版，本课件在保留原课件特色的基础上，做了相应的补充和修改。

该课件与《计算机组成原理》第3版教材在体系上完全一致，它以课堂教学为依托，帮助读者边阅读边思考，通过点击鼠标，逐行显示精练的文字和简明的图表，既可从文稿中对教材的重点和难点加深理解，又可从视图中看到动画演示效果，形象地理解各种电路的工作原理和设计思路。

新版课件提供了章节目录和节目录。操作上除了顺序播放外，还可任选一章、任选一节播放，并可从任意一页幻灯片返回到上一级目录。具体功能如下。

1. 进入章节目录后，可点击任一章播放。

2. 进入节目录后，可点击任一节播放。

3. 每一节播放结束时，可继续播放，也可点击本节最后一页幻灯片右下角的按钮，回到上一级目录，再点击节目录幻灯片右下角的按钮，即可回到章节目录。

4. 点击任一页幻灯片右下角的按钮，均可回到上一级目录。

在修改课件过程中，哈尔滨工业大学计算机科学与技术学院张丽杰、罗丹彦为课件的录入、排版、绘图、动画演示做了大量工作，在此表示衷心感谢。

由于时间仓促，难免有不妥之处，敬请读者和专家批评指正。

唐朔飞

2020年05月

第1章 计算机系统概论

第2章 计算机的发展及应用

第3章 系统总线

第4章 存储器

第5章 输入输出系统

第6章 计算机的运算方法

第7章 指令系统

第8章 CPU 的结构和功能

第9章 控制单元的功能

第10章 控制单元的设计



第1章 计算机系统概论

1.1 计算机系统简介

1.2 计算机的基本组成

1.3 计算机硬件的主要技术指标

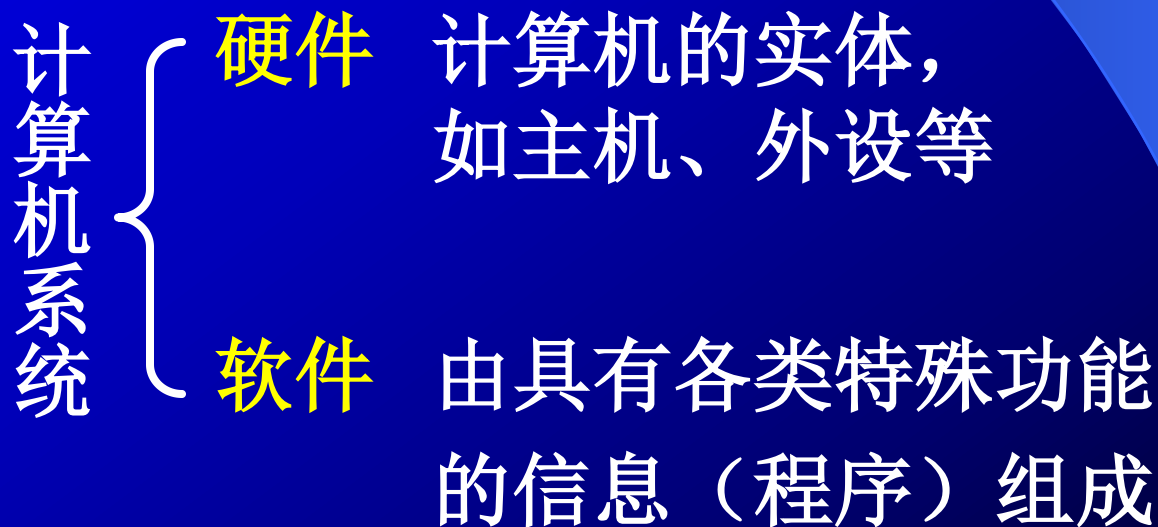
1.4 本书结构



1.1 计算机系统简介

一、计算机的软硬件概念

1. 计算机系统



软件

系统软件 用来管理整个计算机系统

语言处理程序

操作系统

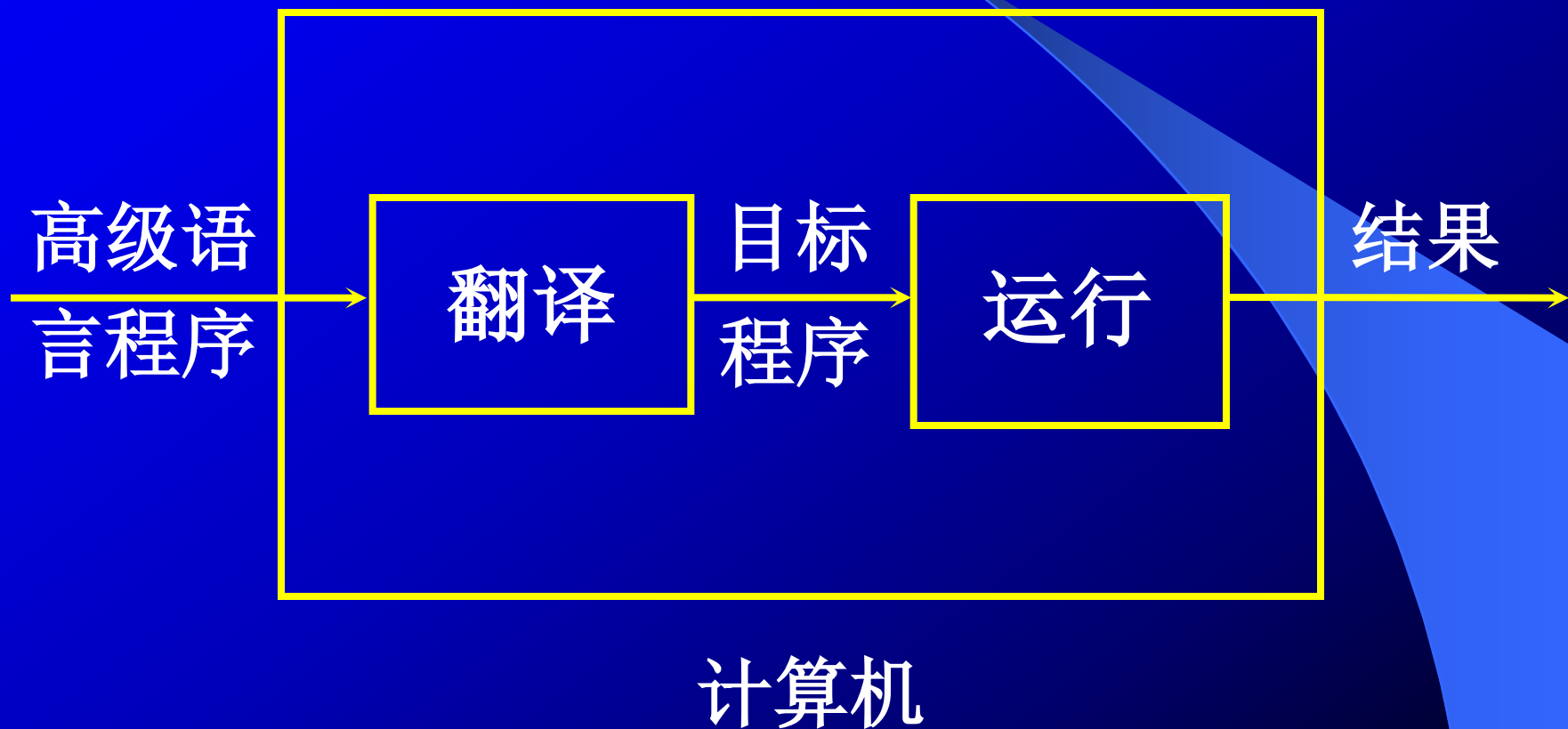
服务性程序

数据库管理系统

网络软件

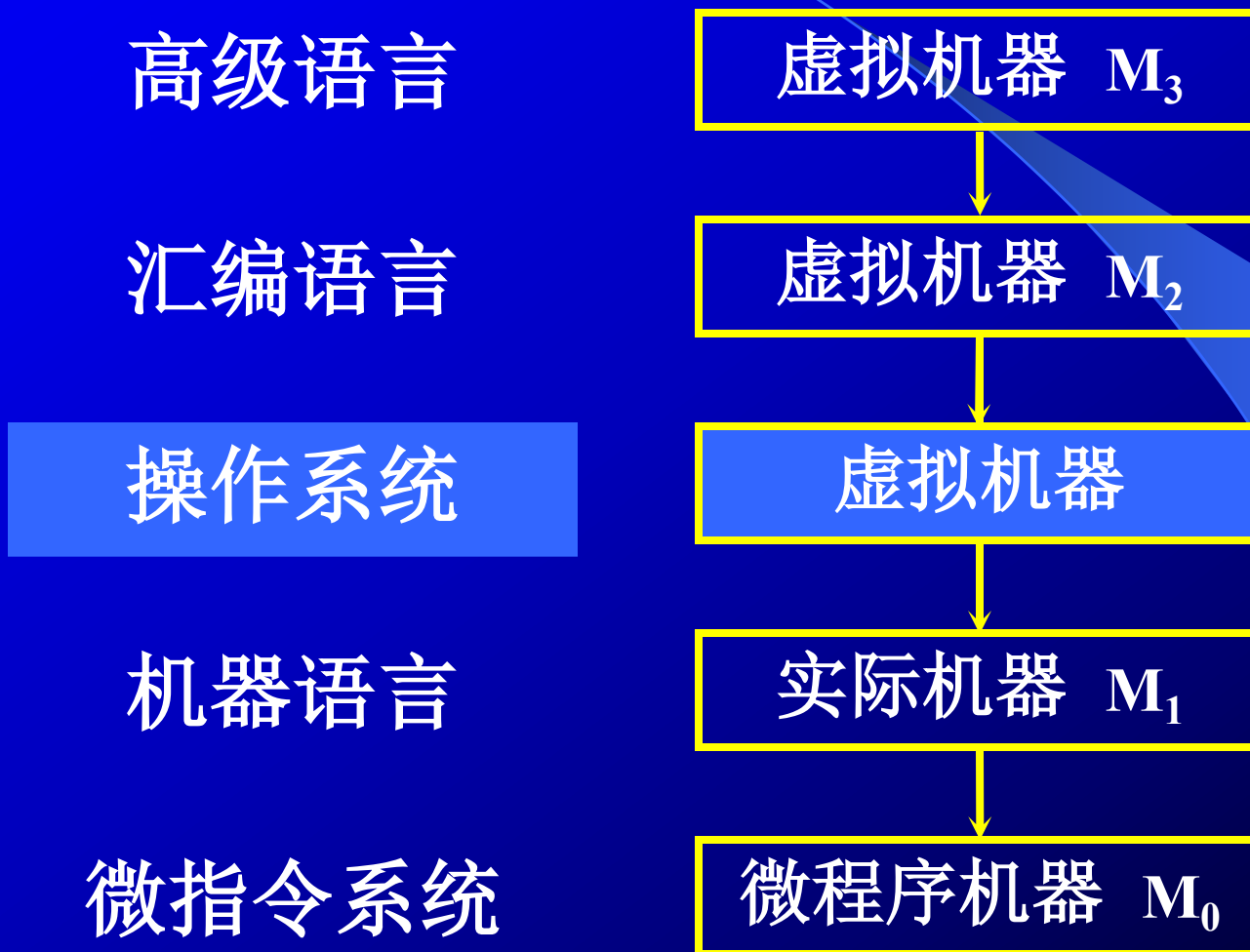
应用软件 按任务需要编制成的各种程序

2. 计算机的解题过程



二、计算机系统的层次结构

1.1



软件

虚拟机 M_4

用编译程序翻译成汇编语言程序

虚拟机 M_3

用汇编程序翻译成机器语言程序

虚拟机 M_2

用机器语言解释操作系统

硬件

实际机器 M_1

用微指令解释机器指令

微程序机器 M_0

由硬件直接执行微指令



三、计算机体系结构和计算机组成

1.1

计算机 体系结构

有无乘法指令

程序员所见到的计算机系统的属性
概念性的结构与功能特性

（指令系统、数据类型、寻址技术、I/O机理）

计算机 组成

实现计算机体系结构所体现的属性
（具体指令的实现）

如何实现乘法指令



1.2 计算机的基本组成

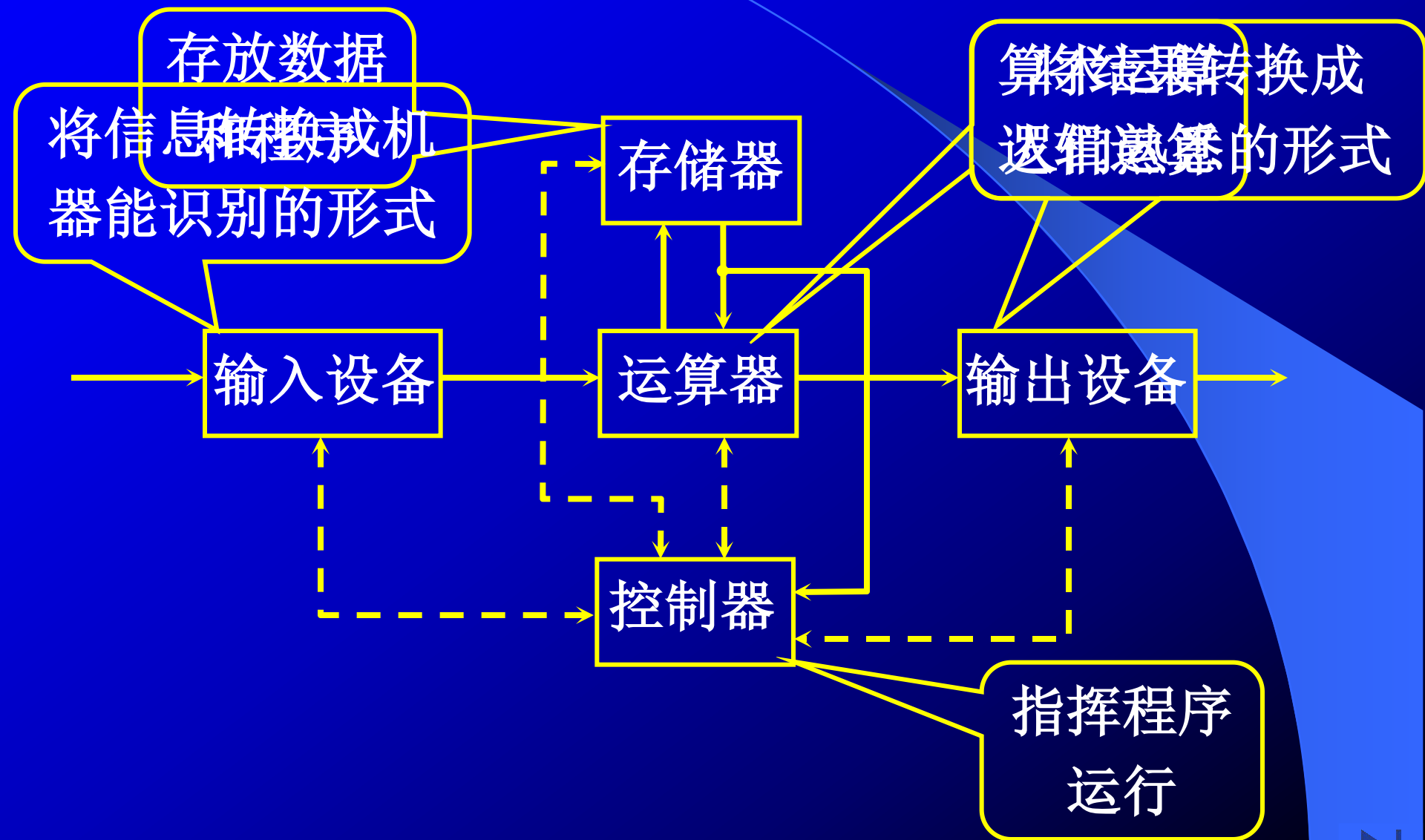
一、冯·诺依曼计算机的特点

1. 计算机由五大部件组成
2. 指令和数据以同等地位存于存储器，可按地址寻访
3. 指令和数据用二进制表示
4. 指令由操作码和地址码组成
5. 存储程序
6. 以运算器为中心



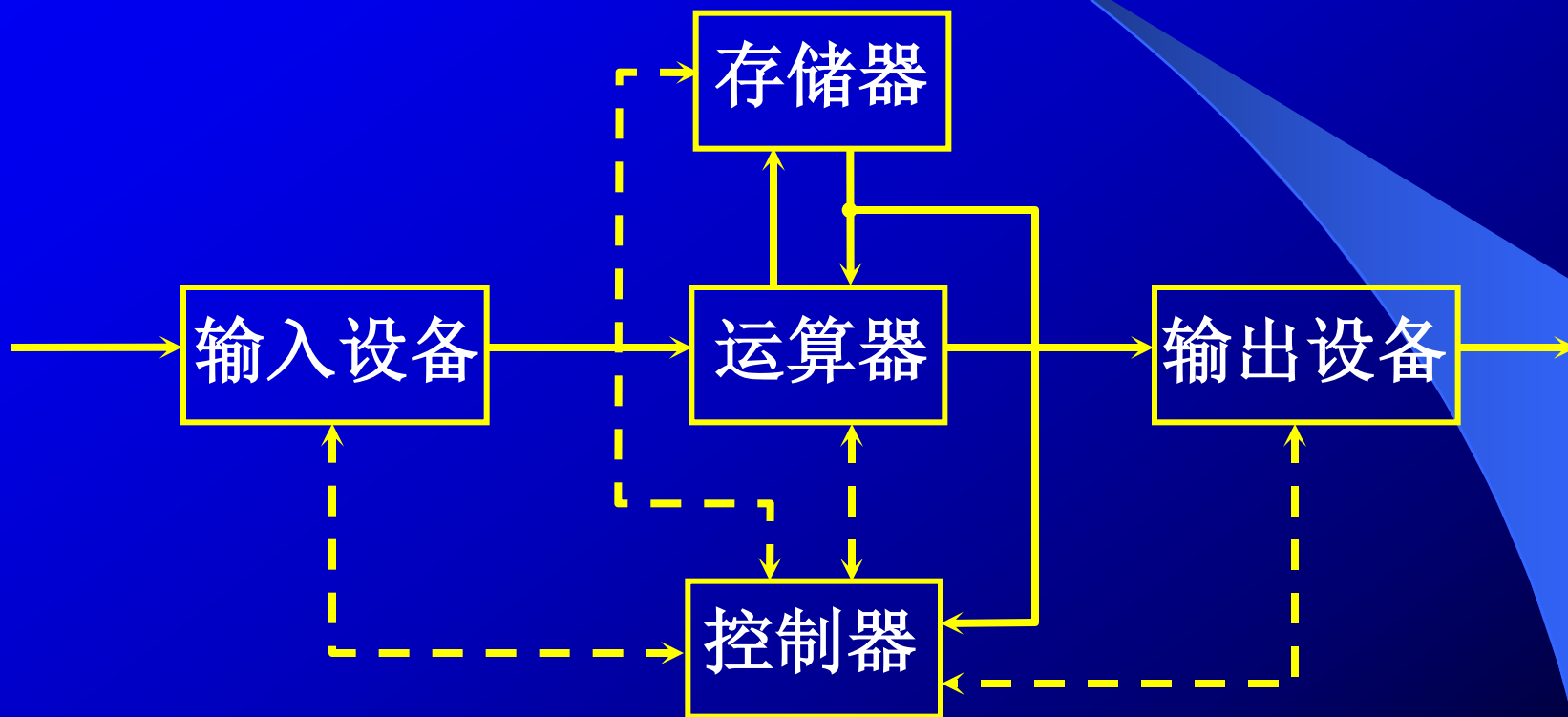
冯·诺依曼计算机硬件框图

1.2



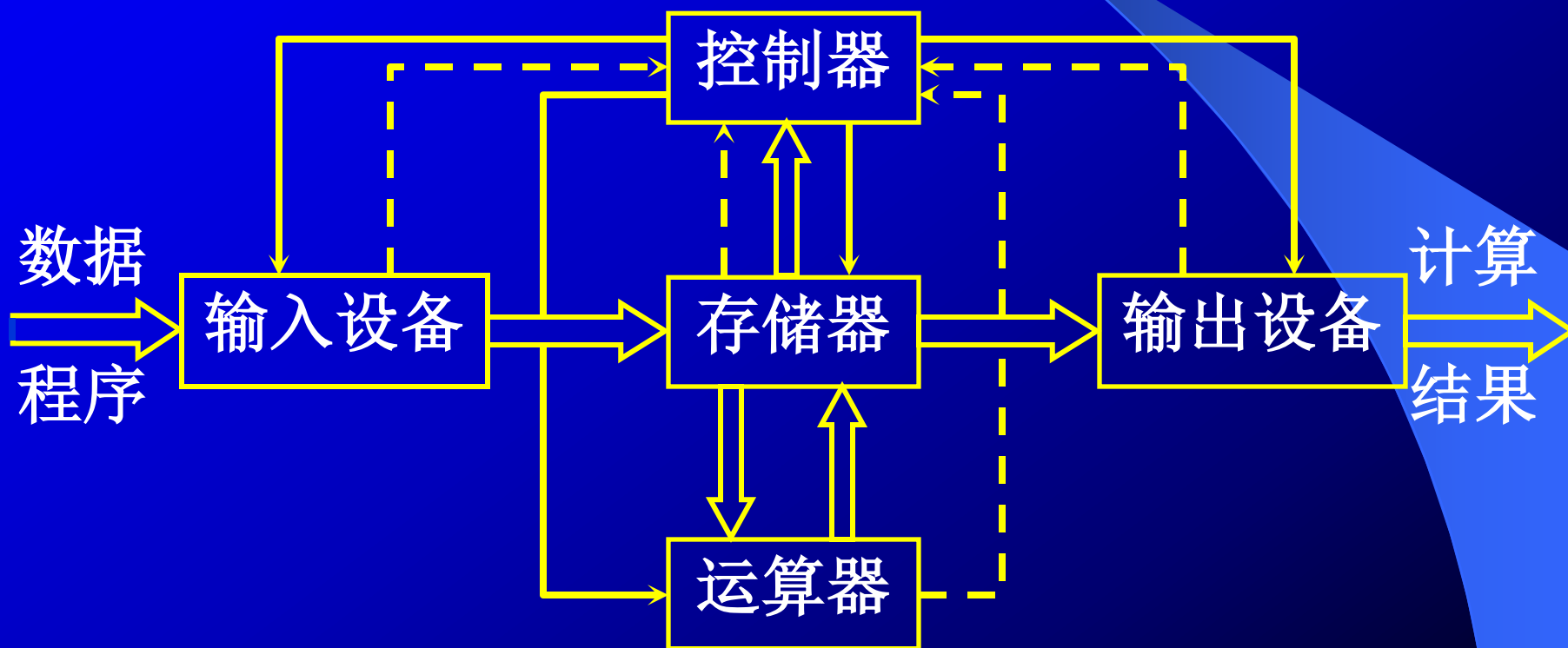
冯·诺依曼计算机硬件框图

1.2

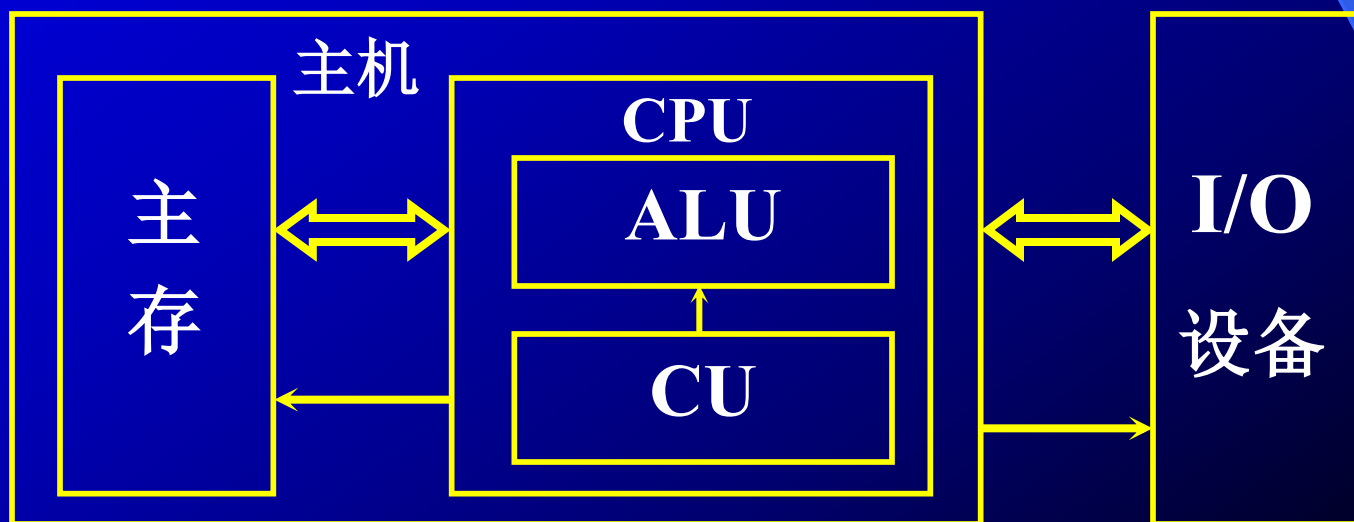


二、计算机硬件框图

1. 以存储器为中心的计算机硬件框图



2. 现代计算机硬件框图



三、计算机的工作步骤

1.2

1. 上机前的准备

- 建立数学模型
- 确定计算方法

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \dots$$

$$\sqrt{x} = \frac{1}{2} \left(y_n + \frac{x}{y_n} \right) \quad (n = 0, 1, 2, \dots)$$

- 编制解题程序

程序 —— 运算的 全部步骤

指令 —— 每 一个步骤



编程举例

1.2

计算 $ax^2 + bx + c = (ax + b)x + c$

取 x 至运算器中

乘以 x 在运算器中

乘以 a 在运算器中

存 ax^2 在存储器中

取 b 至运算器中

乘以 x 在运算器中

加 ax^2 在运算器中

加 c 在运算器中

取 x 至运算器中

乘以 a 在运算器中

加 b 在运算器中

乘以 x 在运算器中

加 c 在运算器中



指令格式举例

操作码	地址码
-----	-----

取数

 α $[\alpha] \rightarrow \text{ACC}$

000001

0000001000

存数

 β $[\text{ACC}] \rightarrow \beta$

加

 γ $[\text{ACC}] + [\gamma] \rightarrow \text{ACC}$

乘

 δ $[\text{ACC}] \times [\delta] \rightarrow \text{ACC}$

打印

 σ $[\sigma] \rightarrow \text{打印机}$

停机

计算 $ax^2 + bx + c$ 程序清单

1.2

指令和数据存于主存单元的地址	指令		注释
	操作码	地址码	
0	000001	0000001000	取数 x 至ACC
1	000100	0000001001	乘 a 得 ax ,存于ACC中
2	000011	0000001010	加 b 得 $ax+b$,存于ACC中
3	000100	0000001000	乘 x 得 $(ax+b)x$,存于ACC中
4	000011	0000001011	加 c 得 $ax^2 + bx + c$,存于ACC
5	000010	0000001100	将 $ax^2 + bx + c$,存于主存单元
6	000101	0000001100	打印
7	000110		停机
8	x		原始数据 x
9	a		原始数据 a
10	b		原始数据 b
11	c		原始数据 c
12			存放结果

2. 计算机的解题过程

1.2

(1) 存储器的基本组成



存储体 – 存储单元 – 存储元件 (0/1)

大楼 – 房间 – 床位 (无人/有人)

存储单元 存放一串二进制代码

存储字 存储单元中二进制代码的组合

存储字长 存储单元中二进制代码的位数

每个存储单元赋予一个地址号

按地址寻访



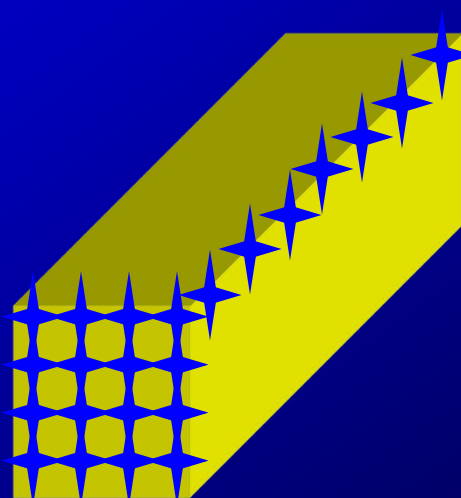
(1) 存储器的基本组成



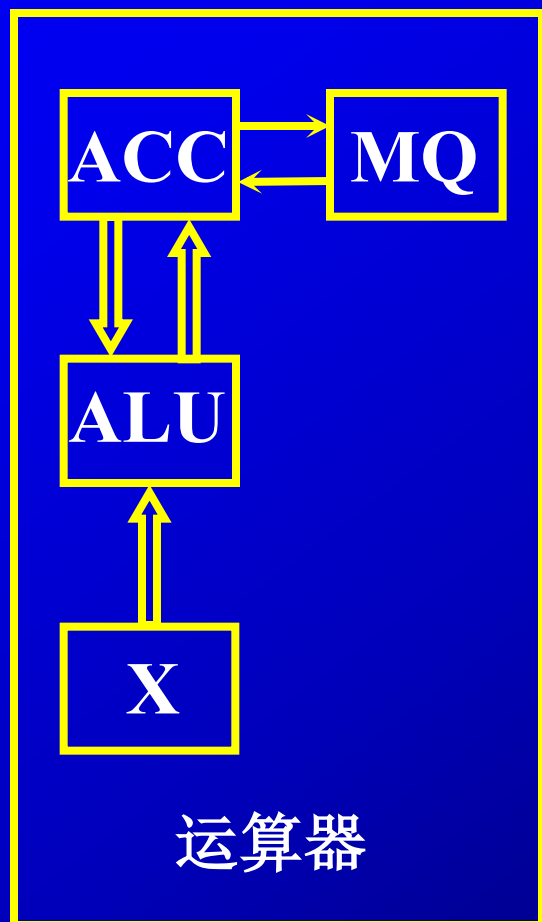
MAR 存储器地址寄存器
反映存储单元的个数

MDR 存储器数据寄存器
反映存储字长

设 MAR=4 位
MDR=8 位
存储单元个数 16
存储字长 8

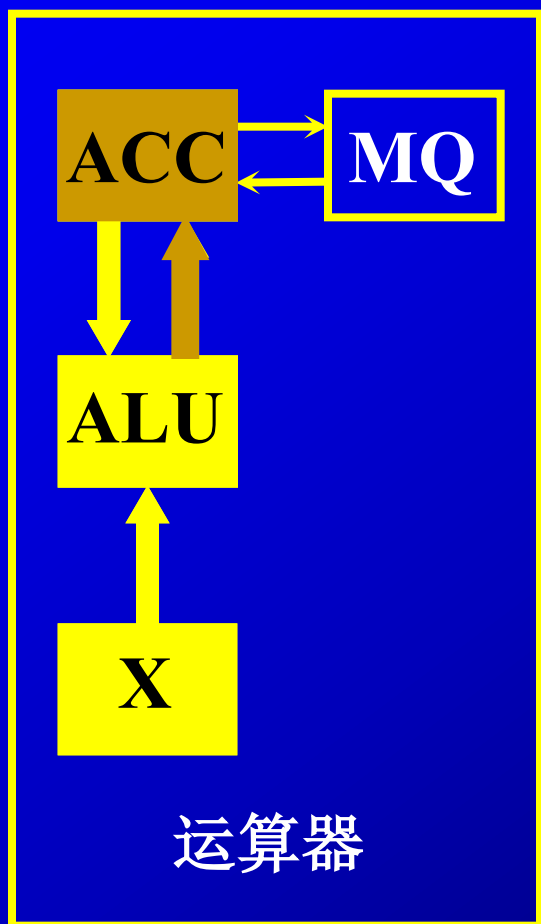


(2)运算器的基本组成及操作过程



	ACC	MQ	X
加法	被加数 和		加数
减法	被减数 差		减数
乘法	乘积高位	乘数 乘积低位	被乘数
除法	被除数 余数	商	除数

① 加法操作过程



指令

加

M

初态

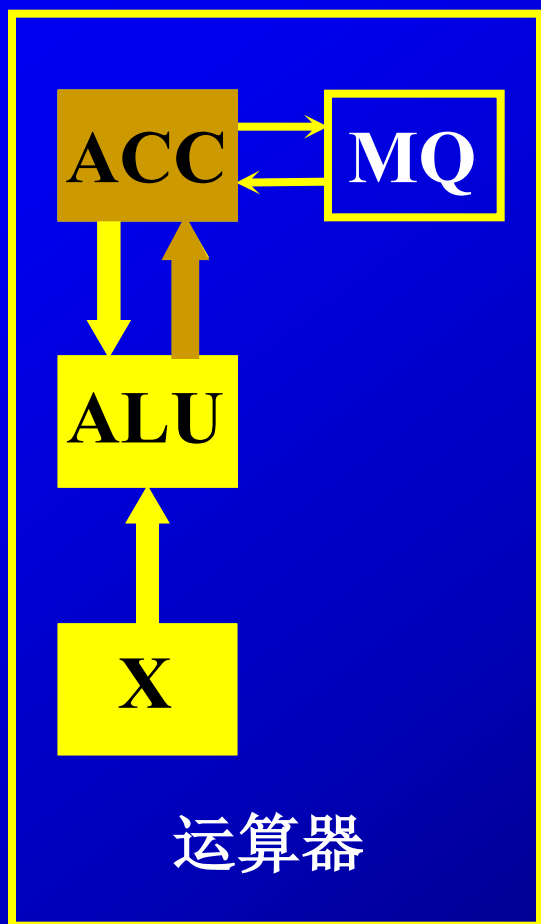
ACC

被加数

$[M] \longrightarrow X$

$[ACC] + [X] \longrightarrow ACC$

② 减法操作过程



指令

减

M

初态

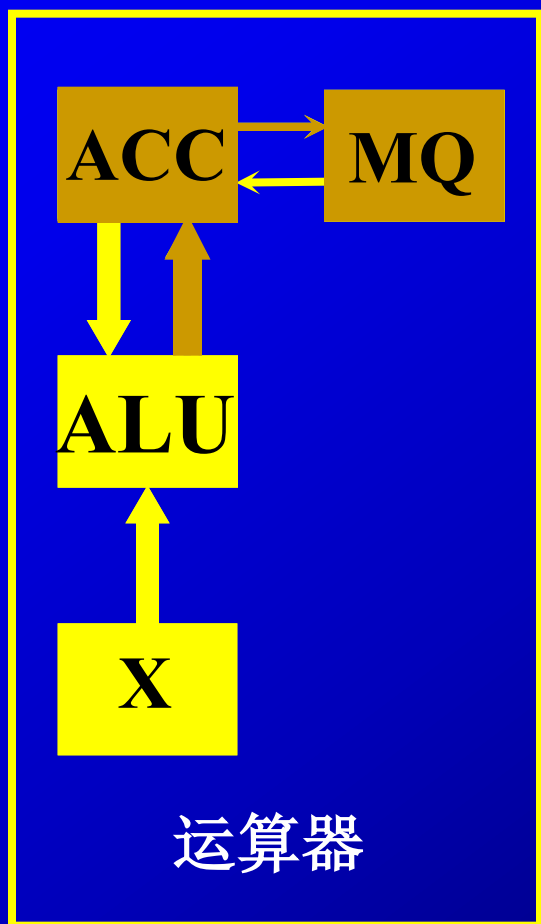
ACC

被减数

$[M] \longrightarrow X$

$[ACC] - [X] \longrightarrow ACC$

③ 乘法操作过程



指令

乘

M

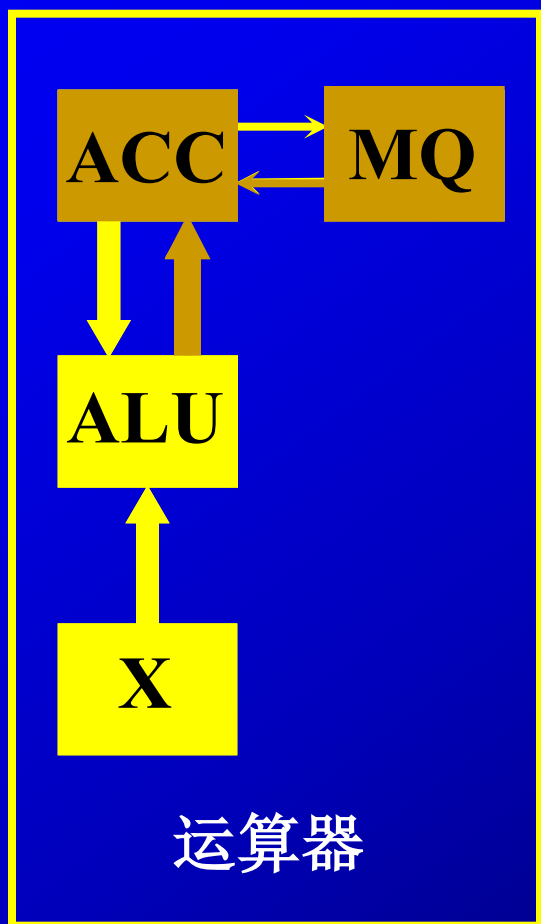
初态

ACC

被乘数

[M] $\longrightarrow$ MQ[ACC] $\longrightarrow$ X0 $\longrightarrow$ ACC[X] $\times$ [MQ] $\longrightarrow$ ACC // MQ

④ 除法操作过程



指令

除

M

初态

ACC

被除数

$[M] \longrightarrow X$

$[ACC] \div [X] \longrightarrow MQ$

余数在ACC中

(3) 控制器的基本组成

1.2



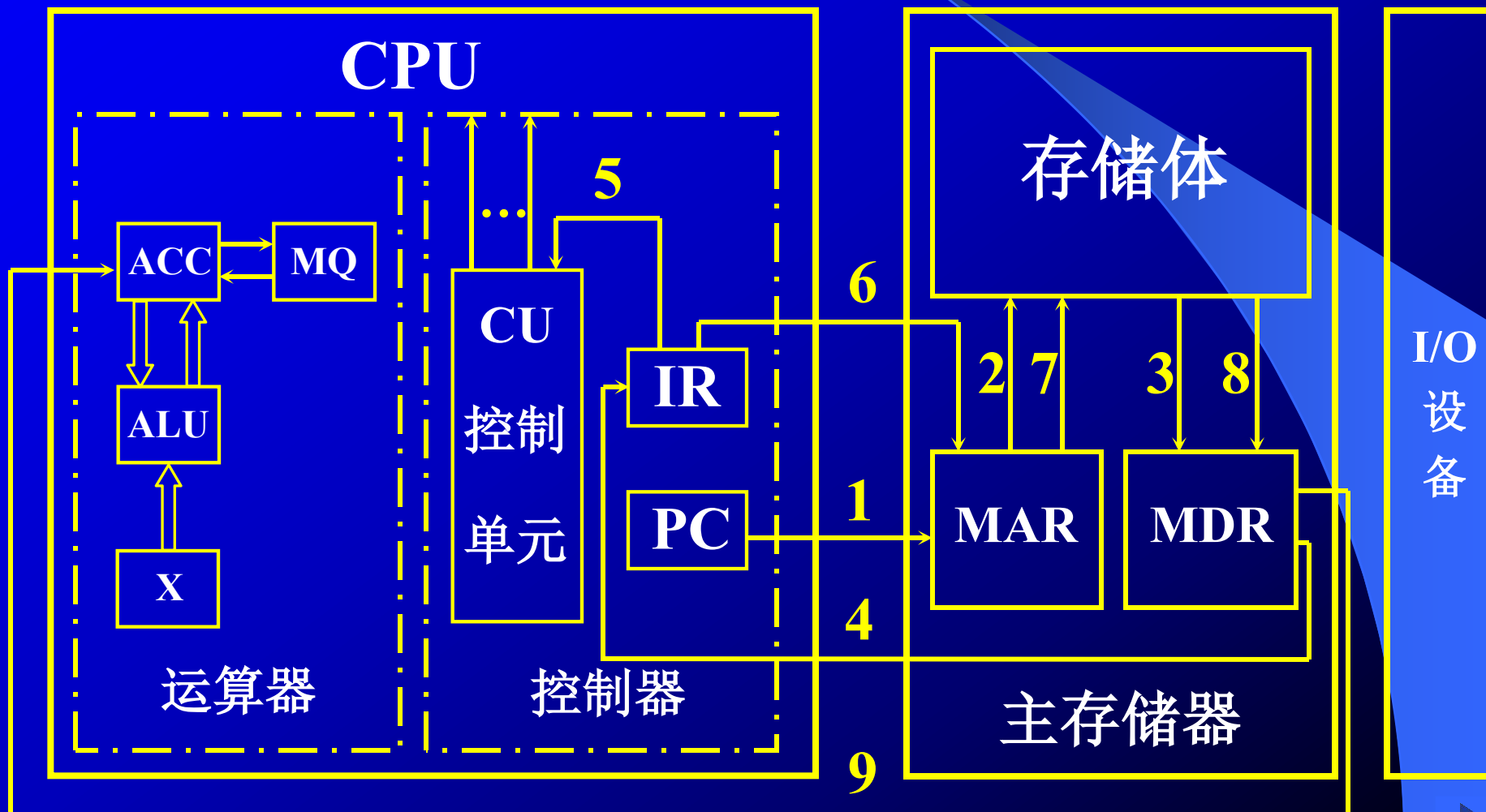
PC 存放当前欲执行指令的地址，
具有计数功能 $(PC) + 1 \rightarrow PC$

IR 存放当前欲执行的指令

CU 控制单元

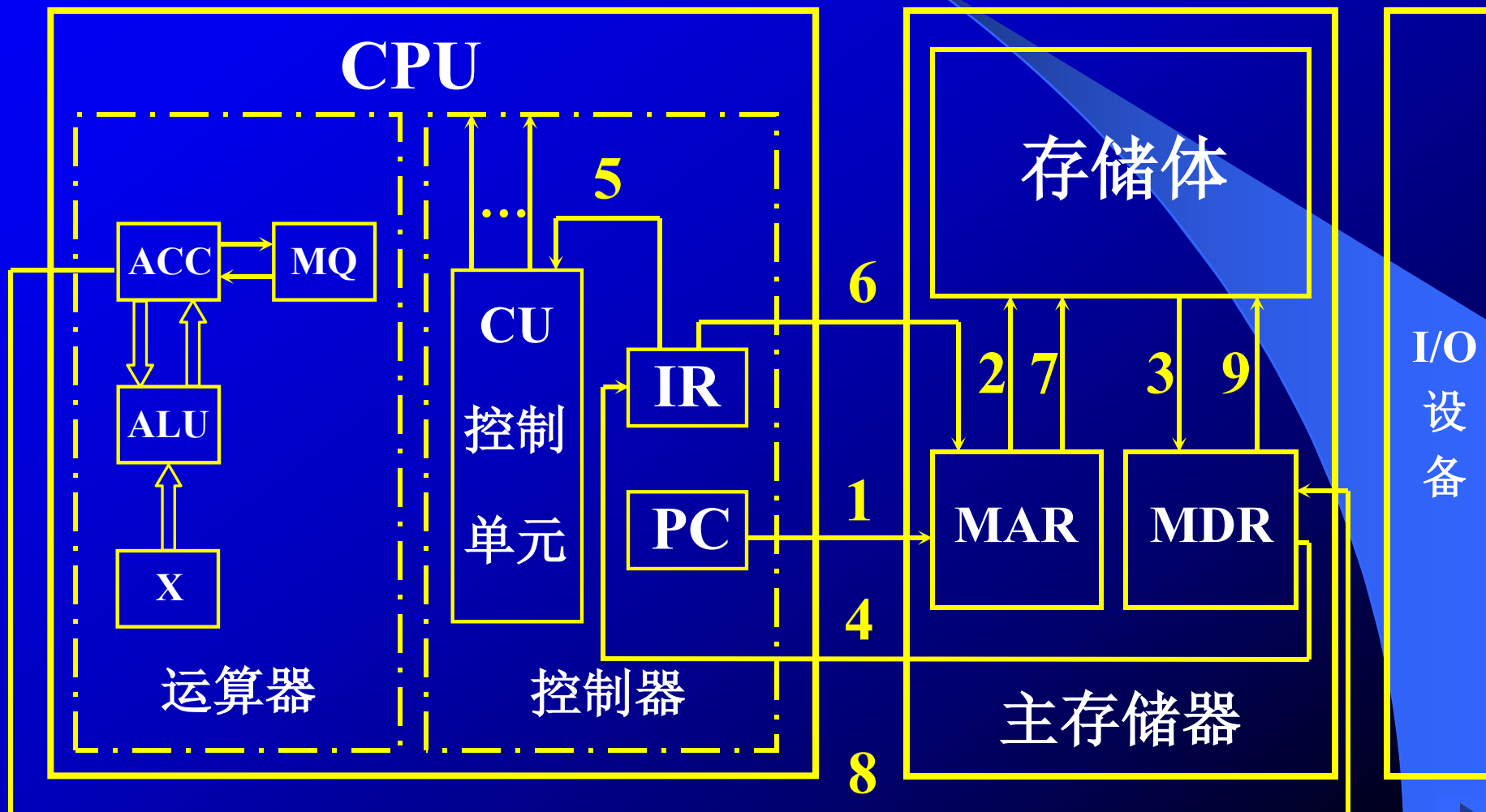
(4) 主机完成一条指令的过程

以取数指令为例



(4) 主机完成一条指令的过程

以存数指令为例



(5) $ax^2 + bx + c$ 程序的运行过程

- 将程序通过输入设备送至计算机
- 程序首地址 $\longrightarrow$ PC
- 启动程序运行
- 取指令 $PC \longrightarrow MAR \longrightarrow M \longrightarrow MDR \longrightarrow IR$, $(PC) + 1 \longrightarrow PC$
- 分析指令 $OP(IR) \longrightarrow CU$
- 执行指令 $Ad(IR) \longrightarrow MAR \longrightarrow M \longrightarrow MDR \longrightarrow ACC$
- $\vdots$
- 打印结果
- 停机

1.3 计算机硬件的主要技术指标

1. 机器字长 CPU 一次能处理数据的位数
与 CPU 中的 寄存器位数 有关

2. 运算速度 { 主频
吉普森法 $T_M = \sum_{i=1}^n f_i t_i$
MIPS 每秒执行百万条指令
CPI 执行一条指令所需时钟周期数
FLOPS 每秒浮点运算次数



3. 存储容量 存放二进制信息的总位数

存储单元个数 $\times$ 存储字长

主存容量

如 MAR MDR 容量

10 8 1 K $\times$ 8位

16 32 64 K $\times$ 32位

字节数

$$1\text{K} = 2^{10}$$

如

$$2^{13} = 1 \text{ KB}$$

$$1\text{B} = 2^3\text{b}$$

$$2^{21} = 256 \text{ KB}$$

辅存容量

字节数

80 GB

$$1\text{GB} = 2^{30}\text{b}$$

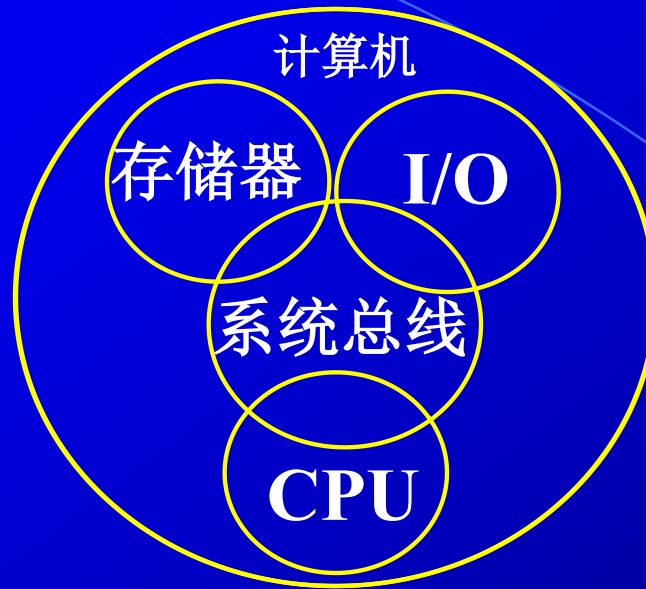
1.4 本书结构

计算机

第 1 篇 概论



1.4 本书结构



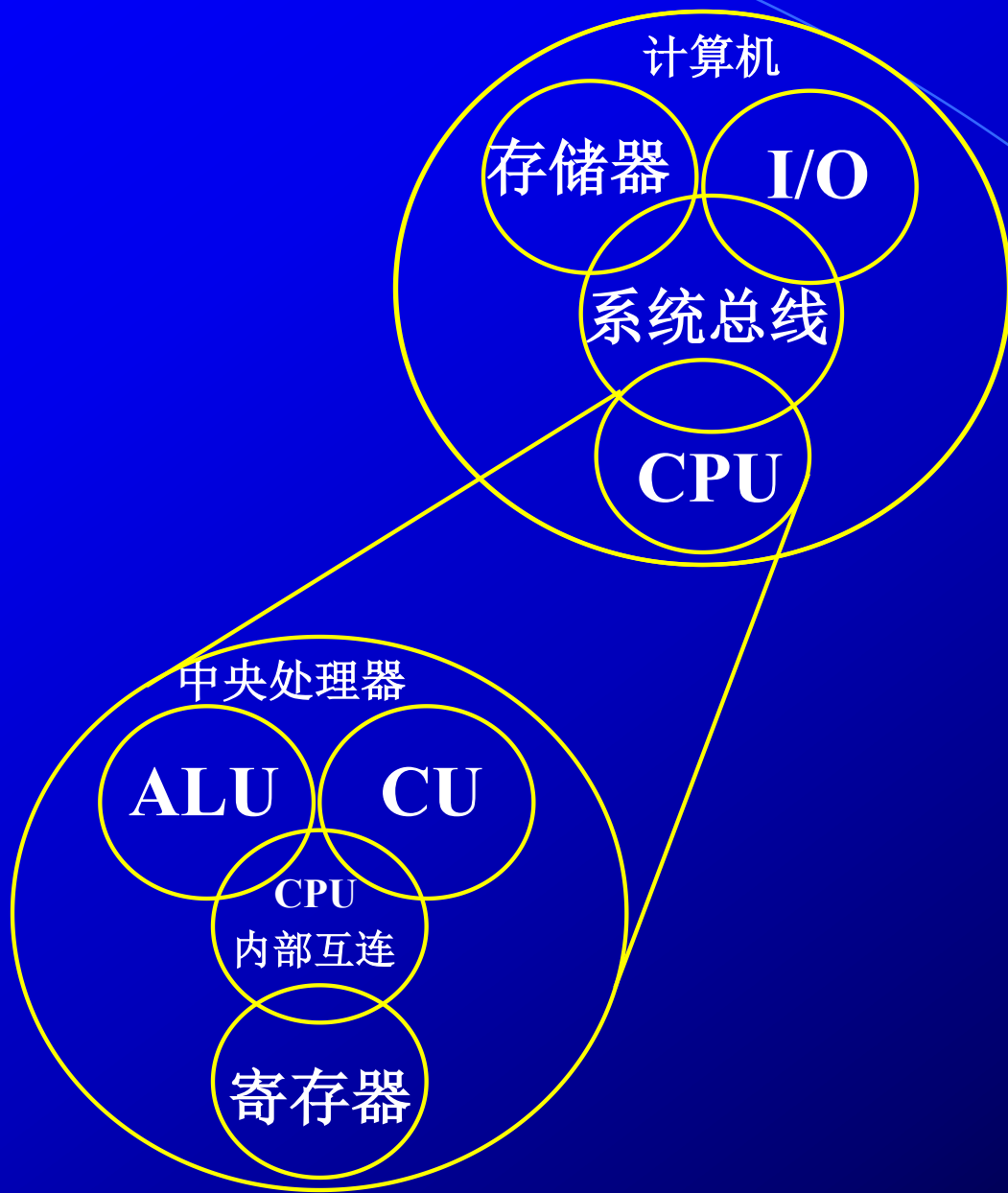
第 2 篇

计算机系统的硬件结构



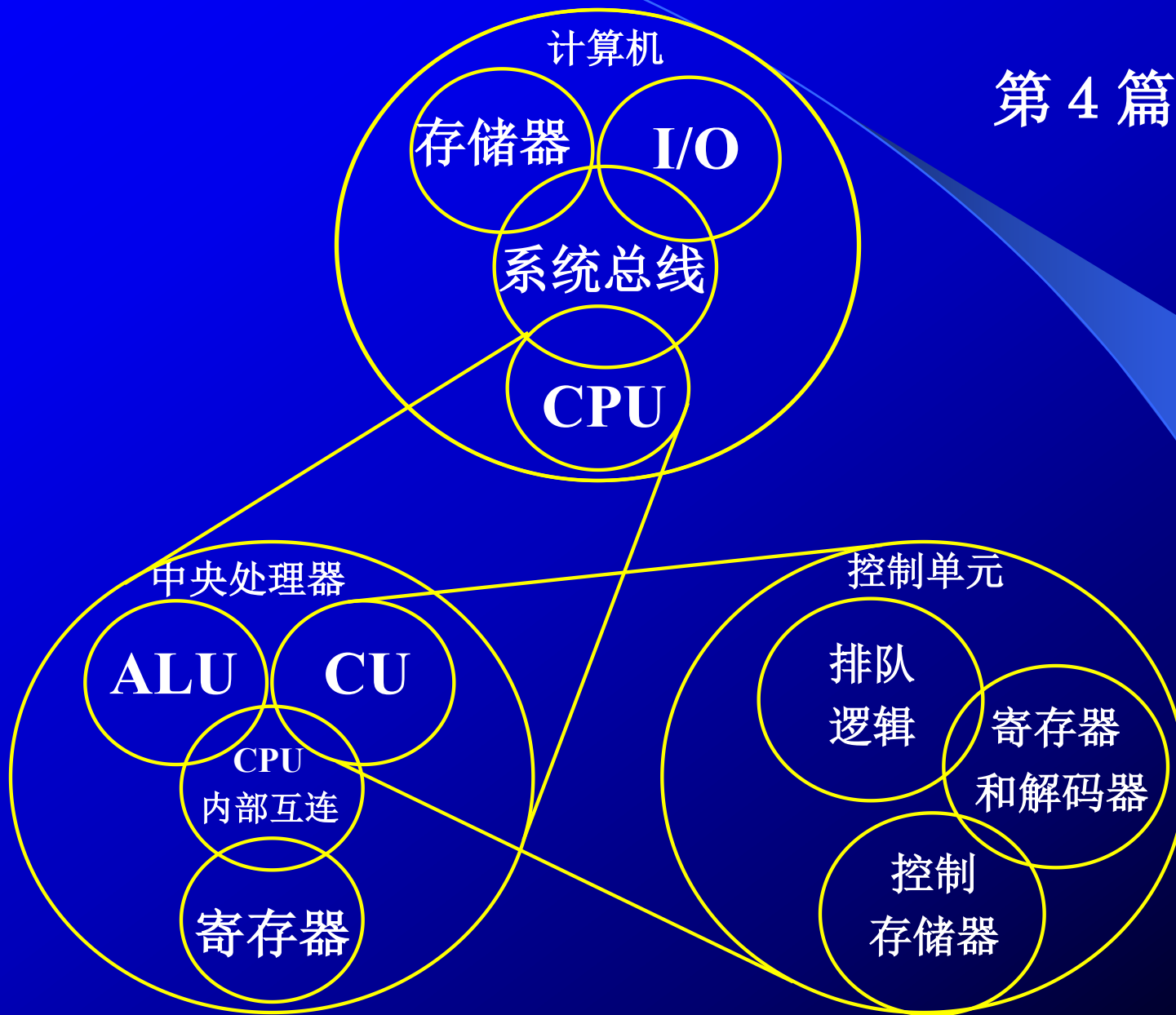
1.4 本书结构

第3篇 CPU



1.4 本书结构

第4篇 CU



第 2 章 计算机的发展及应用

2.1 计算机的发展史

2.2 计算机的应用

2.3 计算机的展望



2.1 计算机的发展史

一、计算机的产生和发展

1946年 美国 ENIAC 1955年退役
十进制运算

18 000 多个电子管

1 500 多个继电器

150 千瓦

30 吨

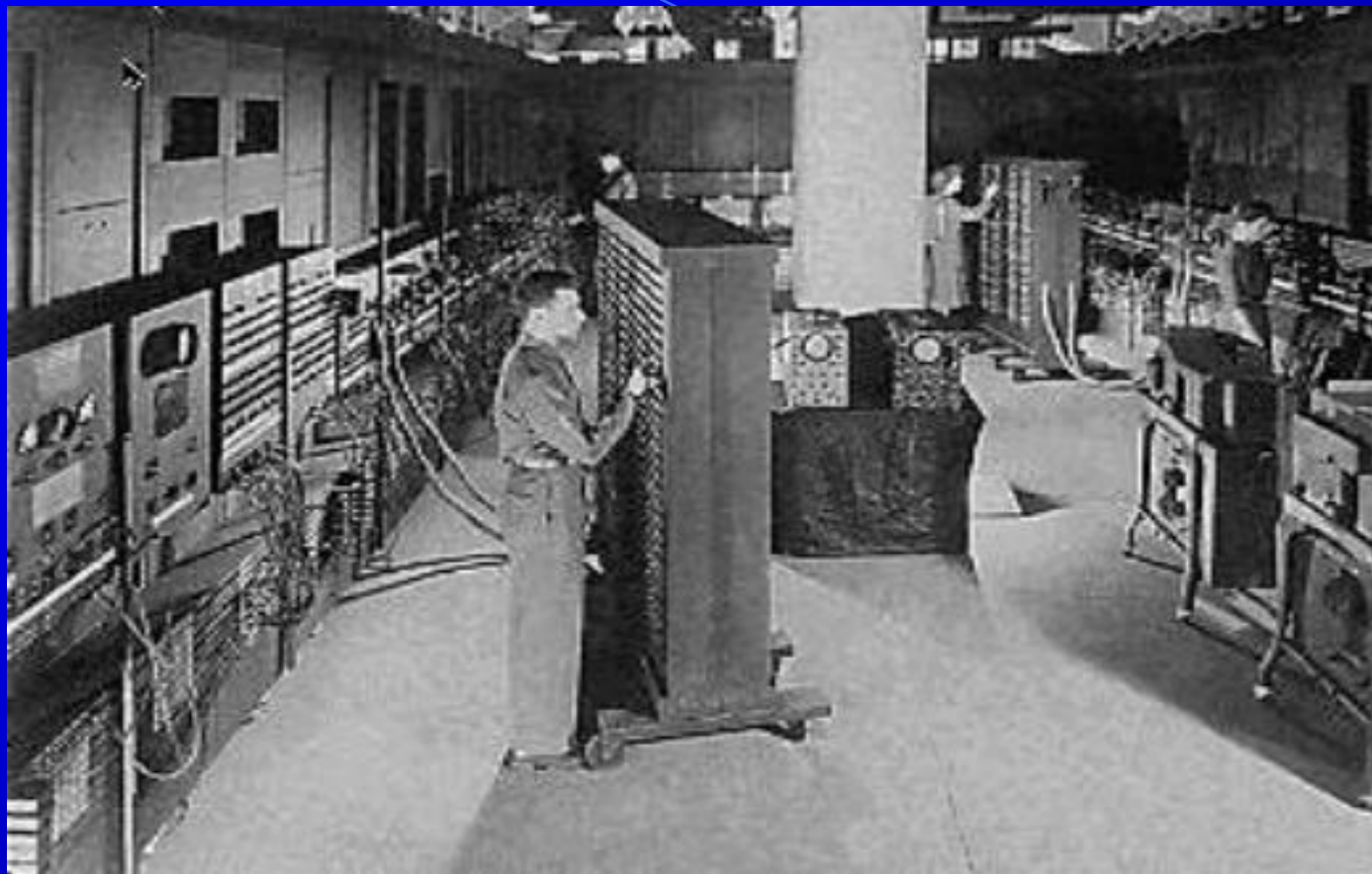
1 500 平方英尺

5 000 次加法 / 秒

用手工搬动开关和拔插电缆来编程



2.1



世界上第一台电子计算机 ENIAC(1946)

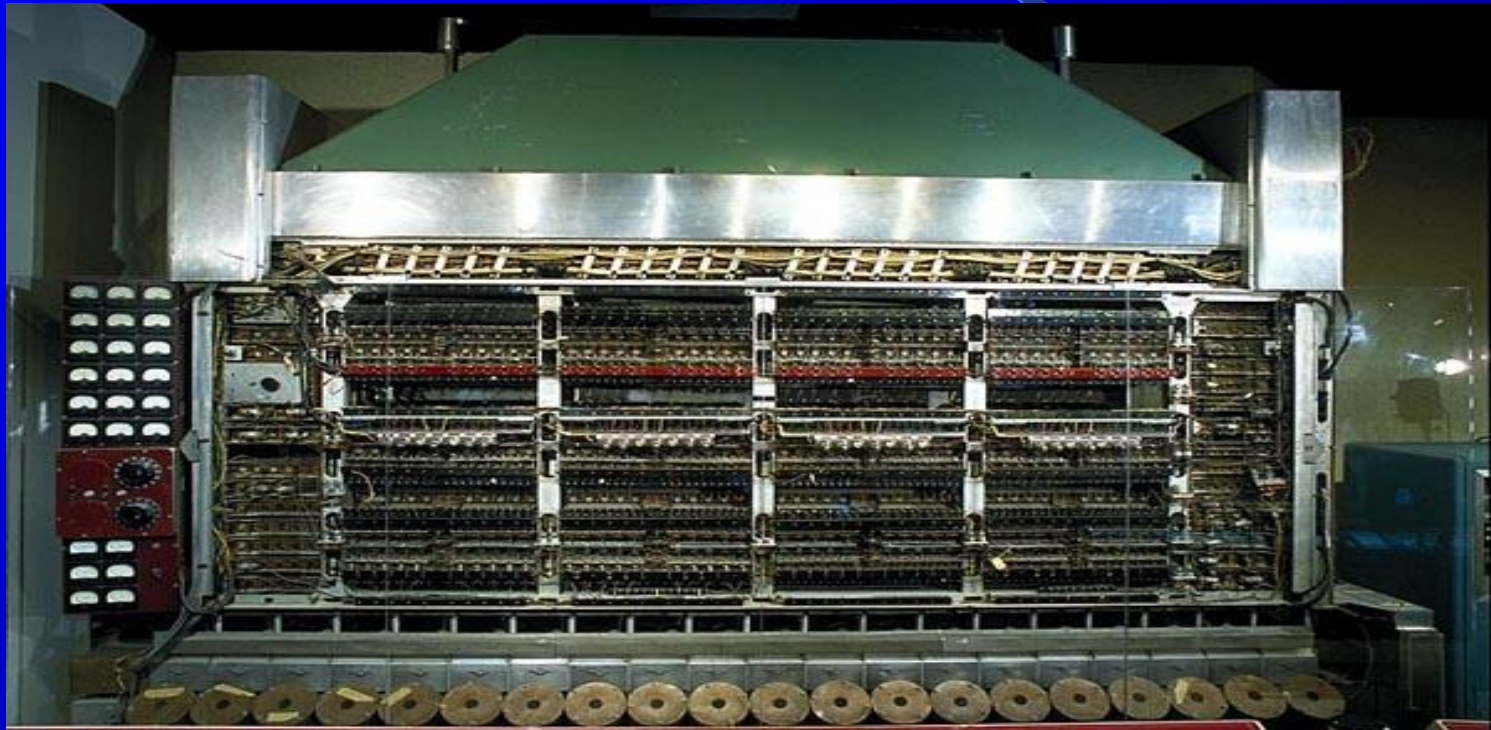
硬件技术对计算机更新换代的影响

2.1

代	时间	硬件技术	速度 / (次/秒)
一	1946—1957	电子管	40 000
二	1958—1964	晶体管	200 000
三	1965—1971	中小规模 集成电路	1 000 000
四	1972—1977	大规模 集成电路	10 000 000
五	1978—现在	超大规模 集成电路	100 000 000



第一台von Neumann 系统结构的计算机 2.1



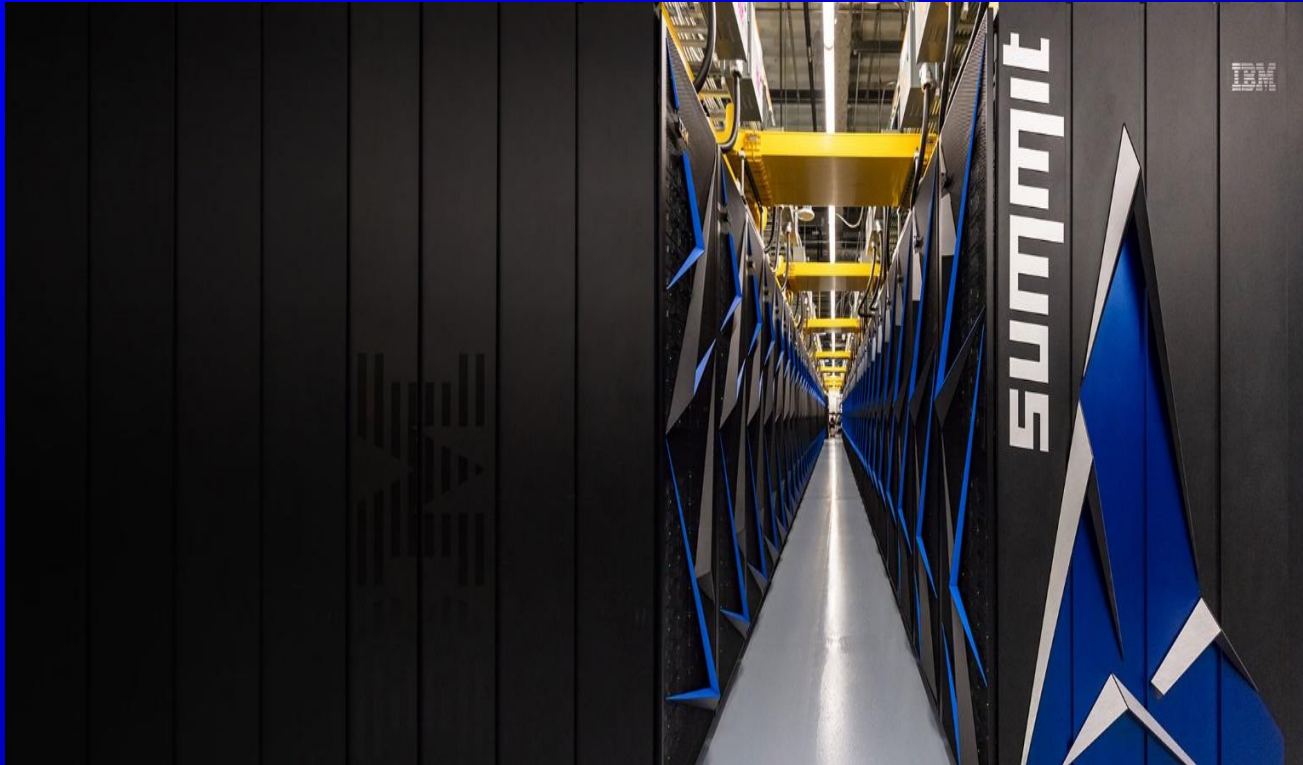
IBM System / 360

2.1



最快的五台超级计算机（截止到 2019.11） **2.1**

1. Summit



Cores: 2,414,592

Linpack Performance [Rmax] 148,600 TFlop/s

最快的五台超级计算机（截止到 2019.11） 2.1

2. Sierra



Cores: 1,572,480

Linpack Performance [Rmax] 94,640 TFlop/s

最快的五台超级计算机（截止到 2019.11） 2.1

3. Sunway TaihuLight--神威 太湖之光



Cores: 10,649,600

Linpack Performance [Rmax] 93,014.6 TFlop/s

最快的五台超级计算机（截止到 2019.11） 2.1

4. Tianhe-2A -- 天河2A



Cores: 4,981,760

Linpack Performance [Rmax] 61,444.5 TFlop/s

最快的五台超级计算机（截止到 2019.11） **2.1**

5. Frontera



Cores: 448,448

Linpack Performance [Rmax] 23,516.4 TFlop/s

最权威的超级计算机排名的参考网址

<http://www.top500.org>

二、微型计算机的出现和发展

微处理器芯片 1971年

4位 (4004)
8位
16位
32位
64位

存储器芯片 1970年

256位
1K位
4K位
16K位
64K位
256K位
1M位
4M位
16M位
64M位



Moore 定律

Intel 公司的缔造者之一
Gordon Moore 提出

微芯片上集成的

晶体管数目每三年翻两番



Intel 公司的典型微处理器产品

2.1

8080	8位	1974年	
8086	16位	1979年	2.9 万个晶体管
80286	16位	1982年	13.4 万个晶体管
80386	32位	1985年	27.5 万个晶体管
80486	32位	1989年	120.0 万个晶体管
Pentium	64位 (准)	1993年	310.0 万个晶体管
Pentium Pro	64位 (准)	1995年	550.0 万个晶体管
Pentium II	64位 (准)	1997年	750.0 万个晶体管
Pentium III	64位 (准)	1999年	950.0 万个晶体管
Pentium IV	64位	2000年	4 200.0 万个晶体管

2013年的Haswell是英特尔最后一次透露普通 cpu 的晶体管数目，4核的型号是 14 亿



三、软件技术的兴起和发展

2.1

1. 各种语言

机器语言 面向机器

汇编语言 面向机器

高级语言 面向问题

FORTRAN 科学计算和工程计算

PASCAL 结构化程序设计

C++ 面向对象

Java 适应网络环境



2. 系统软件

语言处理程序 汇编程序 编译程序 解释程序

操作系统 DOS UNIX Windows

服务性程序 装配 调试 诊断 排错

数据库管理系统 数据库和数据库管理软件

网络软件



3. 软件发展的特点

- (1) 开发周期长
- (2) 制作成本昂贵
- (3) 检测软件产品质量的特殊性

软件是程序以及开发、使用和维护程序所需要的所有文档



2.2 计算机的应用

一、科学计算和数据处理

二、工业控制和实时控制

三、网络技术

1. 电子商务

2. 网络教育

3. 敏捷制造



四、虚拟现实

五、办公自动化和管理信息系统

六、CAD/CAM/CIMS

七、多媒体技术

八、人工智能

2.3 计算机的展望

一、计算机具有类似人脑的一些超级智能功能

要求计算机的速度达 10^{15} /秒

二、芯片集成度的提高受以下三方面的限制

- 芯片集成度受物理极限的制约
- 按几何级数递增的制作成本
- 芯片的功耗、散热、线延迟

三、？ 替代传统的硅芯片

1. 光计算机

利用光子取代电子进行运算和存储

2. DNA生物计算机

通过控制DNA分子间的生化反应

3. 量子计算机

利用原子所具有的量子特性



第3章 系统总线

3.1 总线的基本概念

3.2 总线的分类

3.3 总线特性及性能指标

3.4 总线结构

3.5 总线控制



3.1 总线的基本概念

一、为什么要用总线

二、什么是总线

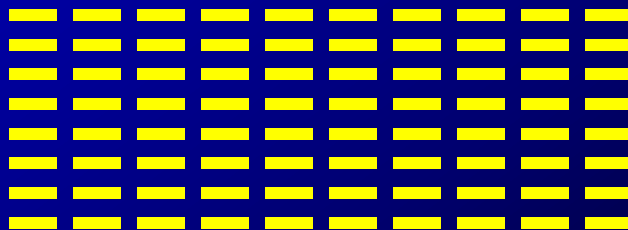
总线是连接各个部件的信息传输线，
是各个部件共享的传输介质

三、总线上信息的传送

串行



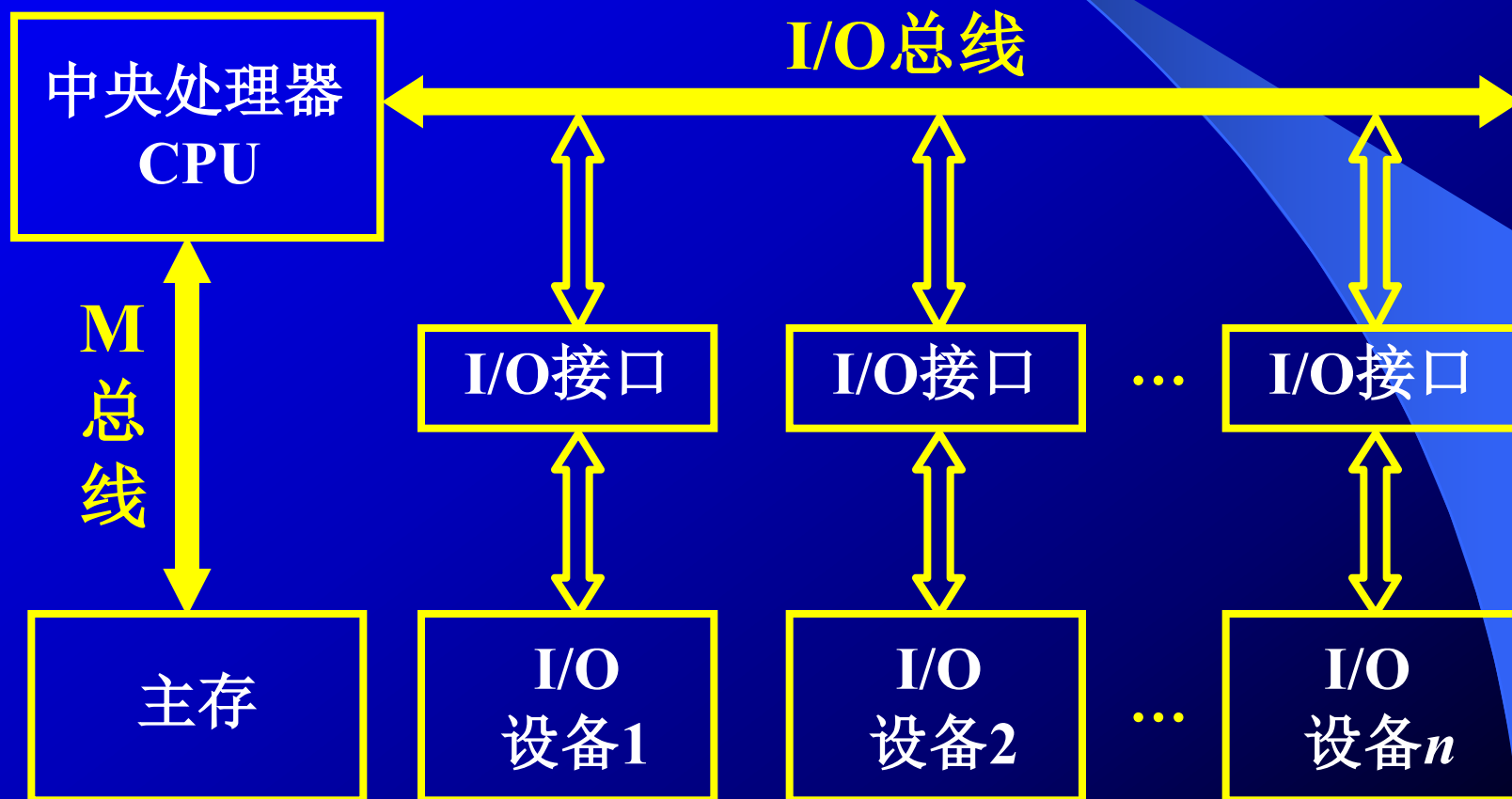
并行



四、总线结构的计算机举例

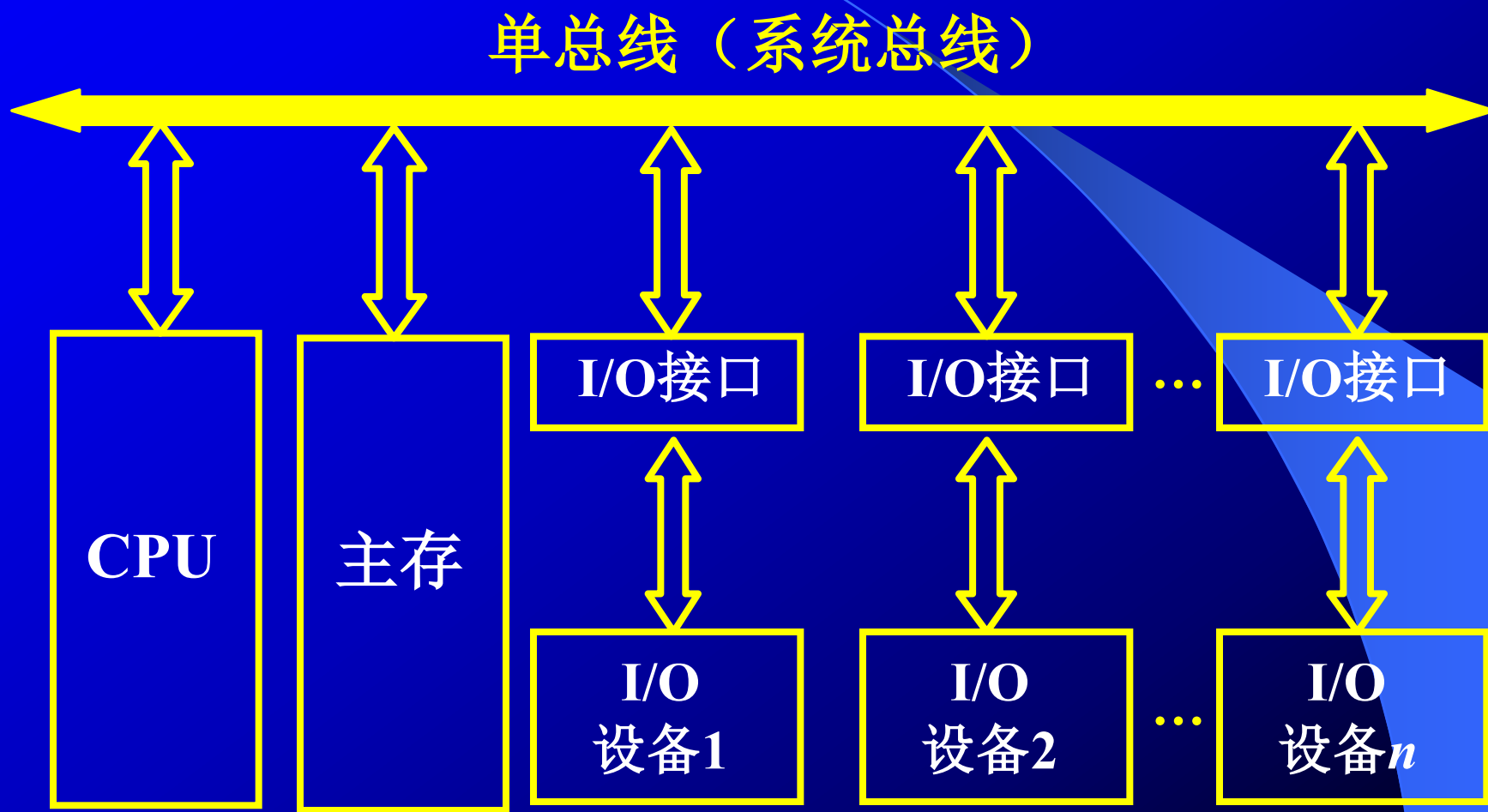
3.1

1. 面向 CPU 的双总线结构框图

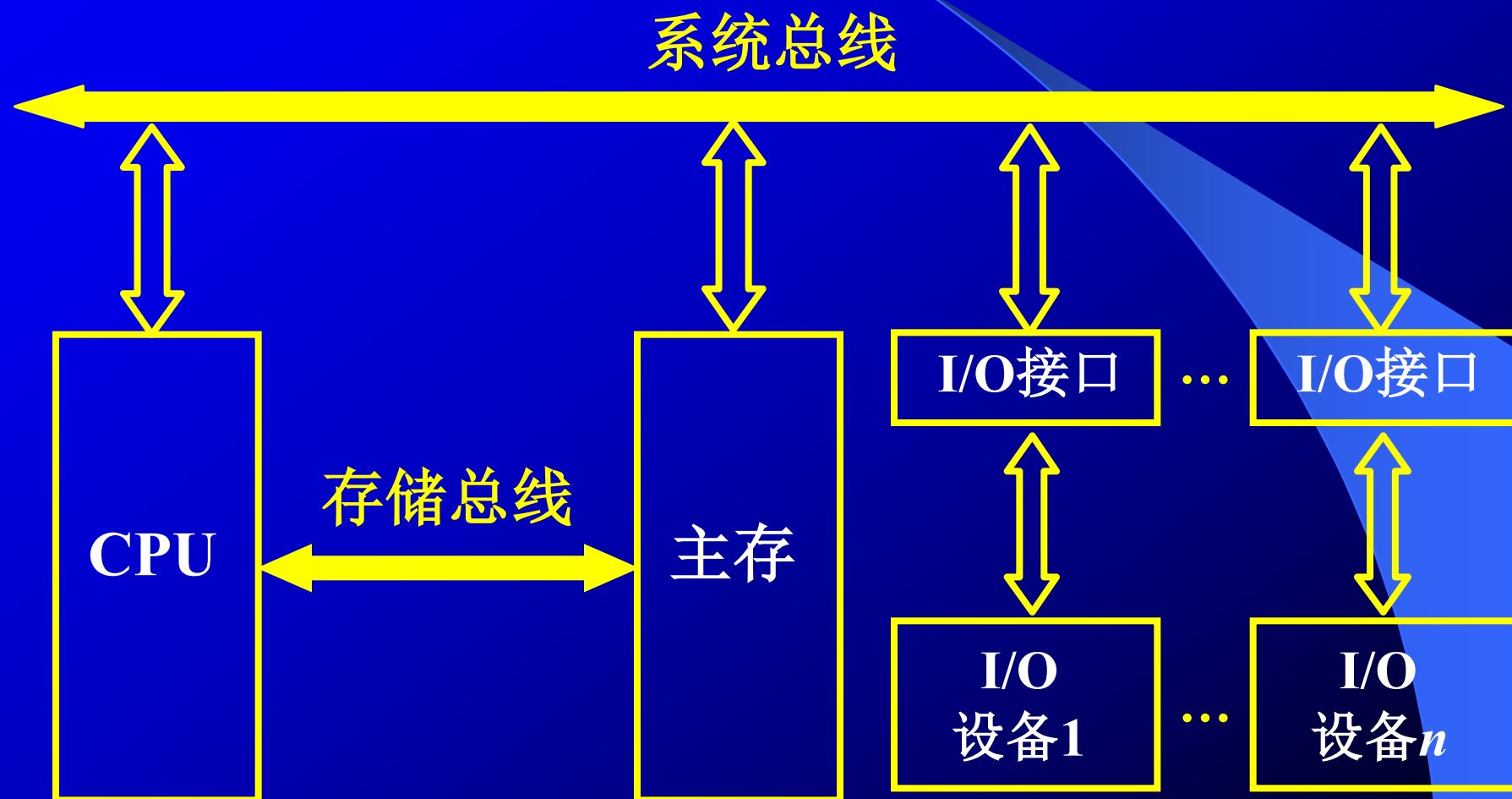


2. 单总线结构框图

3.1



3. 以存储器为中心的双总线结构框图



3.2 总线的分类

1. 片内总线 芯片内部 的总线

2. 系统总线 计算机各部件之间 的信息传输线

数据总线 双向 与机器字长、存储字长有关

地址总线 单向 与存储地址、I/O地址有关

控制总线 有出 有入

中断请求、总线请求

存储器读、存储器写
总线允许、中断确认

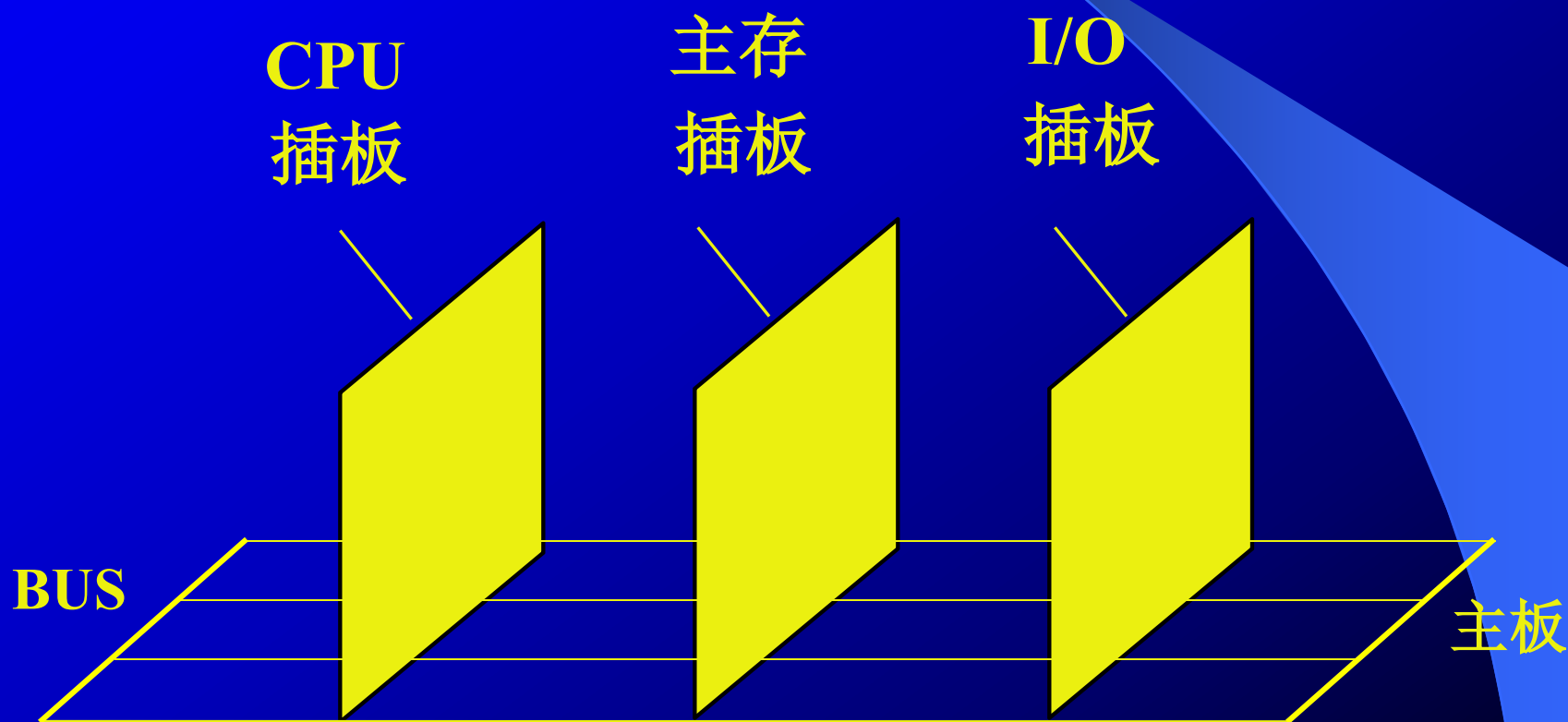
3. 通信总线

用于 计算机系统之间 或 计算机系统
与其他系统（如控制仪表、移动通信等）
之间的通信

传输方式 { 串行通信总线
并行通信总线

3.3 总线特性及性能指标

一、总线物理实现

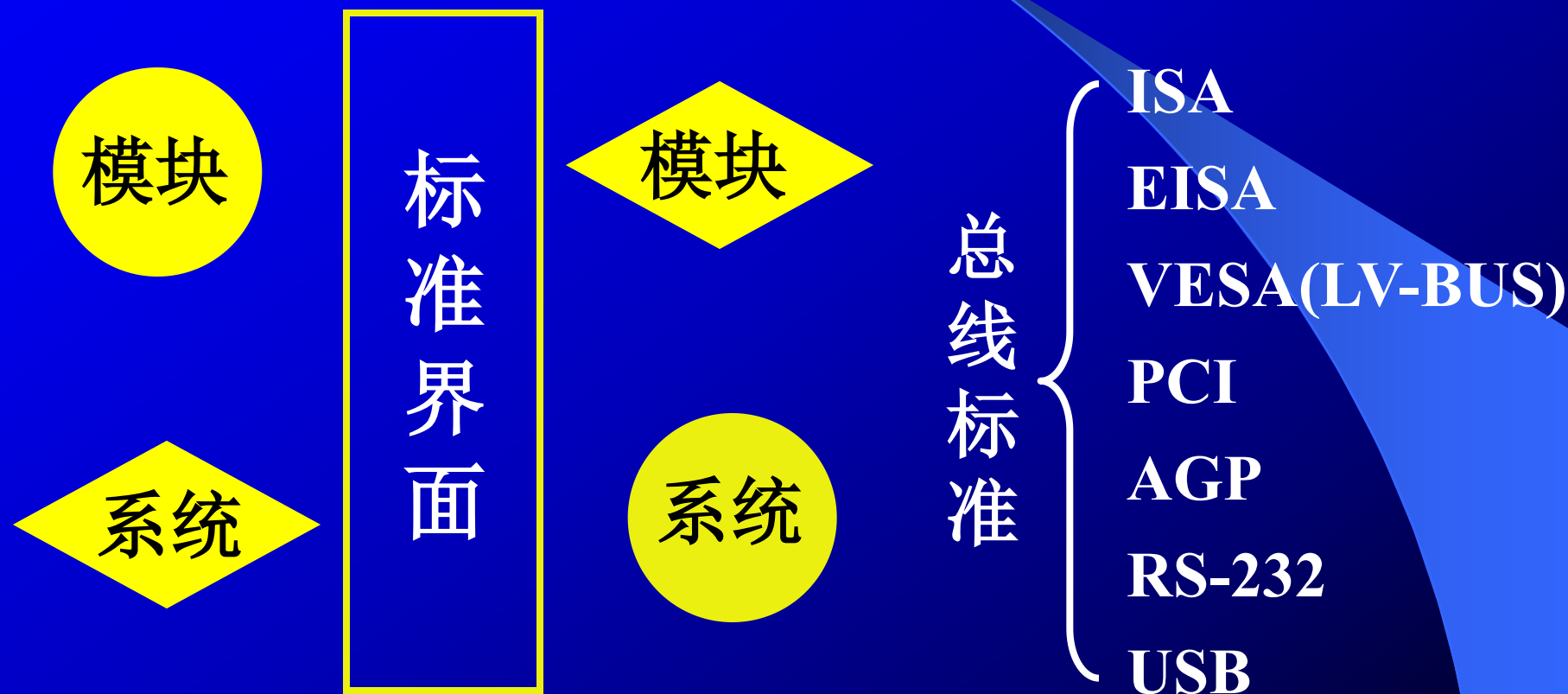


三、总线的性能指标

1. 总线宽度 数据线的根数
2. 总线带宽 每秒传输的最大字节数 (MBps)
3. 时钟同步/异步 同步、不同步
4. 总线复用 地址线与数据线复用
5. 信号线数 地址线、数据线和控制线的总和
6. 总线控制方式 并发、自动、仲裁、逻辑、计数
7. 其他指标 负载能力



四、总线标准



四、总线标准（教材）

3.3

总线标准	数据线	总线时钟	带宽
ISA	16	8 MHz（独立）	16 MBps
EISA	32	8 MHz（独立）	33 MBps
VESA (VL-BUS)	32	33 MHz（CPU）	133 MBps
PCI	32	33 MHz（独立）	132 MBps
	64	66 MHz（独立）	528 MBps
AGP	32	66.7 MHz（独立）	266 MBps
		133 MHz（独立）	533 MBps
RS-232	串行通信 总线标准	数据终端设备（计算机）和数据通信设备（调制解调器）之间的标准接口	
USB	串行接口 总线标准	普通无屏蔽双绞线 带屏蔽双绞线 最高	1.5 Mbps (USB1.0) 12 Mbps (USB1.0) 480 Mbps (USB2.0)

四、总线标准（目前）

3.3

PCI Express 版本	推出	Line 编码	原始传输率	有效带宽（带宽）				
				x1	x2	x4	X8	x16
1.0	2003	8b/10b	2.5 GT/s	250 MB/s	0.50 GB/s	1.0 GB/s	2.0 GB/s	4.0 GB/s
2.0	2007	8b/10b	5.0 GT/s	500 MB/s	1.0 GB/s	2.0 GB/s	4.0 GB/s	8.0 GB/s
3.0	2010	128b/130b	8.0 GT/s	984.6 MB/s	1.97 GB/s	3.94 GB/s	7.88 GB/s	15.8 GB/s
4.0	2017	128b/130b	16.0 GT/s	1969 MB/s	3.94 GB/s	7.88 GB/s	15.75 GB/s	31.5 GB/s
5.0	2019	NRZ 128b/130b	32.0 GT/s	3938 MB/s	7.88 GB/s	15.75 GB/s	31.51 GB/s	63.0 GB/s
6.0	2021	PAM4 & FEC 128b/130b	64.0 GT/s	7877 MB/s	15.75 GB/s	31.51 GB/s	63.02 GB/s	126.03 GB/s

以PCI-E 2.0为例，每秒 5 GT（Gigatransfer）原始数据传输率，编码方式为 8b/10b（每10个比特只有8个有效数据），即有效带宽为 4Gb/s = 500 MByte/s

四、总线标准（目前）

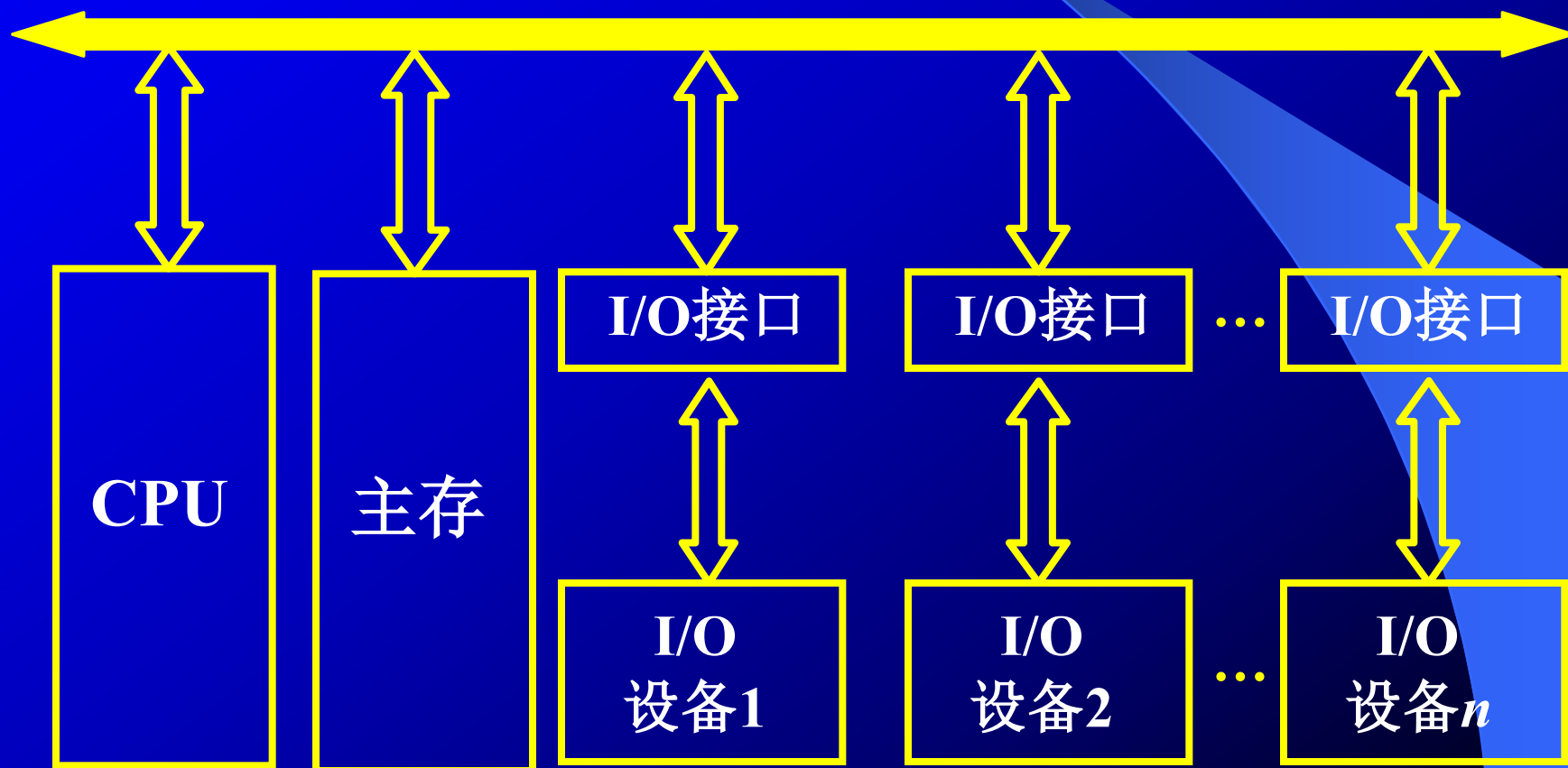
3.3

版本	名称	带宽	编码	备注
USB 3.0	SuperSpeed	5 Gbit/s	8b/10b	
USB 3.1 Gen 1	SuperSpeed	5 Gbit/s	8b/10b	与USB 3.0相同
USB 3.1 Gen 2	SuperSpeed+	10 Gbit/s	128b/132b	新标准
USB 3.2 Gen 1×1	SuperSpeed	5 Gbit/s	8b/10b	与USB 3.1 Gen 1相同
USB 3.2 Gen 1×2	SuperSpeed+	10 Gbit/s	8b/10b	双通道模式，需要USB 3.2电缆
USB 3.2 Gen 2×1	SuperSpeed+	10 Gbit/s	128b/132b	与USB 3.1 Gen 2相同

3.4 总线结构

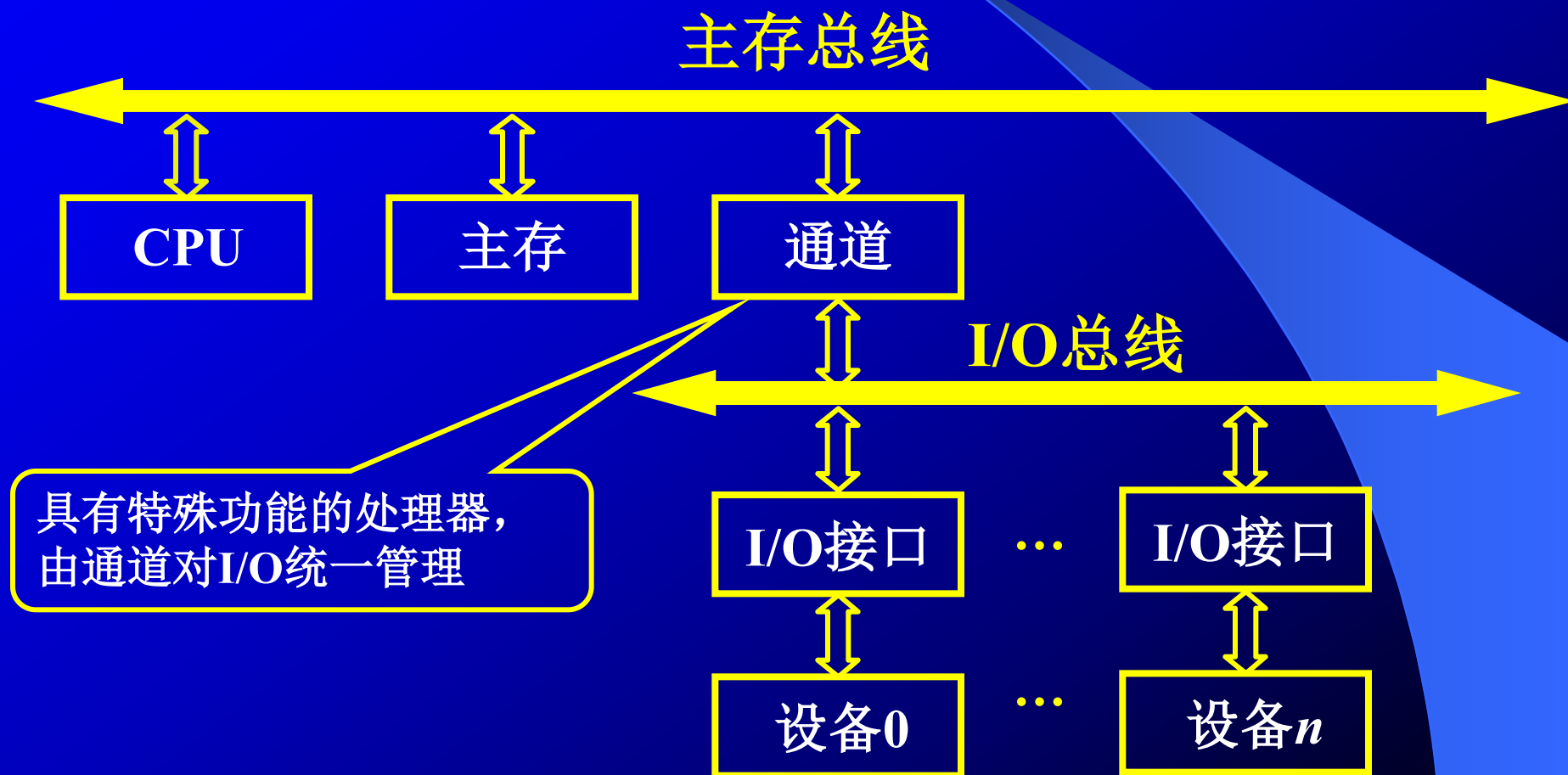
一、单总线结构

单总线（系统总线）

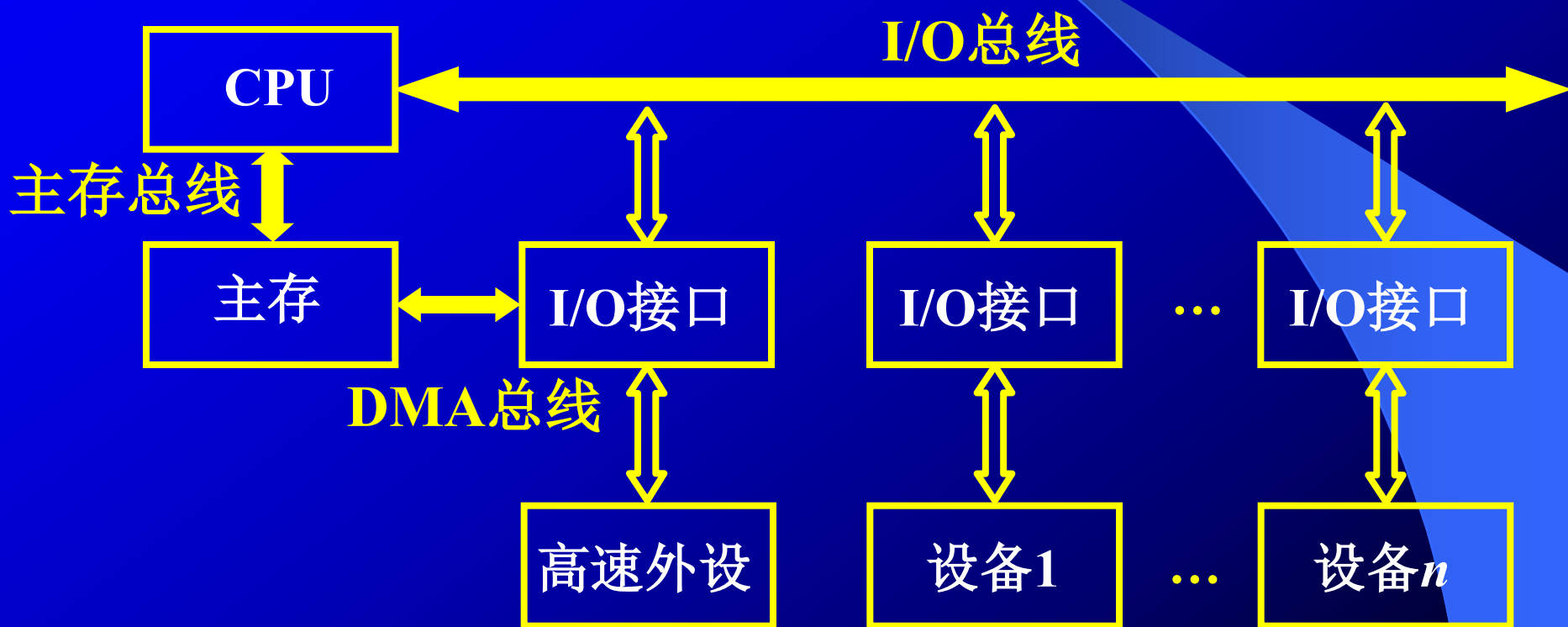


二、多总线结构

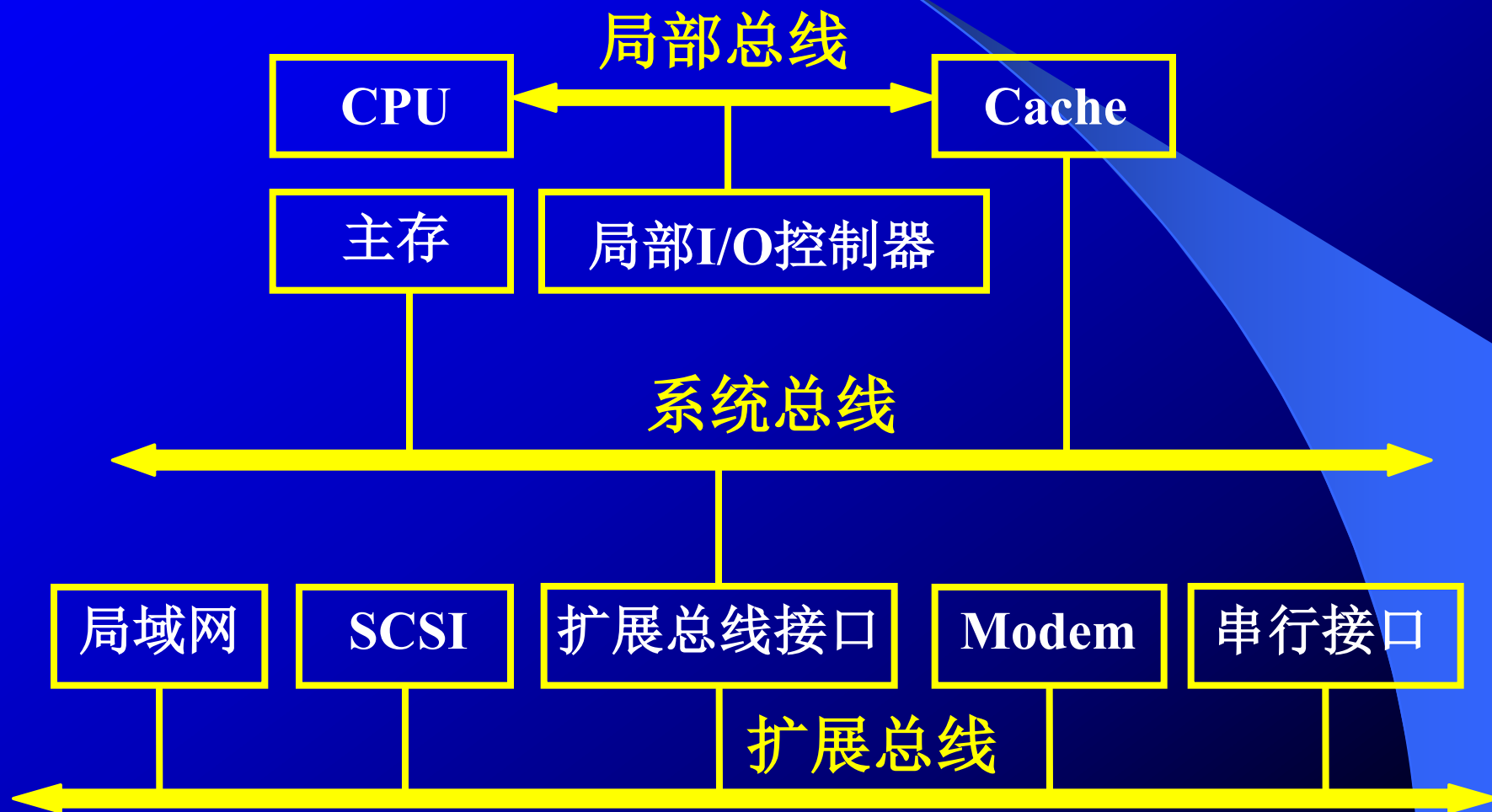
1. 双总线结构



2. 三总线结构

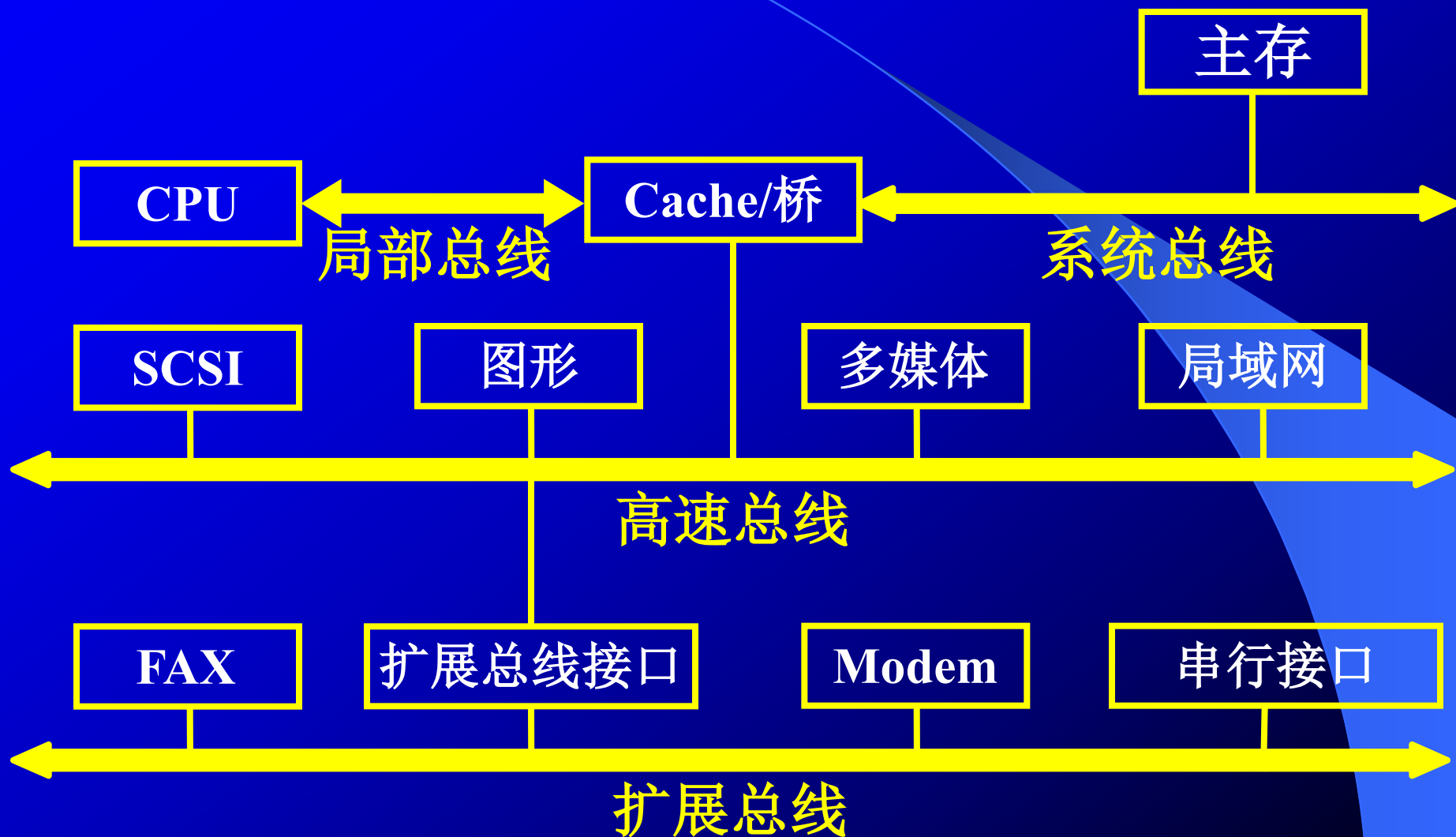


3. 三总线结构的又一形式



4. 四总线结构

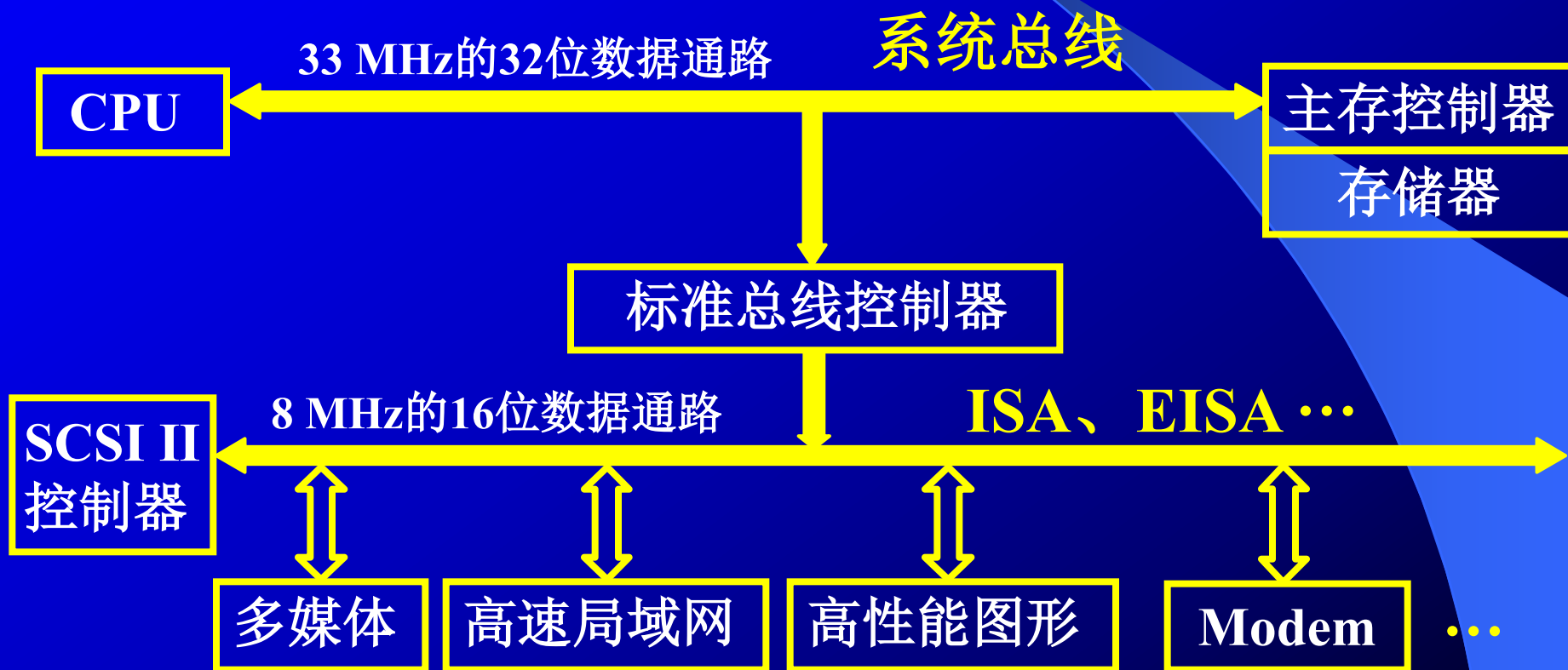
3.4



三、总线结构举例

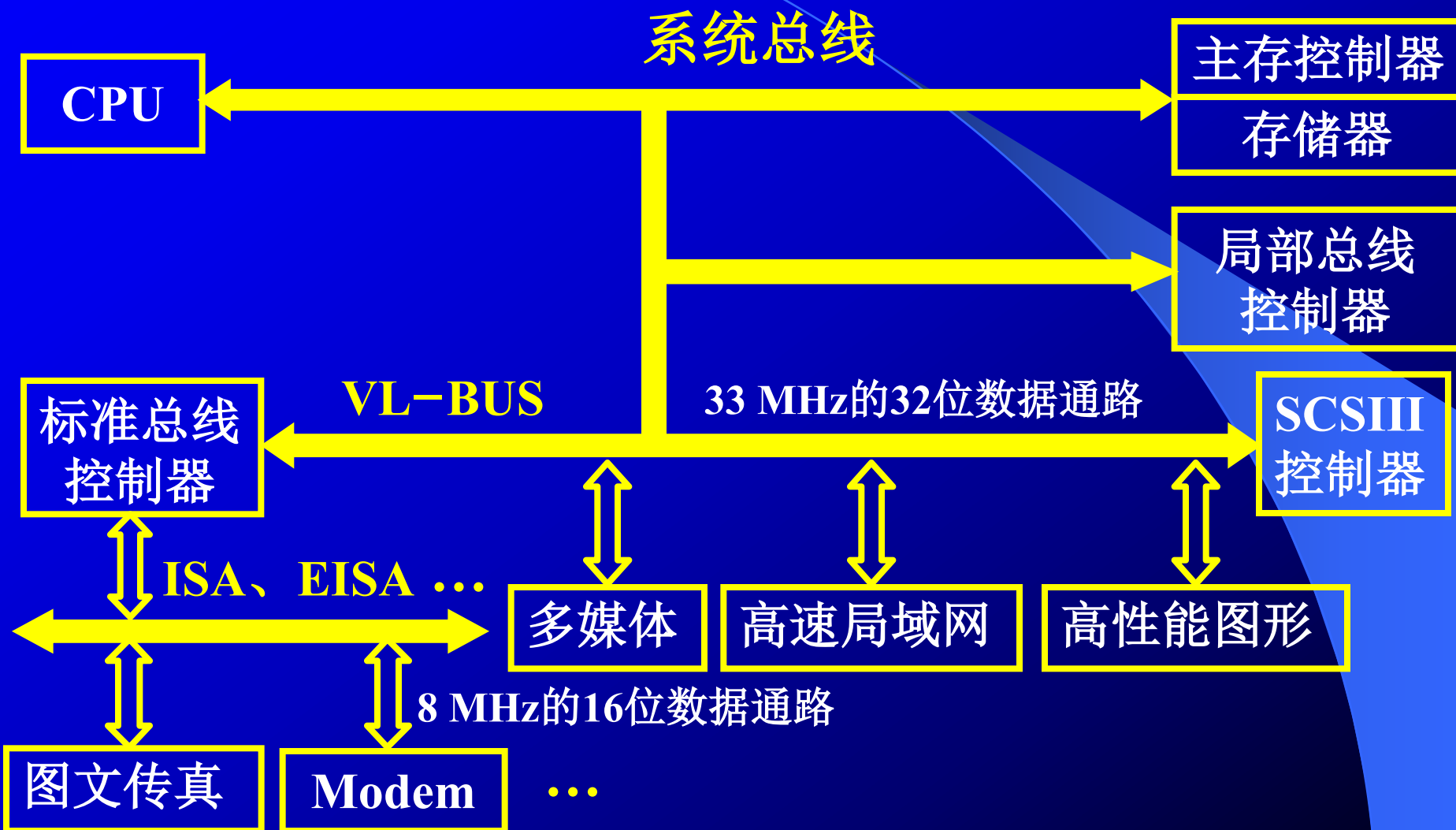
3.4

1. 传统微型机总线结构



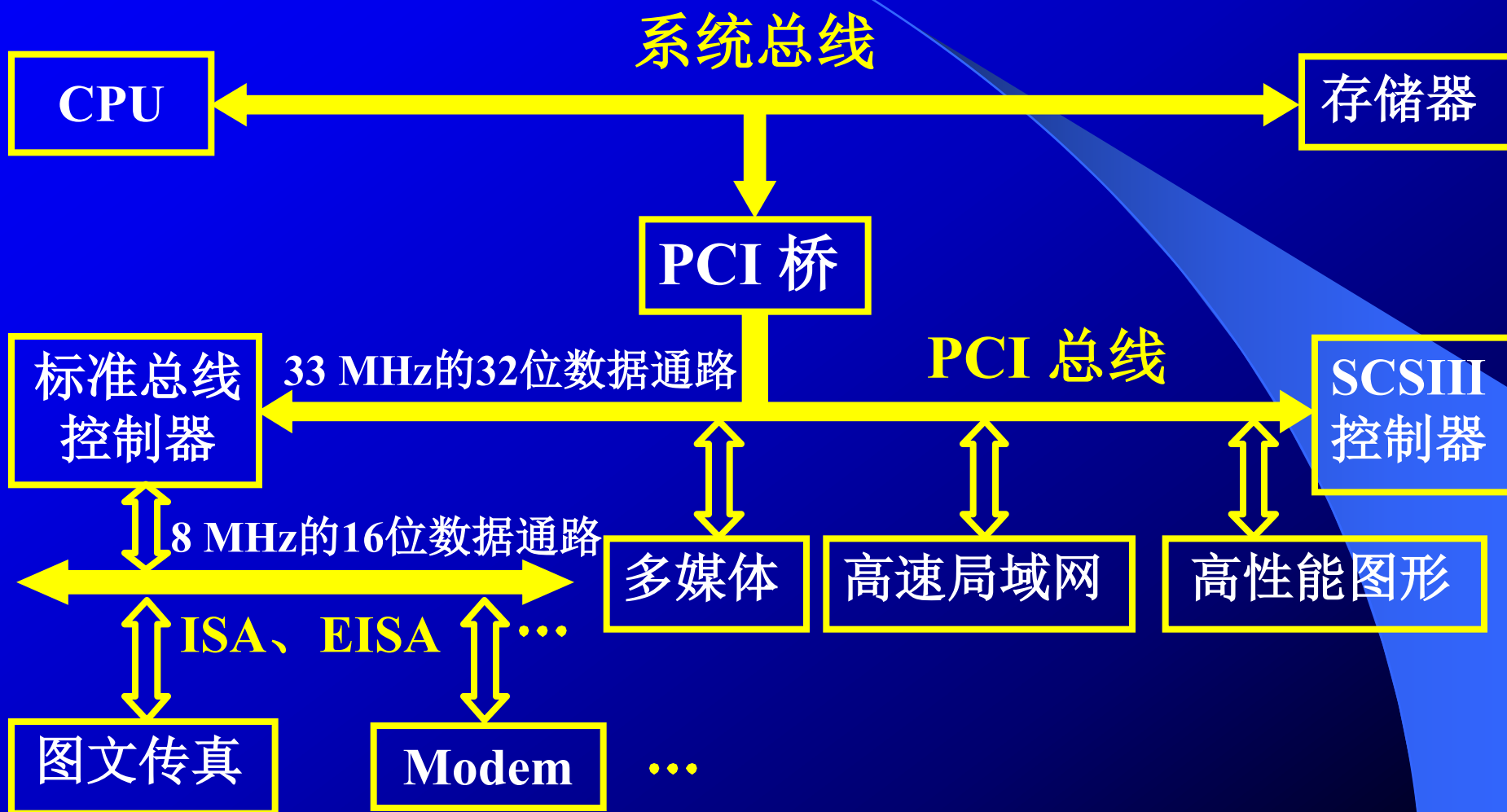
2. VL-BUS局部总线结构

3.4



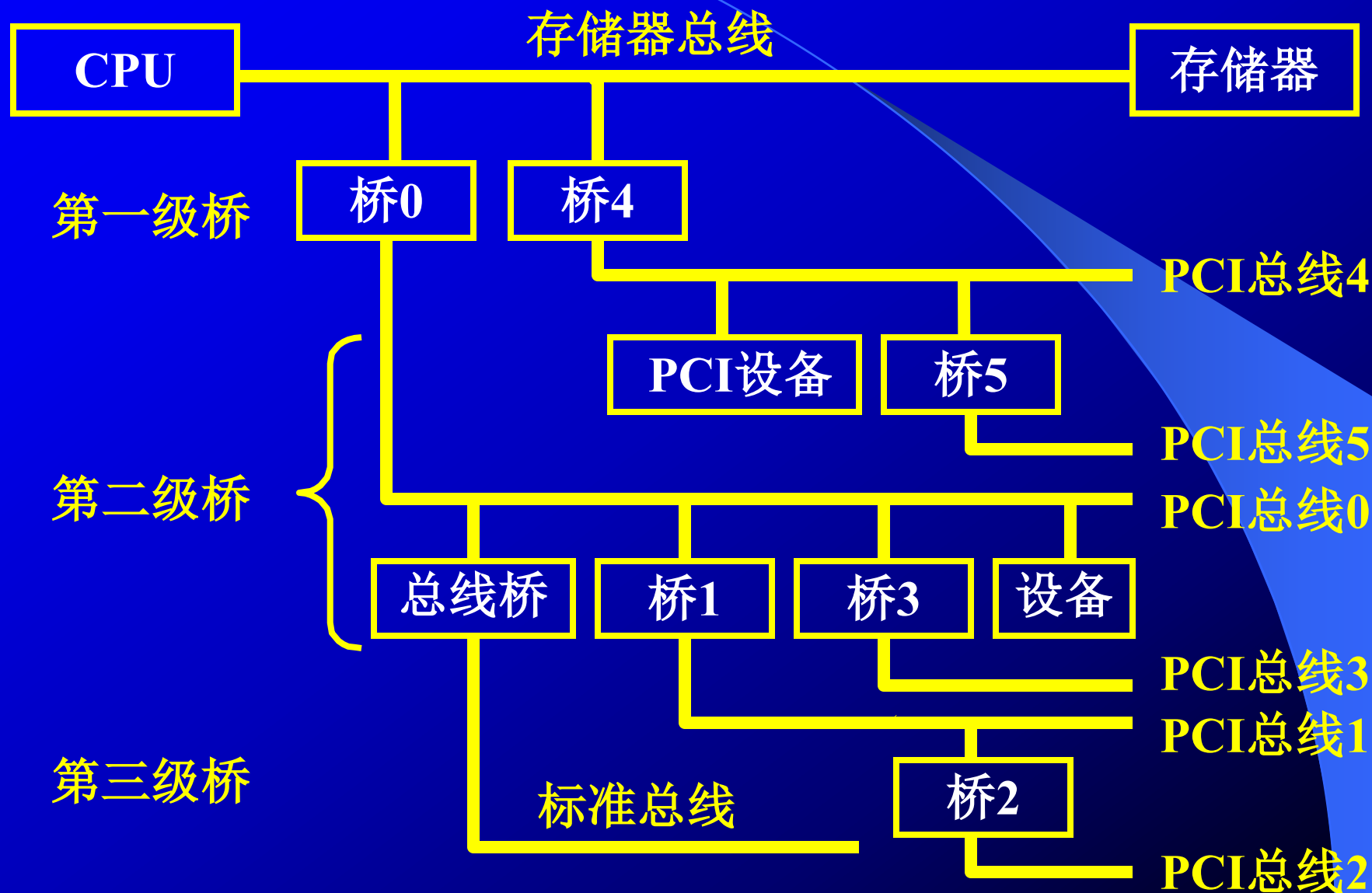
3. PCI 总线结构

3.4



4. 多层 PCI 总线结构

3.4



3.5 总线控制

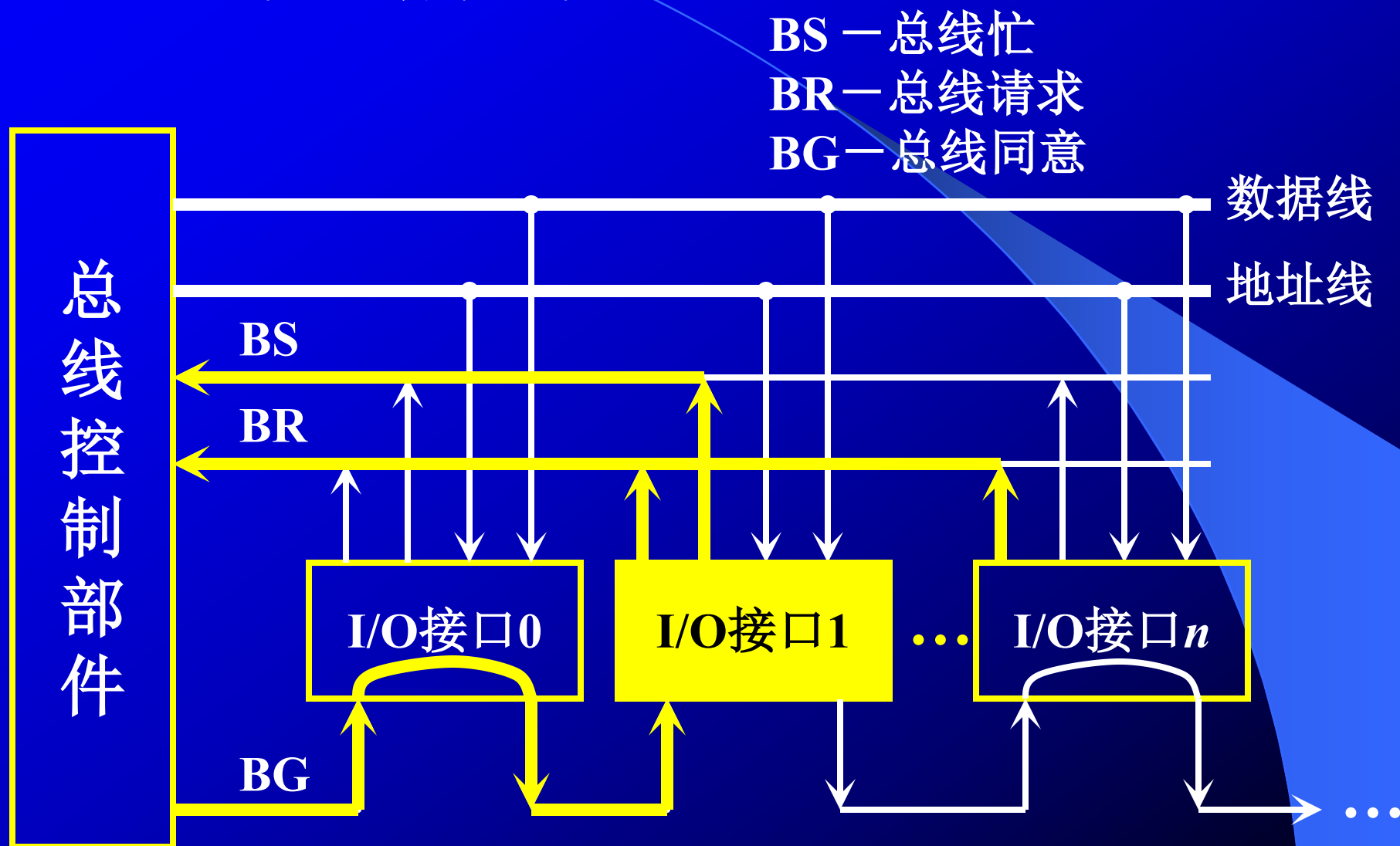
一、总线判优控制

1. 基本概念

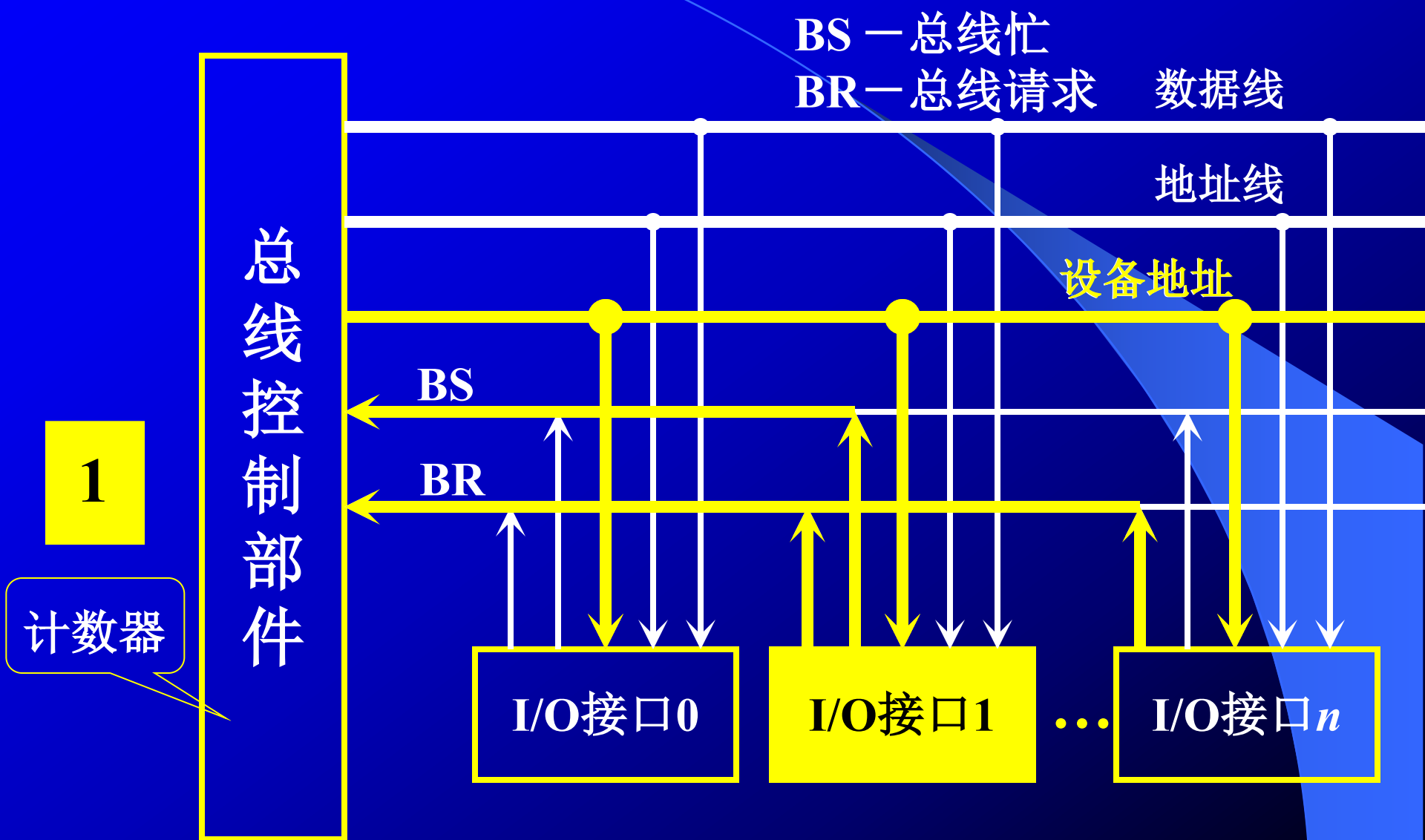
- 主设备(模块) 对总线有 **控制权**
- 从设备(模块) **响应** 从主设备发来的总线命令

- 总线判优控制
 - 集中式
 - 链式查询
 - 计数器定时查询
 - 独立请求方式
 - 分布式

2. 链式查询方式

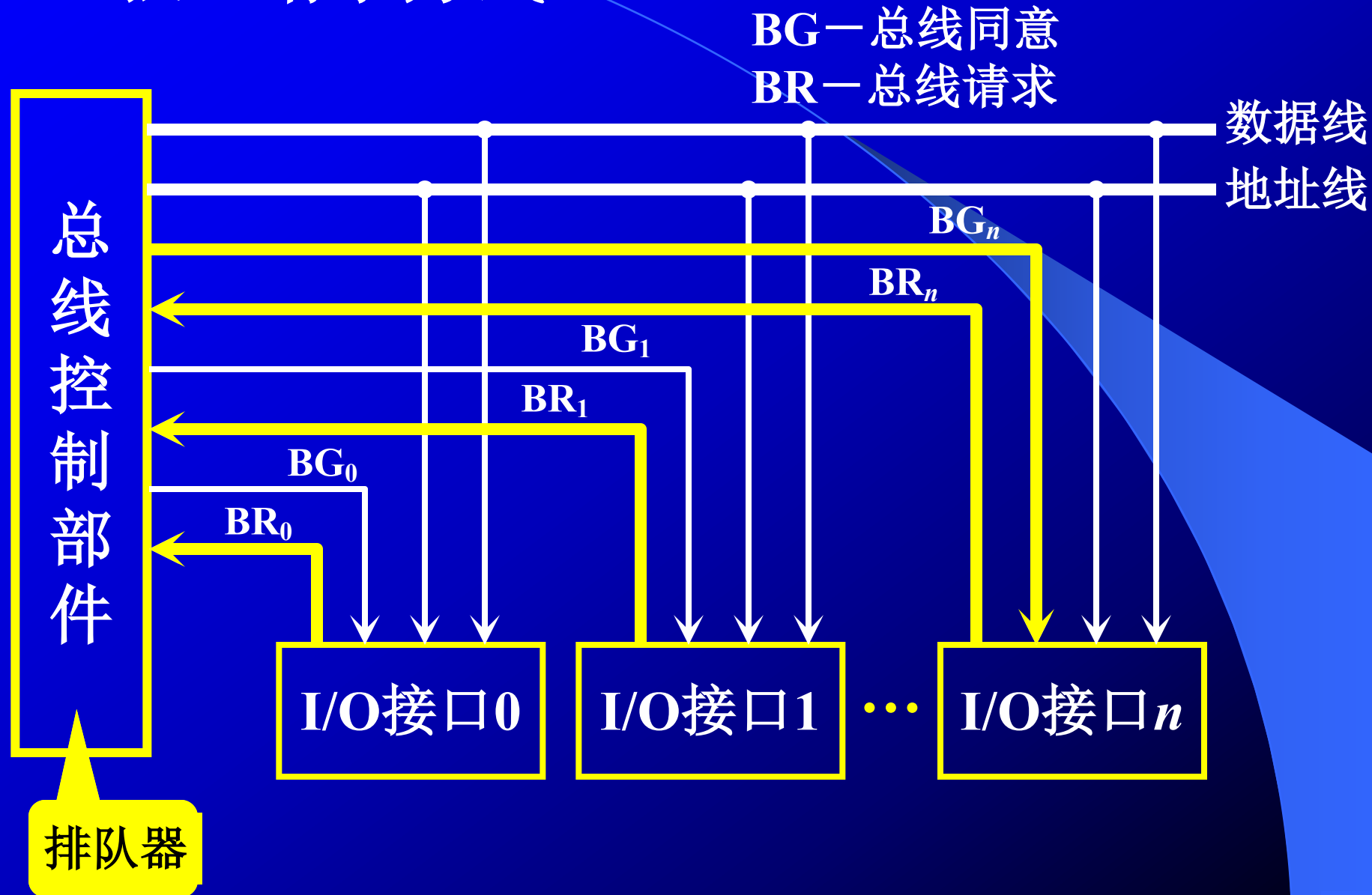


3. 计数器定时查询方式



4. 独立请求方式

3.5



二、总线通信控制

1. 目的 解决通信双方 **协调配合** 问题

2. 总线传输周期

申请分配阶段 **主模块申请**，总线仲裁决定

寻址阶段 主模块向从模块 **给出地址** 和 **命令**

传数阶段 主模块和从模块 **交换数据**

结束阶段 主模块 **撤消有关信息**

3. 总线通信的四种方式

同步通信 由 统一时标 控制数据传送

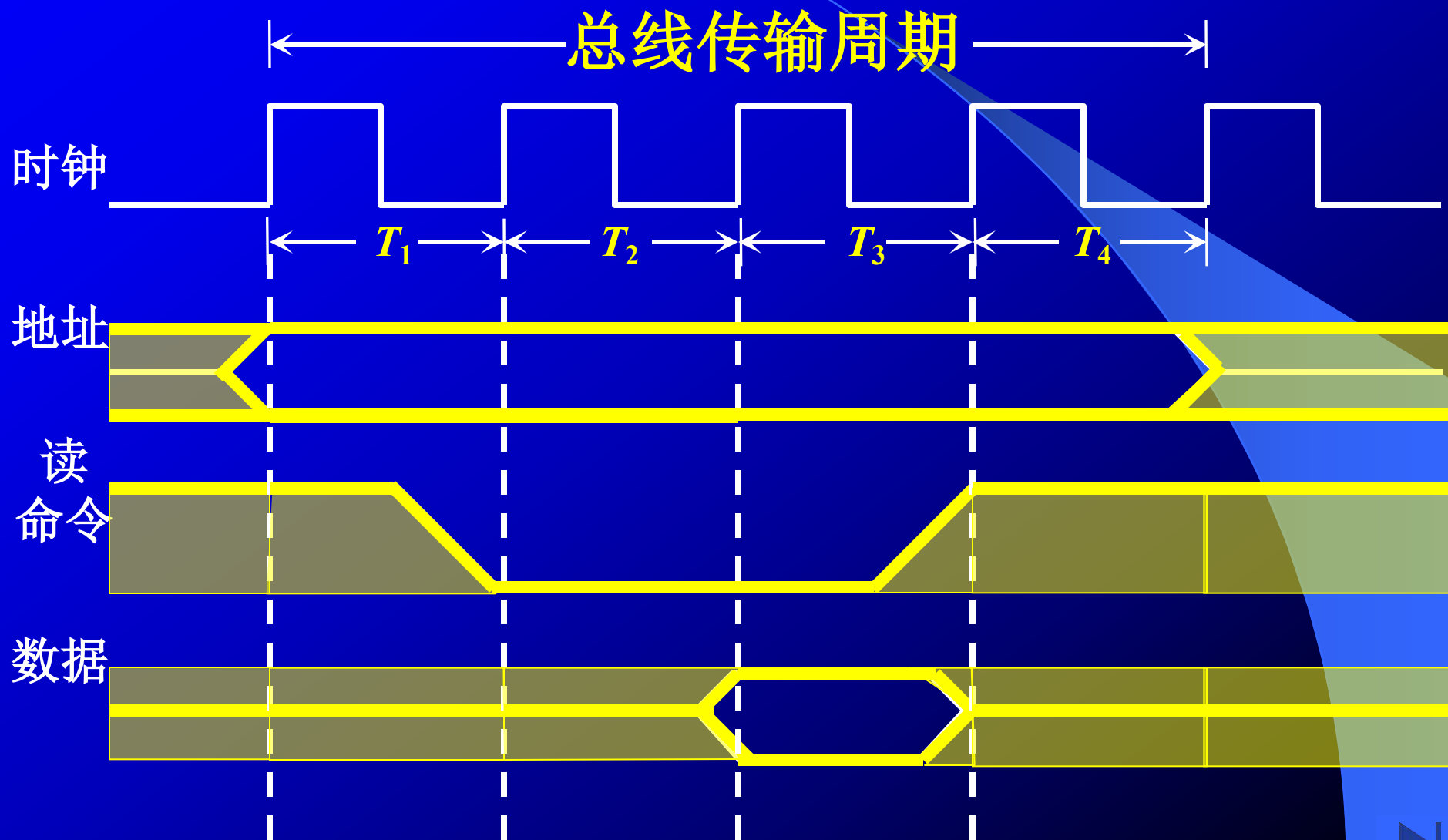
异步通信 采用 应答方式，没有公共时钟标准

半同步通信 同步、异步结合

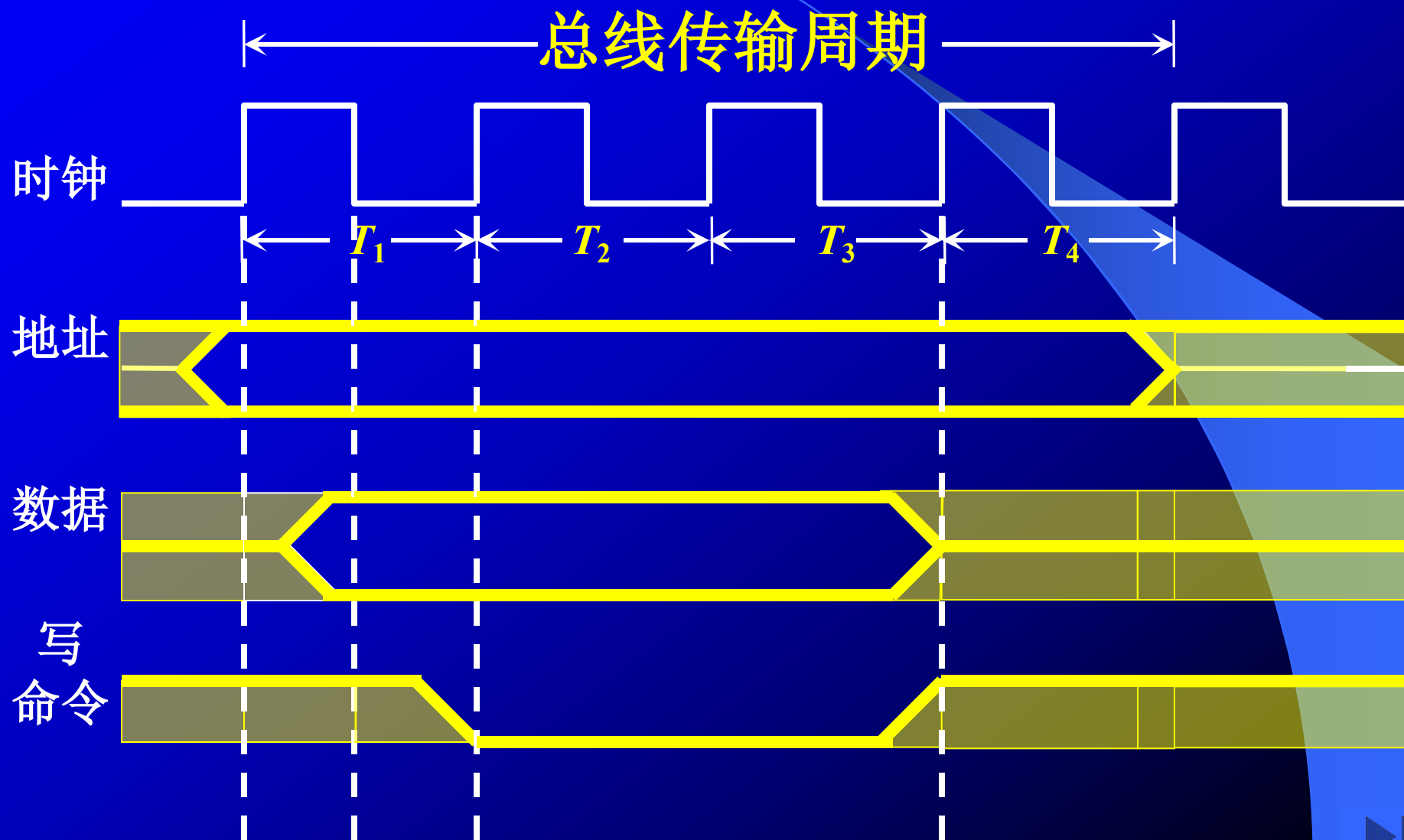
分离式通信 充分 挖掘 系统 总线每个瞬间 的 潜力



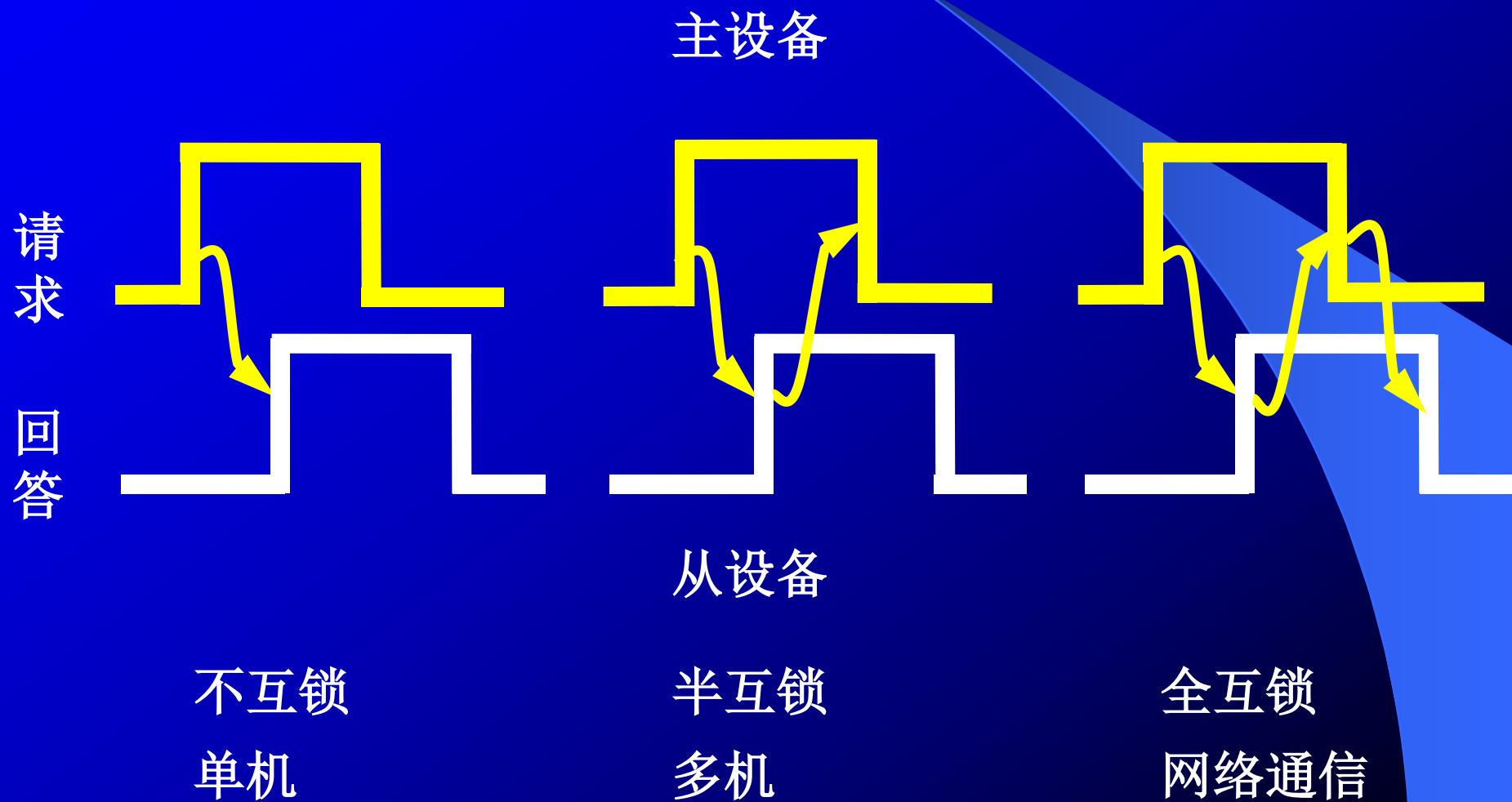
(1) 同步式数据输入



(2) 同步式数据输出



异步通信



(4) 半同步通信 (同步、异步 结合)

3.5

同步 发送方 用系统 时钟前沿 发信号

接收方 用系统 时钟后沿 判断、识别

异步 允许不同速度的模块和谐工作

增加一条 “等待” 响应信号 $\overline{\text{WAIT}}$



以输入数据为例的半同步通信时序

T_1 主模块发地址

T_2 主模块发命令

T_w 当 $\overline{\text{WAIT}}$ 为低电平时，等待一个 T

T_w 当 $\overline{\text{WAIT}}$ 为低电平时，等待一个 T

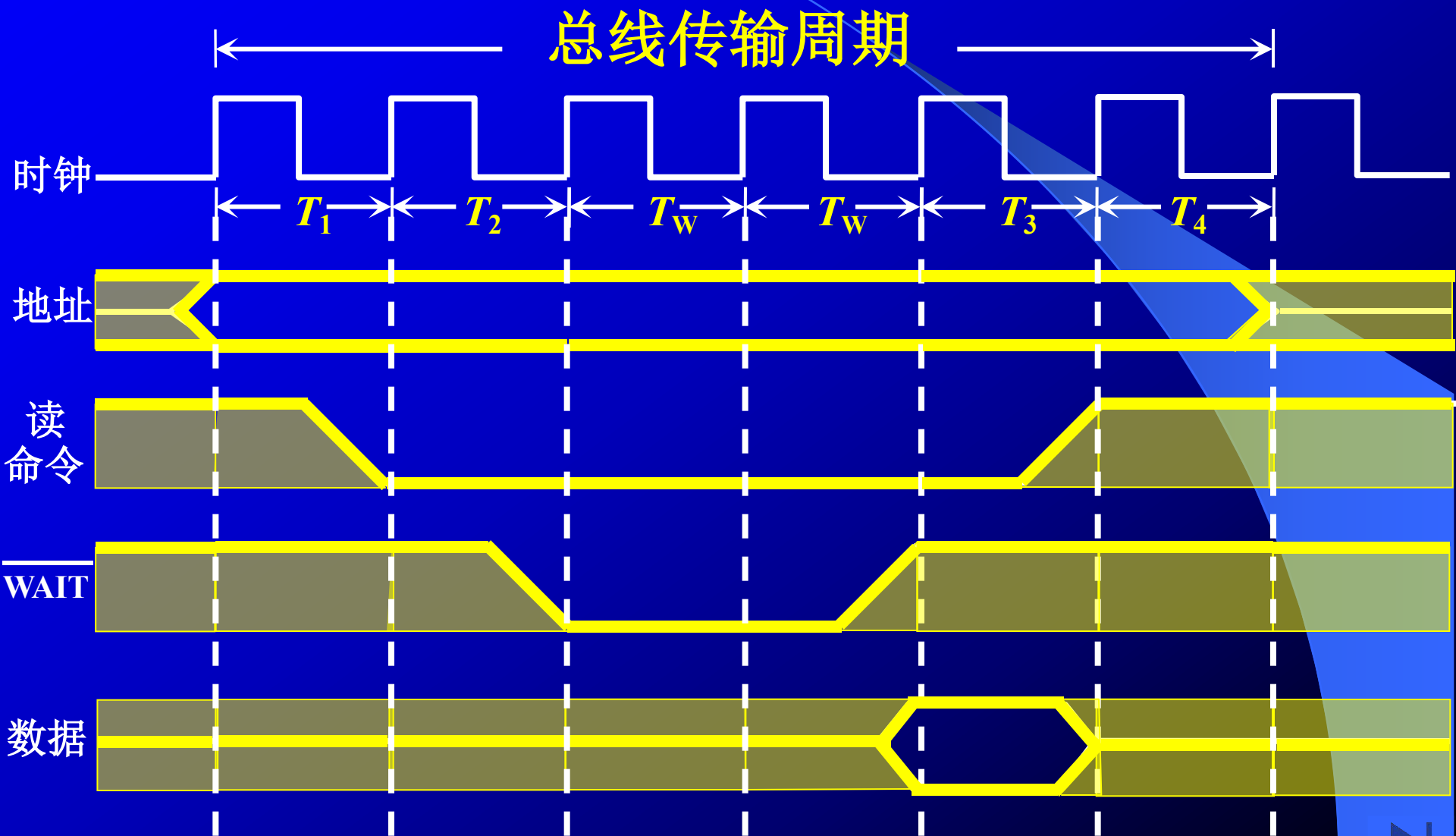
⋮

T_3 从模块提供数据

T_4 从模块撤销数据，主模块撤销命令



(4) 半同步通信 (同步、异步 结合) 3.5



上述三种通信的共同点

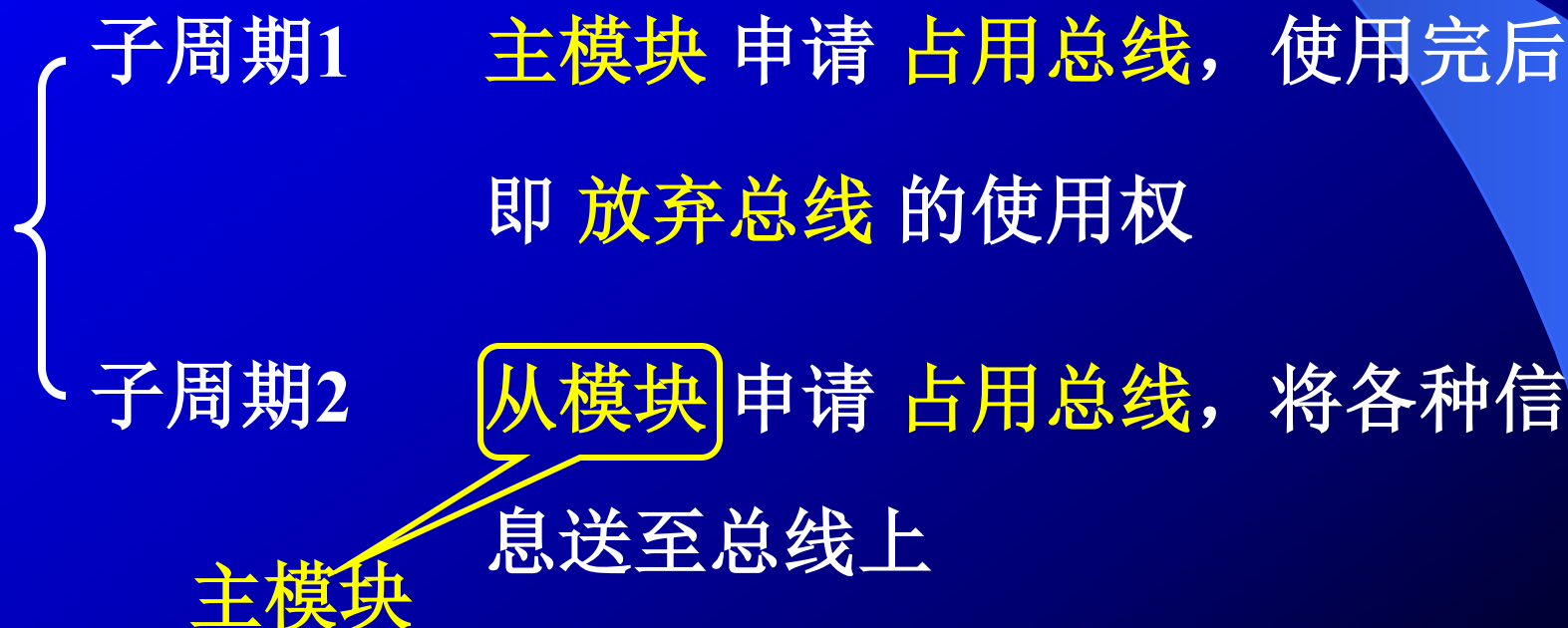
一个总线传输周期（以输入数据为例）

- 主模块发地址、命令 占用总线
- 从模块准备数据 不占用总线 总线空闲
- 从模块向主模块发数据 占用总线

(5) 分离式通信

充分挖掘系统总线每个瞬间的潜力

一个总线传输周期



分离式通信特点

1. 各模块有权申请占用总线
2. 采用同步方式通信，不等对方回答
3. 各模块准备数据时，不占用总线
4. 总线被占用时，无空闲

充分提高了总线的有效占用



第4章 存储器

4.1 概述

4.2 主存储器

4.3 高速缓冲存储器

4.4 辅助存储器



4.1 概 述

一、存储器分类

1. 按存储介质分类

(1) 半导体存储器

TTL、MOS

易失

(2) 磁表面存储器

磁头、载磁体

(3) 磁芯存储器

硬磁材料、环状元件

(4) 光盘存储器

激光、磁光材料

非
易
失



2. 按存取方式分类

(1) 存取时间与物理地址无关（随机访问）

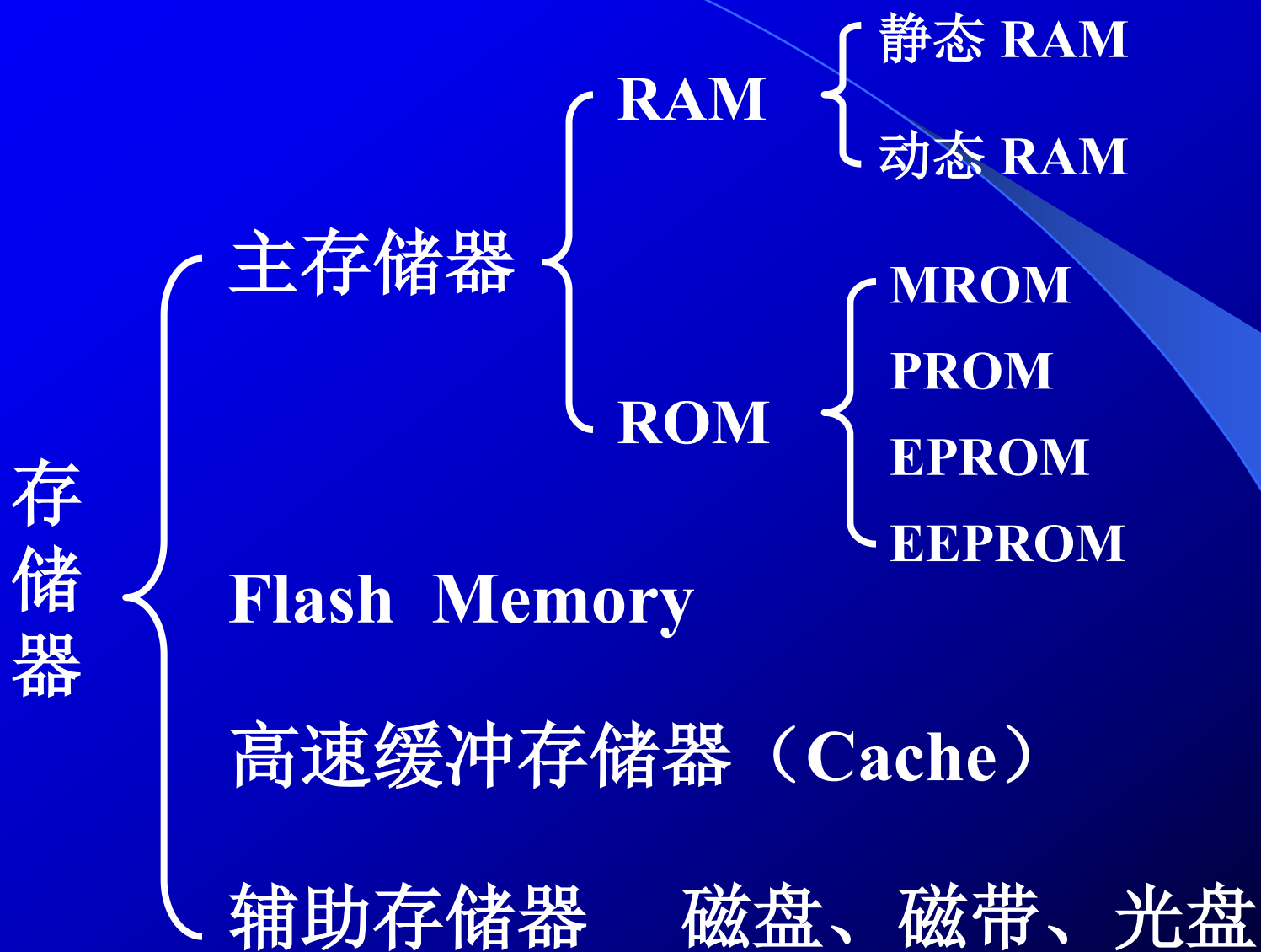
- 随机存储器 在程序的执行过程中 可读 可 写
- 只读存储器 在程序的执行过程中 只 读

(2) 存取时间与物理地址有关（串行访问）

- 顺序存取存储器 磁带
- 直接存取存储器 磁盘

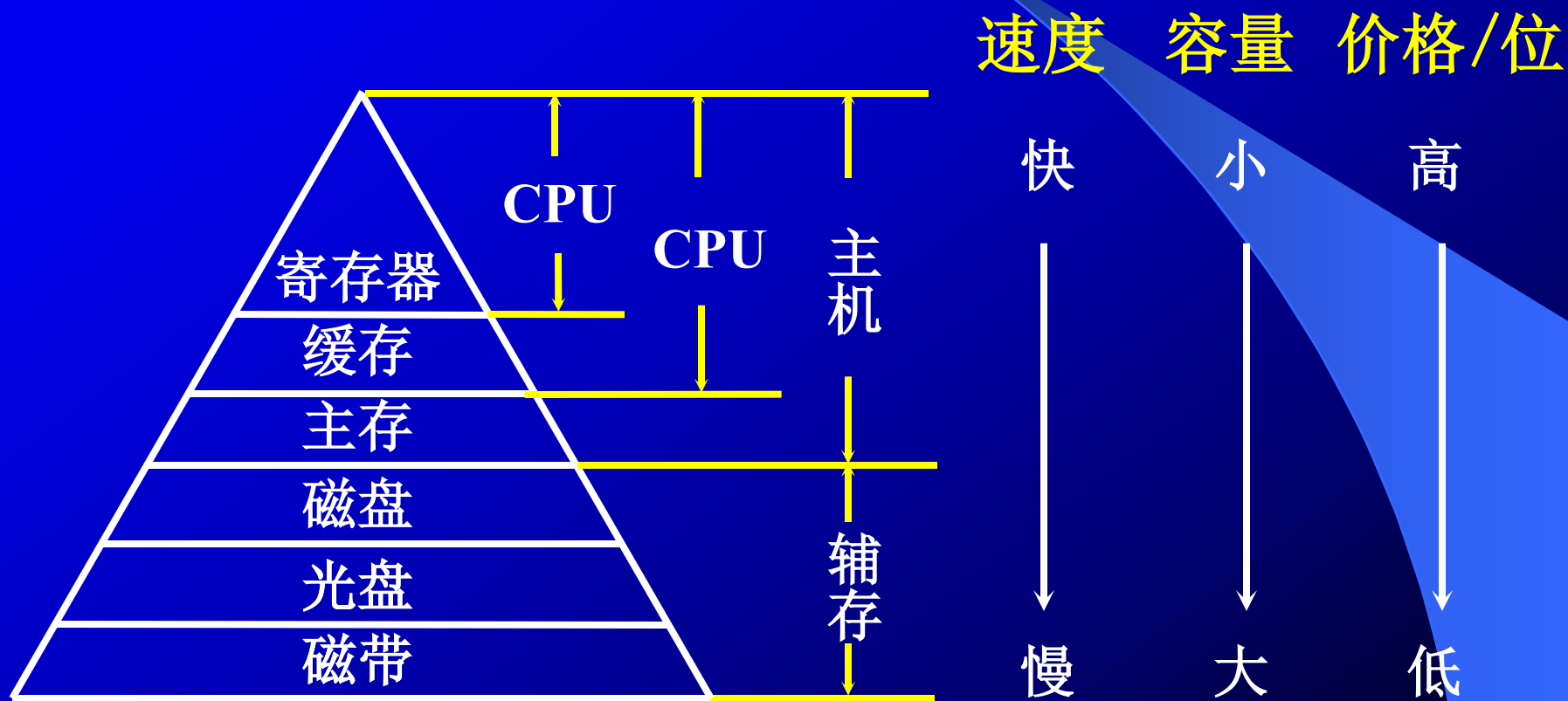
3. 按在计算机中的作用分类

4.1

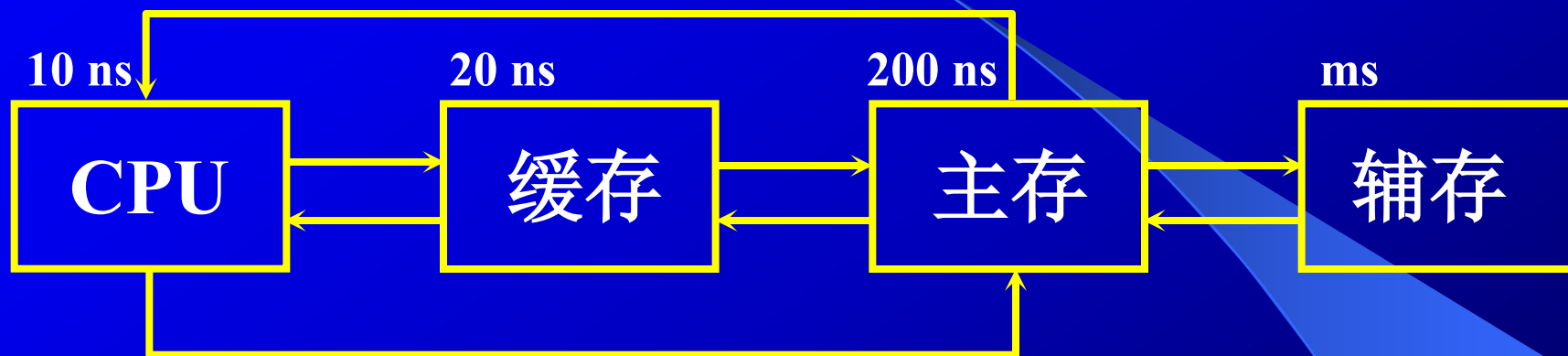


二、存储器的层次结构

1. 存储器三个主要特性的关系



2. 缓存—主存层次和主存—辅存层次 4.1



(速度) (容量)
缓存—主存 主存—辅存

主存储器

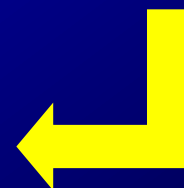
虚拟存储器

实地址

虚地址

物理地址

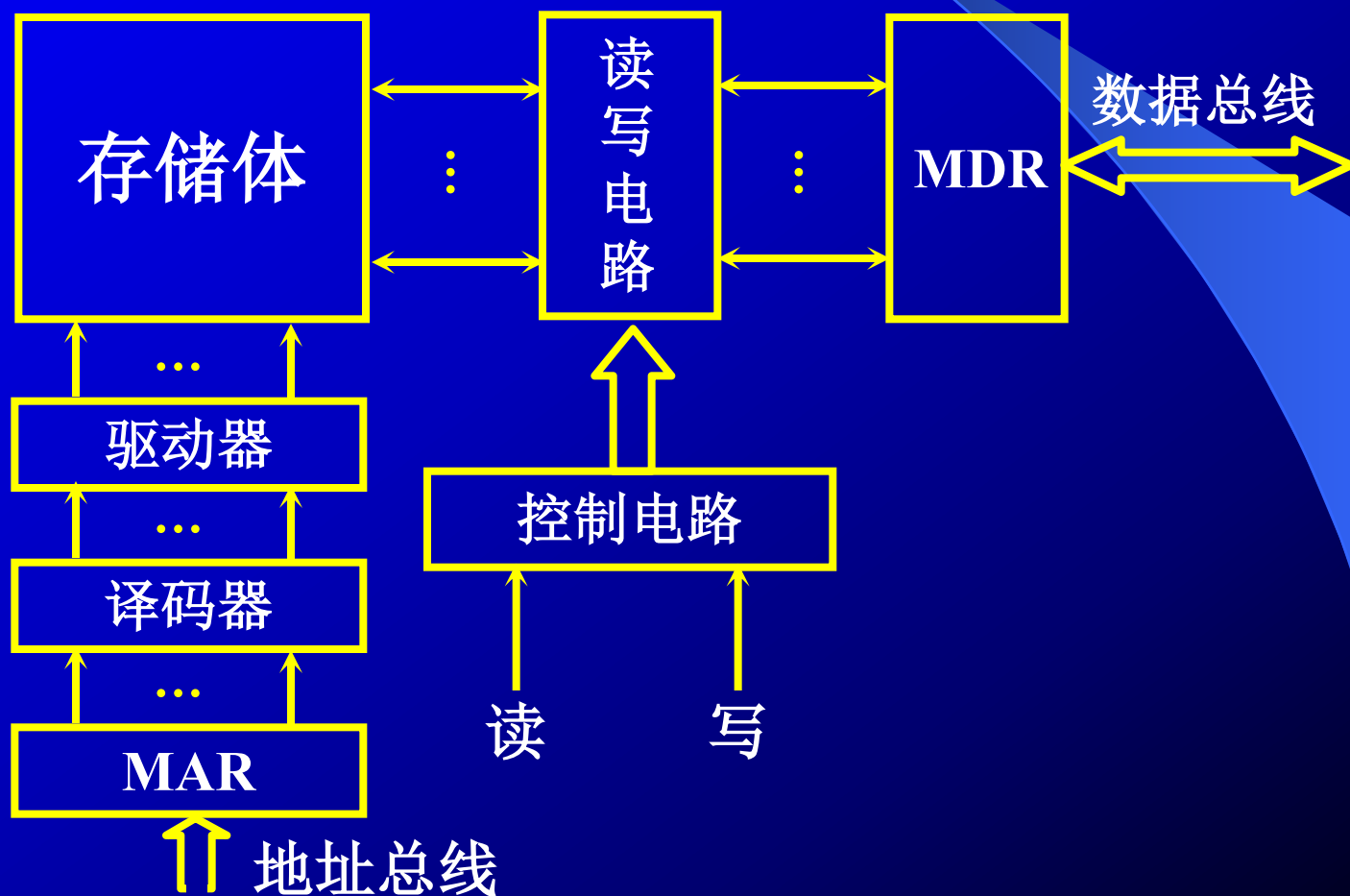
逻辑地址



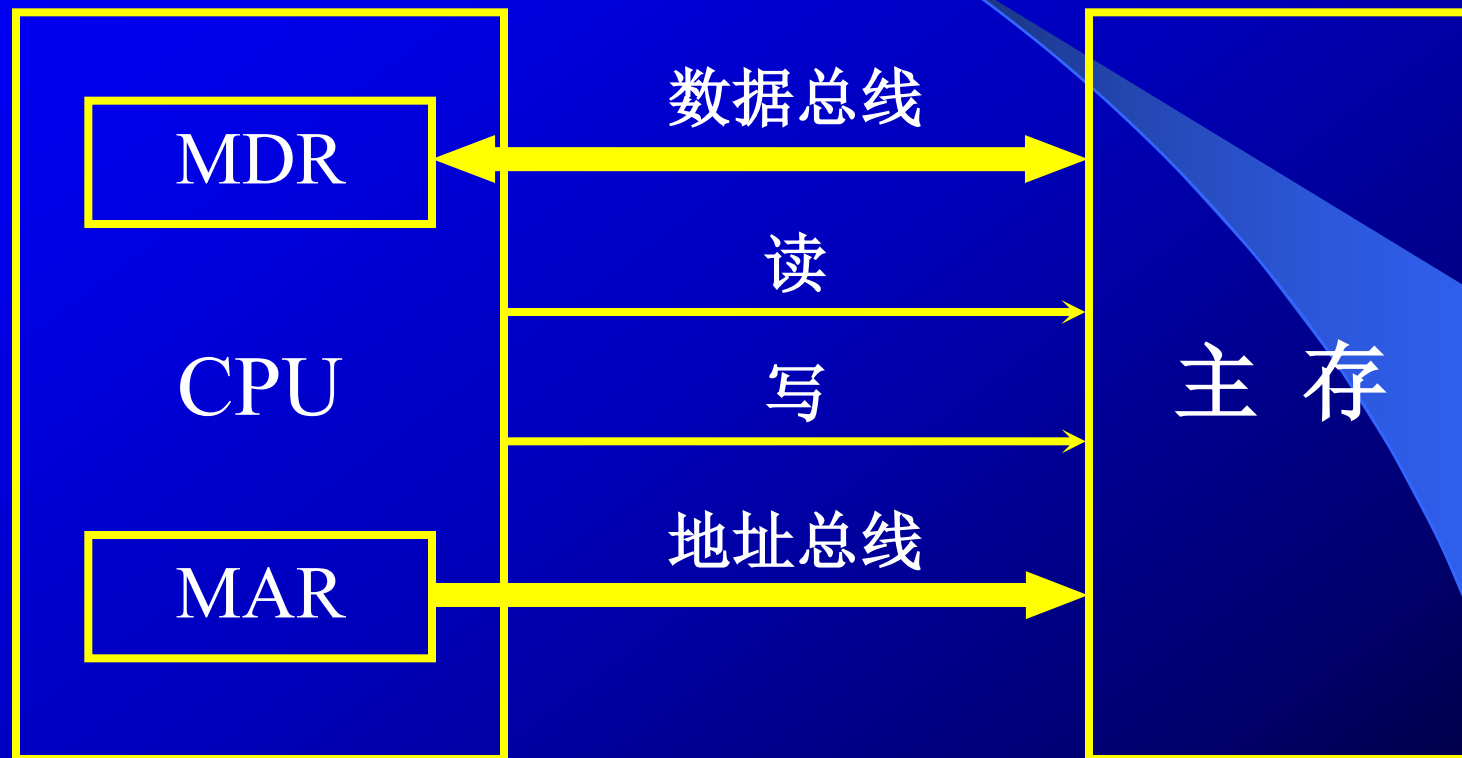
4.2 主存储器

一、概述

1. 主存的基本组成



2. 主存和 CPU 的联系



3. 主存中存储单元地址的分配

高位字节 地址为字地址

低位字节 地址为字地址

字地址	字节地址			
0	0	1	2	3
4	4	5	6	7
8	8	9	10	11

字地址	字节地址	
0	1	0
2	3	2
4	5	4

设地址线 24 根

若字长为 16 位

若字长为 32 位

按 字节 寻址 $2^{24} = 16 \text{ M}$

按 字 寻址 8 M

按 字 寻址 4 M

4. 主存的技术指标

4.2

(1) 存储容量 主存 存放二进制代码的总位数

(2) 存储速度

• 存取时间 存储器的 访问时间

读出时间 写入时间

• 存取周期 连续两次独立的存储器操作

(读或写) 所需的 最小间隔时间

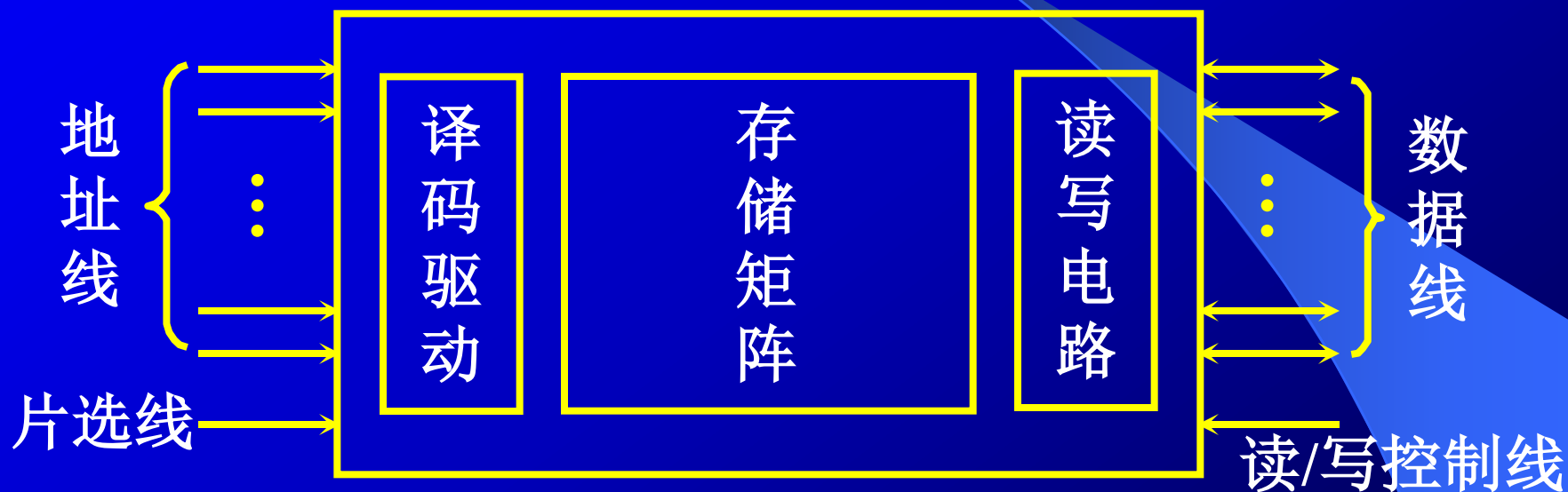
读周期 写周期

(3) 存储器的带宽 位/秒



二、半导体存储芯片简介

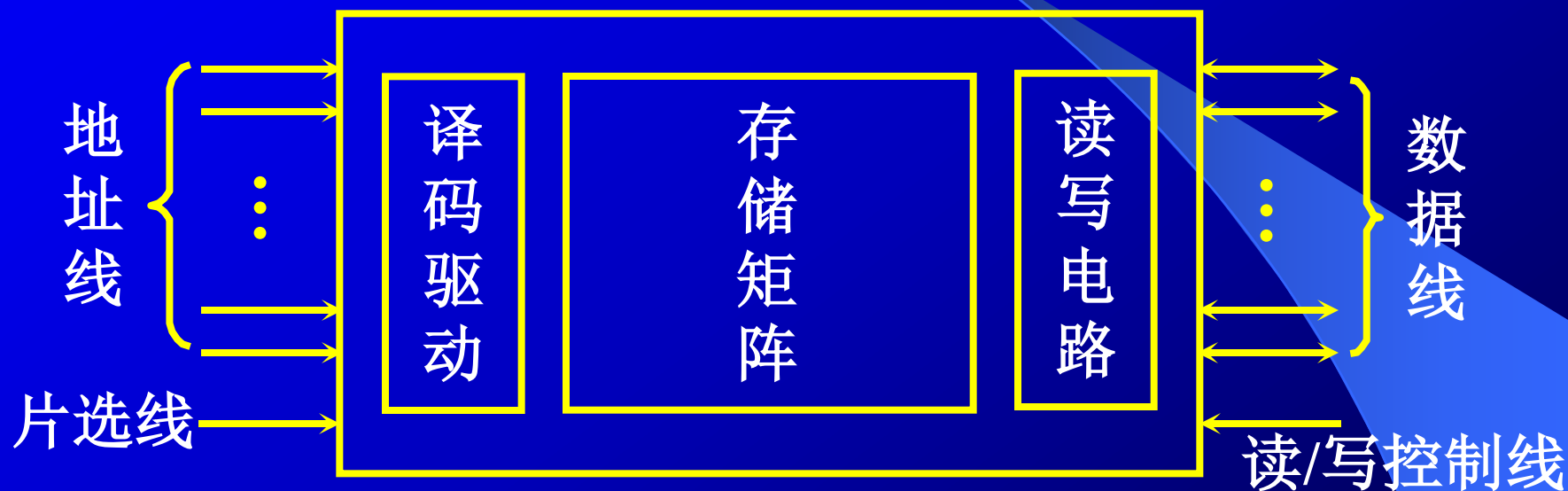
1. 半导体存储芯片的基本结构



地址线（单向）	数据线（双向）	芯片容量
10	4	1K×4位
14	1	16K×1位
13	8	8K×8位

二、半导体存储芯片简介

1. 半导体存储芯片的基本结构

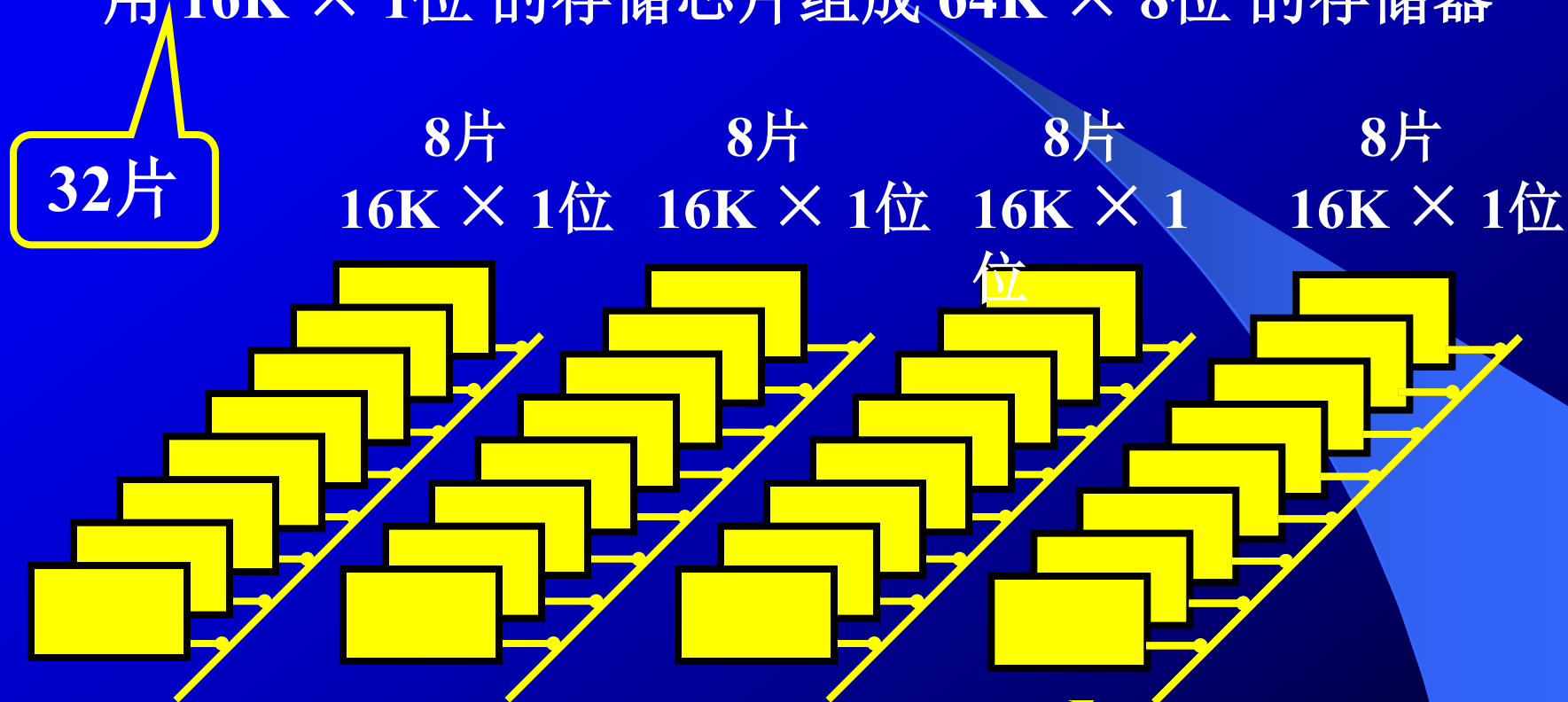


片选线 $\overline{\text{CS}}$ $\overline{\text{CE}}$

读/写控制线 $\overline{\text{WE}}$ (低电平写 高电平读)
 $\overline{\text{OE}}$ (允许读) $\overline{\text{WE}}$ (允许写)

存储芯片片选线的作用

用 $16\text{K} \times 1$ 位的存储芯片组成 $64\text{K} \times 8$ 位的存储器

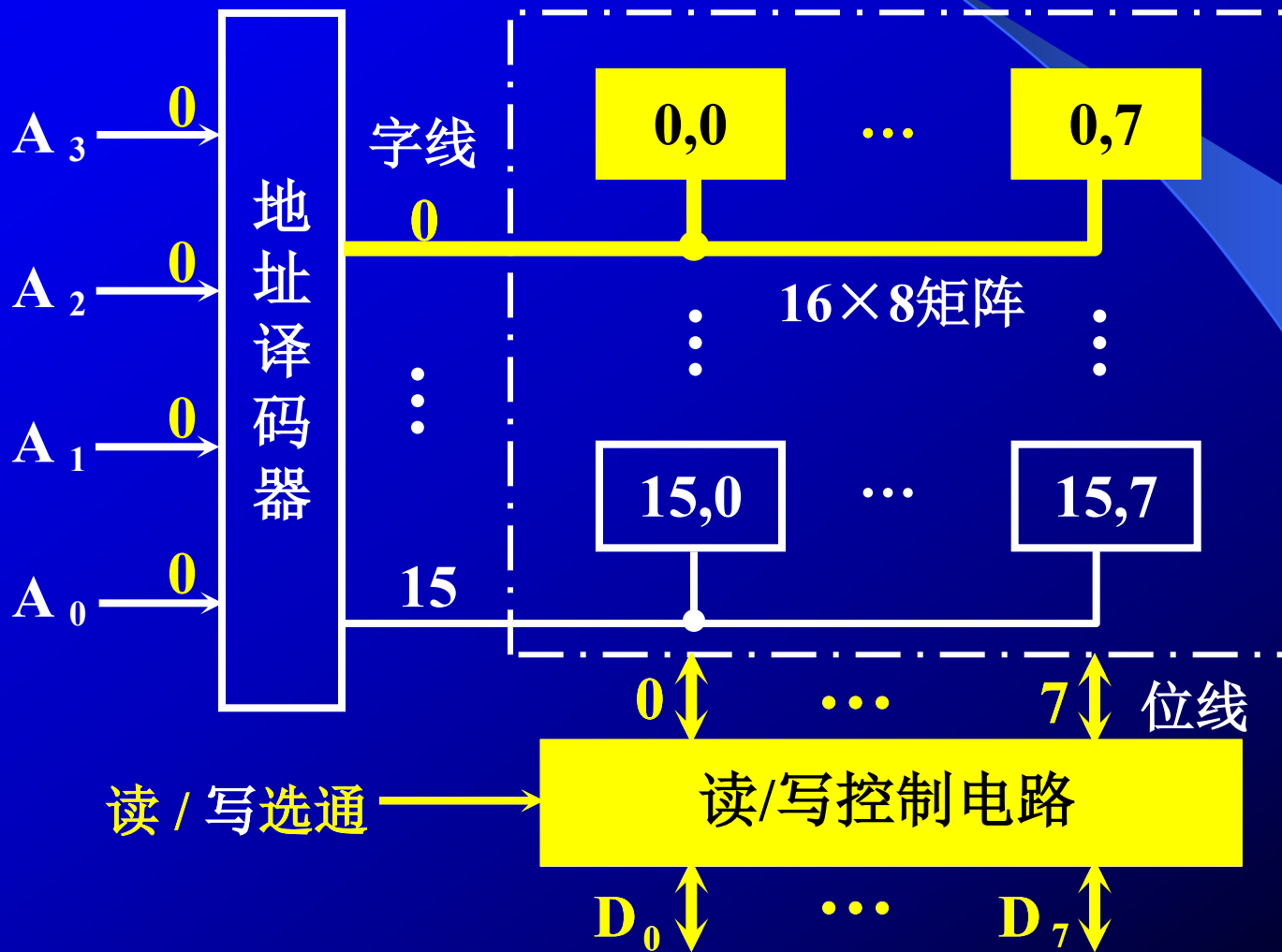


当地址为 65 535 时，此 8 片的片选有效

2. 半导体存储芯片的译码驱动方式

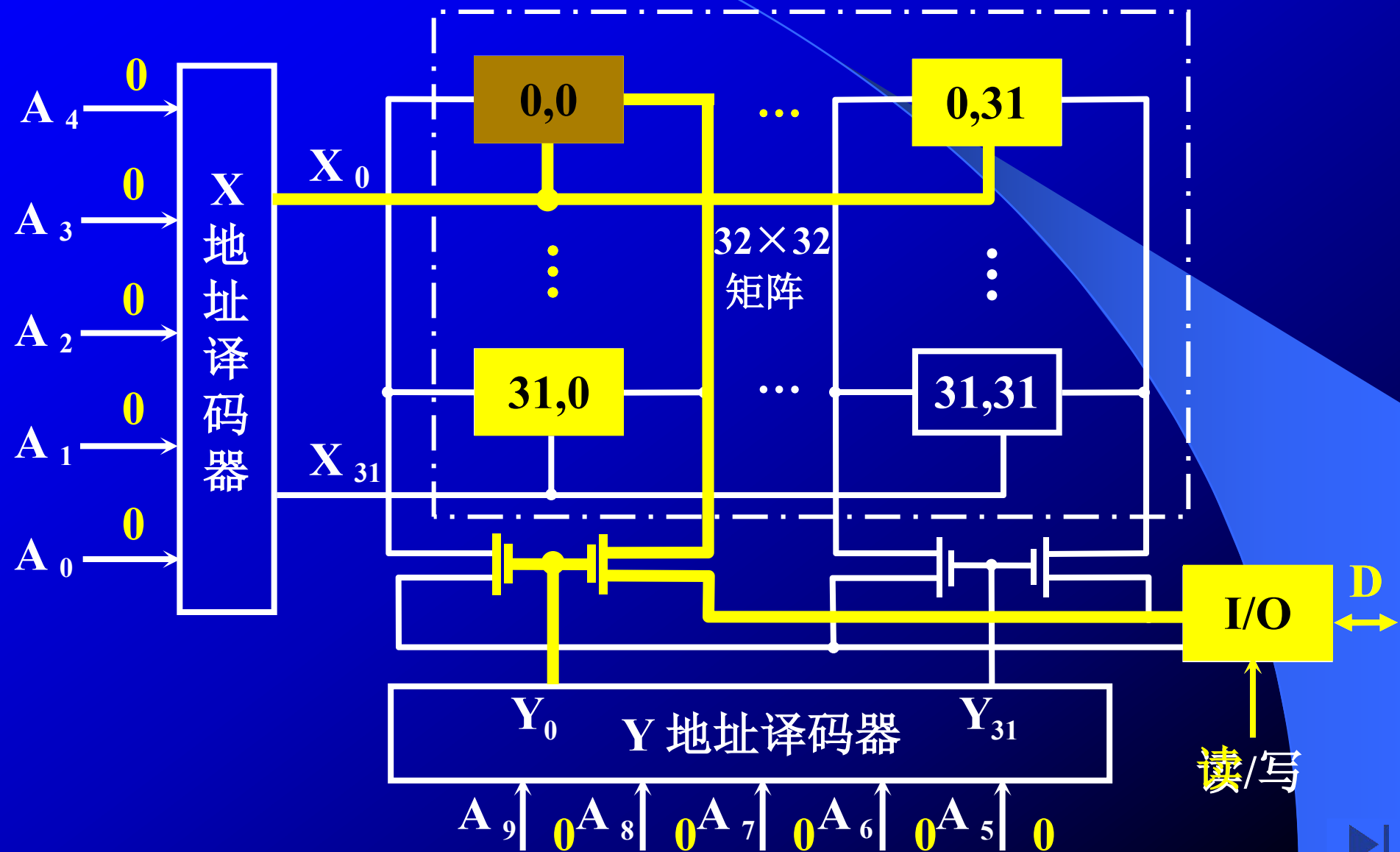
4.2

(1) 线选法



(2) 重合法

4.2

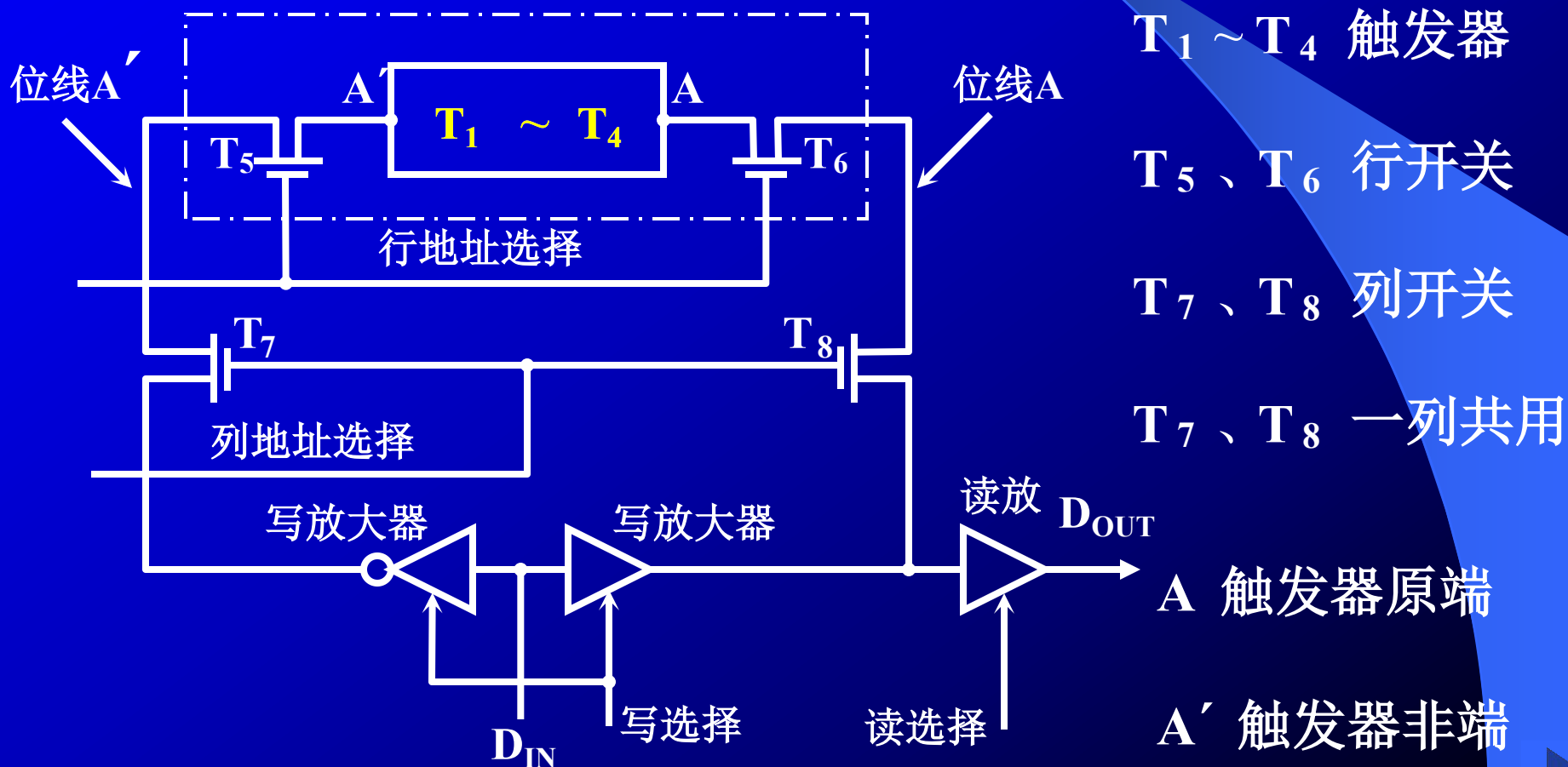


三、随机存取存储器 (RAM)

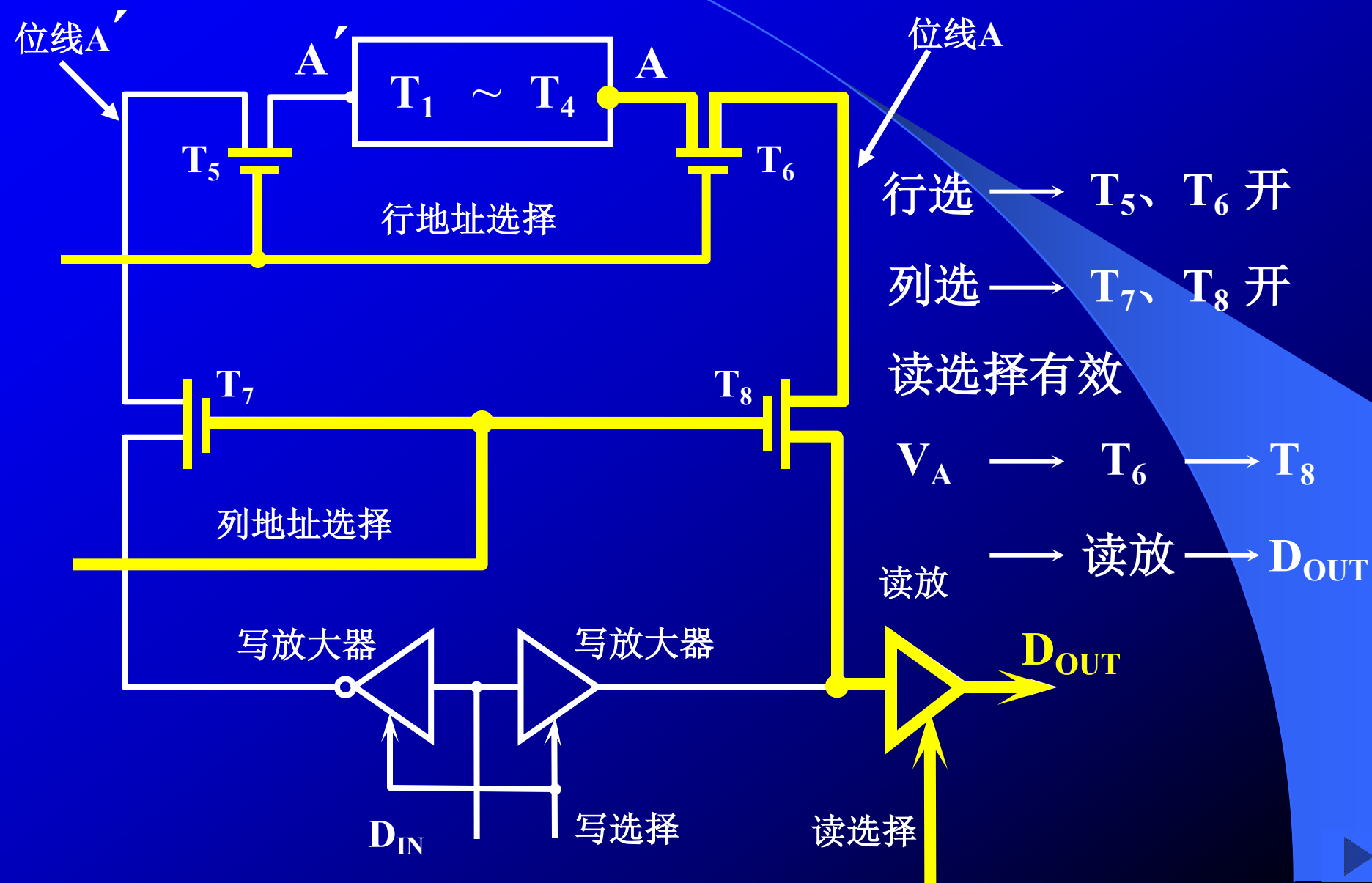
4.2

1. 静态 RAM (SRAM)

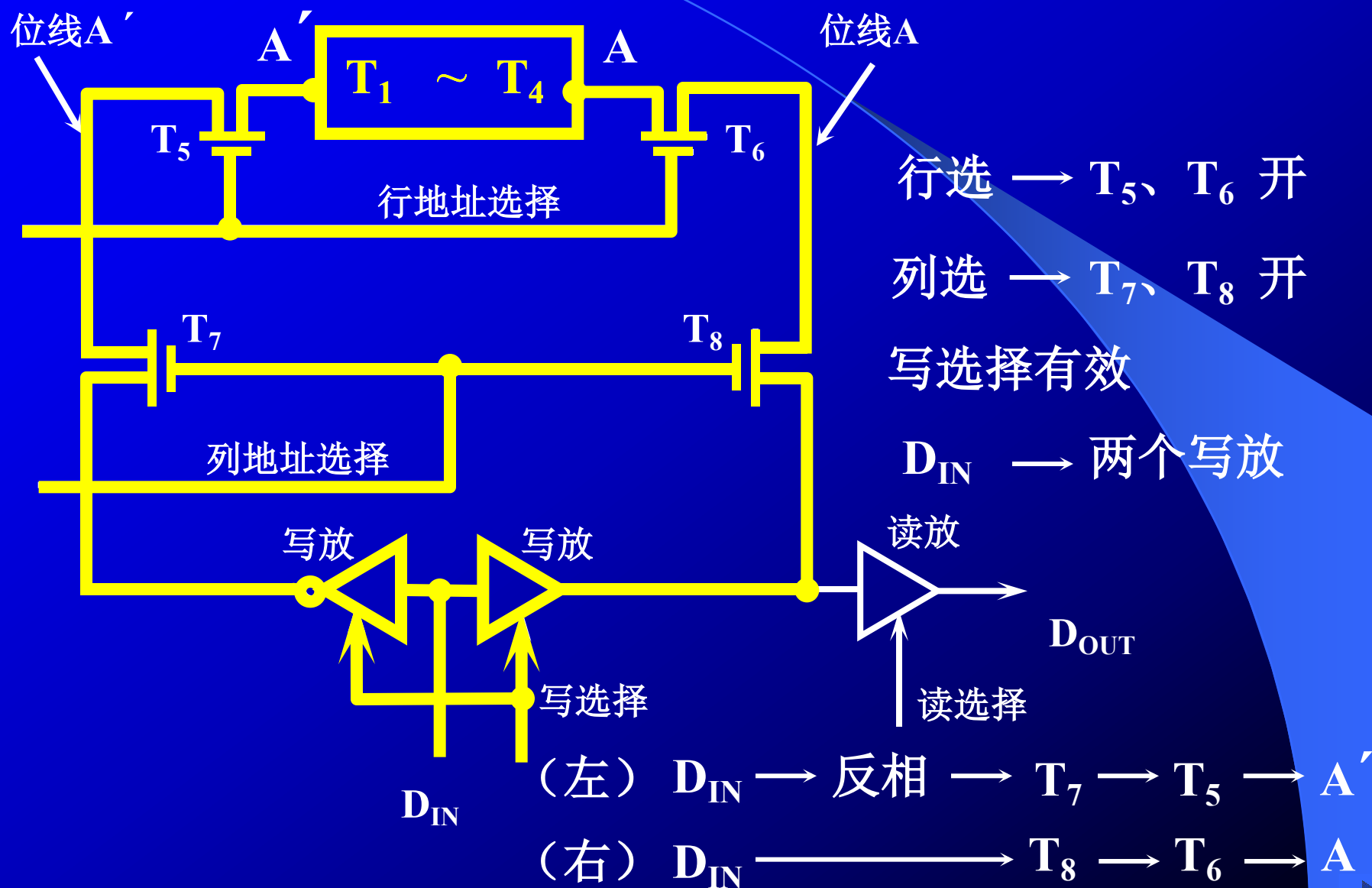
(1) 静态 RAM 基本电路



① 静态 RAM 基本电路的 读 操作

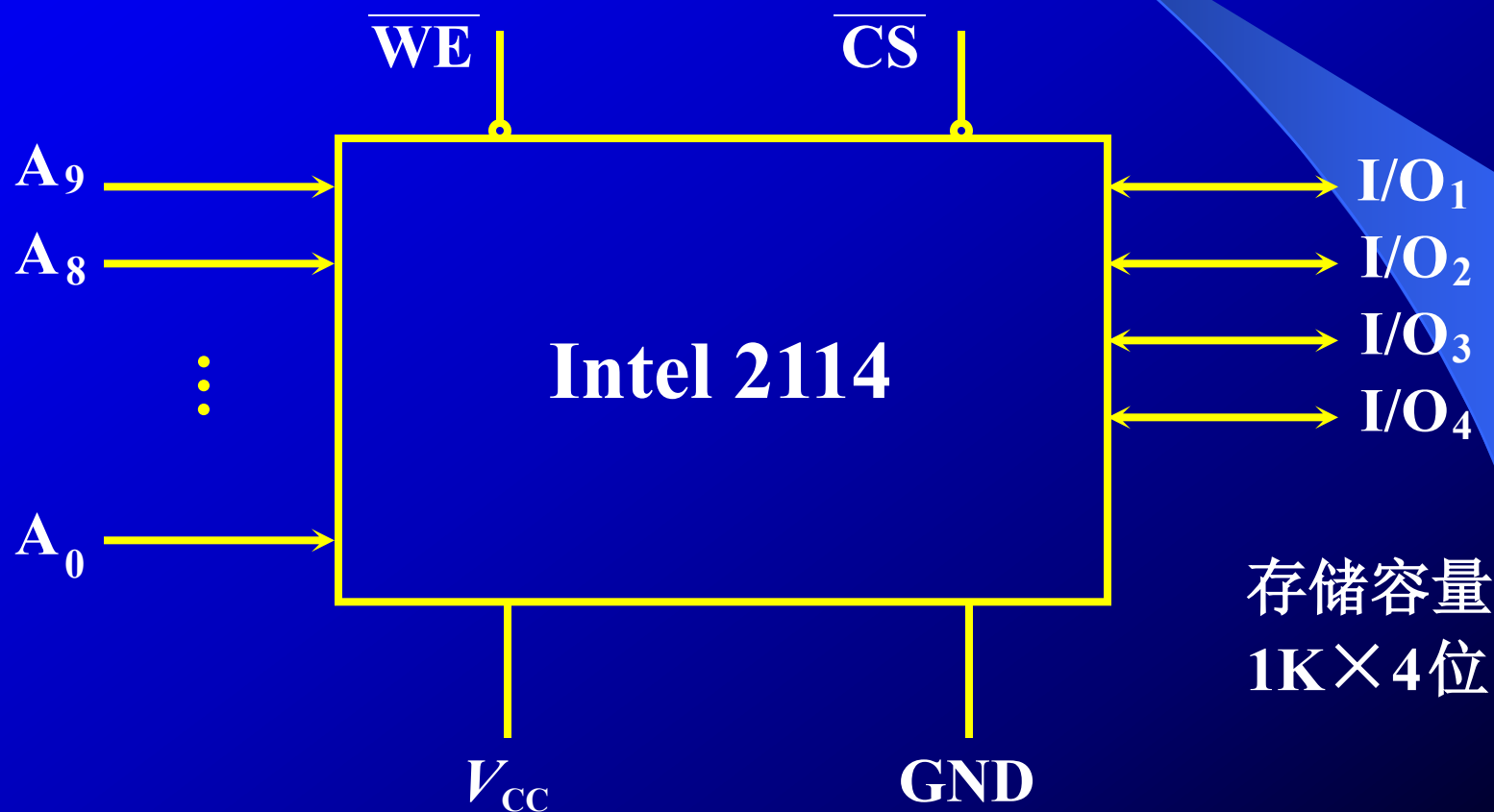


② 静态 RAM 基本电路的 写 操作

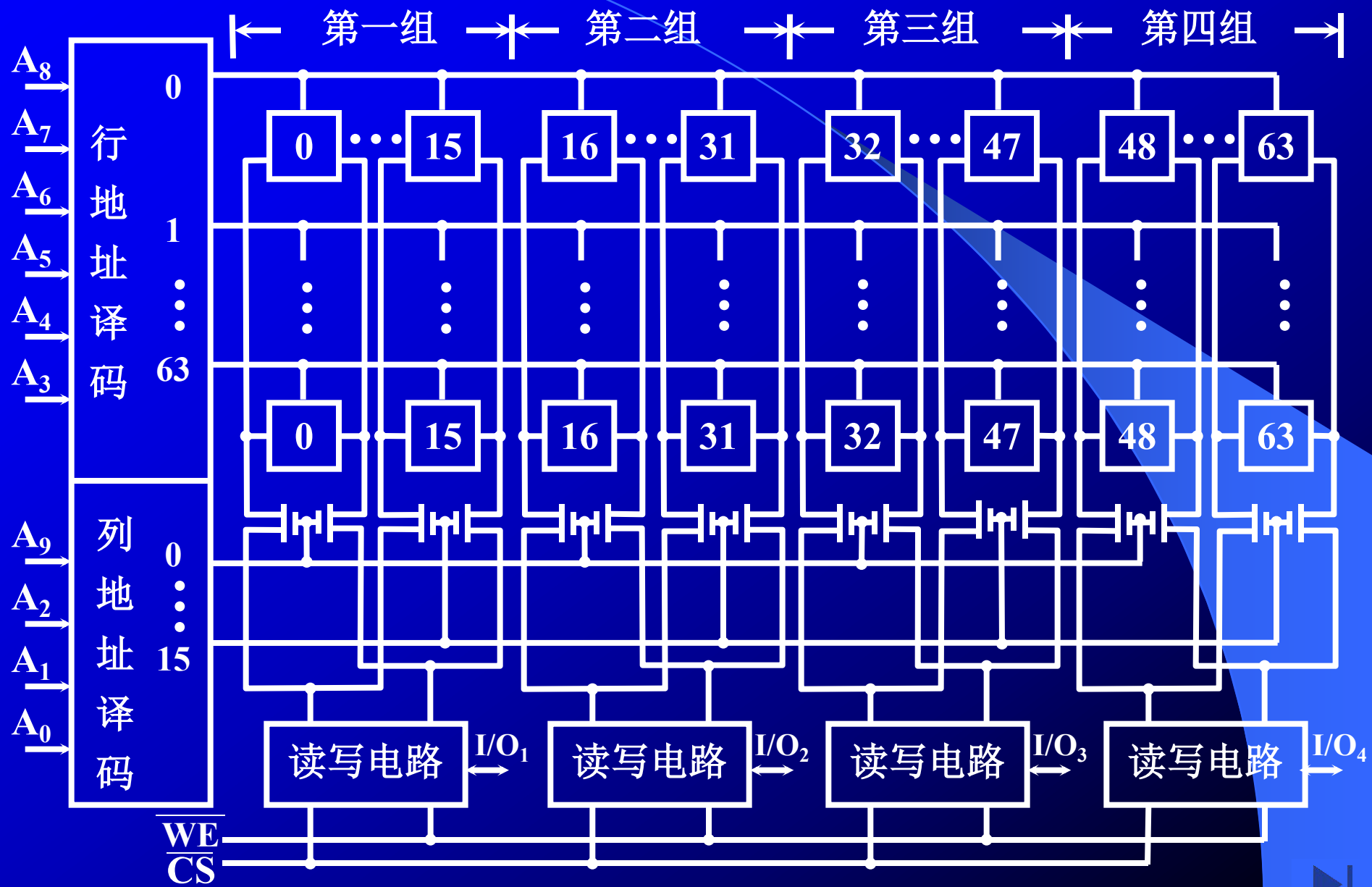


(2) 静态 RAM 芯片举例

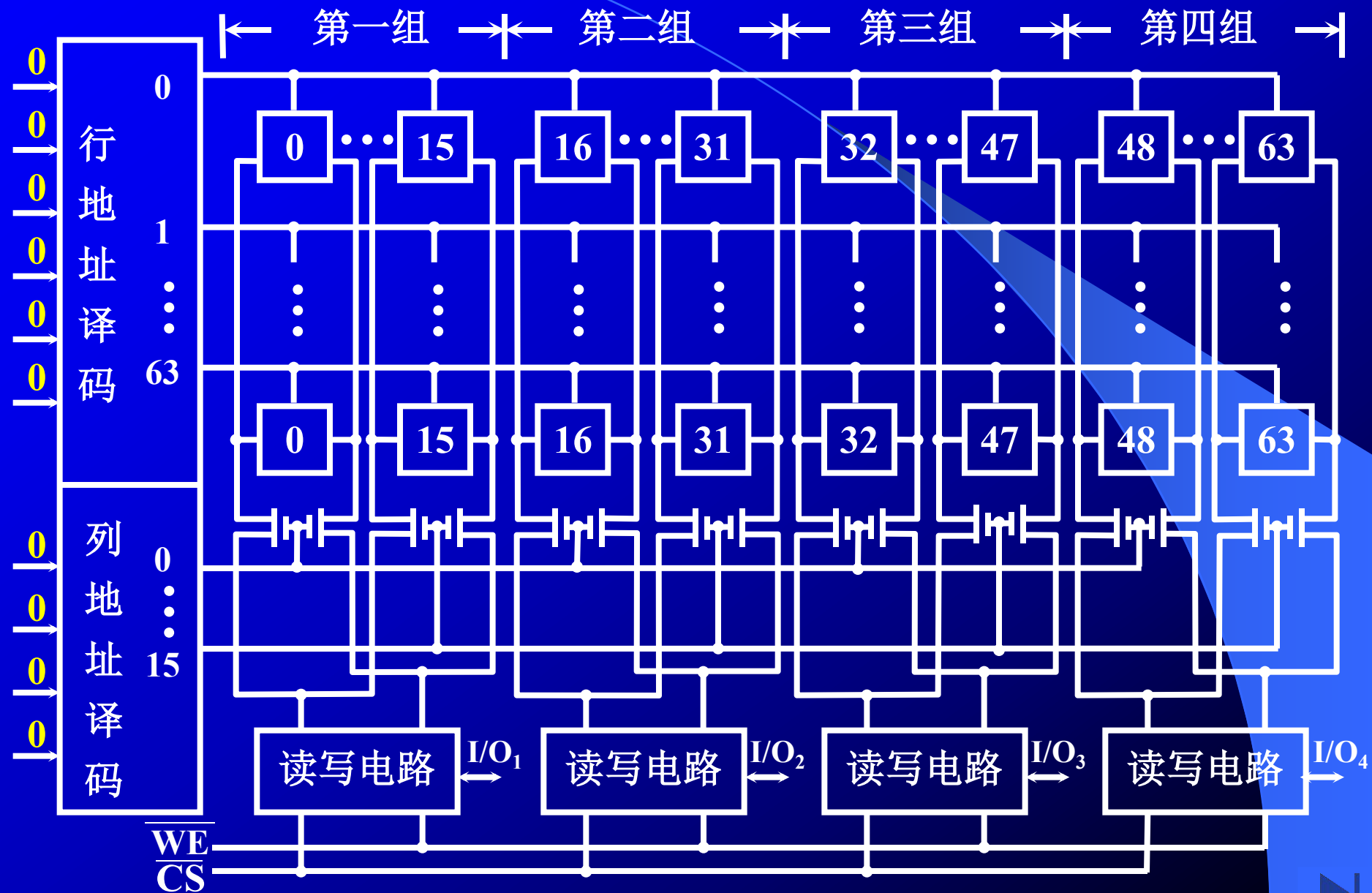
① Intel 2114 外特性



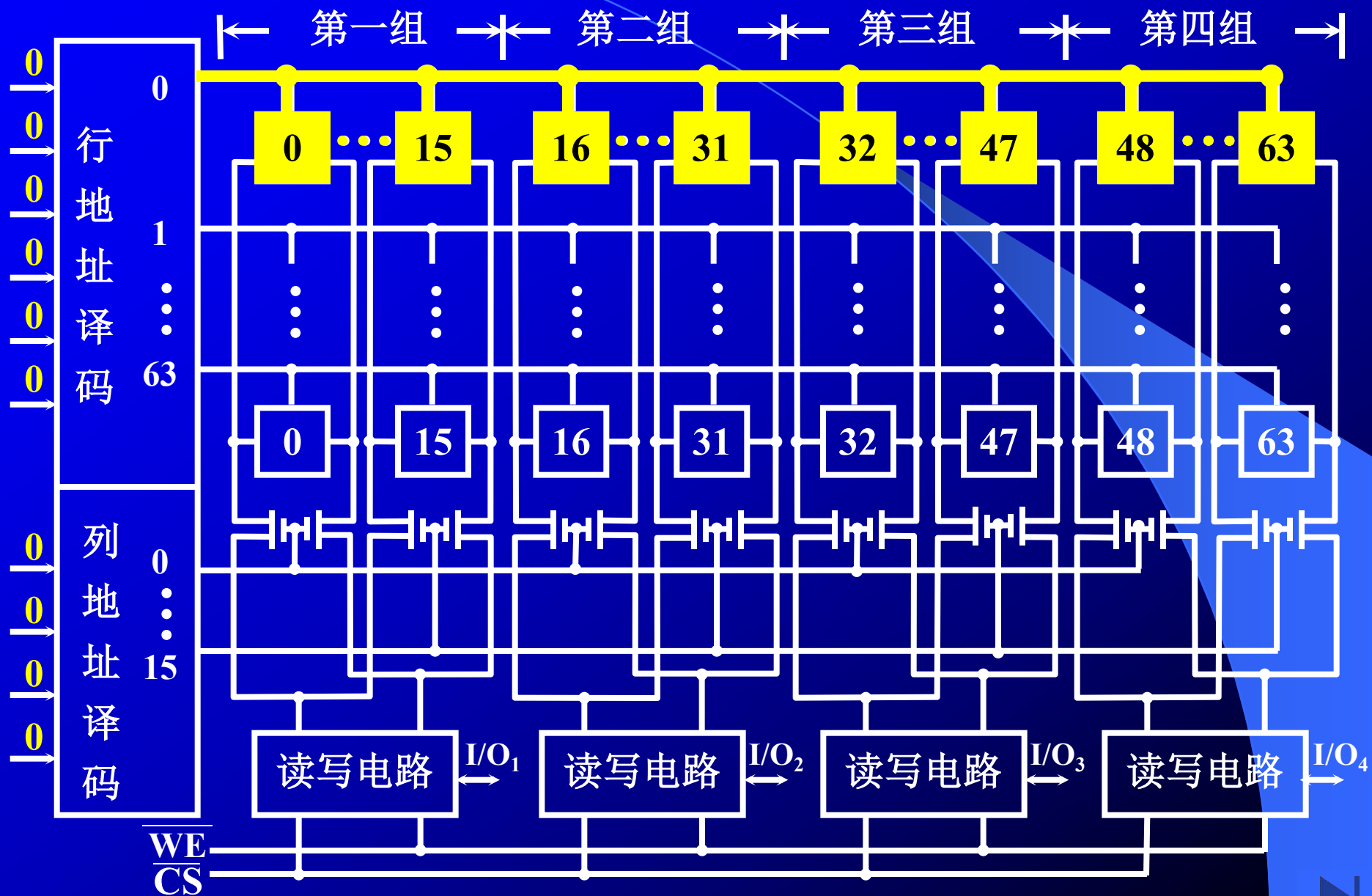
② Intel 2114 RAM 矩阵 (64×64) 读 4.2



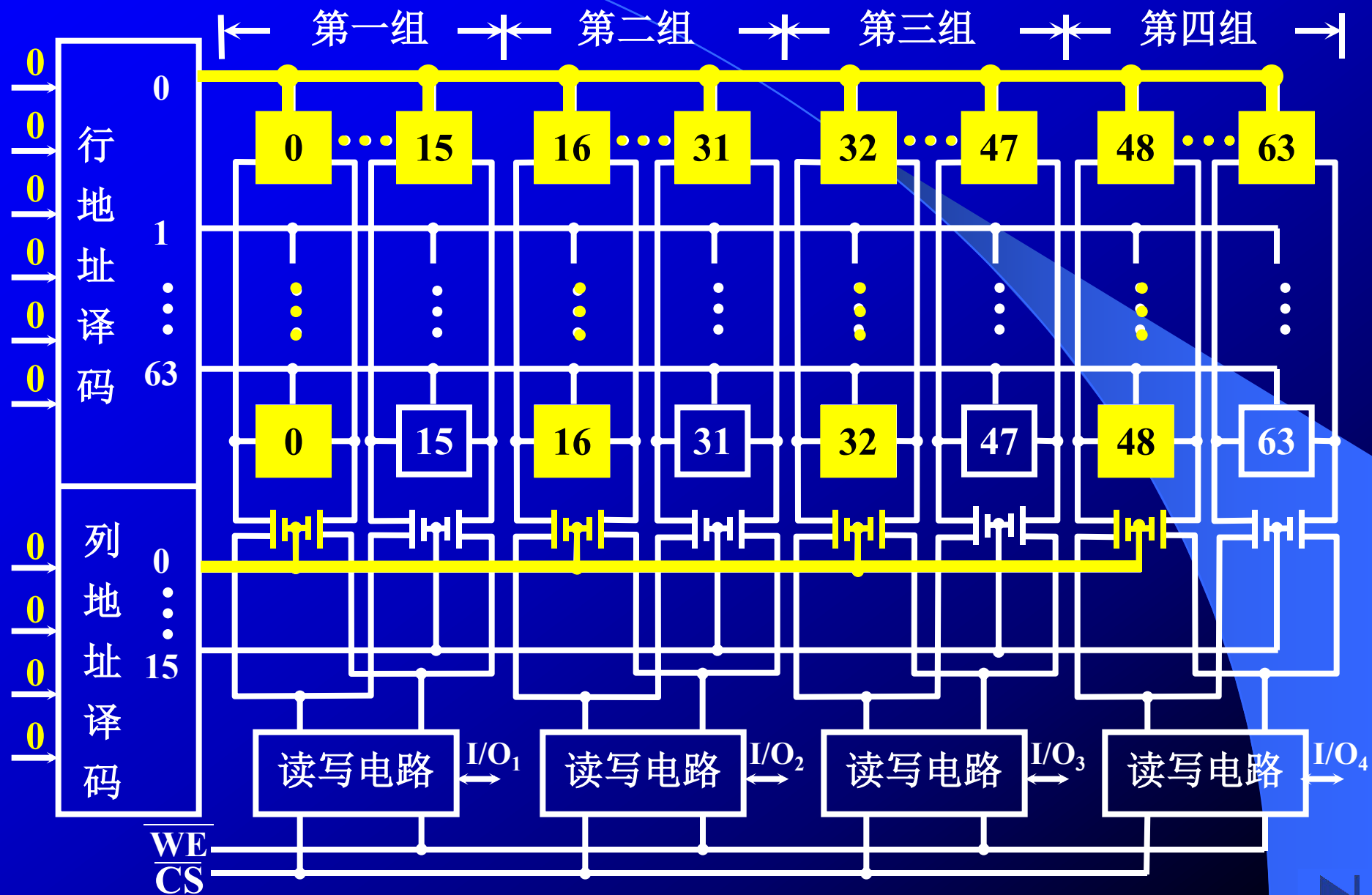
② Intel 2114 RAM 矩阵 (64 × 64) 读 4.2



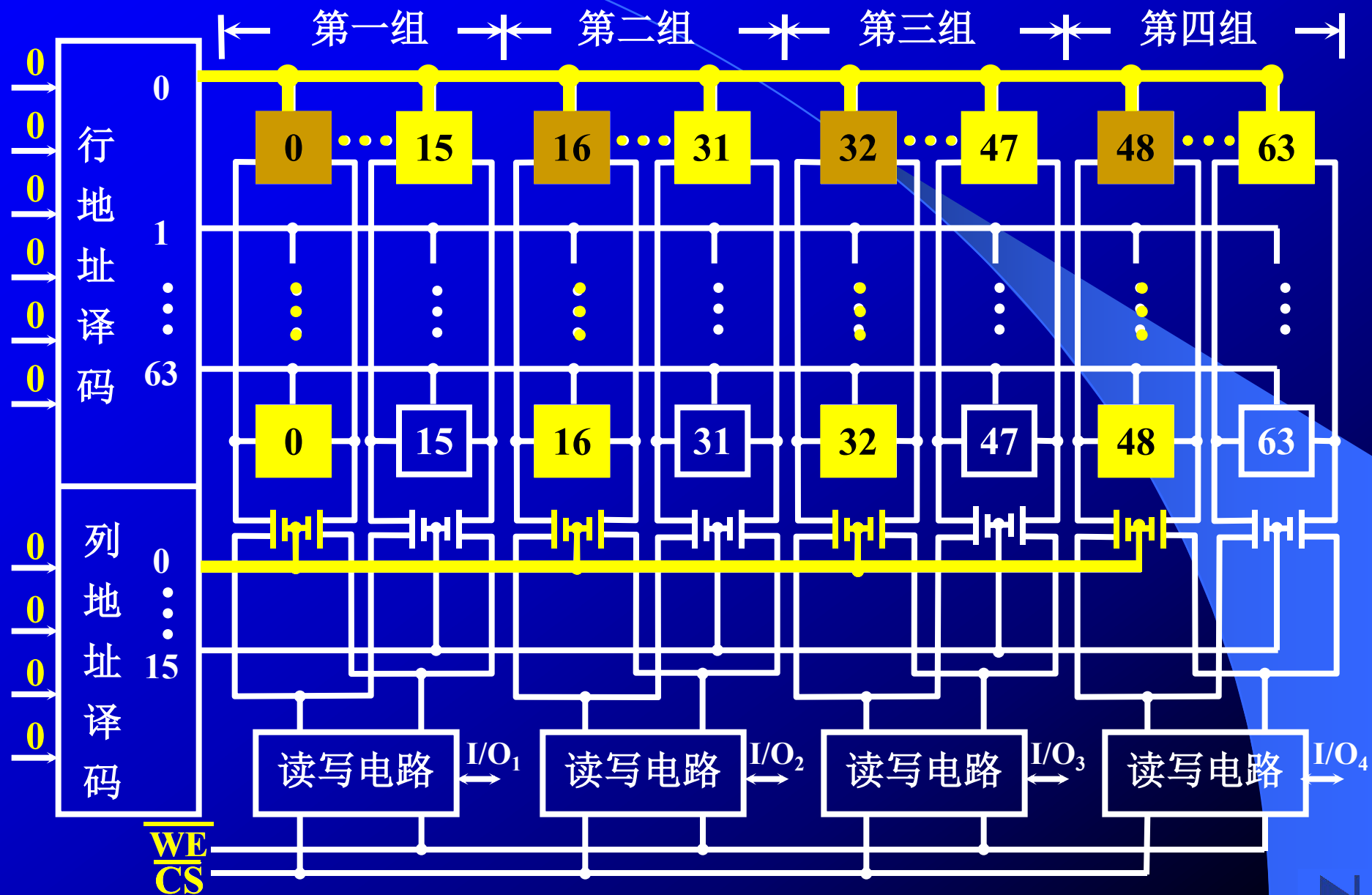
② Intel 2114 RAM 矩阵 (64 × 64) 读 4.2



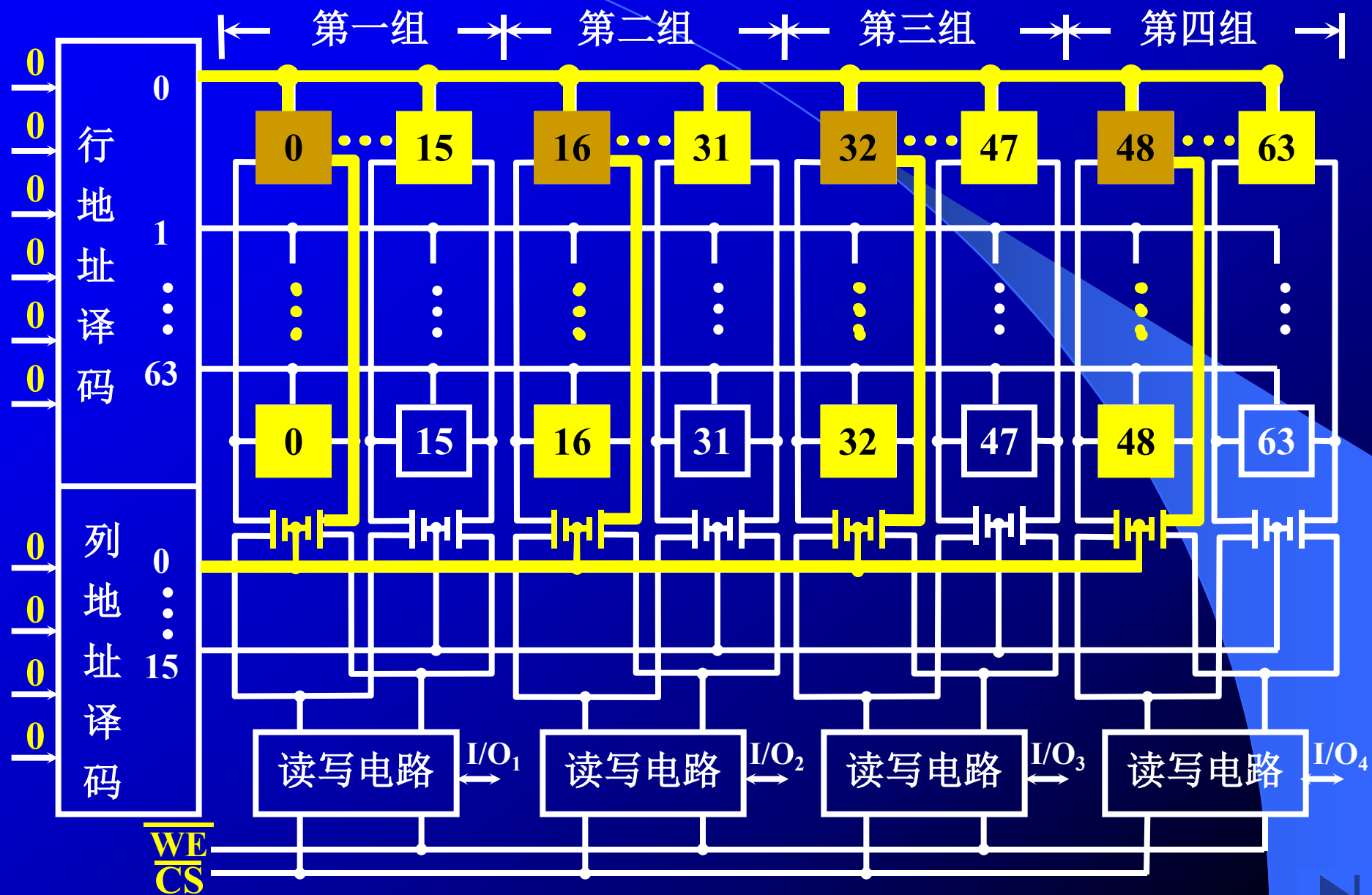
② Intel 2114 RAM 矩阵 (64 × 64) 读 4.2



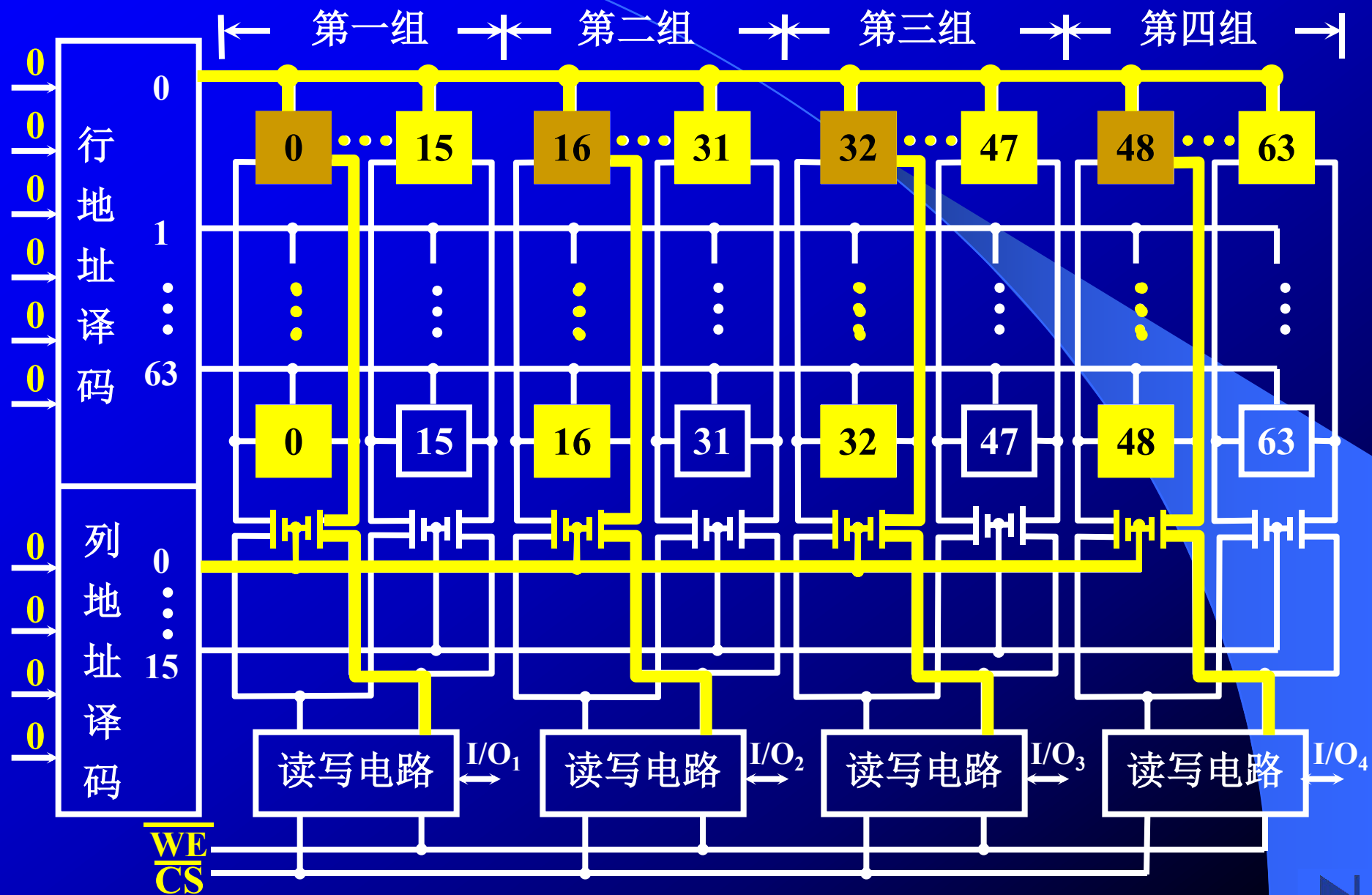
② Intel 2114 RAM 矩阵 (64 × 64) 读 4.2



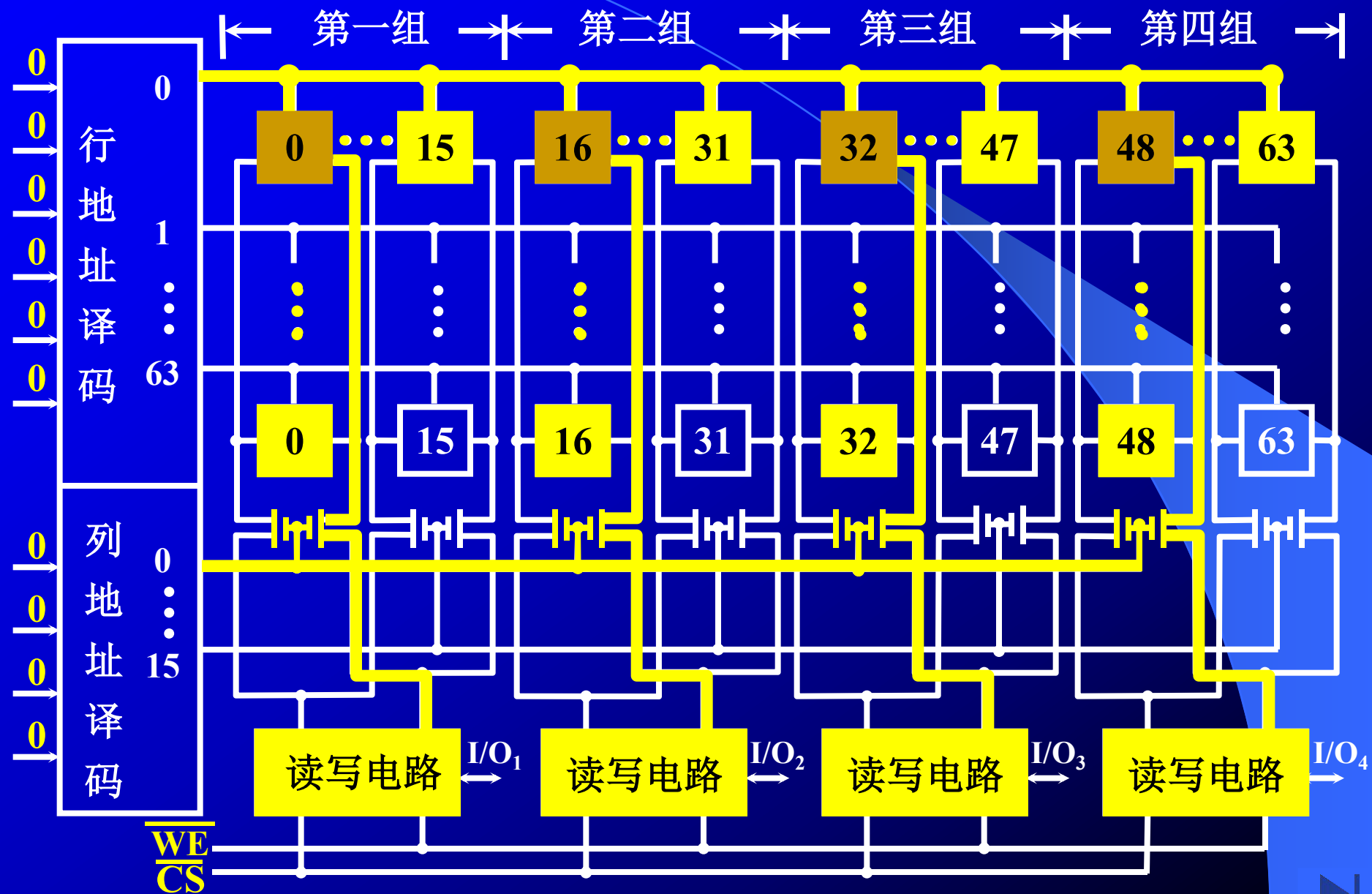
② Intel 2114 RAM 矩阵 (64×64) 读 4.2



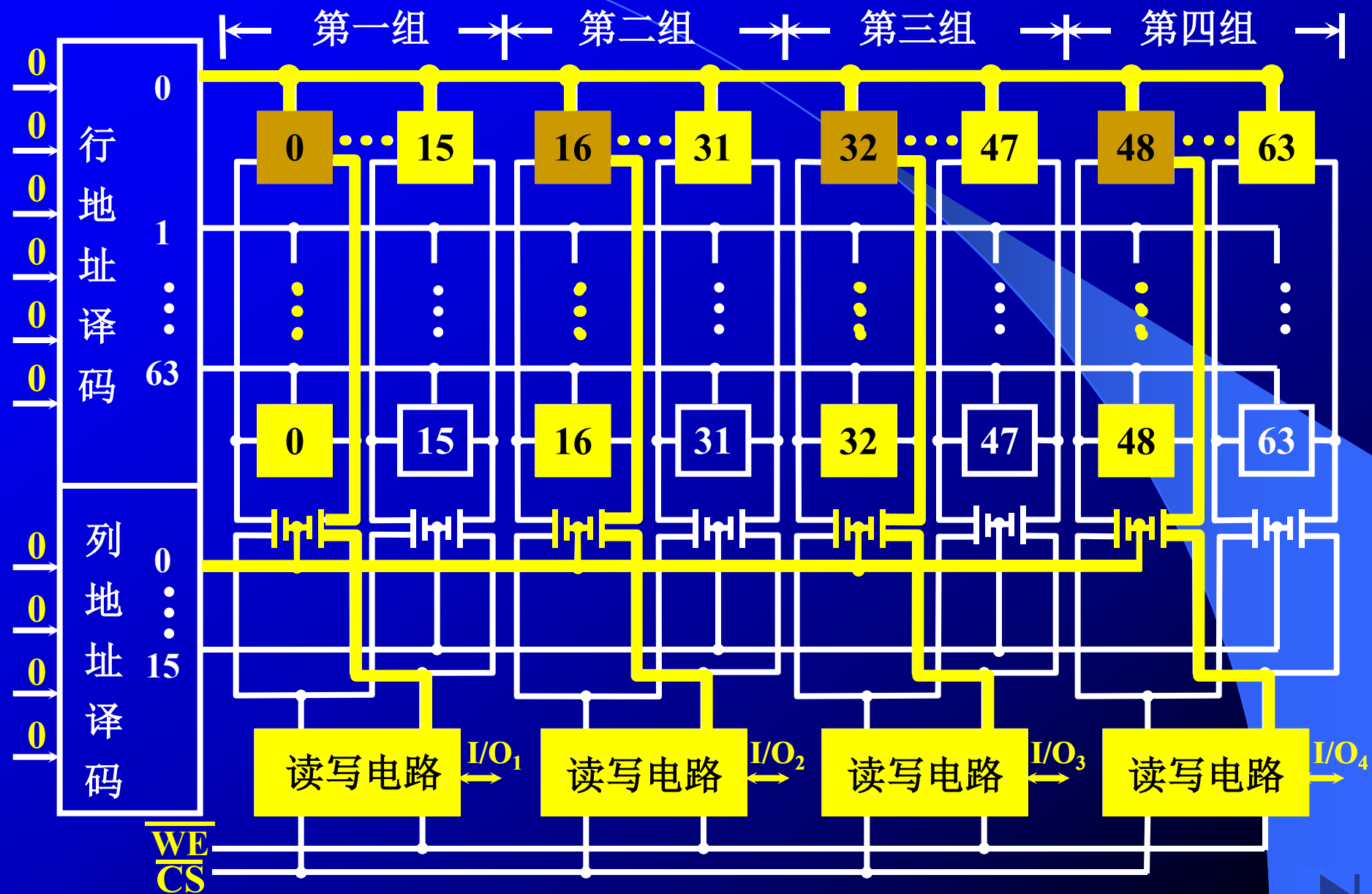
② Intel 2114 RAM 矩阵 (64×64) 读 4.2



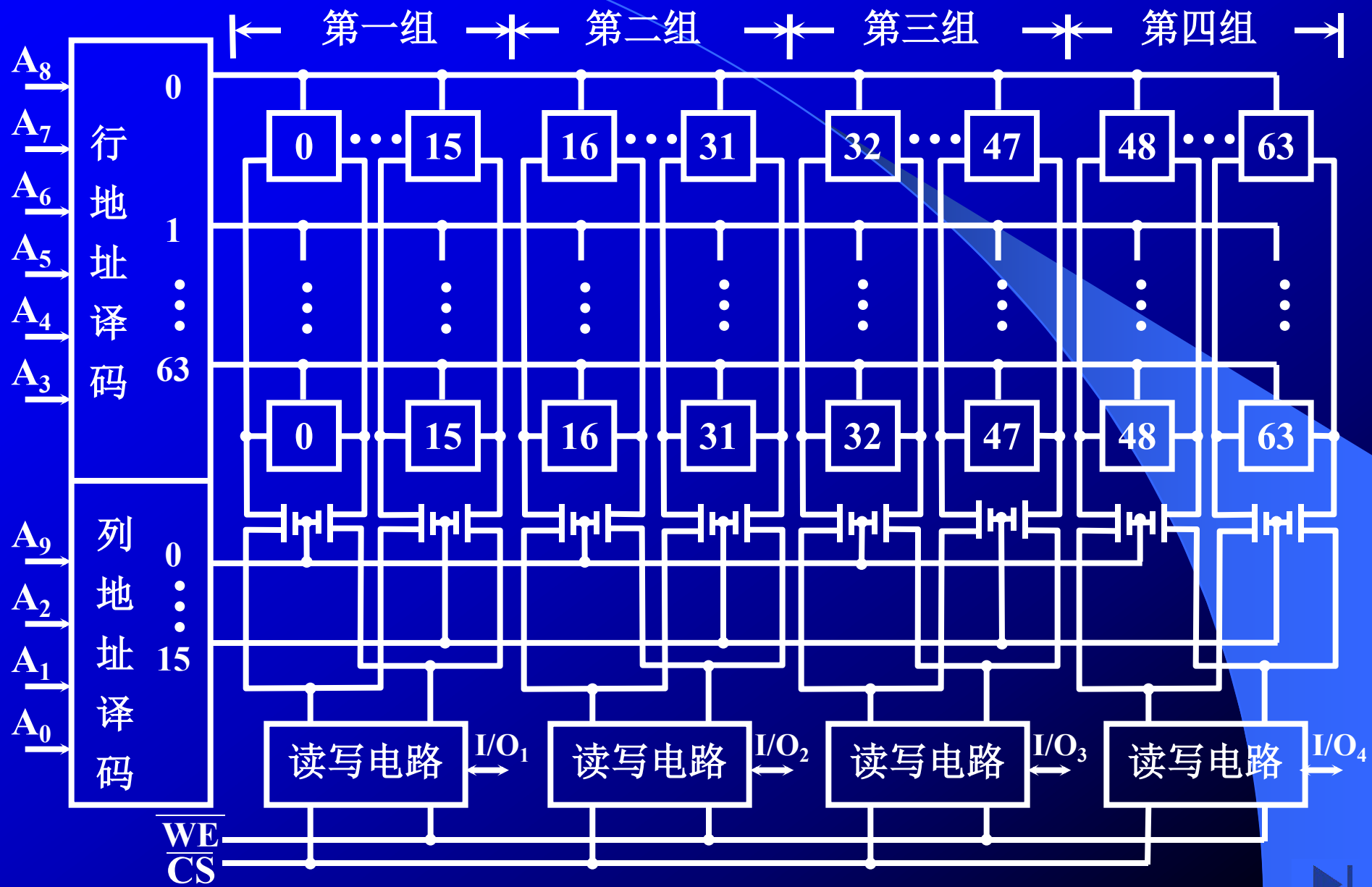
② Intel 2114 RAM 矩阵 (64×64) 读 4.2



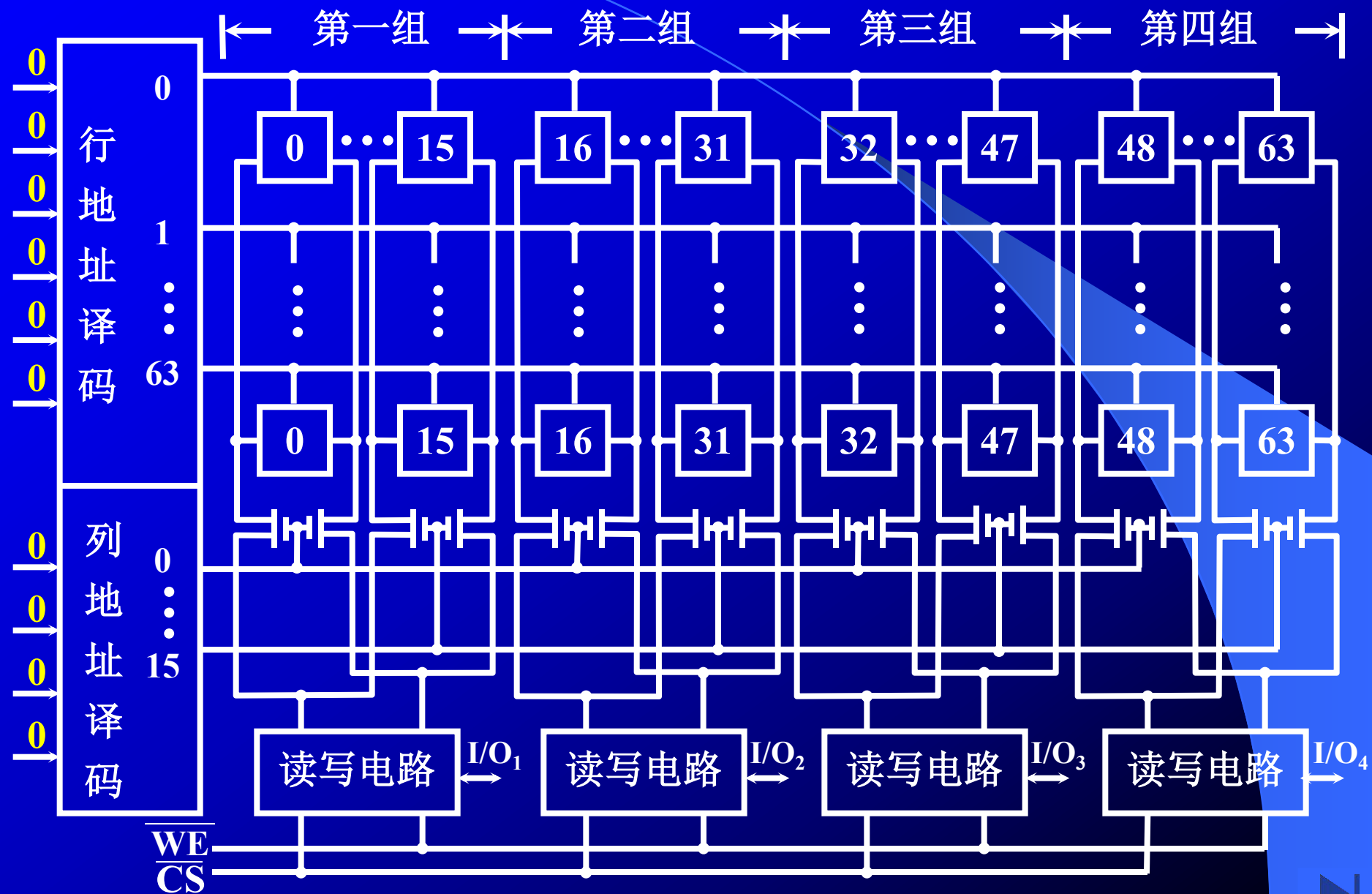
② Intel 2114 RAM 矩阵 (64×64) 读 4.2



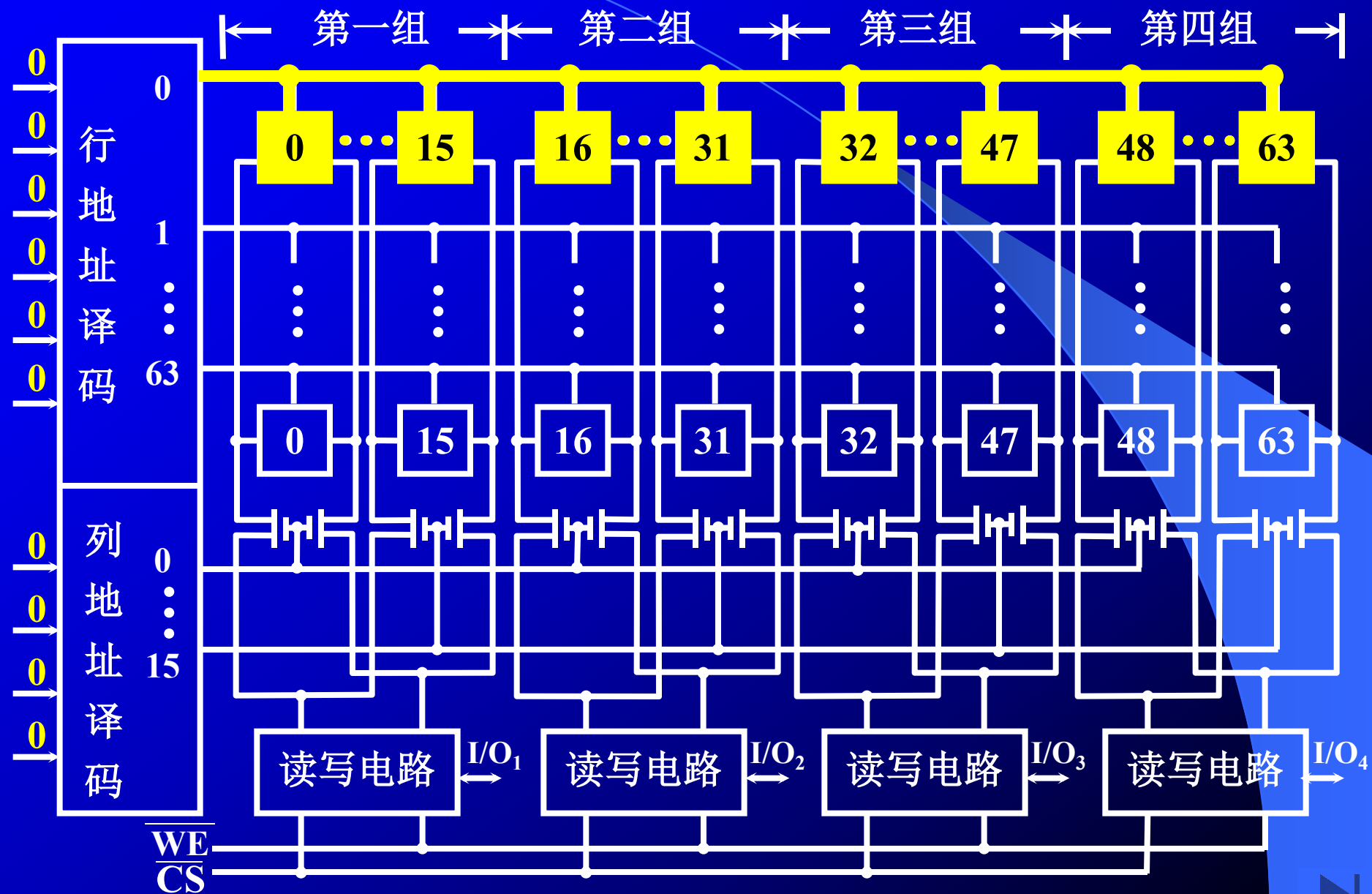
③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2



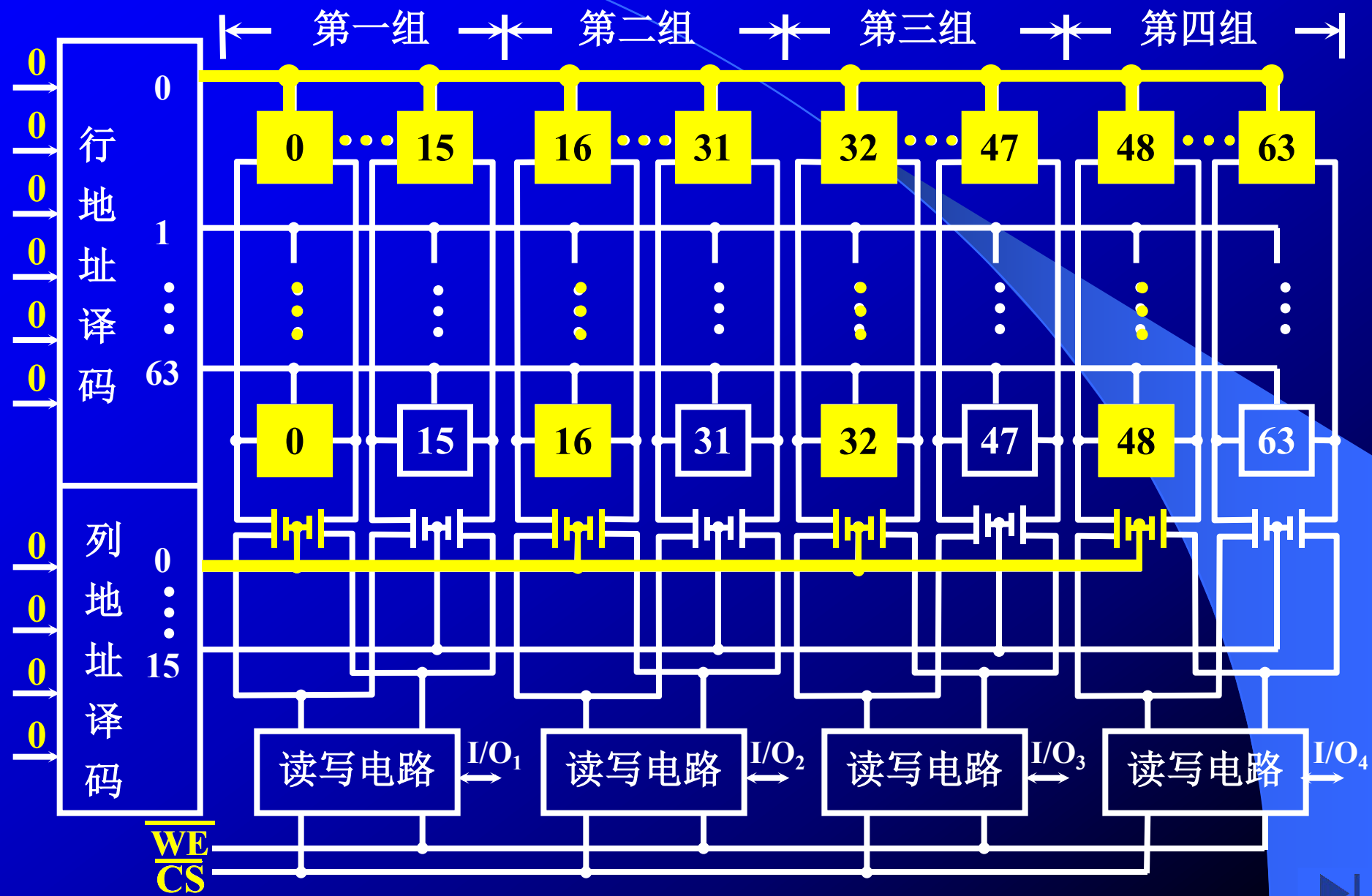
③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2



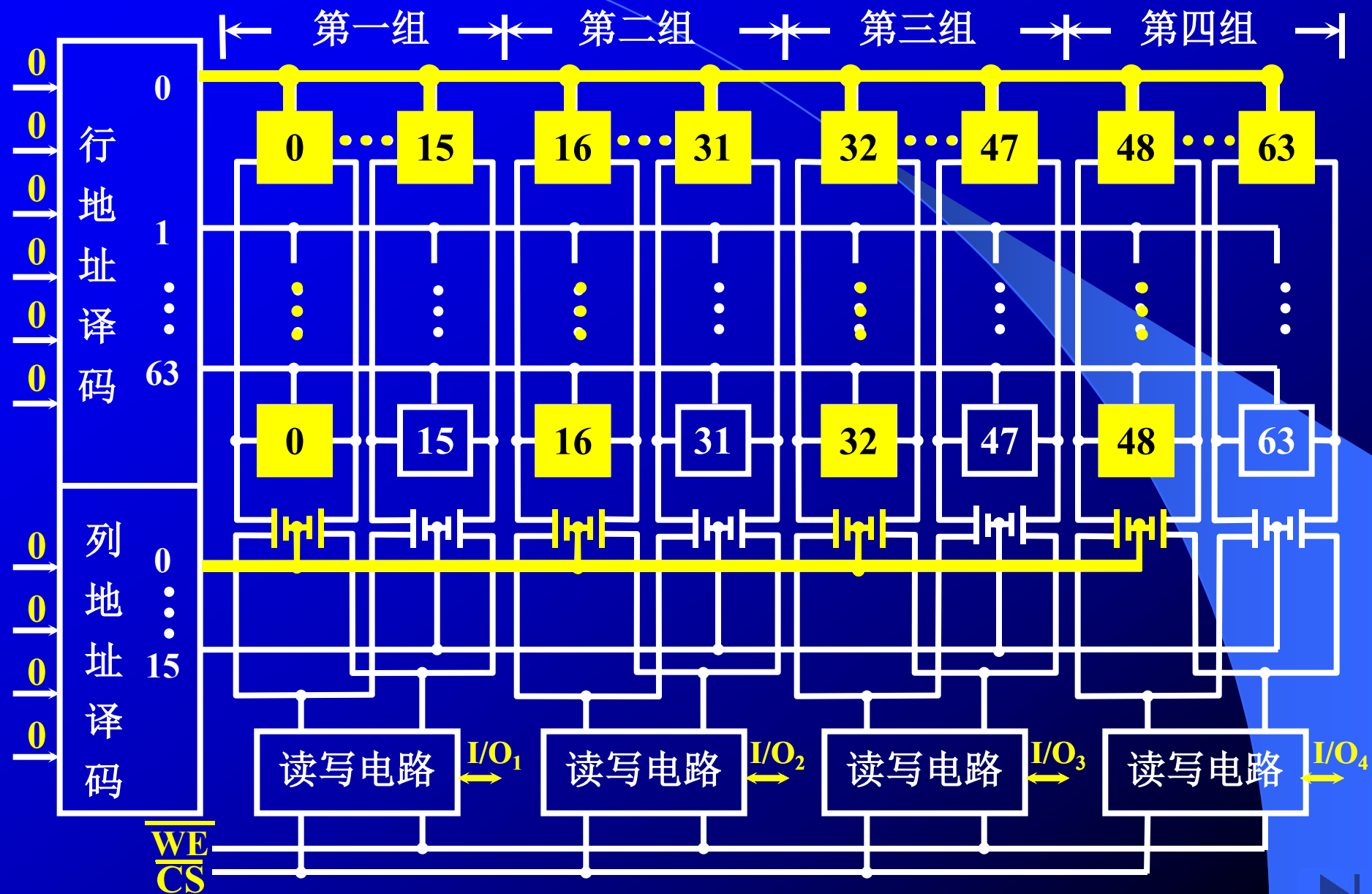
③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2



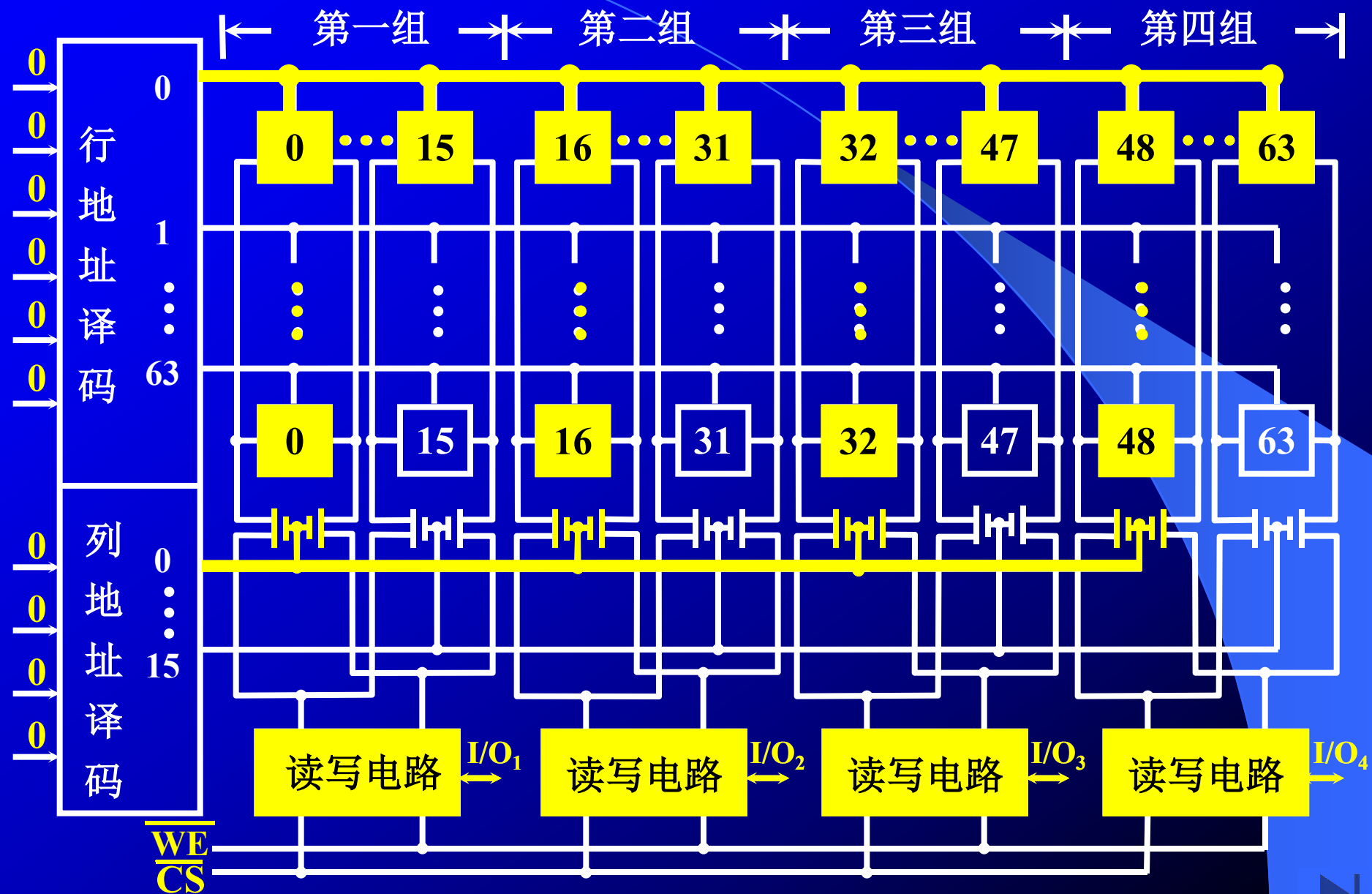
③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2



③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2



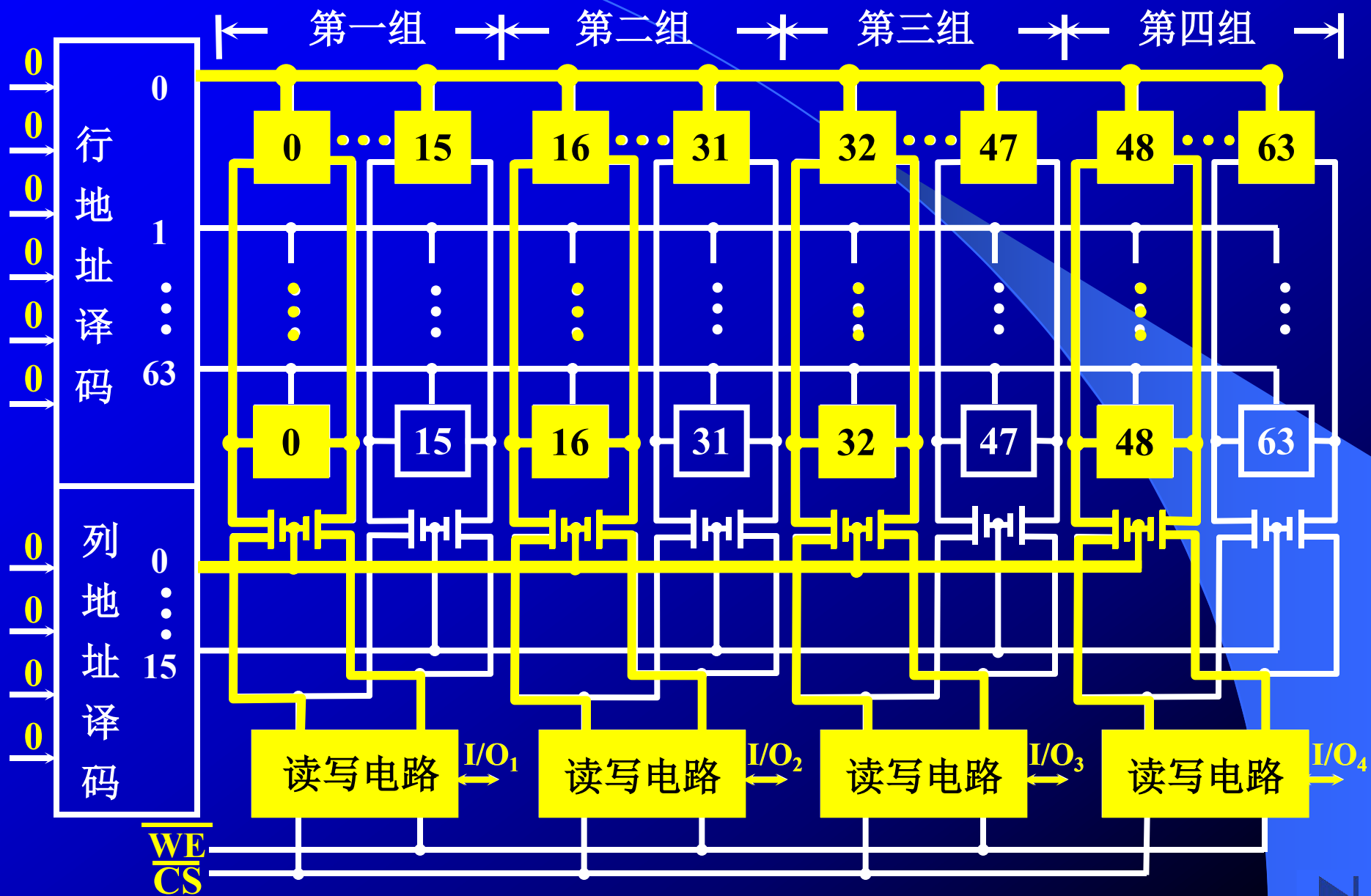
③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2



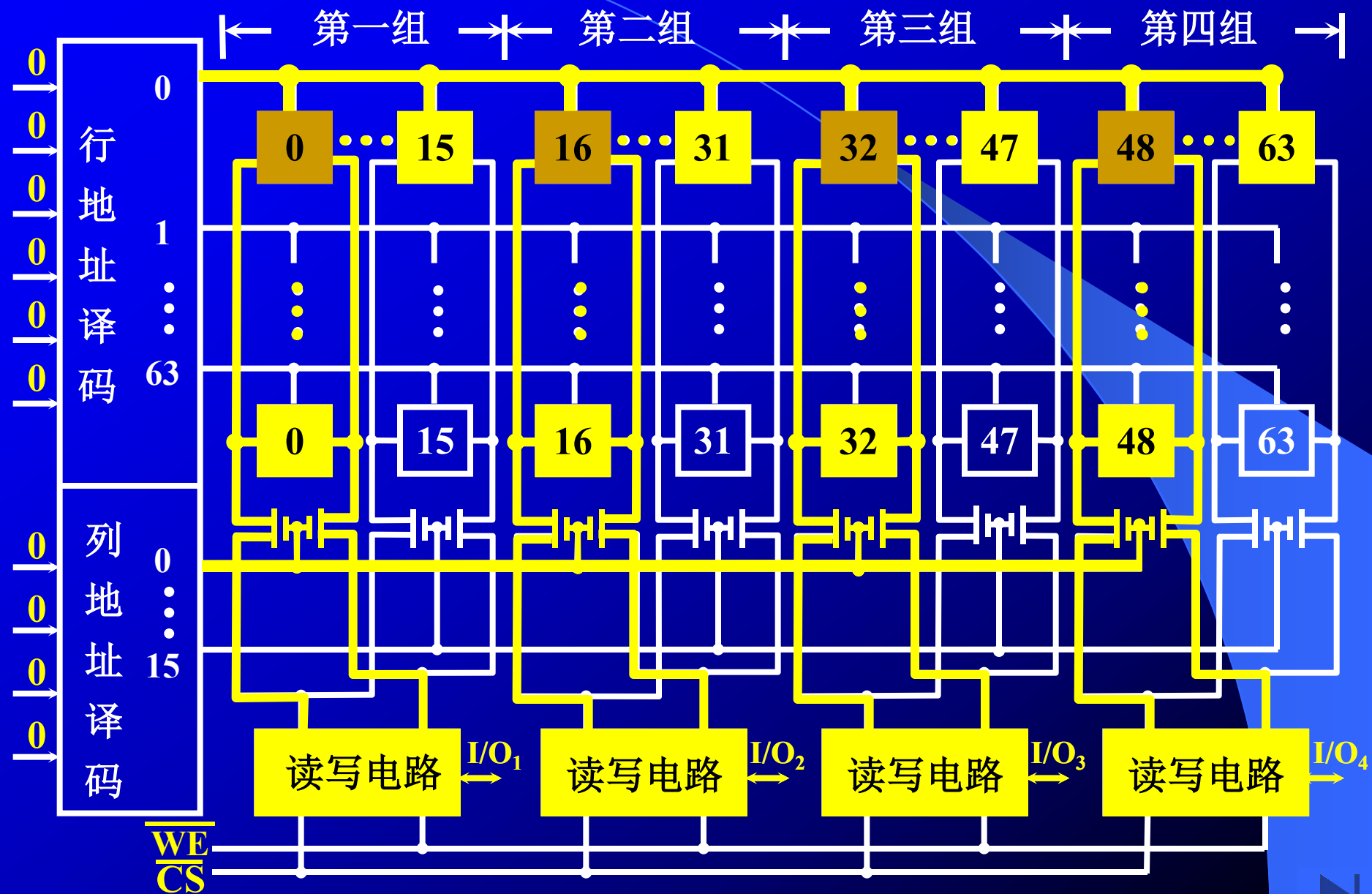
4.2



③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2

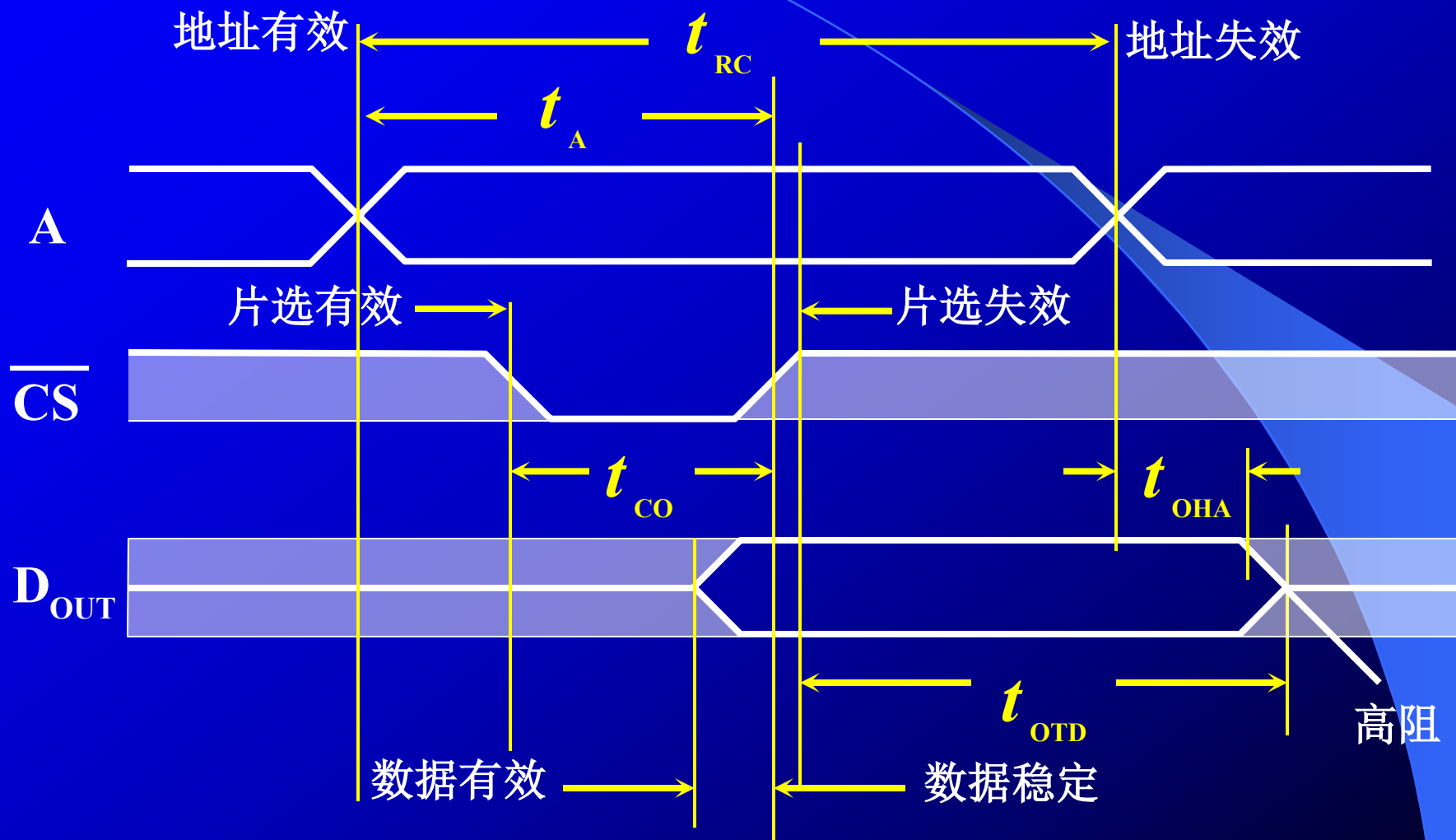


③ Intel 2114 RAM 矩阵 (64 × 64) 写 4.2



(3) 静态 RAM 读 时序

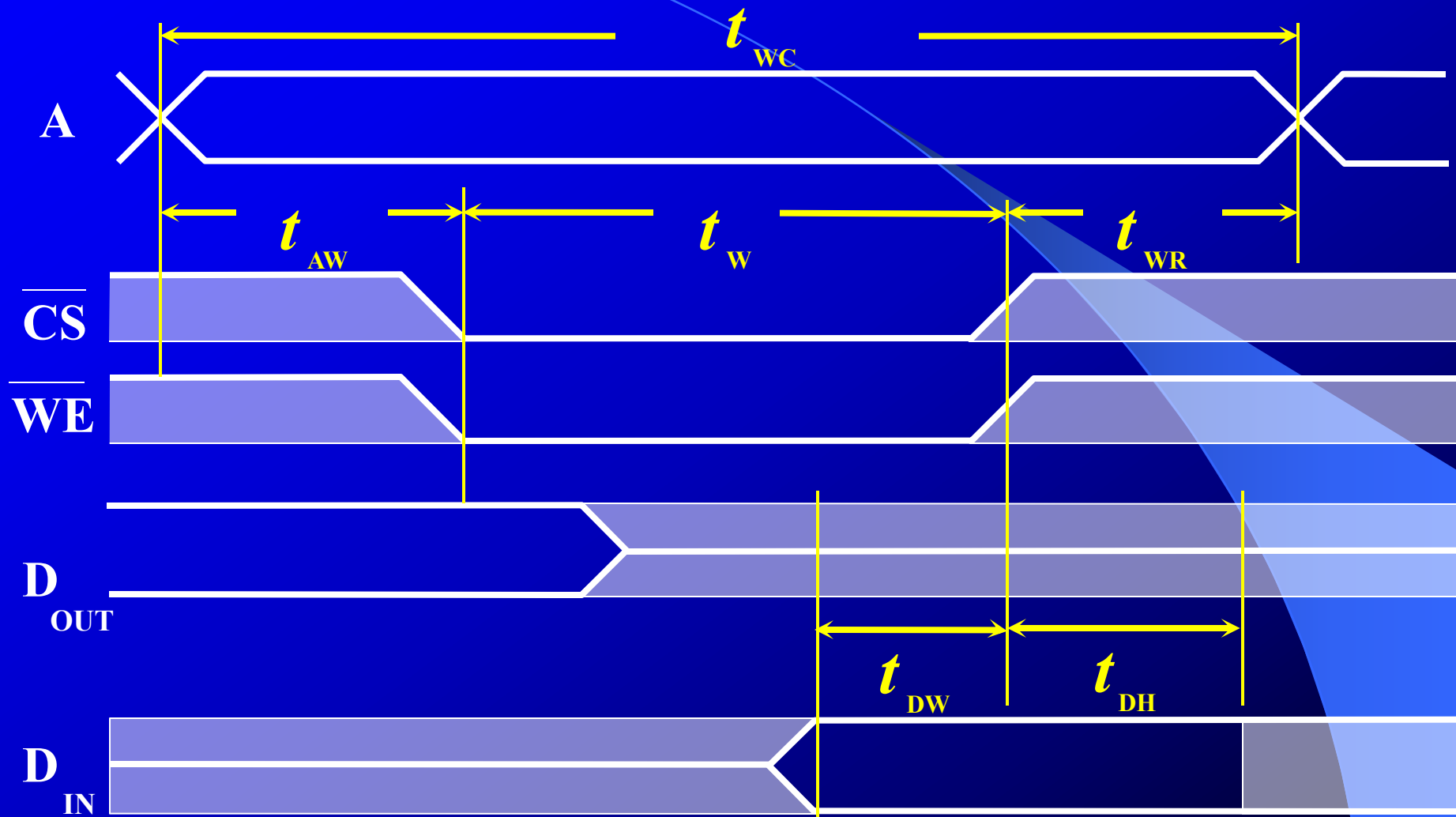
4.2



t_{OHA} 地址失效后的数据维持时间

(4) 静态 RAM (2114) 写时序

4.2



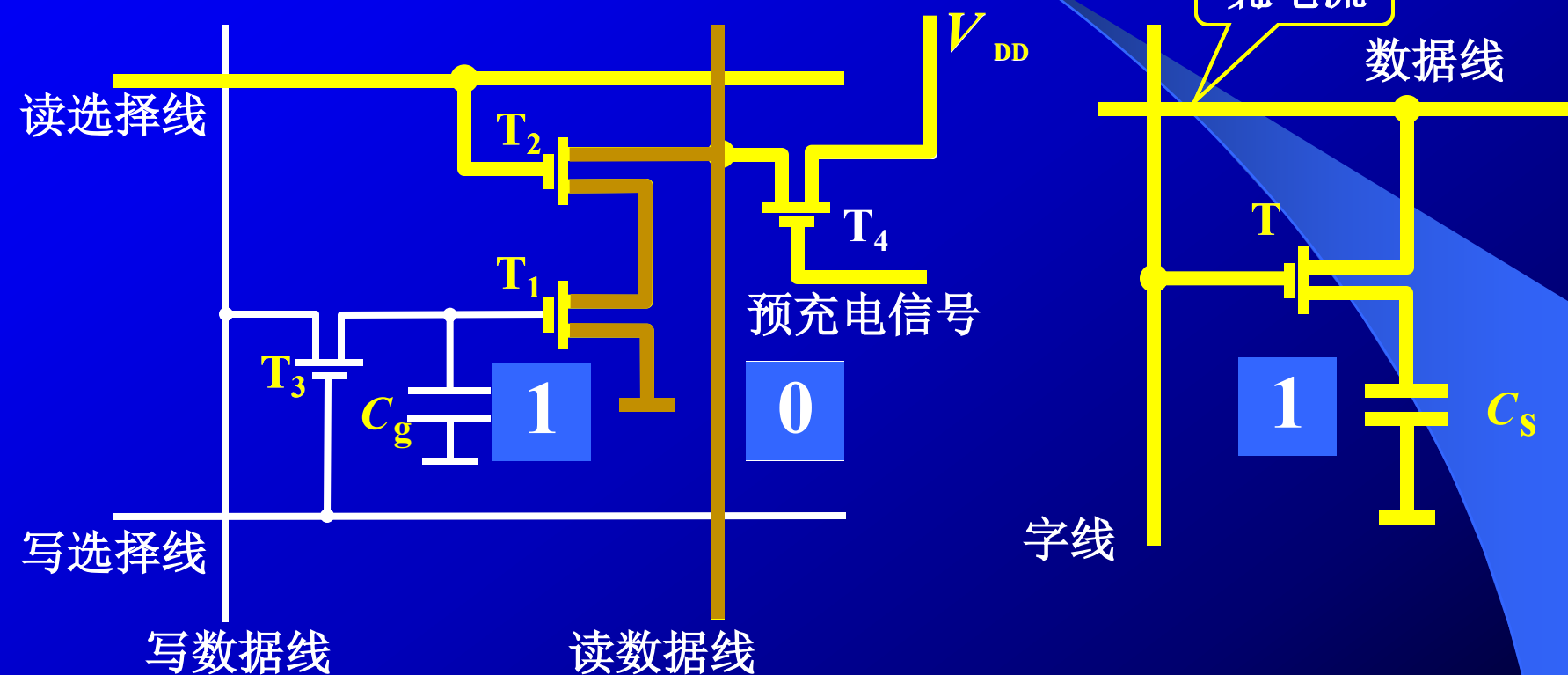
t_{DH} $\overline{WE}$ 失效后的数据维持时间

效



2. 动态 RAM (DRAM)

(1) 动态 RAM 基本单元电路



读出与原存信息相反

写入与输入信息相同

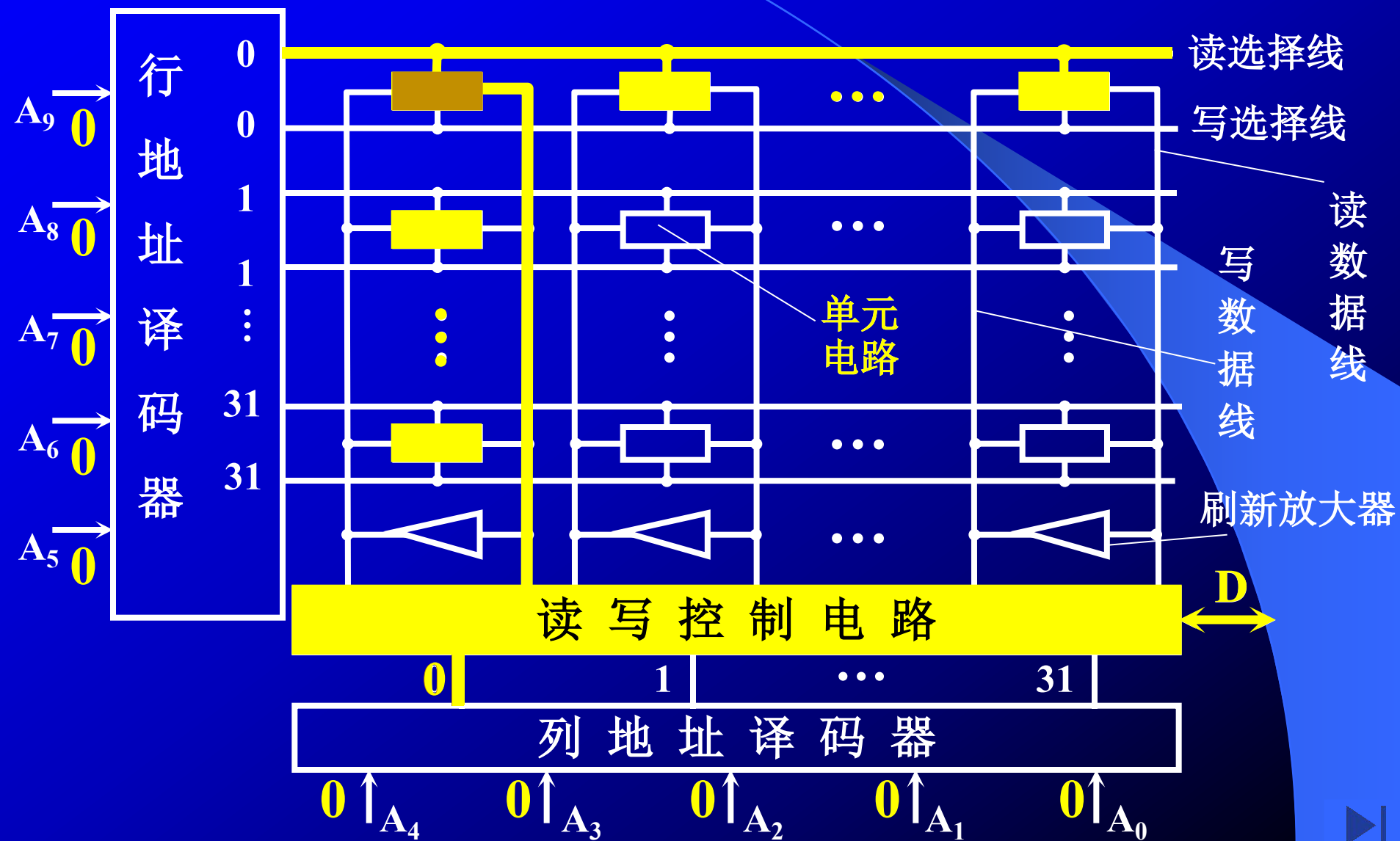
读出时数据线有电流为“1”

写入时 C_s 充电为“1”放电为“0”

(2) 动态 RAM 芯片举例

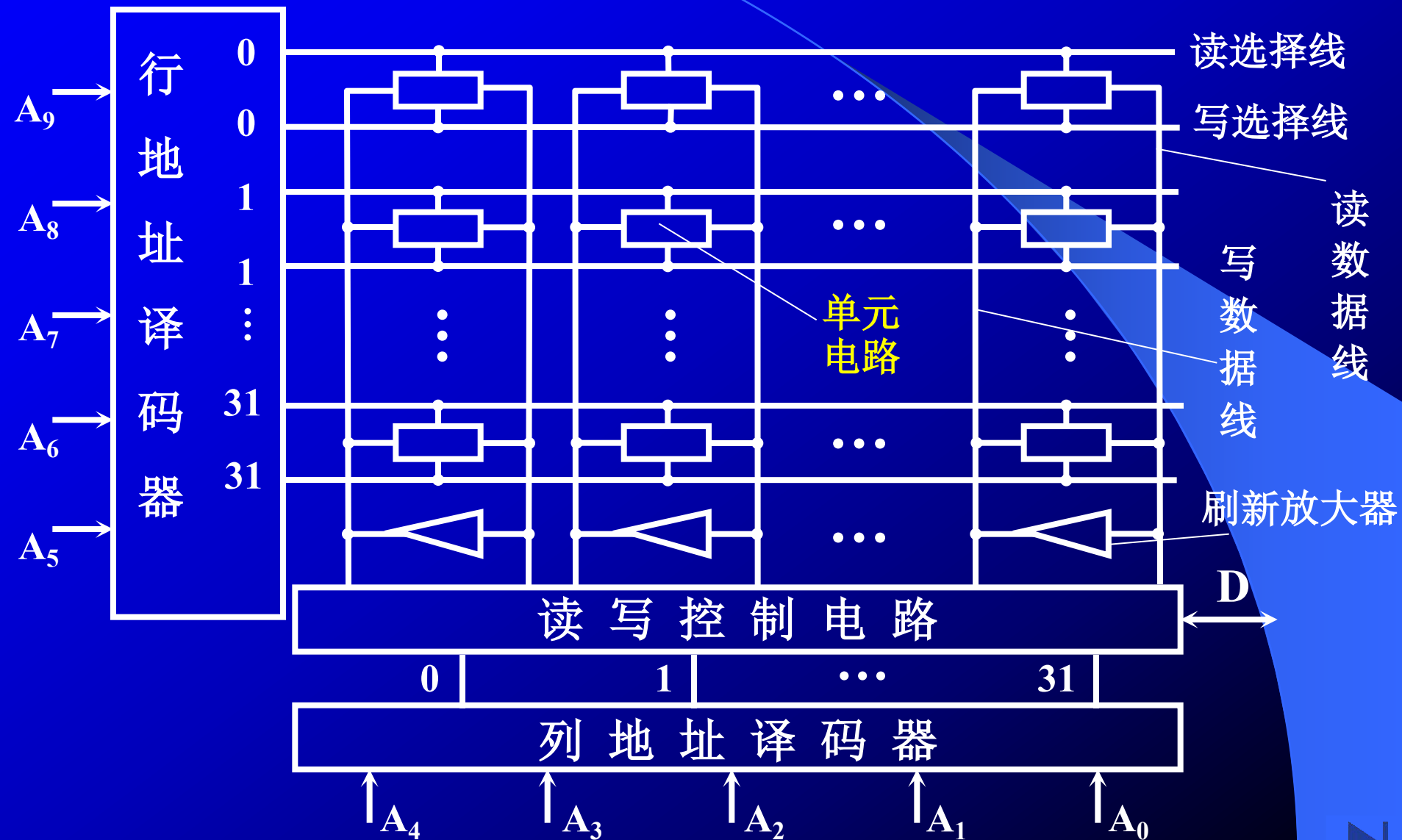
4.2

① 三管动态 RAM 芯片 (Intel 1103) 读



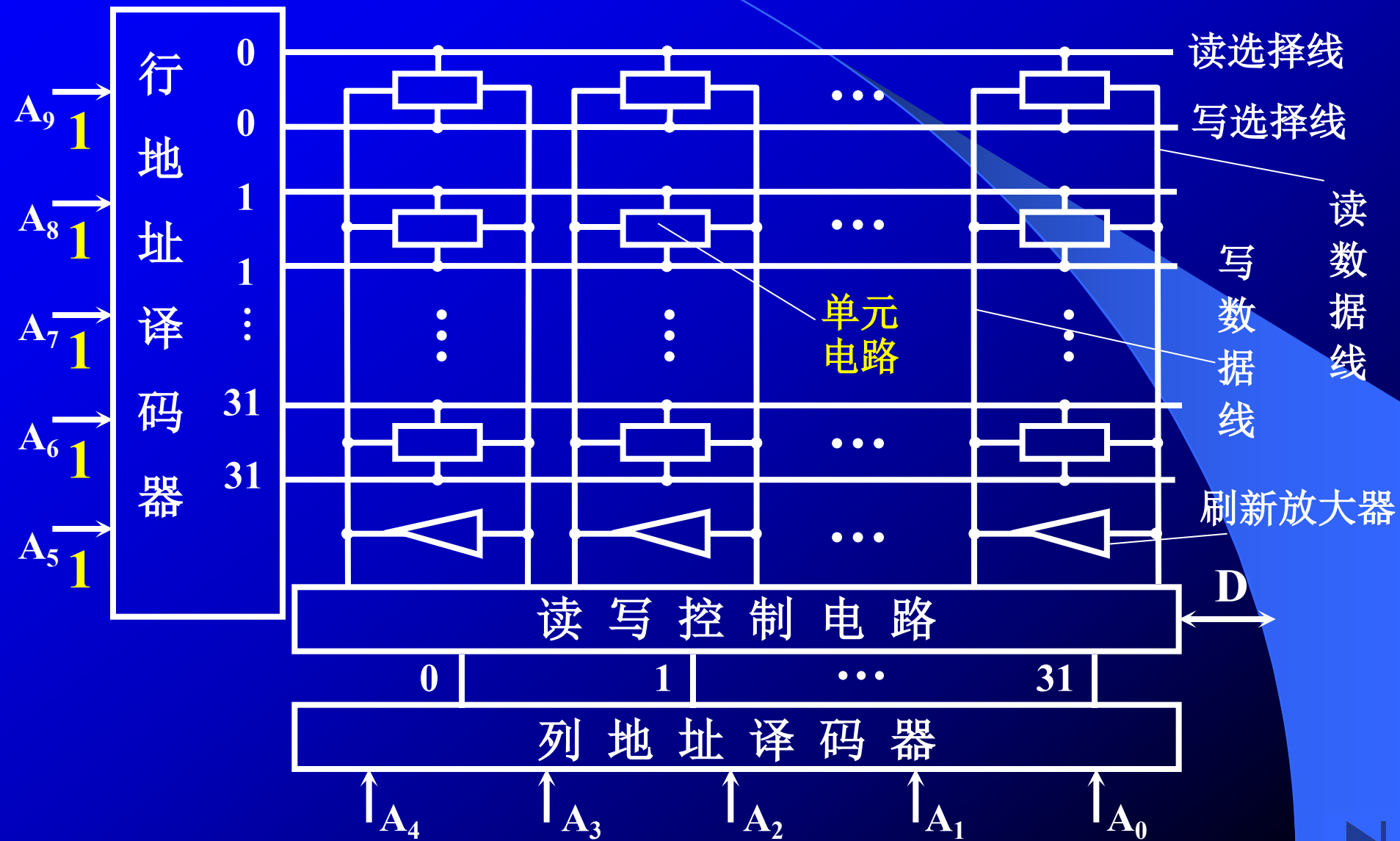
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2



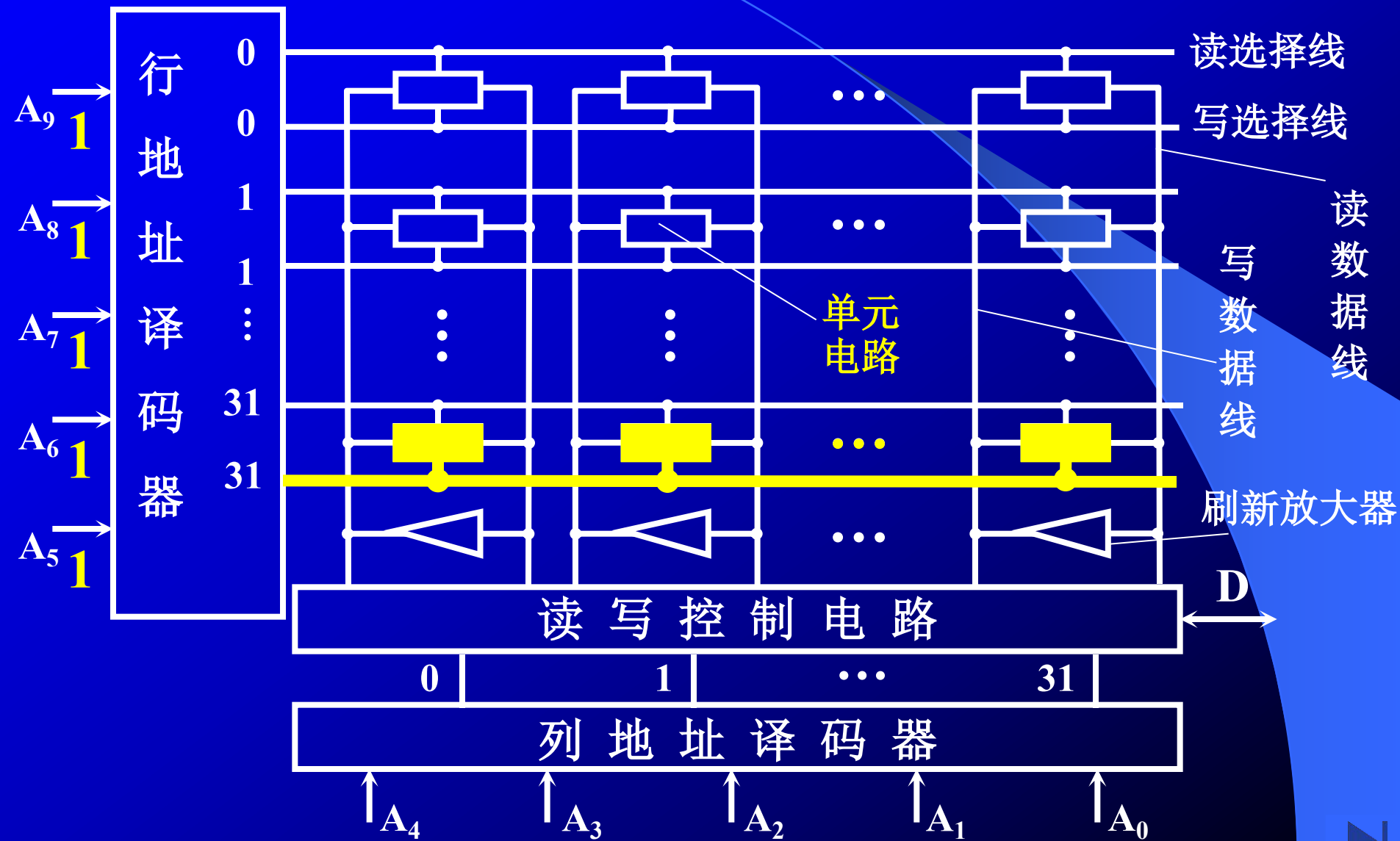
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2



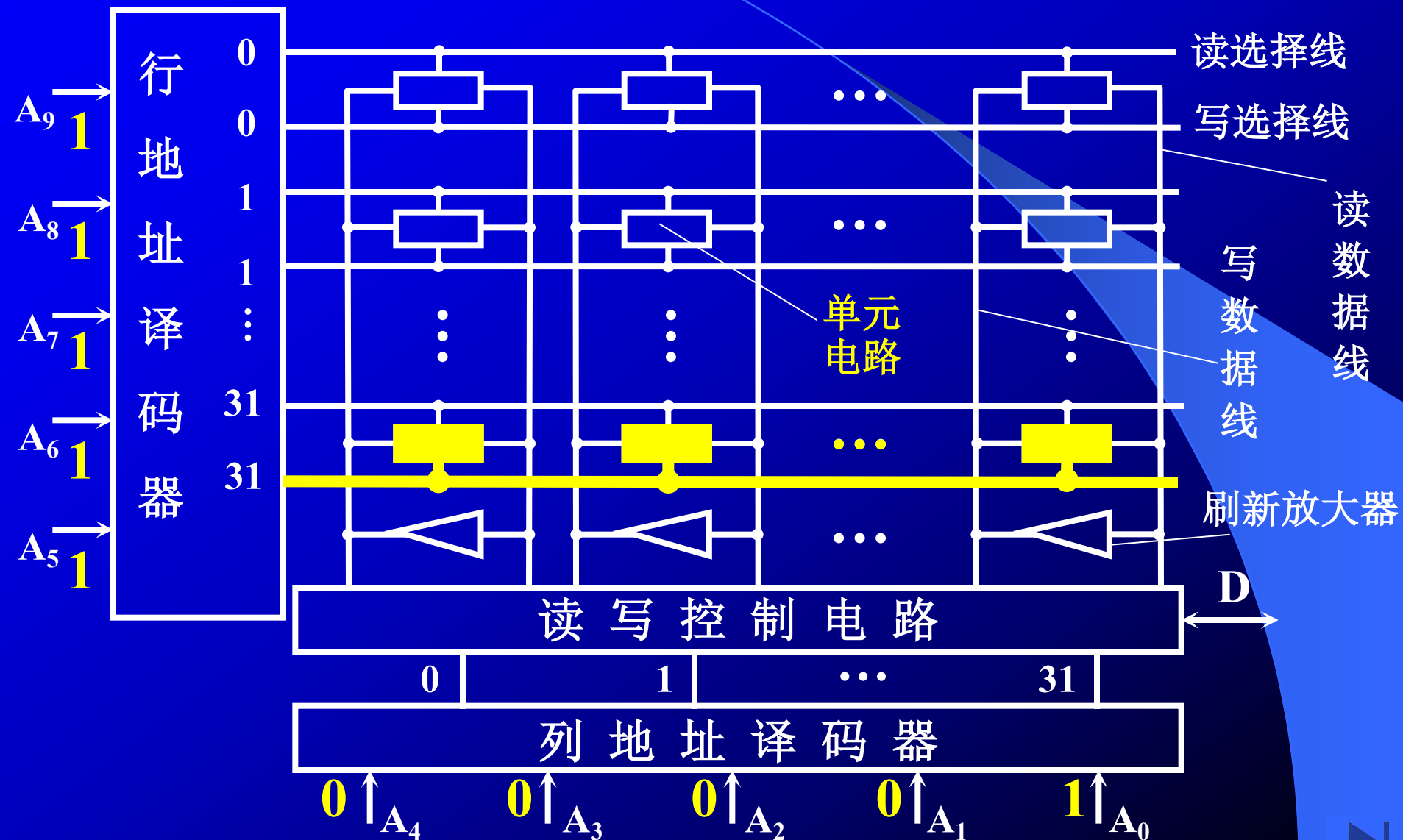
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2



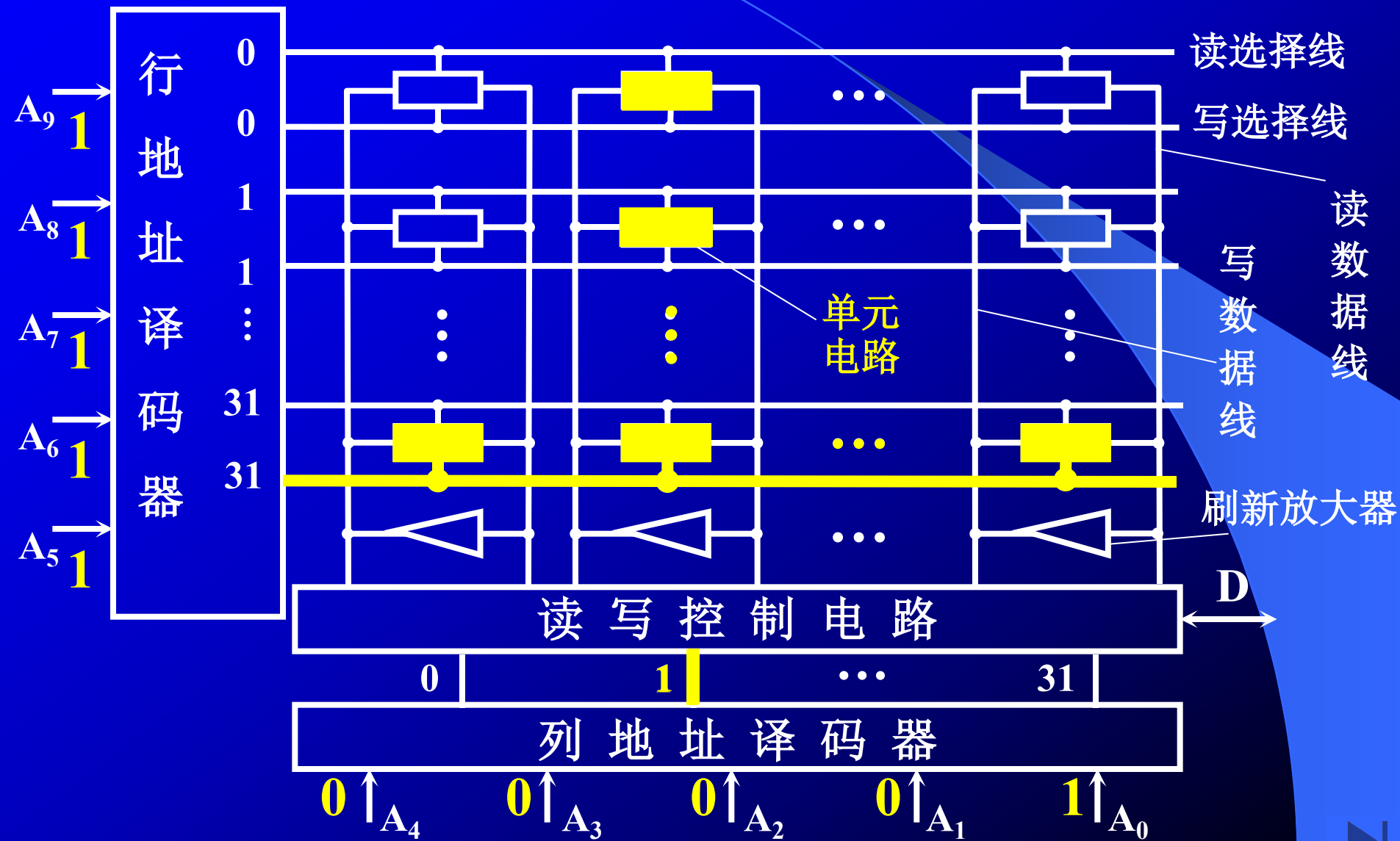
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2



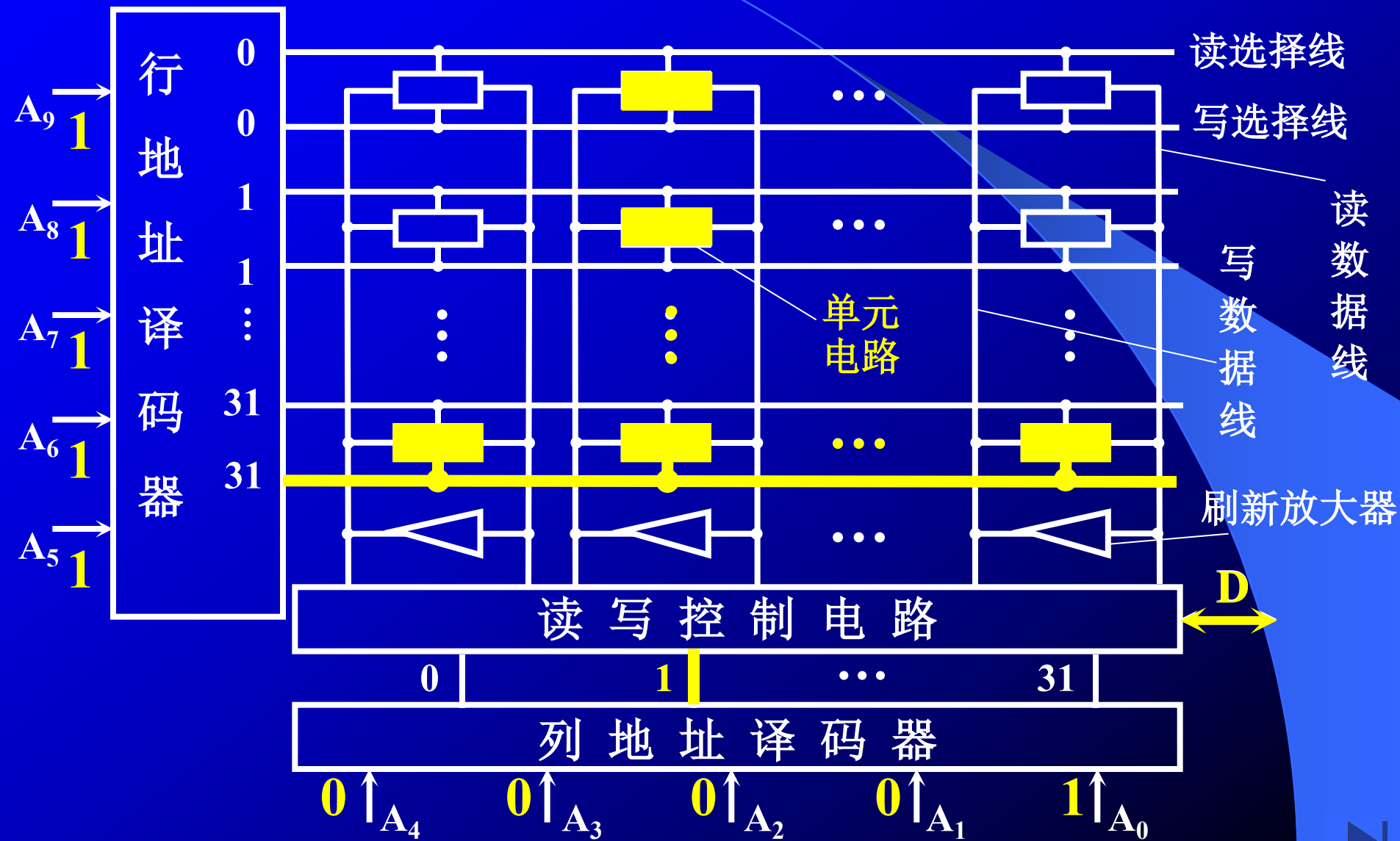
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2



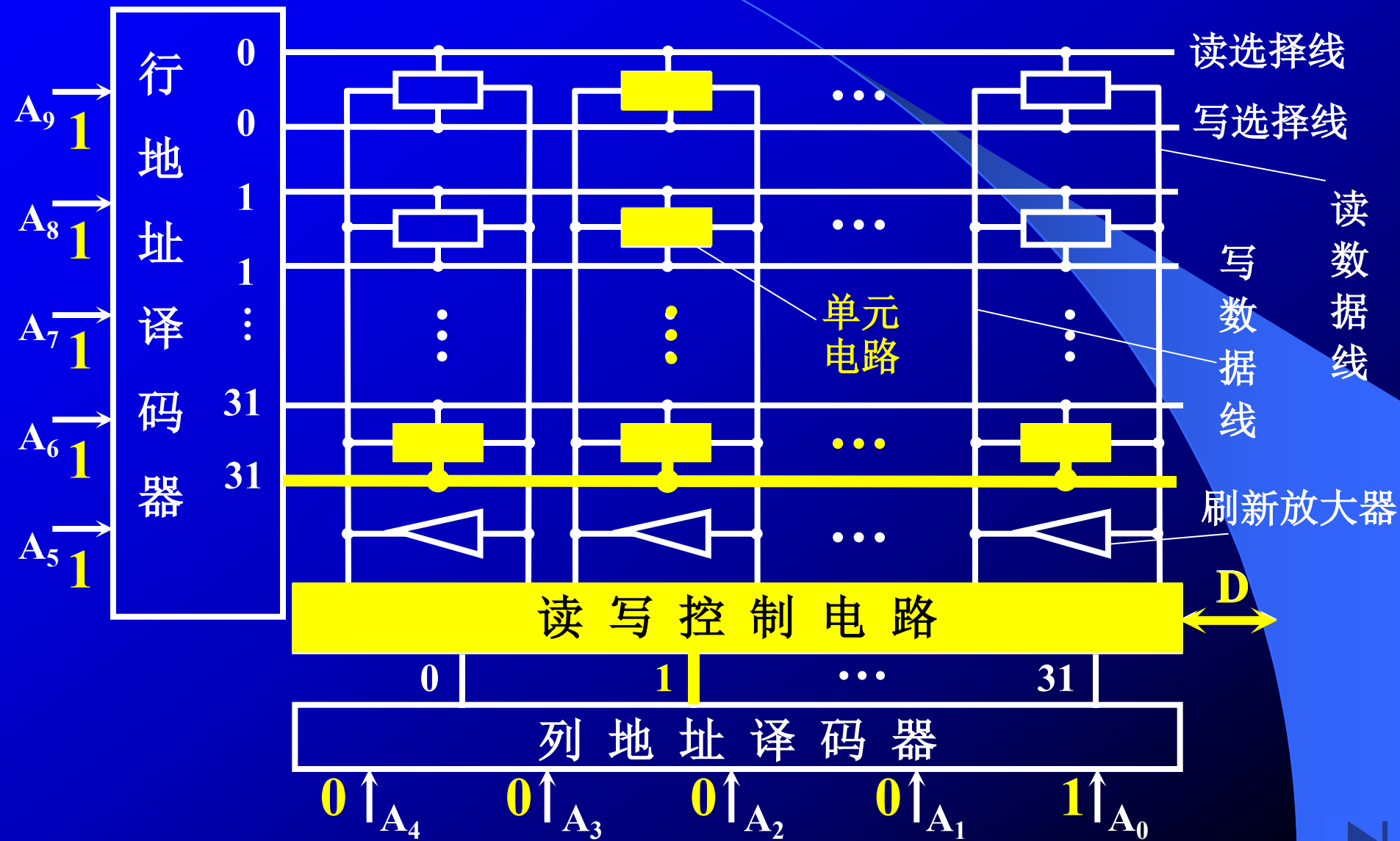
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2



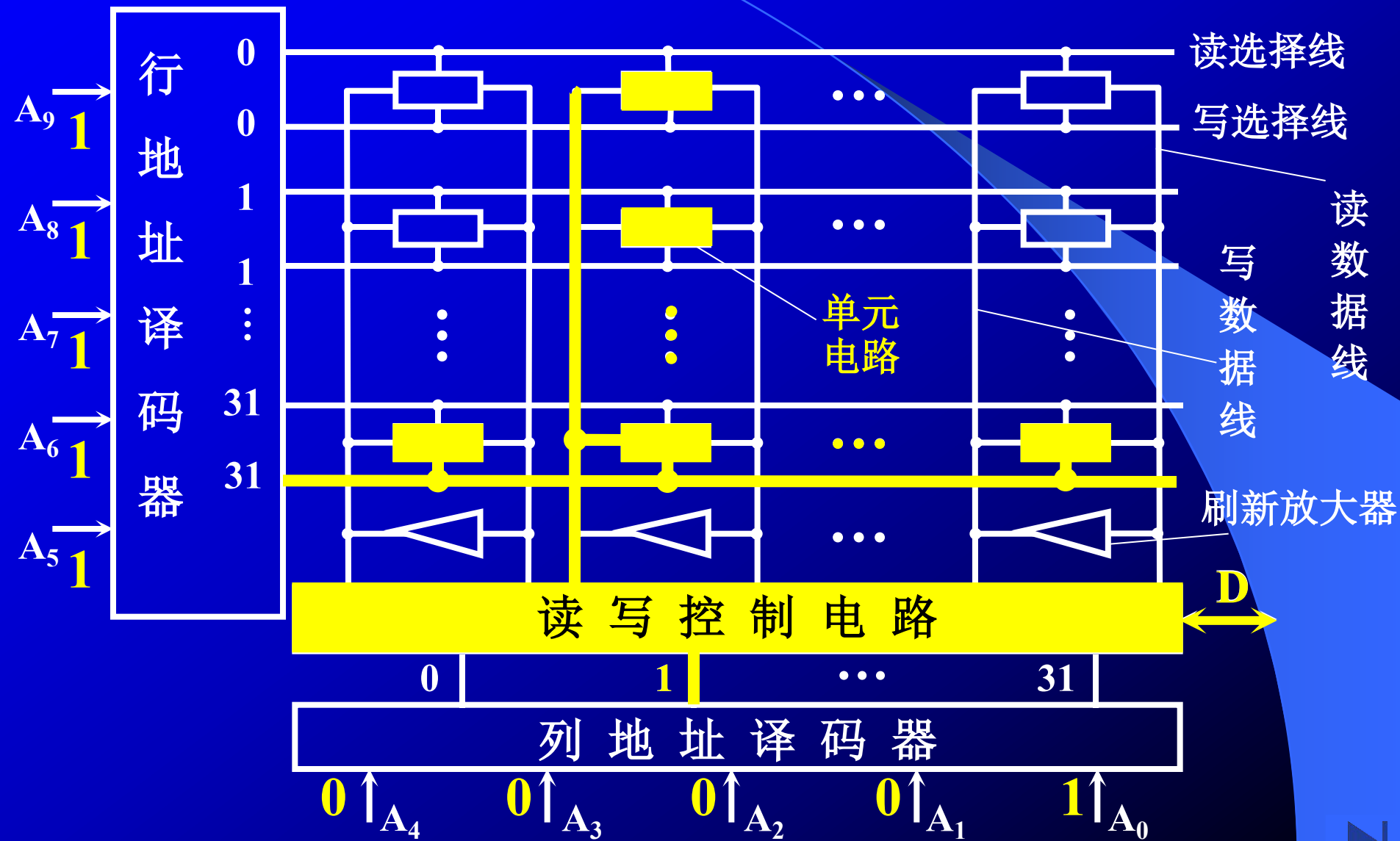
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2



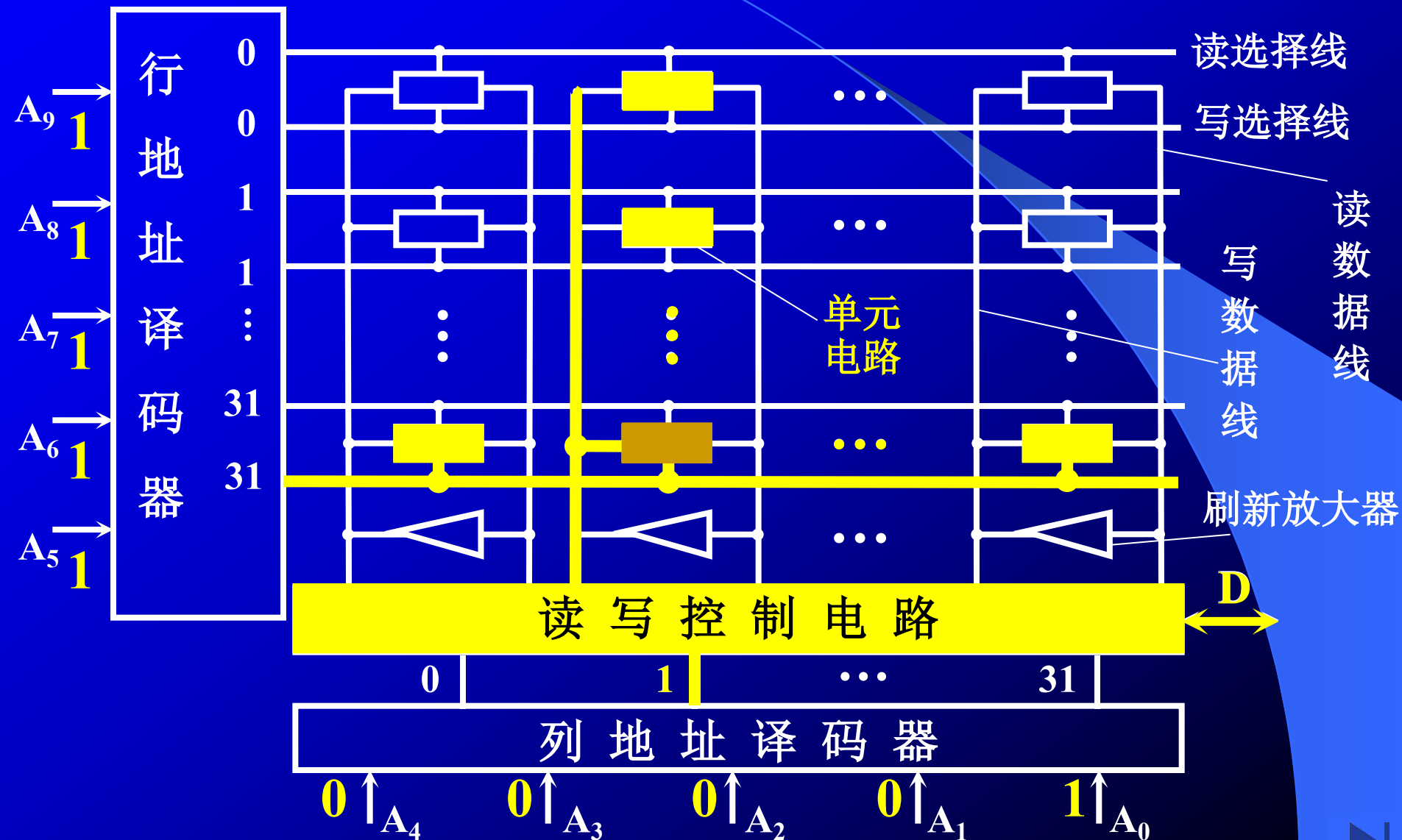
② 三管动态 RAM 芯片 (Intel 1103) 写

4.2

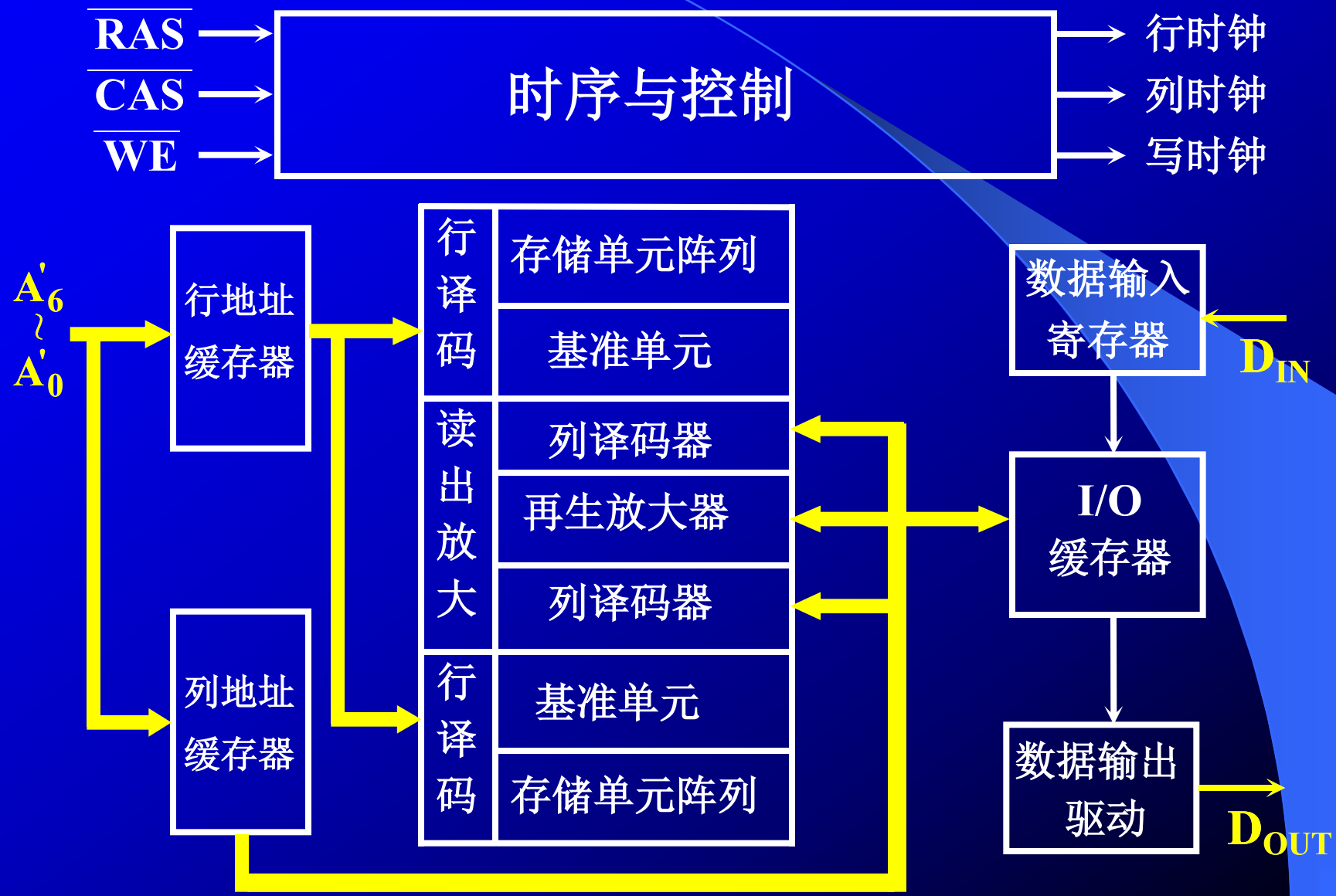


② 三管动态 RAM 芯片 (Intel 1103) 写

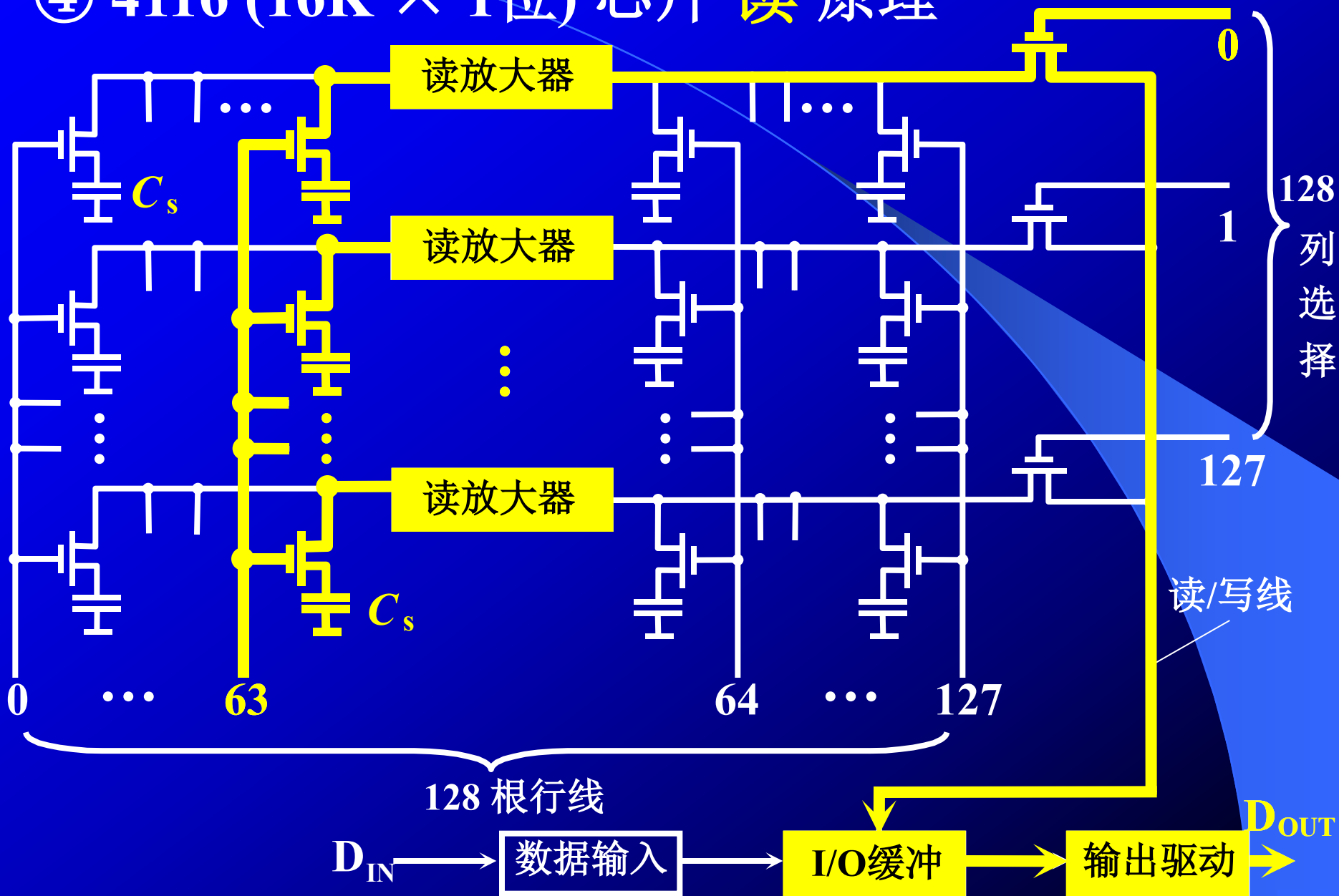
4.2



③ 单管动态 RAM 4116 (16K × 1位) 外特性 4.2

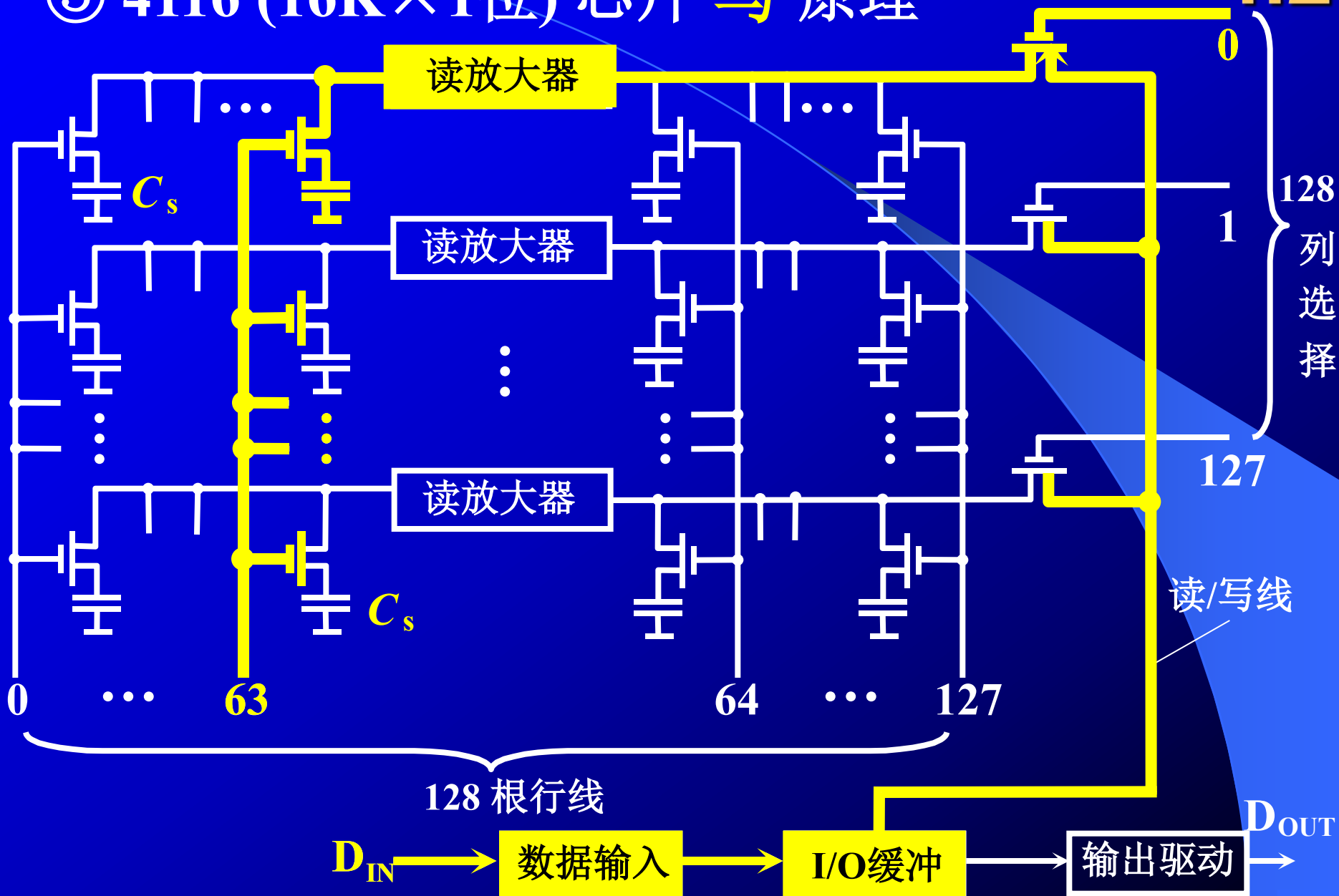


④ 4116 (16K × 1位) 芯片 读 原理



⑤ 4116 (16K×1位) 芯片 写 原理

4.2



(3) 动态 RAM 时序

行、列地址分开传送

读时序

行地址 $\overline{\text{RAS}}$ 有效
写允许 $\overline{\text{WE}}$ 有效(高)
列地址 $\overline{\text{CAS}}$ 有效
数据 D_{OUT} 有效

写时序

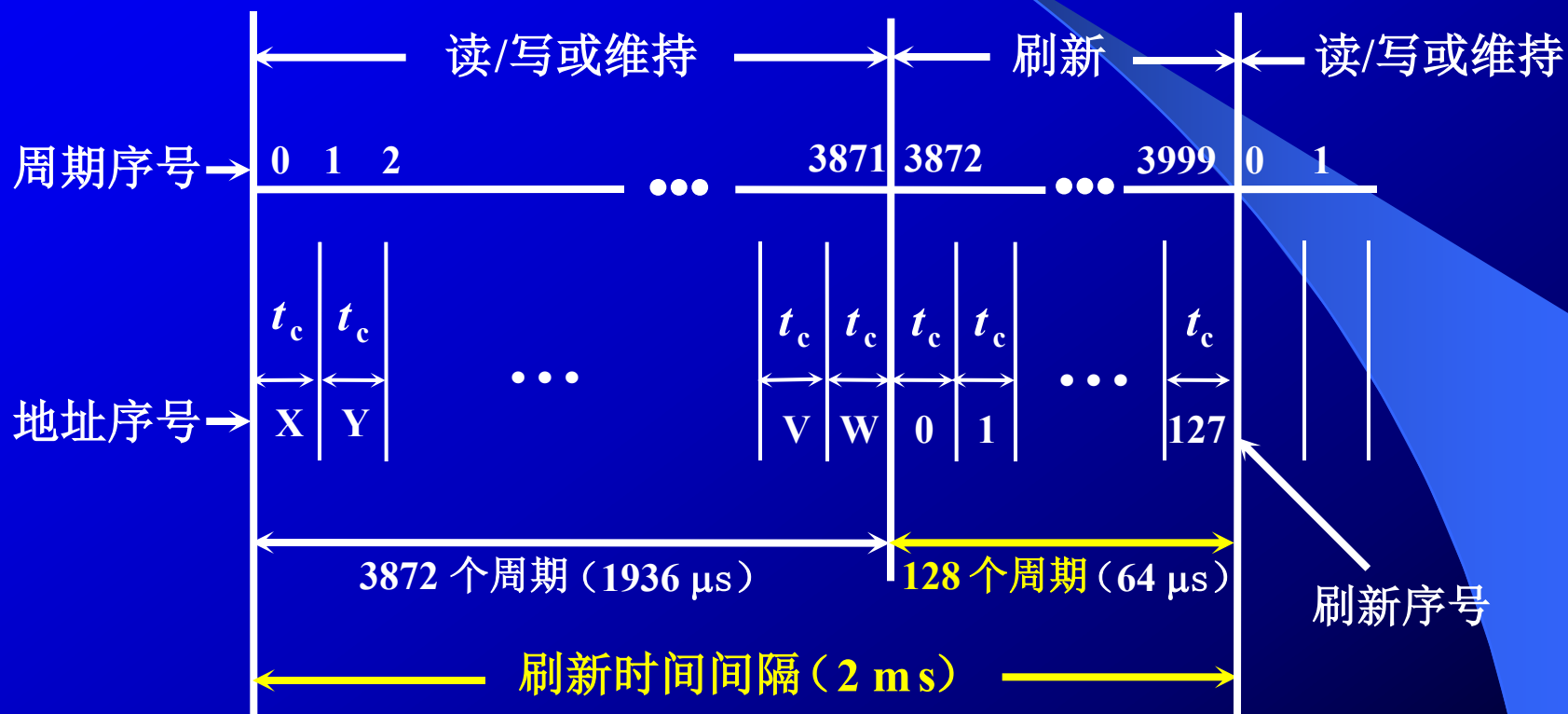
行地址 $\overline{\text{RAS}}$ 有效
写允许 $\overline{\text{WE}}$ 有效(低)
数据 D_{IN} 有效
列地址 $\overline{\text{CAS}}$ 有效

(4) 动态 RAM 刷新

刷新与行地址有关

4.2

① 集中刷新 (存取周期为 $0.5\ \mu\text{s}$) 以 128×128 矩阵为例



“死区” 为 $0.5\ \mu\text{s} \times 128 = 64\ \mu\text{s}$

“死时间率” 为 $128 / 4000 \times 100\% = 3.2\%$

② 分散刷新（存取周期为 $1\mu\text{s}$ ）

4.2

以 128×128 矩阵为例



$$t_C = t_M + t_R$$

无“死区”

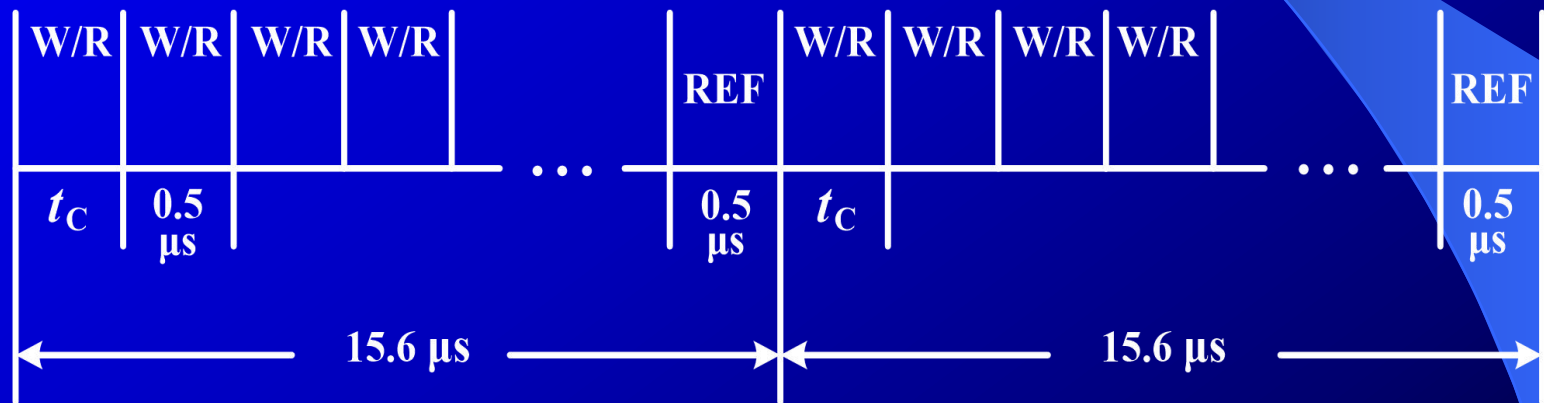
↓ ↓
读写 刷新

(存取周期为 $0.5\mu\text{s} + 0.5\mu\text{s}$)

③ 分散刷新与集中刷新相结合（异步刷新）^{4.2}

对于 128×128 的存储芯片（存取周期为 $0.5 \mu\text{s}$ ）

若每隔 $15.6 \mu\text{s}$ 刷新一次行



每行每隔 2 ms 刷新一次

“死区”为 $0.5 \mu\text{s}$

将刷新安排在指令译码阶段，不会出现“死区”

3. 动态 RAM 和静态 RAM 的比较

	主存 DRAM	SRAM 缓存
存储原理	电容	触发器
集成度	高	低
芯片引脚	少	多
功耗	小	大
价格	低	高
速度	慢	快
刷新	有	无

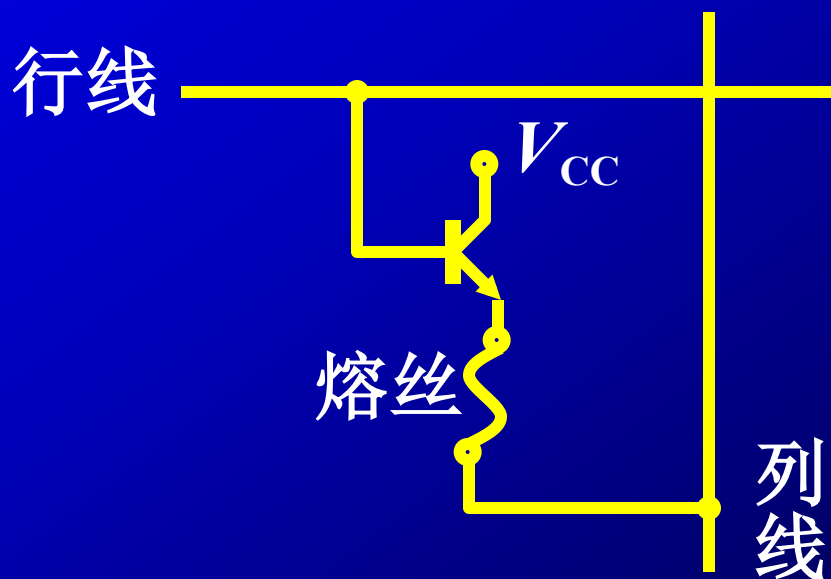
四、只读存储器 (ROM)

1. 掩模 ROM (MROM)

行列选择线交叉处有 MOS 管为 “1”

行列选择线交叉处无 MOS 管为 “0”

2. PROM (一次性编程)

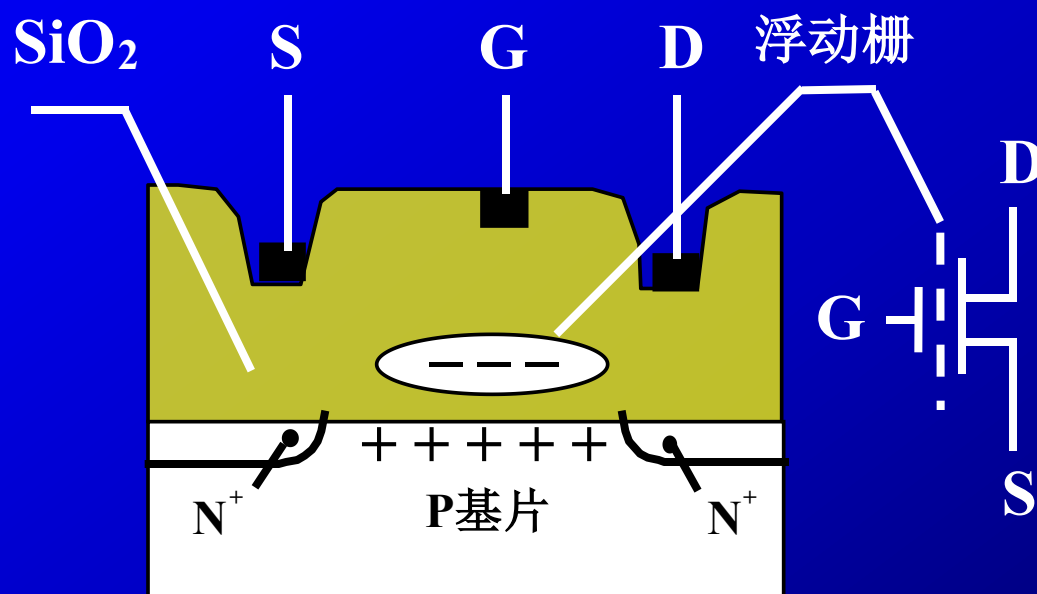


熔丝断 为 “0”

熔丝未断 为 “1”

3. EPROM (多次性编程)

(1) N型沟道浮动栅 MOS 电路



G 栅极

S 源

D 漏

紫外线全部擦洗

D 端加正电压

形成浮动栅

S 与 D 不导通为 “0”

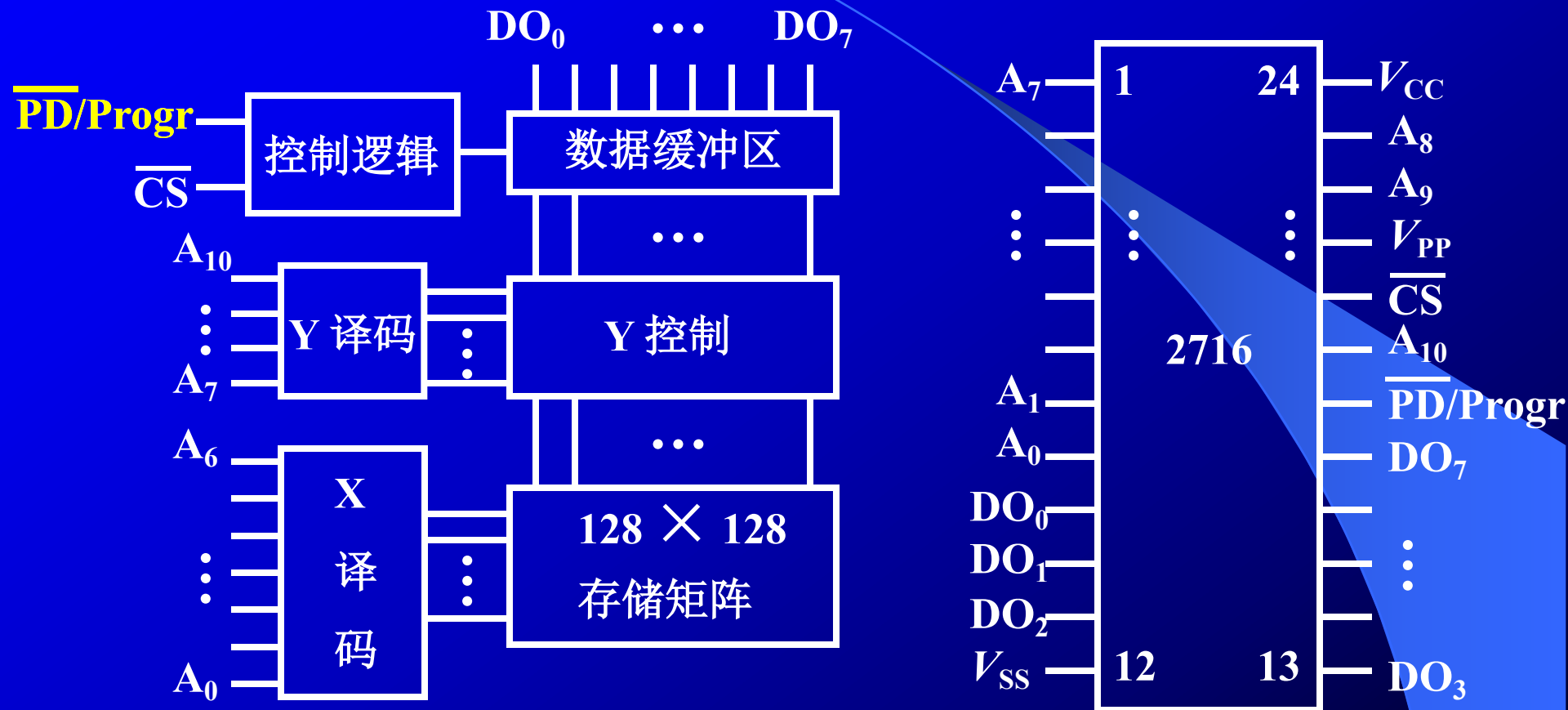
D 端不加正电压

不形成浮动栅

S 与 D 导通为 “1”

(2) 2716 EPROM 的逻辑图和引脚

4.2



$\overline{\text{PD/Progr}}$ 功率下降 / 编程输入端 读出时为低电平

4. EEPROM (多次性编程)

电可擦写

局部擦写

全部擦写

5. Flash Memory (闪速型存储器)

EPROM 价格便宜 集成度高

EEPROM 电可擦写重写

比 EEPROM 快 具备 RAM 功能

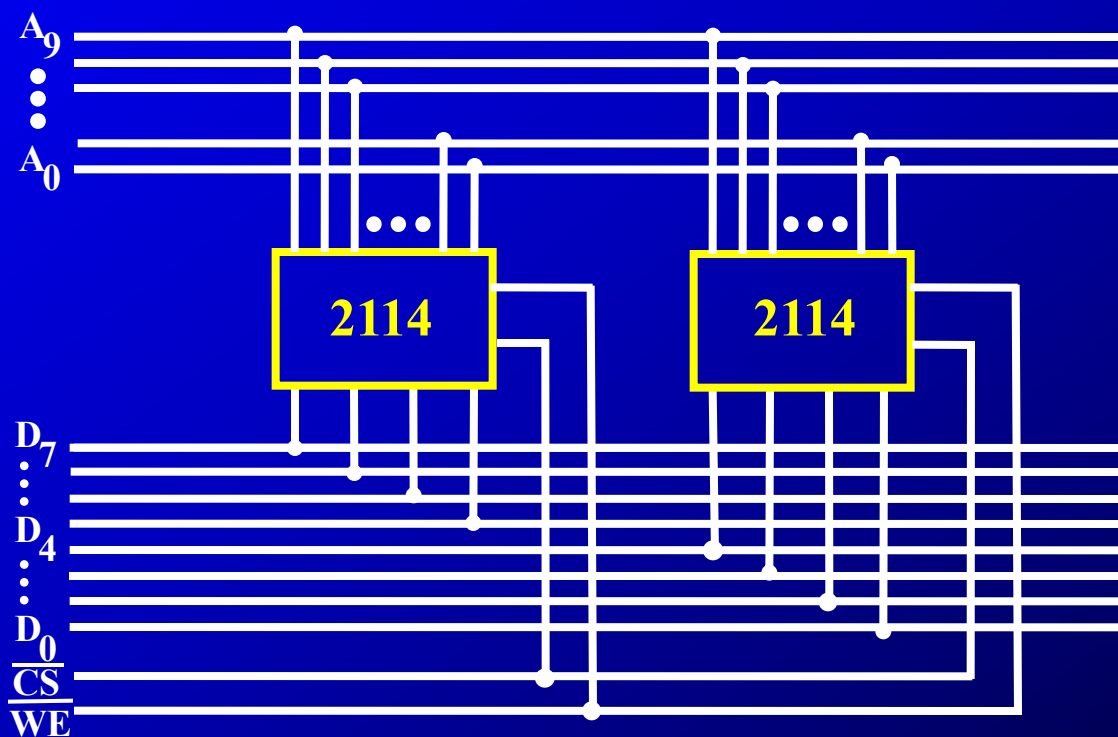


五、存储器与 CPU 的连接

1. 存储器容量的扩展

(1) 位扩展（增加存储字长）

用 **2片** $1K \times 4$ 位 存储芯片组成 $1K \times 8$ 位的存储器



10根地址线

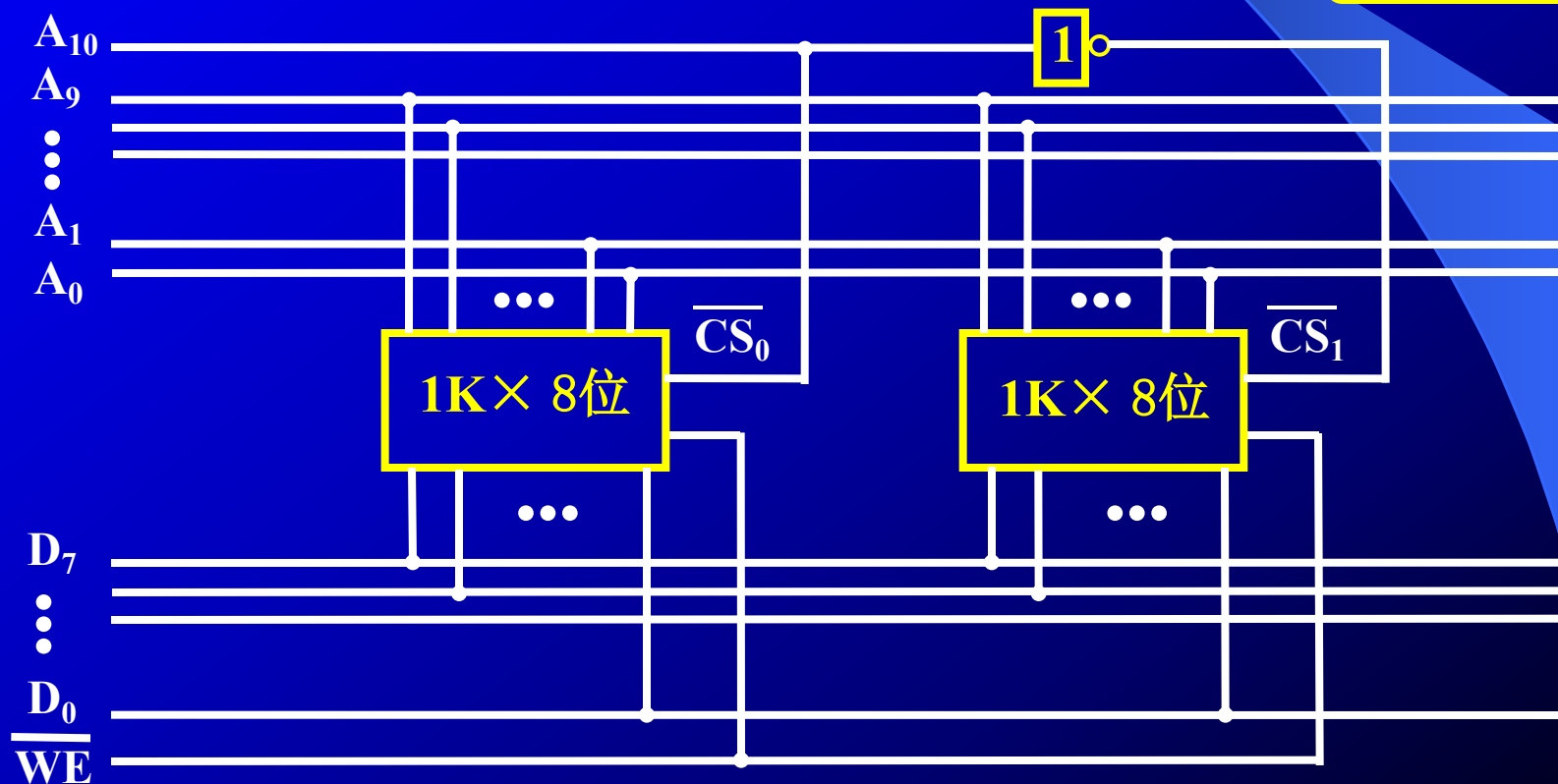
8根数据线

(2) 字扩展（增加存储字的数量）

11根地址线

用 2片 $1\text{K} \times 8\text{位}$ 存储芯片组成 $2\text{K} \times 8\text{位}$ 的存储器

8根数据线



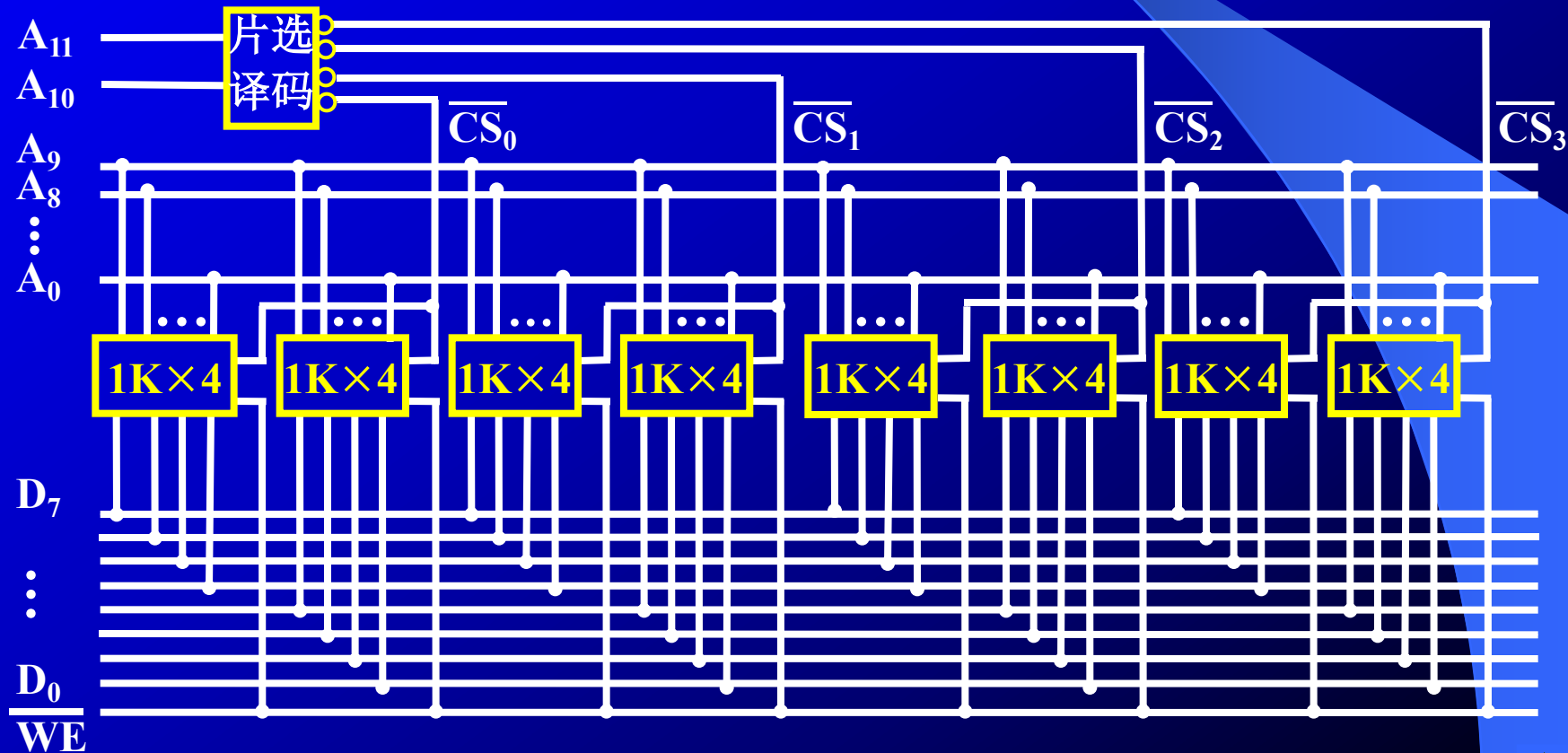
(3) 字、位扩展

4.2

用 8 片 $1\text{K} \times 4$ 位 存储芯片组成 $4\text{K} \times 8$ 位的存储器

12根地址线

8根数据线



2. 存储器与 CPU 的连接

- (1) 地址线的连接
- (2) 数据线的连接
- (3) 读/写命令线的连接
- (4) 片选线的连接
- (5) 合理选择存储芯片
- (6) 其他 时序、负载



例4.1 解:

4.2

(1) 写出对应的二进制地址码

$A_{15}A_{14}A_{13}$ $A_{11}A_{10}$... A_7 ... A_4A_3 ... A_0

0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0

⋮

0 1 1 0 0 1 1 1 1 1 1 1 1 1 1 1

0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0

⋮

0 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1

1片 $2K \times 8$ 位

ROM

$2K \times 8$ 位

$1K \times 8$ 位

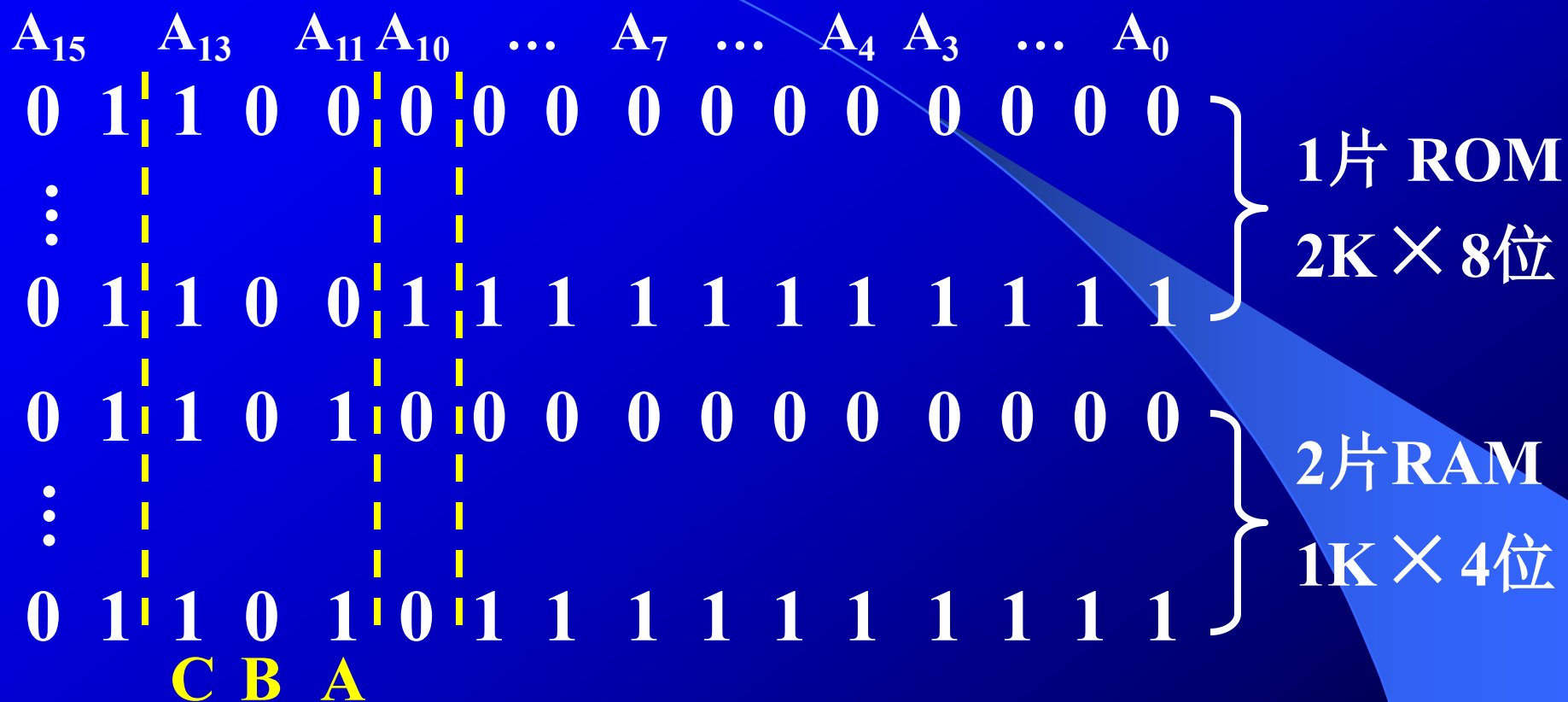
RAM

2片 $1K \times 4$ 位

(2) 确定芯片的数量及类型



(3) 分配地址线



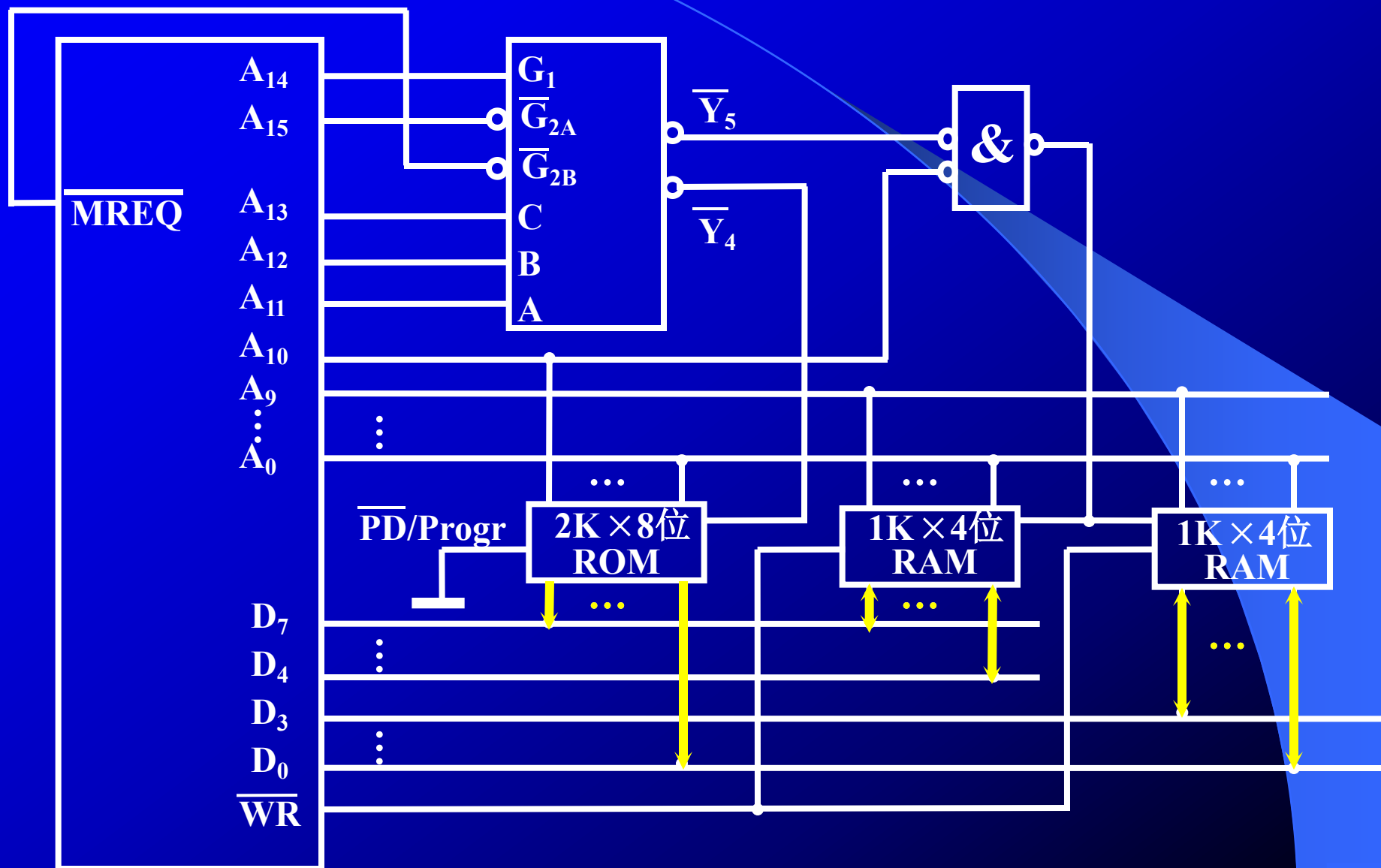
A₁₀~A₀ 接 2K × 8位 ROM 的地址线

A₉~A₀ 接 1K × 4位 RAM 的地址线

(4) 确定片选信号

例 4.1 CPU 与存储器的连接图

4.2



例4.2 假设同前，要求最小 4K为系统程序区，相邻 8K为用户程序区。

(1) 写出对应的二进制地址码

(2) 确定芯片的数量及类型

1片 4K×8位 ROM 2片 4K×8位 RAM

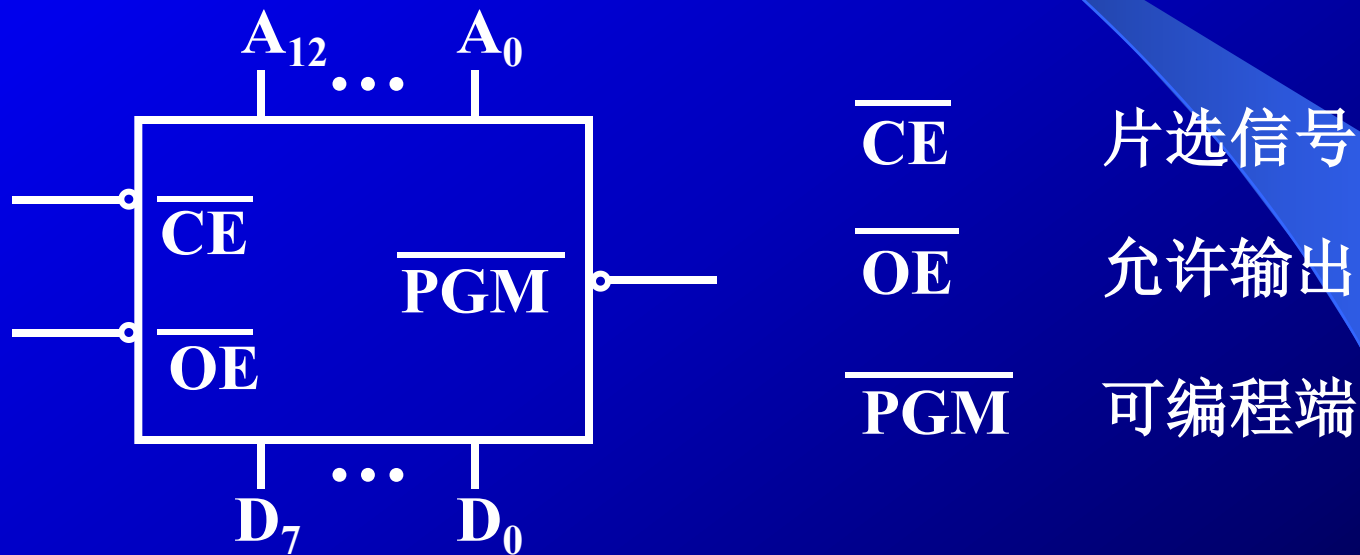
(3) 分配地址线

$A_{11} \sim A_0$ 接 ROM 和 RAM 的地址线

(4) 确定片选信号

例 4.3 设 CPU 有 20 根地址线, 8 根数据线。 4.2

并用 $\overline{\text{IO}/\text{M}}$ 作访存控制信号。 $\overline{\text{RD}}$ 为读命令,
 $\overline{\text{WR}}$ 为写命令。现有 2764 EPROM (8K × 8位),
外特性如下:



用 138 译码器及其他门电路 (门电路自定) 画出 CPU
和 2764 的连接图。要求地址为 F0000H~FFFFFH, 并
写出每片 2764 的地址范围。

六、存储器的校验

4.2

1. 编码的最小距离

任意两组合法代码之间 二进制位数 的 最少差异
编码的纠错、检错能力与编码的最小距离有关

$$L - 1 = D + C \quad (D \geq C)$$

L —— 编码的最小距离 $L = 3$

D —— 检测错误的位数 具有 一位 纠错能力

C —— 纠正错误的位数

汉明码是具有一位纠错能力的编码



2. 汉明码的组成

组成汉明码的三要素

汉明码的组成需增添 ? 位检测位

$$2^k \geq n + k + 1$$

检测位的位置 ?

$$2^i \ (i = 0, 1, 2, 3, \dots)$$

检测位的取值 ?

检测位的取值与该位所在的检测“小组”中
承担的奇偶校验任务有关



各检测位 C_i 所承担的检测小组为

C_1 检测的 g_1 小组包含第 1, 3, 5, 7, 9, 11, ...

C_2 检测的 g_2 小组包含第 2, 3, 6, 7, 10, 11, ...

C_4 检测的 g_3 小组包含第 4, 5, 6, 7, 12, 13, ...

C_8 检测的 g_4 小组包含第 8, 9, 10, 11, 12, 13, 14, 15, 24, ...

g_i 小组独占第 2^{i-1} 位

g_i 和 g_j 小组共同占第 $2^{i-1} + 2^{j-1}$ 位

g_i 、 g_j 和 g_l 小组共同占第 $2^{i-1} + 2^{j-1} + 2^{l-1}$ 位



例4.4 求 0101 按“偶校验”配置的汉明码

解：∵ $n = 4$

根据 $2^k \geq n + k + 1$

得 $k = 3$

汉明码排序如下：

二进制序号	1	2	3	4	5	6	7
名称	C_1	C_2	0	C_4	1	0	1
	0	1		0			

∴ 0101 的汉明码为 **0100101**

练习1 按配偶原则配置 0011 的汉明码 4.2

解: $\because n = 4$ 根据 $2^k \geq n + k + 1$

取 $k = 3$

二进制序号	1	2	3	4	5	6	7
名称	C_1	C_2	0	C_4	0	1	1
	1	0		0			

$$C_1 = 3 \oplus 5 \oplus 7 = 1$$

$$C_2 = 3 \oplus 6 \oplus 7 = 0$$

$$C_4 = 5 \oplus 6 \oplus 7 = 0$$

$\therefore$ 0011 的汉明码为 **1000011**



3. 汉明码的纠错过程

4.2

形成新的检测位 P_i ，其位数与增添的检测位有关，如增添 3 位 ($k=3$)，新的检测位为 $P_4 P_2 P_1$ 。

以 $k=3$ 为例， P_i 的取值为

$$P_1 = \overset{C_1}{1} \oplus 3 \oplus 5 \oplus 7$$

$$P_2 = \overset{C_2}{2} \oplus 3 \oplus 6 \oplus 7$$

$$P_4 = \overset{C_4}{4} \oplus 5 \oplus 6 \oplus 7$$

对于按“偶校验”配置的汉明码
不出错时 $P_1=0, P_2=0, P_4=0$



例4.5 已知接收到的汉明码为 0100111

(按配偶原则配置) 试问要求传送的信息是什么?

解: 纠错过程如下

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7 = 0 \quad \text{无错}$$

$$P_2 = 2 \oplus \underset{\checkmark}{3} \oplus \boxed{6} \oplus \underset{\checkmark}{7} = 1 \quad \text{有错}$$

$$P_4 = 4 \oplus \underset{\checkmark}{5} \oplus \boxed{6} \oplus \underset{\checkmark}{7} = 1 \quad \text{有错}$$

$$\therefore P_4 P_2 P_1 = 110$$

第 6 位出错, 可纠正为 0100101,
故要求传送的信息为 0101。

练习2 写出按偶校验配置的汉明码

0101101 的纠错过程

$$P_4 = 4 \oplus 5 \oplus 6 \oplus 7 = 1$$

$$P_2 = 2 \oplus 3 \oplus 6 \oplus 7 = 0$$

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7 = 0$$

$\therefore P_4 P_2 P_1 = 100$ 第 4 位错，可不纠

练习3 按配奇原则配置 0011 的汉明码

配奇的汉明码为 0101011



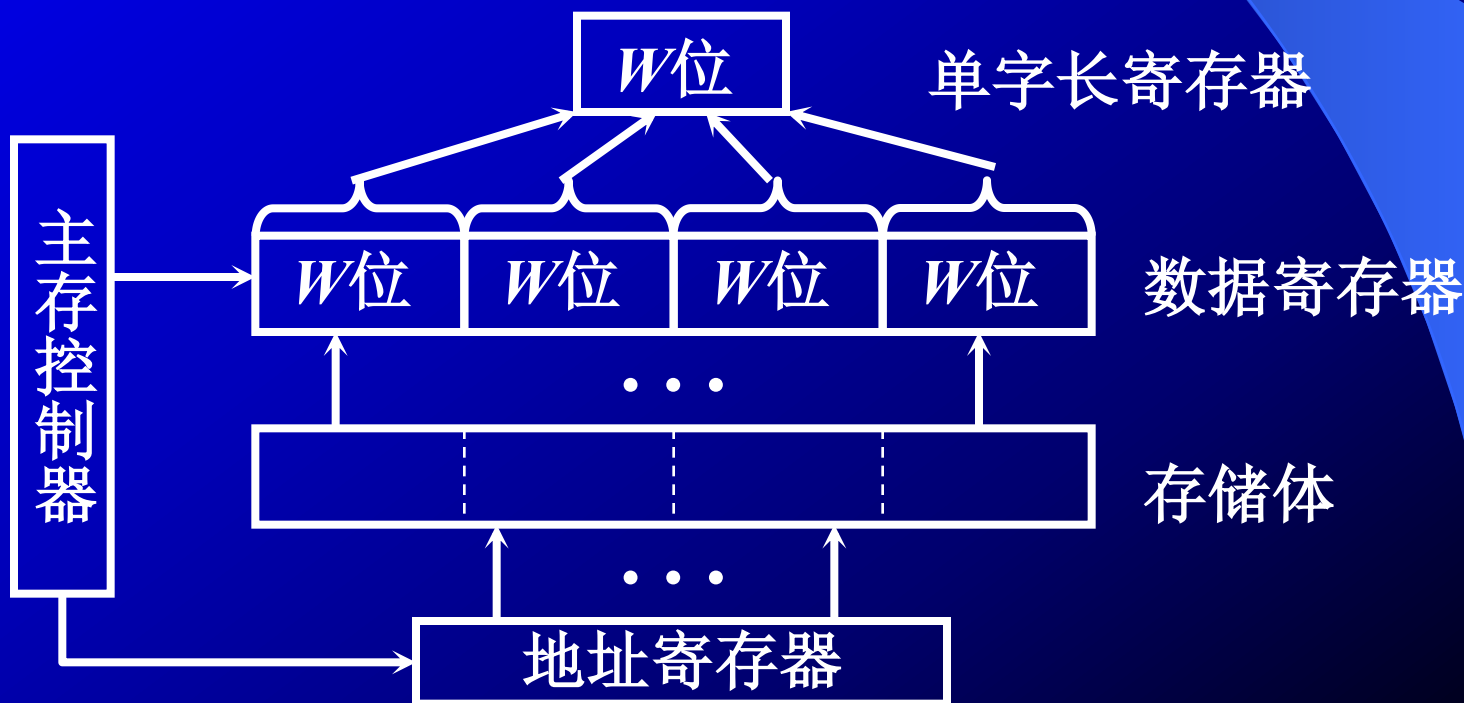
七、提高访存速度的措施

4.2

- 采用高速器件
- 采用层次结构 Cache-主存
- 调整主存结构

1. 单体多字系统

增加存储器的带宽

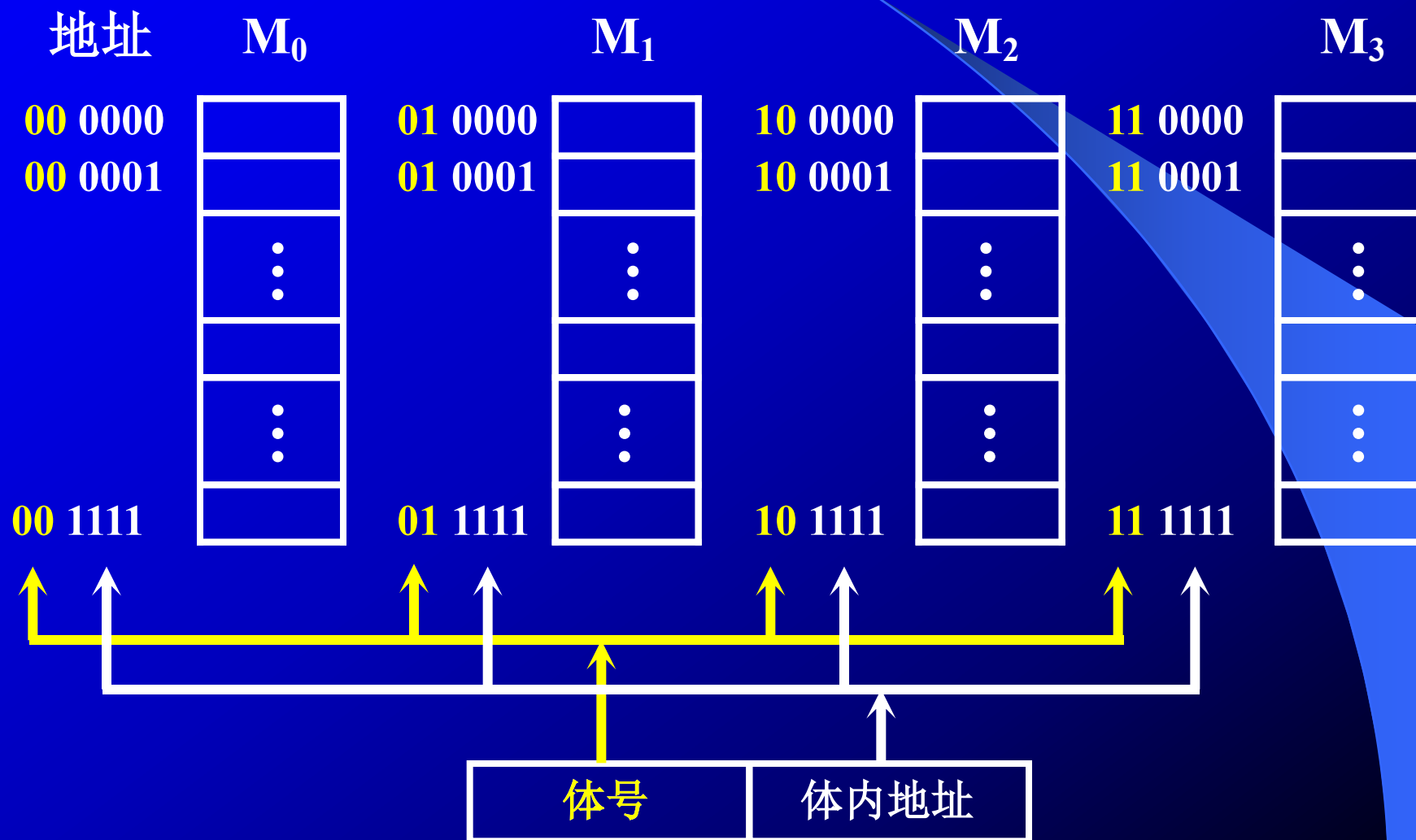


2. 多体并行系统

4.2

(1) 高位交叉

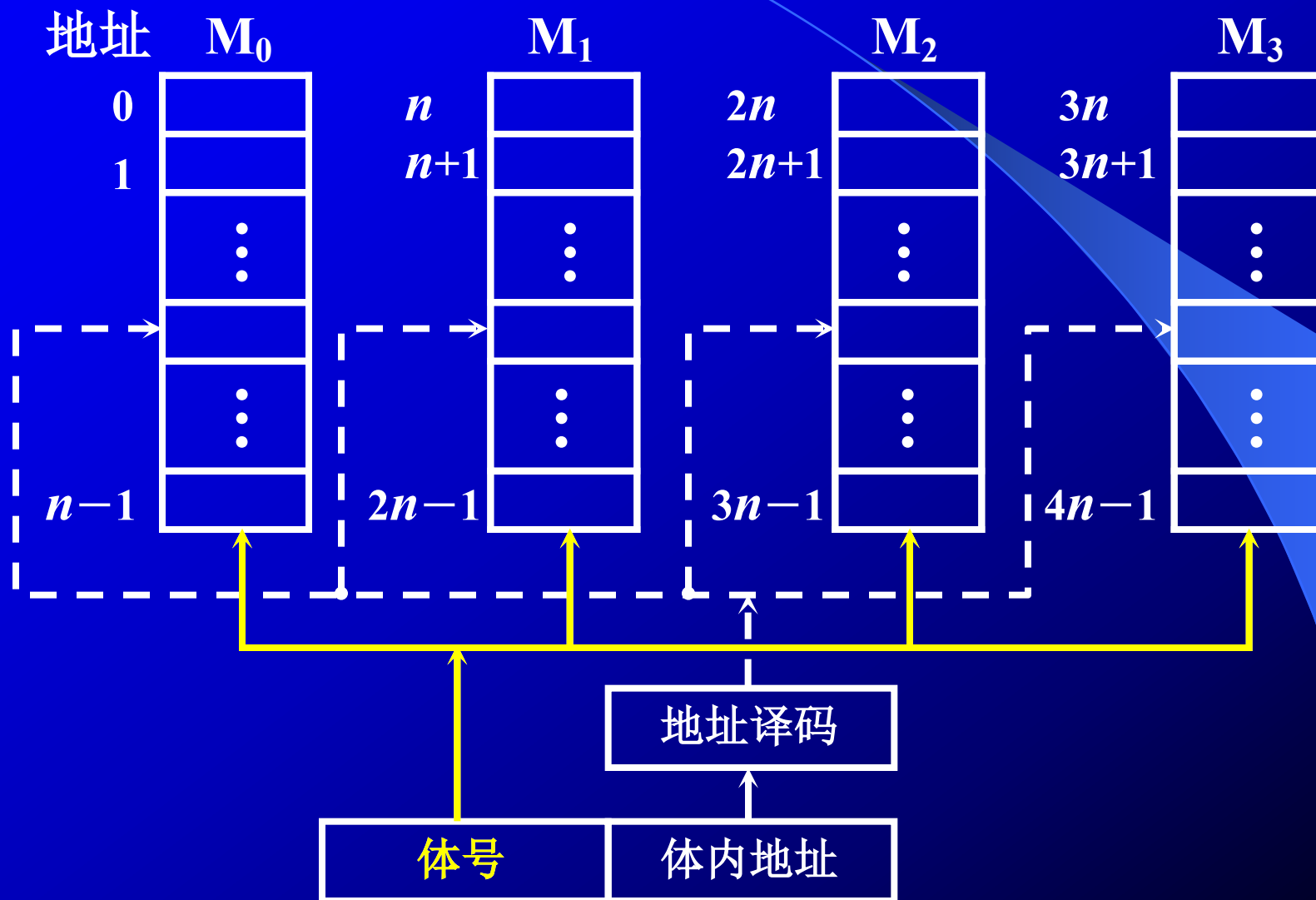
顺序编址



(1) 高位交叉

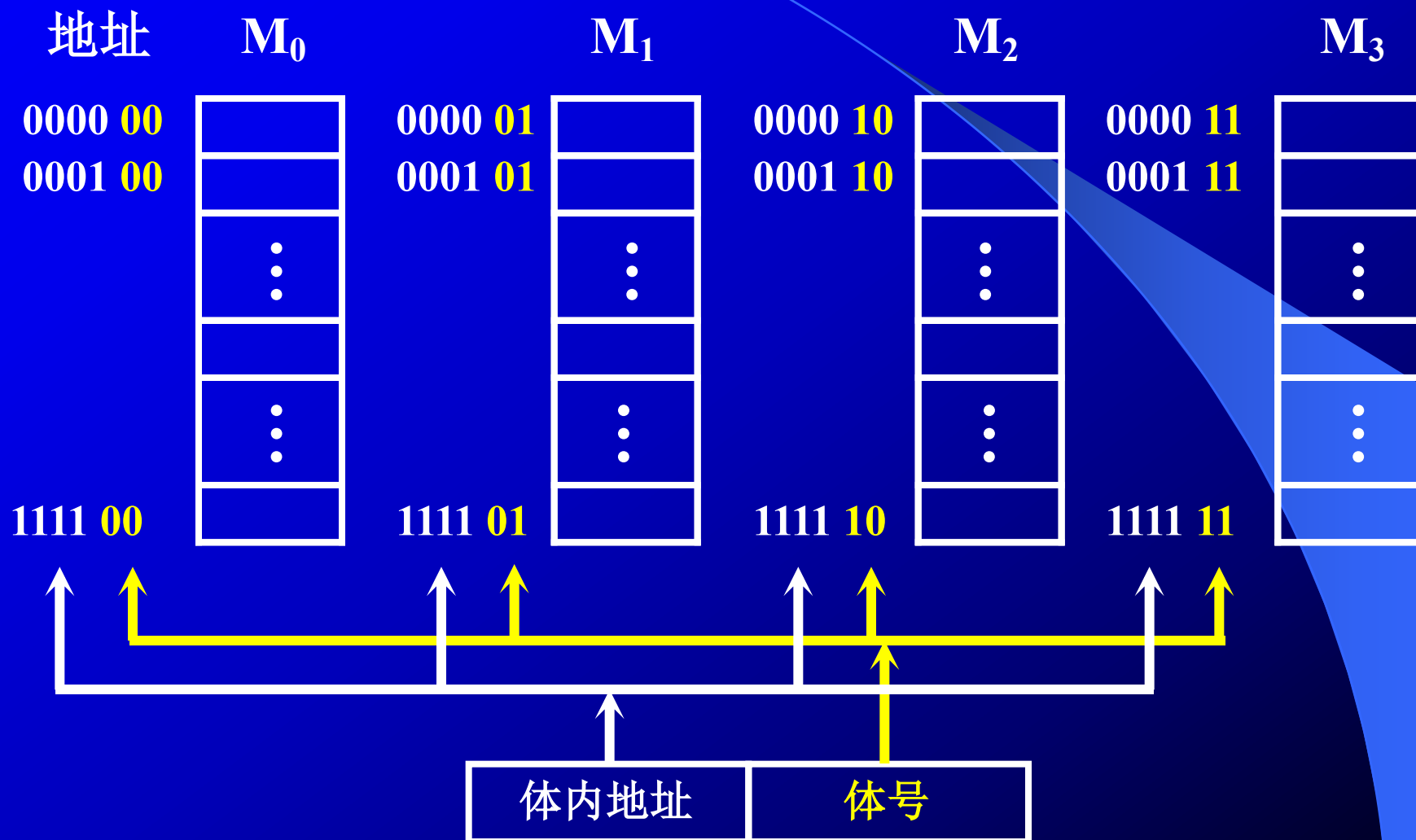
各个体并行工作

4.2



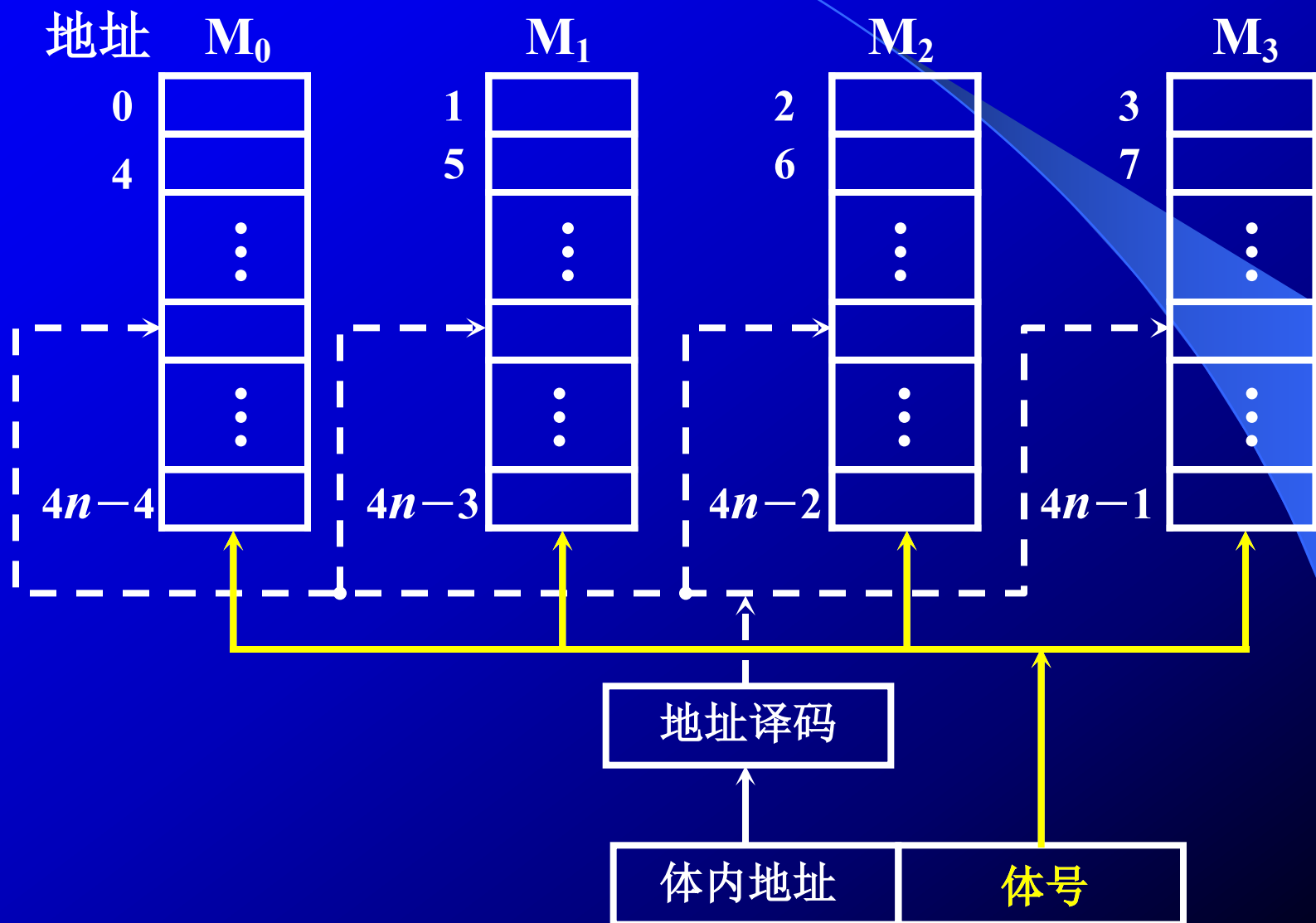
(2) 低位交叉 各个体轮流编址

4.2



(2) 低位交叉 各个体轮流编址

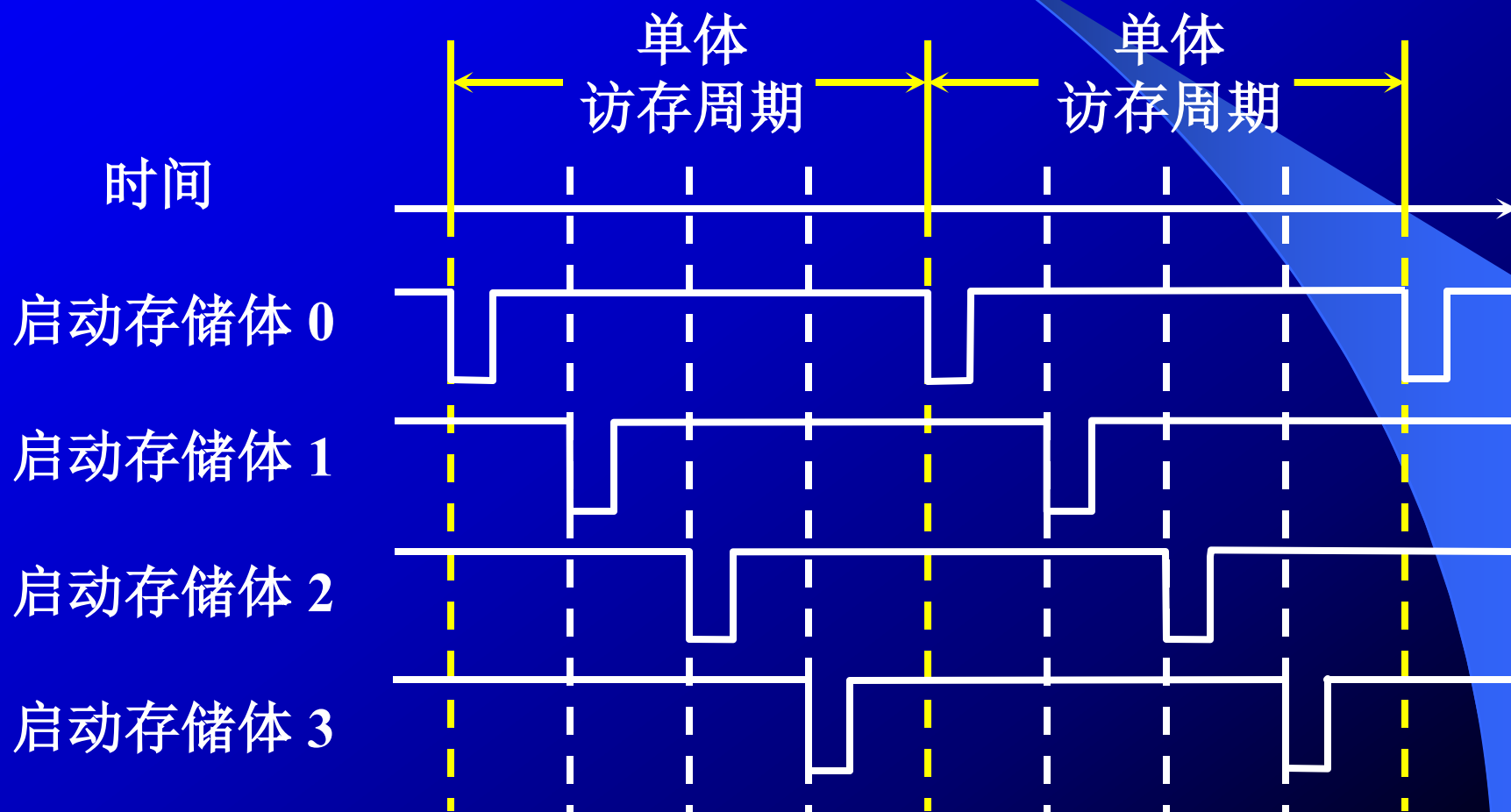
4.2



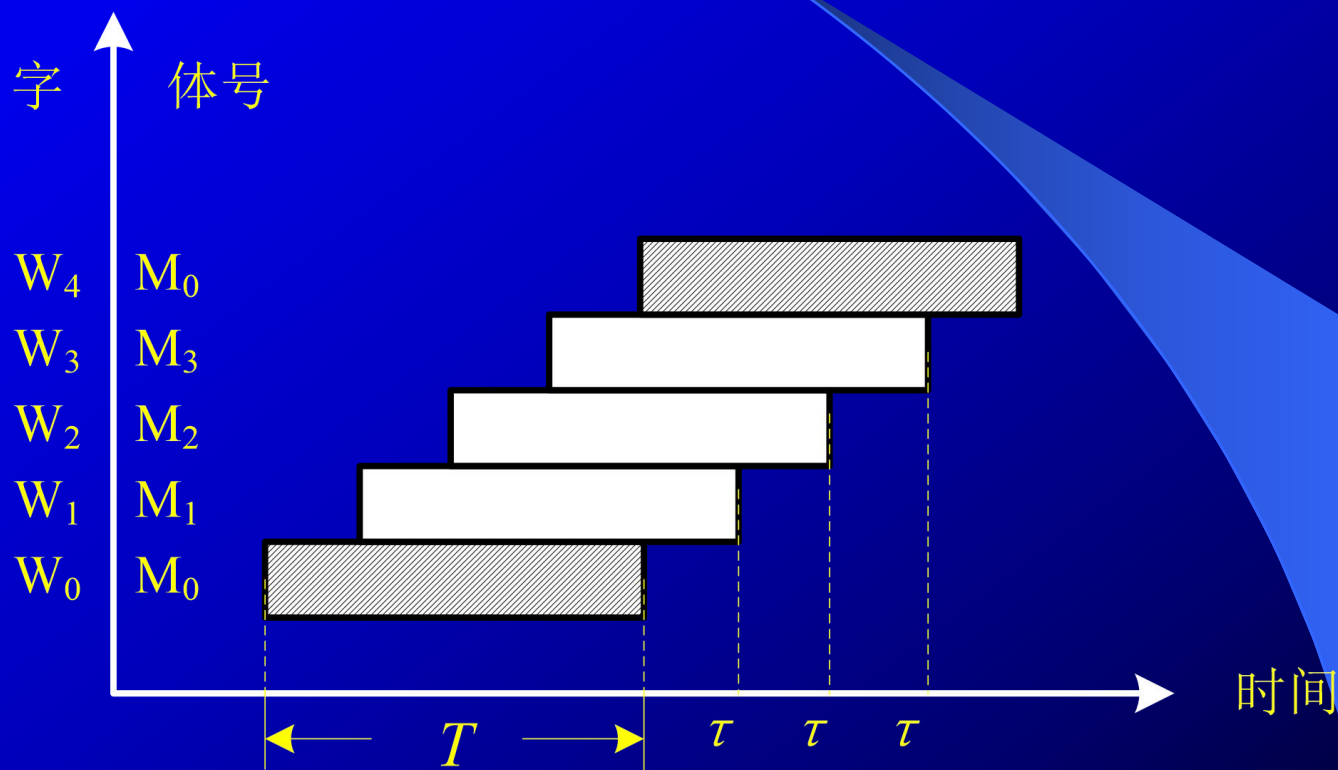
低位交叉的特点

4.2

在不改变存取周期的前提下，增加存储器的带宽

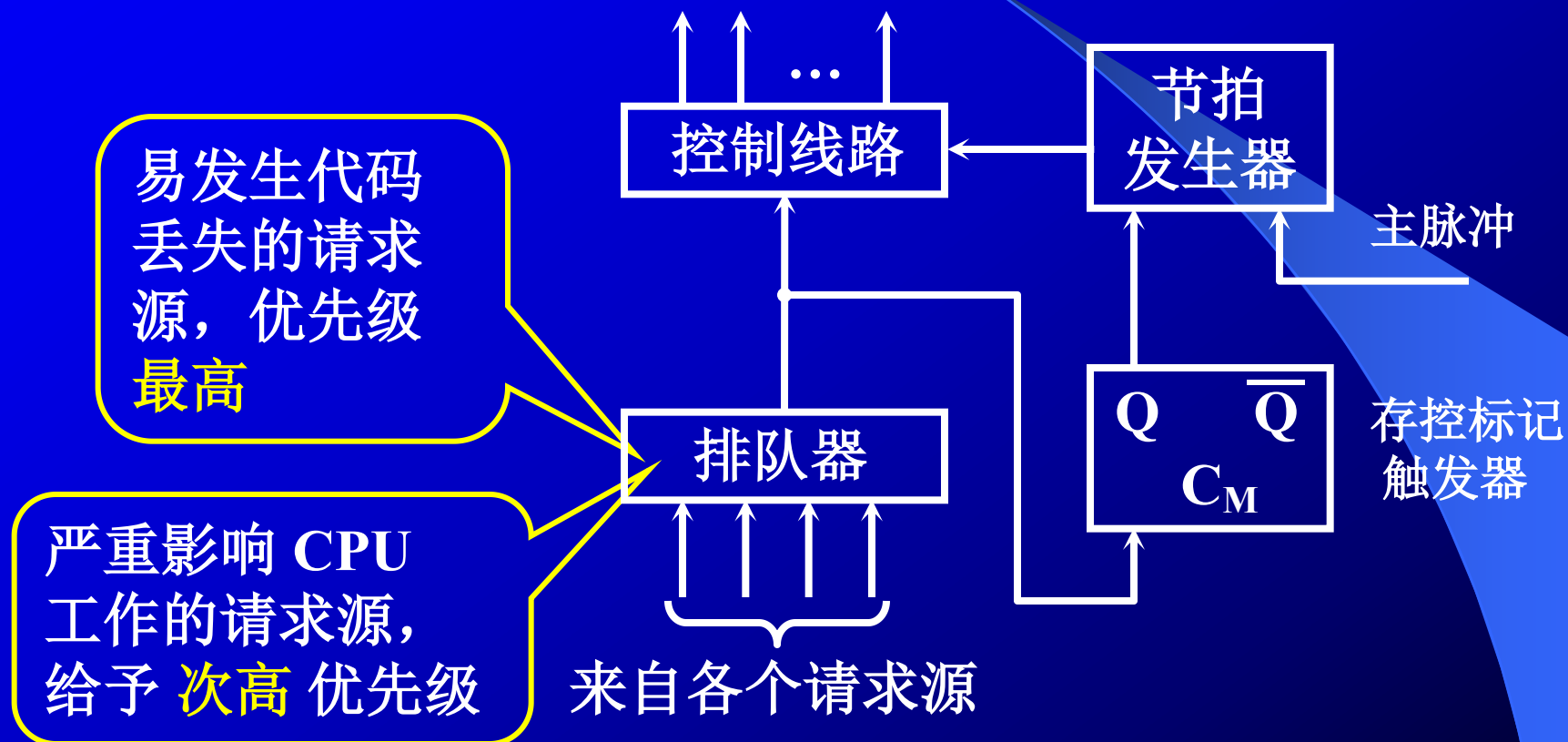


设四体低位交叉存储器，存取周期为 T ，总线传输周期为 τ ，为实现流水线方式存取，应满足 $T = 4\tau$ 。



连续读取 4 个字所需的时间为 $T + (4 - 1)\tau$

(3) 存储器控制部件（简称存控）



3.高性能存储芯片

4.2

(1) SDRAM (同步 DRAM)

在系统时钟的控制下进行读出和写入

CPU 无须等待

(2) RDRAM

由 **Rambus** 开发，主要解决 存储器带宽 问题

(3) 带 Cache 的 DRAM

在 DRAM 的芯片内 集成 了一个由 **SRAM** 组成的

Cache，有利于 猝发式读取



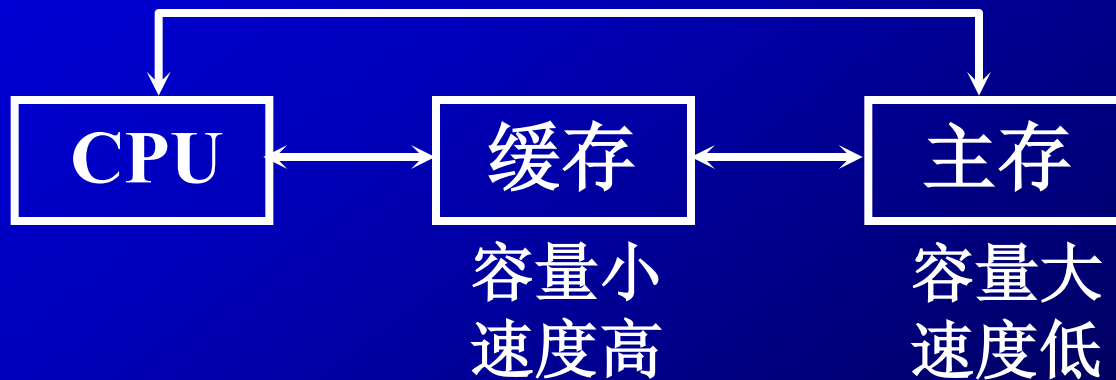
4.3 高速缓冲存储器

一、概述

1. 问题的提出

避免 CPU “空等” 现象

CPU 和主存（DRAM）的速度差异

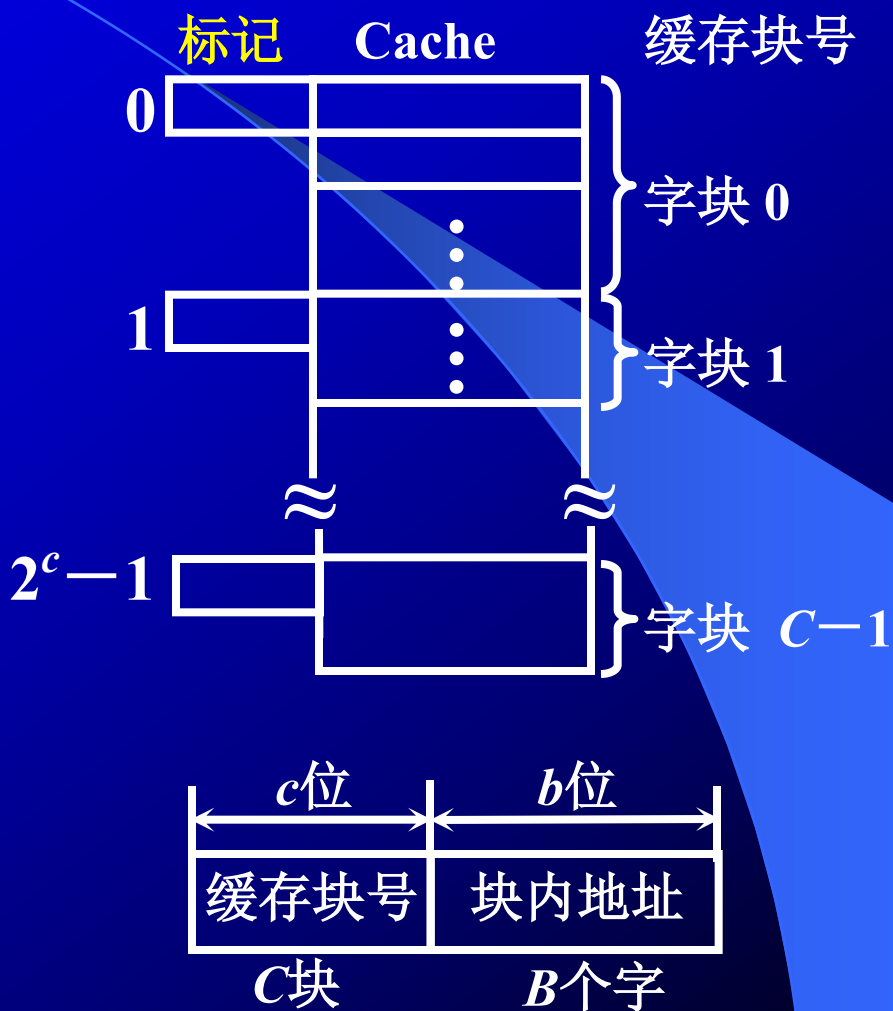
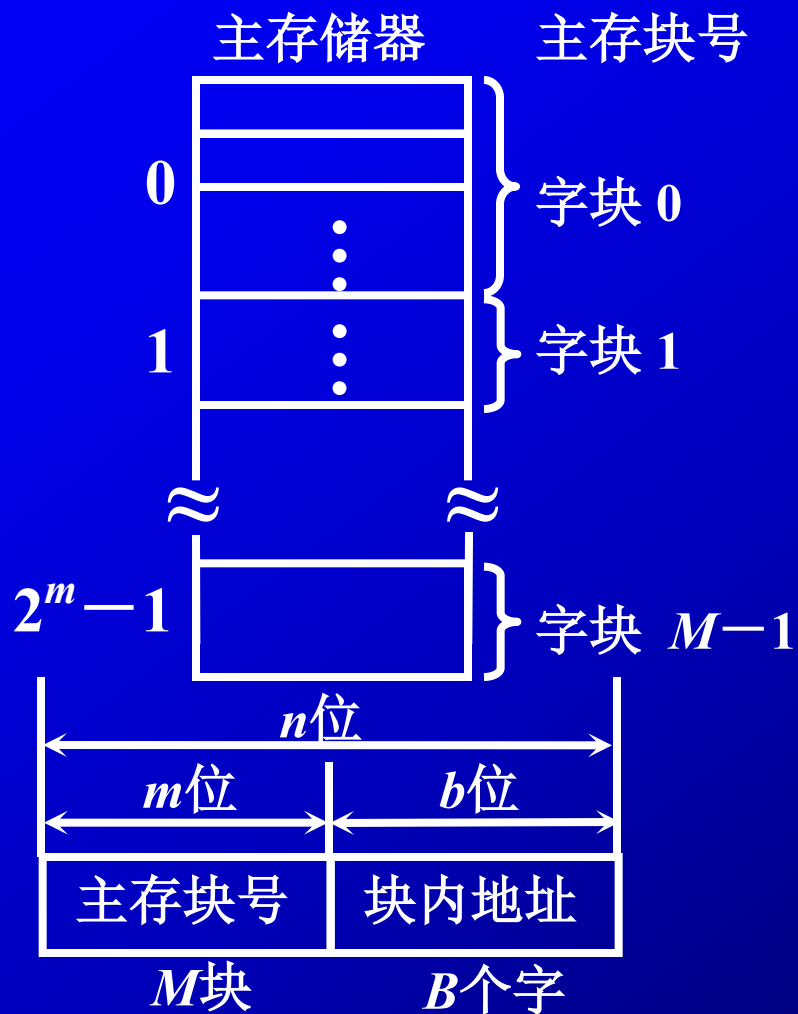


程序访问的局部性原理

2. Cache 的工作原理

4.3

(1) 主存和缓存的编址



主存和缓存按块存储

块的大小相同 B 为块长

(2) 命中与未命中

缓存共有 C 块

主存共有 M 块 $M \gg C$

命中

主存块 调入 缓存

主存块与缓存块 建立 了对应关系

用 标记记录 与某缓存块建立了对应关系的 主存块号

未命中

主存块 未调入 缓存

主存块与缓存块 未建立 对应关系



(3) Cache 的命中率

CPU 欲访问的信息在 Cache 中的 **比率**

命中率 与 Cache 的 **容量** 与 **块长** 有关

一般每块可取 4 ~ 8 个字

块长 取一个存取周期内从主存调出的信息长度

CRAY_1	16体交叉	块长取 16 个存储字
IBM 370/168	4体交叉	块长取 4 个存储字 (64位 $\times$ 4 = 256位)



(4) Cache –主存系统的效率

效率 e 与 命中率 有关

$$e = \frac{\text{访问 Cache 的时间}}{\text{平均访问时间}} \times 100\%$$

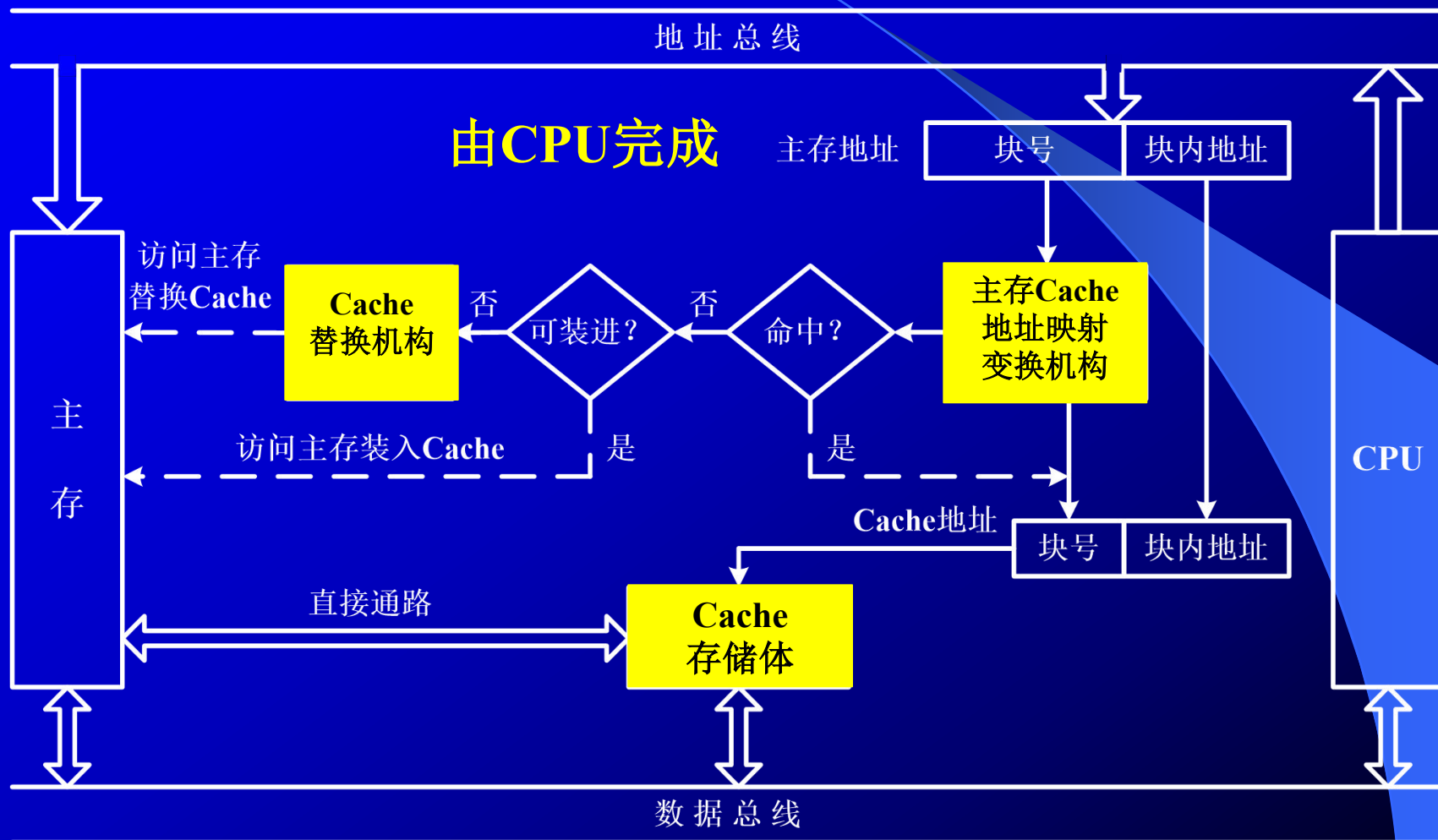
设 Cache 命中率为 h ，访问 Cache 的时间为 t_c ，
访问主存的时间为 t_m

$$\text{则 } e = \frac{t_c}{h \times t_c + (1-h) \times t_m} \times 100\%$$



3. Cache 的基本结构

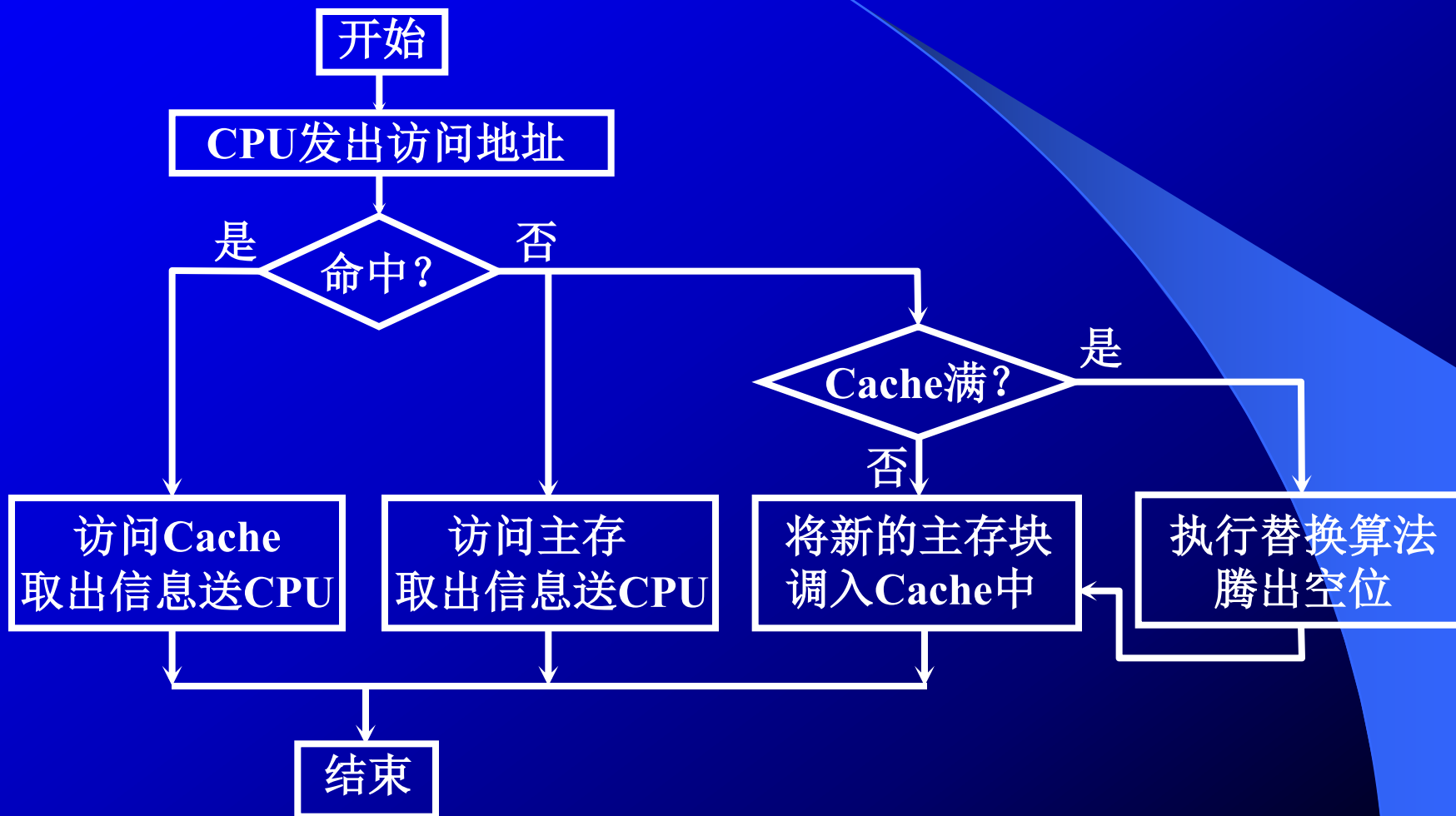
4.3



4. Cache 的读写操作

读

4.3



4. Cache 的 读写 操作

4.3

写 Cache 和主存的一致性

- 写直达法 (Write-through)

写操作时数据既写入Cache又写入主存

写操作时间就是访问主存的时间，读操作时不涉及对主存的写操作，更新策略比较容易实现

- 写回法 (Write-back)

写操作时只把数据写入 Cache 而不写入主存
当 Cache 数据被替换出去时才写回主存

写操作时间就是访问 Cache 的时间，

读操作 Cache 失效发生数据替换时，

被替换的块需写回主存，增加了 Cache 的复杂性



5. Cache 的改进

4.3

(1) 增加 Cache 的级数

片载（片内）Cache

片外 Cache

(2) 统一缓存和分立缓存

指令 Cache 数据 Cache

与主存结构有关

与指令执行的控制方式有关 是否流水

Pentium

8K 指令 Cache

8K 数据 Cache

PowerPC620

32K 指令 Cache

32K 数据 Cache

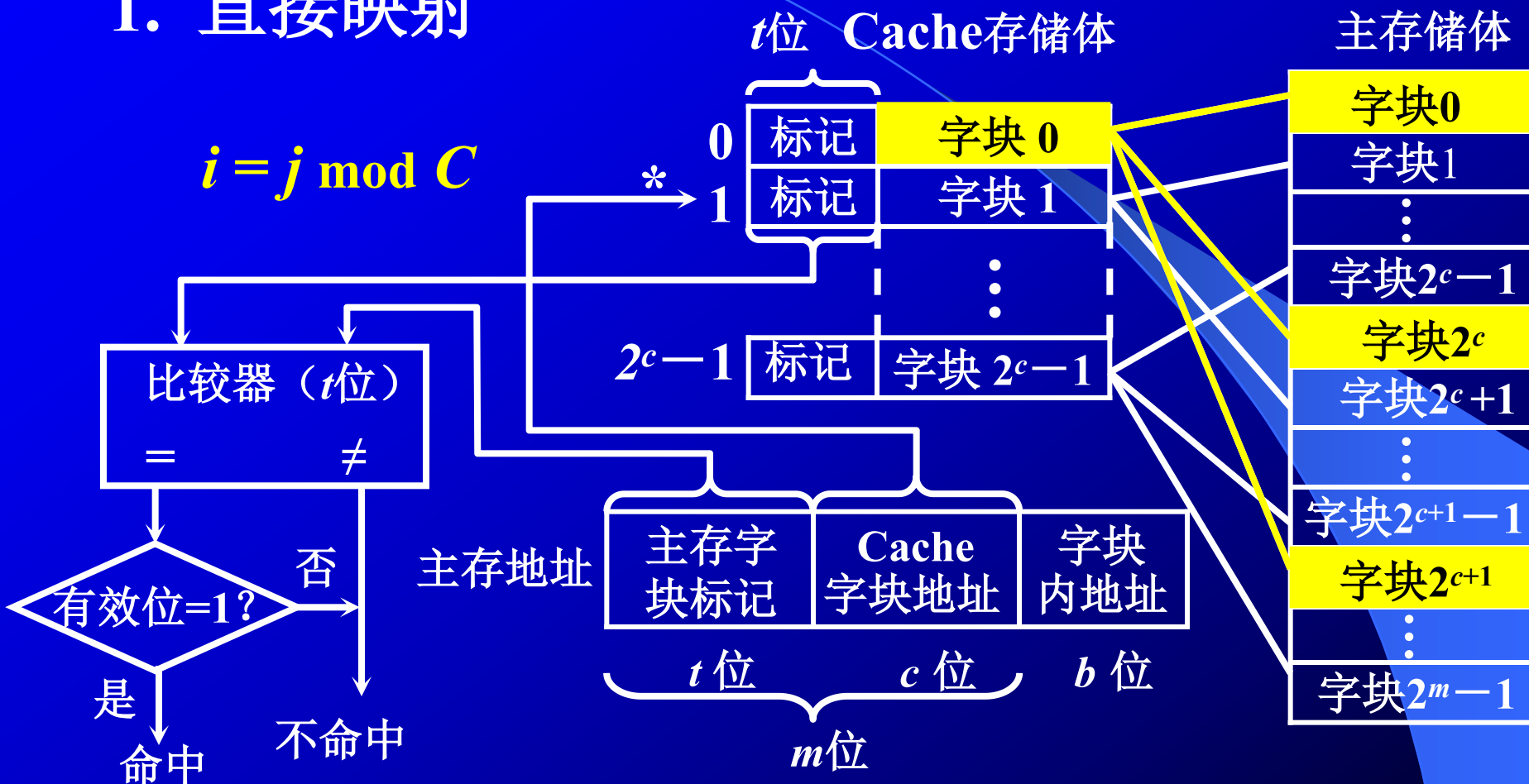


二、Cache – 主存的地址映射

4.3

1. 直接映射

$$i = j \bmod C$$

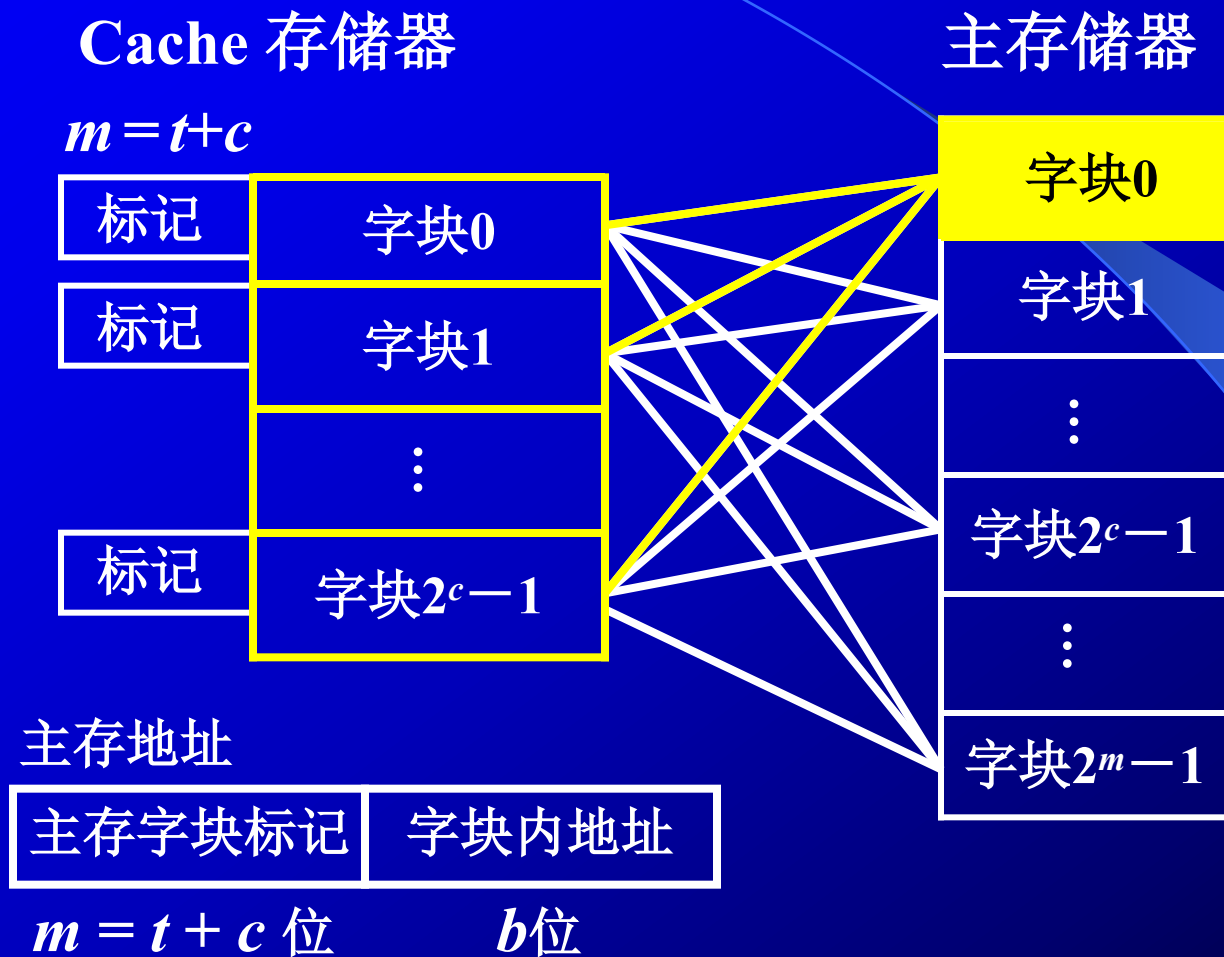


每个缓存块 i 可以和若干个主存块对应

每个主存块 j 只能和一个缓存块对应

2. 全相联映射

4.3



主存 中的 任一块 可以映射到 缓存 中的 任一块

3. 组相联映射

4.3

主存储器

组 Cache 共 Q 组，每组内两块 ($r=1$)

0	标记	字块 0	标记	字块 1
1	标记	字块 2	标记	字块 3
	⋮	⋮	⋮	⋮
$2^{c-r}-1$	标记	字块 2^c-2	标记	字块 2^c-1

主存地址

主存字块标记	组地址	字块内地址
$s = t + r$ 位	$q = c - r$ 位	b 位
m 位		

字块0
字块1
⋮
字块 $2^{c-r}-1$
字块 2^{c-r}
字块 $2^{c-r}+1$
⋮
字块 2^{c-r+1}
⋮
字块 2^m-1

$$i = j \bmod Q$$

直接映射

某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块

三、替换算法

1. 先进先出（FIFO）算法

2. 近期最少使用（LRU）算法

小结

成本与活

直接 某一主存块只能固定映射到某一缓存块

全相联 某一主存块能映射到任一缓存块

组相联 某一主存块只能映射到某一缓存组中的任一块

4.4 辅助存储器

一、概述

1. 特点 不直接与 CPU 交换信息

2. 磁表面存储器的技术指标

(1) 记录密度 道密度 D_t 位密度 D_b

(2) 存储容量 $C = n \times k \times s$

(3) 平均寻址时间 寻道时间 + 等待时间

辅存的速度 { 寻址时间
 磁头读写时间

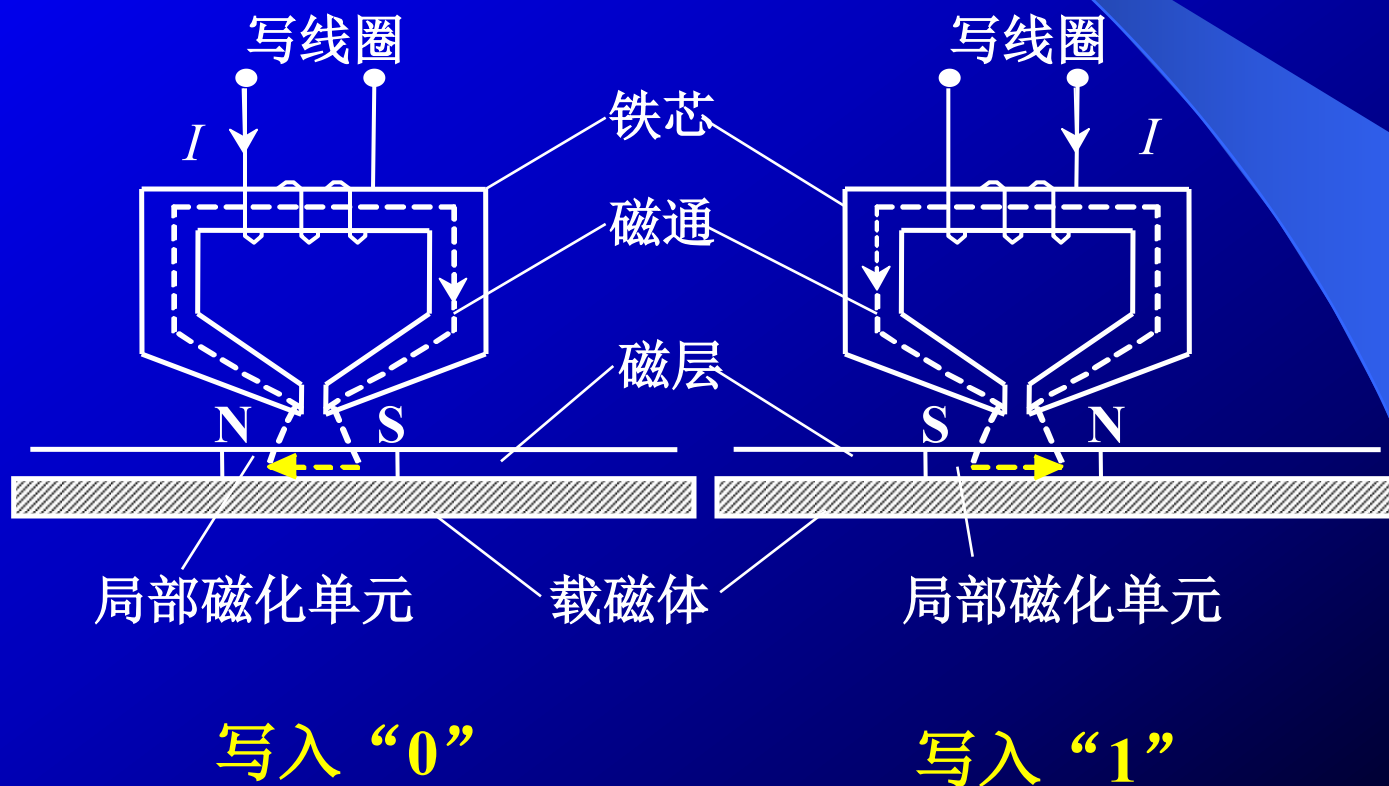
(4) 数据传输率 $D_r = D_b \times V$

(5) 误码率 出错信息位数与读出信息的总位数之比



二、磁记录原理和记录方式

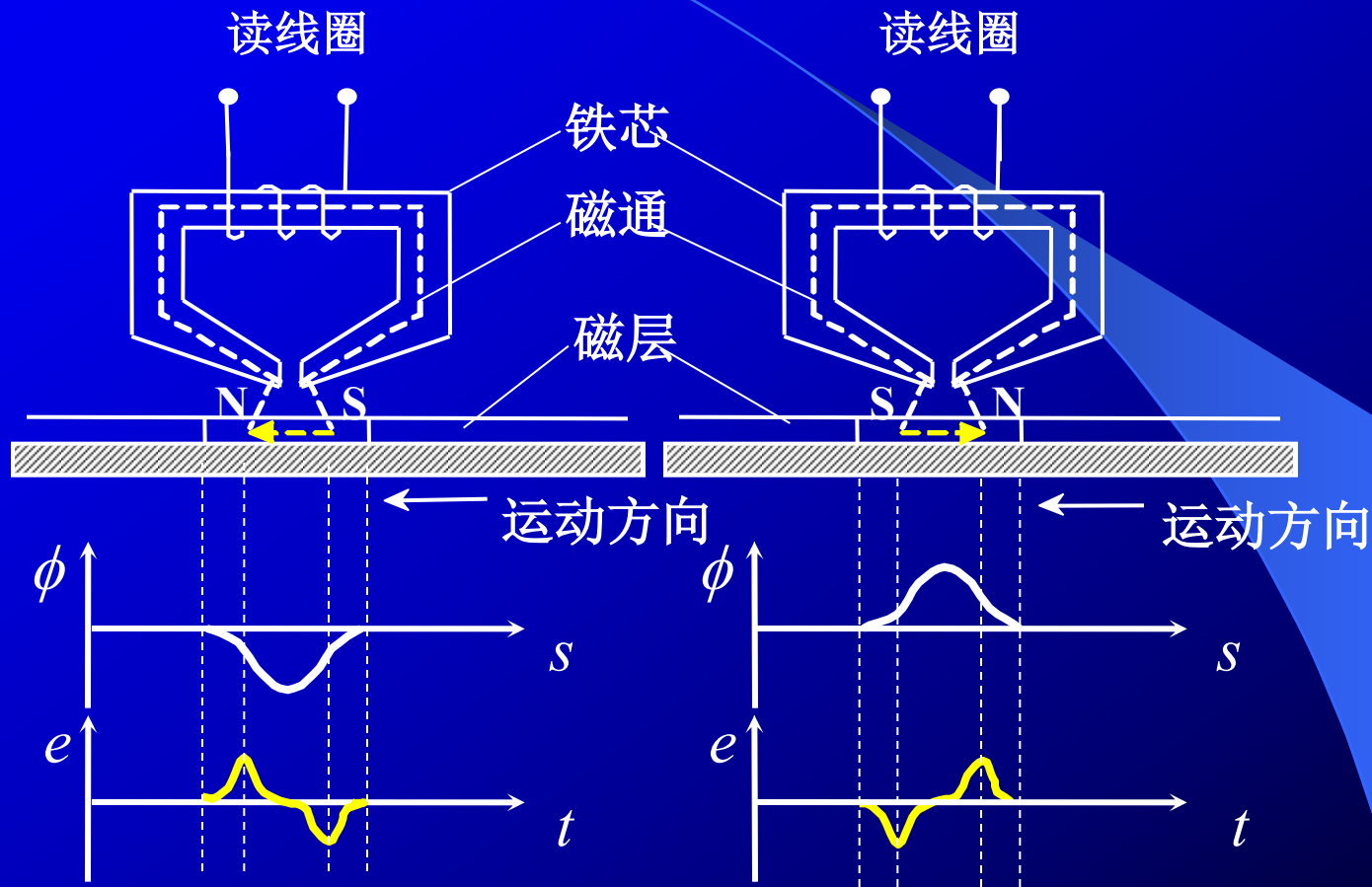
1. 磁记录原理 写



1. 磁记录原理

4.4

读



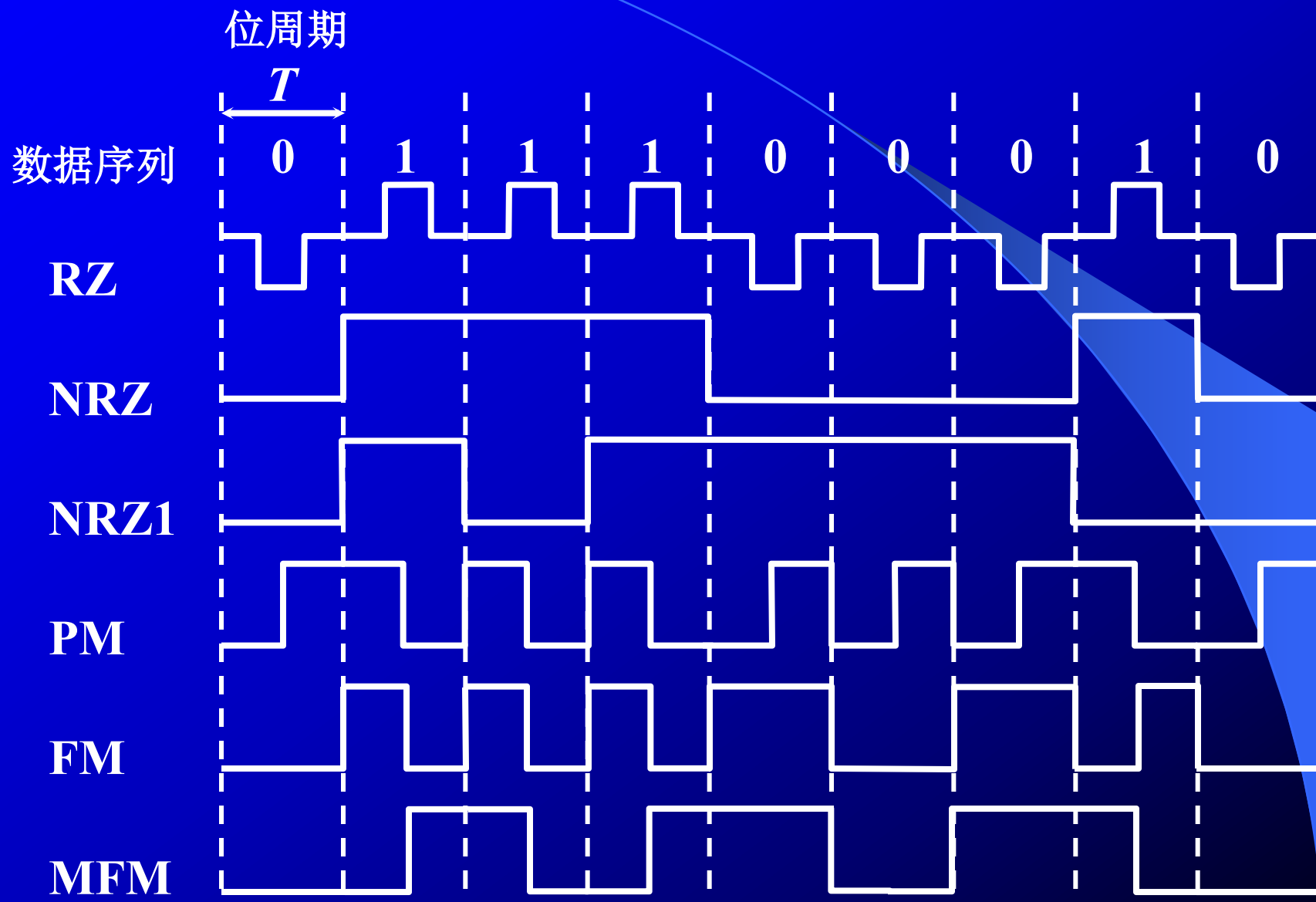
读出“0”

读出“1”

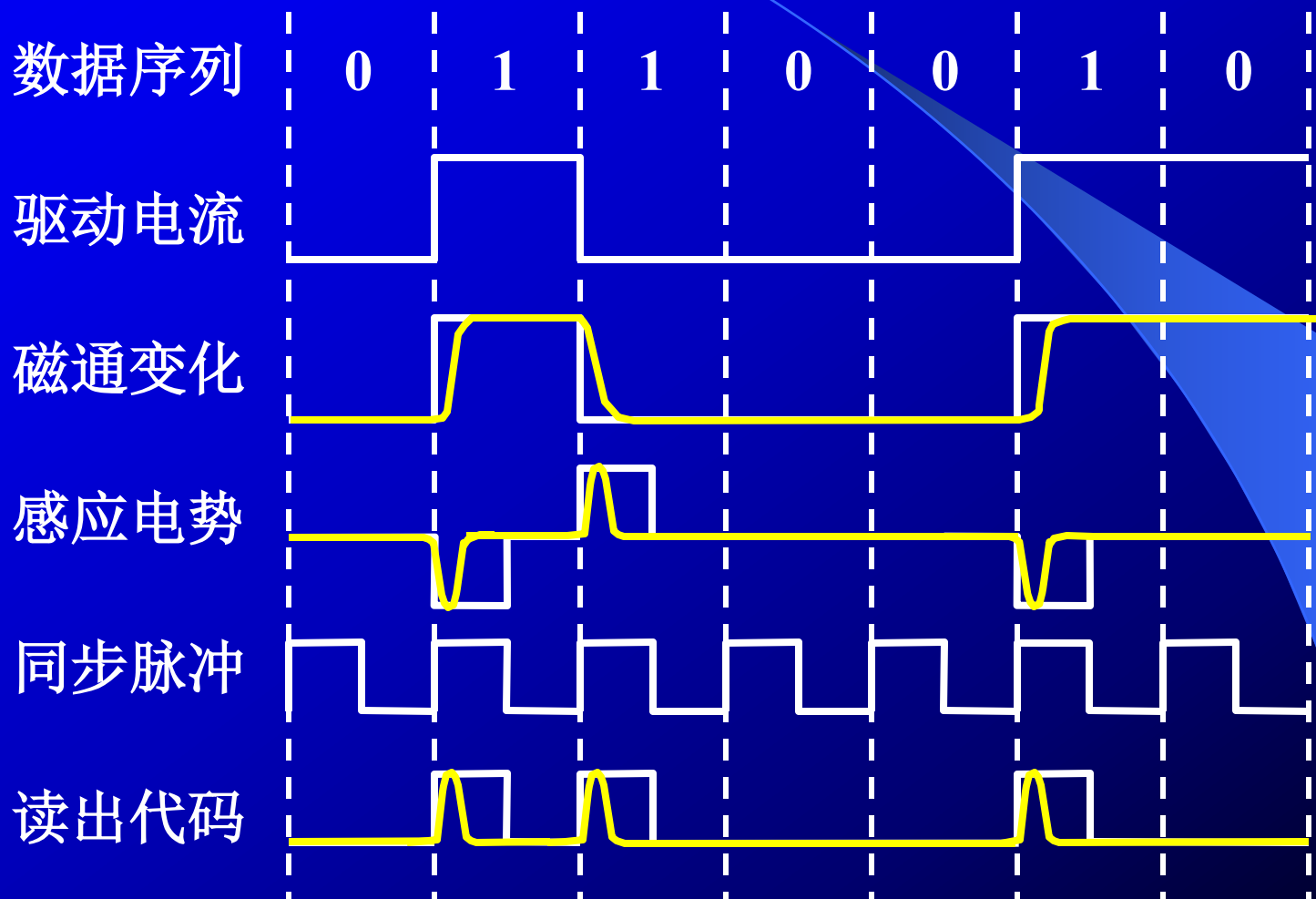


2. 磁表面存储器的记录方式

4.4



例 NRZ1 的读出代码波形



三、硬磁盘存储器

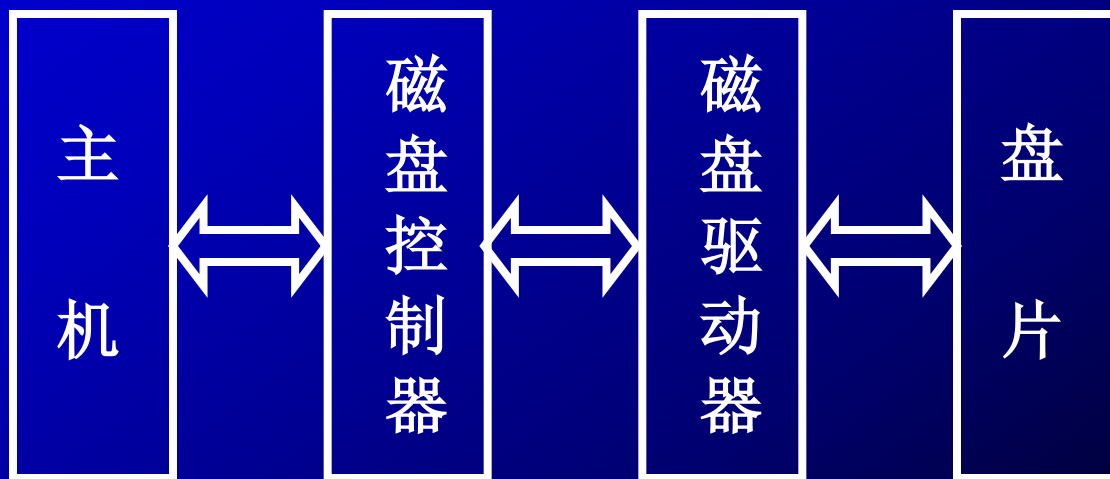
4.4

1. 硬磁盘存储器的类型

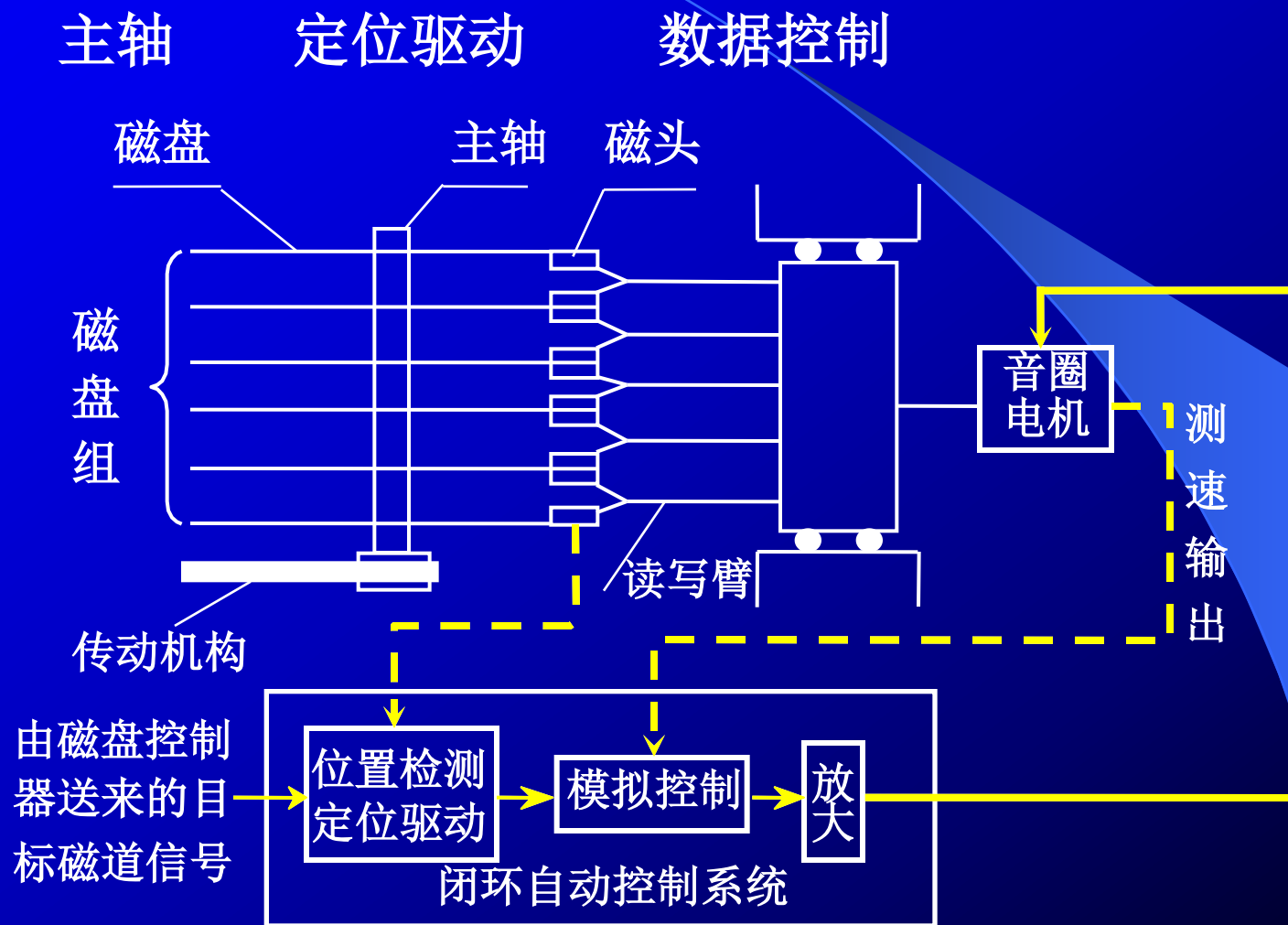
(1) 固定磁头和移动磁头

(2) 可换盘和固定盘

2. 硬磁盘存储器结构



(1) 磁盘驱动器



(2) 磁盘控制器

- 接收主机发来的命令，转换成磁盘驱动器的控制命令
- 实现主机和驱动器之间的数据格式转换
- 控制磁盘驱动器读写

磁盘控制器 是

主机与磁盘驱动器之间的 接口 { 对主机 通过总线
对硬盘 (设备)

(3) 盘片

由硬质铝合金材料制成



四、软磁盘存储器

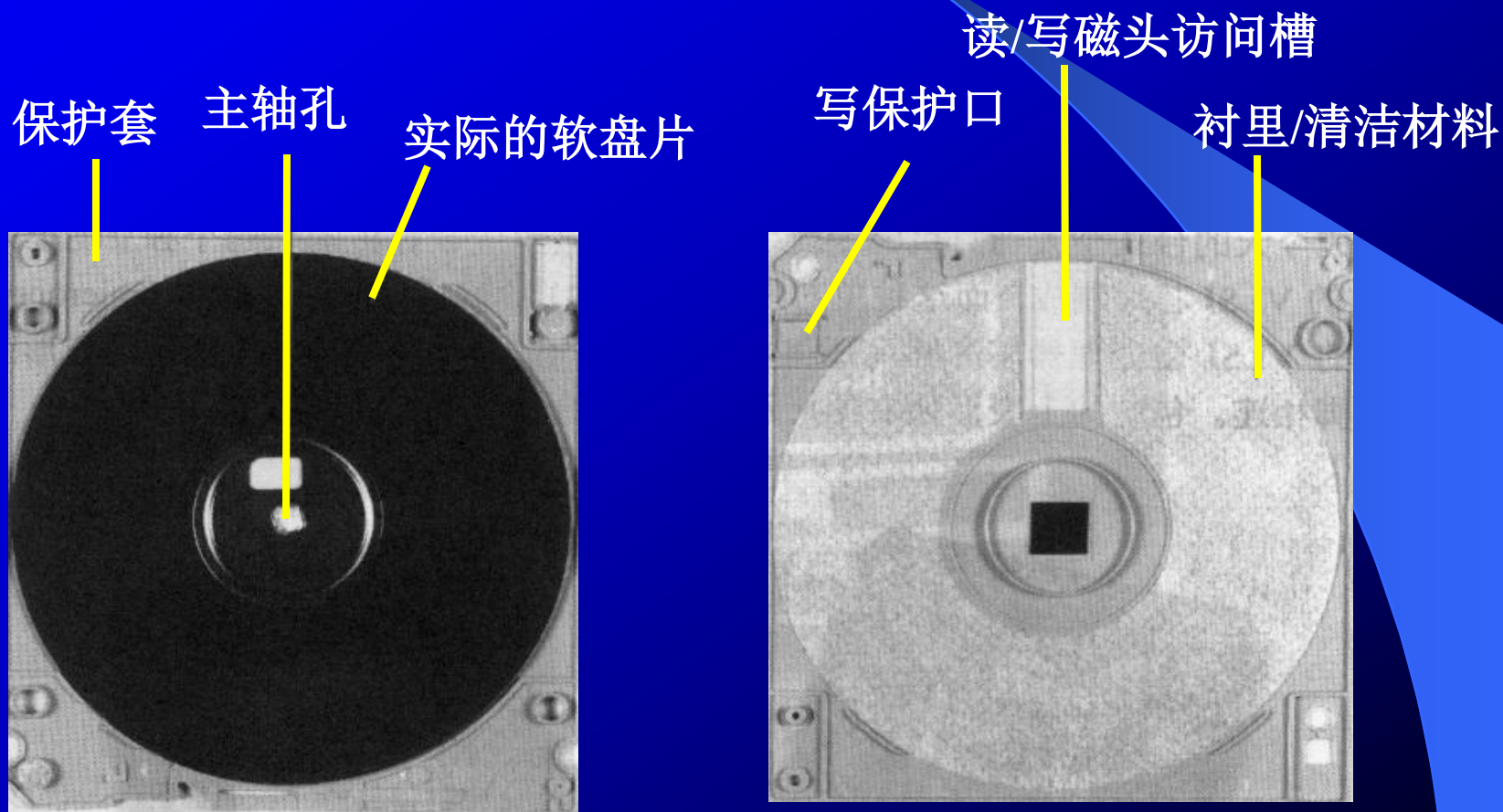
1. 概述

	硬盘	软盘
速度	高	低
磁头	固定、活动 浮动	活动 接触盘片
盘片	固定盘、盘组 大部分不可换	可换盘片
价格	高	低
环境	苛刻	



2. 软盘片

由聚酯薄膜制成



五、光盘存储器

1. 概述

采用光存储技术

利用激光写入和读出

{ 第一代光存储技术
第二代光存储技术

采用非磁性介质

不可擦写

采用磁性介质

可擦写

2. 光盘的存储原理

只读型和只写一次型

热作用（物理或化学变化）

可擦写光盘

热磁效应



第5章 输入输出系统

5.1 概述

5.2 外部设备

5.3 I/O接口

5.4 程序查询方式

5.5 程序中断方式

5.6 DMA方式



5.1 概述

一、输入输出系统的发展概况

1. 早期

分散连接

CPU 和 I/O设备 串行 工作 程序查询方式

2. 接口模块和 DMA 阶段

总线连接

CPU 和 I/O设备 并行 工作 { 中断方式
DMA 方式

3. 具有通道结构的阶段

4. 具有 I/O 处理机的阶段



二、输入输出系统的组成

5.1

1. I/O 软件

(1) I/O 指令 CPU 指令的一部分

操作码	命令码	设备码
-----	-----	-----

(2) 通道指令 通道自身的指令

指出数组的首地址、传送字数、操作命令

如 IBM/370 通道指令为 64 位

2. I/O 硬件

设备

I/O 接口

设备

设备控制器

通道



三、I/O 设备与主机的联系方式

5.1

1. I/O 设备编址方式

(1) 统一编址 用取数、存数指令

(2) 不统一编址 有专门的 I/O 指令

2. 设备选址

用设备选择电路识别是否被选中

3. 传送方式

(1) 串行

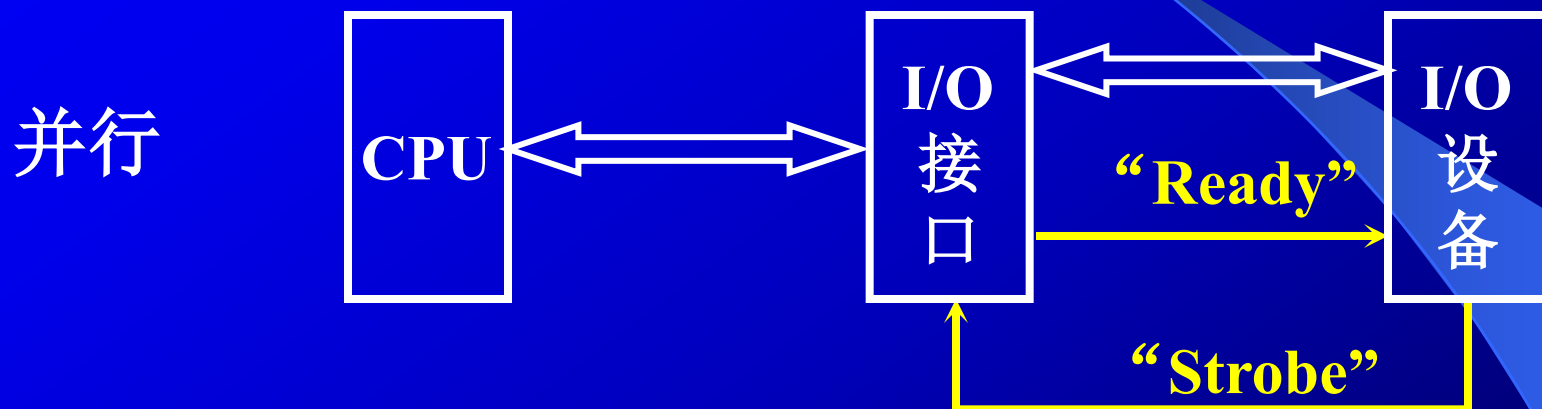
(2) 并行



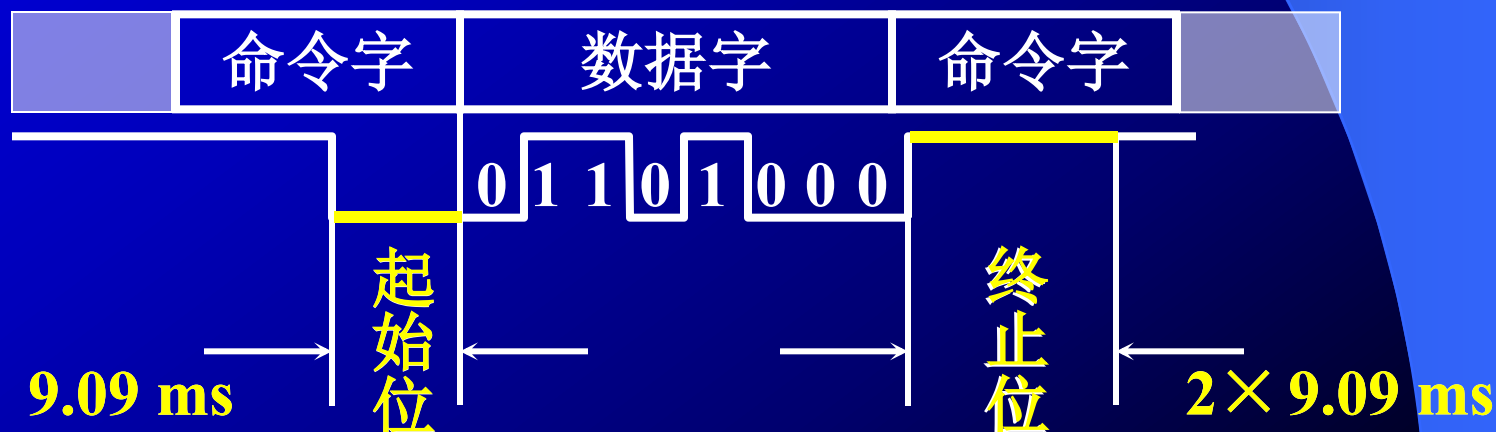
4. 联络方式

(1) 立即响应

(2) 异步工作采用应答信号



串行

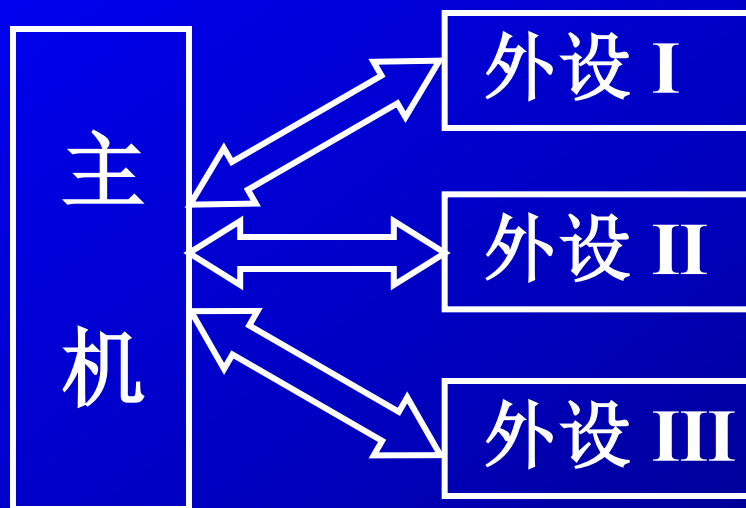


(3) 同步工作采用同步时标

5. I/O 设备与主机的连接方式

5.1

(1) 辐射式连接



每台设备都配有一套
控制线路和一组信号线

不便于增删设备

(2) 总线连接

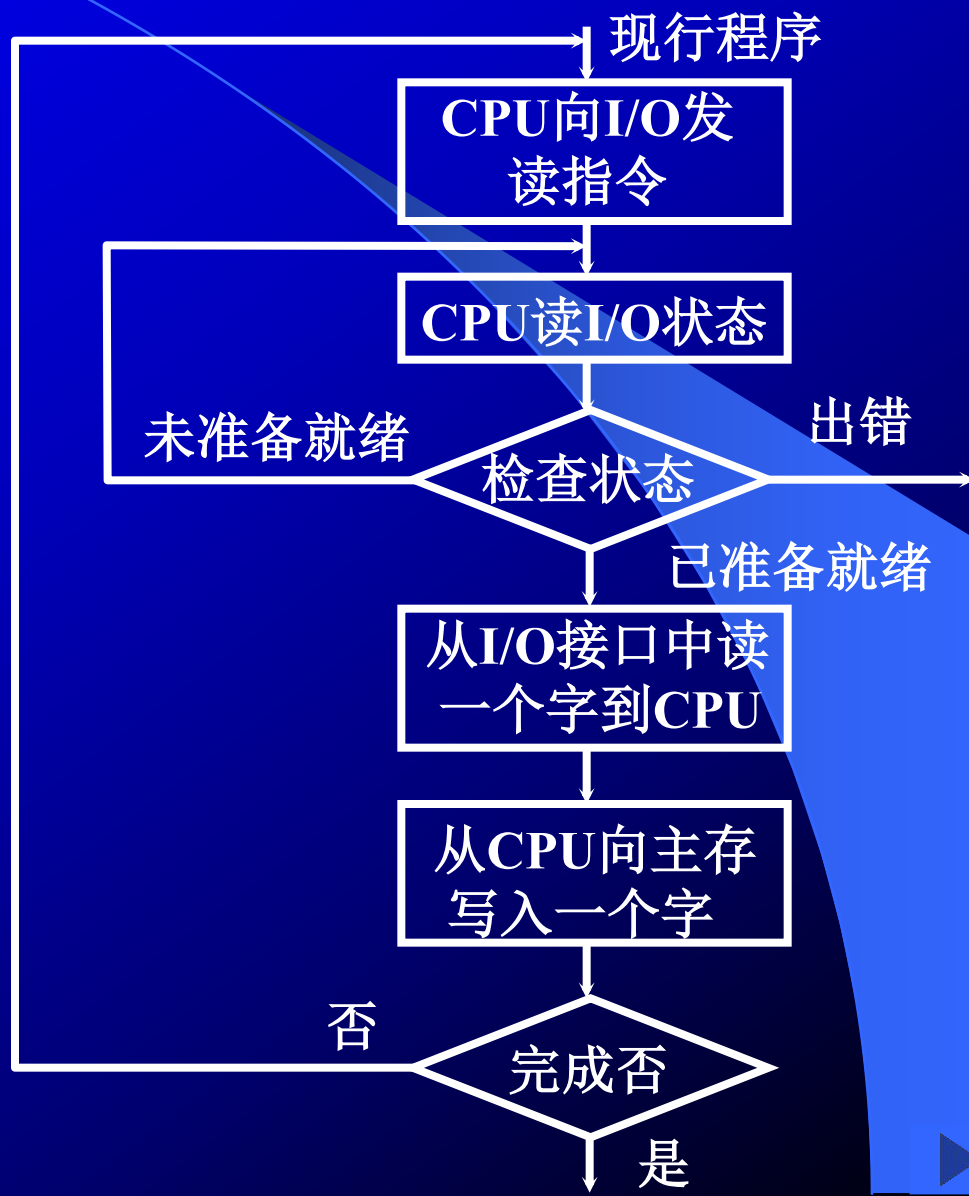
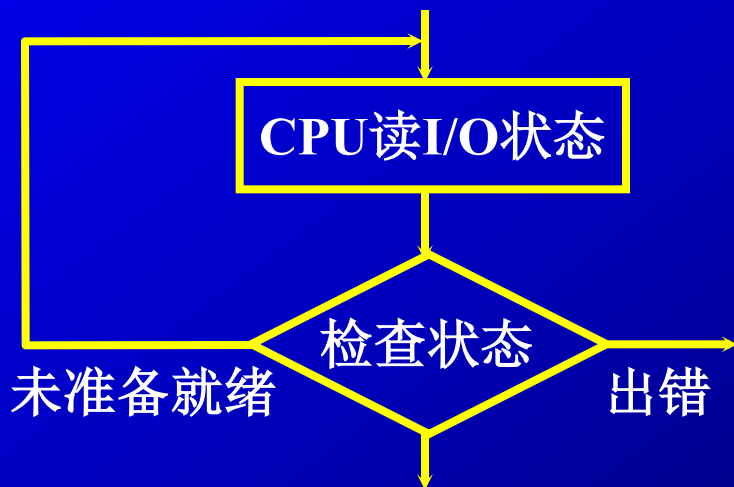
便于增删设备

四、I/O设备与主机信息传送的控制方式 5.1

1. 程序查询方式

CPU 和 I/O 串行工作

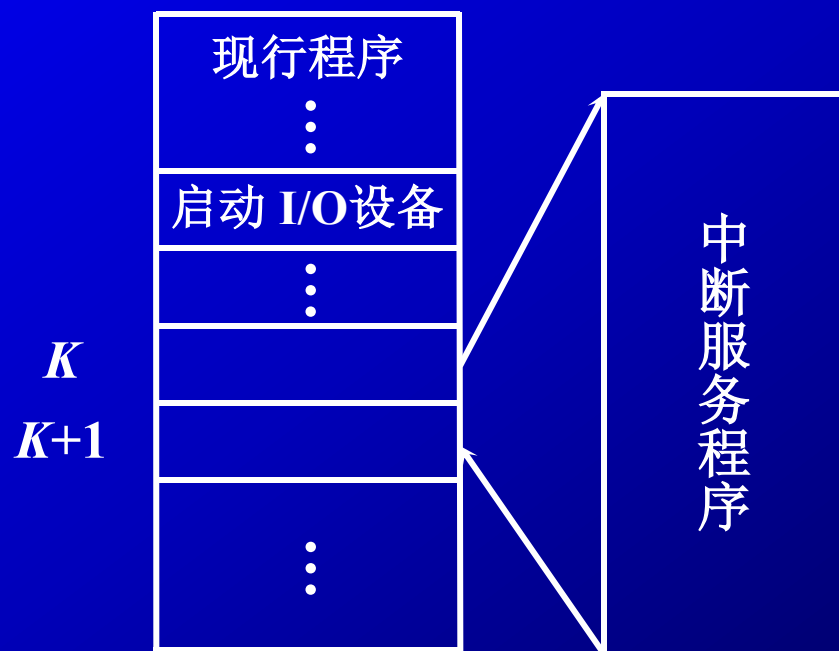
踏步等待



2. 程序中中断方式

I/O 工作 { 自身准备 CPU 不查询
与主机交换信息 CPU 暂停现程序

CPU 和 I/O 并行工作

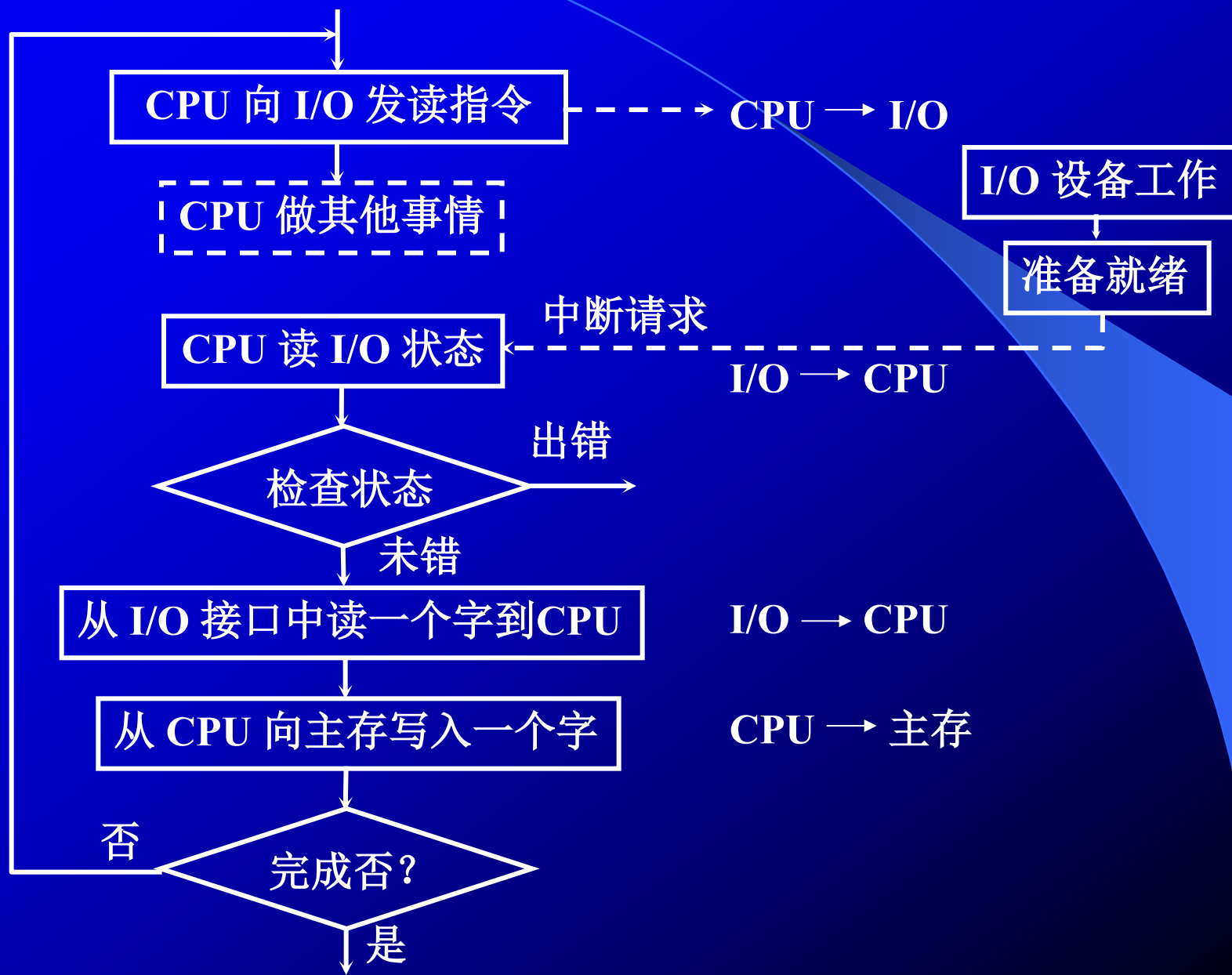


没有踏步等待现象

中断现程序

程序中中断方式流程

5.1



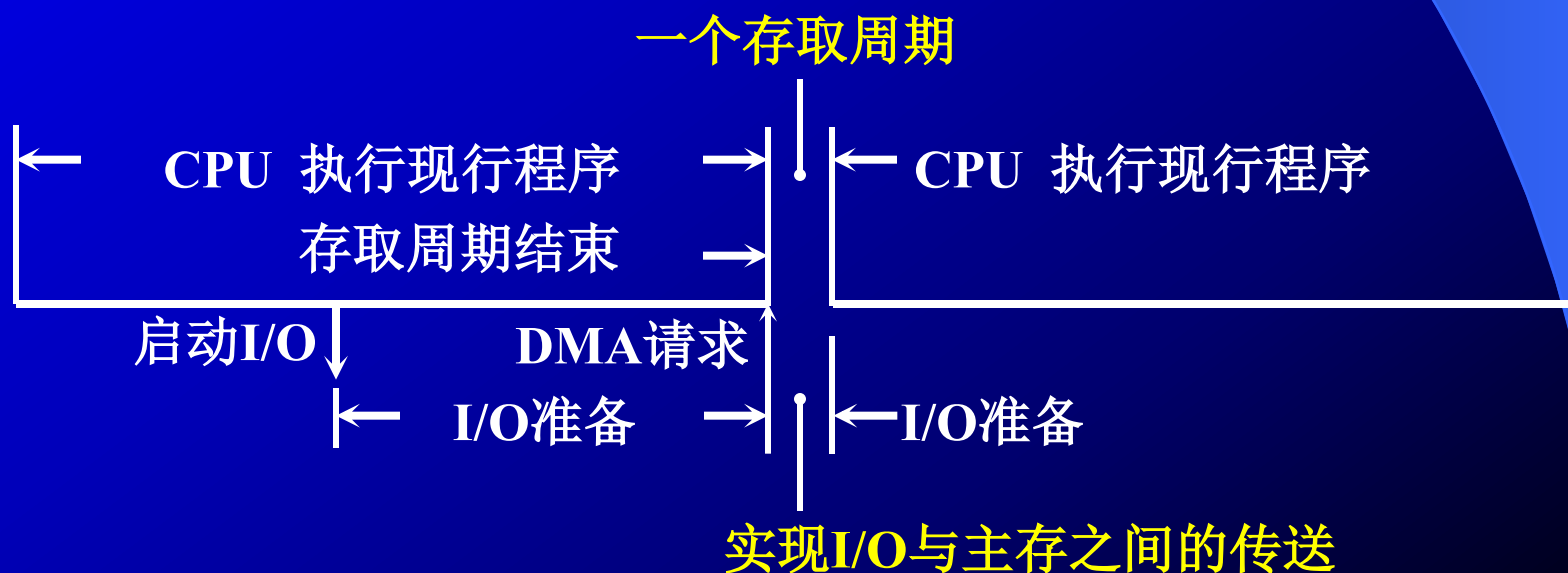
3. DMA 方式

主存和 I/O 之间有一条直接数据通道

不中断现行程序

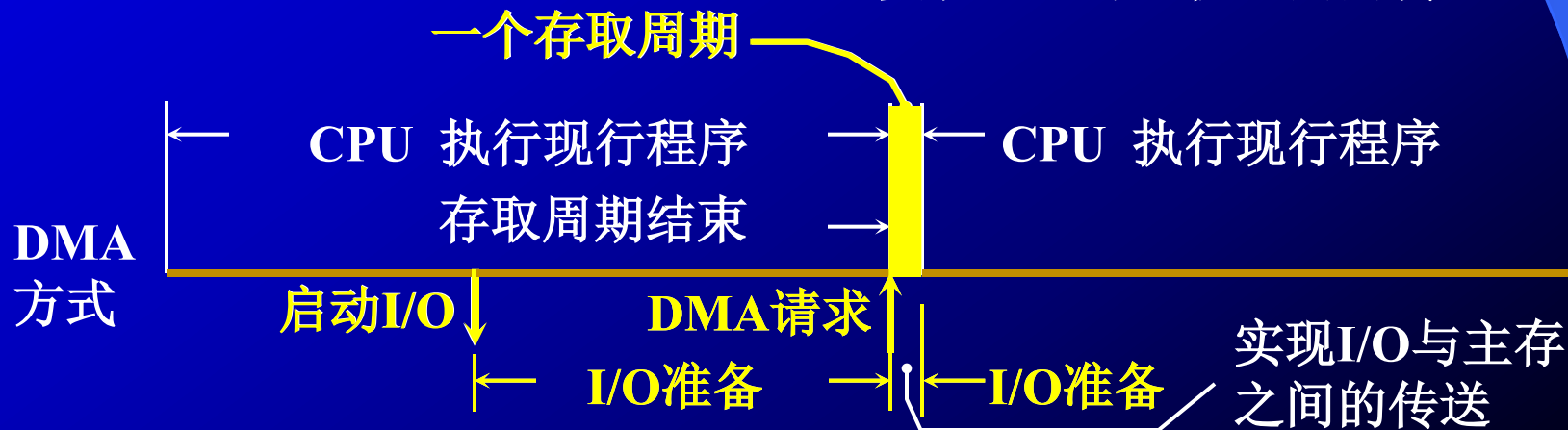
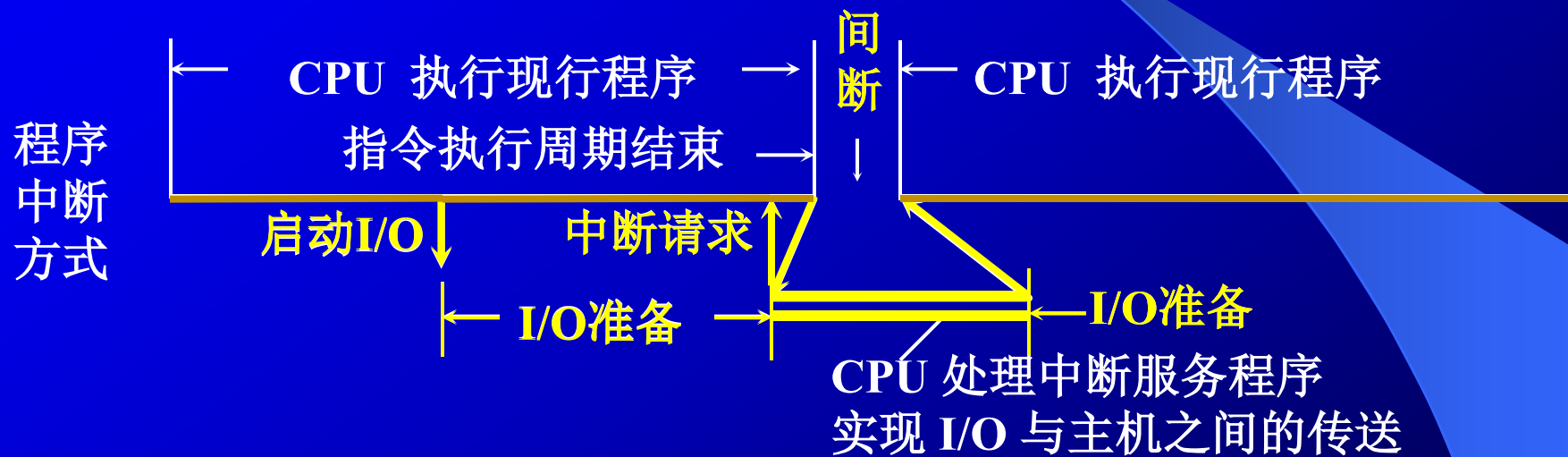
周期挪用（周期窃取）

CPU 和 I/O 并行工作



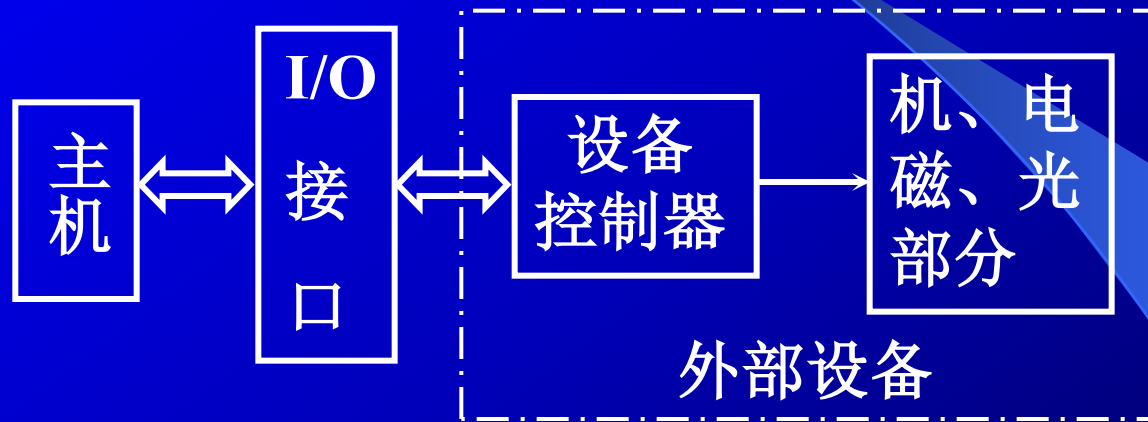
三种方式的 CPU 工作效率比较

5.1



5.2 I/O设备

一、概述



外部设备大致分三类

- | | |
|--------------|---------------|
| 1. 人机交互设备 | 键盘、鼠标、打印机、显示器 |
| 2. 计算机信息存储设备 | 磁盘、光盘、磁带 |
| 3. 机—机通信设备 | 调制解调器等 |

二、输入设备

1. 键盘

按键

判断哪个键按下

将此键翻译成 ASCII 码（编码键盘法）

2. 鼠标

机械式 金属球 电位器

光电式 光电转换器

3. 触摸屏

三、输出设备

5.2

1. 显示器

(1) 字符显示 字符发生器

(2) 图形显示 主观图像

(3) 图像显示 客观图像

2. 打印机

(1) 击打式 点阵式（逐字、逐行）

(2) 非击打式 激光（逐页）喷墨（逐字）



四、其他

1. A/D、D/A 模拟/数字（数字/模拟）转换器
2. 终端 由键盘和显示器组成
完成显示控制与存储、键盘管理及通信控制
3. 汉字处理 汉字输入、汉字存储、汉字输出

五、多媒体技术

1. 什么是多媒体
2. 多媒体计算机的关键技术

5.3 I/O 接口

一、概述

为什么要设置接口？

1. 实现设备的选择
2. 实现数据缓冲达到速度匹配
3. 实现数据串—并格式转换
4. 实现电平转换
5. 传送控制命令
6. 反映设备的状态（“忙”、“就绪”、“中断请



二、接口的功能和组成

5.3

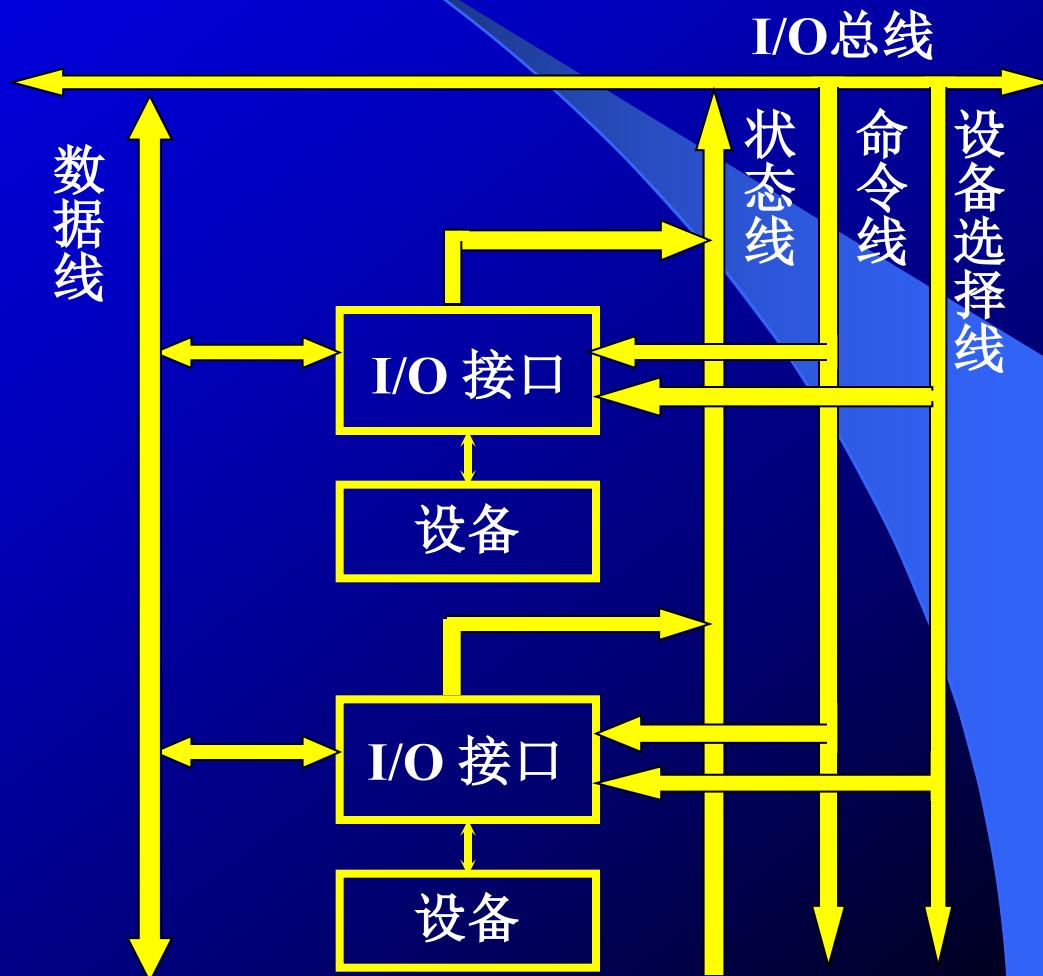
1. 总线连接方式的 I/O 接口电路

(1) 设备选择线

(2) 数据线

(3) 命令线

(4) 状态线



2. 接口的功能和组成

功能

选址功能

传送命令的功能

传送数据的功能

反映设备状态的功能

完成触发器 D

工作触发器 B

中断请求触发器 INTR

屏蔽触发器 MASK

组成

设备选择电路

命令寄存器、命令译码器

数据缓冲寄存器

设备状态标记

3. I/O 接口的基本组成



三、接口类型

1. 按数据 传送方式 分类

并行接口 Intel 8255

串行接口 Intel 8251

2. 按功能 选择的灵活性 分类

可编程接口 Intel 8255、 Intel 8251

不可编程接口 Intel 8212

3. 按 通用性 分类

通用接口 Intel 8255、 Intel 8251

专用接口 Intel 8279、 Intel 8275

4. 按数据传送的 控制方式 分类

中断接口 Intel 8259

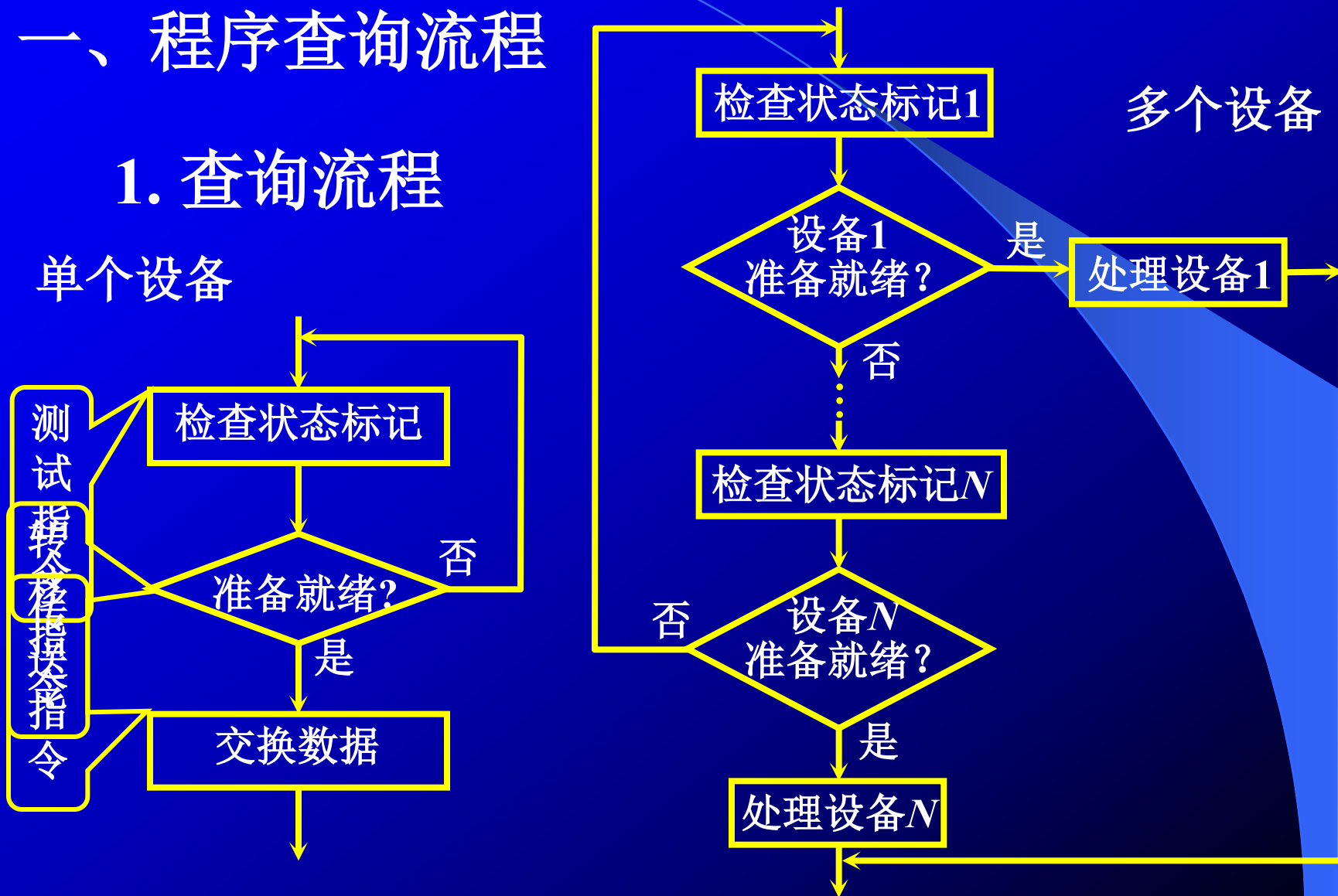
DMA 接口 Intel 8257



5.4 程序查询方式

一、程序查询流程

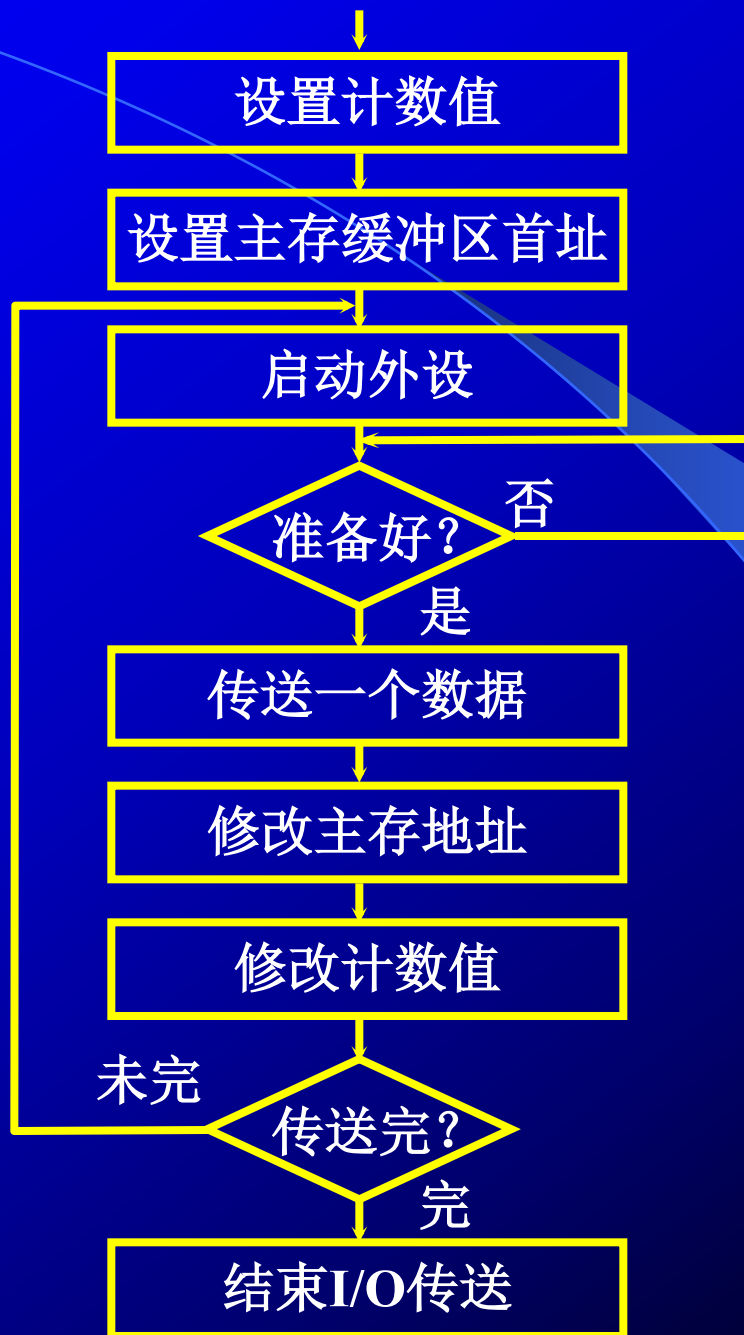
1. 查询流程



2. 程序流程

5.4

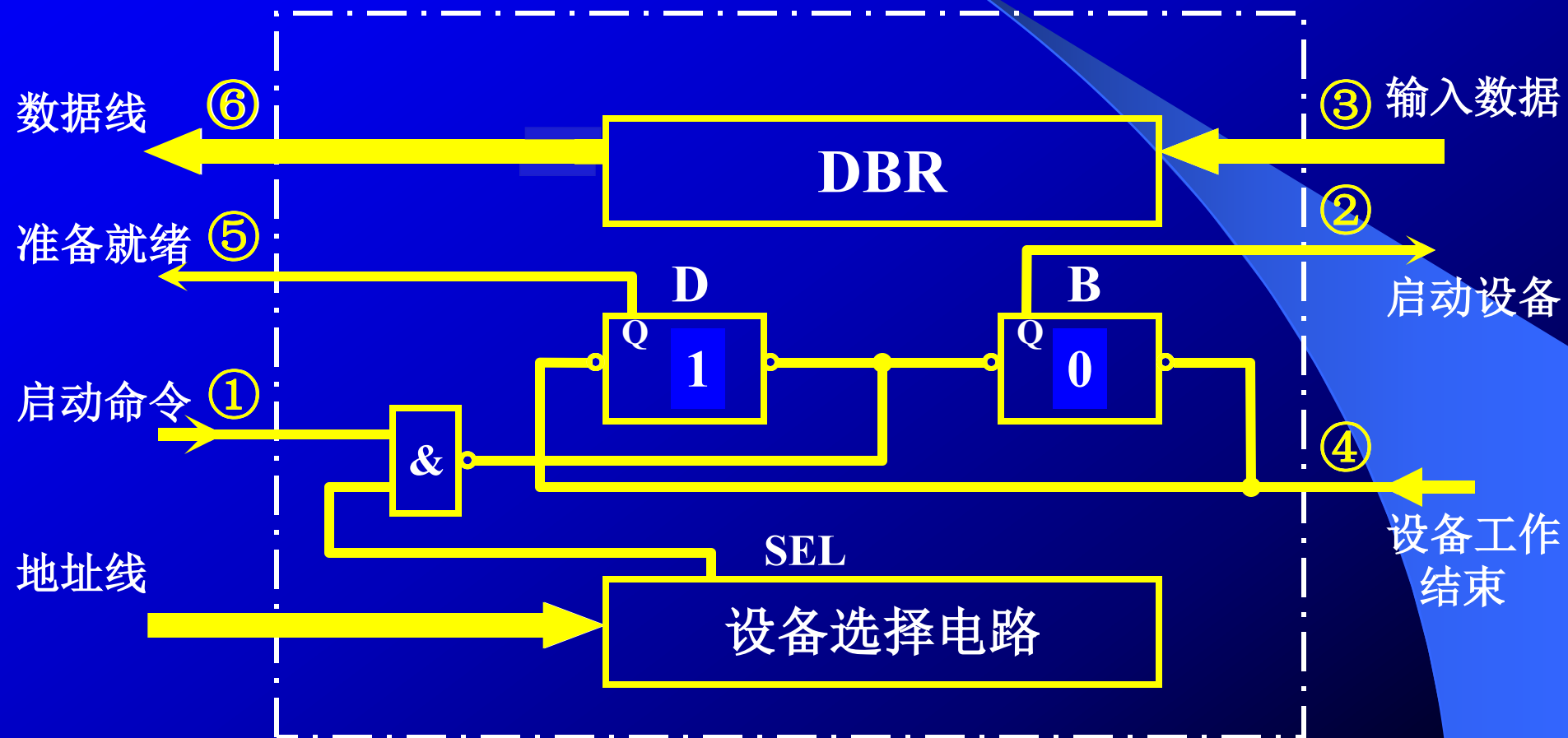
保存
寄存器内容



二、程序查询方式的接口电路

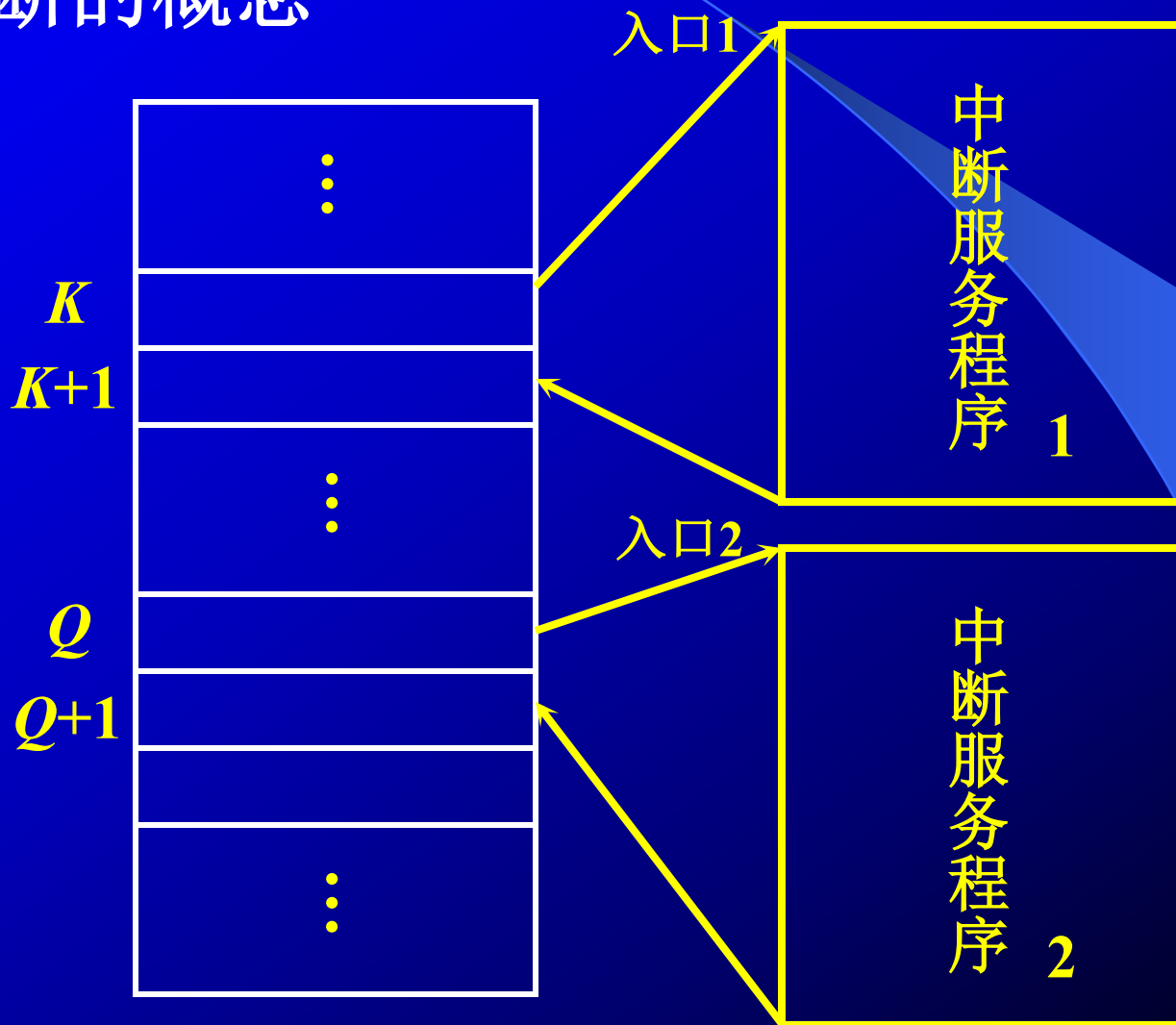
5.4

以输入为例



5.5 程序中中断方式

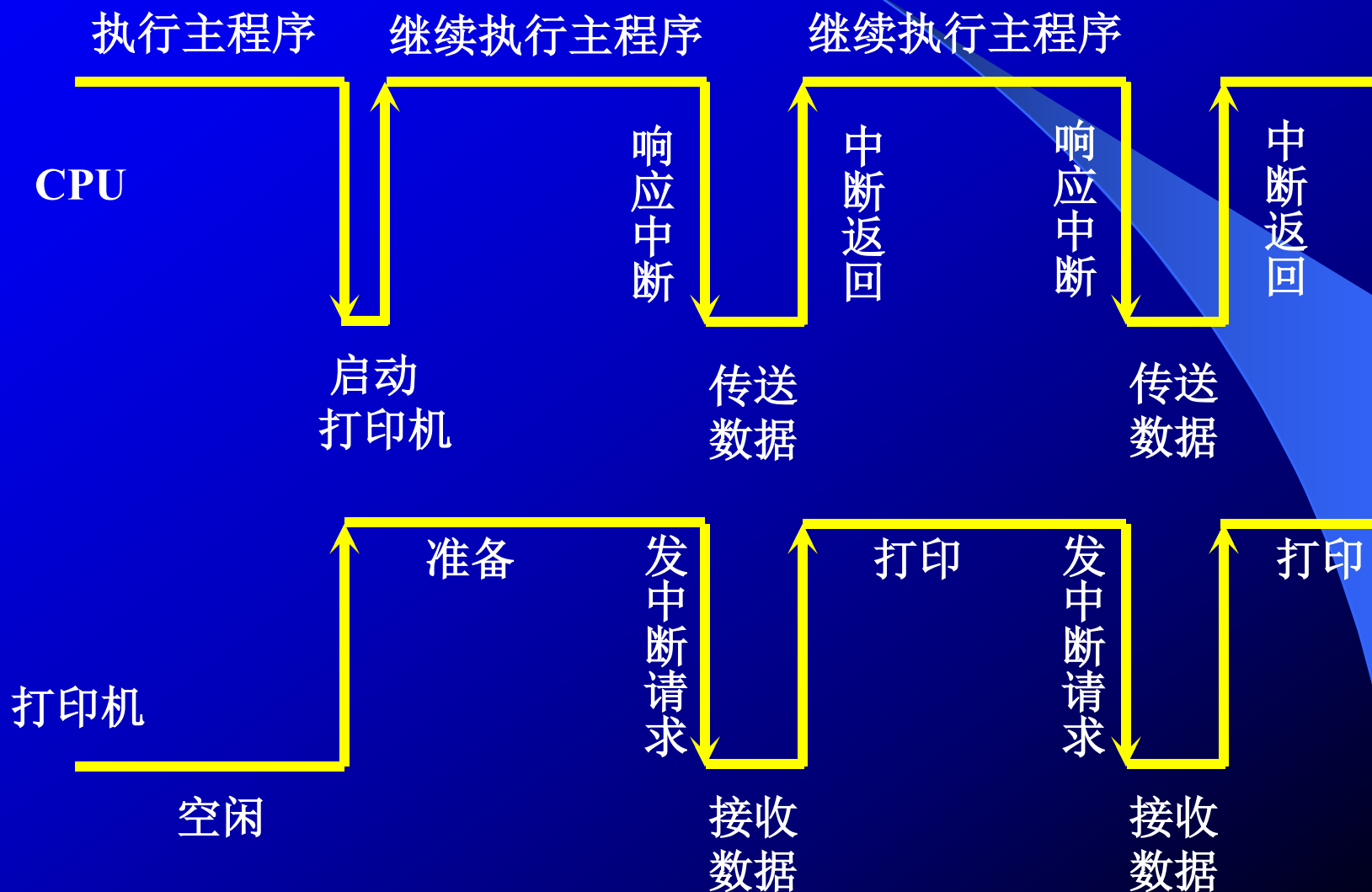
一、中断的概念



二、I/O 中断的产生

5.5

以打印机为例 CPU 与打印机并行工作

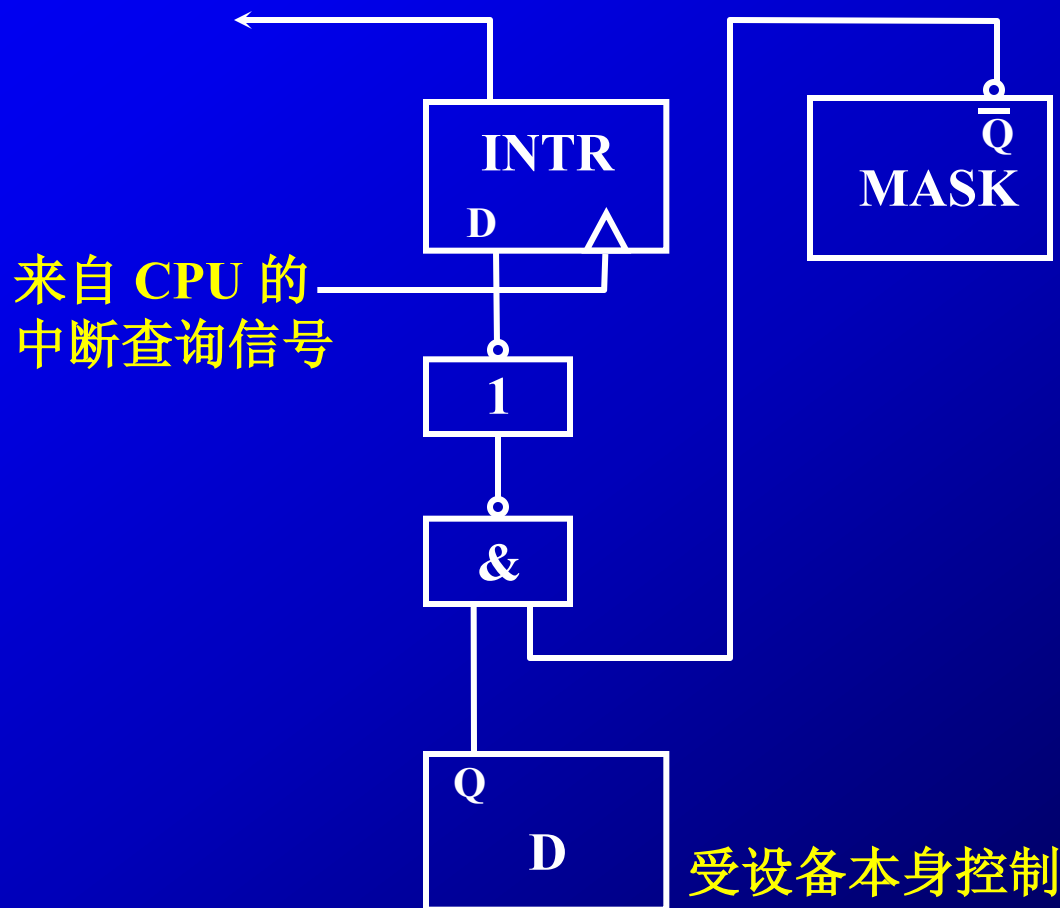


三、程序中中断方式的接口电路

5.5

1. 配置中断请求触发器和中断屏蔽触发器

中断请求



INTR

中断请求触发器

INTR = 1 有请求

MASK

中断屏蔽触发器

MASK = 1 被屏蔽

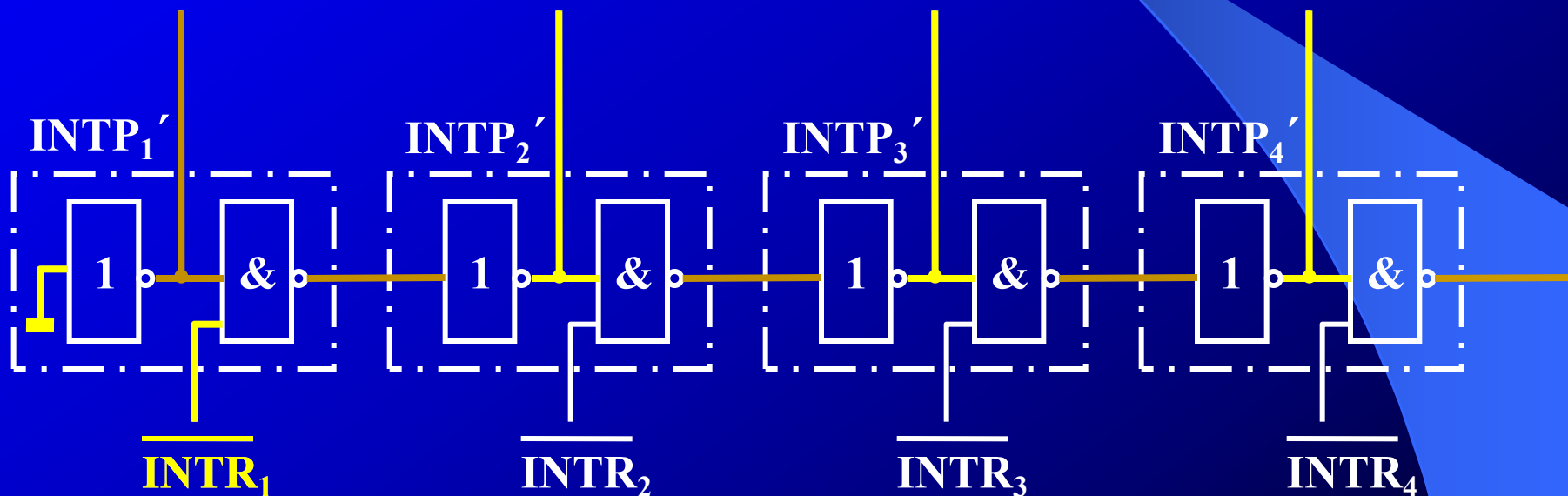
D 完成触发器

受设备本身控制

2. 排队器

5.5

排队 { 硬件 在 CPU 内或在接口电路中 (链式排队器)
软件 详见第八章



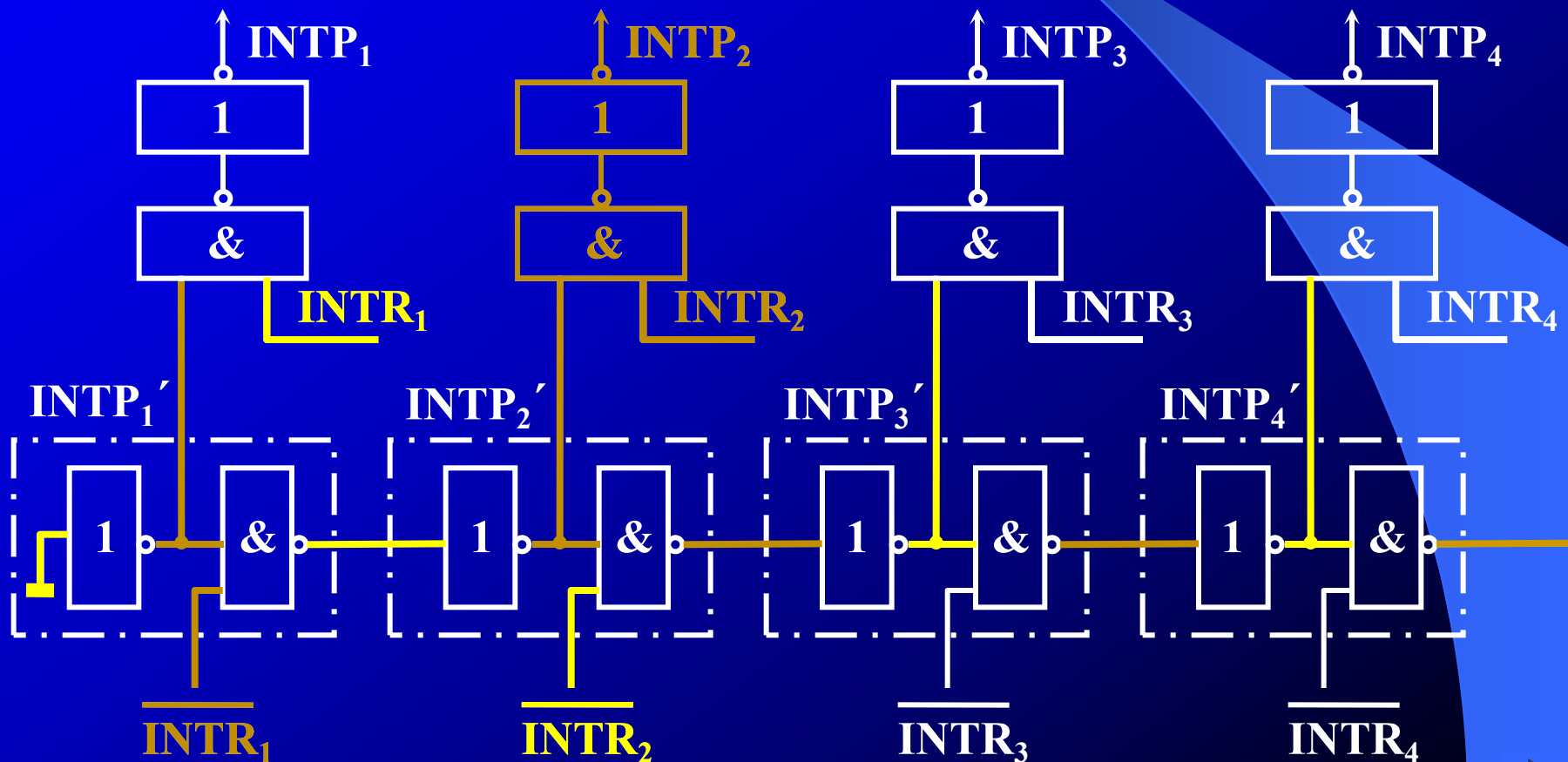
设备 1[#]、2[#]、3[#]、4[#] 优先级按 降序排列

$INTR_i = 1$ 有请求 即 $\overline{INTR}_i = 0$

2. 排队器

5.5

排队 { 硬件 在 CPU 内或在接口电路中（链式排队器）
软件 详见第八章



3. 中断向量地址形成部件

5.5

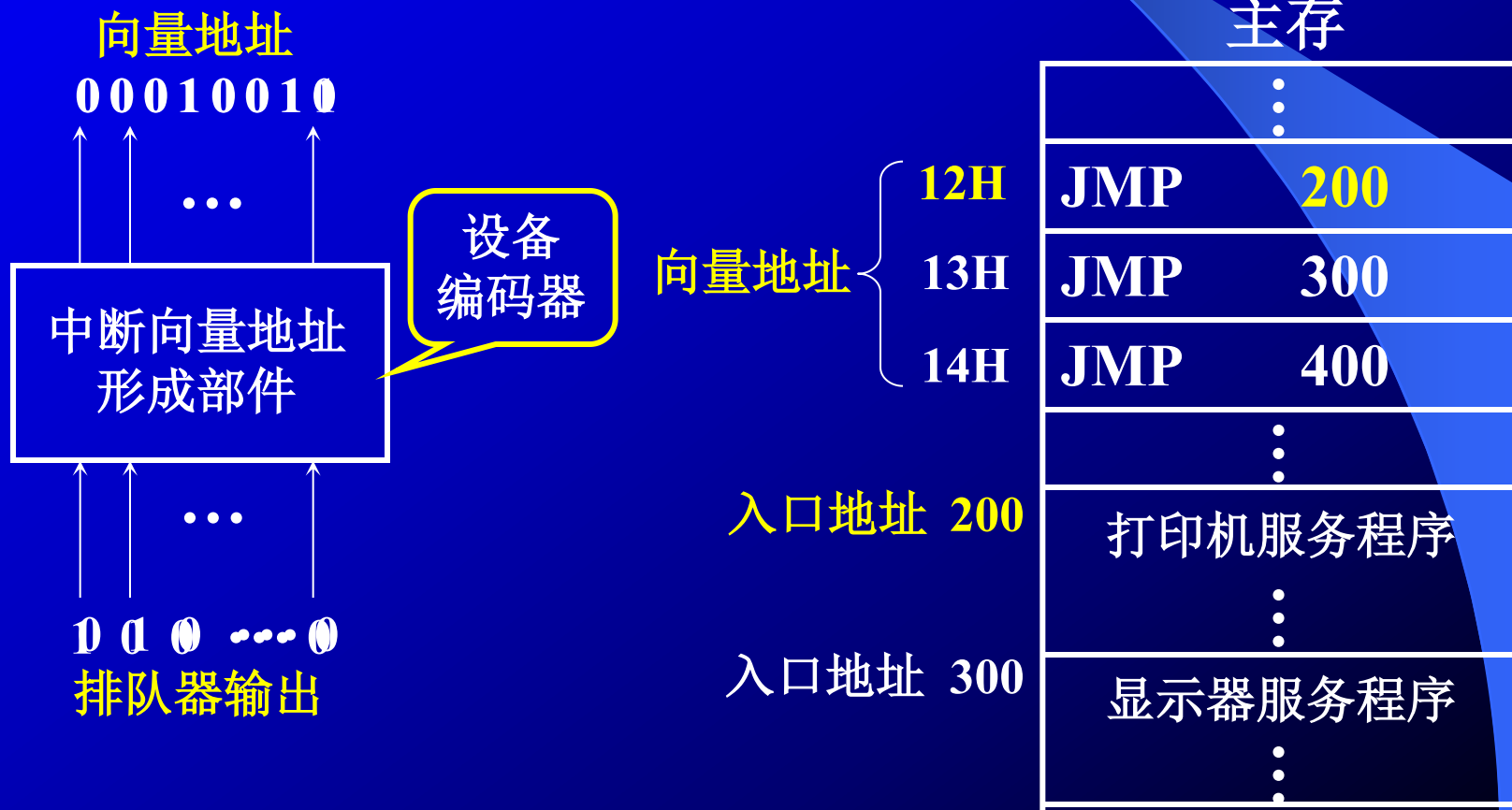
入口地址 { 由软件产生
 硬件向量法

详见第八章

由 **硬件** 产生 **向量地址**

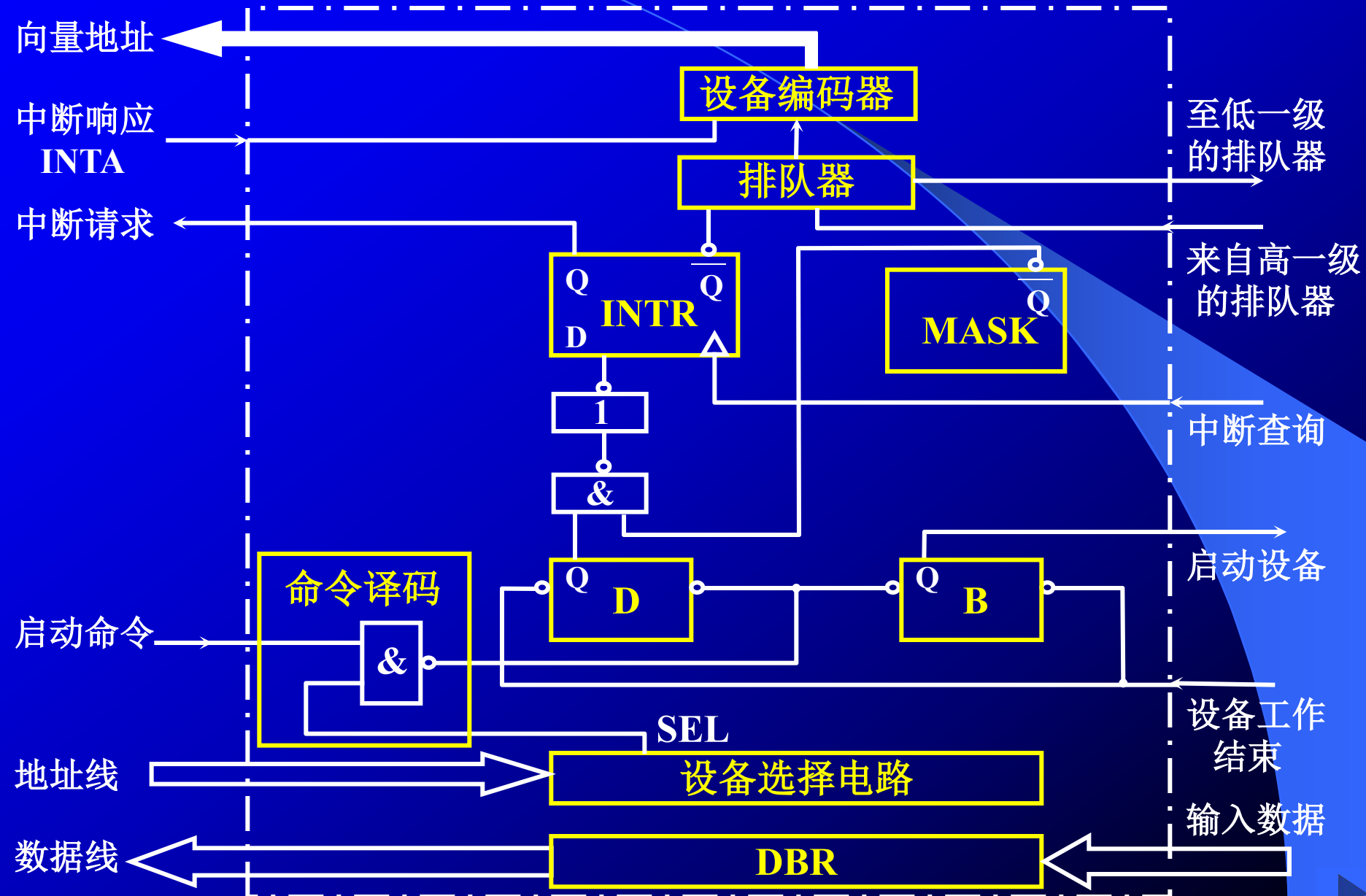
再由 **向量地址** 找到 **入口地址**

主存



4. 程序中断方式接口电路的基本组成

5.5



四、I/O 中断处理过程

5.5

1. CPU 响应中断的条件和时间

(1) 条件

允许中断触发器 **EINT = 1**

用 **开中断** 指令将 **EINT** 置 “**1**”

用 **关中断** 指令将 **EINT** 置 “**0**” 或硬件 **自动复**

(2) 时间

当 **D = 1**（随机）且 **MASK = 0** 时

在每条指令执行阶段的结束前

CPU 发 **中断查询信号**（将 **INTR** 置 “**1**”）



5.5



五、中断服务程序流程

5.5

1. 中断服务程序的流程

(1) 保护现场

{	程序断点的保护	中断隐指令完成
	寄存器内容的保护	进栈指令

(2) 中断服务

对不同的 I/O 设备具有不同内容的设备服务

(3) 恢复现场

出栈指令

(4) 中断返回

中断返回指令

2. 单重中断和多重中断

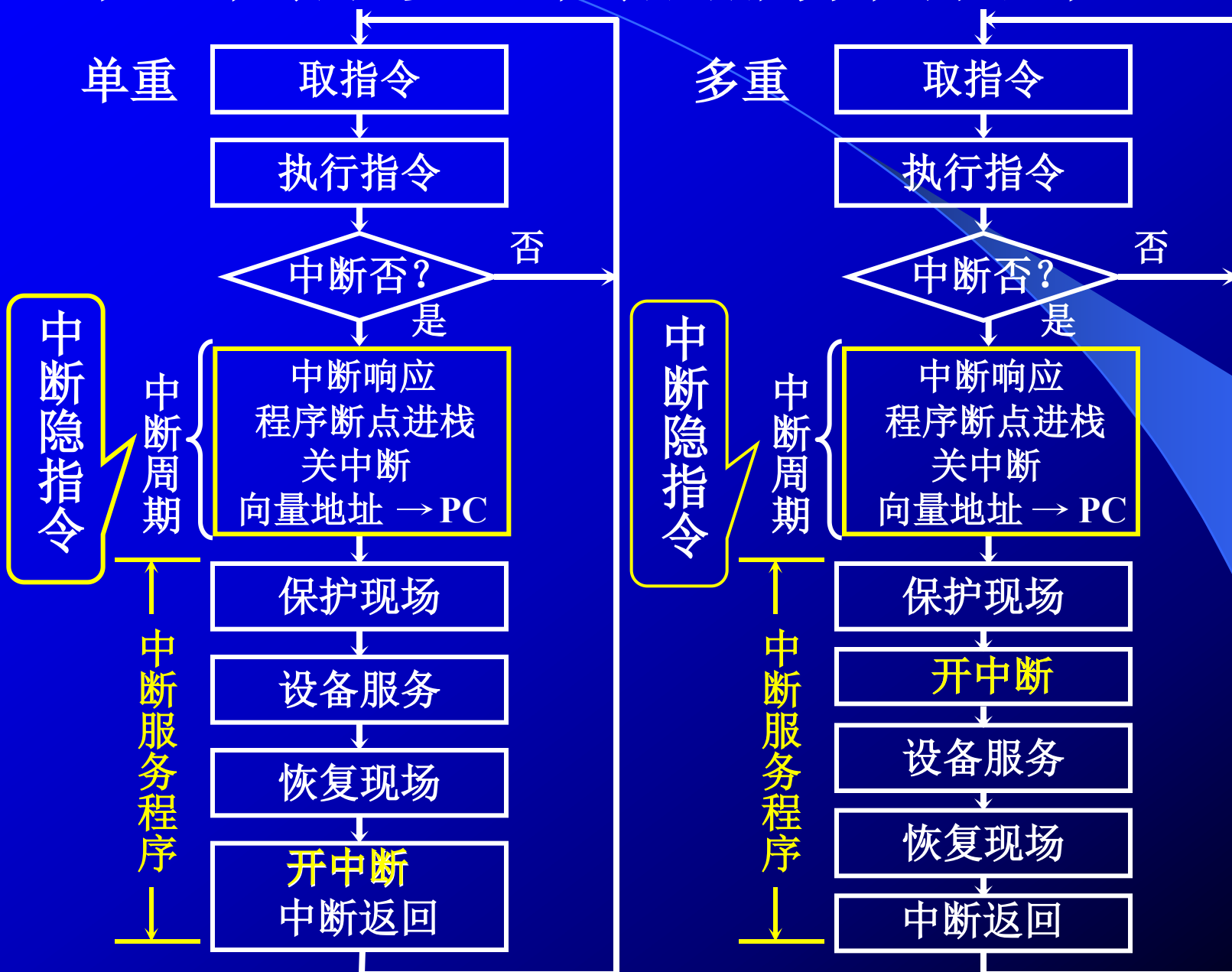
单重 中断 **不允许**中断 现行的 **中断服务程序**

多重 中断 **允许**级别更高 的中断源
中断 现行的 **中断服务程序**



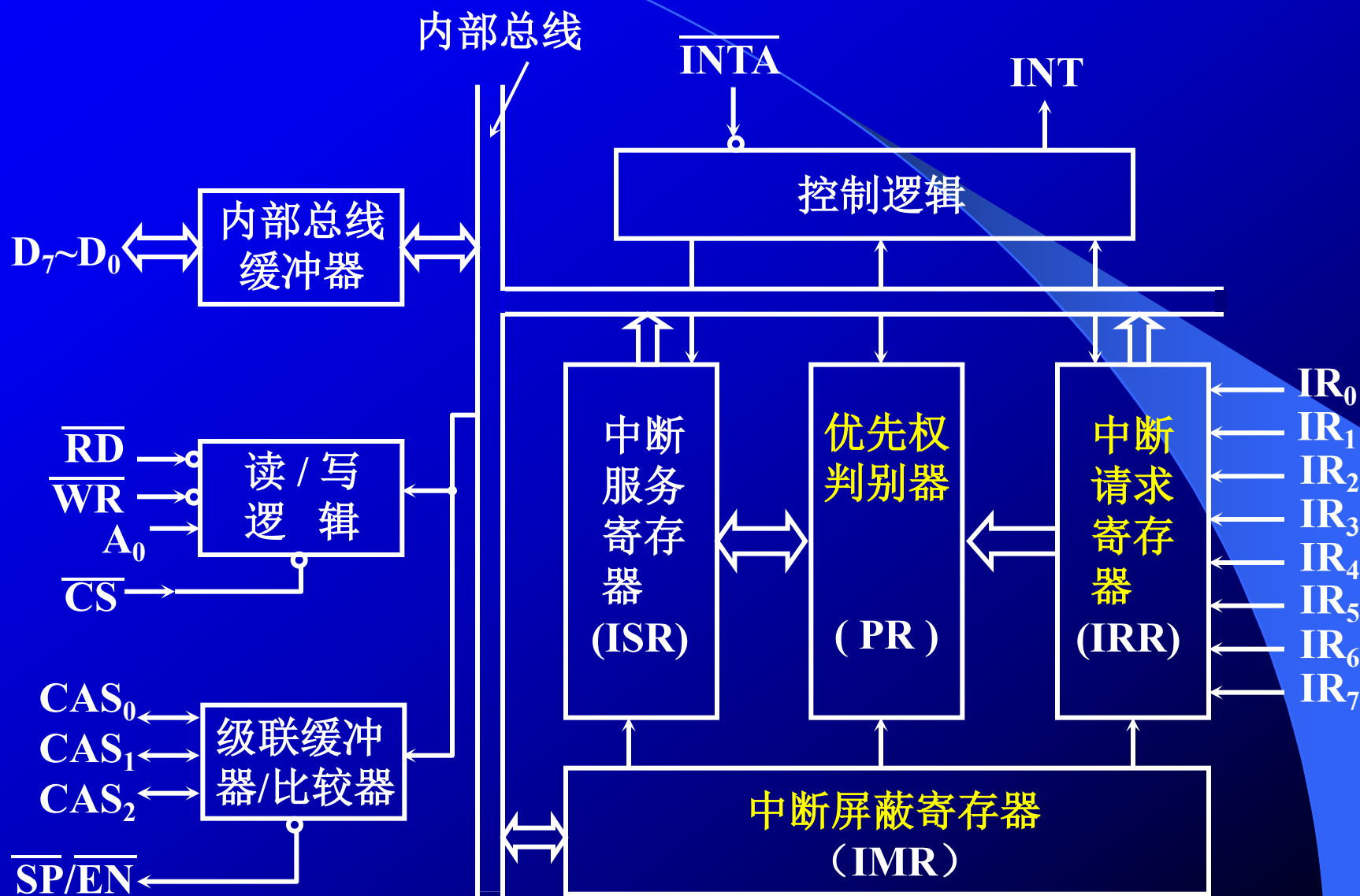
3. 单重中断和多重中断的服务程序流程

5.5

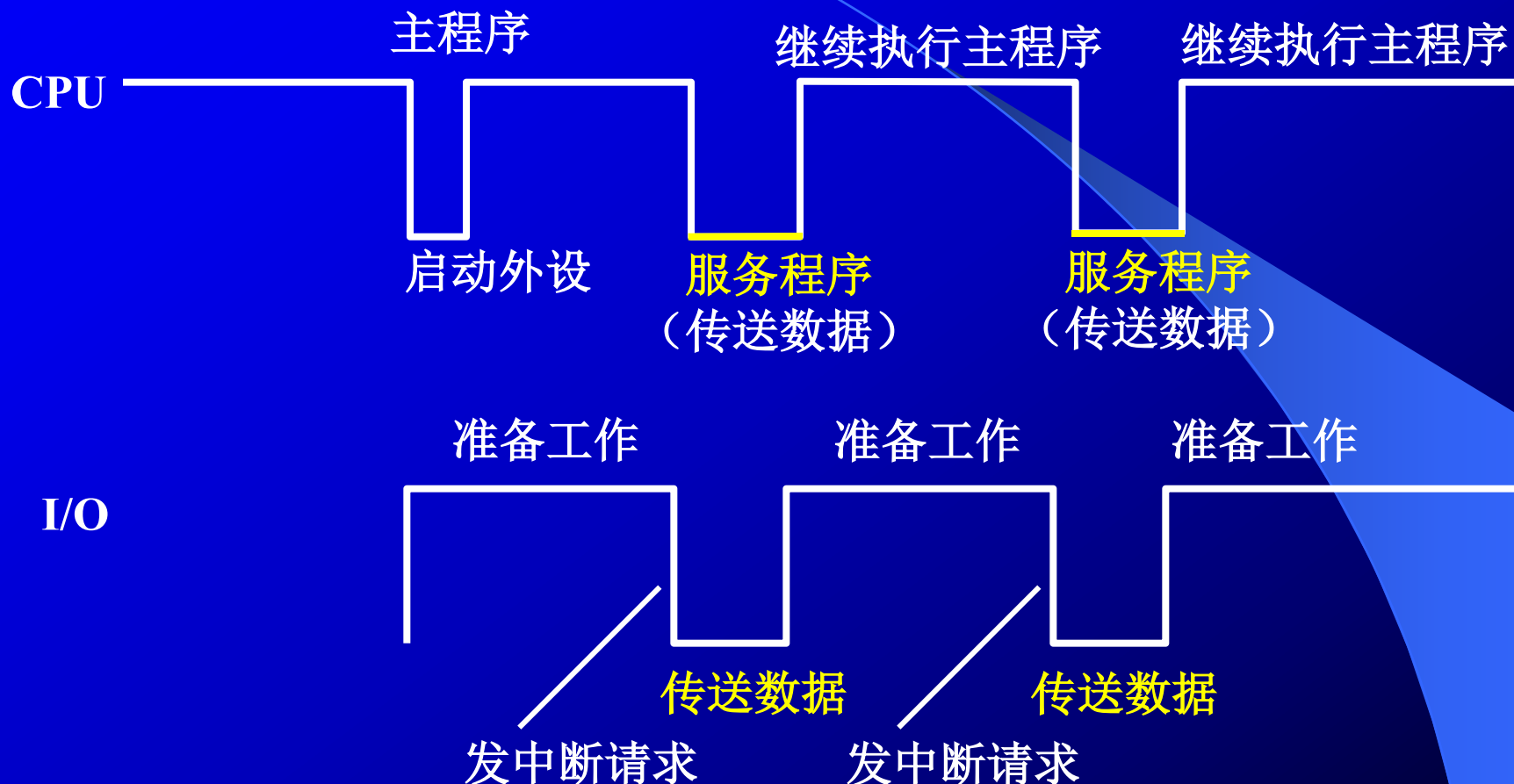


程序中中断接口芯片 8259A 的内部结构

5.5



主程序和服务程序抢占 CPU 示意图 5.5



宏观上 CPU 和 I/O 并行工作

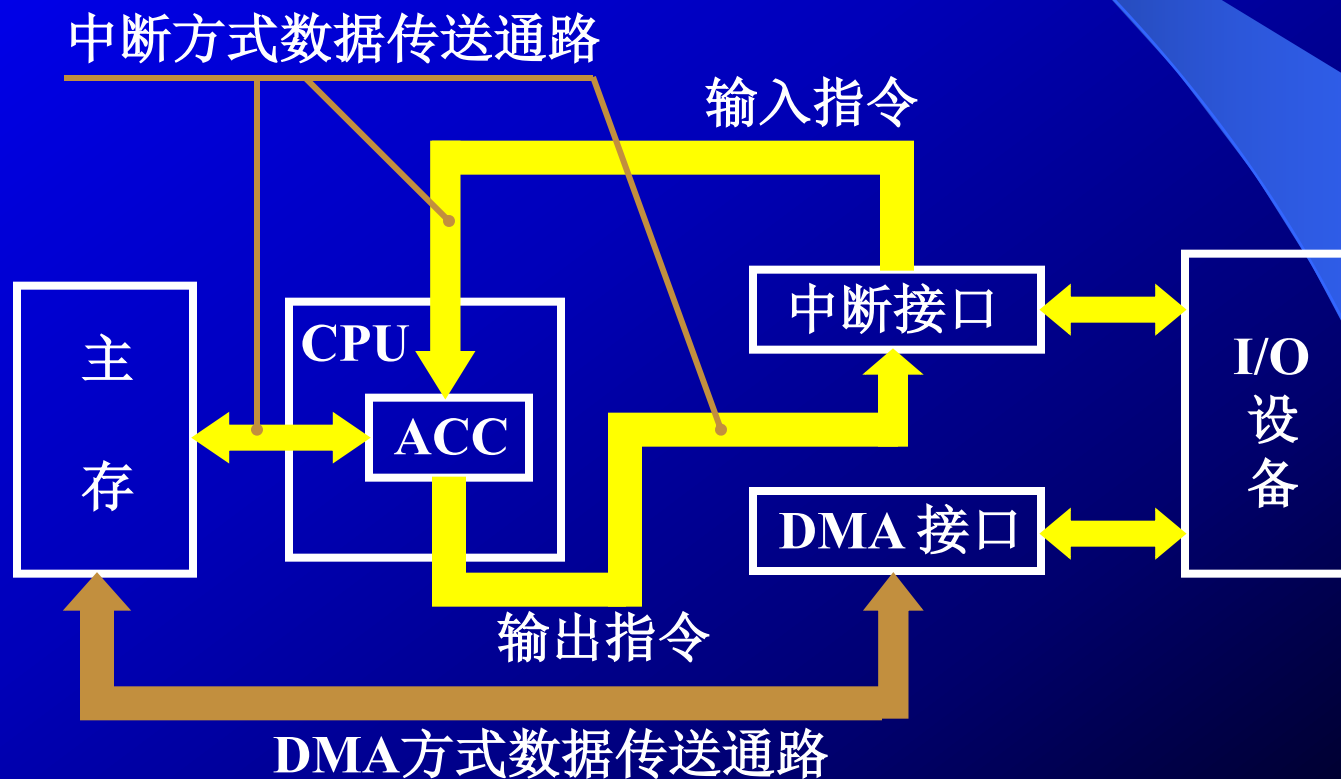
微观上 CPU 中断现行政程序为 I/O 服务



5.6 DMA 方式

一、DMA 方式的特点

1. DMA 和程序中中断两种方式的数据通路



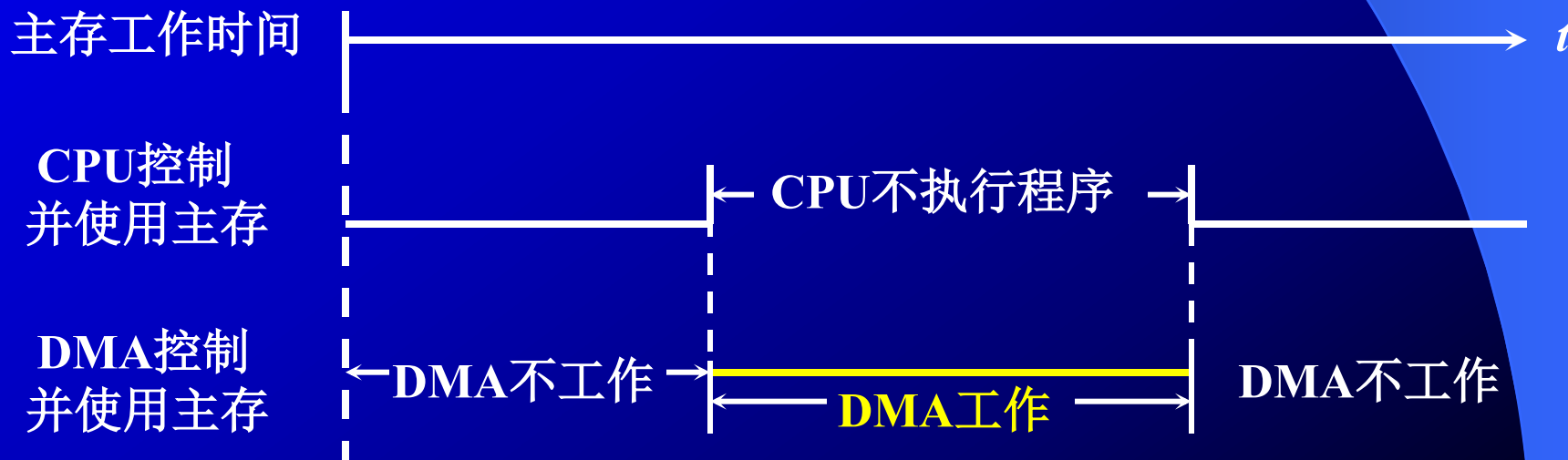
2. DMA 与主存交换数据的三种方式 5.6

(1) 停止 CPU 访问主存

控制简单

CPU 处于不工作状态或保持状态

未充分发挥 CPU 对主存的利用率



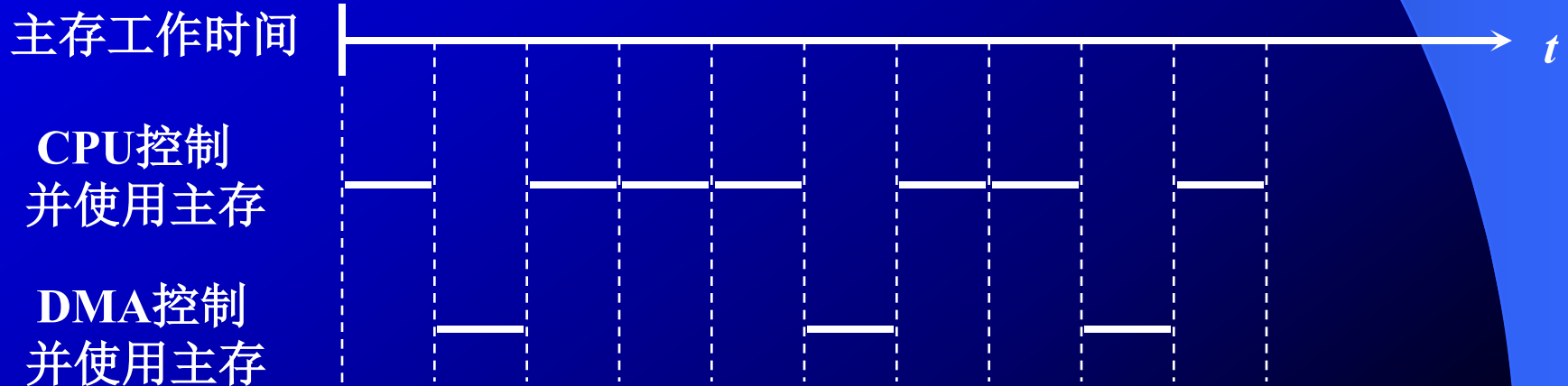
(2) 周期挪用 (或周期窃取)

5.6

DMA 访问主存有三种可能

- CPU 此时不访存
- CPU 正在访存
- CPU 与 DMA 同时请求访存

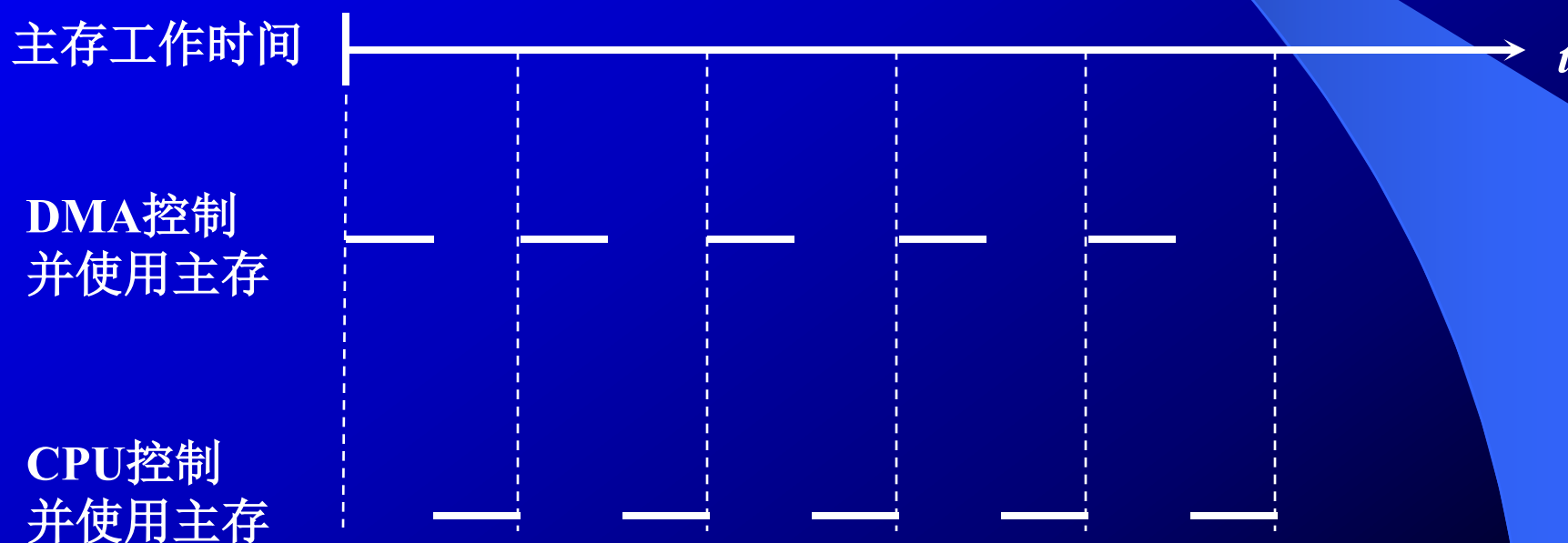
此时 CPU 将总线控制权让给 DMA



(3) DMA 与 CPU 交替访问

CPU 工作周期 $\begin{cases} C_1 \text{ 专供 DMA 访存} \\ C_2 \text{ 专供 CPU 访存} \end{cases}$

所有指令执行过程中的一个基准时间



不需要 申请建立和归还 总线的使用权

二、DMA 接口的功能和组成

5.6

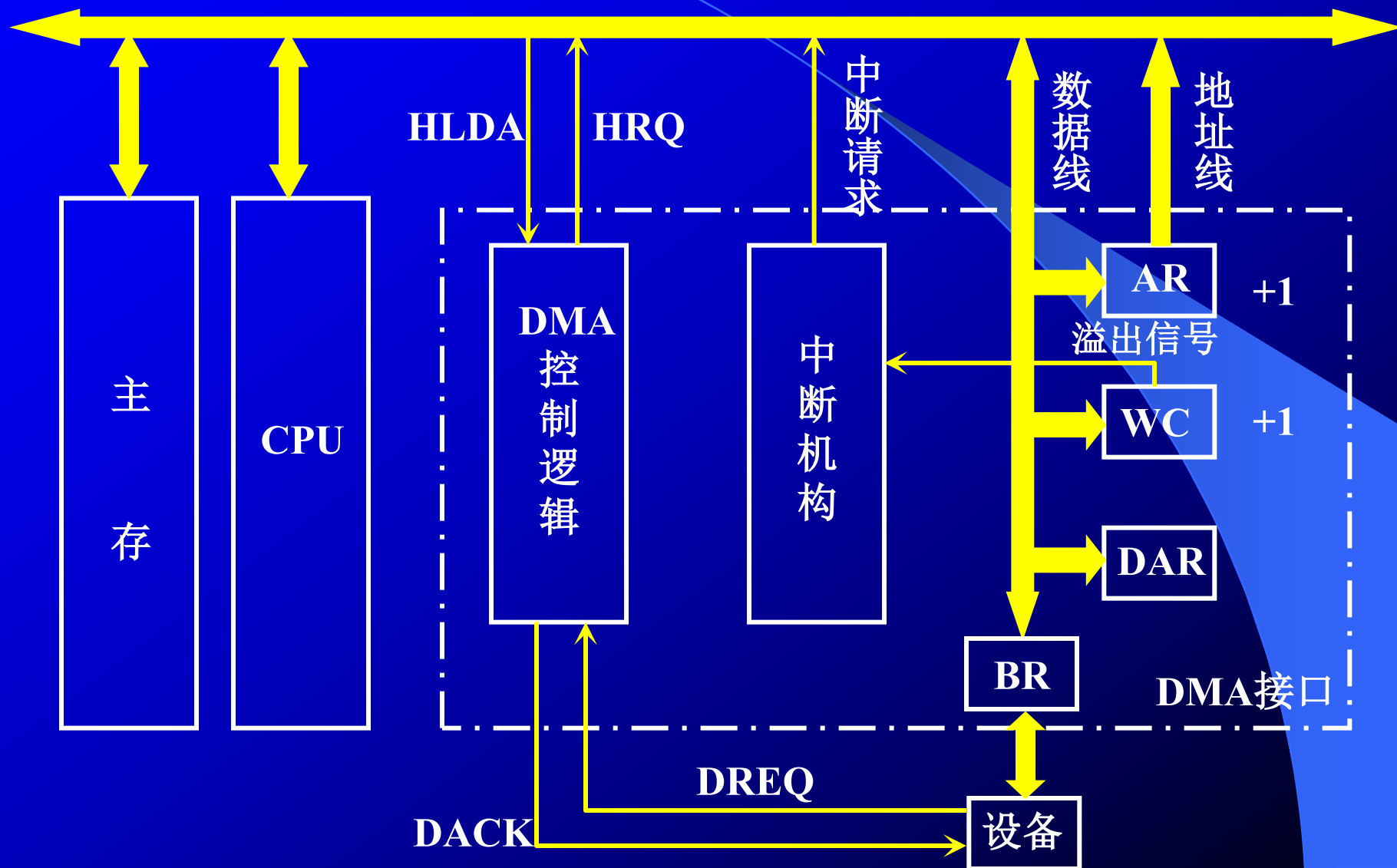
1. DMA 接口功能

- (1) 向 CPU 申请 DMA 传送
- (2) 处理总线控制权的转交
- (3) 管理系统总线、控制数据传送
- (4) 确定数据传送的首地址和长度
修正 传送过程中的数据 地址 和 长度
- (5) DMA 传送结束时，给出操作完成信号



2. DMA 接口组成

5.6



三、DMA 的工作过程

5.6

1. DMA 传送过程

预处理、数据传送、后处理

(1) 预处理

通过几条输入输出指令预置如下信息

- 通知 DMA 控制逻辑传送方向（入/出）
- 设备地址——DMA 的 DAR
- 主存地址——DMA 的 AR
- 传送字数——DMA 的 WC



(2) DMA 传送过程示意

5.6

CPU

预处理:

主存起始地址 → DMA
设备地址 → DMA
传送数据个数 → DMA
启动设备

数据传送:

继续执行主程序
同时完成一批数据传送

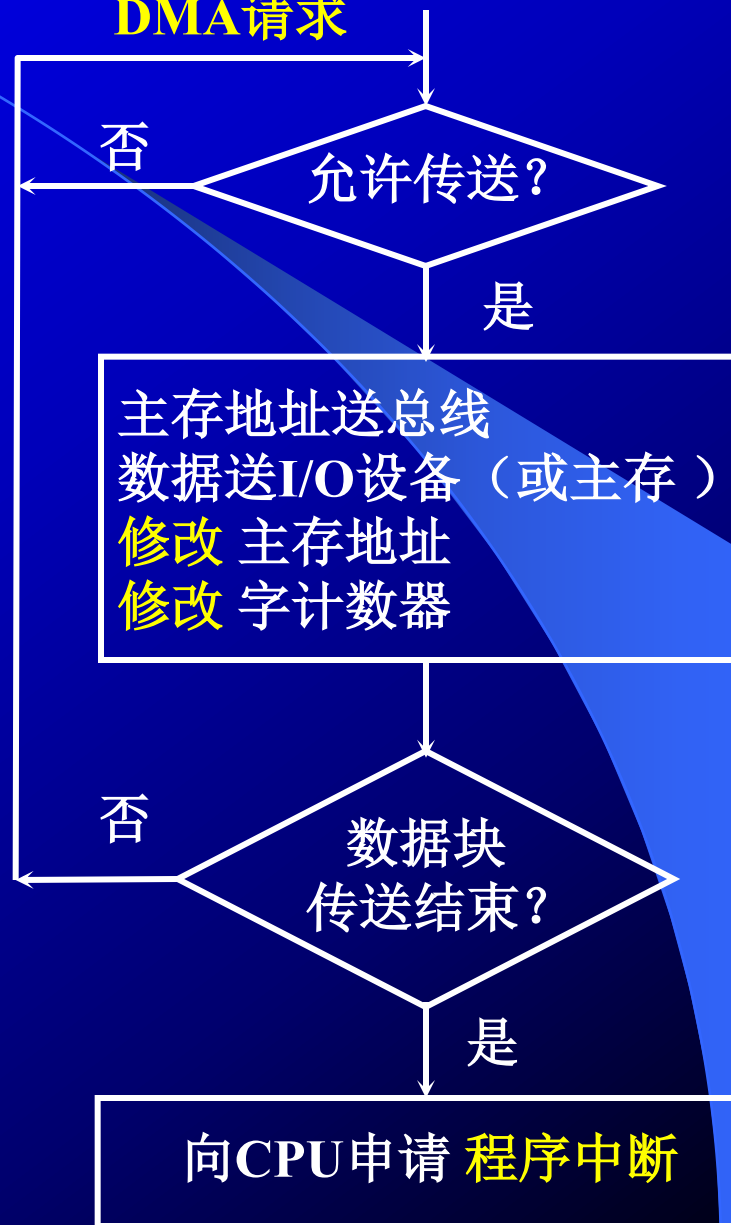
后处理:

中断服务程序
做 DMA 结束处理

继续执行主程序

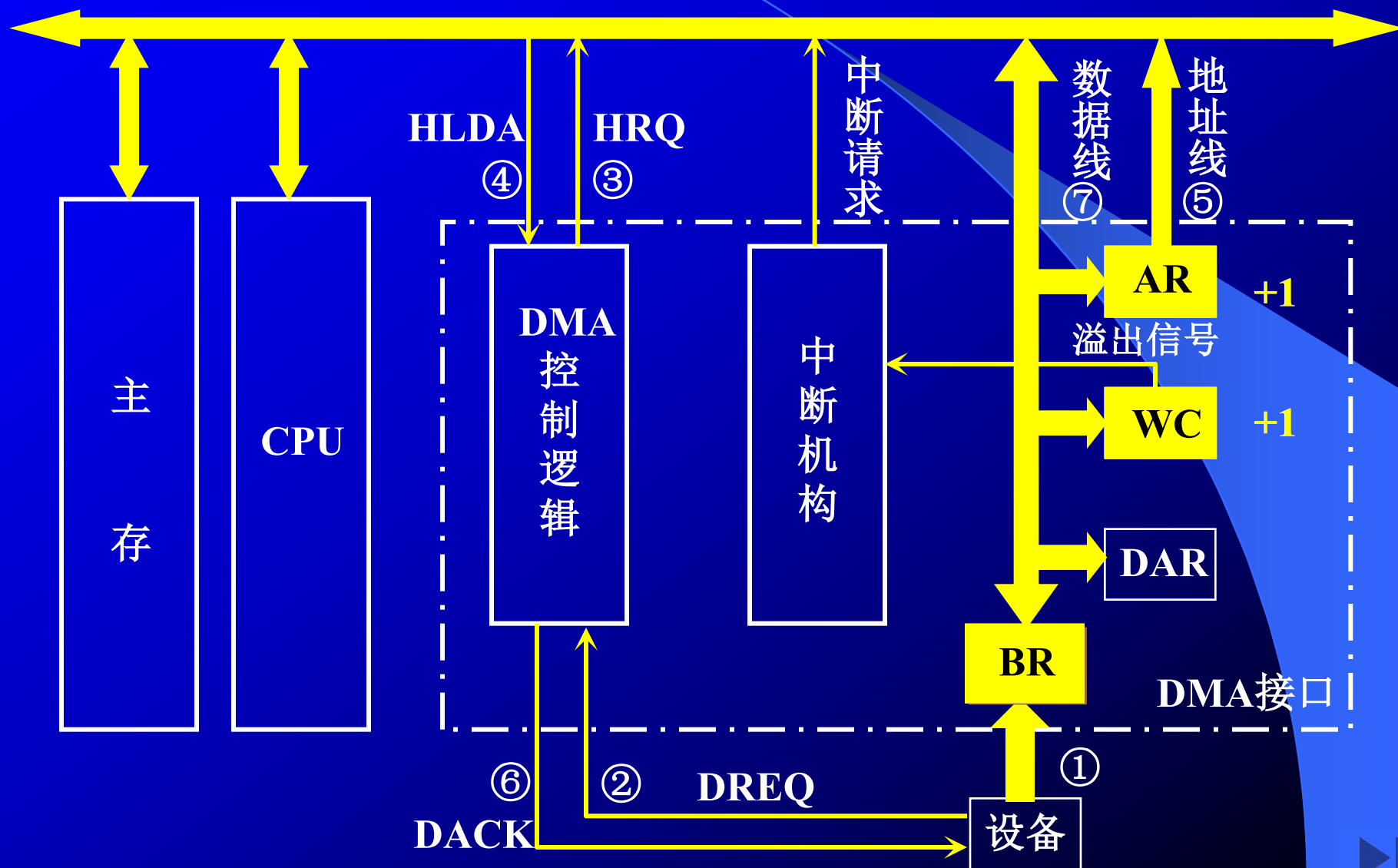
数据传送

DMA请求



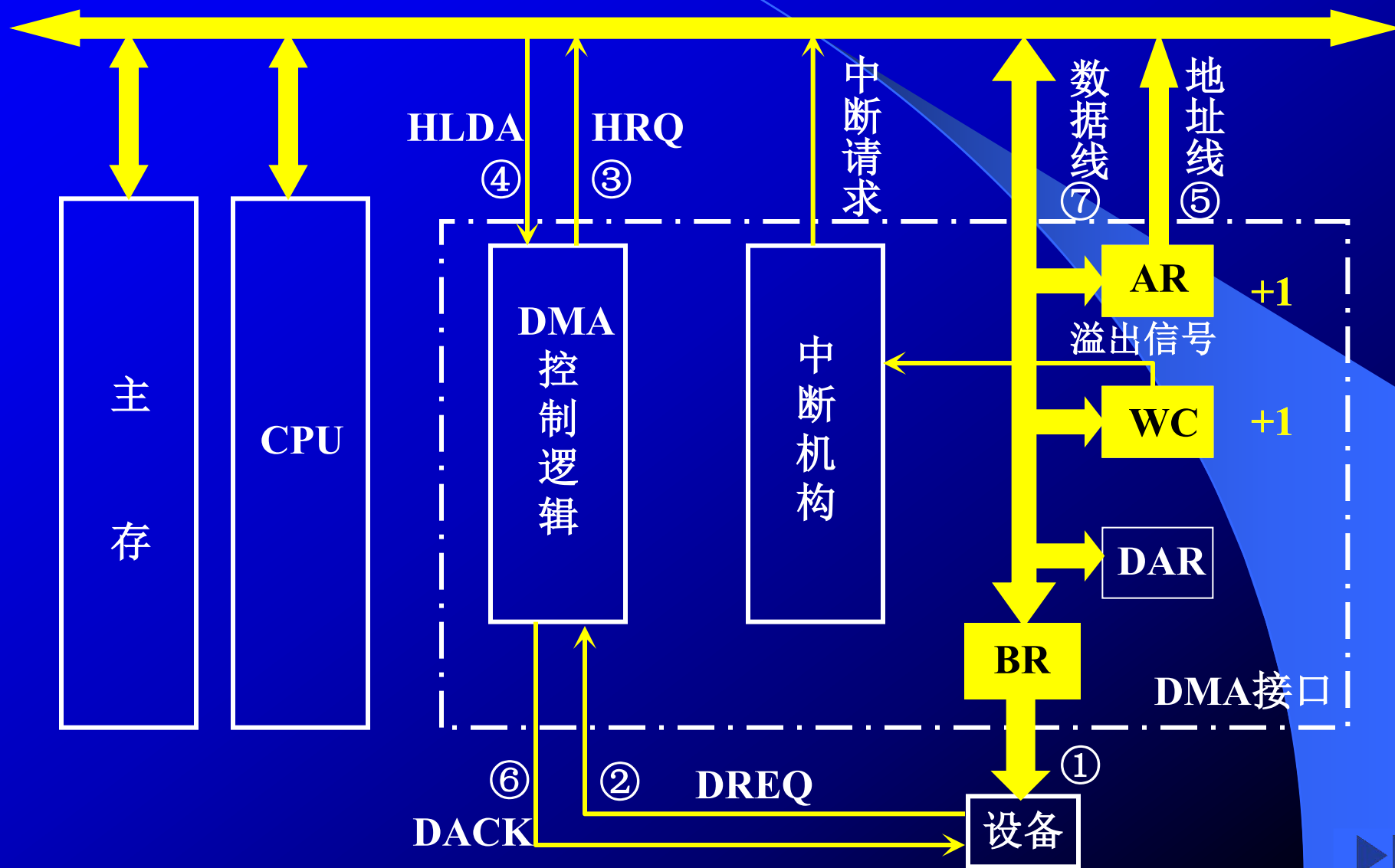
(3) 数据传送过程（输入）

5.6



(4) 数据传送过程（输出）

5.6



(5) 后处理

校验送入主存的数是否正确

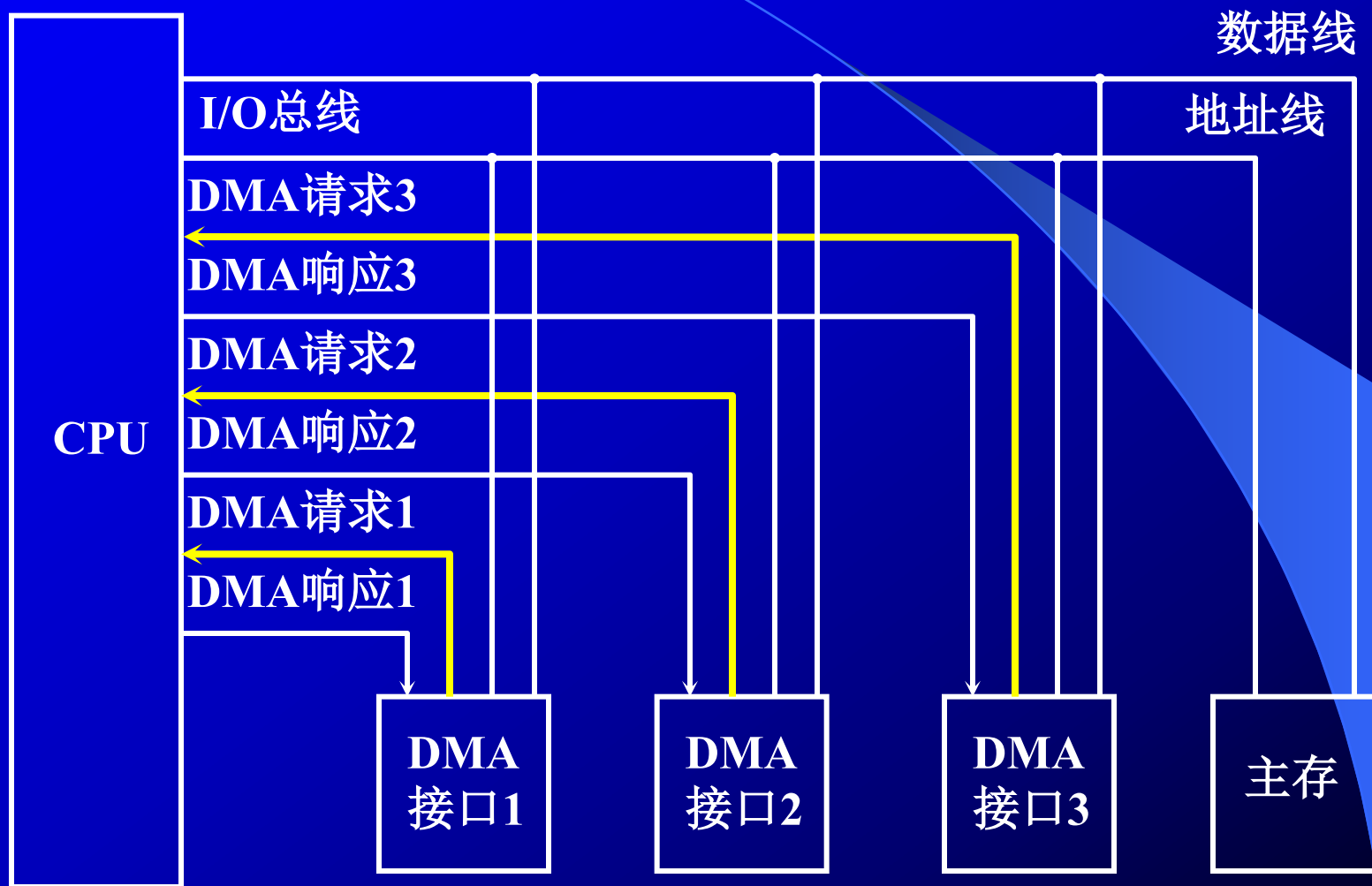
是否继续用 DMA

测试传送过程是否正确，错则转诊断程序

由中断服务程序完成

(2) 独立的 DMA 请求

5.6



3. DMA 方式与程序中断方式的比较 5.6

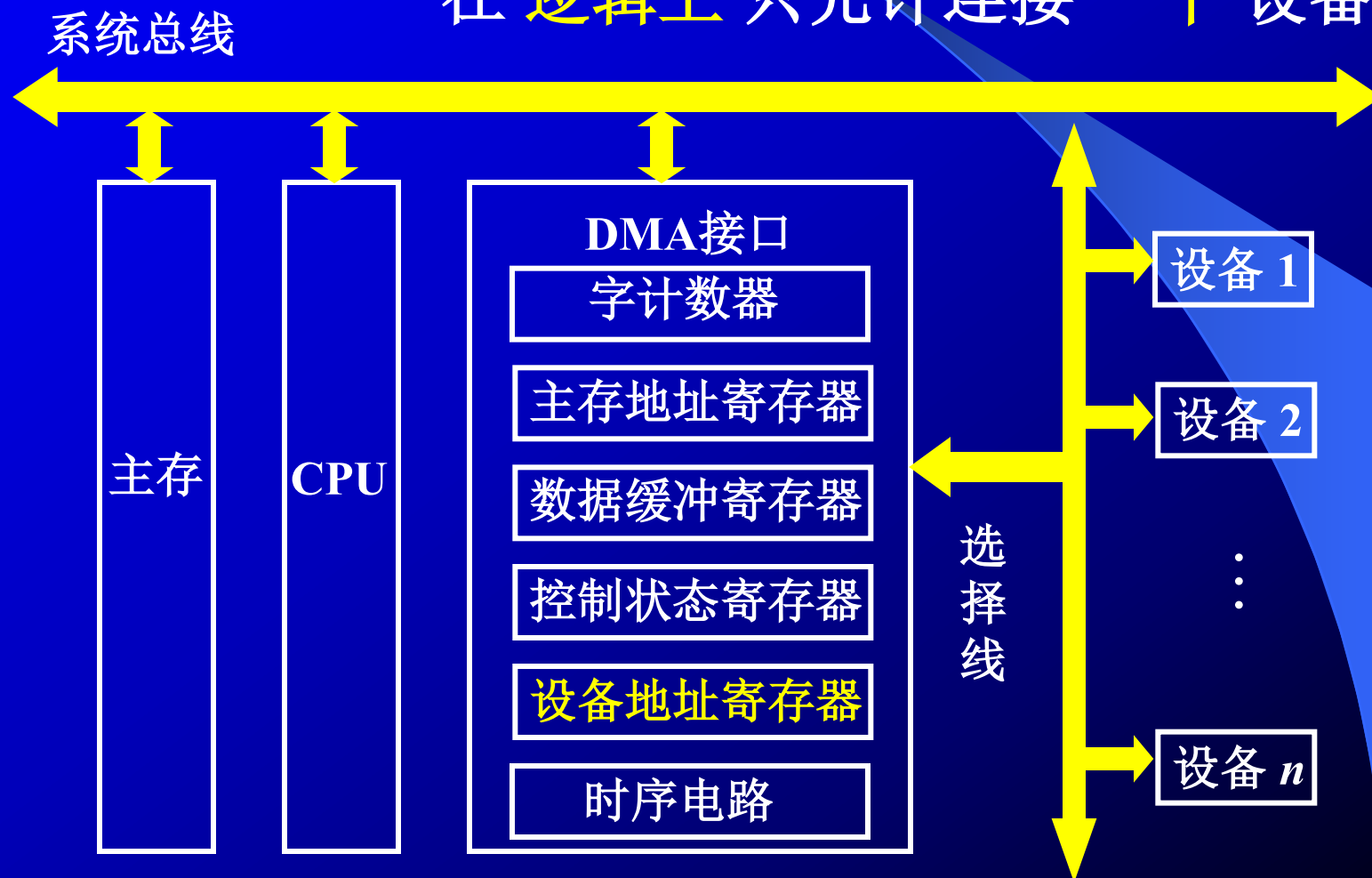
	中断方式	DMA 方式
(1) 数据传送	程序	硬件
(2) 响应时间	指令执行结束	存取周期结束
(3) 处理异常情况	能	不能
(4) 中断请求	传送数据	后处理
(5) 优先级	低	高



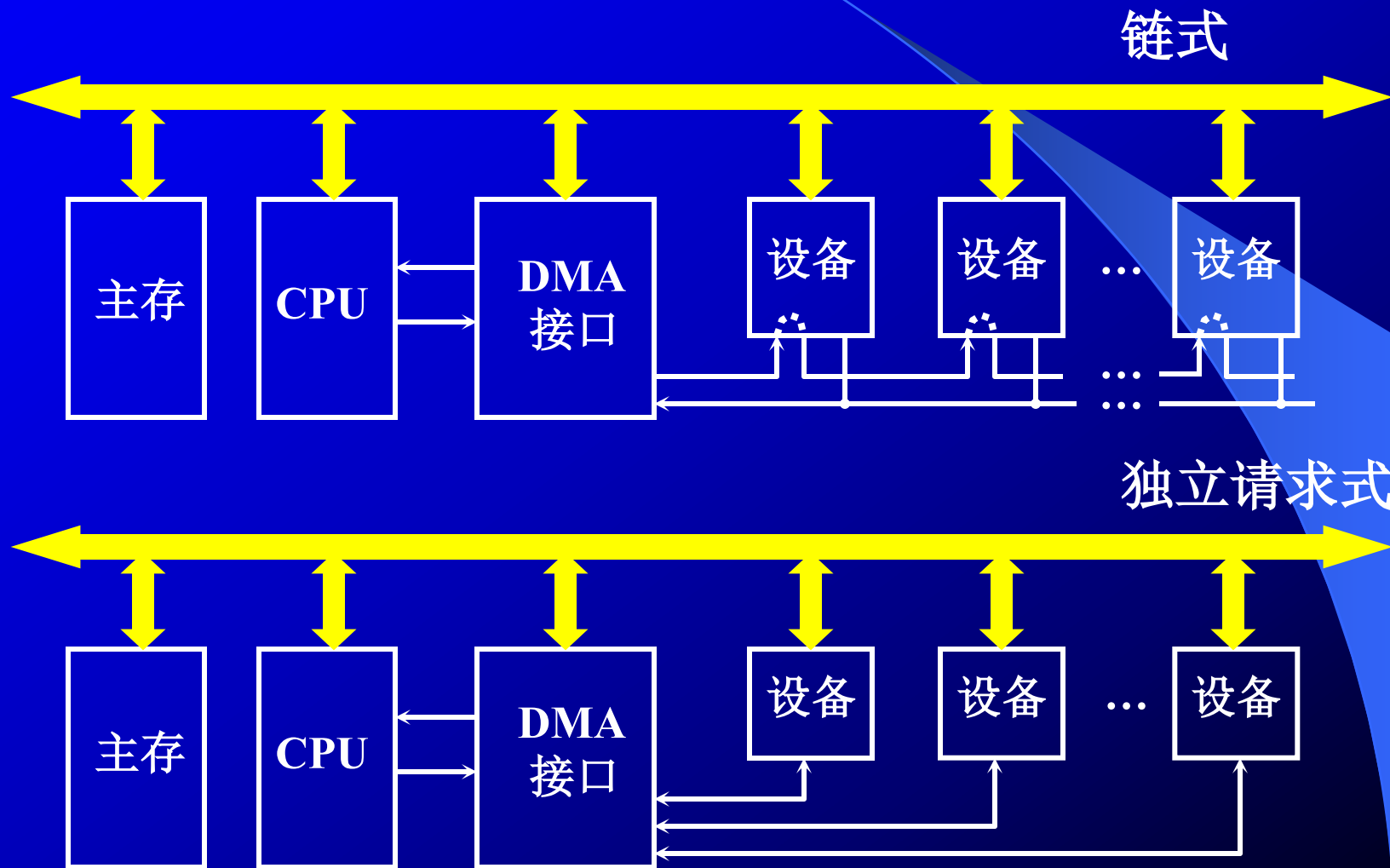
四、DMA 接口的类型

1. 选择型

在物理上连接多个设备
在逻辑上只允许连接一个设备

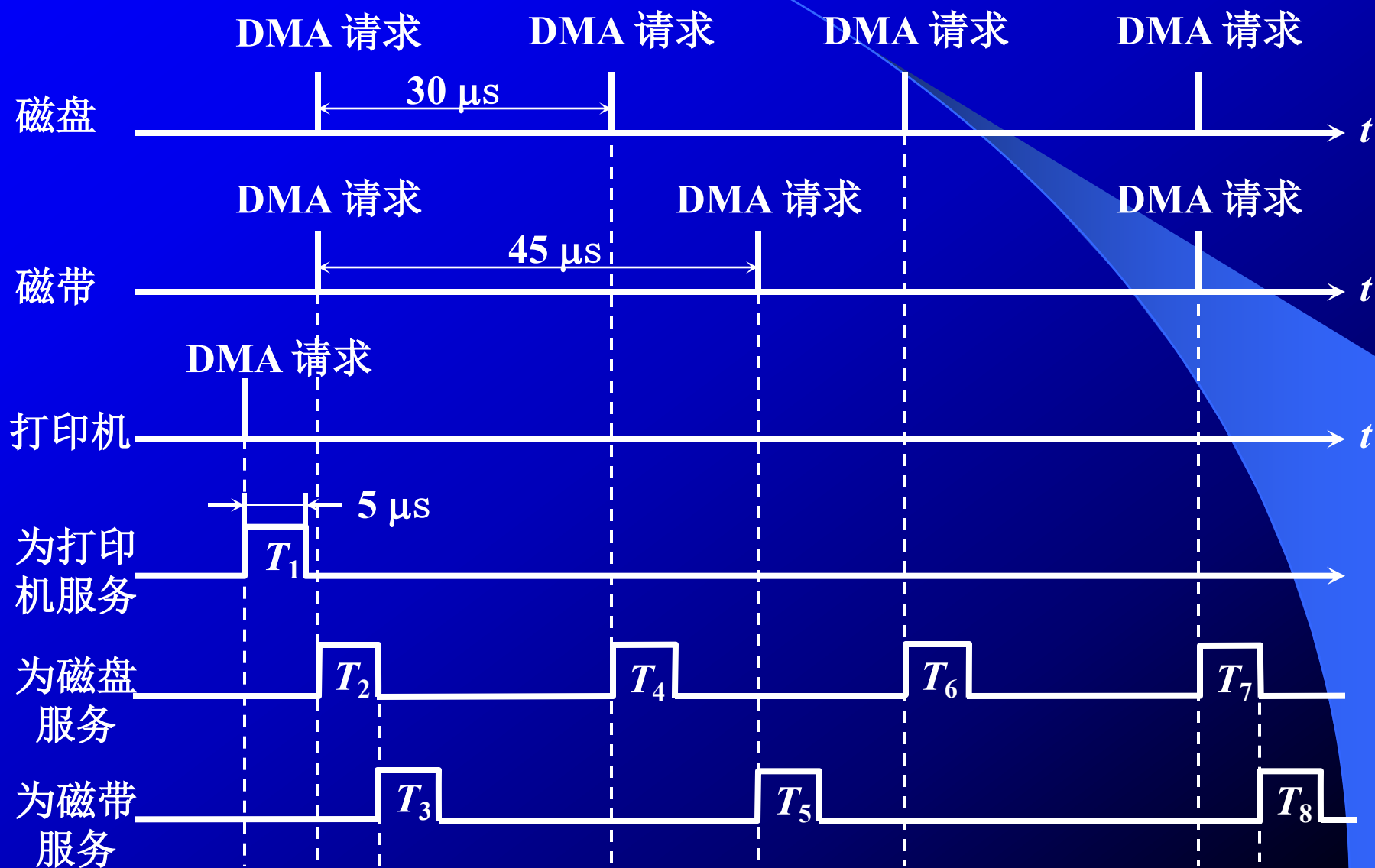


2. 多路型 在物理上连接多个设备 在逻辑上允许连接多个设备同时工作



3. 多路型 DMA 接口的工作原理

5.6



第 6 章 计算机的运算方法

6.1 无符号数和有符号数

6.2 数的定点表示和浮点表示

6.3 定点运算

6.4 浮点四则运算

6.5 算术逻辑单元



6.1 无符号数和有符号数

一、无符号数

寄存器的位数

反映无符号数的表示范围



8 位

0 ~ 255



16 位

0 ~ 65535

二、有符号数

6.1

1. 机器数与真值

真值

带符号的数

$+ 0.1011$

$- 0.1011$

$+ 1100$

$- 1100$

机器数

符号数字化的数

0	1011
---	------

↑ 小数点的位置

1	1011
---	------

↑ 小数点的位置

0	1100
---	------

↑ 小数点的位置

1	1100
---	------

↑ 小数点的位置

2. 原码表示法

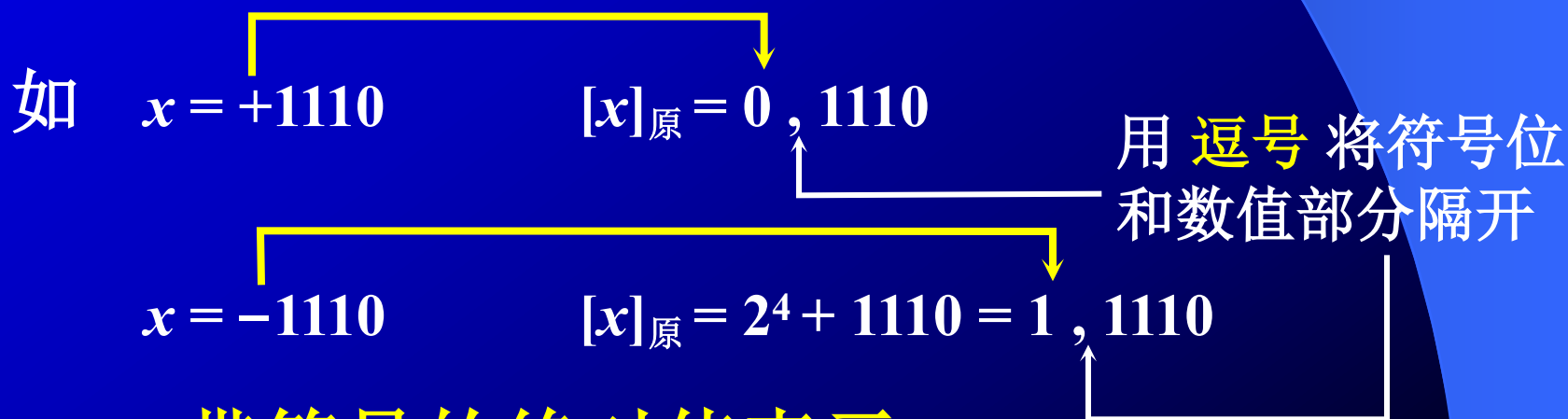
6.1

(1) 定义

整数

$$[x]_{\text{原}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases}$$

x 为真值 n 为整数的位数



带符号的绝对值表示

小数

6.1

$$[x]_{\text{原}} = \begin{cases} x & 1 > x \geq 0 \\ 1 - x & 0 \geq x > -1 \end{cases}$$

x 为真值

如 $x = +0.1101$ $[x]_{\text{原}} = 0.1101$ 用 **小数点** 将符号位和数值部分隔开

$x = -0.1101$ $[x]_{\text{原}} = 1 - (-0.1101) = 1.1101$

$x = +0.1000000$ $[x]_{\text{原}} = 0.1000000$ 用 **小数点** 将符号位和数值部分隔开

$x = -0.1000000$ $[x]_{\text{原}} = 1 - (-0.1000000) = 1.1000000$

(2) 举例

6.1

例 6.1 已知 $[x]_{\text{原}} = 1.0011$ 求 $x - 0.0011$



解: 由定义得

$$x = 1 - [x]_{\text{原}} = 1 - 1.0011 = -0.0011$$

例 6.2 已知 $[x]_{\text{原}} = 1,1100$ 求 $x - 1100$



解: 由定义得

$$x = 2^4 - [x]_{\text{原}} = 10000 - 1,1100 = -1100$$

例 6.3 已知 $[x]_{\text{原}} = 0.1101$ 求 x

解: 根据定义 $\because [x]_{\text{原}} = 0.1101$

$$\therefore x = +0.1101$$

例 6.4 求 $x = 0$ 的原码

解: 设 $x = +0.0000$ $[+0.0000]_{\text{原}} = 0.0000$

$x = -0.0000$ $[-0.0000]_{\text{原}} = 1.0000$

同理, 对于整数 $[+0]_{\text{原}} = 0,0000$

$[-0]_{\text{原}} = 1,0000$

$\therefore [+0]_{\text{原}} \neq [-0]_{\text{原}}$



原码的特点：简单、直观

6.1

但是用原码作加法时，会出现如下问题：

要求	数1	数2	实际操作	结果符号
加法	正	正	加	正
加法	正	负	减	可正可负
加法	负	正	减	可正可负
加法	负	负	加	负

能否 只作加法？

找到一个与负数等价的正数 来代替这个负数
就可使 减 $\longrightarrow$ 加



3. 补码表示法

(1) 补的概念

• 时钟

逆时针

$$\begin{array}{r} 6 \\ - 3 \\ \hline 3 \end{array}$$

顺时针

$$\begin{array}{r} 6 \\ + 9 \\ \hline 15 \\ - 12 \\ \hline 3 \end{array}$$

可见 -3 可用 $+9$ 代替 减法 $\longrightarrow$ 加法
称 $+9$ 是 -3 以 12 为模的 补数

记作 $-3 \equiv +9 \pmod{12}$

同理 $-4 \equiv +8 \pmod{12}$

$-5 \equiv +7 \pmod{12}$

时钟以
12为模

结论

6.1

- 一个负数加上“模”即得该负数的补数
- 一个正数和一个负数互为补数时
它们绝对值之和即为模数

• 计数器（模 16） $1011 \longrightarrow 0000$?

$$\begin{array}{r} 1011 \\ - 1011 \\ \hline 0000 \end{array}$$

$$\begin{array}{r} 1011 \\ + 0101 \\ \hline 10000 \end{array}$$

自然去掉

可见 -1011 可用 $+0101$ 代替

记作 $-1011 \equiv +0101 \pmod{2^4}$

同理 $-011 \equiv +101 \pmod{2^3}$

$-0.1001 \equiv +1.0111 \pmod{2}$

6.1

$$\boxed{-1011 \equiv +0101 \pmod{2^4}}$$

+ 10000

+ 10000

11



(mod 2⁴)

丢掉

01 ?

– 1011

? 0,0101 $\rightarrow$ + 0101

? **1**,0101 $\rightarrow$ 1011

$$2^{4+1} - 1011 = 100000$$

- 1011

1,0101

 $(\text{mod } 2^{4+1})$

用逗号将符号位和数值部分隔开

(3) 补码定义

6.1

整数

$$[x]_{\text{补}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

x 为真值

n 为整数的位数

如

$$x = +1010$$

$$x = -1011000$$

$$[x]_{\text{补}} = 0,1010$$

$$[x]_{\text{补}} = 2^{7+1} + (-1011000)$$

$$= 100000000$$

$$- 1011000$$

$$\hline 1,0101000$$

用 逗号 将符号位
和数值部分隔开



小数

6.1

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$$

x 为真值

如

$$x = +0.1110$$

$$x = -0.1100000$$

$$[x]_{\text{补}} = 0.1110$$

$$[x]_{\text{补}} = 2 + (-0.1100000)$$

$$= 10.0000000$$

$$- 0.1100000$$

$$\hline 1.0100000$$

用 小数点 将符号位
和数值部分隔开



(4) 求补码的快捷方式

6.1

设 $x = -1010$ 时

$$\begin{aligned} \text{则 } [x]_{\text{补}} &= 2^{4+1} - 1010 &= 11111 + 1 - 1010 \\ &= 100000 &= 11111 + 1 \\ &\quad - 1010 &\quad - 1010 \\ \hline &= 1,0110 &\quad \boxed{10101} + 1 \\ & &= 1,0110 \end{aligned}$$

$$\text{又 } [x]_{\text{原}} = \boxed{1,1010}$$

当真值为负时，补码可用原码除符号位外每位取反，末位加1求得



(5) 举例

6.1

例 6.5 已知 $[x]_{\text{补}} = 0.0001$

求 x

解：由定义得 $x = +0.0001$

例 6.6 已知 $[x]_{\text{补}} = 1.0001$

求 x

解：由定义得

$$\begin{aligned} x &= [x]_{\text{补}} - 2 \\ &= 1.0001 - 10.0000 \\ &= -0.1111 \end{aligned}$$

$$\begin{aligned} [x]_{\text{补}} &\xrightarrow{?} [x]_{\text{原}} \\ [x]_{\text{原}} &= 1.1111 \\ \therefore x &= -0.1111 \end{aligned}$$



例 6.7 已知 $[x]_{\text{补}} = 1,1110$

6.1

求 x

解：由定义得

$$\begin{aligned}x &= [x]_{\text{补}} - 2^{4+1} \\&= 1,1110 - 100000 \\&= -0010\end{aligned}$$

$$\begin{aligned}[x]_{\text{补}} &\xrightarrow{?} [x]_{\text{原}} \\[x]_{\text{原}} &= 1,0010 \\ \therefore x &= -0010\end{aligned}$$

当真值为负时，原码可用补码除符号位外
每位取反，末位加1求得



练习 求下列真值的补码

6.1

真值	$[x]_{\text{补}}$	$[x]_{\text{原}}$
$x = +70 = 1000110$	0, 1000110	0, 1000110
$x = -70 = -1000110$	1, 0111010	1, 1000110
$x = 0.1110$	0.1110	0.1110
$x = -0.1110$	1.0010	1.1110
$x = \boxed{0.0000} \quad [+0]_{\text{补}} = [-0]_{\text{补}}$	$\boxed{0.0000}$	0.0000
$x = \boxed{-0.0000}$	$\boxed{0.0000}$	1.0000
$x = -1.0000$	1.0000	不能表示

由小数补码定义
$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$$

$$[-1]_{\text{补}} = 2 + x = 10.0000 - 1.0000 = 1.0000$$



4. 反码表示法

6.1

(1) 定义

整数

$$[x]_{\text{反}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ (2^{n+1} - 1) + x & 0 \geq x > -2^n \pmod{2^{n+1} - 1} \end{cases}$$

x 为真值

n 为整数的位数

如 $x = +1101$

$$[x]_{\text{反}} = 0,1101$$



用 逗号 将符号位
和数值部分隔开

$x = -1101$

$$\begin{aligned} [x]_{\text{反}} &= (2^{4+1} - 1) - 1101 \\ &= 11111 - 1101 \\ &= 1,0010 \end{aligned}$$



小数

6.1

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ (2 - 2^{-n}) + x & 0 \geq x > -1 \pmod{2 - 2^{-n}} \end{cases}$$

x 为真值 n 为小数的位数

如

$$x = +0.1101$$

$$x = -0.1010$$

$$[x]_{\text{反}} = 0.1101$$

$$[x]_{\text{反}} = (2 - 2^{-4}) - 0.1010$$

$$= 1.1111 - 0.1010$$

$$= 1.0101$$

用 小数点 将符号位

和数值部分隔开



(2) 举例

例6.8 已知 $[x]_{\text{反}} = 0,1110$ 求 x

解: 由定义得 $x = +1110$

例6.9 已知 $[x]_{\text{反}} = 1,1110$ 求 x

解: 由定义得
$$\begin{aligned} x &= [x]_{\text{反}} - (2^{4+1} - 1) \\ &= 1,1110 - 11111 \\ &= -0001 \end{aligned}$$

例 6.10 求 0 的反码

解: 设 $x = +0.0000$ $[+0.0000]_{\text{反}} = 0.0000$

$x = -0.0000$ $[-0.0000]_{\text{反}} = 1.1111$

同理, 对于整数 $[+0]_{\text{反}} = 0,0000$ $[-0]_{\text{反}} = 1,1111$

$\therefore [+0]_{\text{反}} \neq [-0]_{\text{反}}$

三种机器数的小结

- 最高位为符号位，书写上用 “,”（整数）或 “.”（小数）将数值部分和符号位隔开
- 对于正数，原码 = 补码 = 反码
- 对于负数，符号位为 1，其数值部分
原码除符号位外每位取反末位加 1 → 补码
原码除符号位外每位取反 → 反码

例6.11 设机器数字长为8位（其中1位为符号位）**6.1**
对于整数，当其分别代表无符号数、原码、补码和反码时，对应的真值范围各为多少？

二进制代码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	<u>+0</u>	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

例6.12 已知 $[y]_{\text{补}}$ 求 $[-y]_{\text{补}}$

6.1

解： 设 $[y]_{\text{补}} = y_0 \cdot y_1 y_2 \cdots y_n$

<I> $[y]_{\text{补}} = 0.y_1 y_2 \cdots y_n$

$[y]_{\text{补}}$ 连同符号位在内， 每位取反， 末位加 1

即得 $[-y]_{\text{补}}$

$$[-y]_{\text{补}} = 1.\overline{y_1} \overline{y_2} \cdots \overline{y_n} + 2^{-n}$$

<II> $[y]_{\text{补}} = 1.y_1 y_2 \cdots y_n$

$[y]_{\text{补}}$ 连同符号位在内， 每位取反， 末位加 1

即得 $[-y]_{\text{补}}$

$$[-y]_{\text{补}} = 0.\overline{y_1} \overline{y_2} \cdots \overline{y_n} + 2^{-n}$$



5. 移码表示法

6.1

补码表示很难直接判断其真值大小

如	十进制	二进制	补码	
	$x = +21$	+10101	0,10101	错
	$x = -21$	-10101	1,01011	大
	$x = +31$	+11111	0,11111	错
	$x = -31$	-11111	1,00001	大

$x + 2^5$

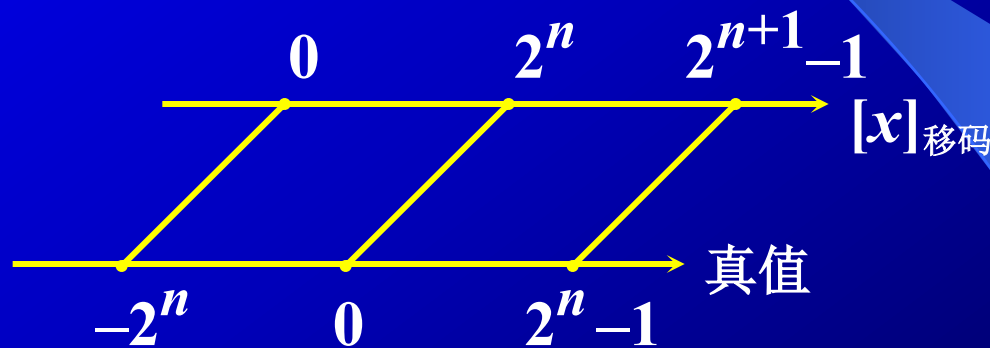
$+10101 + 100000 = 110101$	大	正确
$-10101 + 100000 = 001011$		
$+11111 + 100000 = 111111$	大	正确
$-11111 + 100000 = 000001$		

(1) 移码定义

$$[x]_{\text{移}} = 2^n + x \quad (2^n > x \geq -2^n)$$

x 为真值, n 为 整数的位数

移码在数轴上的表示



如 $x = 10100$

$$[x]_{\text{移}} = 2^5 + 10100 = 1,10100$$

$$x = -10100$$

$$[x]_{\text{移}} = 2^5 - 10100 = 0,01100$$

用 逗号 将符号位
和数值部分隔开

(2) 移码和补码的比较

设 $x = +1100100$

$$[x]_{\text{移}} = 2^7 + 1100100 = \mathbf{1},1100100$$

$$[x]_{\text{补}} = \mathbf{0},1100100$$

设 $x = -1100100$

$$[x]_{\text{移}} = 2^7 - 1100100 = \mathbf{0},0011100$$

$$[x]_{\text{补}} = \mathbf{1},0011100$$

补码与移码只差一个符号位

(3) 真值、补码和移码的对照表

6.1

真值 x ($n=5$)	$[x]_{\text{补}}$	$[x]_{\text{移}}$	$[x]_{\text{移}}$ 对应的 十进制整数
-100000	100000	000000	0
- 11111	100001	000001	1
- 11110	100010	000010	2
⋮	⋮	⋮	⋮
- 00001	111111	011111	31
± 00000	000000	100000	32
+ 00001	000001	100001	33
+ 00010	000010	100010	34
⋮	⋮	⋮	⋮
+ 11110	011110	111110	62
+ 11111	011111	111111	63

(4) 移码的特点

6.1

➤ 当 $x = 0$ 时 $[+0]_{\text{移}} = 2^5 + 0 = 1,00000$

$$[-0]_{\text{移}} = 2^5 - 0 = 1,00000$$

$$\therefore [+0]_{\text{移}} = [-0]_{\text{移}}$$

➤ 当 $n = 5$ 时 最小的真值为 $-2^5 = -100000$

$$[-100000]_{\text{移}} = 2^5 - 100000 = 000000$$

可见，最小真值的移码为全 0

用移码表示浮点数的阶码

能方便地判断浮点数的阶码大小



6.2 数的定点表示和浮点表示

小数点按约定方式标出

一、定点表示



或



定点机

小数定点机

整数定点机

原码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

补码

$$-1 \sim +(1 - 2^{-n})$$

$$-2^n \sim +(2^n - 1)$$

反码

$$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$$

$$-(2^n - 1) \sim +(2^n - 1)$$

二、浮点表示

6.2

$N = S \times r^j$ 浮点数的一般形式

S 尾数 j 阶码 r 基数（基值）

计算机中 r 取 2、4、8、16 等

当 $r = 2$

$$N = 11.0101$$

$$\checkmark = 0.110101 \times 2^{10}$$

$$= 1.10101 \times 2^1$$

$$= 1101.01 \times 2^{-10}$$

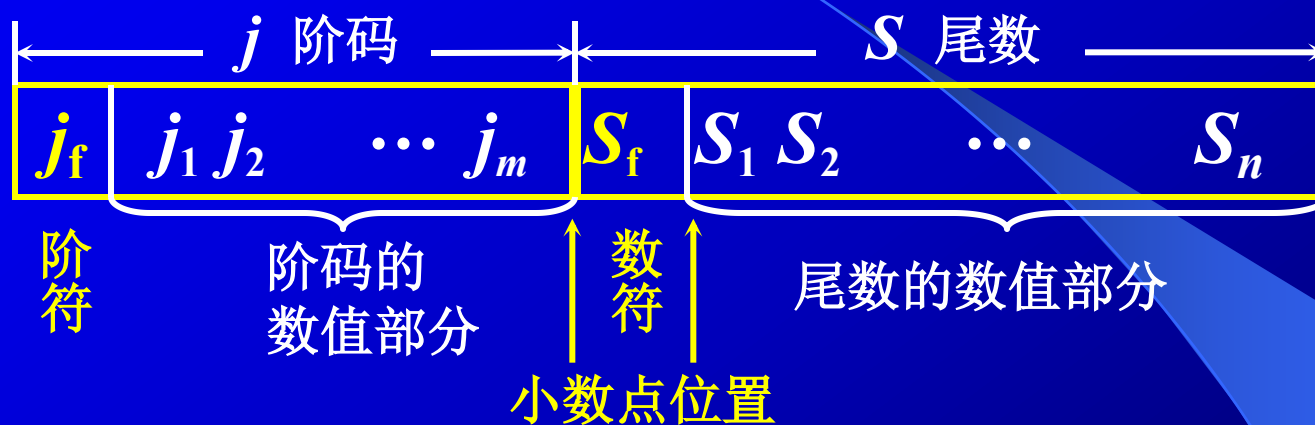
$$\checkmark = 0.00110101 \times 2^{100}$$

二进制表示

规格化数

计算机中 S 小数、可正可负
 j 整数、可正可负

1. 浮点数的表示形式



S_f 代表浮点数的符号

n 其位数反映浮点数的精度

m 其位数反映浮点数的表示范围

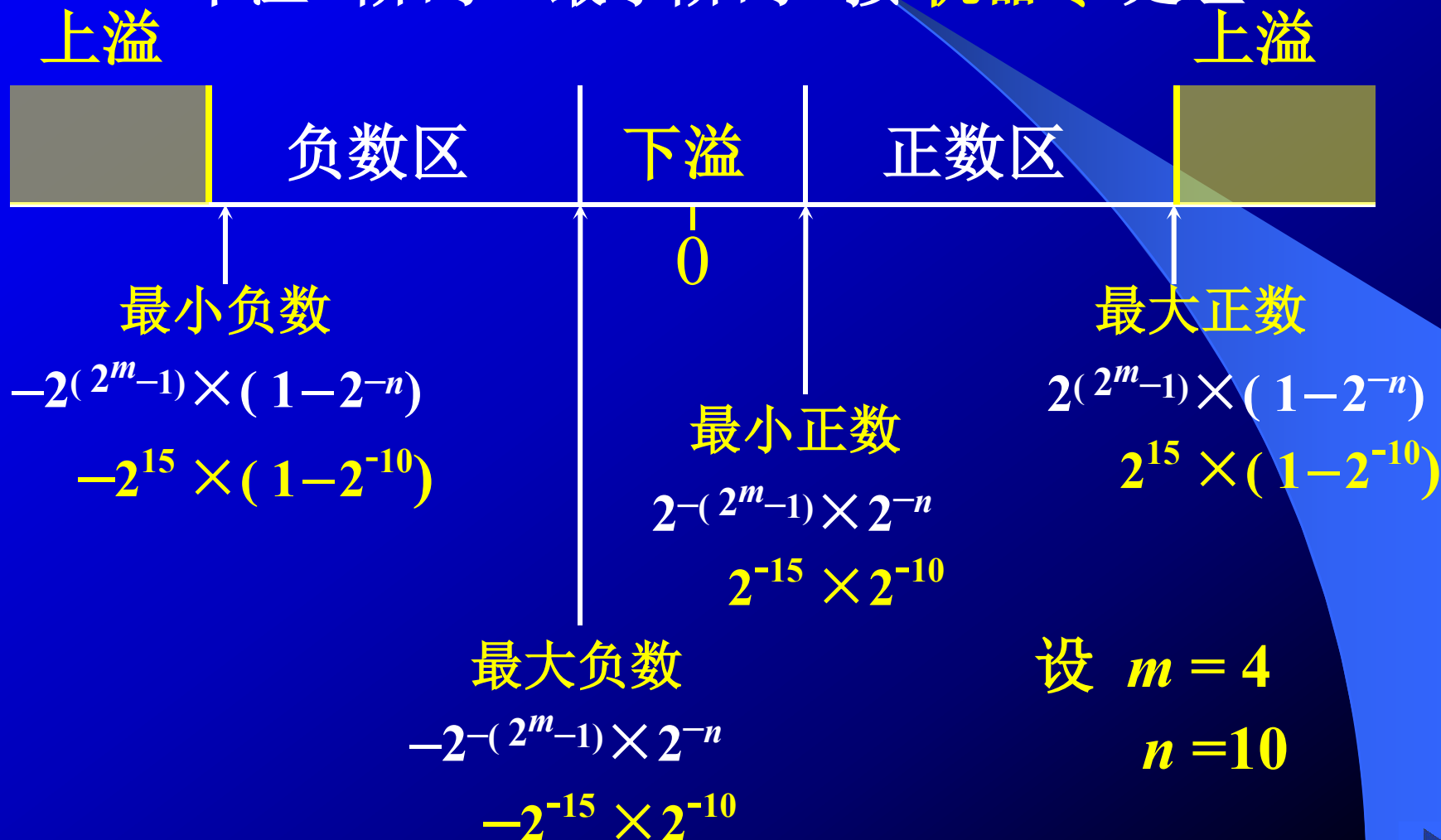
j_f 和 m 共同表示小数点的实际位置

2. 浮点数的表示范围

6.2

上溢 阶码 > 最大阶码

下溢 阶码 < 最小阶码 按 机器零 处理



练习

6.2

设机器数字长为 24 位，欲表示 ± 3 万的十进制数，试问在保证数的最大精度的前提下，除阶符、数符各取 1 位外，阶码、尾数各取几位？

解： $\because 2^{14} = 16384 \quad 2^{15} = 32768$

$\therefore$ 15 位二进制数可反映 ± 3 万之间的十进制数

$$2^{15} \times 0.\underbrace{\times \times \times \dots \times \times \times}_{15\text{位}}$$

$m = 4, 5, 6, \dots$

满足最大精度 可取 $m = 4, n = 18$

3. 浮点数的规格化形式

$r = 2$ 尾数最高位为 1

$r = 4$ 尾数最高 2 位不全为 0

$r = 8$ 尾数最高 3 位不全为 0

基数不同，浮点数的规格化形式不同

4. 浮点数的规格化

$r = 2$ 左规 尾数左移 1 位，阶码减 1

右规 尾数右移 1 位，阶码加 1

$r = 4$ 左规 尾数左移 2 位，阶码减 1

右规 尾数右移 2 位，阶码加 1

$r = 8$ 左规 尾数左移 3 位，阶码减 1

右规 尾数右移 3 位，阶码加 1

基数 r 越大，可表示的浮点数的范围越大

基数 r 越大，浮点数的精度降低



例如：设 $m = 4$, $n = 10$, $r = 2$

尾数规格化后的浮点数表示范围

最大正数 $2^{+1111} \times 0.\underbrace{1111111111}_{10 \uparrow 1} = 2^{15} \times (1 - 2^{-10})$

最小正数 $2^{-1111} \times 0.1\underbrace{0000000000}_{9 \uparrow 0} = 2^{-15} \times 2^{-1} = 2^{-16}$

最大负数 $2^{-1111} \times (-0.1\underbrace{0000000000}_{9 \uparrow 0}) = -2^{-15} \times 2^{-1} = -2^{-16}$

最小负数 $2^{+1111} \times (-0.\underbrace{1111111111}_{10 \uparrow 1}) = -2^{15} \times (1 - 2^{-10})$

三、举例

6.2

例 6.13 将 $+\frac{19}{128}$ 写成二进制定点数、浮点数及在定点机和浮点机中的机器数形式。其中数值部分均取 10 位，数符取 1 位，浮点数阶码取 5 位（含 1 位阶符）。

解： 设 $x = +\frac{19}{128}$

二进制形式 $x = 0.0010011$

定点表示 $x = 0.0010011\mathbf{000}$

浮点规格化形式 $x = 0.1001100000 \times 2^{-10}$

定点机中 $[x]_{\text{原}} = [x]_{\text{补}} = [x]_{\text{反}} = 0.0010011000$

浮点机中 $[x]_{\text{原}} = 1, 0010; 0.1001100000$

$[x]_{\text{补}} = 1, 1110; 0.1001100000$

$[x]_{\text{反}} = 1, 1101; 0.1001100000$



例 6.14 将 -58 表示成二进制定点数和浮点数，**6.2**
并写出它在定点机和浮点机中的三种机器数及阶码
为移码、尾数为补码的形式（其他要求同上例）。

解： 设 $x = -58$

二进制形式 $x = -111010$

定点表示 $x = -0000111010$

浮点规格化形式 $x = -(0.1110100000) \times 2^{110}$

定点机中

$[x]_{\text{原}} = 1, 0000111010$

$[x]_{\text{补}} = 1, 1111000110$

$[x]_{\text{反}} = 1, 1111000101$

浮点机中

$[x]_{\text{原}} = 0, 0110; 1.1110100000$

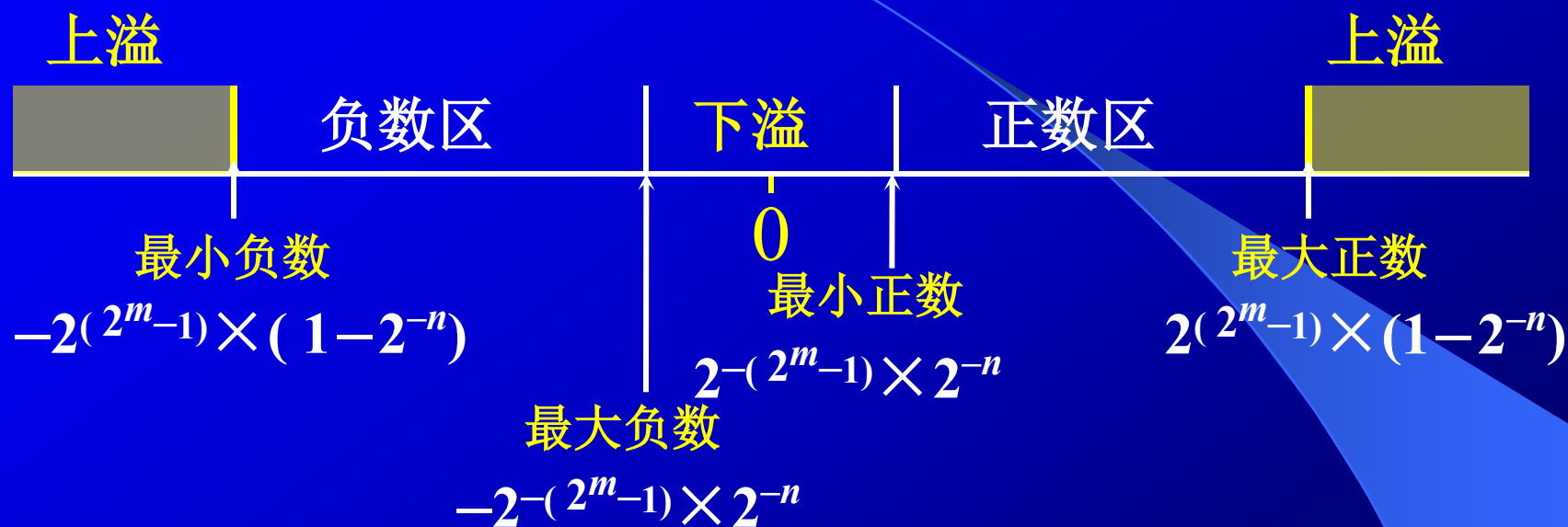
$[x]_{\text{补}} = 0, 0110; 1.0001100000$

$[x]_{\text{反}} = 0, 0110; 1.0001011111$

$[x]_{\text{阶移、尾补}} = 1, 0110; 1.0001100000$



例6.15 写出对应下图所示的浮点数的补码形式。 6.2
形式。 设 $n = 10$, $m = 4$, 阶符、数符各取 1 位。



解:

真值

补码

最大正数	$2^{15} \times (1-2^{-10})$	0,1111; 0.1111111111
最小正数	$2^{-15} \times 2^{-10}$	1,0001; 0.0000000001
最大负数	$-2^{-15} \times 2^{-10}$	1,0001; 1.1111111111
最小负数	$-2^{15} \times (1-2^{-10})$	0,1111; 1.0000000001

- 当浮点数 **尾数为 0** 时，不论其阶码为何值 按机器零处理
- 当浮点数 **阶码等于或小于它所表示的最小 数** 时，不论尾数为何值，按机器零处理

如 $m = 4$ $n = 10$

当阶码和尾数都用补码表示时，机器零为

$\times, \times \times \times \times; \quad 0.00 \dots 0$

(阶码 = -16) $1, 0000; \quad \times.\times\times \dots \times$

当阶码用移码，尾数用补码表示时，机器零为

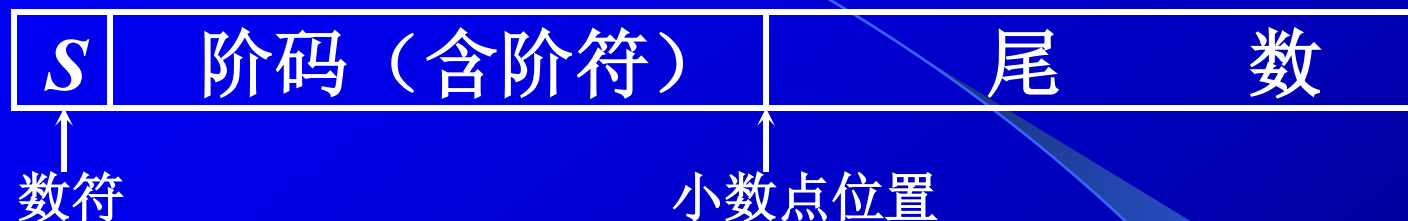
$0, 0000; \quad 0.00 \dots 0$

有利于机器中“判 0”电路的实现



四、IEEE 754 标准

6.2



尾数为规格化表示

非“0”的有效位最高位为“1”（隐含）

	符号位 S	阶码	尾数	总位数
短实数	1	8	23	32
长实数	1	11	52	64
临时实数	1	15	64	80

6.3 定点运算

一、移位运算

1. 移位的意义

$$15.\text{m} = 1500.\text{cm}$$

小数点右移 2 位

机器用语 15 相对于小数点 左移 2 位
(小数点不动)

左移 绝对值扩大

右移 绝对值缩小

在计算机中，移位与加减配合，能够实现乘除运算

2. 算术移位规则

符号位不变

真值	码 制	添补代码
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移 添 0
		右移 添 1
	反 码	1

例6.16

6.3

设机器数字长为 8 位（含 1 位符号位），写出 $A = +26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A = +26 = +11010$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0,0011010$

移位操作	机 器 数	对应的真值
	$[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}}$	
移位前	0,0011010	+26
左移一位	0,0110100	+52
左移两位	0,1101000	+104
右移一位	0,0001101	+13
右移两位	0,0000110	+6



例6.17

6.3

设机器数字长为 8 位（含 1 位符号位），写出 $A = -26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A = -26 = -11010$

原码	移位操作	机 器 数	对应的真值
	移位前	1,0011010	- 26
	左移一位	1,0110100	- 52
	左移两位	1,1101000	- 104
	右移一位	1,0001101	- 13
	右移两位	1,0000110	- 6

补码

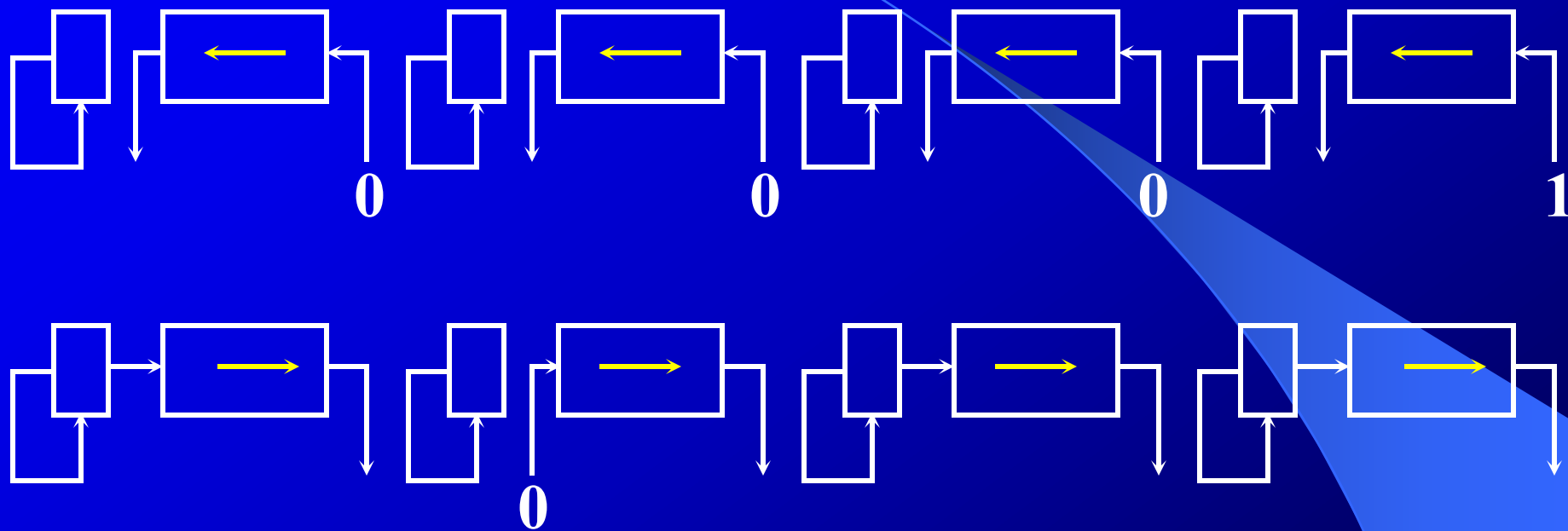
移位操作	机 器 数	对应的真值
移位前	1,1100110	-26
左移一位	1,1001100	-52
左移两位	1,0011000	-104
右移一位	1,1110011	-13
右移两位	1,1111001	-7

反码

移位操作	机 器 数	对应的真值
移位前	1,1100101	-26
左移一位	1,1001011	-52
左移两位	1,0010111	-104
右移一位	1,1110010	-13
右移两位	1,1111001	-6

3. 算术移位的硬件实现

6.3



(a) 真值为正

(b) 负数的原码

(c) 负数的补码

(d) 负数的反码

← 丢 1 出错

出错

正确

正确

→ 丢 1 影响精度

影响精度

影响精度

正确



4. 算术移位和逻辑移位的区别

6.3

算术移位 有符号数的移位

逻辑移位 无符号数的移位

逻辑左移 低位添 0，高位移丢

逻辑右移 高位添 0，低位移丢

例如 **01010011**

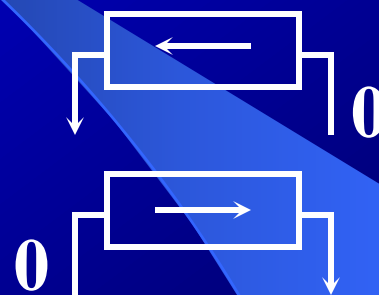
逻辑左移 **10100110**

算术左移 **00100110**

逻辑右移 **01011001**

算术右移 **11011001** (补码)

高位 1 移丢



二、加减法运算

6.3

1. 补码加减运算公式

(1) 加法

整数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2}$

(2) 减法

$$A-B = A+(-B)$$

整数 $[A-B]_{\text{补}} = [A+(-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A-B]_{\text{补}} = [A+(-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2}$

连同符号位一起相加，符号位产生的进位自然丢掉



2. 举例

6.3

例 6.18 设 $A = 0.1011$, $B = -0.0101$

求 $[A + B]_{\text{补}}$

验证

解: $[A]_{\text{补}} = 0.1011$

$+ [B]_{\text{补}} = 1.1011$

$[A]_{\text{补}} + [B]_{\text{补}} = 10.0110 = [A + B]_{\text{补}}$

$\therefore A + B = 0.0110$

$$\begin{array}{r} 0.1011 \\ - 0.0101 \\ \hline 0.0110 \end{array}$$

例 6.19 设 $A = -9$, $B = -5$

求 $[A + B]_{\text{补}}$

验证

解: $[A]_{\text{补}} = 1, 0111$

$+ [B]_{\text{补}} = 1, 1011$

$[A]_{\text{补}} + [B]_{\text{补}} = 11, 0010 = [A + B]_{\text{补}}$

$\therefore A + B = -1110$

$$\begin{array}{r} -1001 \\ + -0101 \\ \hline -1110 \end{array}$$

例 6.20 设机器数字长为 8 位（含 1 位符号位） 6.3

且 $A = 15$, $B = 24$, 用补码求 $A - B$

解: $A = 15 = 0001111$

$$B = 24 = 0011000$$

$$[A]_{\text{补}} = 0, 0001111 \quad [B]_{\text{补}} = 0, 0011000$$

$$+ [-B]_{\text{补}} = 1, 1101000$$

$$[A]_{\text{补}} + [-B]_{\text{补}} = 1, 1110111 = [A - B]_{\text{补}}$$

$$\therefore A - B = -1001 = -9$$

练习 1 设 $x = \frac{9}{16}$ $y = \frac{11}{16}$, 用补码求 $x + y$

$$x + y = -0.1100 = -\frac{12}{16} \quad \text{错}$$

练习 2 设机器数字长为 8 位（含 1 位符号位）
且 $A = -97$, $B = +41$, 用补码求 $A - B$

$$A - B = +1110110 = +118 \quad \text{错}$$

3. 溢出判断

6.3

(1) 一位符号位判溢出

参加操作的 **两个数**（减法时即为被减数和“求补”以后的减数）**符号相同**，其结果的符号与原操作数的符号不同，即为溢出

硬件实现

最高有效位的进位 $\oplus$ 符号位的进位 = 1 溢出

如

$1 \oplus 0 = 1$	} 有 溢出
$0 \oplus 1 = 1$	
$0 \oplus 0 = 0$	} 无 溢出
$1 \oplus 1 = 0$	



(2) 两位符号位判溢出

6.3

$$[x]_{\text{补}'} = \begin{cases} x & 1 > x \geq 0 \\ 4 + x & 0 > x \geq -1 \pmod{4} \end{cases}$$

$$[x]_{\text{补}'} + [y]_{\text{补}'} = [x + y]_{\text{补}'} \pmod{4}$$

$$[x - y]_{\text{补}'} = [x]_{\text{补}'} + [-y]_{\text{补}'} \pmod{4}$$

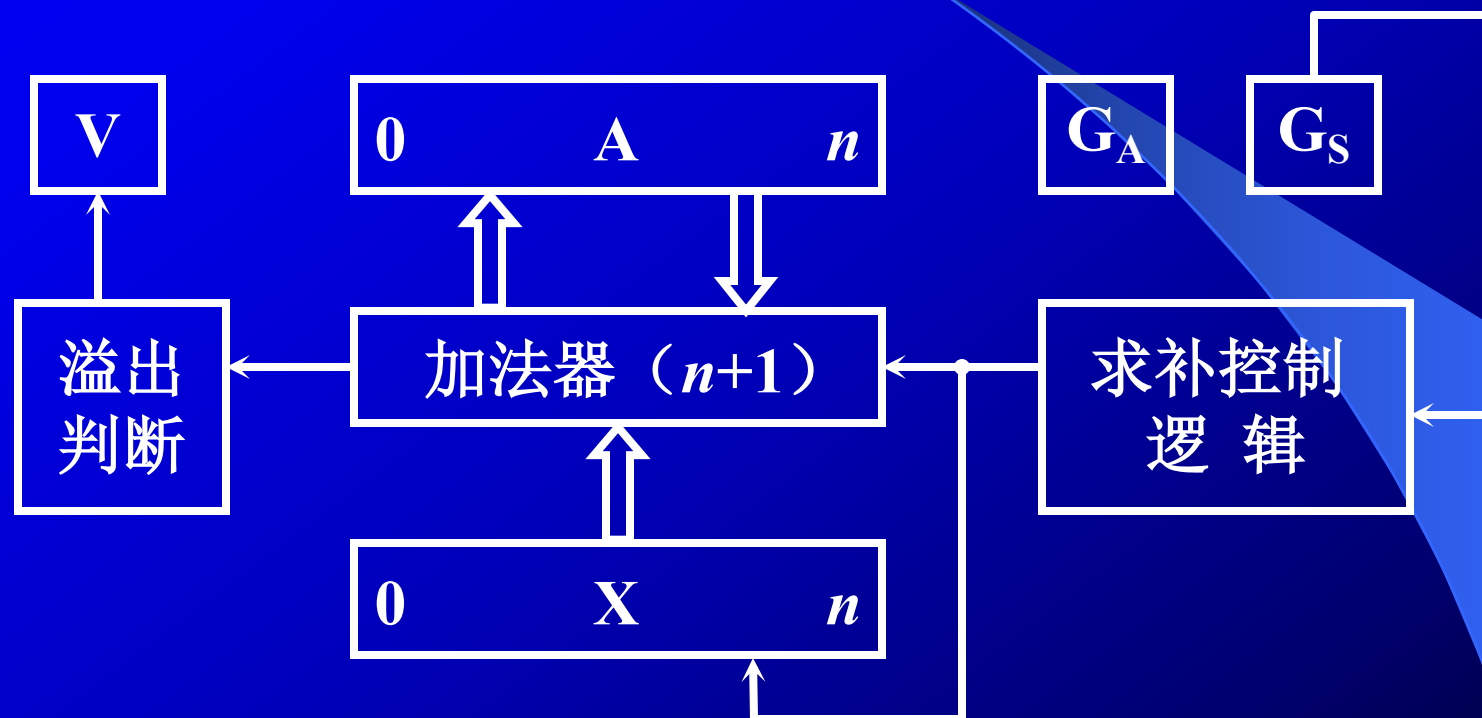
结果的双符号位	相同	未溢出	00, ×××××
			11, ×××××

结果的双符号位	不同	溢出	10, ×××××
			01, ×××××

最高符号位 代表其 真正的符号



4. 补码加减法的硬件配置



A、X 均 $n+1$ 位

用减法标记 G_S 控制求补逻辑

三、乘法运算

1. 分析笔算乘法

$$A = -0.1101 \quad B = 0.1011$$

$$A \times B = -0.10001111 \quad \text{乘积的符号心算求得}$$

$$\begin{array}{r}
 0.1101 \\
 \times 0.1011 \\
 \hline
 1101 \\
 1101 \\
 0000 \\
 1101 \\
 \hline
 0.10001111
 \end{array}$$

✓ 符号位单独处理

✓ 乘数的某一位决定是否加被乘数

? 4个位积一起相加

✓ 乘积的位数扩大一倍

2. 笔算乘法改进

6.3

$$A \cdot B = A \cdot 0.1011$$

$$= 0.1A + 0.00A + 0.001A + 0.0001A$$

$$= 0.1A + 0.00A + 0.001(A + 0.1A)$$

$$= 0.1A + 0.01[0 \cdot A + 0.1(A + 0.1A)]$$

右移一位

$$= 0.1\{A + 0.1[0 \cdot A + 0.1(A + 0.1A)]\}$$

$$= 2^{-1}\{A + 2^{-1}[0 \cdot A + 2^{-1}(A + 2^{-1}(A + 0))]\}$$

第一步 被乘数 $A + 0$

第二步 右移一位，得新的部分积

第三步 部分积 + 被乘数

⋮

第八步 右移一位，得结果

①

②

③

⑧



3. 改进后的笔算乘法过程（竖式） 6.3

部分积	乘数	说明
$\begin{array}{r} 0.0000 \\ + 0.1101 \\ \hline \end{array}$	$1011\underline{=}$	初态，部分积 = 0 乘数为 1，加被乘数
$\begin{array}{r} 0.1101 \\ 0.0110 \\ + 0.1101 \\ \hline \end{array}$	$1101\underline{=}$	→ 1，形成新的部分积 乘数为 1，加被乘数
$\begin{array}{r} 1.0011 \\ 0.1001 \\ + 0.0000 \\ \hline \end{array}$	$1110\underline{=}$	→ 1，形成新的部分积 乘数为 0，加 0
$\begin{array}{r} 0.1001 \\ 0.0100 \\ + 0.1101 \\ \hline \end{array}$	$1111\underline{=}$	→ 1，形成新的部分积 乘数为 1，加被乘数
$\begin{array}{r} 1.0001 \\ 0.1000 \\ \hline \end{array}$	1111	→ 1，得结果

- 乘法 运算可用 加和移位实现
 $n = 4$, 加 4 次, 移 4 次
- 由乘数的末位决定被乘数是否与原部分积相加,
然后 $\rightarrow$ 1 位形成新的部分积, 同时 乘数 $\rightarrow$ 1 位
(末位移丢), 空出高位存放部分积的低位。
- 被乘数只与部分积的高位相加

硬件 3 个寄存器, 具有移位功能

1 个全加器

4. 原码乘法

(1) 原码一位乘运算规则

以小数为例

$$\text{设 } [x]_{\text{原}} = x_0. x_1 x_2 \cdots x_n$$

$$[y]_{\text{原}} = y_0. y_1 y_2 \cdots y_n$$

$$\begin{aligned} [x \cdot y]_{\text{原}} &= (x_0 \oplus y_0). (0. x_1 x_2 \cdots x_n) (0. y_1 y_2 \cdots y_n) \\ &= (x_0 \oplus y_0). x^* y^* \end{aligned}$$

式中 $x^* = 0. x_1 x_2 \cdots x_n$ 为 x 的绝对值

$y^* = 0. y_1 y_2 \cdots y_n$ 为 y 的绝对值

乘积的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相乘 $x^* \cdot y^*$

(2) 原码一位乘递推公式

$$\begin{aligned}
 x^* \cdot y^* &= x^*(0.y_1y_2 \dots y_n) \\
 &= x^*(y_12^{-1} + y_22^{-2} + \dots + y_n2^{-n}) \\
 &= 2^{-1}(y_1x^* + 2^{-1}(y_2x^* + \dots 2^{-1}(y_nx^* + \underbrace{0}_{z_0}) \dots)) \\
 &\quad \underbrace{\dots}_{z_n}
 \end{aligned}$$

$$z_0 = 0$$

$$z_1 = 2^{-1}(y_nx^* + z_0)$$

$$z_2 = 2^{-1}(y_{n-1}x^* + z_1)$$

$$\vdots$$

$$z_n = 2^{-1}(y_1x^* + z_{n-1})$$

例6.21 已知 $x = -0.1110$ $y = 0.1101$ 求 $[x \cdot y]_{\text{原}}$ **6.3**

解: 数值部分的运算

部分积	乘数	说明
0.0000	<u>1101</u>	部分积 初态 $z_0 = 0$
+ 0.1110		+ x^*
0.1110		
逻辑右移 0.0111	<u>0110</u>	$\rightarrow 1$, 得 z_1
+ 0.0000		+ 0
0.0111	0	
逻辑右移 0.0011	<u>1011</u>	$\rightarrow 1$, 得 z_2
+ 0.1110		+ x^*
1.0001	10	
逻辑右移 0.1000	<u>1101</u>	$\rightarrow 1$, 得 z_3
+ 0.1110		+ x^*
1.0110	110	
逻辑右移 0.1011	0110	$\rightarrow 1$, 得 z_4

例6.21 结果

6.3

① 乘积的符号位 $x_0 \oplus y_0 = 1 \oplus 0 = 1$

② 数值部分按绝对值相乘

$$x^* \cdot y^* = 0.10110110$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.10110110$$

特点 绝对值运算

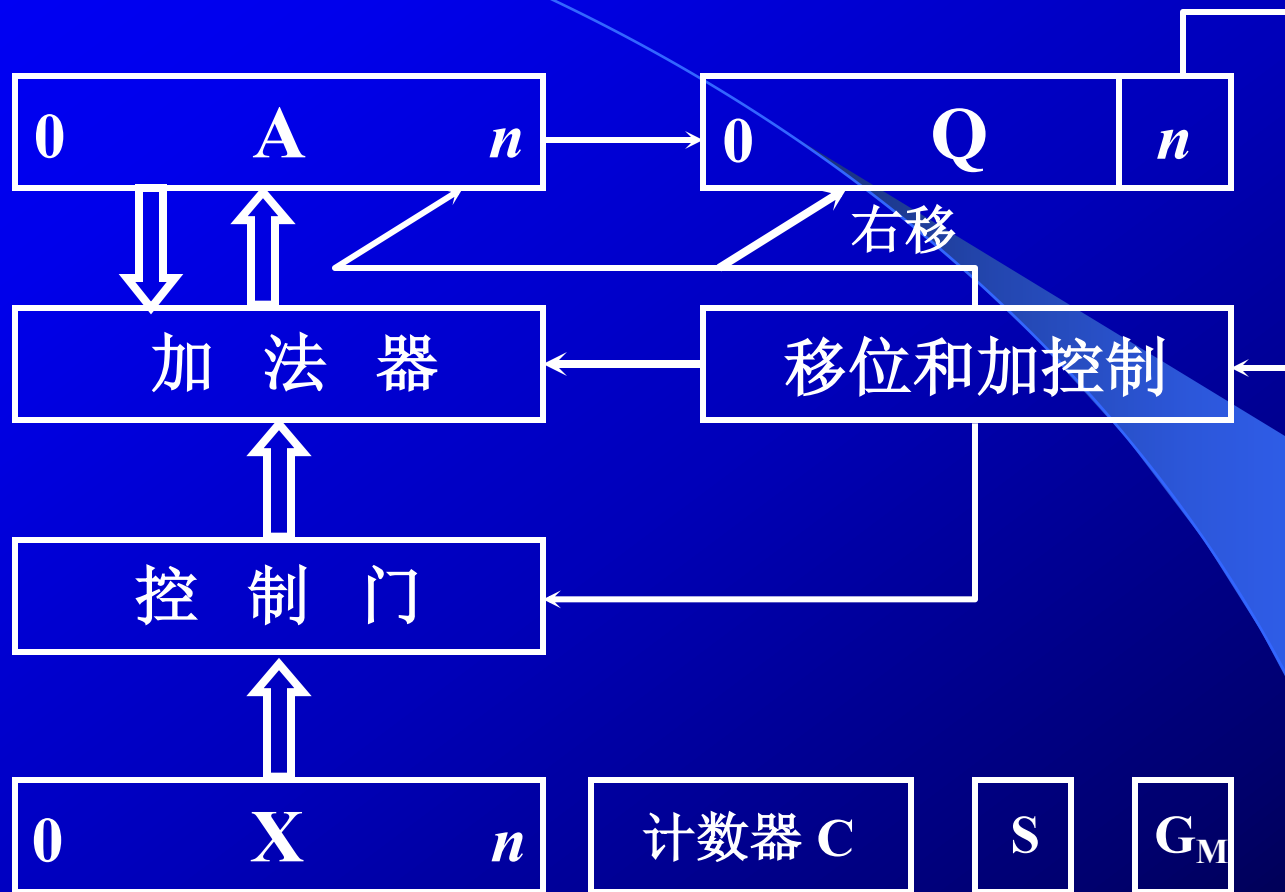
用移位的次数判断乘法是否结束

逻辑移位



(3) 原码一位乘的硬件配置

6.3



A、X、Q 均 $n+1$ 位

移位和加受末位乘数控制

(4) 原码两位乘

6.3

原码乘

符号位 和 数值位 部分 分开运算

两位乘

每次用 乘数的 2 位判断 原部分积
是否加 和 如何加 被乘数

乘数 $y_{n-1} y_n$	新的部分积
0 0	加 “0” $\rightarrow 2$
0 1	加 1 倍的被乘数 $\rightarrow 2$
1 0	加 2 倍的被乘数 $\rightarrow 2$
1 1	加 3 倍的被乘数 $\rightarrow 2$

$$\begin{array}{r} 3 ? \quad 4 \quad 100 \\ -1 \quad -01 \\ \hline 3 \quad 11 \end{array}$$

先 减 1 倍 的被乘数
再 加 4 倍 的被乘数

(5) 原码两位乘运算规则

6.3

乘数判断位 $y_{n-1}y_n$	标志位 C_j	操作内容
0 0	0	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
0 1	0	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 0	0	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “0”
1 1	0	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “1”
0 0	1	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “0”
0 1	1	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置 “0”
1 0	1	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”
1 1	1	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持 “1”

共有操作 $+x^*$ $+2x^*$ $-x^*$ $\rightarrow 2$

实际操作 $+ [x^*]_{\text{补}}$ $+ [2x^*]_{\text{补}}$ $+ [-x^*]_{\text{补}}$ $\rightarrow 2$ 补码移

例6.22 已知 $x = 0.111111$ $y = -0.111001$ 求 $[x \cdot y]_{\text{原}}$

6.3

解：数值部分补码运算

	乘数	C_j	说明
补码右移	<u>00.111001</u>	<u>0</u>	初态 $z_0 = 0$
			$+x^*, C_j = 0$
	000.111111		
补码右移	<u>11 001110</u>	<u>0</u>	$\rightarrow 2$
			$+2x^*, C_j = 0$
	+001.111110		
补码右移	11		
	<u>0111 0011</u>	<u>0</u>	$\rightarrow 2$
			$-x^*, C_j = 1$
补码右移	0111		
	<u>000111 00</u>	<u>1</u>	$\rightarrow 2$
			$+x^*, C_j = 0$
	000.111111		
	000.111000	000111	



例6.22 结果

6.3

① 乘积的符号位 $x_0 \oplus y_0 = 0 \oplus 1 = 1$

② 数值部分的运算

$$x^* \cdot y^* = 0.111000000111$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.111000000111$$

特点 绝对值的补码运算

用移位的次数判断乘法是否结束

算术移位



(6) 原码两位乘和原码一位乘比较 6.3

	原码一位乘	原码两位乘
符号位	$x_0 \oplus y_0$	$x_0 \oplus y_0$
操作数	绝对值	绝对值的补码
移位	逻辑右移	算术右移
移位次数	n	$\frac{n}{2}$ (n 为偶数)
最多加法次数	n	$\frac{n}{2}+1$ (n 为偶数)

思考 n 为奇数时，原码两位乘 移？次 最多加？次



5. 补码乘法

6.3

(1) 补码一位乘运算规则

以小数为例 设被乘数 $[x]_{\text{补}} = x_0.x_1x_2 \cdots x_n$
乘数 $[y]_{\text{补}} = y_0.y_1y_2 \cdots y_n$

① 被乘数任意，乘数为正

同原码乘 但加和移位按补码规则运算
乘积的符号自然形成

② 被乘数任意，乘数为负

乘数 $[y]_{\text{补}}$ ，去掉符号位，操作同 ①

最后加 $[-x]_{\text{补}}$ ，校正



③ Booth 算法（被乘数、乘数符号任意） 6.3

设 $[x]_{\text{补}} = x_0 \cdot x_1 x_2 \cdots x_n$ $[y]_{\text{补}} = y_0 \cdot y_1 y_2 \cdots y_n$

$[x \cdot y]_{\text{补}}$

$$-[x]_{\text{补}} = +[-x]_{\text{补}}$$

$$= [x]_{\text{补}} (0 \cdot y_1 \cdots y_n) - [x]_{\text{补}} \cdot y_0$$

$$= [x]_{\text{补}} (y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n}) - [x]_{\text{补}} \cdot y_0$$

$$2^{-1} = 2^0 - 2^{-1}$$

$$= [x]_{\text{补}} (-y_0 + y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n})$$

$$2^{-2} = 2^{-1} - 2^{-2}$$

$$= [x]_{\text{补}} [-y_0 + (y_1 - y_1 2^{-1}) + (y_2 2^{-1} - y_2 2^{-2}) + \cdots + (y_n 2^{-(n-1)} - y_n 2^{-n})]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_n - y_{n-1}) 2^{-(n-1)} + (0 - y_n) 2^{-n}]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_{n+1} - y_n) 2^{-n}]$$

附加位 y_{n+1}

$$y_1 2^{-1} + \cdots + y_n 2^{-n}$$

④ Booth 算法递推公式

6.3

$$[z_0]_{\text{补}} = 0$$

$$[z_1]_{\text{补}} = 2^{-1} \{ (y_{n+1} - y_n) [x]_{\text{补}} + [z_0]_{\text{补}} \} \quad y_{n+1} = 0$$

⋮

$$[z_n]_{\text{补}} = 2^{-1} \{ (y_2 - y_1) [x]_{\text{补}} + [z_{n-1}]_{\text{补}} \}$$

$$[x \cdot y]_{\text{补}} = [z_n]_{\text{补}} + (y_1 - y_0) [x]_{\text{补}}$$

最后一步不移位

如何实现
 $y_{i+1} - y_i$?

y_i	y_{i+1}	$y_{i+1} - y_i$	操作
0	0	0	$\rightarrow 1$
0	1	1	$+ [x]_{\text{补}} \rightarrow 1$
1	0	-1	$+ [-x]_{\text{补}} \rightarrow 1$
1	1	0	$\rightarrow 1$



例6.23 已知 $x = +0.0011$ $y = -0.1011$ 求 $[x \cdot y]_{\text{补}}$ **6.3**

解:

0 0 . 0 0 0 0	1 . 0 1 0 <u>1</u> <u>0</u>	
+ 1 1 . 1 1 0 1		$+[-x]_{\text{补}}$

$$[x]_{\text{补}} = 0.0011$$

$$[y]_{\text{补}} = 1.0101$$

$$[-x]_{\text{补}} = 1.1101$$

补码
右移

1 1 . 1 1 0 1		
1 1 . <u>1</u> 1 1 0	1 <u>1</u> 0 1 <u>0</u> <u>1</u>	$\rightarrow 1$
+ 0 0 . 0 0 1 1		$+ [x]_{\text{补}}$

补码
右移

0 0 . 0 0 0 1	1	
0 0 . <u>0</u> 0 0 0	1 1 <u>1</u> 0 <u>1</u> <u>0</u>	$\rightarrow 1$
+ 1 1 . 1 1 0 1		$+ [-x]_{\text{补}}$

补码
右移

1 1 . 1 1 0 1	1 1	
1 1 . <u>1</u> 1 1 0	1 1 1 <u>1</u> 0 <u>1</u>	$\rightarrow 1$
+ 0 0 . 0 0 1 1		$+ [x]_{\text{补}}$

补码
右移

0 0 . 0 0 0 1	1 1 1	
0 0 . <u>0</u> 0 0 0	1 1 1 1 <u>1</u> <u>0</u>	$\rightarrow 1$
+ 1 1 . 1 1 0 1		$+ [-x]_{\text{补}}$

1 1 . 1 1 0 1	1 1 1 1	
---------------	---------	--

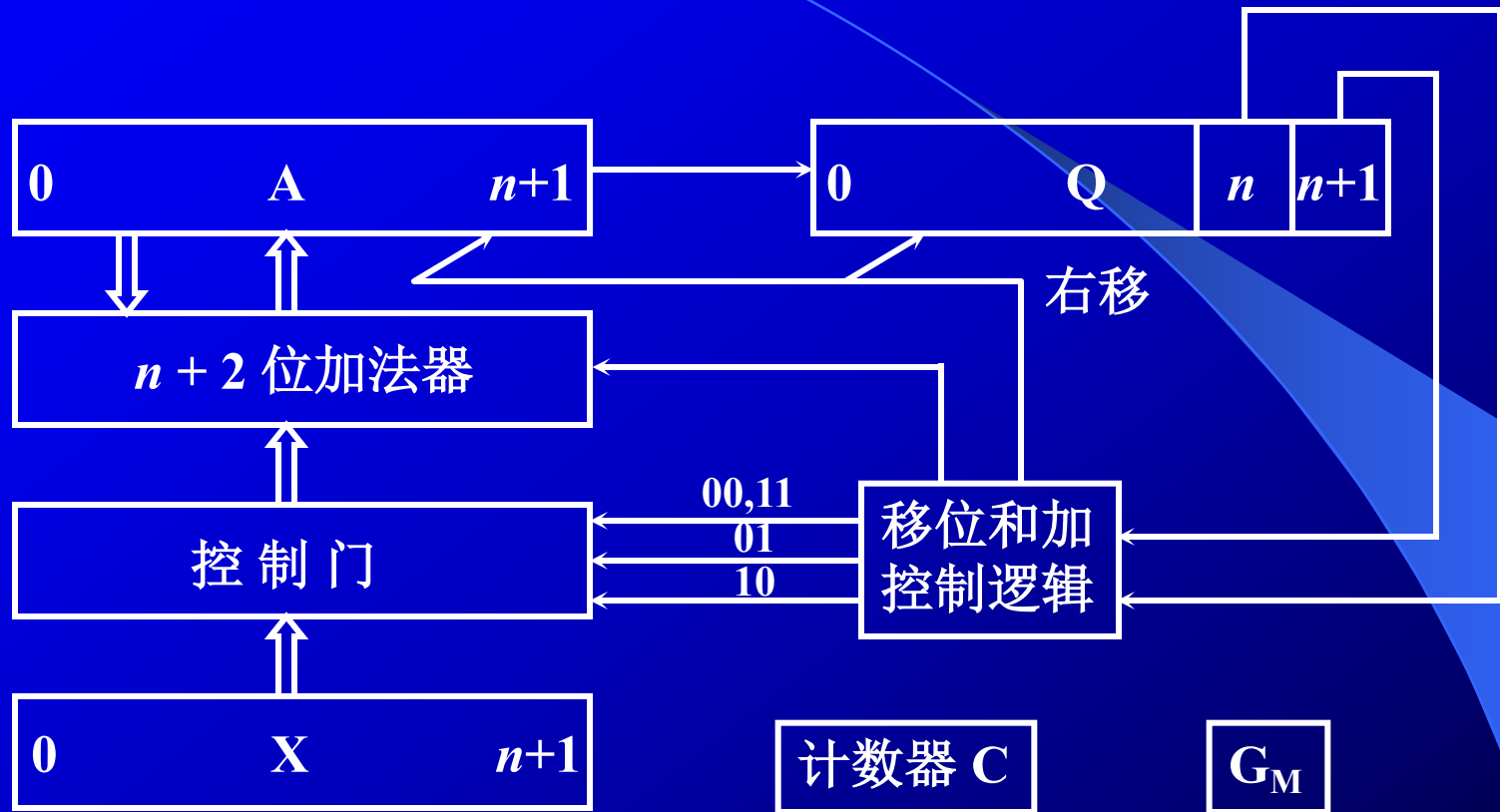
最后一步不移位

$$\therefore [x \cdot y]_{\text{补}} = 1.11011111$$



(2) Booth 算法的硬件配置

6.3



A、X、Q 均 $n+2$ 位
移位和加受末两位乘数控制

乘法小结

- 整数乘法与小数乘法完全相同
可用 **逗号** 代替小数点
- 原码乘 符号位 **单独处理**
补码乘 符号位 **自然形成**
- 原码乘去掉符号位运算 即为无符号数乘法
- 不同的乘法运算需有不同的硬件支持

四、除法运算

6.3

1. 分析笔算除法

$$x = -0.1011 \quad y = 0.1101 \quad \text{求 } x \div y$$

$$\begin{array}{r} 0.1101 \overline{) 0.1101} \\ \underline{0.1101} \\ 0.0000 \\ \underline{0.0000} \\ 0.0000 \\ \underline{0.0000} \\ 0.0000 \\ \underline{0.0000} \\ 0.0000 \end{array}$$

✓ 商符单独处理

? 心算上商

? 余数不动低位补“0”
减右移一位的除数

? 上商位置不固定

$$x \div y = -0.1101 \quad \text{商符心算求得}$$

$$\text{余数 } 0.00000111$$



2. 笔算除法和机器除法的比较

6.3

笔算除法

商符单独处理

心算上商

余数 **不动** 低位补“0”
减右移一位 的除数

2 倍字长加法器

上商位置 **不固定**

机器除法

符号位异或形成

$|x| - |y| > 0$ 上商 1

$|x| - |y| < 0$ 上商 0

余数 **左移一位** 低位补“0”
减 除数

1 倍字长加法器

在寄存器 **最末位上商**



3. 原码除法

6.3

以小数为例

$$[x]_{\text{原}} = x_0.x_1x_2 \dots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2 \dots y_n$$

$$\left[\frac{x}{y}\right]_{\text{原}} = (x_0 \oplus y_0). \frac{x^*}{y^*}$$

式中 $x^* = 0.x_1x_2 \dots x_n$ 为 x 的绝对值
 $y^* = 0.y_1y_2 \dots y_n$ 为 y 的绝对值

商的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相除 $\frac{x^*}{y^*}$

约定 小数定点除法 $x^* < y^*$ 整数定点除法 $x^* > y^*$
被除数不等于 0
除数不能为 0



(1) 恢复余数法

6.3

例6.24 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{\text{原}}$

解: $[x]_{\text{原}} = 1.1011$ $[y]_{\text{原}} = 1.1101$ $[y^*]_{\text{补}} = 0.1101$ $[-y^*]_{\text{补}} = 1.0011$

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$

② 被除数 (余数)	商	说 明
0.1011	0.0000	
+ 1.0011		$+ [-y^*]_{\text{补}}$
1.1110	0	余数为负, 上商 0
+ 0.1101		恢复余数 $+ [y^*]_{\text{补}}$
0.1011	0	恢复后的余数
逻辑左移 1.0110	0	$\leftarrow 1$
+ 1.0011		$+ [-y^*]_{\text{补}}$
0.1001	01	余数为正, 上商 1
逻辑左移 1.0010	01	$\leftarrow 1$
+ 1.0011		$+ [-y^*]_{\text{补}}$

6.3

被除数 (余数)	商	说 明
0.0101	011	余数为正, 上商 1
逻辑左移 0.1010	011	← 1
+ 1.0011		+ $[-y^*]_{\text{补}}$
1.1101	0110	余数为负, 上商 0
+ 0.1101		恢复余数 + $[y^*]_{\text{补}}$
0.1010	0110	恢复后的余数
逻辑左移 1.0100	0110	← 1
+ 1.0011		+ $[-y^*]_{\text{补}}$
0.0111	01101	余数为正, 上商 1

$$\frac{x^*}{y^*} = 0.1101$$

$$\therefore \left[\frac{x}{y}\right]_{\text{原}} = 0.1101$$

余数为正 上商 1

余数为负 上商 0, 恢复余数

上商 5 次

第一次上商判溢出

移 4 次



(2) 不恢复余数法（加减交替法）

6.3

• 恢复余数法运算规则

余数 $R_i > 0$ 上商 “1”， $2R_i - y^*$

余数 $R_i < 0$ 上商 “0”， $R_i + y^*$ 恢复余数

$$2(R_i + y^*) - y^* = 2R_i + y^*$$

• 不恢复余数法运算规则

上商 “1” $2R_i - y^*$

上商 “0” $2R_i + y^*$

加减交替



例6.25 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{\text{原}}$

6.3

解:

0.1011	0.0000	$+[-y^*]_{\text{补}}$
$+1.0011$		余数为负, 上商 0
1.1110	0	$\leftarrow 1$
1.1100	0	$+ [y^*]_{\text{补}}$
$+0.1101$		
0.1001	01	余数为正, 上商 1
1.0010	01	$\leftarrow 1$
$+1.0011$		$+ [-y^*]_{\text{补}}$
0.0101	011	余数为正, 上商 1
0.1010	011	$\leftarrow 1$
$+1.0011$		$+ [-y^*]_{\text{补}}$
1.1101	0110	余数为负, 上商 0
1.1010	0110	$\leftarrow 1$
$+0.1101$		$+ [y^*]_{\text{补}}$
0.0111	01101	余数为正, 上商 1

$$[x]_{\text{原}} = 1.1011$$

$$[y]_{\text{原}} = 1.1101$$

$$[x^*]_{\text{补}} = 0.1011$$

$$[y^*]_{\text{补}} = 0.1101$$

$$[-y^*]_{\text{补}} = 1.0011$$

逻辑左移

逻辑左移

逻辑左移

逻辑左移

逻辑左移



例6.25 结果

6.3

$$\textcircled{1} x_0 \oplus y_0 = 1 \oplus 1 = 0$$

$$\textcircled{2} \frac{x^*}{y^*} = 0.1101$$

$$\therefore [\frac{x}{y}]_{\text{原}} = 0.1101$$

特点 上商 $n+1$ 次

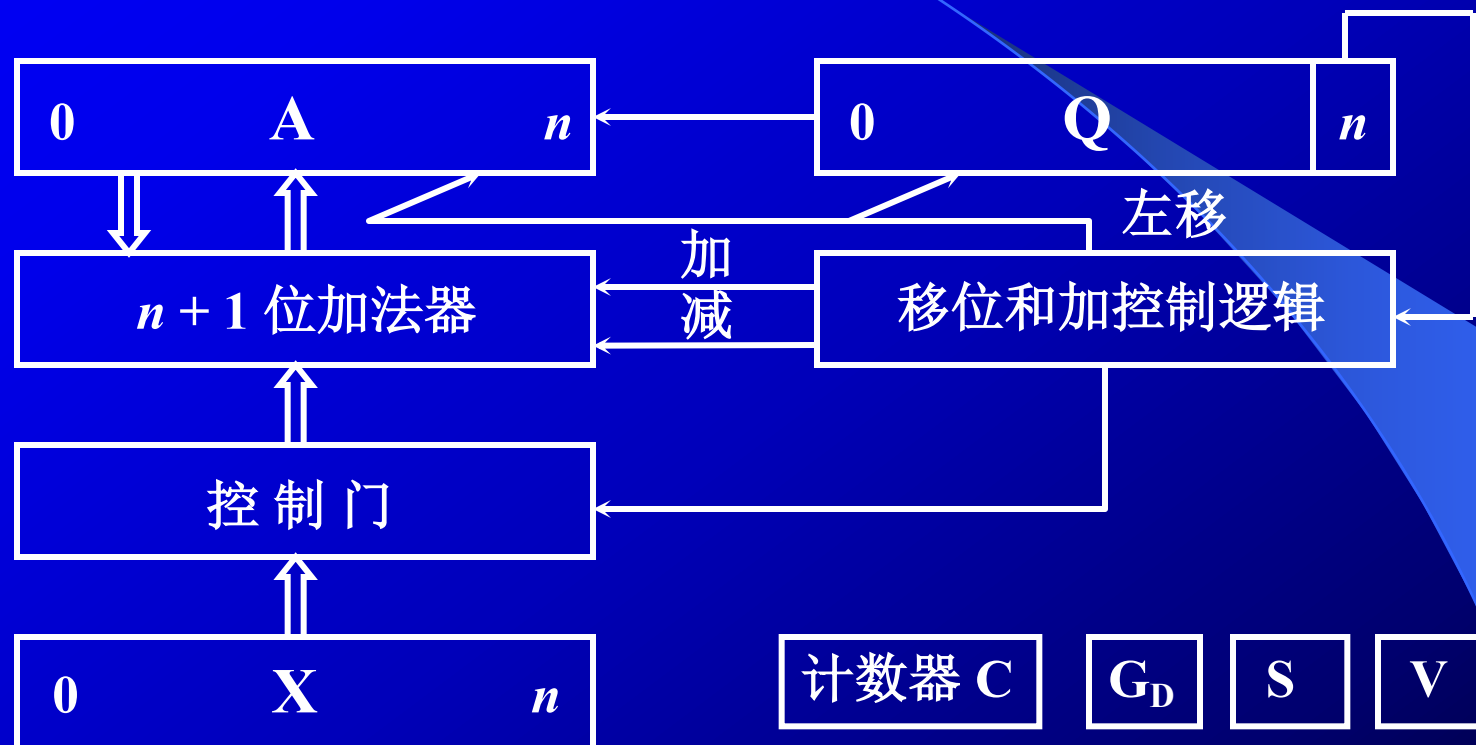
第一次上商判溢出

移 n 次，加 $n+1$ 次

用移位的次数判断除法是否结束



(3) 原码加减交替除法硬件配置



A、X、Q 均 $n+1$ 位
用 Q_n 控制加减交替

4. 补码除法

6.3

(1) 商值的确定

① 比较被除数和除数绝对值的大小

x 与 y 同号

$$x = 0.1011 \quad [x]_{\text{补}} = 0.1011$$

$$y = 0.0011 \quad [y]_{\text{补}} = \boxed{0}.0011$$

$$[x]_{\text{补}} = 0.1011$$

$$+ [-y]_{\text{补}} = 1.1101$$

$$\hline [R_i]_{\text{补}} = \boxed{0}.1000$$

$$x^* > y^*$$

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号

“够减”

$$x = -0.0011 \quad [x]_{\text{补}} = 1.1101$$

$$y = -0.1011 \quad [y]_{\text{补}} = \boxed{1}.0101$$

$$[x]_{\text{补}} = 1.1101$$

$$+ [-y]_{\text{补}} = 0.1011$$

$$\hline [R_i]_{\text{补}} = \boxed{0}.1000$$

$$x^* < y^*$$

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号

“不够减”



6.3

x 与 y 异号

$$x = 0.1011 \quad [x]_{\text{补}} = 0.1011$$

$$y = -0.0011 \quad [y]_{\text{补}} = \boxed{1}.1101$$

$$\begin{array}{r} [x]_{\text{补}} = 0.1011 \\ + [y]_{\text{补}} = 1.1101 \\ \hline [R_i]_{\text{补}} = \boxed{0}.1000 \end{array}$$

$$x^* > y^*$$

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号

“够减”

$$x = -0.0011 \quad [x]_{\text{补}} = 1.1101$$

$$y = 0.1011 \quad [y]_{\text{补}} = \boxed{0}.1011$$

$$\begin{array}{r} [x]_{\text{补}} = 1.1101 \\ + [y]_{\text{补}} = 0.1011 \\ \hline [R_i]_{\text{补}} = \boxed{0}.1000 \end{array}$$

$$x^* < y^*$$

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号

“不够减”

小结

$[x]_{\text{补}}$ 和 $[y]_{\text{补}}$	求 $[R_i]_{\text{补}}$	$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$
同号	$[x]_{\text{补}} - [y]_{\text{补}}$	同号, “够减”
异号	$[x]_{\text{补}} + [y]_{\text{补}}$	异号, “够减”



② 商值的确定 末位恒置“1”法

6.3

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号
正商

$\times.\times\times\times\times 1$
 $0.\underbrace{\times\times\times\times\times}_{\text{原码}} 1$

按原码上商

“够减”上“1”
“不够减”上“0”

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号
负商

$1.\underbrace{\times\times\times\times\times}_{\text{反码}} 1$

按反码上商

“够减”上“0”
“不够减”上“1”

小 结

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$	商	$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商 值
同 号	正	够减 (同号) 不够减 (异号)	1 0 原码上商
异 号	负	够减 (异号) 不够减 (同号)	0 1 反码上商

简 化 为

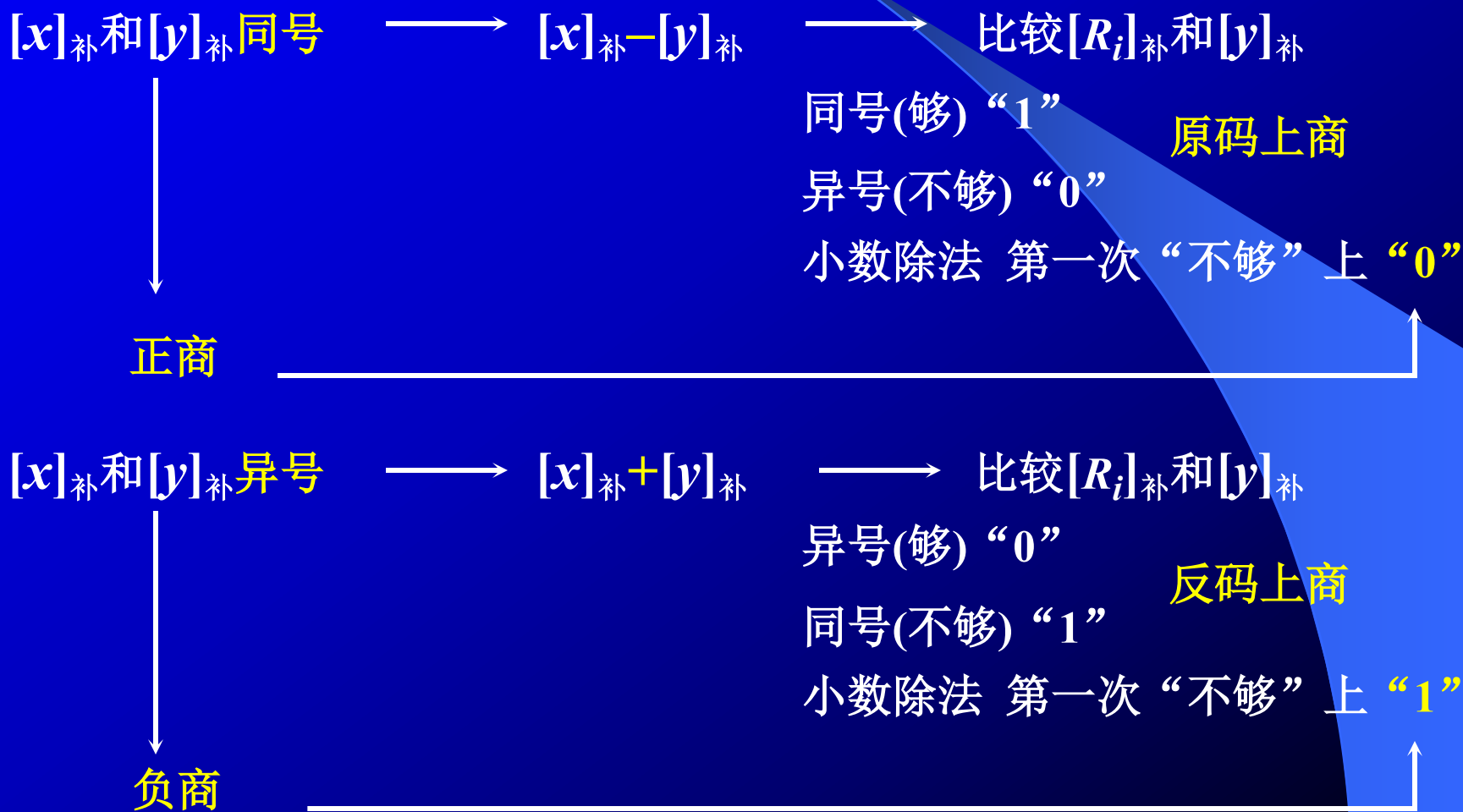
$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商值
同 号	1
异 号	0



(2) 商符的形成

6.3

除法过程中自然形成



(3) 新余数的获得

加减交替

$[R_i]_{\text{补}}$ 和 $[y]_{\text{补}}$	商	新余数
同 号	1	$2[R_i]_{\text{补}} + [-y]_{\text{补}}$
异 号	0	$2[R_i]_{\text{补}} + [y]_{\text{补}}$

例6.26 设 $x = -0.1011$ $y = 0.1101$
求 $[\frac{x}{y}]_{\text{补}}$ 并还原成真值

解: $[x]_{\text{补}} = 1.0101$ $[y]_{\text{补}} = 0.1101$ $[-y]_{\text{补}} = 1.0011$

	1.0101	0.0000	
	+ 0.1101		异号做加法
逻辑左移	0.0010	1	同号上“1”
	0.0100	1	←1
	+ 1.0011		+ $[-y]_{\text{补}}$
逻辑左移	1.0111	10	异号上“0”
	0.1110	10	←1
	+ 0.1101		+ $[y]_{\text{补}}$
逻辑左移	1.1011	100	异号上“0”
	1.0110	100	←1
	+ 0.1101		+ $[y]_{\text{补}}$
逻辑左移	0.0011	1001	同号上“1”
	0.0110	1001	←1 末位恒置“1”

$$\therefore [\frac{x}{y}]_{\text{补}} = 1.0011$$

$$\text{则 } \frac{x}{y} = -0.1101$$

(4) 小结

6.3

- 补码除法共上商 $n+1$ 次（末位恒置 1）
第一次为商符
- 第一次商可判溢出
- 加 n 次 移 n 次
- 用移位的次数判断除法是否结束
- 精度误差最大为 2^{-n}

(5) 补码除和原码除（加减交替法）比较

6.3

	原码除	补码除
商符	$x_0 \oplus y_0$	自然形成
操作数	绝对值补码	补码
上商原则	余数的正负	比较余数和除数的符号
上商次数	$n + 1$	$n + 1$
加法次数	$n + 1$	n
移位	逻辑左移	逻辑左移
移位次数	n	n
第一步操作	$[x^*]_{\text{补}} - [y^*]_{\text{补}}$	同号 $[x]_{\text{补}} - [y]_{\text{补}}$ 异号 $[x]_{\text{补}} + [y]_{\text{补}}$

6.4 浮点四则运算

一、浮点加减运算

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

1. 对阶

(1) 求阶差

$$\Delta j = j_x - j_y = \begin{cases} = 0 & j_x = j_y & \text{已对齐} \\ > 0 & j_x > j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & S_x \leftarrow 1, j_x - 1 \\ y \text{ 向 } x \text{ 看齐} & \checkmark S_y \rightarrow 1, j_y + 1 \end{cases} \\ < 0 & j_x < j_y & \begin{cases} x \text{ 向 } y \text{ 看齐} & \checkmark S_x \rightarrow 1, j_x + 1 \\ y \text{ 向 } x \text{ 看齐} & S_y \leftarrow 1, j_y - 1 \end{cases} \end{cases}$$

(2) 对阶原则

小阶向大阶看齐



例如 $x = 0.1101 \times 2^{01}$ $y = (-0.1010) \times 2^{11}$ **6.4**

求 $x+y$

解: $[x]_{\text{补}} = 00, 01; 00.1101$ $[y]_{\text{补}} = 00, 11; 11.0110$

1. 对阶

① 求阶差 $[\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = 00, 01$

$$\begin{array}{r} + \quad 11, 01 \\ \hline 11, 10 \end{array}$$

阶差为负 (-2) $\therefore S_x \rightarrow 2 \quad j_x + 2$

② 对阶 $[x]_{\text{补}}' = 00, 11; 00.0011$

2. 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}}' = 00.0011 \quad \text{对阶后的}[S_x]_{\text{补}}' \\ + \quad [S_y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \\ \therefore [x+y]_{\text{补}} = 00, 11; 11.1001 \end{array}$$

3. 规格化

6.4

(1) 规格化数的定义

$$r = 2 \quad \frac{1}{2} \leq |S| < 1$$

(2) 规格化数的判断

$S > 0$	规格化形式	$S < 0$	规格化形式
真值	$0.1 \times \times \dots \times$	真值	$-0.1 \times \times \dots \times$
原码	$0.\boxed{1} \times \times \dots \times$	原码	$1.\boxed{1} \times \times \dots \times$
补码	$\boxed{0.1} \times \times \dots \times$	补码	$\boxed{1.0} \times \times \dots \times$
反码	$0.1 \times \times \dots \times$	反码	$1.0 \times \times \dots \times$

原码 不论正数、负数，第一数位为1

补码 符号位和第一数位不同



特例

6.4

$$S = -\frac{1}{2} = -0.100 \dots 0$$

$$[S]_{\text{原}} = 1.100 \dots 0$$

$$[S]_{\text{补}} = \boxed{1.1}00 \dots 0$$

$\therefore [-\frac{1}{2}]_{\text{补}}$ 不是规格化的数

$$S = -1$$

$$[S]_{\text{补}} = \boxed{1.0}00 \dots 0$$

$\therefore [-1]_{\text{补}}$ 是规格化的数



(3) 左规

尾数左移一位，阶码减 1，直到数符和第一数位不同为止

上例 $[x+y]_{\text{补}} = 00, 11; 11.1001$

左规后 $[x+y]_{\text{补}} = 00, 10; 11.0010$

$$\therefore x + y = (-0.1110) \times 2^{10}$$

(4) 右规

当尾数溢出 (>1) 时，需右规

即尾数出现 $01.\times\times \dots\times$ 或 $10.\times\times \dots\times$ 时

尾数右移一位，阶码加 1

例6.27 $x = 0.1101 \times 2^{10}$ $y = 0.1011 \times 2^{01}$ **6.4**

求 $x+y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $[x]_{\text{补}} = 00, 010; 00. 110100$
 $[y]_{\text{补}} = 00, 001; 00. 101100$

① 对阶

$$[\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = \begin{array}{r} 00, 010 \\ + 11, 111 \\ \hline 100, 001 \end{array}$$

阶差为 +1 $\therefore S_y \rightarrow 1, j_y + 1$

$\therefore [y]_{\text{补}}' = 00, 010; 00. 010110$

② 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}} = 00. 110100 \\ + [S_y]_{\text{补}}' = 00. 010110 \\ \hline 01. 001010 \end{array} \quad \begin{array}{l} \text{对阶后的 } [S_y]_{\text{补}}' \\ \text{尾数溢出需右规} \end{array}$$

③ 右规

$$[x+y]_{\text{补}} = 00, 010; 01. 001010$$

右规后

$$[x+y]_{\text{补}} = 00, 011; 00. 100101$$

$$\therefore x+y = 0. 100101 \times 2^{11}$$

4. 舍入

在 对阶 和 右规 过程中，可能出现 尾数末位丢失 引起误差，需考虑舍入

(1) 0 舍 1 入法

(2) 恒置 “1” 法

6.4

例 6.28 $x = (-\frac{5}{8}) \times 2^{-5}$ $y = (-\frac{7}{8}) \times 2^{-4}$

求 $x-y$ (除阶符、数符外, 阶码取 3 位, 尾数取 6 位)

解: $x = (-0.101000) \times 2^{-101}$ $y = (0.111000) \times 2^{-100}$

$[x]_{\text{补}} = 11, 011; 11. 011000$ $[y]_{\text{补}} = 11, 100; 00. 111000$

① 对阶

$$\begin{array}{r} [\Delta j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = 11, 011 \\ + 00, 100 \\ \hline 11, 111 \end{array}$$

阶差为 -1 $\therefore S_x \longrightarrow 1, j_x + 1$

$\therefore [x]_{\text{补}}' = 11, 100; 11. 101100$



② 尾数求和

$$\begin{array}{r}
 [S_x]_{\text{补}} = 11.101100 \\
 + [-S_y]_{\text{补}} = 11.001000 \\
 \hline
 110.110100
 \end{array}$$

③ 右规

$$[x - y]_{\text{补}} = 11, 100; 10.110100$$

右规后

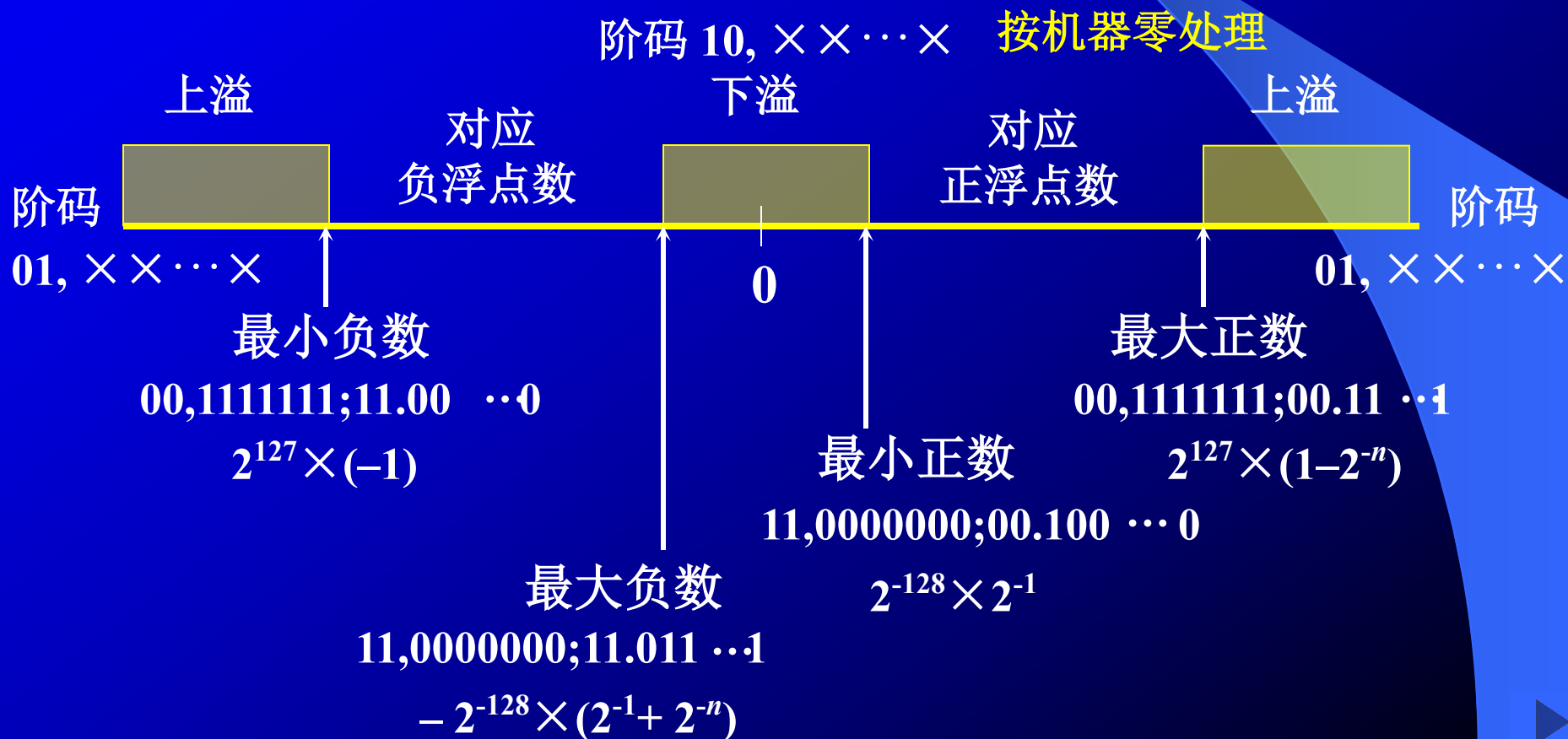
$$[x - y]_{\text{补}} = 11, 101; 11.011010$$

$$\begin{aligned}
 \therefore x - y &= (-0.100110) \times 2^{-11} \\
 &= \left(-\frac{19}{32}\right) \times 2^{-3}
 \end{aligned}$$

5. 溢出判断

6.4

设机器数为补码，尾数为规格化形式，并假设阶符取 2 位，阶码的数值部分取 7 位，数符取 2 位，尾数取 n 位，则该补码在数轴上的表示为



二、浮点乘除运算

$$x = S_x \cdot 2^{j_x} \quad y = S_y \cdot 2^{j_y}$$

1. 乘法

$$x \cdot y = (S_x \cdot S_y) \times 2^{j_x + j_y}$$

2. 除法

$$\frac{x}{y} = \frac{S_x}{S_y} \times 2^{j_x - j_y}$$

3. 步骤

(1) 阶码采用 **补码定点加**（乘法）**减**（除法）运算

(2) 尾数乘除同 **定点** 运算

(3) 规格化

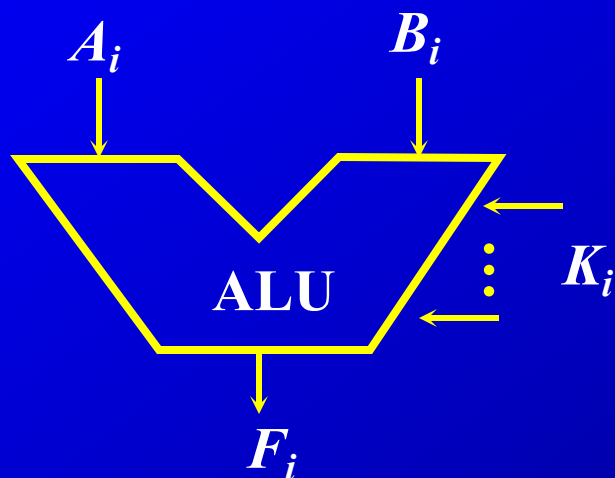
4. 浮点运算部件

阶码运算部件，尾数运算部件



6.5 算术逻辑单元

一、ALU 电路



组合逻辑电路

K_i 不同取值

F_i 不同

四位 ALU 74181

$M = 0$ 算术运算

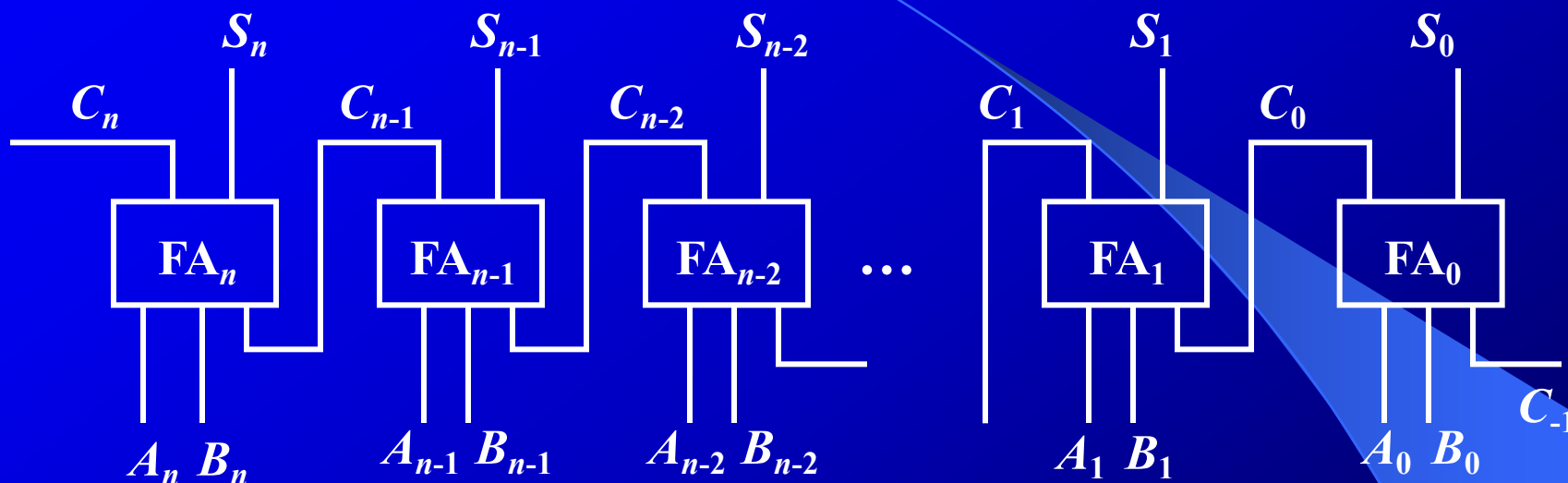
$M = 1$ 逻辑运算

$S_3 \sim S_0$ 不同取值，可做不同运算

二、快速进位链

6.5

1. 并行加法器



$$S_i = \overline{A_i} \overline{B_i} C_{i-1} + \overline{A_i} B_i \overline{C_{i-1}} + A_i \overline{B_i} \overline{C_{i-1}} + A_i B_i C_{i-1}$$

$$C_i = \overline{A_i} B_i C_{i-1} + A_i \overline{B_i} C_{i-1} + A_i B_i \overline{C_{i-1}} + A_i B_i C_{i-1}$$

$$= A_i B_i + (A_i + B_i) C_{i-1}$$

$$d_i = A_i B_i \quad \text{本地进位}$$

$$t_i = A_i + B_i \quad \text{传送条件}$$

则 $C_i = d_i + t_i C_{i-1}$



2. 串行进位链

6.5

进位链

传送进位的电路

串行进位链

进位串行传送

以 4 位全加器为例，每一位的进位表达式为

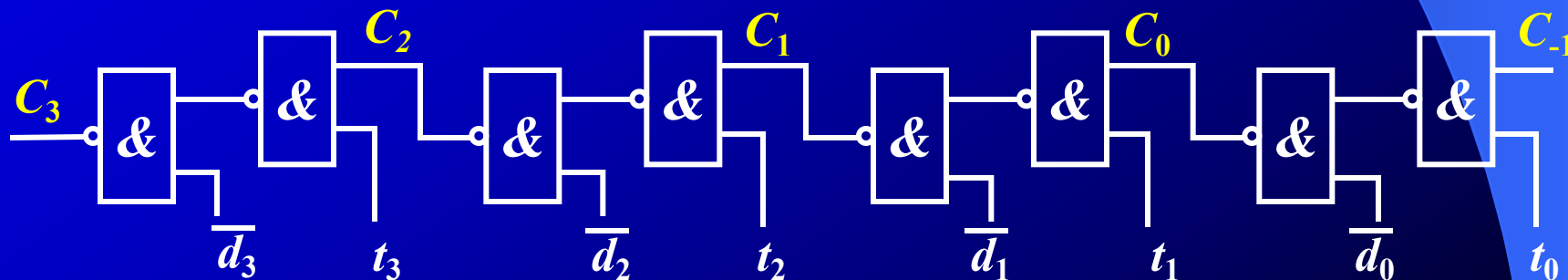
$$C_0 = d_0 + t_0 C_{-1} = \overline{\overline{d_0} \cdot \overline{t_0 C_{-1}}}$$

$$C_1 = d_1 + t_1 C_0$$

$$C_2 = d_2 + t_2 C_1$$

$$C_3 = d_3 + t_3 C_2$$

设与非门的级延迟时间为 t_y



4位全加器产生进位的全部时间为 $8t_y$

n 位全加器产生进位的全部时间为 $2nt_y$

3. 并行进位链（先行进位，跳跃进位）

6.5

n 位加法器的进位同时产生 以 4 位加法器为例

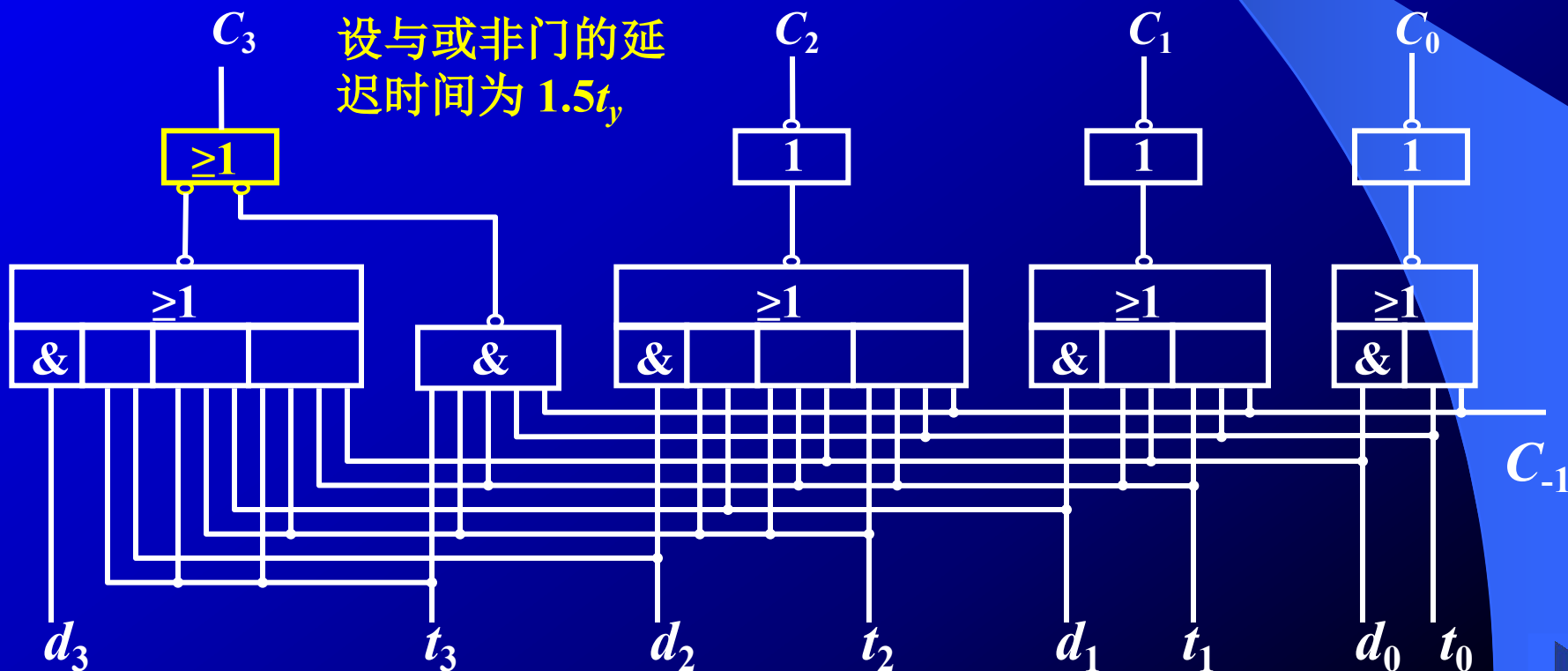
$$C_0 = d_0 + t_0 C_{-1}$$

$$C_1 = d_1 + t_1 C_0 = d_1 + t_1 d_0 + t_1 t_0 C_{-1}$$

$$C_2 = d_2 + t_2 C_1 = d_2 + t_2 d_1 + t_2 t_1 d_0 + t_2 t_1 t_0 C_{-1}$$

$$C_3 = d_3 + t_3 C_2 = d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0 + t_3 t_2 t_1 t_0 C_{-1}$$

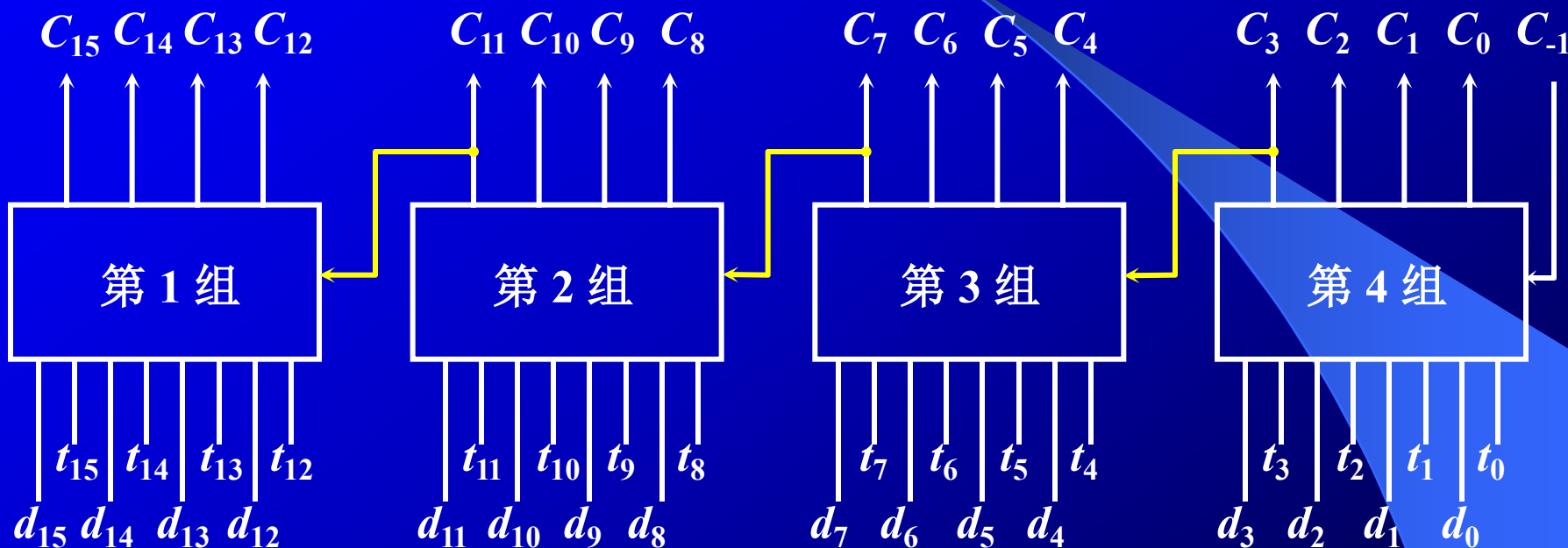
当 $d_i t_i$ 形成后，只需 $2.5t_y$ 产生全部进位



(1) 单重分组跳跃进位链

6.5

n 位全加器分若干小组，小组中的进位同时产生，
小组与小组之间采用串行进位 以 $n = 16$ 为例



当 $d_i t_i$ 形成后	经 $2.5 t_y$	产生 $C_3 \sim C_0$
	$5 t_y$	产生 $C_7 \sim C_4$
	$7.5 t_y$	产生 $C_{11} \sim C_8$
	$10 t_y$	产生 $C_{15} \sim C_{12}$

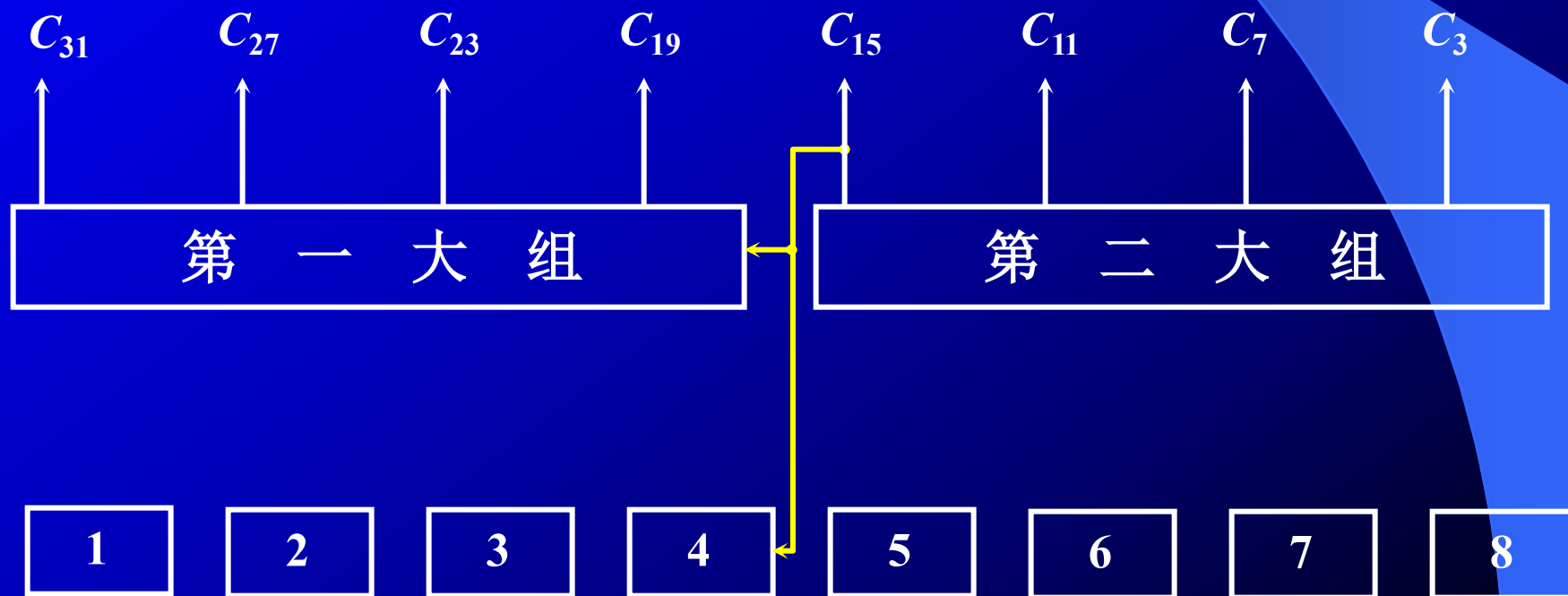


(2) 双重分组跳跃进位链

6.5

n 位全加器分若干大组，大组中又包含若干小组。每个大组中小组的最高位进位同时产生。大组与大组之间采用串行进位。

以 $n = 32$ 为例



(3) 双重分组跳跃进位链 大组进位分析

6.5

以第 8 小组为例

$$\begin{aligned} C_3 &= d_3 + t_3 C_2 = d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0 + t_3 t_2 t_1 t_0 C_{-1} \\ &= \underbrace{d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0}_{D_8} + \underbrace{t_3 t_2 t_1 t_0}_{T_8} C_{-1} \end{aligned}$$

D_8 小组的本地进位 与外来进位无关

T_8 小组的传送条件 与外来进位无关 传递外来进位

同理 第 7 小组 $C_7 = D_7 + T_7 C_3$

第 6 小组 $C_{11} = D_6 + T_6 C_7$

第 5 小组 $C_{15} = D_5 + T_5 C_{11}$

进一步展开得

$$C_3 = D_8 + T_8 C_{-1}$$

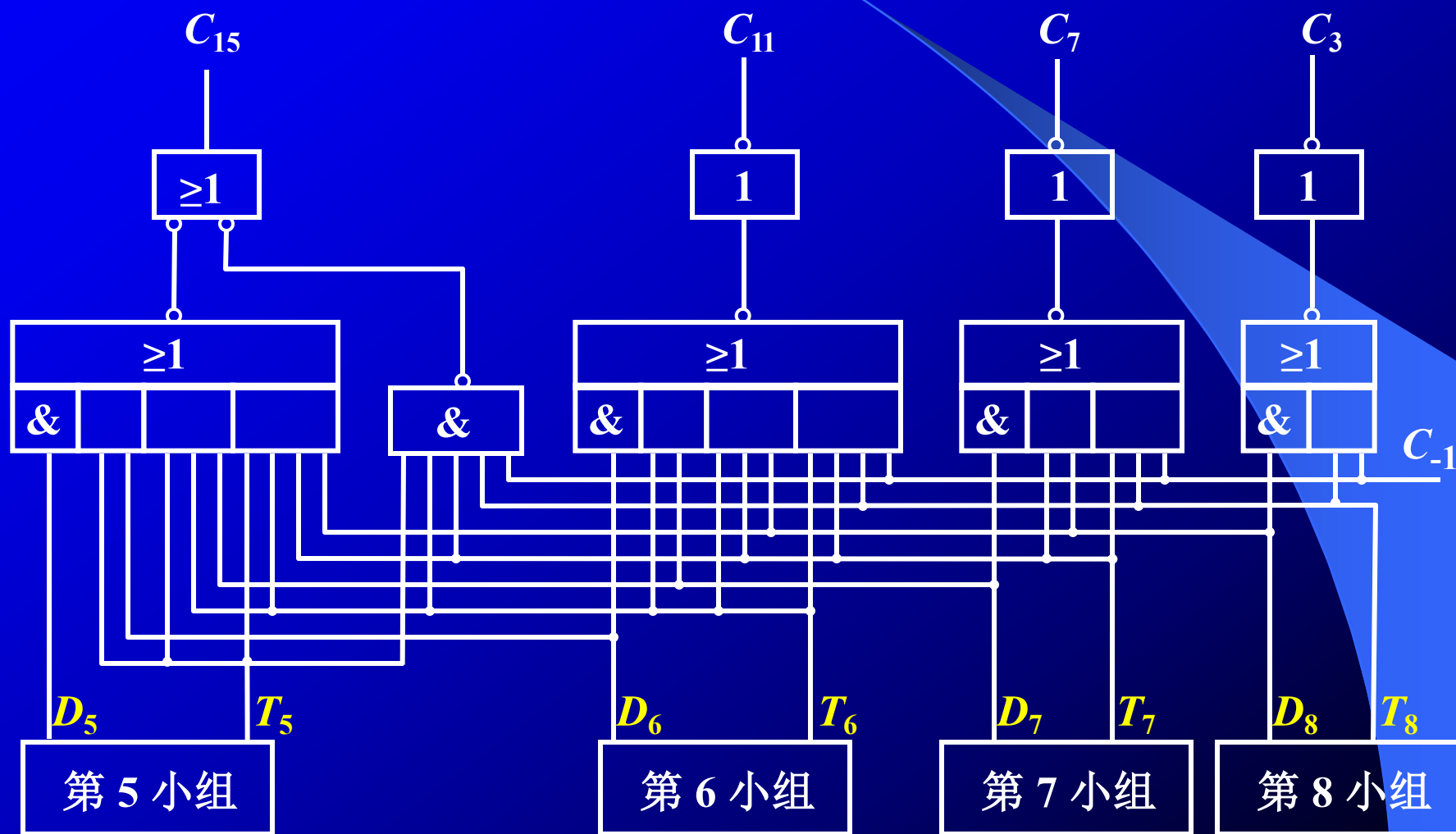
$$C_7 = D_7 + T_7 C_3 = D_7 + T_7 D_8 + T_7 T_8 C_{-1}$$

$$C_{11} = D_6 + T_6 C_7 = D_6 + T_6 D_7 + T_6 T_7 D_8 + T_6 T_7 T_8 C_{-1}$$

$$C_{15} = D_5 + T_5 C_{11} = D_5 + T_5 D_6 + T_5 T_6 D_7 + T_5 T_6 T_7 D_8 + T_5 T_6 T_7 T_8 C_{-1}$$

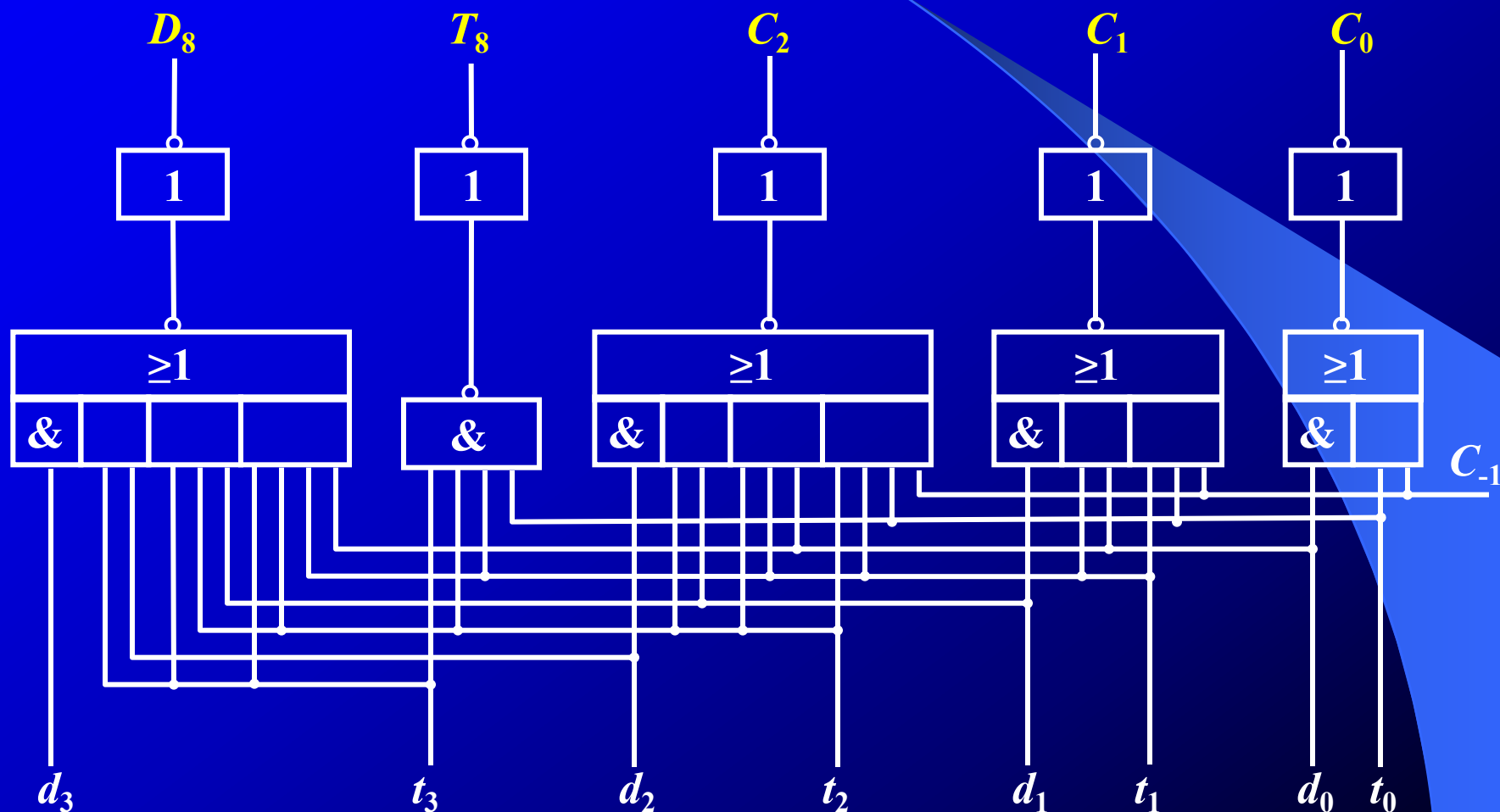
(4) 双重分组跳跃进位链的 **大组** 进位线路 6.5

以第 2 大组为例



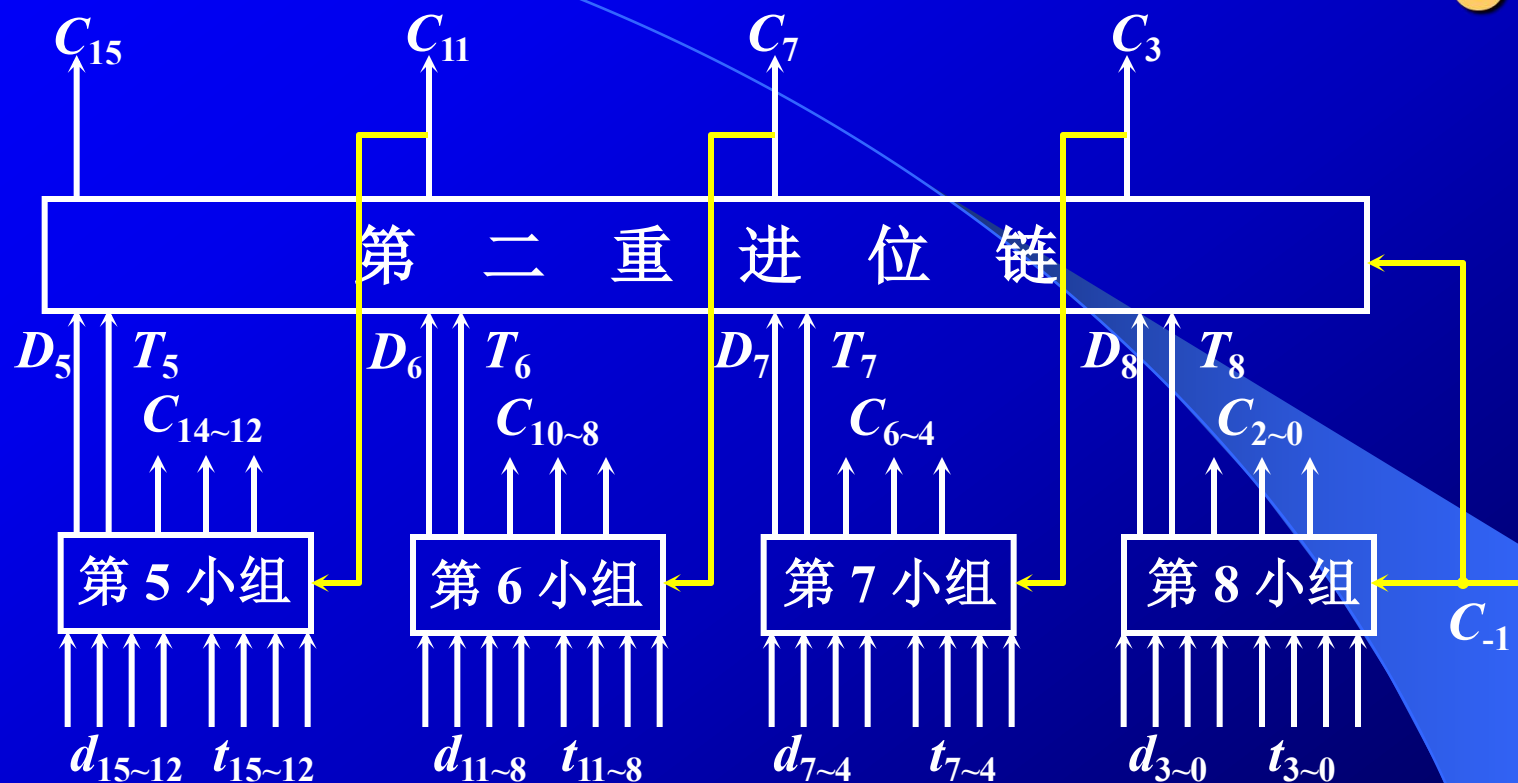
(5) 双重分组跳跃进位链的 小组 进位线路 6.5

以第 8 小组为例 只产生 低 3 位 的进位和 本小组的 $D_8 T_8$



(6) $n=16$ 双重分组跳跃进位链

6.5



当 $d_i t_i$ 和 C_{-1} 形成后

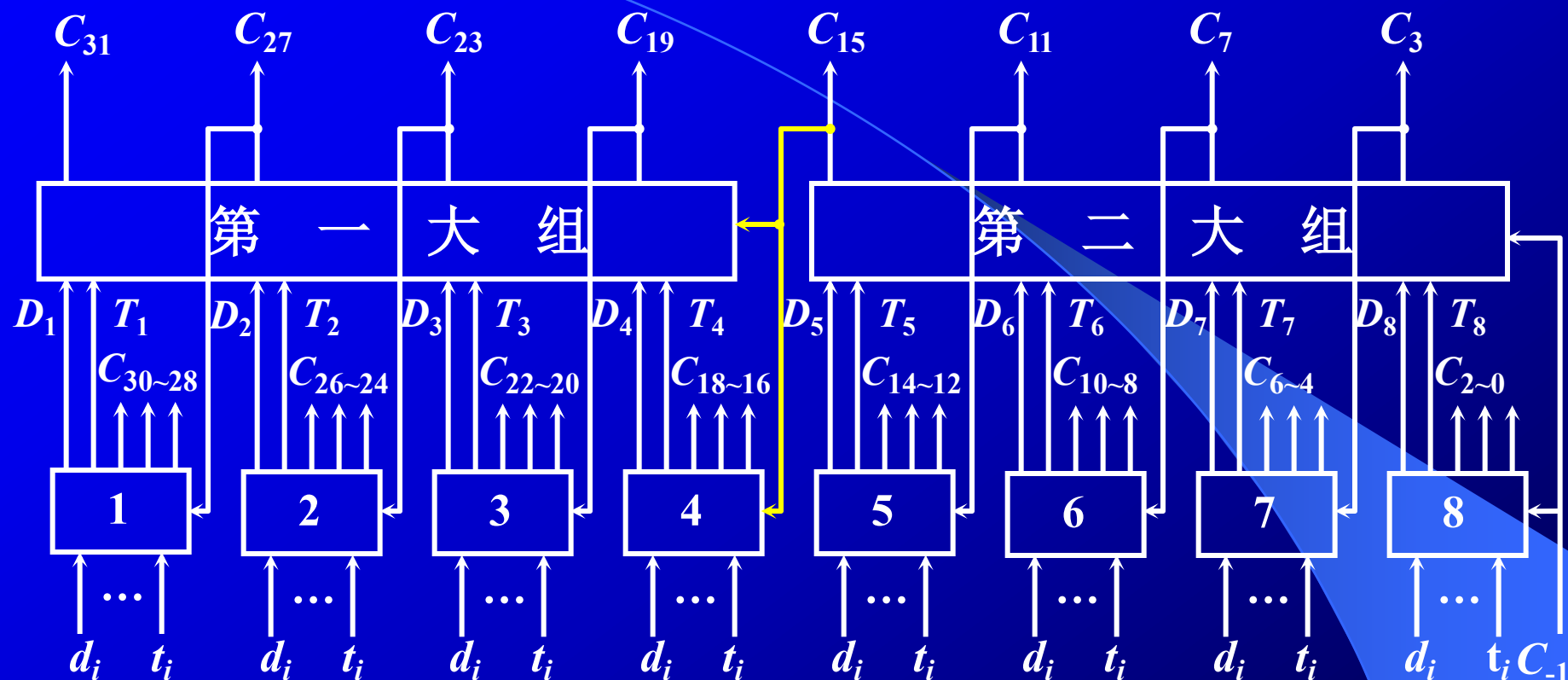
经 $2.5 t_y$	产生 $C_2, C_1, C_0, D_5 \sim D_8, T_5 \sim T_8$
经 $5 t_y$	产生 C_{15}, C_{11}, C_7, C_3
经 $7.5 t_y$	产生 $C_{14} \sim C_{12}, C_{10} \sim C_8, C_6 \sim C_4$

串行进位链 经 $32 t_y$ 产生 全部进位

单重分组跳跃进位链 经 $10 t_y$ 产生 全部进位

(7) $n=32$ 双重分组跳跃进位链

6.5



当 $d_i t_i$ 形成后

- 经 $2.5 t_y$ 产生 C_2 、 C_1 、 C_0 、 $D_1 \sim D_8$ 、 $T_1 \sim T_8$
- $5 t_y$ 产生 C_{15} 、 C_{11} 、 C_7 、 C_3
- $7.5 t_y$ 产生 $C_{18} \sim C_{16}$ 、 $C_{14} \sim C_{12}$ 、 $C_{10} \sim C_8$ 、 $C_6 \sim C_4$
 C_{31} 、 C_{27} 、 C_{23} 、 C_{19}
- $10 t_y$ 产生 $C_{30} \sim C_{28}$ 、 $C_{26} \sim C_{24}$ 、 $C_{22} \sim C_{20}$

第7章 指令系统

7.1 机器指令

7.2 操作数类型和操作类型

7.3 寻址方式

7.4 指令格式举例

7.5 RISC 技术



7.1 机器指令

一、指令的一般格式



1. 操作码 反映机器做什么操作

(1) 长度固定

用于指令字长较长的情况，RISC

如 IBM 370 操作码 8 位

(2) 长度可变

操作码分散在指令字的不同字段中



(3) 扩展操作码技术

7.1

操作码的位数随地址数的减少而增加

	OP	A ₁	A ₂	A ₃	
4 位操作码	0000	A ₁	A ₂	A ₃	最多15条三地址指令
	0001	A ₁	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1110	A ₁	A ₂	A ₃	
8 位操作码	1111	0000	A ₂	A ₃	最多15条二地址指令
	1111	0001	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1110	A ₂	A ₃	
12 位操作码	1111	1111	0000	A ₃	最多15条一地址指令
	1111	1111	0001	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1111	1110	A ₃	
16 位操作码	1111	1111	1111	0000	16条零地址指令
	1111	1111	1111	0001	
	⋮	⋮	⋮	⋮	
	1111	1111	1111	1111	

(3) 扩展操作码技术

7.1

操作码的位数随地址数的减少而增加

	OP	A ₁	A ₂	A ₃
4 位操作码	0000	A ₁	A ₂	A ₃
	0001	A ₁	A ₂	A ₃
	⋮	⋮	⋮	⋮
	1110	A ₁	A ₂	A ₃
8 位操作码	1111	0000	A ₂	A ₃
	1111	0001	A ₂	A ₃
	⋮	⋮	⋮	⋮
	1111	1110	A ₂	A ₃
12 位操作码	1111	1111	0000	A ₃
	1111	1111	0001	A ₃
	⋮	⋮	⋮	⋮
	1111	1111	1110	A ₃
16 位操作码	1111	1111	1111	0000
	1111	1111	1111	0001
	⋮	⋮	⋮	⋮
	1111	1111	1111	1111

三地址指令操作码
每减少一种可多构成
2⁴ 种二地址指令

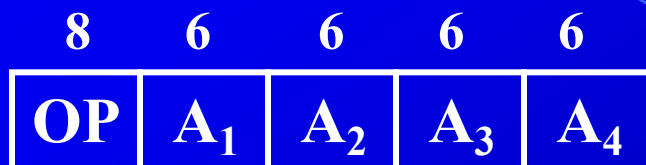
二地址指令操作码
每减少一种可多构成
2⁴ 种一地址指令



2. 地址码

7.1

(1) 四地址



A₁ 第一操作数地址

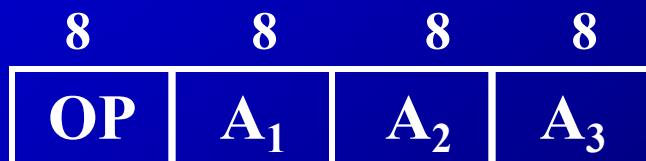
A₂ 第二操作数地址

A₃ 结果的地址

A₄ 下一条指令地址

$(A_1) \text{ OP } (A_2) \longrightarrow A_3$

(2) 三地址



$(A_1) \text{ OP } (A_2) \longrightarrow A_3$

设指令字长为 32 位

操作码固定为 8 位

4 次访存

寻址范围 $2^6 = 64$

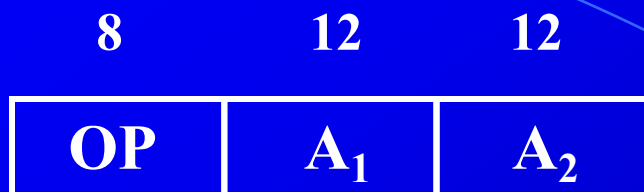
若 PC 代替 A₄

4 次访存

寻址范围 $2^8 = 256$

若 A₃ 用 A₁ 或 A₂ 代替

(3) 二地址



或 $(A_1) \text{ OP } (A_2) \longrightarrow A_1$

$(A_1) \text{ OP } (A_2) \longrightarrow A_2$

若结果存于 ACC 3次访存

4 次访存

寻址范围 $2^{12} = 4 \text{ K}$

若ACC 代替 A₁ (或A₂)

(4) 一地址



$(\text{ACC}) \text{ OP } (A_1) \longrightarrow \text{ACC}$

2 次访存

寻址范围 $2^{24} = 16 \text{ M}$

(5) 零地址 无地址码

二、指令字长

指令字长决定于 { 操作码的长度
操作数地址的长度
操作数地址的个数

1. 指令字长 固定

指令字长 = 存储字长

2. 指令字长 可变

按字节的倍数变化

➤ 当用一些硬件资源代替指令字中的地址码字段后

- 可扩大指令操作数的寻址范围
- 可缩短指令字长
- 可减少访存次数

➤ 当指令的地址字段为寄存器时

三地址 **OP** **R_1** , **R_2** , **R_3**

二地址 **OP** **R_1** , **R_2**

一地址 **OP** **R_1**

- 可缩短指令字长
- 指令执行阶段不访存



7.2 操作数类型和操作种类

一、操作数类型

地址 无符号整数

数字 定点数、浮点数、十进制数

字符 ASCII

逻辑数 逻辑运算

二、数据在存储器中的存放方式

字地址 低字节

0	3	2	1	0
4	7	6	5	4

字地址 为 低字节 地址

字地址 低字节

0	0	1	2	3
4	4	5	6	7

字地址 为 高字节 地址



存储器中的数据存放（存储字长为32位）

7.2

边界对准

地址（十进制）

字（地址 0）				0
字（地址 4）				4
字节（地址11）	字节（地址10）	字节（地址 9）	字节（地址 8）	8
字节（地址15）	字节（地址14）	字节（地址13）	字节（地址12）	12
半字（地址18）✓		半字（地址16）✓		16
半字（地址22）✓		半字（地址20）✓		20
双字（地址24）▲				24
双字				28
双字（地址32）▲				32
双字				36

边界未对准

地址（十进制）

字(地址2)		半字(地址0)	0
字节(地址7)	字节(地址6)	字(地址4)	4
半字(地址10)		半字(地址8)	8

三、操作类型

7.2

1. 数据传送

源	寄存器	寄存器	存储器	存储器
目的	寄存器	存储器	寄存器	存储器
例如	MOVE	STORE MOVE PUSH	LOAD MOVE POP	MOVE
置“1”，清“0”				

2. 算术逻辑操作

加、减、乘、除、增 1、减 1、求补、浮点运算、十进制运算
与、或、非、异或、位操作、位测试、位清除、位求反

如 8086 ADD SUB MUL DIV INC DEC CMP NEG
 AAA AAS AAM AAD
 AND OR NOT XOR TEST



3. 移位操作

算术移位 逻辑移位

循环移位（带进位和不带进位）

4. 转移

(1) 无条件转移 **JMP**

(2) 条件转移

结果为零转 ($Z = 1$) **JZ**

结果溢出转 ($O = 1$) **JO**

结果有进位转 ($C = 1$) **JC**

跳过一条指令 **SKP**

如

300
⋮
305

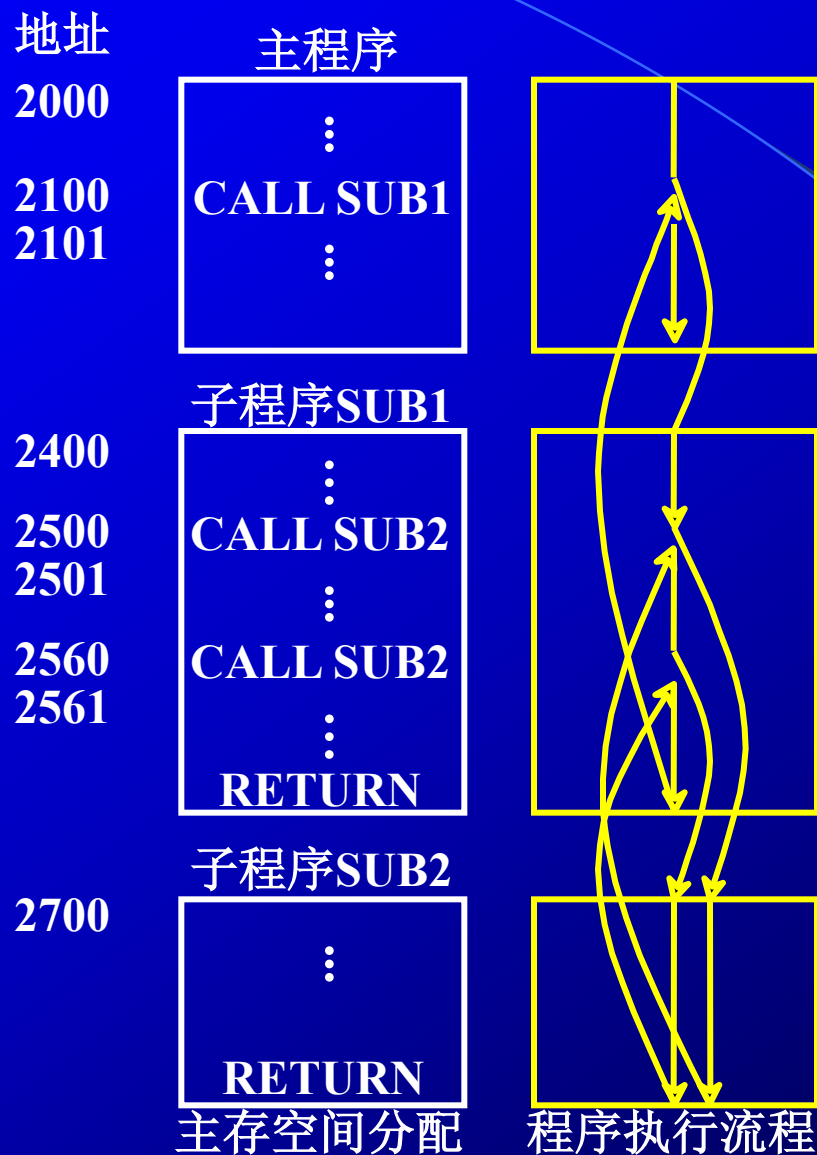
306
307

完成触发器

SKP DZ D = 0 则跳

(3) 调用和返回

7.2



(4) 陷阱 (Trap) 与陷阱指令

意外事故的中断

7.2

- 一般不提供给用户直接使用
在出现事故时, 由 CPU 自动产生并执行 (隐指令)
- 设置供用户使用的陷阱指令

如 8086 INT TYPE 软中断

提供给用户使用的陷阱指令, 完成系统调用

5. 输入输出

入 端口地址 $\longrightarrow$ CPU 的寄存器

如 **IN AX, m** **IN AX, DX**

出 CPU 的寄存器 $\longrightarrow$ 端口地址

如 **OUT n, AX** **OUT DX, AX**



7.3 寻址方式

寻址方式 确定 本条指令 的 操作数地址
下一条 欲执行 指令 的 指令地址

寻址方式 { 指令寻址
数据寻址

7.3 寻址方式

一、指令寻址

顺序 $(PC) + 1 \longrightarrow PC$

跳跃 由转移指令指出



二、数据寻址

7.3

操作码	寻址特征	形式地址 A
-----	------	--------

形式地址 指令字中的地址

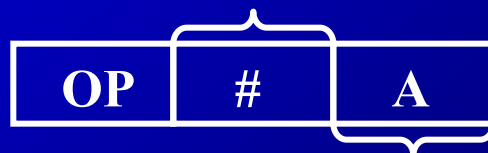
有效地址 操作数的真实地址

约定 指令字长 = 存储字长 = 机器字长

1. 立即寻址

形式地址 A 就是操作数

立即寻址特征



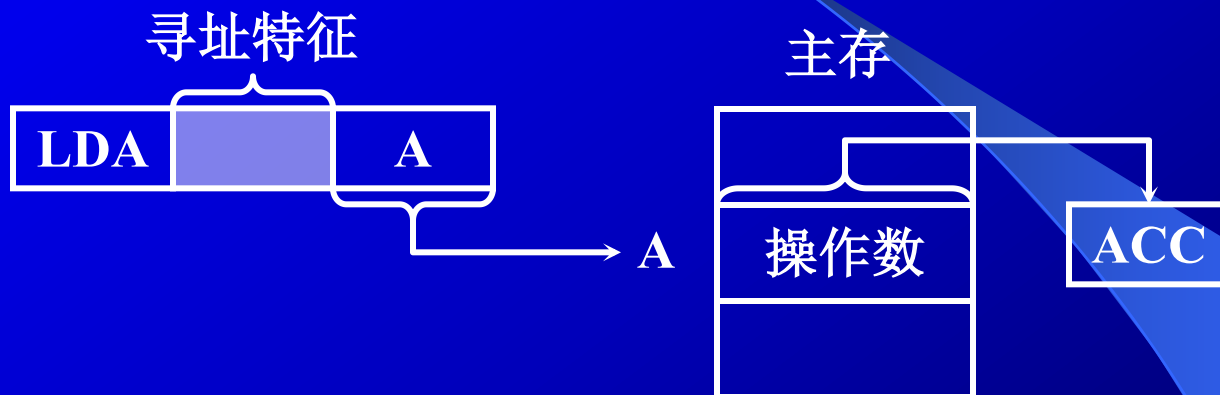
立即数 可正可负 补码

- 指令执行阶段不访存
- A 的位数限制了立即数的范围

2. 直接寻址

7.3

$EA = A$ 有效地址由形式地址直接给出

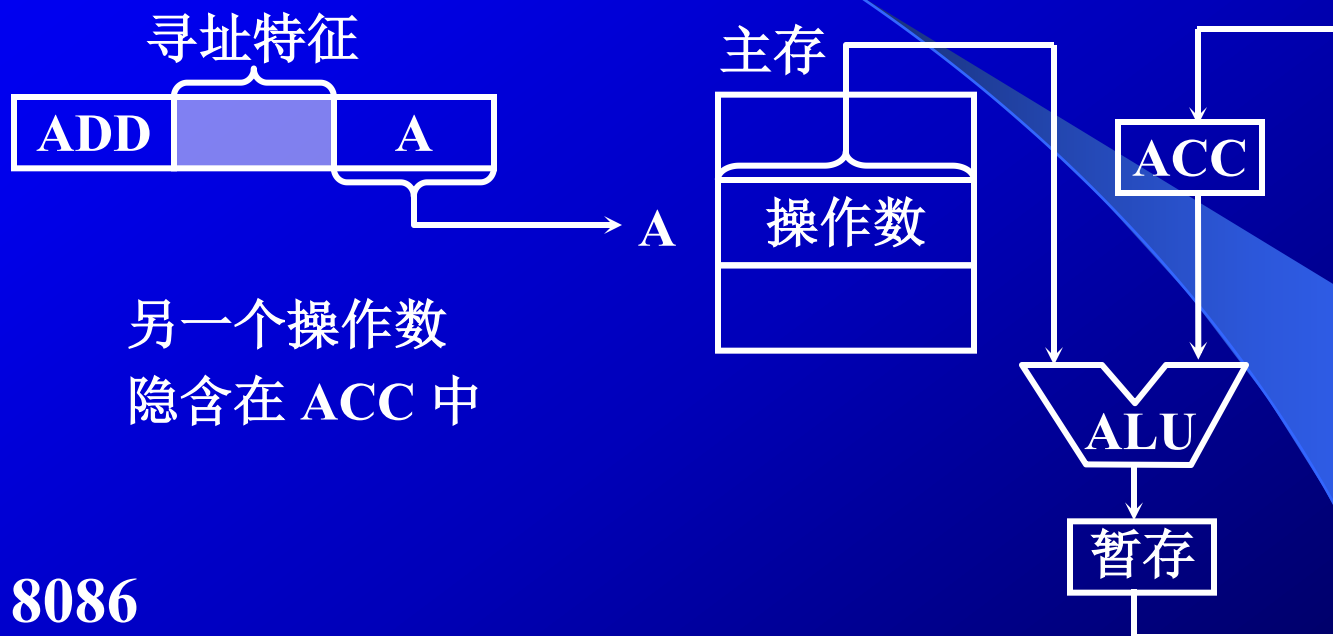


- 执行阶段访问一次存储器
- A 的位数决定了该指令操作数的寻址范围
- 操作数的地址不易修改（必须修改A）

3. 隐含寻址

7.3

操作数地址隐含在操作码中



如 8086

MUL 指令 被乘数隐含在 AX (16位) 或 AL (8位) 中

MOVS 指令 源操作数的地址隐含在 SI 中

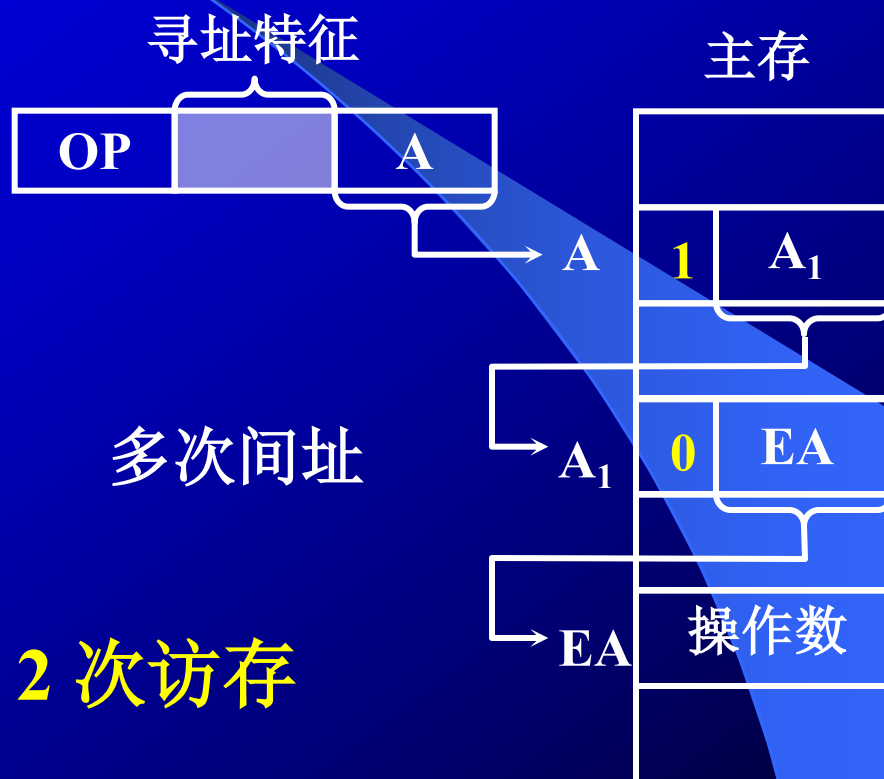
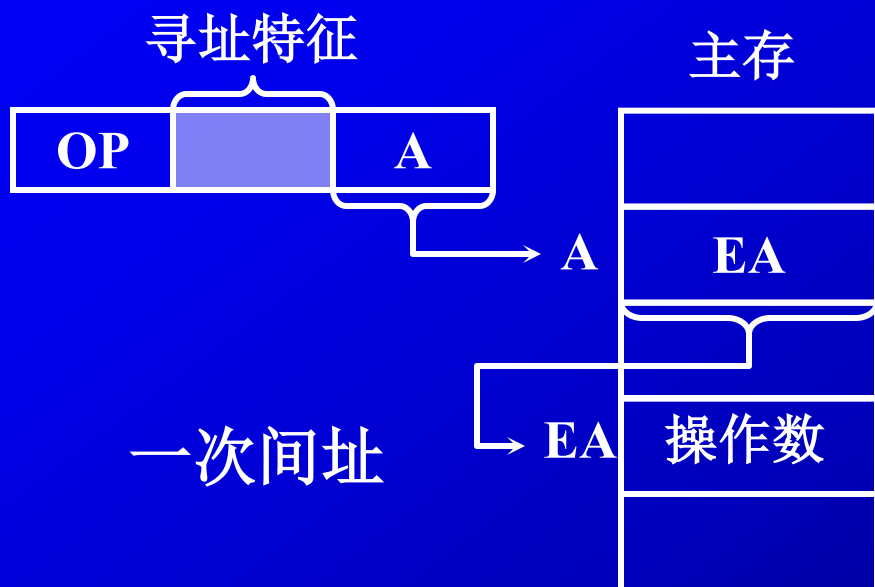
目的操作数的地址隐含在 DI 中

- 指令字中少了一个地址字段，可缩短指令字长

4. 间接寻址

7.3

$EA = (A)$ 有效地址由形式地址间接提供

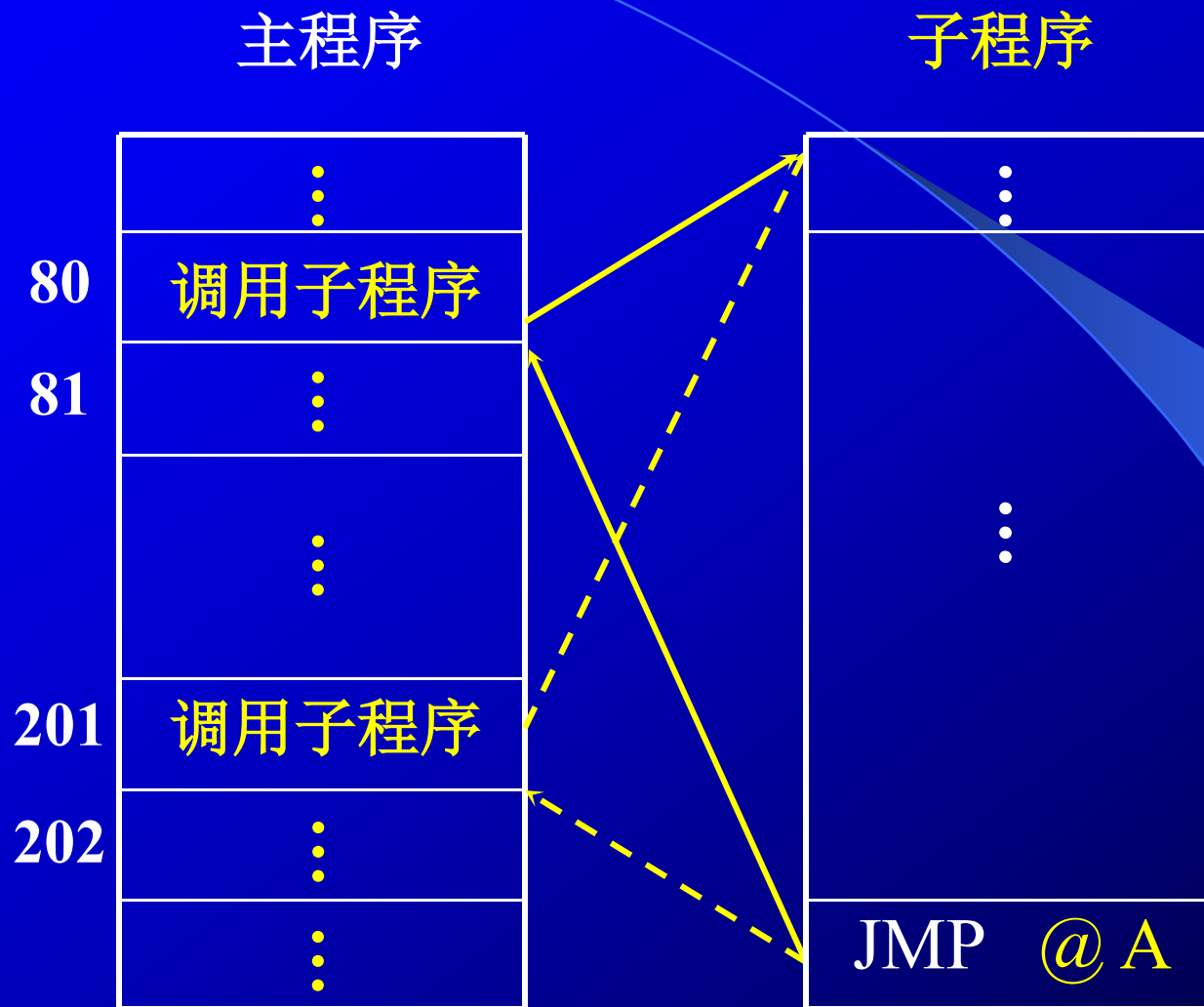


- 执行指令阶段 2 次访存
- 可扩大寻址范围
- 便于编制程序

多次访存

间接寻址编程举例

7.3



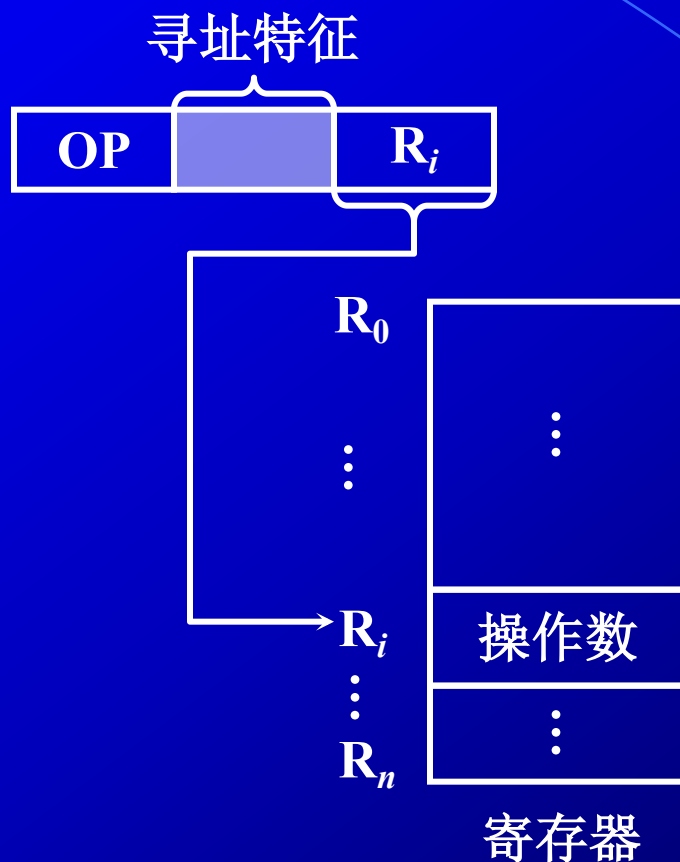
@ 间址特征



5. 寄存器寻址

7.3

$EA = R_i$ 有效地址即为寄存器编号



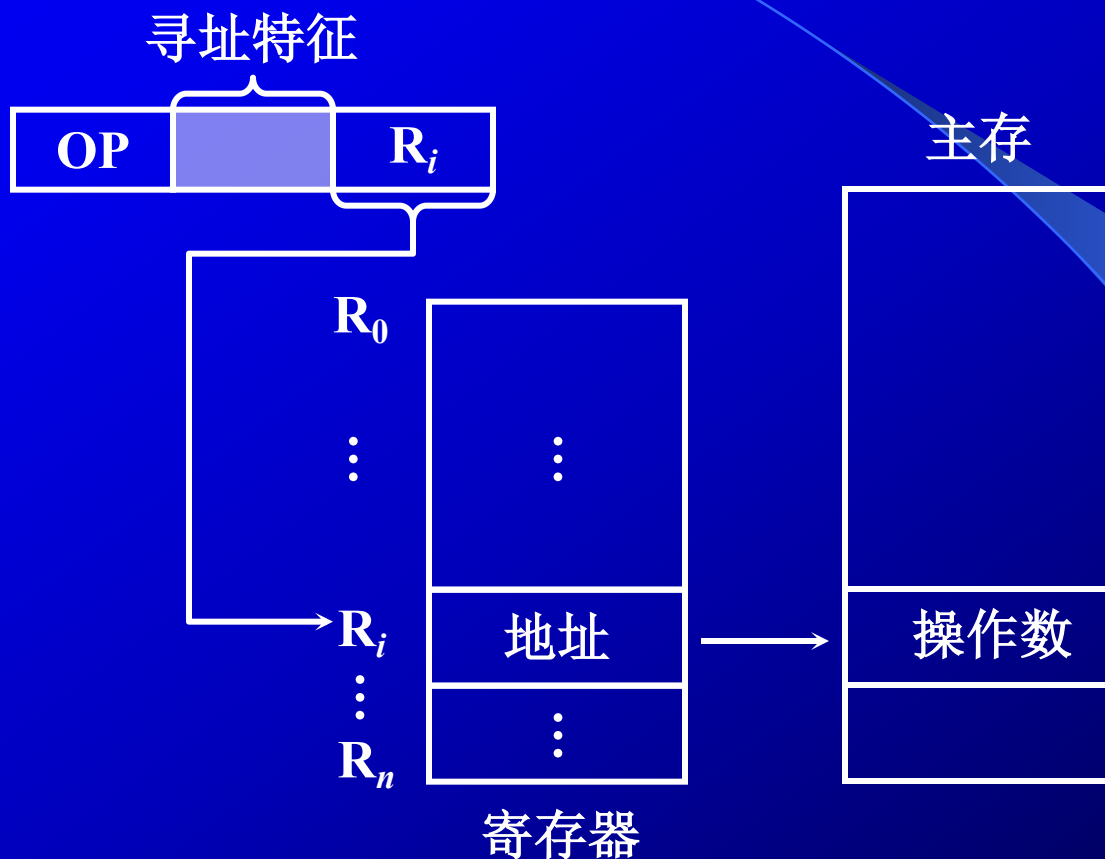
- 执行阶段不访存，只访问寄存器，执行速度快
- 寄存器个数有限，可缩短指令字长

6. 寄存器间接寻址

7.3

$$EA = (R_i)$$

有效地址在寄存器中



- 有效地址在寄存器中，操作数在存储器中，执行阶段访存
- 便于编制循环程序

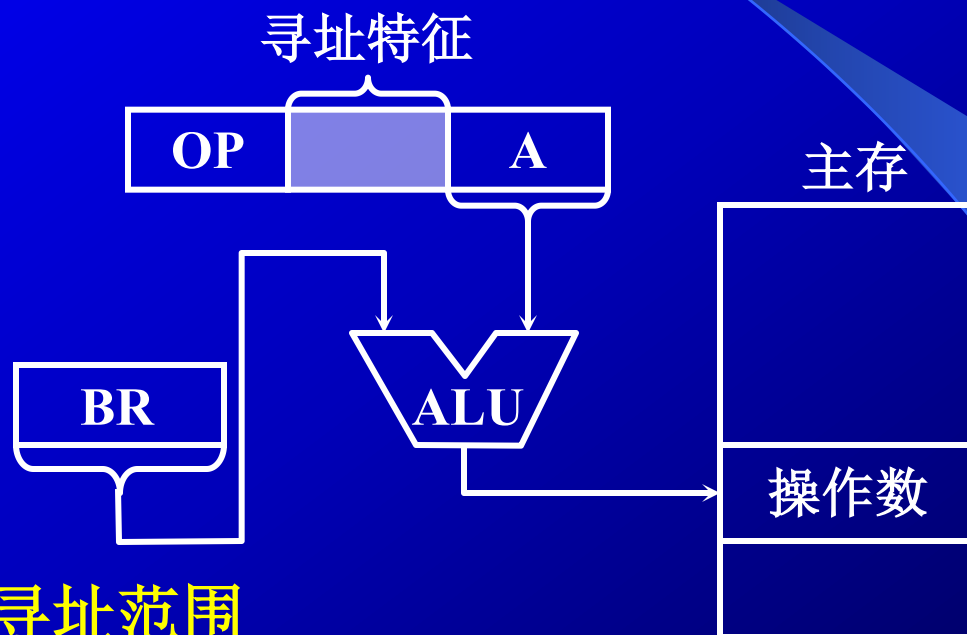
7. 基址寻址

7.3

(1) 采用专用寄存器作基址寄存器

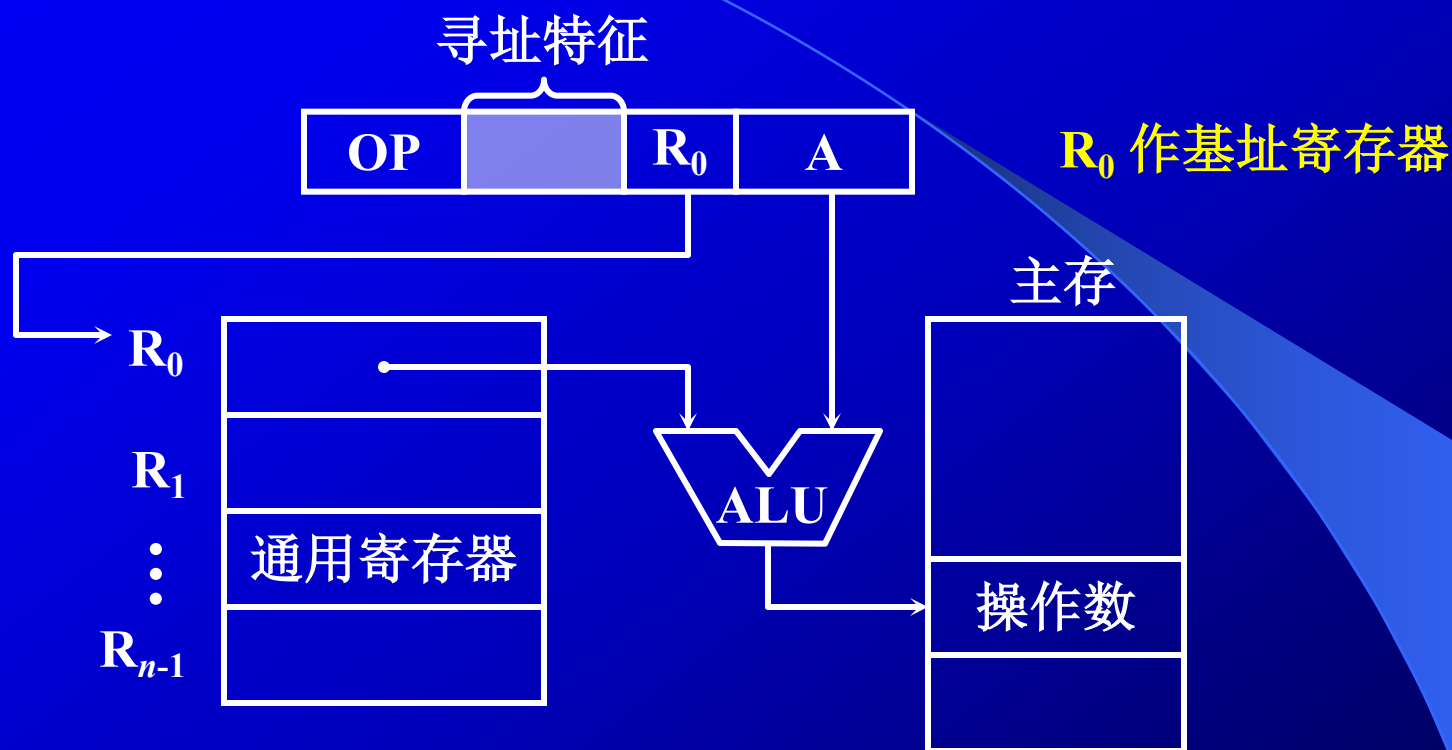
$$EA = (BR) + A$$

BR 为基址寄存器



- 可扩大寻址范围
- 有利于多道程序
- **BR** 内容由操作系统或管理程序确定
- 在程序的执行过程中 **BR** 内容不变，形式地址 **A** 可变

(2) 采用通用寄存器作基址寄存器



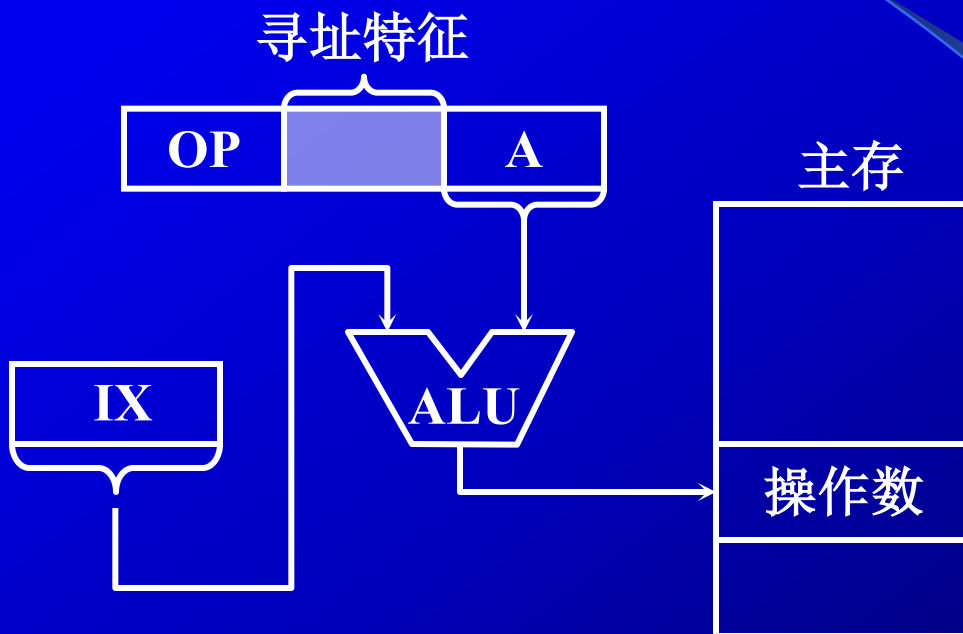
- 由用户指定哪个通用寄存器作为基址寄存器
- 基址寄存器的内容由操作系统确定
- 在程序的执行过程中 R_0 内容不变，形式地址 A 可变

8. 变址寻址

7.3

$EA = (IX) + A$ IX 为变址寄存器（专用）

通用寄存器也可以作为变址寄存器



- 可扩大寻址范围
- IX 的内容由用户给定
- 在程序的执行过程中 IX 内容可变，形式地址 A 不变
- 便于处理数组问题

例 设数据块首地址为 D ，求 N 个数的平均值 7.3

直接寻址

LDA D

ADD $D + 1$

ADD $D + 2$

⋮

ADD $D + (N - 1)$

DIV $\# N$

STA ANS

共 $N + 2$ 条指令

变址寻址

LDA $\# 0$

LDX $\# 0$ X 为变址寄存器

ADD X, D D 为形式地址

INX $(X) + 1 \rightarrow X$

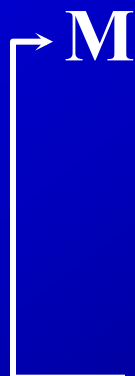
CPX $\# N$ (X) 和 $\# N$ 比较

BNE M 结果不为零则转

DIV $\# N$

STA ANS

共 8 条指令

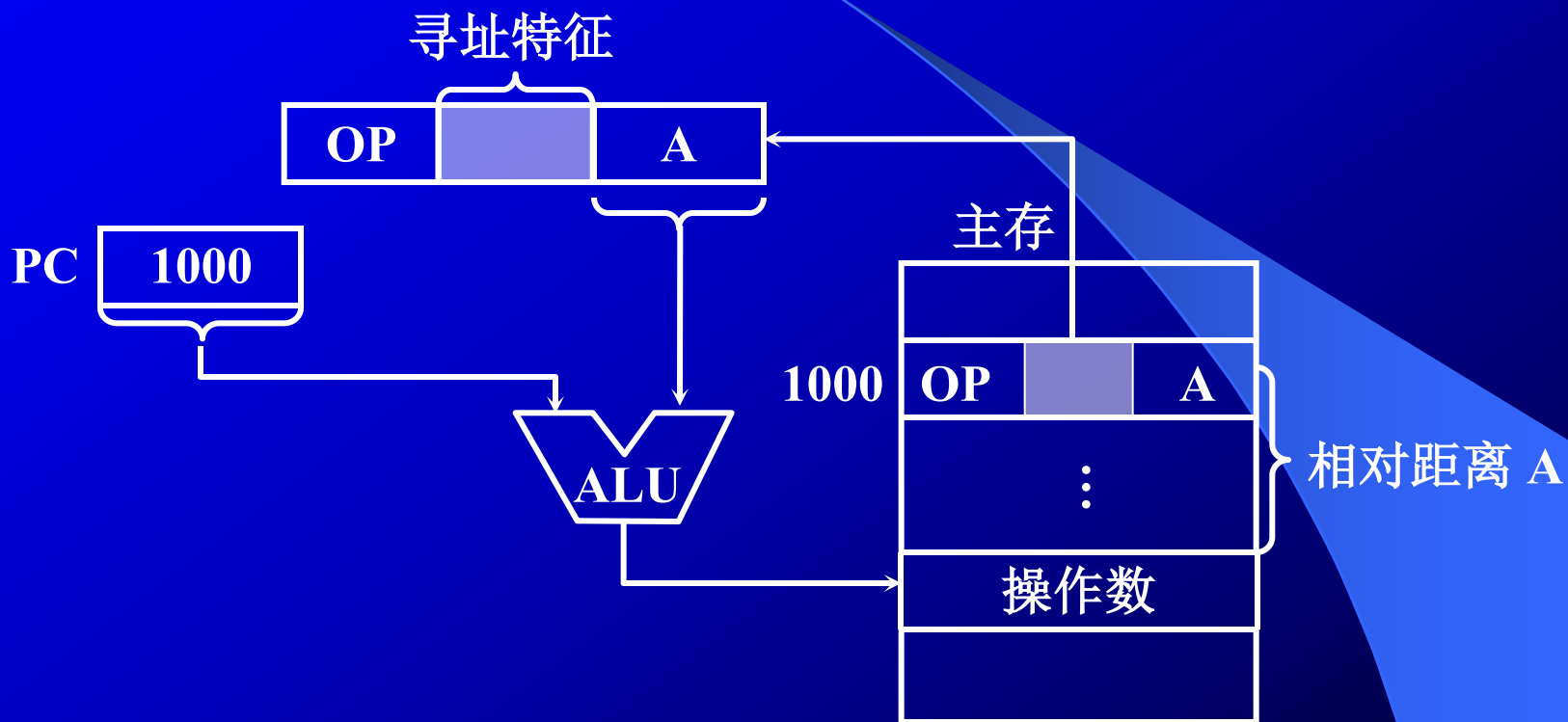


9. 相对寻址

7.3

$$EA = (PC) + A$$

A 是相对于当前指令的位移量（可正可负，补码）



- A 的位数决定操作数的寻址范围
- 程序浮动
- 广泛用于转移指令

(1) 相对寻址举例

7.3

	LDA	# 0	
	LDX	# 0	
→ M	ADD	X, D	
M+1	INX		
M+2	CPX	# N	✱ 相对寻址特征
— M+3	BNE	M → ✱ - 3	
	DIV	# N	
	STA	ANS	

M 随程序所在存储空间的位置不同而不同

而指令 **BNE ✱-3** 与指令 **ADD X, D** 相对位移量不变

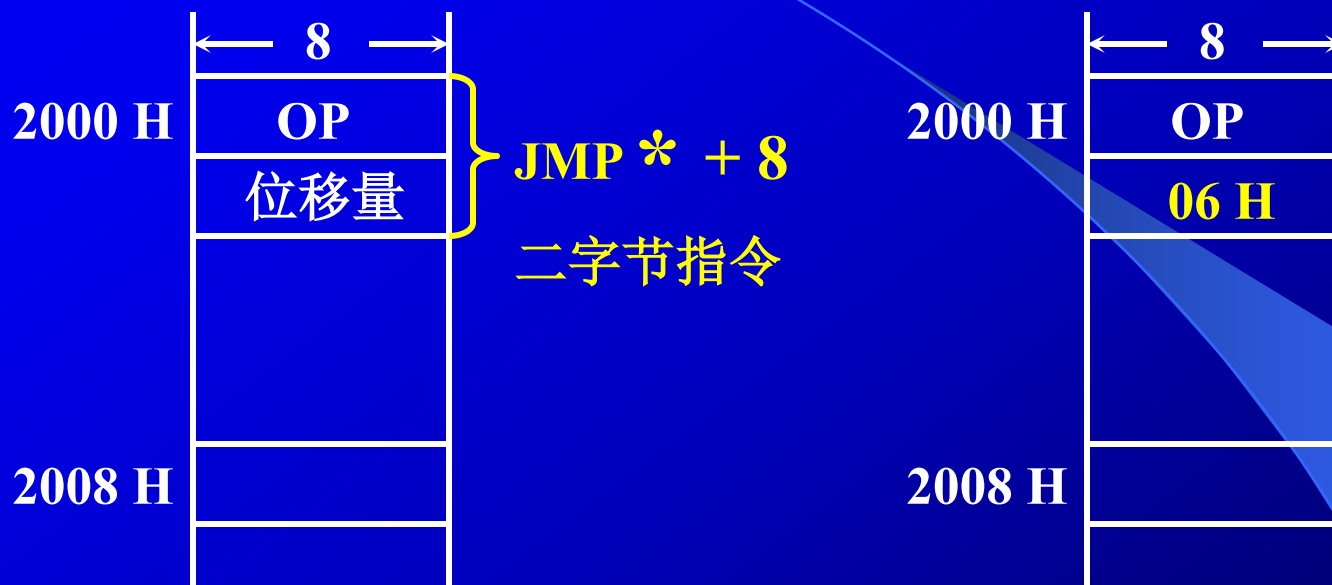
指令 **BNE ✱-3** 操作数的有效地址为

$$EA = (M+3) - 3 = M$$



(2) 按字节寻址的相对寻址举例

7.3



设 当前指令地址 **PC = 2000H**

转移后的目的地址为 **2008H**

因为 取出 **JMP * + 8** 后 **PC = 2002H**

故 **JMP * + 8** 指令 的第二字节为 **2008H - 2002H = 06H**



10. 堆栈寻址

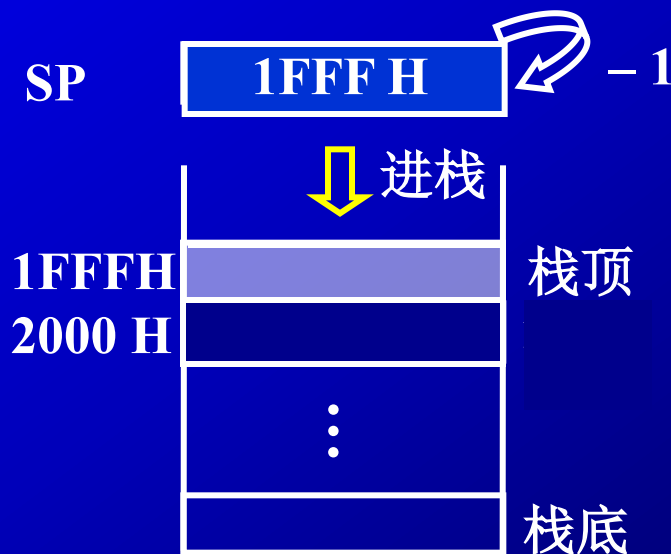
7.3

(1) 堆栈的特点

堆栈 { 硬堆栈 多个寄存器
 软堆栈 指定的存储空间

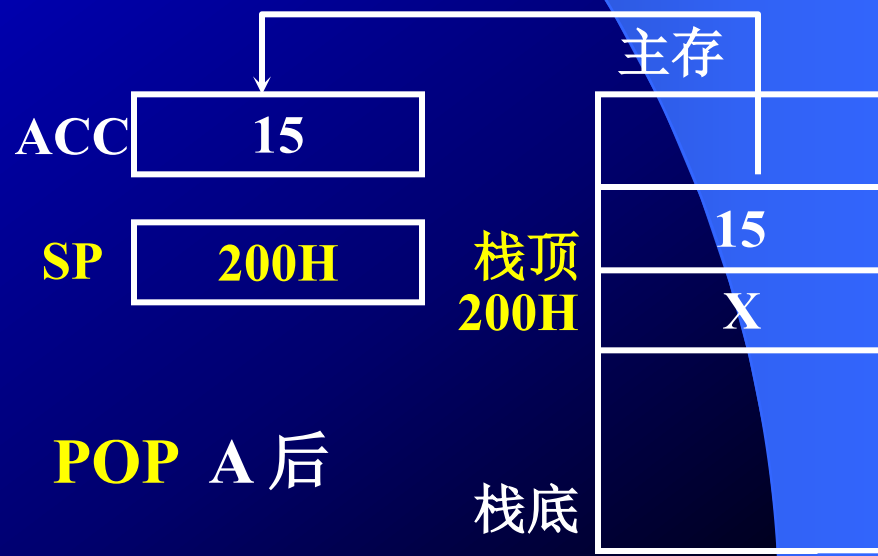
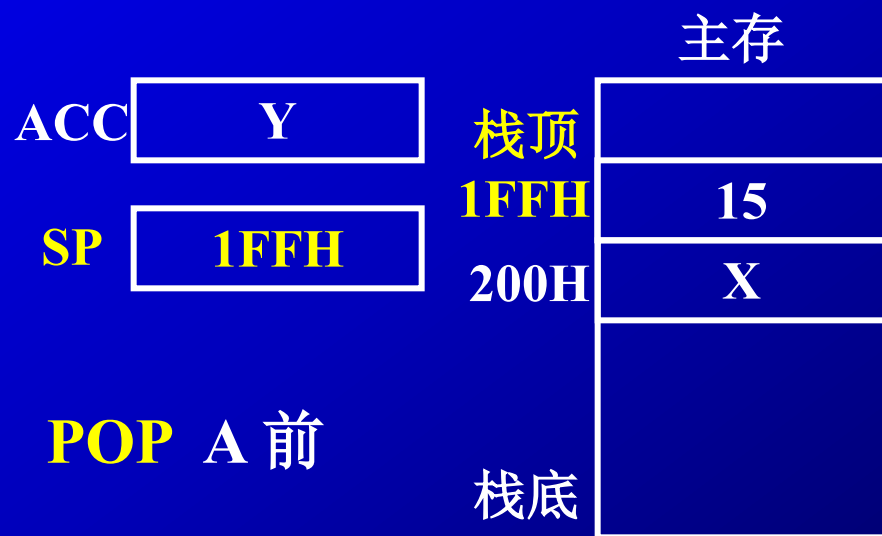
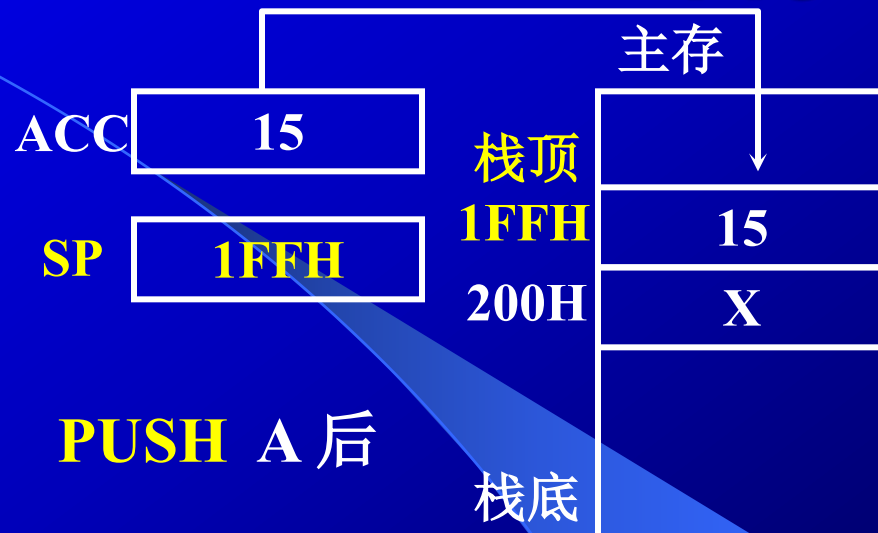
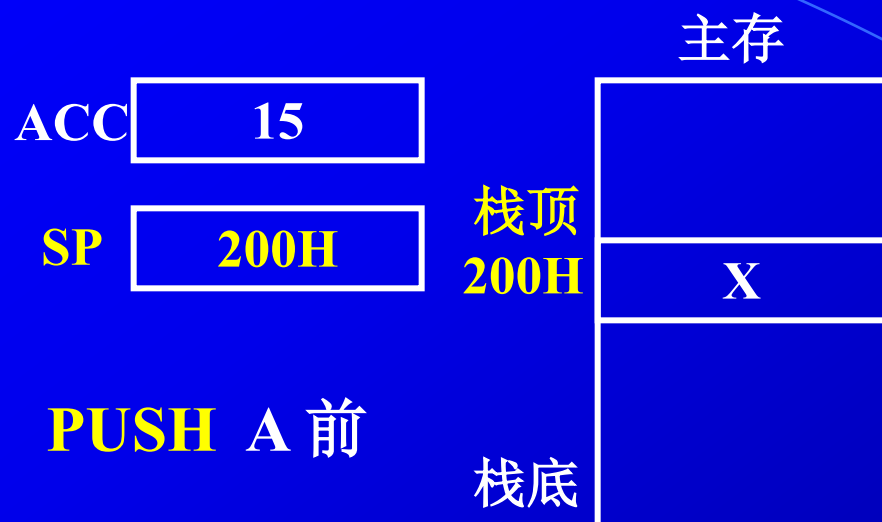
先进后出（一个入出口） 栈顶地址 由 **SP** 指出

进栈 $(SP) - 1 \rightarrow SP$ 出栈 $(SP) + 1 \rightarrow SP$



(2) 堆栈寻址举例

7.3



(3) SP 的修改与主存编址方法有关

7.3

① 按字编址

进栈 $(SP) - 1 \longrightarrow SP$

出栈 $(SP) + 1 \longrightarrow SP$

② 按字节编址

存储字长 16 位 进栈 $(SP) - 2 \longrightarrow SP$

出栈 $(SP) + 2 \longrightarrow SP$

存储字长 32 位 进栈 $(SP) - 4 \longrightarrow SP$

出栈 $(SP) + 4 \longrightarrow SP$



7.4 指令格式举例

一、设计指令格式时应考虑的各种因素

1. 指令系统的 **兼容性** （向上兼容）

2. 其他因素

操作类型

包括指令个数及操作的难易程度

数据类型

确定哪些数据类型可参与操作

指令格式

指令字长是否固定

操作码位数、是否采用扩展操作码技术，
地址码位数、地址个数、寻址方式类型

寻址方式

指令寻址、操作数寻址

寄存器个数

寄存器的多少直接影响指令的执行时间



二、指令格式举例

7.4

1. PDP-8 指令字长固定 12 位

访存类指令



I/O 类指令



寄存器类指令



采用扩展操作码技术



2. PDP – 11

7.4

指令字长有 16 位、32 位、48 位三种



16

零地址 (16 位)

扩展操作码技术



10

6

一地址 (16 位)



4

6

6

二地址 R – R (16 位)



10

6

16

二地址 R – M (32 位)



4

6

6

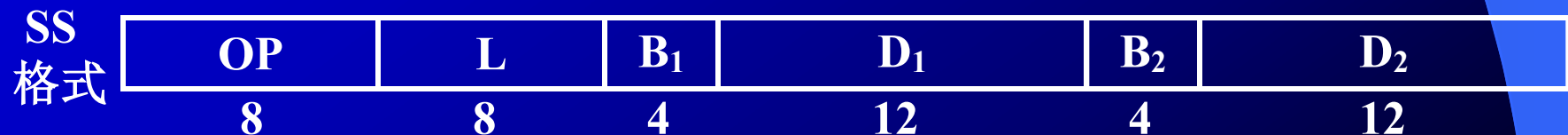
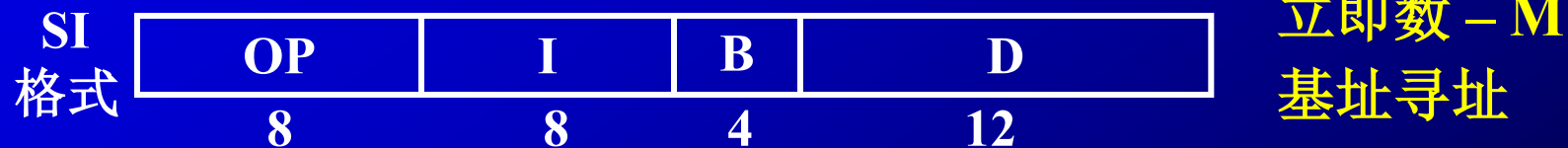
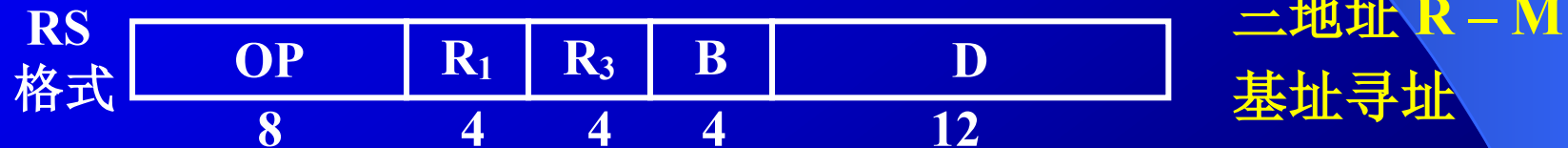
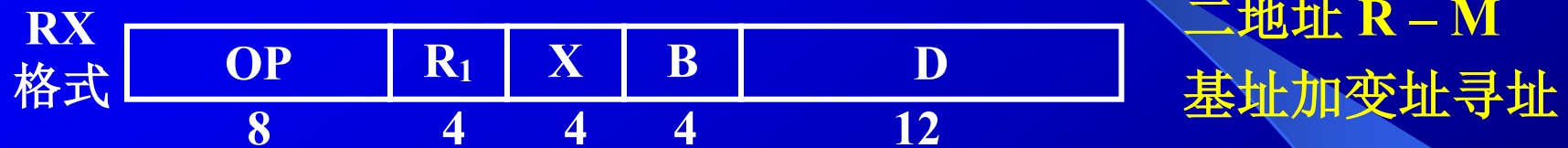
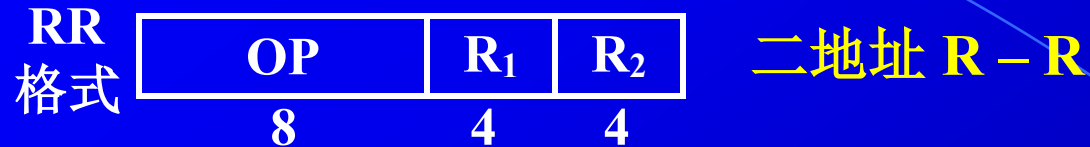
16

16

二地址 M – M (48 位)

3. IBM 360

7.4



4. Intel 8086

7.4

(1) 指令字长 1~6 个字节

INC AX 1 字节

MOV WORD PTR[0204], 0138H 6 字节

(2) 地址格式

零地址 NOP 1 字节

一地址 CALL 段间调用 5 字节

CALL 段内调用 3 字节

二地址 ADD AX, BX 2 字节 寄存器 – 寄存器

ADD AX, 3048H 3 字节 寄存器 – 立即数

ADD AX, [3048H] 4 字节 寄存器 – 存储器



7.5 RISC 技术

一、RISC 的产生和发展

RISC (Reduced Instruction Set Computer)

CISC (Complex Instruction Set Computer)

80 — 20 规律

—— RISC技术

- 典型程序中 80% 的语句仅仅使用处理机中 20% 的指令
- 执行频度高的简单指令，因复杂指令的存在，执行速度无法提高
- ？ 能否用 20% 的简单指令组合不常用的 80% 的指令功能



二、RISC 的主要特征

- 选用使用频度较高的一些简单指令，复杂指令的功能由简单指令来组合
- 指令长度固定、指令格式种类少、寻址方式少
- 只有 **LOAD / STORE** 指令访存
- CPU 中有多个通用寄存器
- 采用流水技术 一个时钟周期 内完成一条指令
- 采用组合逻辑实现控制器
- 采用优化的编译程序

三、CISC 的主要特征

- 系统指令 复杂庞大，各种指令使用频度相差大
- 指令 长度不固定、指令格式种类多、寻址方式多
- 访存 指令 不受限制
- CPU 中设有 专用寄存器
- 大多数指令需要 多个时钟周期 执行完毕
- 采用 微程序 控制器
- 难以 用 优化编译 生成高效的代码



四、RISC和CISC 的比较

1. RISC更能 充分利用 VLSI 芯片的面积
2. RISC 更能 提高计算机运算速度
指令数、指令格式、寻址方式少，
通用 寄存器多，采用 组合逻辑，
便于实现 指令流水
3. RISC 便于设计，可 降低成本，提高 可靠性
4. RISC 有利于编译程序代码优化
5. RISC 不易 实现 指令系统兼容



第 8 章 CPU 的结构和功能

8.1 CPU 的结构

8.2 指令周期

8.3 指令流水

8.4 中断系统



8.1 CPU 的结构

一、CPU 的功能

1. 控制器的功能

取指令

指令控制

分析指令

操作控制

执行指令，发出各种操作命令

时间控制

控制程序输入及结果的输出

总线管理

处理中断

处理异常情况和特殊请求

数据加工

2. 运算器的功能

实现算术运算和逻辑运算



二、CPU 结构框图

8.1

1. CPU 与系统总线

指令控制

PC IR

操作控制

CU 时序电路

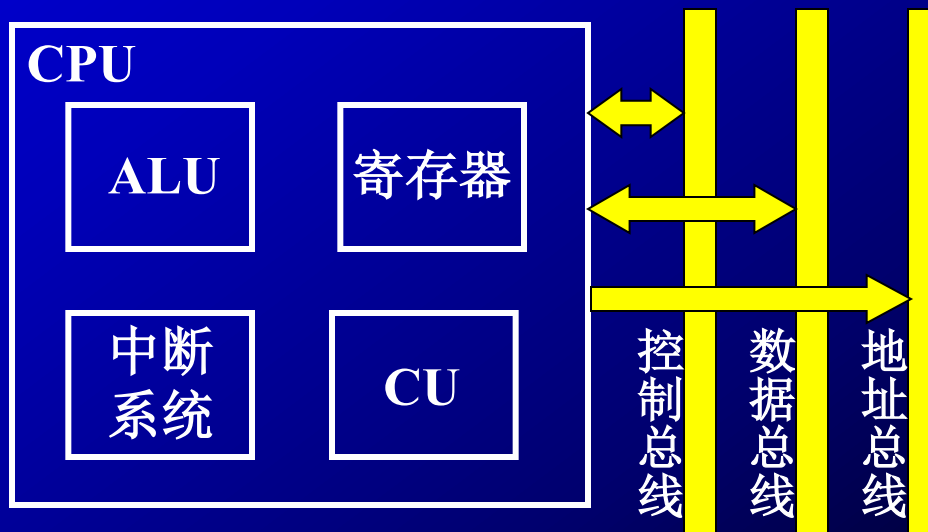
时间控制

数据加工

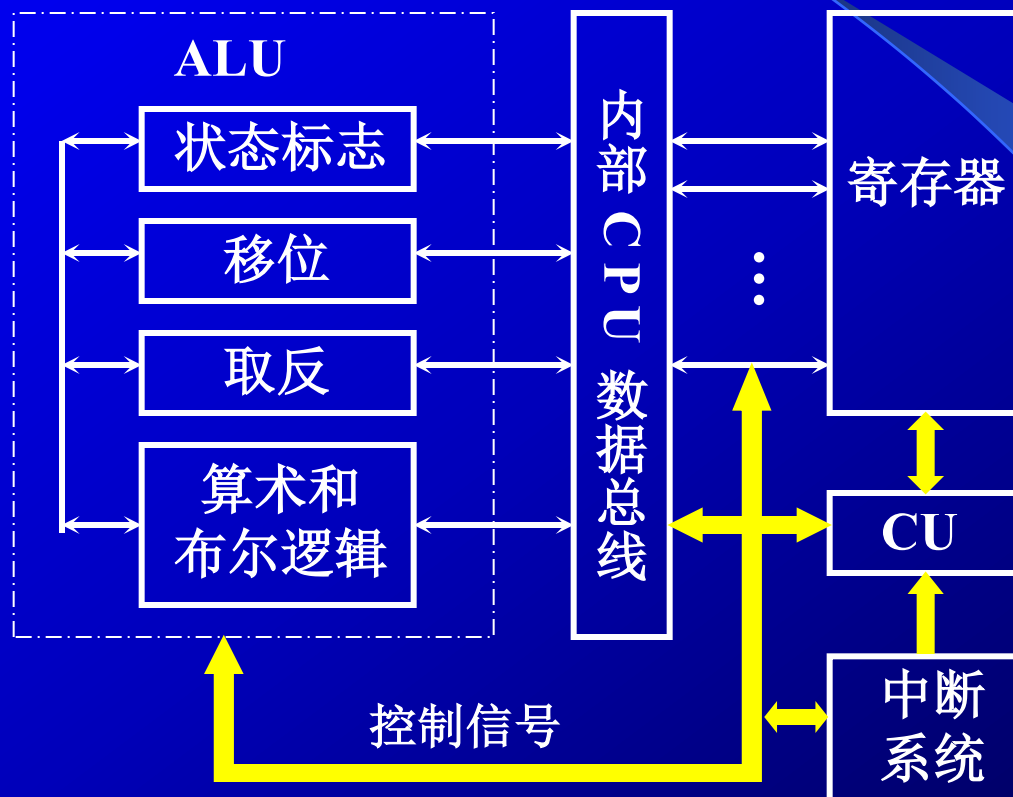
ALU 寄存器

处理中断

中断系统



2. CPU 的内部结构



三、CPU 的寄存器

8.1

1. 用户可见寄存器

(1) 通用寄存器

存放操作数

可作 某种寻址方式所需的 专用寄存器

(2) 数据寄存器

存放操作数（满足各种数据类型）

两个寄存器拼接存放双倍字长数据

(3) 地址寄存器

存放地址，其位数应满足最大的地址范围

用于特殊的寻址方式 段基值 栈指针

(4) 条件码寄存器

存放条件码，可作程序分支的依据

如 正、负、零、溢出、进位等



2. 控制和状态寄存器

(1) 控制寄存器

PC → **MAR** → **M** → **MDR** → **IR**

控制 CPU 操作

其中 **MAR**、**MDR**、**IR**

用户不可见

PC

用户可见

(2) 状态寄存器

状态寄存器

存放条件码

PSW 寄存器

存放程序状态字

3. 举例

Z8000

8086

MC 68000



四、控制单元 CU 和中断系统

1. CU 产生全部指令的微操作命令序列

组合逻辑设计

硬连线逻辑

参见 第 4 篇

微程序设计

存储逻辑

2. 中断系统

参见 8.4 节

五、ALU

参见 第 6 章



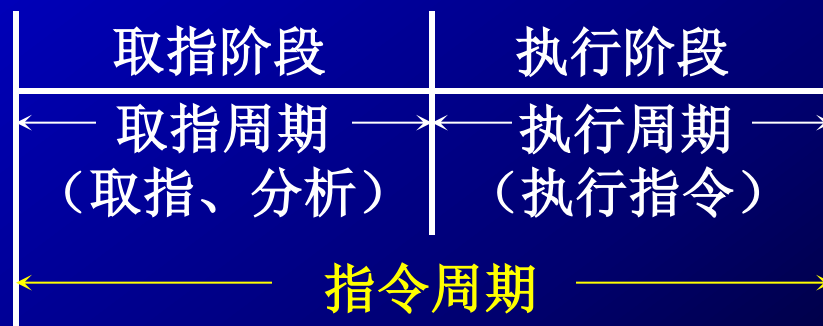
8.2 指令周期

一、指令周期的基本概念

1. 指令周期

取出并执行一条指令所需的全部时间

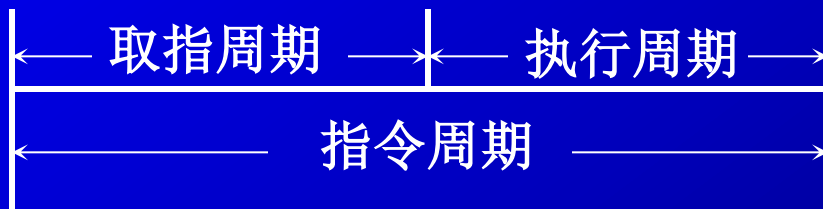
完成一条指令 { 取指、分析 取指周期
 执行 执行周期



2. 每条指令的指令周期不同



NOP



ADD mem

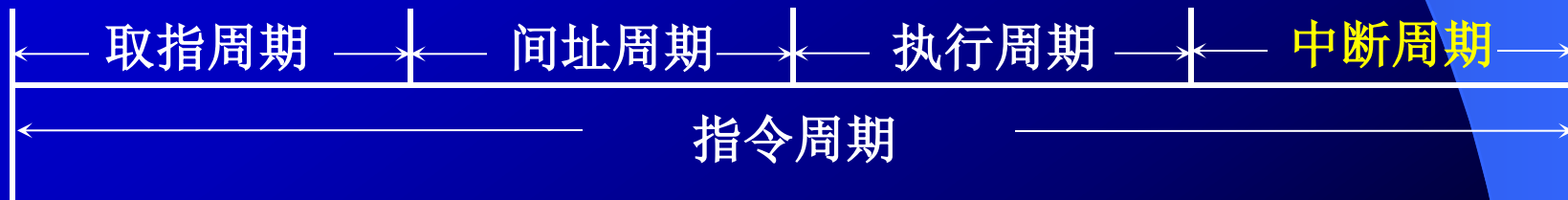


MUL mem

3. 具有间接寻址的指令周期

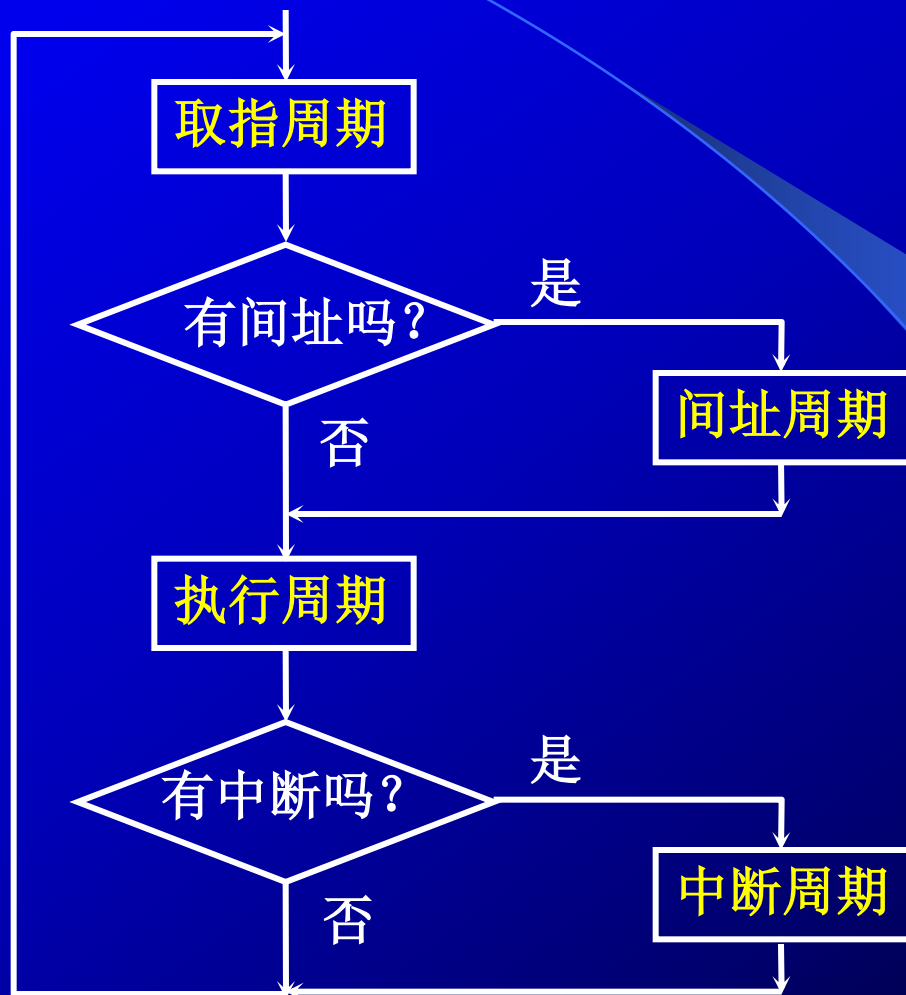


4. 带有中断周期的指令周期



5. 指令周期流程

8.2



6. CPU 工作周期的标志

CPU 访存有四种性质

取 指令

取指周期

取 地址

间址周期

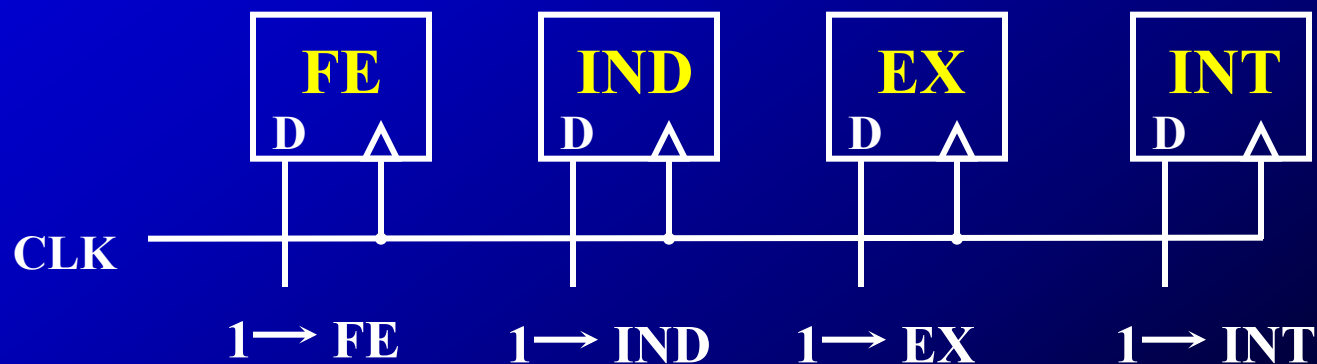
取 操作数

执行周期

存 程序断点

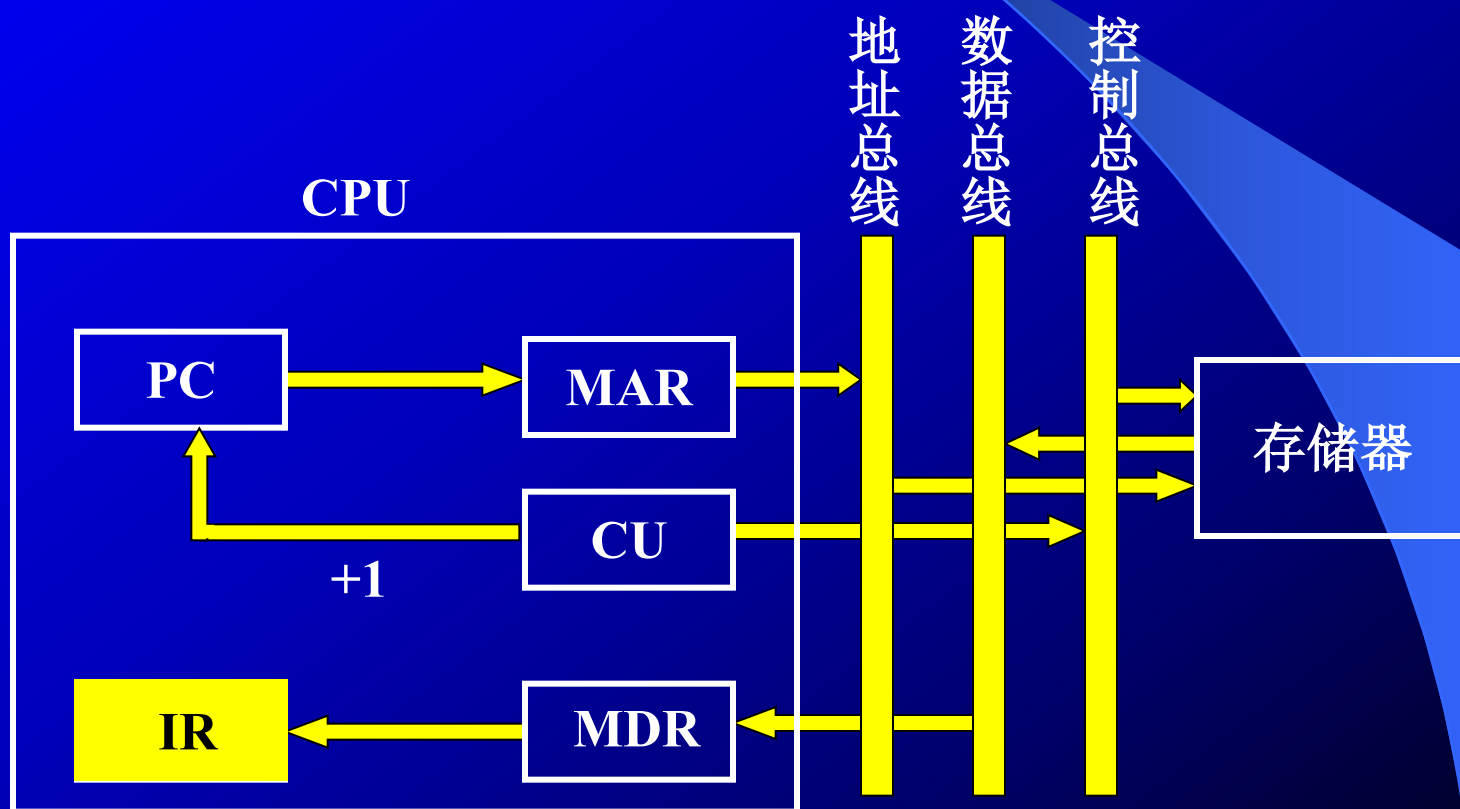
中断周期

CPU 的
4个工作周期

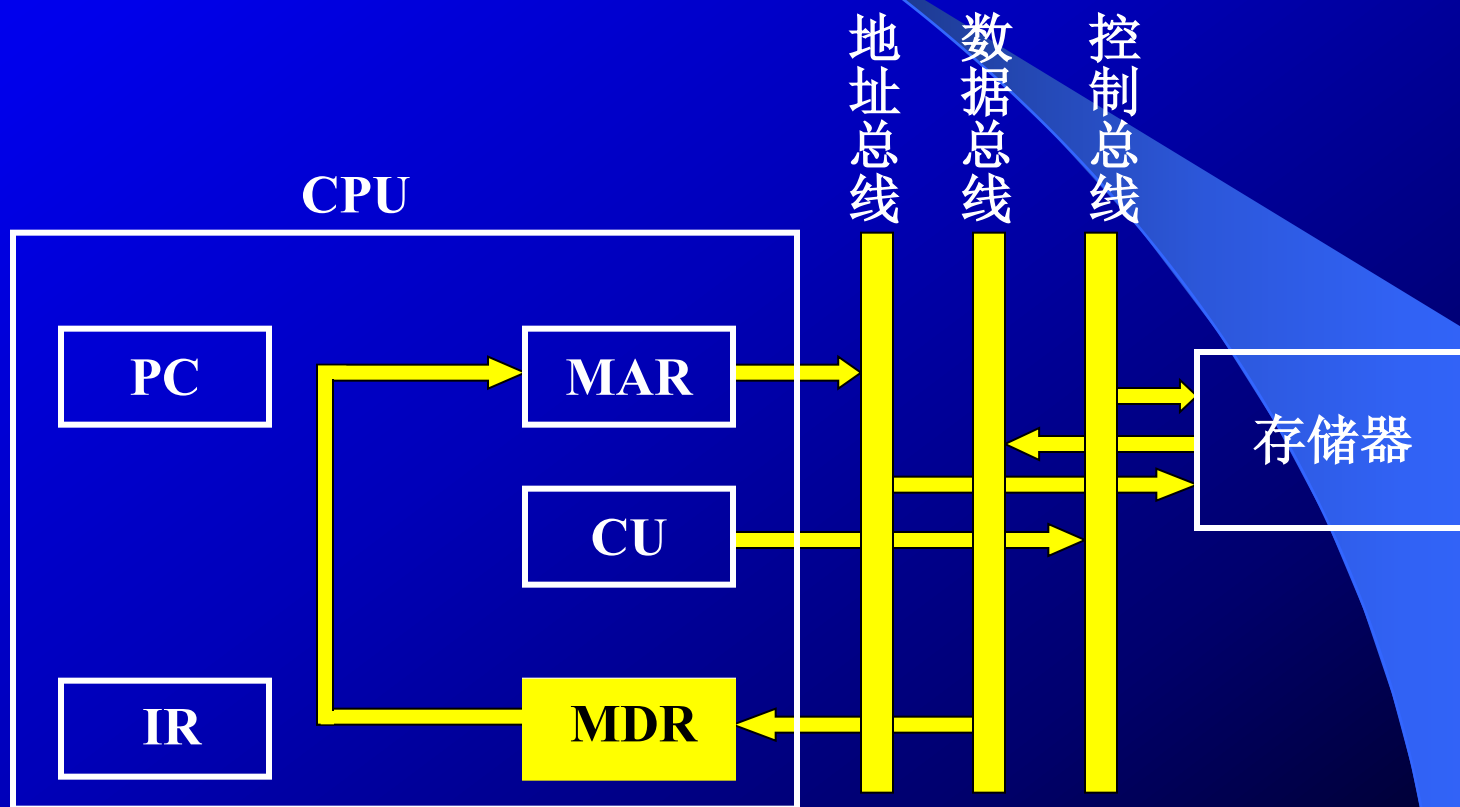


二、指令周期的数据流

1. 取指周期数据流



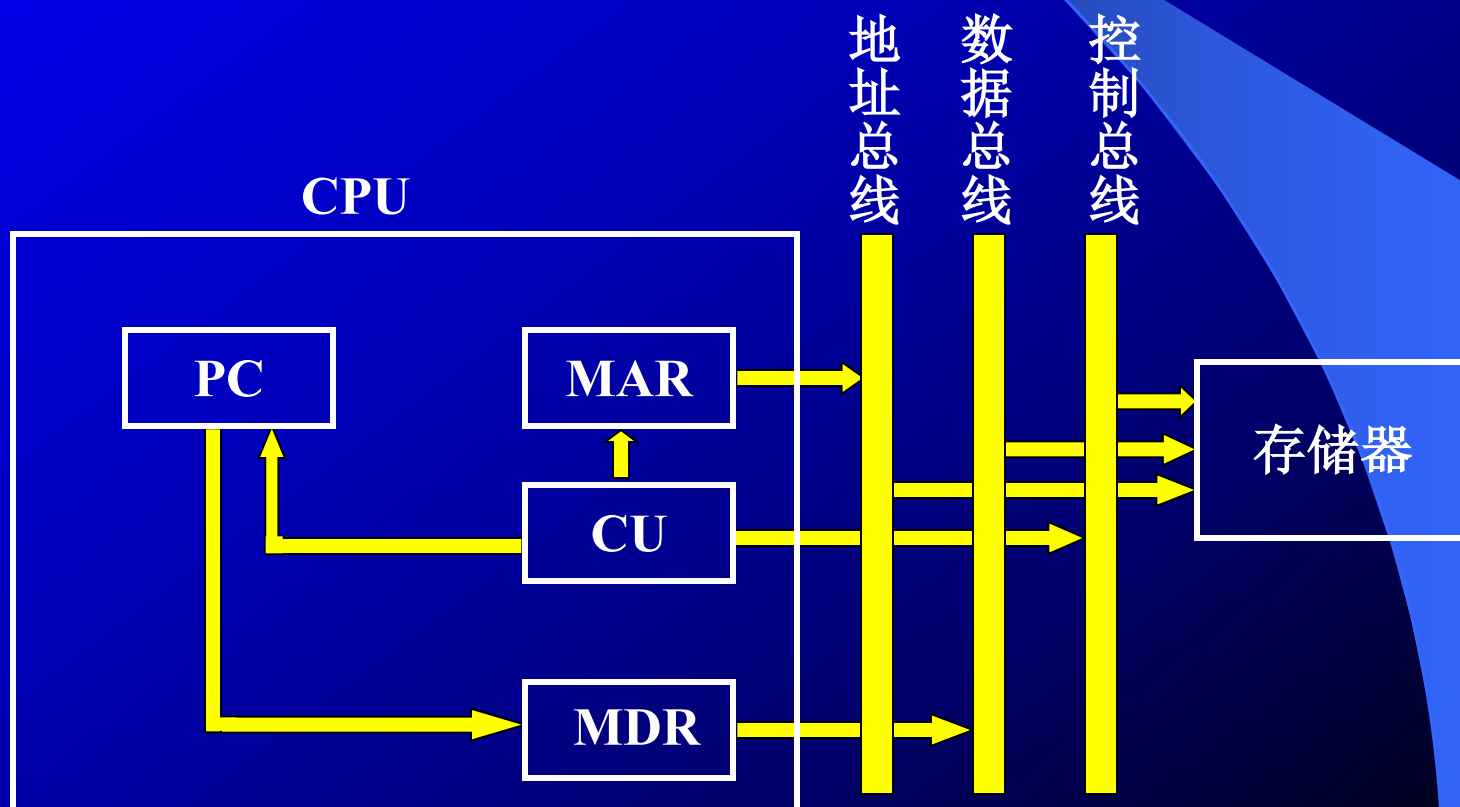
2. 间址周期数据流



3. 执行周期数据流

不同指令的执行周期数据流不同

4. 中断周期数据流



8.3 指令流水

一、如何提高机器速度

1. 提高访存速度

高速芯片

Cache

多体并行

2. 提高 I/O 和主机之间的传送速度

中断

DMA

通道

I/O 处理机

多总线

3. 提高运算器速度

高速芯片

改进算法

快速进位链

• 提高整机处理能力

高速器件

改进系统结构，开发系统的并行性



二、系统的并行性

8.3

1. 并行的概念

并行 { **并发** 两个或两个以上事件在 **同一时间段** 发生
同时 两个或两个以上事件在 **同一时刻** 发生
时间上互相重叠

2. 并行性的等级

过程级（程序、进程）	粗粒度	软件实现
指令级（指令之间） （指令内部）	细粒度	硬件实现



三、指令流水原理

8.3

1. 指令的串行执行



取指令 取指令部件 完成 总有一个部件 空闲
执行指令 执行指令部件 完成

2. 指令的二级流水



指令预取

若 取指 和 执行 阶段时间上 完全重叠

指令周期 减半 速度提高 1 倍



3. 影响指令流水效率加倍的因素

8.3

(1) 执行时间 > 取指时间



(2) 条件转移指令 对指令流水的影响

必须等 **上条** 指令执行结束，才能确定 **下条** 指令的地址，

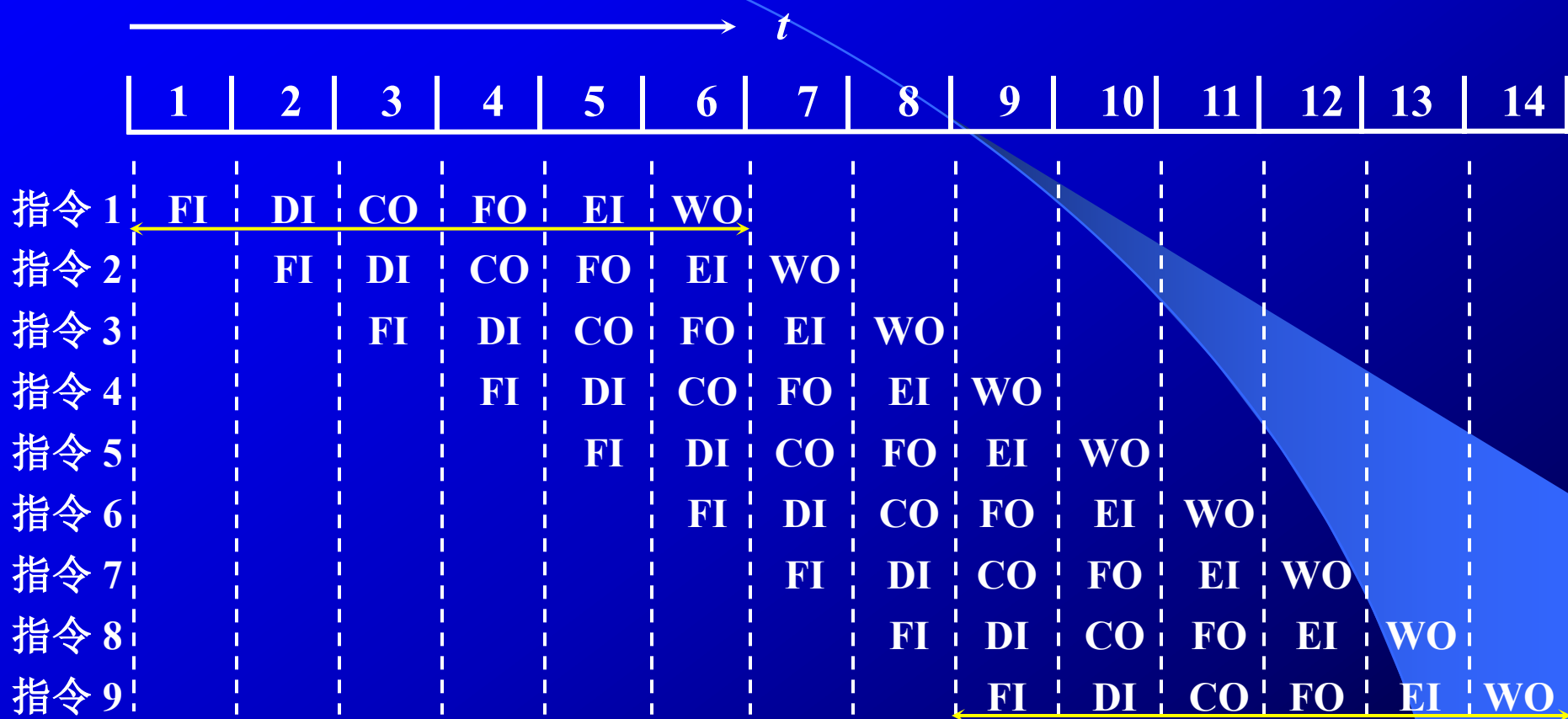
造成时间损失

猜测法

解决办法 ?

4. 指令的六级流水

8.3



完成一条指令
串行执行
六级流水

6 个时间单位

$6 \times 9 = 54$ 个时间单位

14 个时间单位



四、影响指令流水线性能的因素

8.3

1. 结构相近指令之间出现某种关键部件产生资源冲突

使指令流水出现停顿，影响流水线效率

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
指令 1	FI	DI	CO	FO	EI	WO								
指令 2		FI	DI	CO	FO	EI	WO							
指令 3			FI	DI	CO	FO	EI	WO						
指令 4				FI	DI	CO	FO	EI	WO					
指令 5					FI	DI	CO	FO	EI	WO				
指令 6						FI	DI	CO	FO	EI	WO			
指令 7							FI	DI	CO	FO	EI	WO		
指令 8								FI	DI	CO	FO	EI	WO	
指令 9									FI	DI	CO	FO	EI	WO

解决办法

- 停顿
- 指令 1 与指令 4 冲突
- 指令 2 与指令 5 冲突
- 指令 3 与指令 6 冲突
- 指令存储器和数据存储器分开
- 指令预取技术（适用于访存周期短的情况）

2. 数据相关

8.3

不同指令因重叠操作，可能改变操作数的 读/写 访问顺序

- 写后读相关 (RAW)

SUB R_1 , R_2 , R_3 ; $(R_2) - (R_3) \rightarrow R_1$

ADD R_4 , R_5 , R_1 ; $(R_5) + (R_1) \rightarrow R_4$

- 读后写相关 (WAR)

STA M, R_2 ; $(R_2) \rightarrow M$ 存储单元

ADD R_2 , R_4 , R_5 ; $(R_4) + (R_5) \rightarrow R_2$

- 写后写相关 (WAW)

MUL R_3 , R_2 , R_1 ; $(R_2) \times (R_1) \rightarrow R_3$

SUB R_3 , R_4 , R_5 ; $(R_4) - (R_5) \rightarrow R_3$

解决办法

- 后推法

- 采用 旁路技术



3. 控制相关

8.3

由转移指令引起



A diagram showing a jump from the **BNE** instruction to the **ADD** instruction. A line starts from the left of the **BNE** instruction, goes up, then left, then down, ending with an arrow pointing to the **ADD** instruction.

	LDA	# 0
	LDX	# 0
→ M	ADD	X, D
	INX	
	CPX	# N
	BNE	M
	DIV	# N
	STA	ANS

BNE 指令必须等
CPX 指令的结果
才能判断出
是转移
还是顺序执行



3. 控制相关

8.3

设 指令3 是转移指令



五、流水线性能

8.3

1. 吞吐率

单位时间内 流水线所完成指令 或 输出结果 的数量

设 m 段的流水线各段时间为 Δt

- 最大吞吐率

$$T_{pmax} = \frac{1}{\Delta t}$$

- 实际吞吐率

连续处理 n 条指令的吞吐率为

$$T_p = \frac{n}{m \cdot \Delta t + (n-1) \cdot \Delta t}$$



2. 加速比 S_p

8.3

m 段的流水线的速度与等功能的非流水线的速度之比
设流水线各段时间为 Δt

完成 n 条指令在 m 段流水线上共需

$$T = m \cdot \Delta t + (n-1) \cdot \Delta t$$

完成 n 条指令在等效的非流水线上共需

$$T' = nm \cdot \Delta t$$

则

$$S_p = \frac{nm \cdot \Delta t}{m \cdot \Delta t + (n-1) \cdot \Delta t} = \frac{nm}{m + n - 1}$$



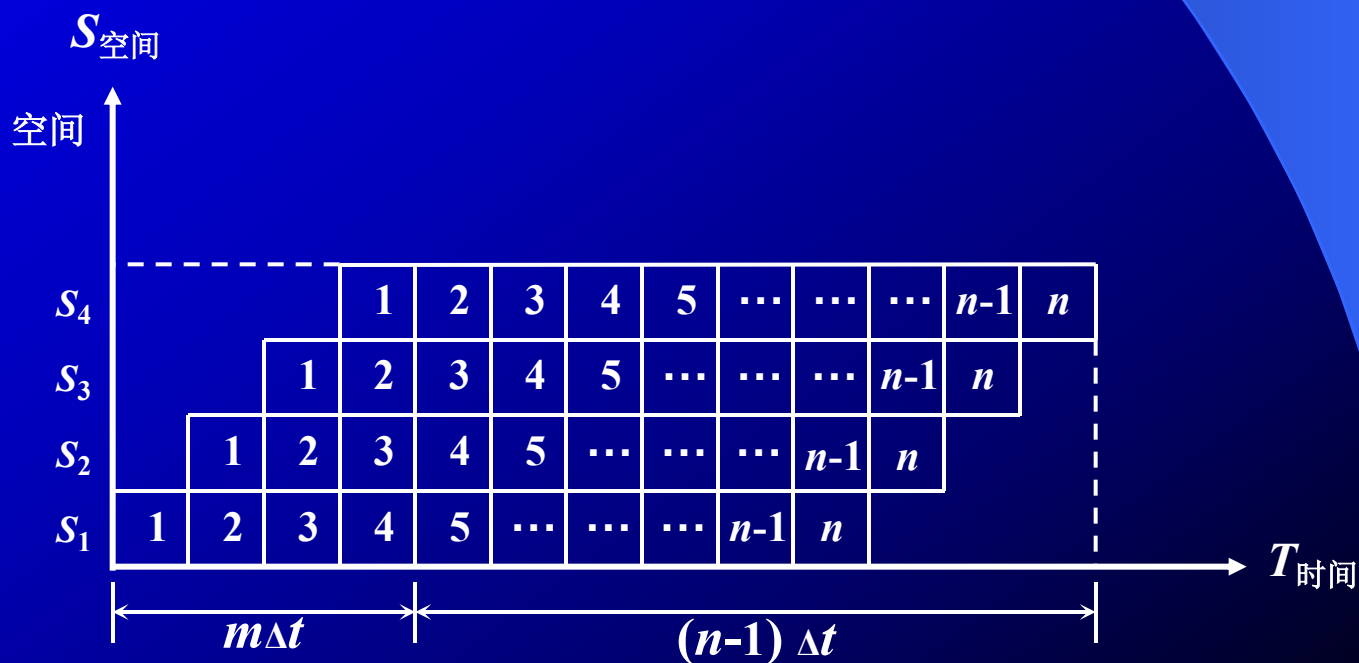
3. 效率

8.3

流水线中各功能段的 **利用率**

由于流水线有 **建立时间** 和 **排空时间**

因此各功能段的 **设备不可能** 一直 处于 **工作** 状态



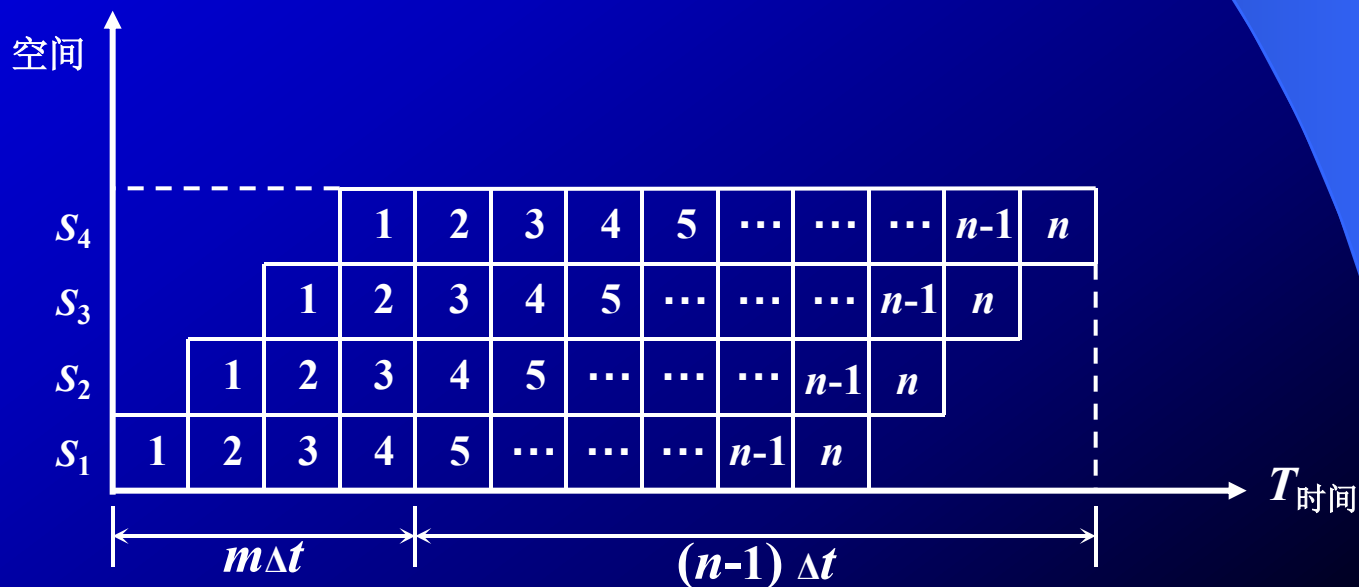
3. 效率

8.3

流水线中各功能段的 **利用率**

效率 = $\frac{\text{流水线各段处于工作时间的时空区}}{\text{流水线中各段总的时空区}}$

$$= \frac{mn\Delta t}{m(m+n-1)\Delta t}$$



六、流水线的多发技术

8.3

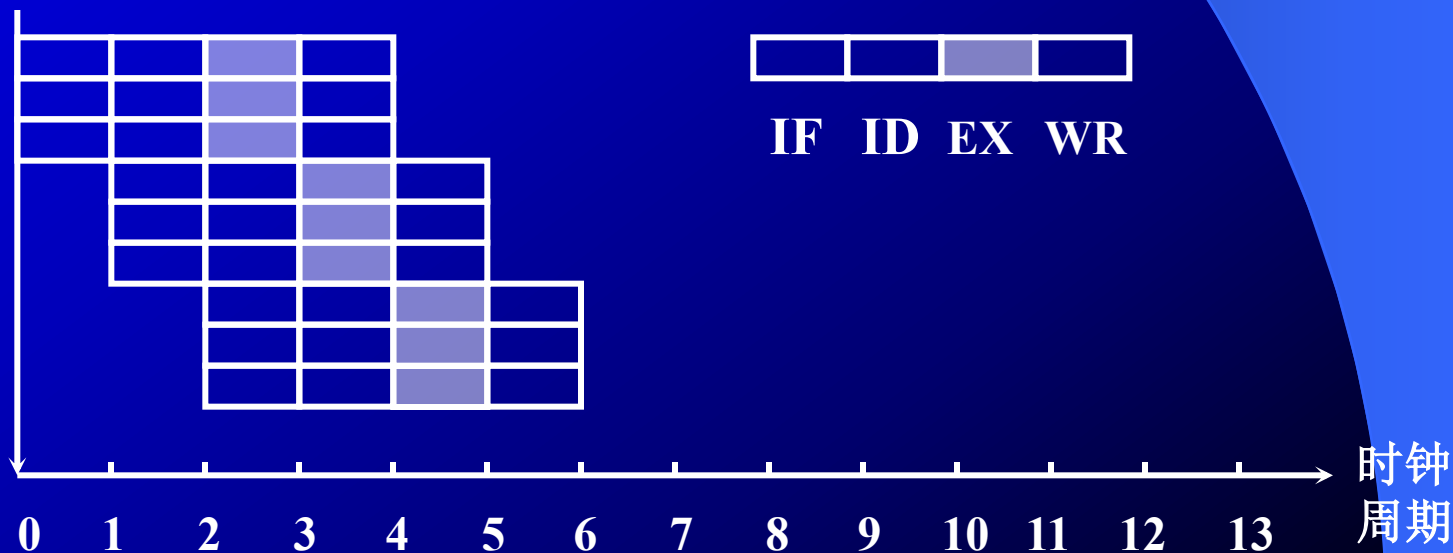
1. 超标量技术

- 每个时钟周期内可 **并发多条独立指令**
配置多个功能部件

- **不能调整** 指令的 **执行顺序**

通过编译优化技术，把可并行执行的指令搭配起来

指令序列



2. 超流水线技术

8.3

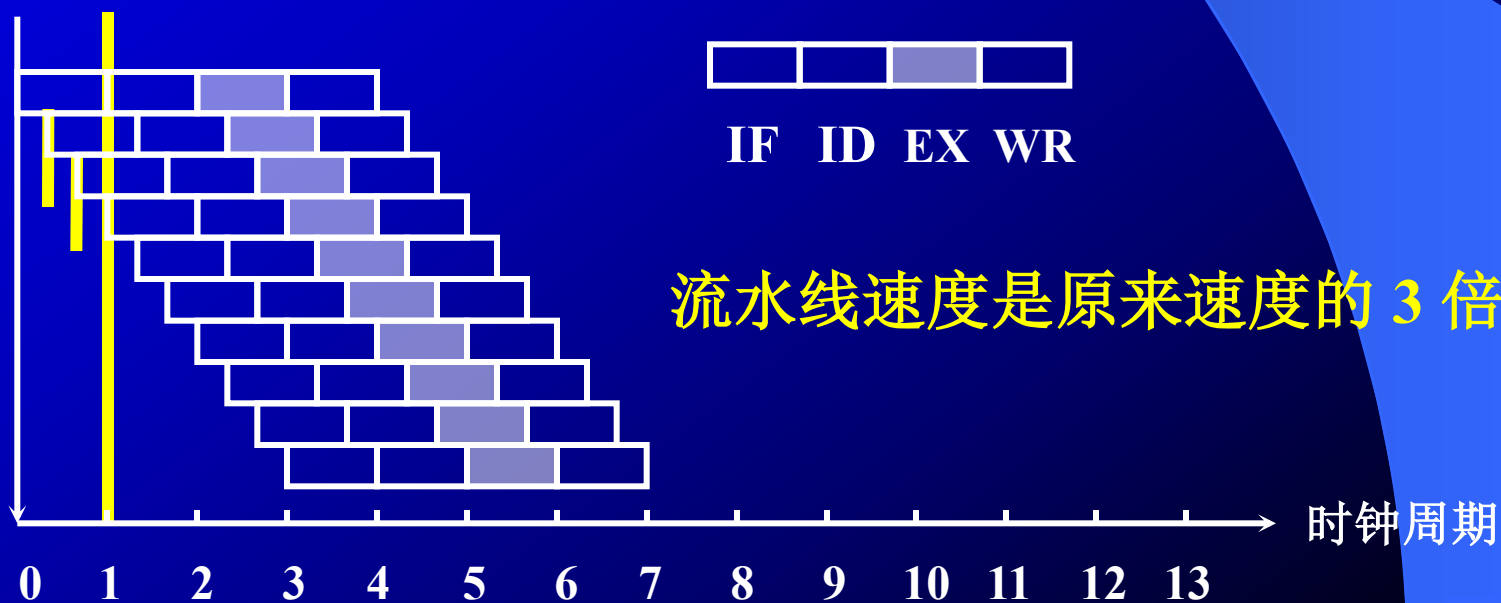
- 在一个时钟周期内再分段（3段）

在一个时钟周期内 一个功能部件使用多次（3次）

- 不能调整 指令的执行顺序

靠编译程序解决优化问题

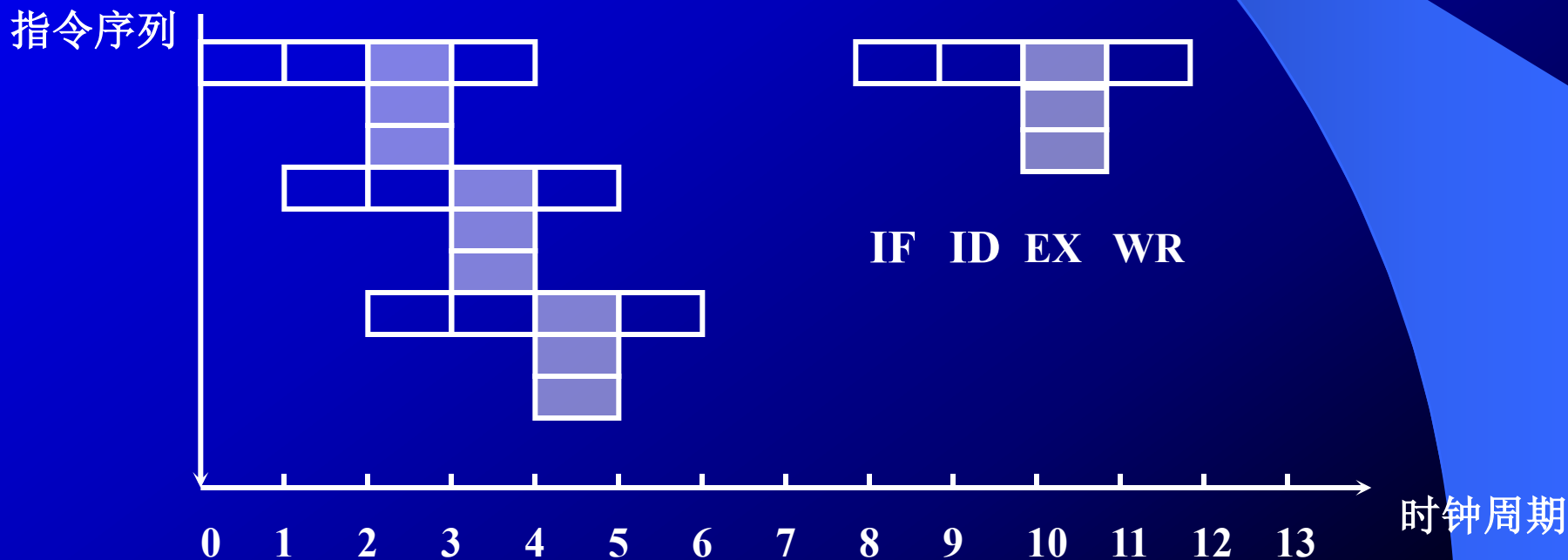
指令序列



3. 超长指令字技术

8.3

- 由编译程序 **挖掘** 出指令间 **潜在的** **并行性**，
将 **多条** 能 **并行操作** 的指令组合成一条
具有 **多个操作码字段** 的 **超长指令字**（可达几百位）
- 采用 **多个处理部件**

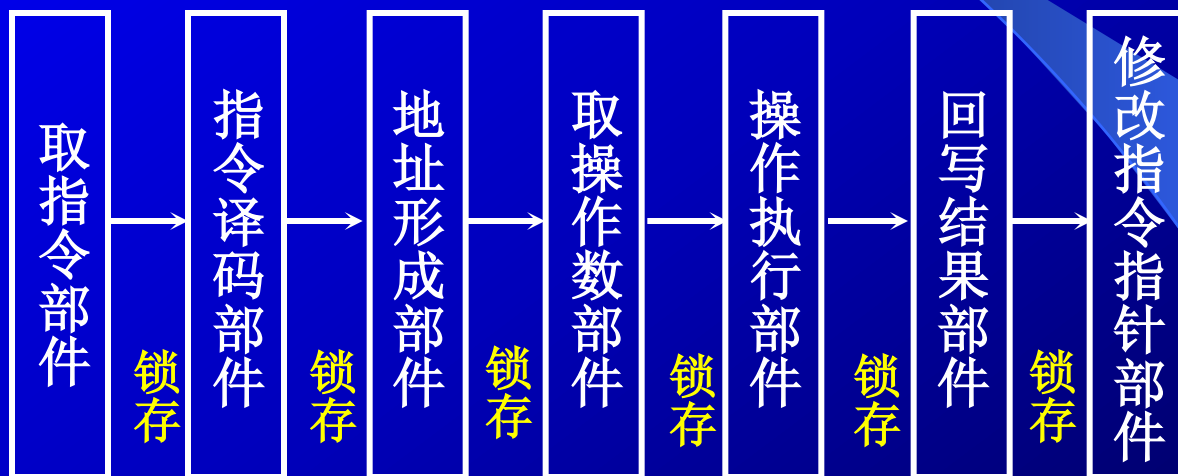


七、流水线结构

8.3

1. 指令流水线结构

完成一条指令分 **7 段**，每段需一个时钟周期



若 **流水线不出现断流**

1 个时钟周期出 **1** 结果

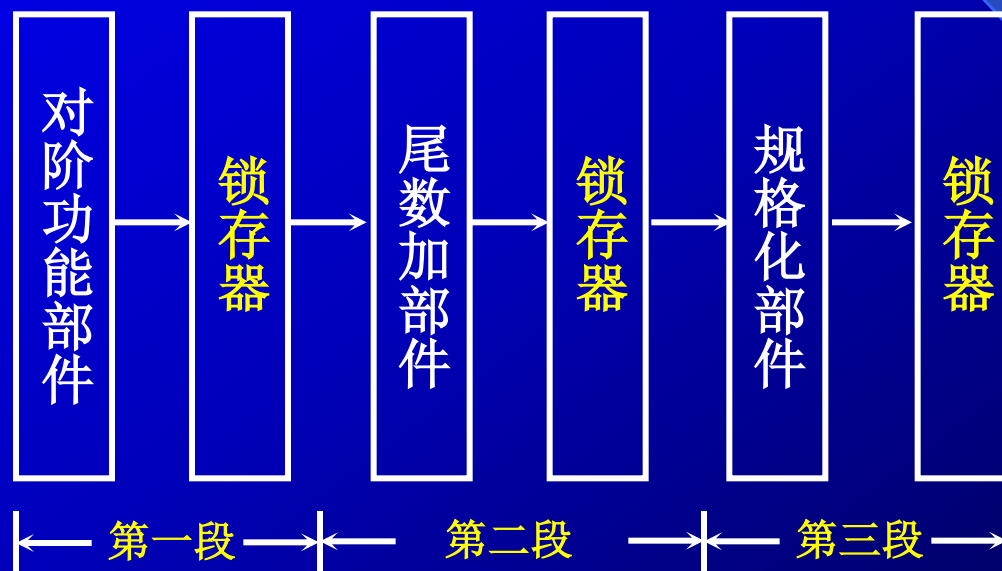
不采用流水技术

7 个时钟周期出 **1** 结果

理想情况下，**7 级流水** 的速度是不采用流水技术的 **7 倍**

2. 运算流水线

完成 浮点加减 运算 可分
对阶、尾数求和、规格化 三段



分段原则 每段 操作时间 尽量 一致

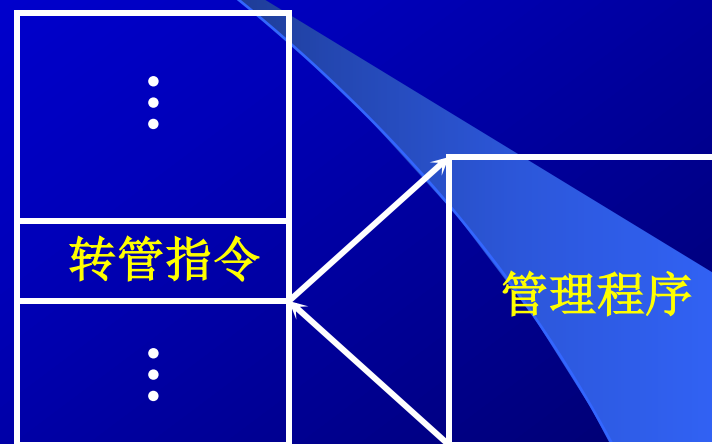
8.4 中断系统

一、概述

1. 引起中断的各种因素

(1) 人为设置的中断

如 转管指令



(2) 程序性事故 溢出、操作码不能识别、除法非法

(3) 硬件故障

(4) I/O 设备

(5) 外部事件 用 键盘中断 现行程序

2. 中断系统需解决的问题

- (1) 各中断源 如何 向 CPU 提出请求？
- (2) 各中断源 同时 提出 请求 怎么办？
- (3) CPU 什么 条件、什么 时间、以什么 方式 响应中断？
- (4) 如何 保护现场？
- (5) 如何 寻找入口地址？
- (6) 如何 恢复现场，如何 返回？
- (7) 处理中断的过程中又 出现新的中断 怎么办？

硬件 + 软件



二、中断请求标记和中断判优逻辑

1. 中断请求标记 **INTR**

一个请求源 一个 **INTR** 中断请求标记触发器

多个**INTR** 组成 中断请求标记寄存器

1	2	3	4	5			<i>n</i>
掉电	过热	主存读写校验错	阶上溢	非法除法		键盘输入	打印机输出

INTR 分散 在各个中断源的 接口电路中

INTR 集中在 **CPU** 的中断系统 内

2. 中断判优逻辑

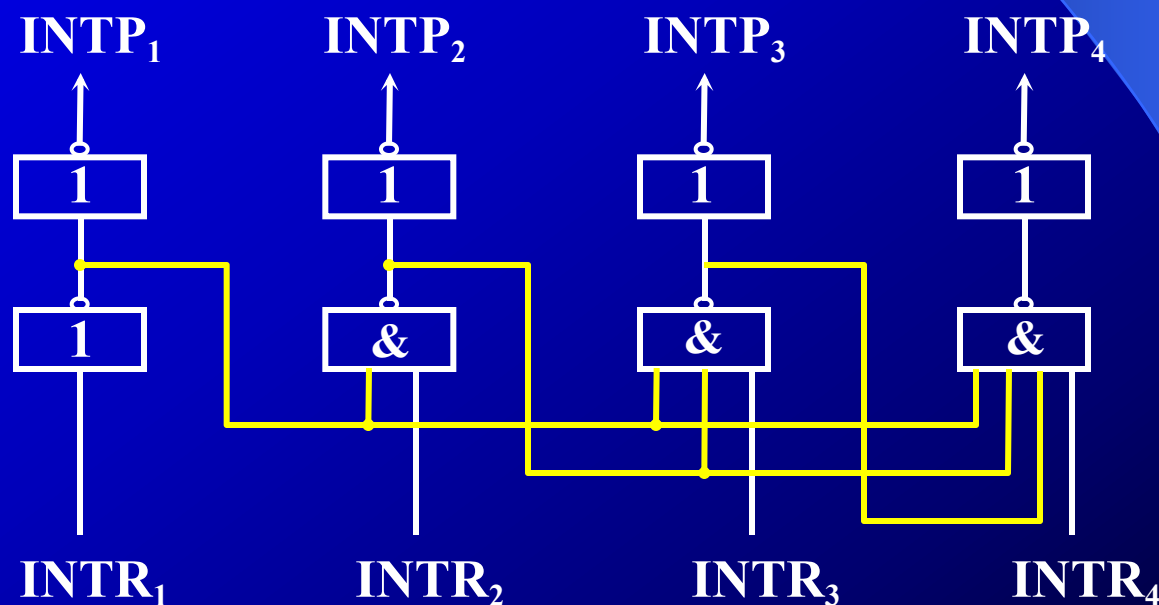
8.4

(1) 硬件实现（排队器）

① 分散 在各个中断源的 接口电路中 链式排队器

参见 第五章

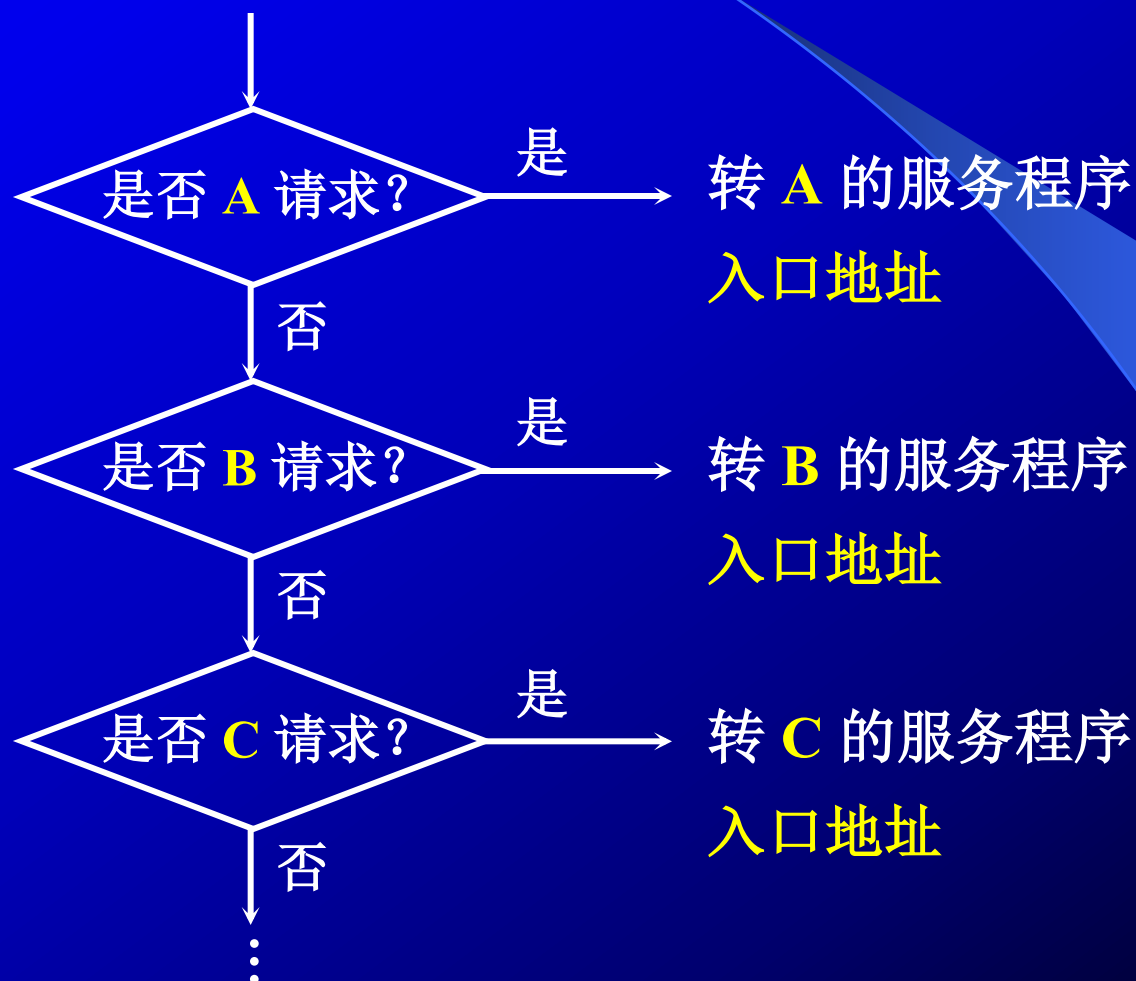
② 集中 在 CPU 内



INTR₁、INTR₂、INTR₃、INTR₄ 优先级按降序排列

(2) 软件实现（程序查询）

A、B、C 优先级按 降序 排列



三、中断服务程序入口地址的寻找

8.4

1. 硬件向量法



向量地址 12H、13H、14H

入口地址 200、300、400

2. 软件查询法

8.4

八个中断源 1, 2, ... 8 按 **降序** 排列

中断识别程序 (入口地址 **M**)

地 址	指 令	说 明
M	SKP DZ 1 [#]	1 [#] D = 0 跳 (D为完成触发器)
	JMP 1 [#] SR	1 [#] D = 1 转1 [#] 服务程序
	SKP DZ 2 [#]	2 [#] D = 0 跳
	JMP 2 [#] SR	2 [#] D = 1 转2 [#] 服务程序
	⋮	
	SKP DZ 8 [#]	8 [#] D = 0 跳
	JMP 8 [#] SR	8 [#] D = 1 转8 [#] 服务程序



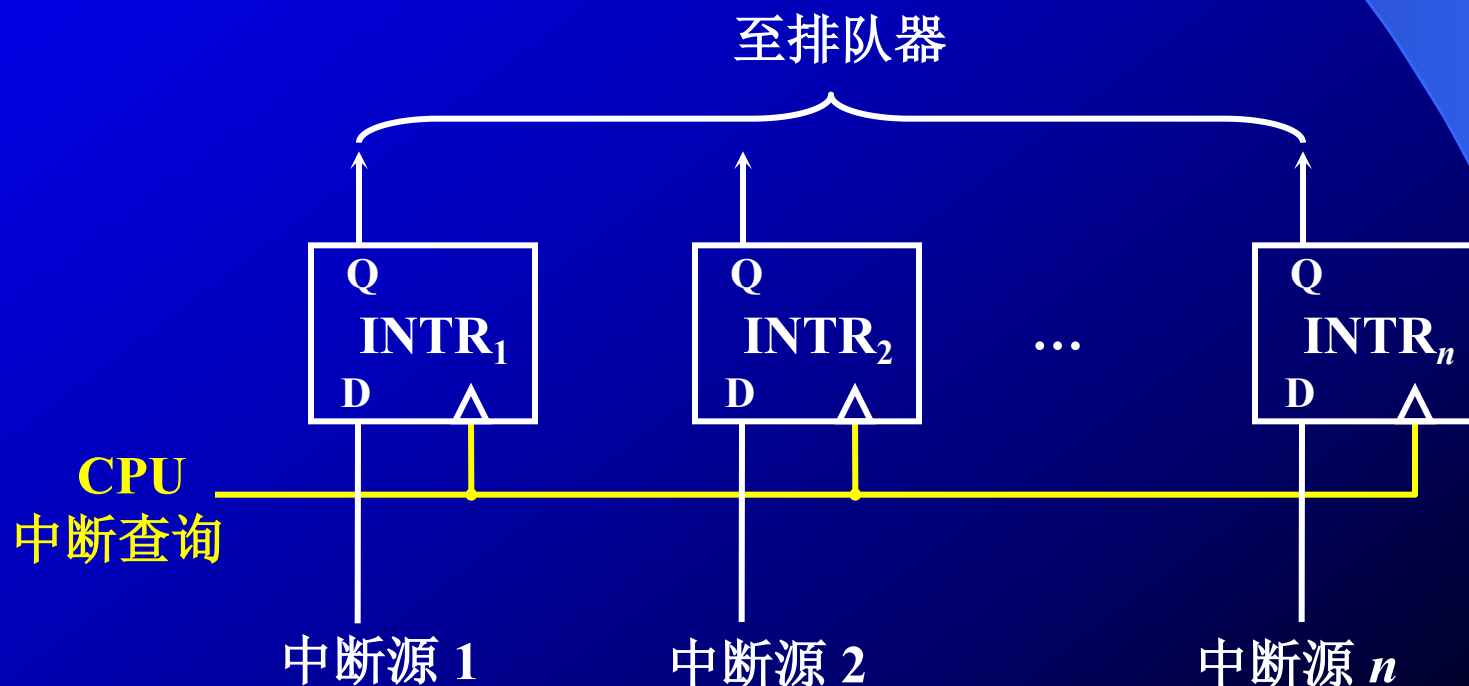
四、中断响应

1. 响应中断的条件

允许中断触发器 $EINT = 1$

2. 响应中断的时间

指令执行周期结束时刻由CPU发查询信号



五、保护现场和恢复现场

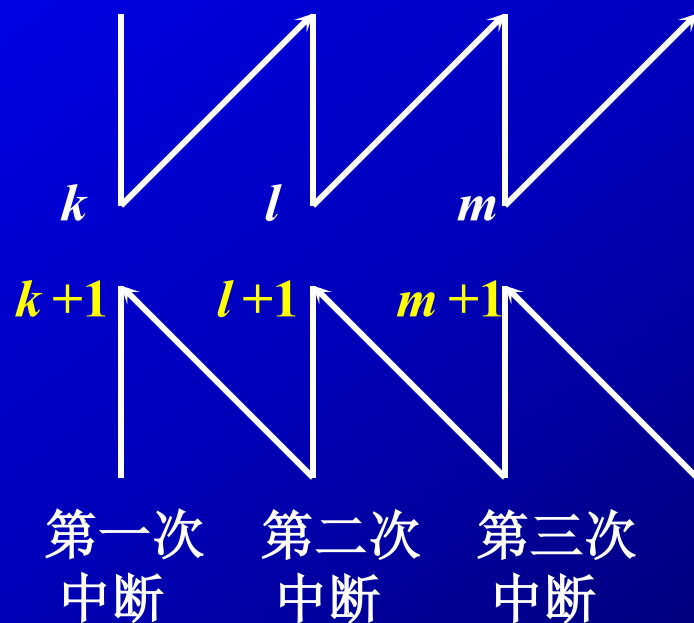
8.4

1. 保护现场 { 断点 中断隐指令 完成
 寄存器 内容 中断服务程序 完成
2. 恢复现场 中断服务程序 完成



六、中断屏蔽技术

1. 多重中断的概念



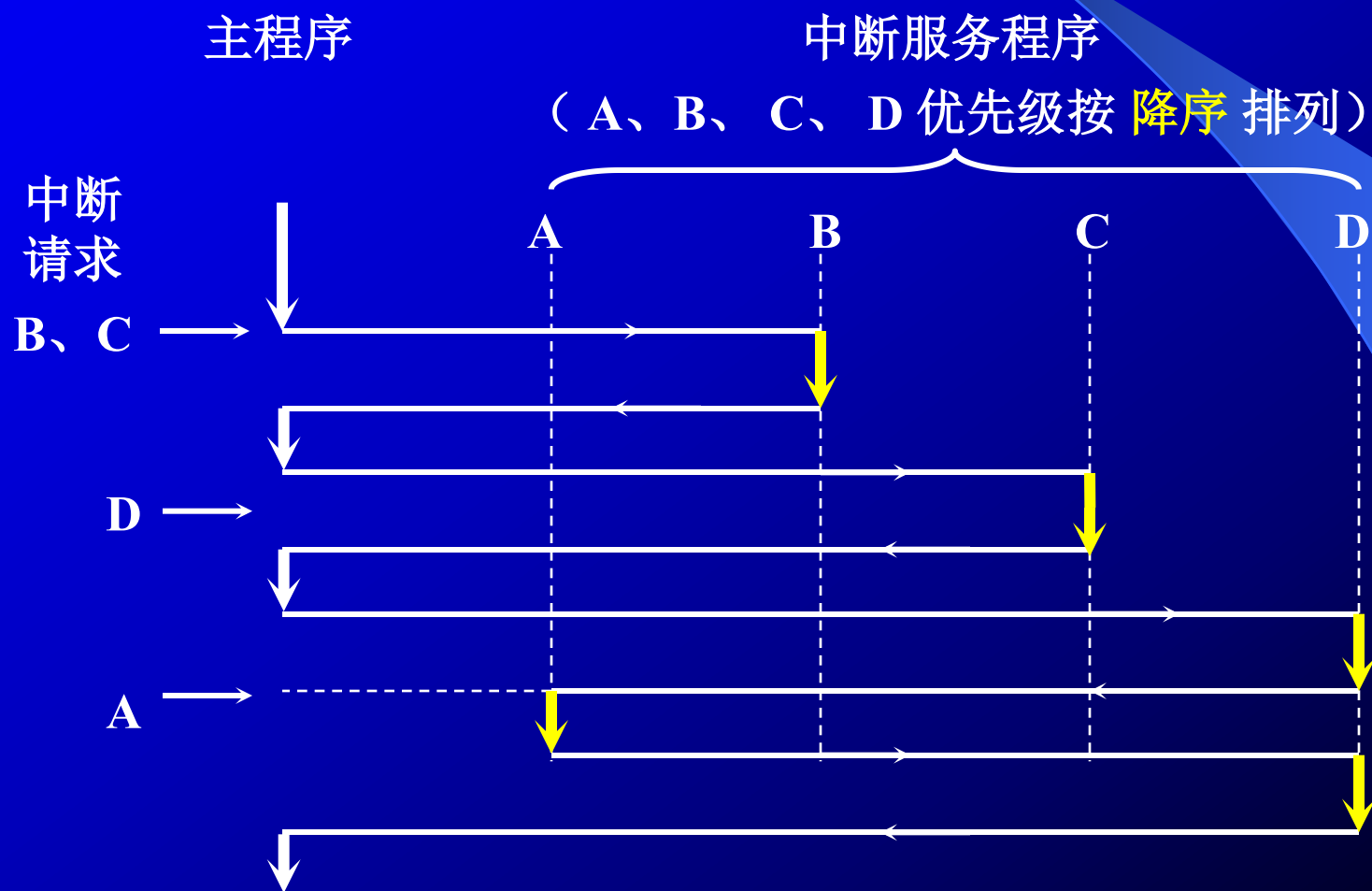
程序断点 ***k+1***, ***l+1***, ***m+1***

2. 实现多重中断的条件

8.4

(1) 提前 设置 开中断 指令

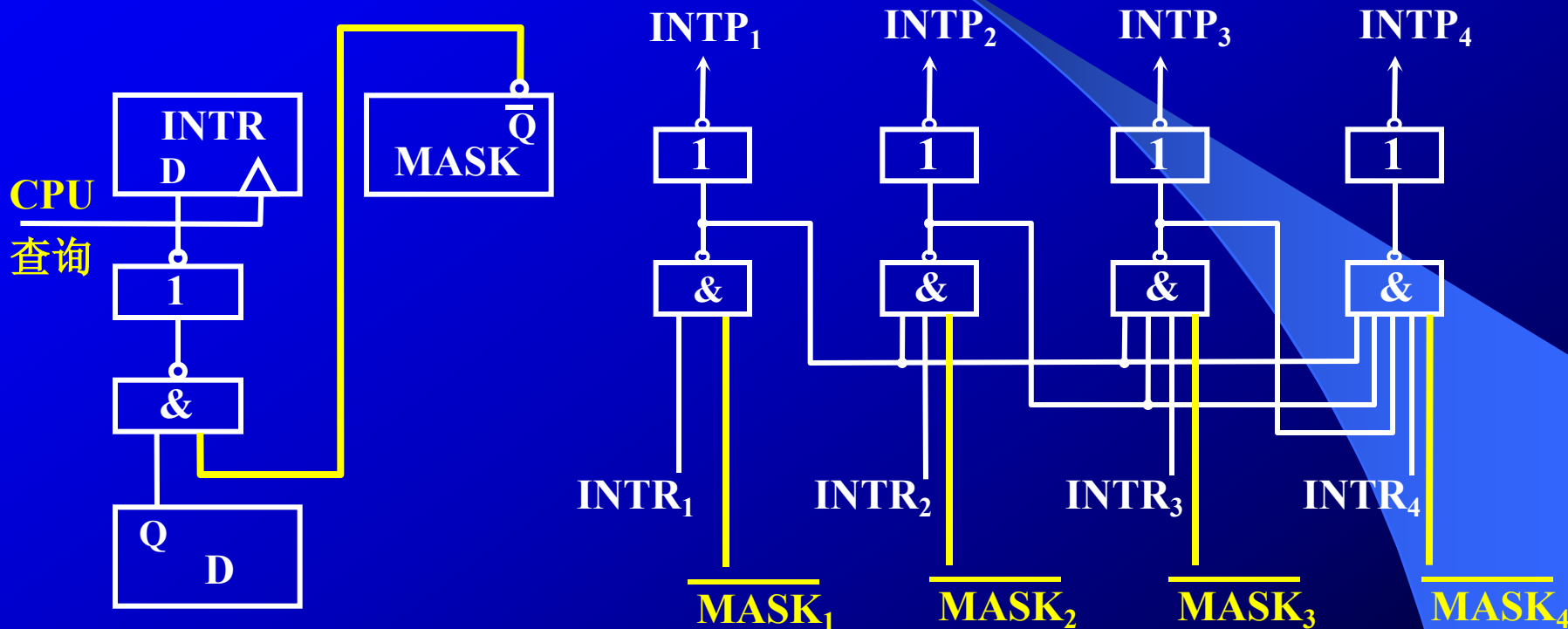
(2) 优先级别高 的中断源 有权中断优先级别低 的中断源



3. 屏蔽技术

8.4

(1) 屏蔽触发器的作用



$MASK = 0$ (未屏蔽)

$INTR$ 能被置“1”

$MASK_i = 1$ (屏蔽)

$INTP_i = 0$ (不能被排队选中)

优先级	屏	蔽	字
1	1	1	1
2	0	1	1
3	0	0	1
4	0	0	0
5	0	0	0
6	0	0	0
⋮			
15	0	0	0
16	0	0	0

(3) 屏蔽技术可改变处理优先等级

8.4

响应优先级 不可改变

处理优先级 可改变（通过重新设置屏蔽字）

中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

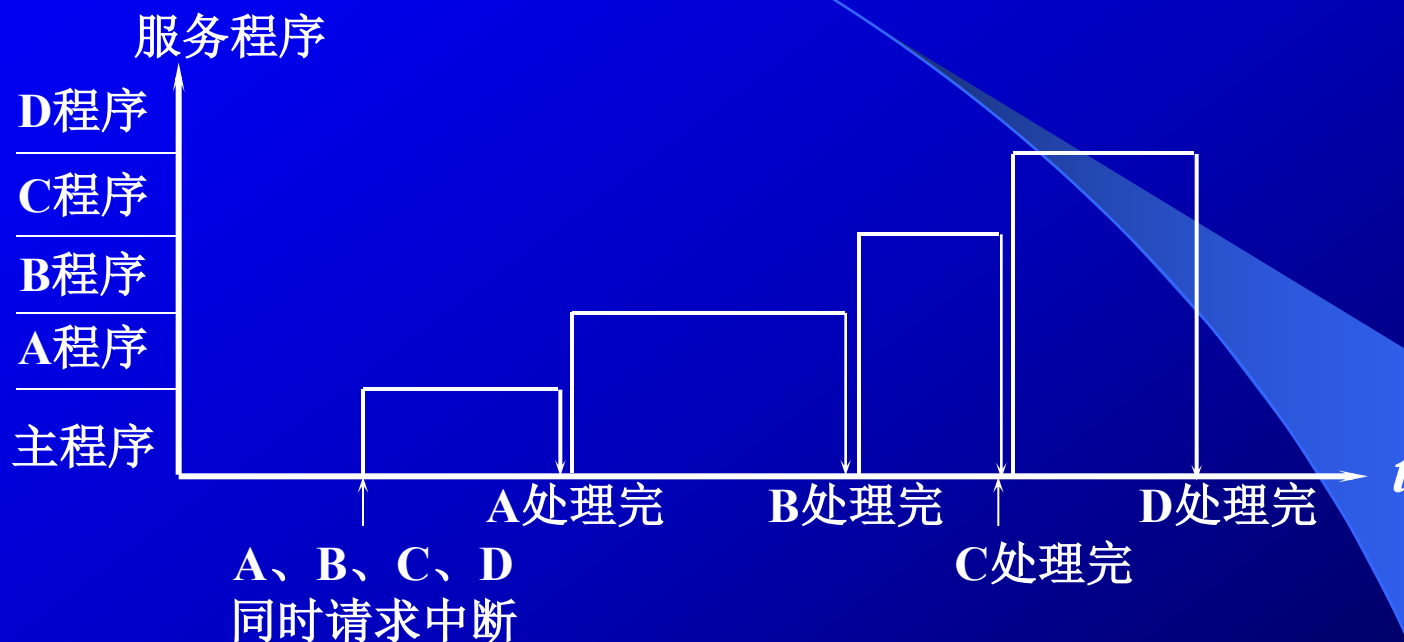
响应优先级 **A→B→C→D** 降序排列

处理优先级 **A→D→C→B** 降序排列



(3) 屏蔽技术可改变处理优先等级

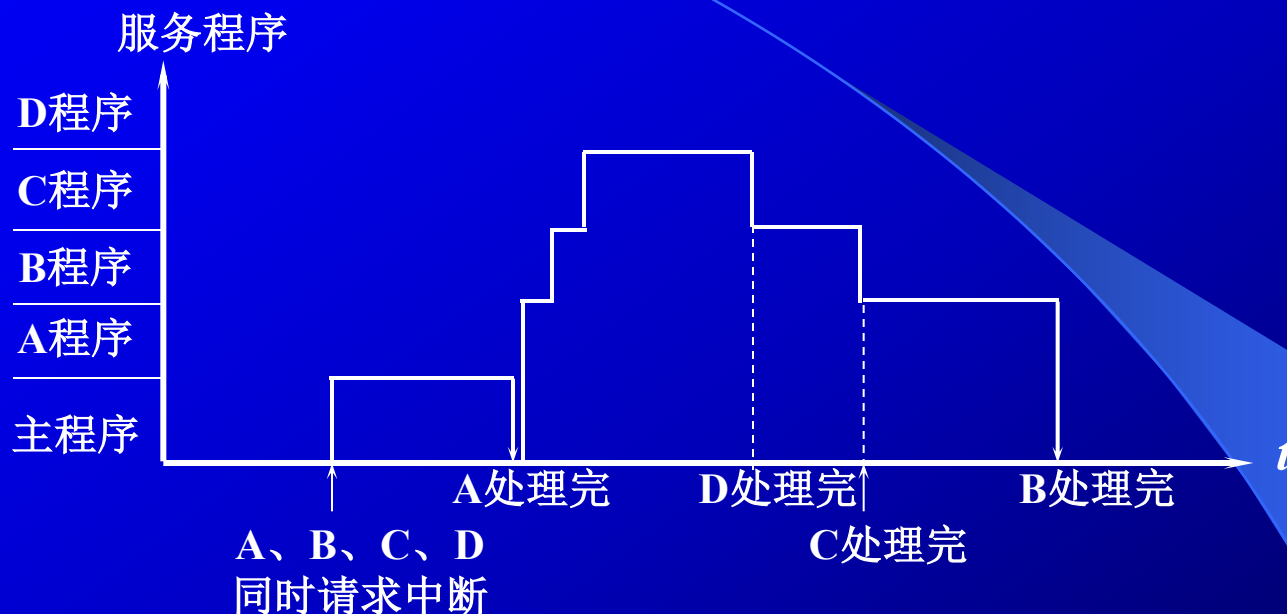
8.4



CPU 执行程序轨迹（原屏蔽字）



(3) 屏蔽技术可改变处理优先等级



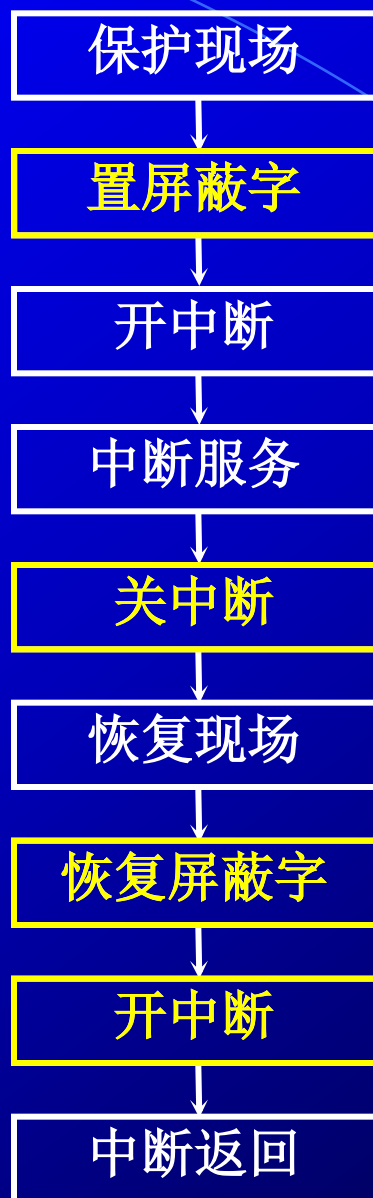
CPU 执行程序轨迹（新屏蔽字）

(4) 屏蔽技术的其他作用

可以 **人为地屏蔽** 某个中断源的请求
便于程序控制

(5) 新屏蔽字的设置

8.4



4. 多重中断的断点保护

(1) 断点进栈 中断隐指令 完成

(2) 断点存入“0”地址 中断隐指令 完成

中断周期 $0 \rightarrow \text{MAR}$

命令存储器写

$\text{PC} \rightarrow \text{MDR}$ 断点 $\rightarrow \text{MDR}$

$(\text{MDR}) \rightarrow$ 存入存储器

三次中断，三个断点都存入“0”地址

？ 如何保证断点不丢失？

(3) 程序断点存入 “0” 地址的断点保护8.4

地 址	内 容	说 明
0	××××	存程序断点
5	JMP SERVE	5 为向量地址
SERVE	STA SAVE	保护现场
	⋮	
置屏蔽字	LDA 0	} 0 地址内容转存
→	STA RETURN	
	ENI	开中断
	⋮	} 其他服务内容
	LDA SAVE	
	JMP @ RETURN	恢复现场
SAVE	××××	间址返回
RETURN	××××	存放 ACC 内容
		转存 0 地址内容

第 9 章 控制单元的功能

9.1 操作命令的分析

9.2 控制单元的功能



9.1 操作命令的分析

完成一条指令分 4 个工作周期

取指周期

间址周期

执行周期

中断周期



9.1 操作命令的分析

一、取指周期

$PC \rightarrow MAR \rightarrow \text{地址线}$

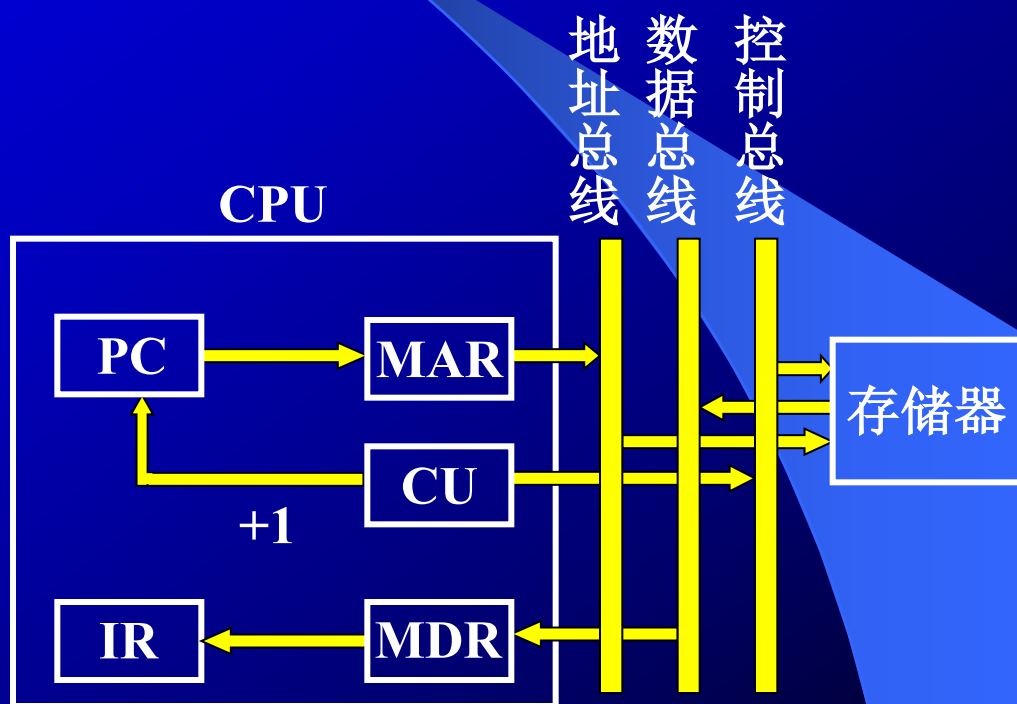
$1 \rightarrow R$

$M(MAR) \rightarrow MDR$

$MDR \rightarrow IR$

$OP(IR) \rightarrow CU$

$(PC) + 1 \rightarrow PC$



二、间址周期

9.1

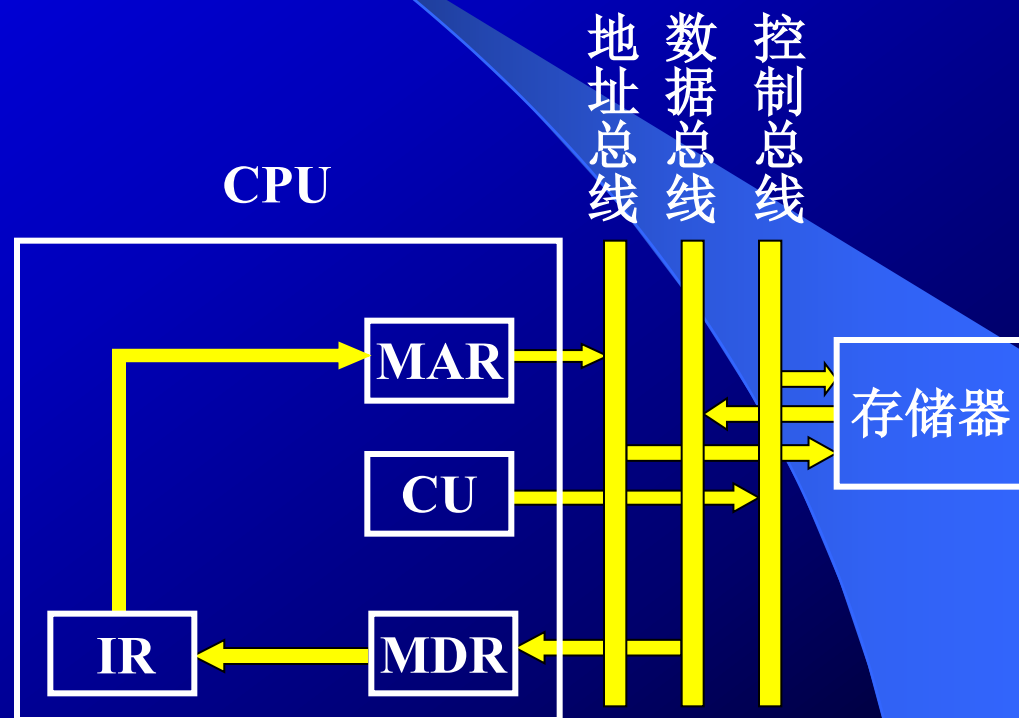
指令形式地址 $\rightarrow$ MAR

$Ad(IR) \rightarrow MAR$

$1 \rightarrow R$

$M(MAR) \rightarrow MDR$

$MDR \rightarrow Ad(IR)$



三、执行周期

9.1

1. 非访存指令

(1) **CLA** 清A $0 \rightarrow \text{ACC}$

(2) **COM** 取反 $\overline{\text{ACC}} \rightarrow \text{ACC}$

(3) **SHR** 算术右移 $\text{L(ACC)} \rightarrow \text{R(ACC)}, \text{ACC}_0 \rightarrow \text{ACC}_0$

(4) **CSL** 循环左移 $\text{R(ACC)} \rightarrow \text{L(ACC)}, \text{ACC}_0 \rightarrow \text{ACC}_n$

(5) **STP** 停机指令 $0 \rightarrow \text{G}$



2. 访存指令

9.1

(1) 加法指令

ADD X

$\text{Ad(IR)} \rightarrow \text{MAR}$

$1 \rightarrow \text{R}$

$\text{M(MAR)} \rightarrow \text{MDR}$

$(\text{ACC}) + (\text{MDR}) \rightarrow \text{ACC}$

(2) 存数指令

STA X

$\text{Ad(IR)} \rightarrow \text{MAR}$

$1 \rightarrow \text{W}$

$\text{ACC} \rightarrow \text{MDR}$

$\text{MDR} \rightarrow \text{M(MAR)}$



(3) 取数指令 **LDA X**

$$\text{Ad}(\text{IR}) \rightarrow \text{MAR}$$
$$1 \rightarrow \text{R}$$
$$\text{M}(\text{MAR}) \rightarrow \text{MDR}$$
$$\text{MDR} \rightarrow \text{ACC}$$

3. 转移指令

(1) 无条件转 **JMP X**

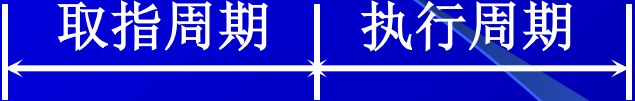
$$\text{Ad}(\text{IR}) \rightarrow \text{PC}$$

(2) 条件转移 **BAN X** (负则转)

$$\text{A}_0 \cdot \text{Ad}(\text{IR}) + \bar{\text{A}}_0(\text{PC}) \rightarrow \text{PC}$$

4. 三类指令的指令周期

非访存 指令周期



取指周期 | 执行周期

直接访存 指令周期



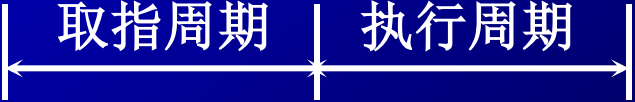
取指周期 | 执行周期

间接访存 指令周期



取指周期 | 间址周期 | 执行周期

转移 指令周期



取指周期 | 执行周期

间接转移 指令周期



取指周期 | 间址周期 | 执行周期

四、中断周期

9.1

程序断点存入 “0” 地址 程序断点 进栈

$0 \rightarrow \text{MAR}$

$(\text{SP}) - 1 \rightarrow \text{MAR}$

$1 \rightarrow \text{W}$

$1 \rightarrow \text{W}$

$\text{PC} \rightarrow \text{MDR}$

$\text{PC} \rightarrow \text{MDR}$

$\text{MDR} \rightarrow \text{M}(\text{MAR})$

$\text{MDR} \rightarrow \text{M}(\text{MAR})$

中断识别程序入口地址 $\text{M} \rightarrow \text{PC}$

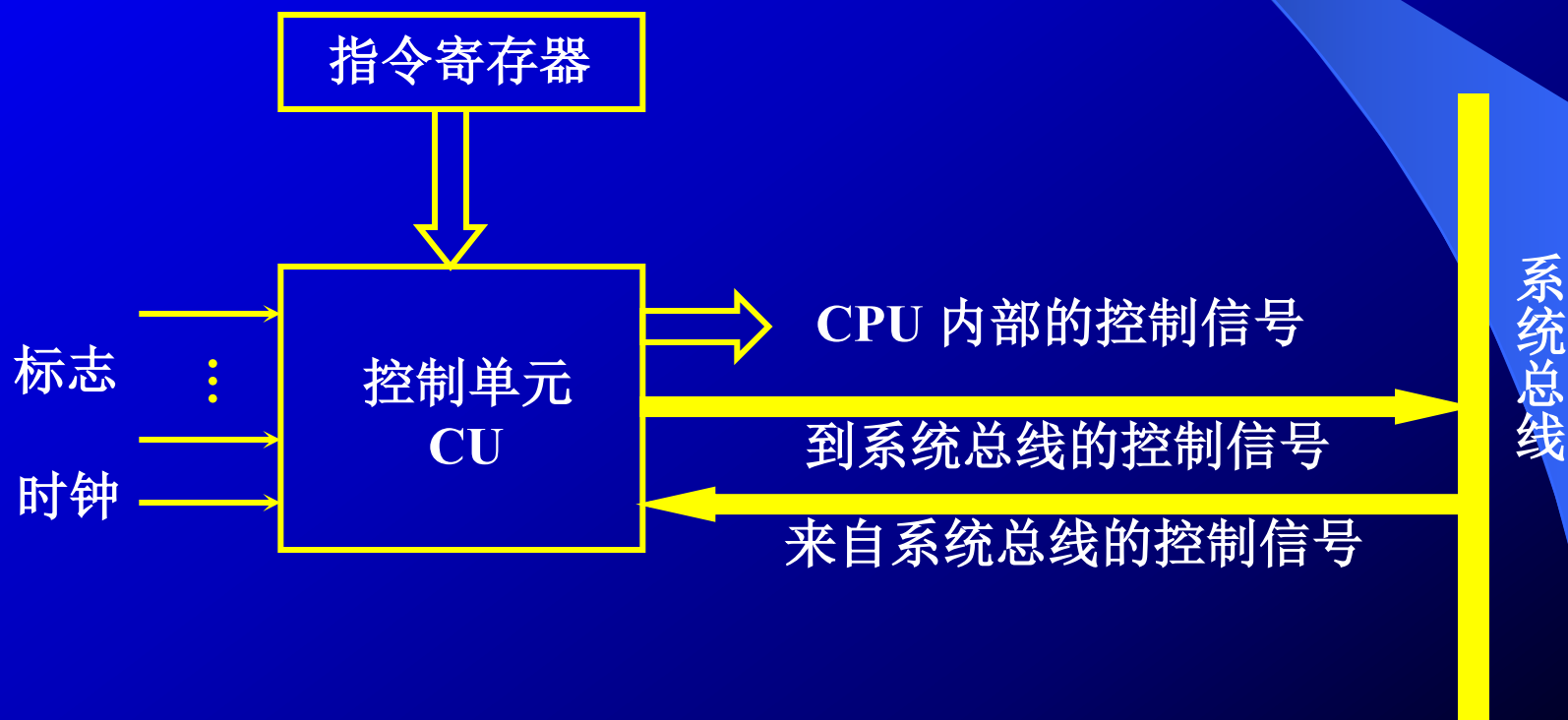
$0 \rightarrow \text{EINT}(\text{置“0”})$

$0 \rightarrow \text{EINT}(\text{置“0”})$



9.2 控制单元的功能

一、控制单元的外特性



1. 输入信号

(1) 时钟

CU 受时钟控制

一个时钟脉冲

发一个操作命令或一组需同时执行的操作命令

(2) 指令寄存器 $OP(IR) \rightarrow CU$

控制信号 与操作码有关

(3) 标志

CU 受标志控制

(4) 外来信号

如 **INTR** 中断请求
 HRQ 总线请求



2. 输出信号

(1) CPU 内的各种控制信号

$R_i \rightarrow R_j$

$(PC) + 1 \rightarrow PC$

ALU +、-、与、或

(2) 送至控制总线的信号

$\overline{MREQ}$

访存控制信号

$\overline{IO/M}$

访 IO/ 存储器的控制信号

$\overline{RD}$

读命令

$\overline{WR}$

写命令

INTA

中断响应信号

HLDA

总线响应信号

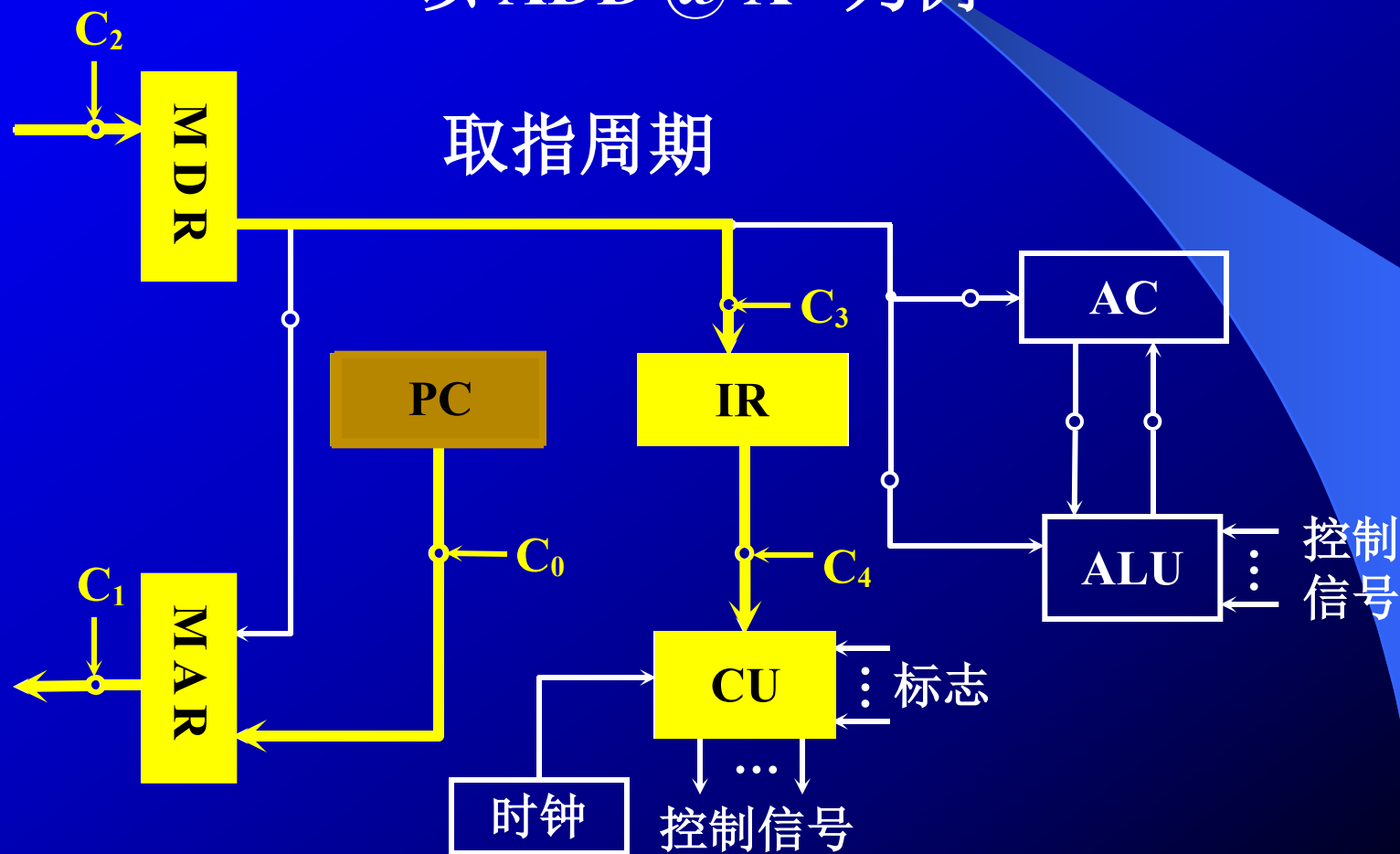


二、控制信号举例

9.2

1. 不采用 CPU 内部总线的方式

以 $\text{ADD } @X$ 为例

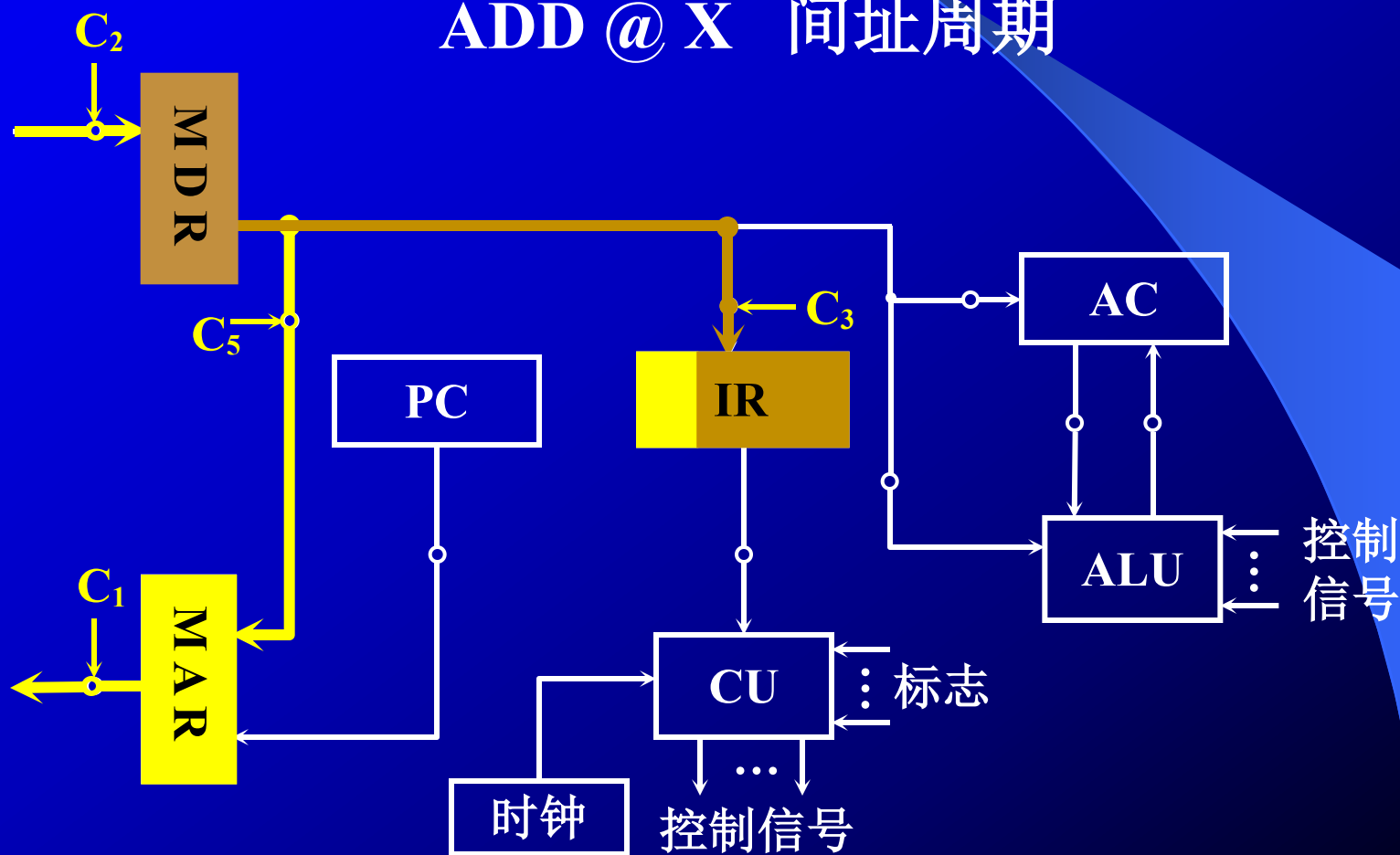


二、控制信号举例

9.2

1. 不采用 CPU 内部总线的方式

ADD @ X 间址周期

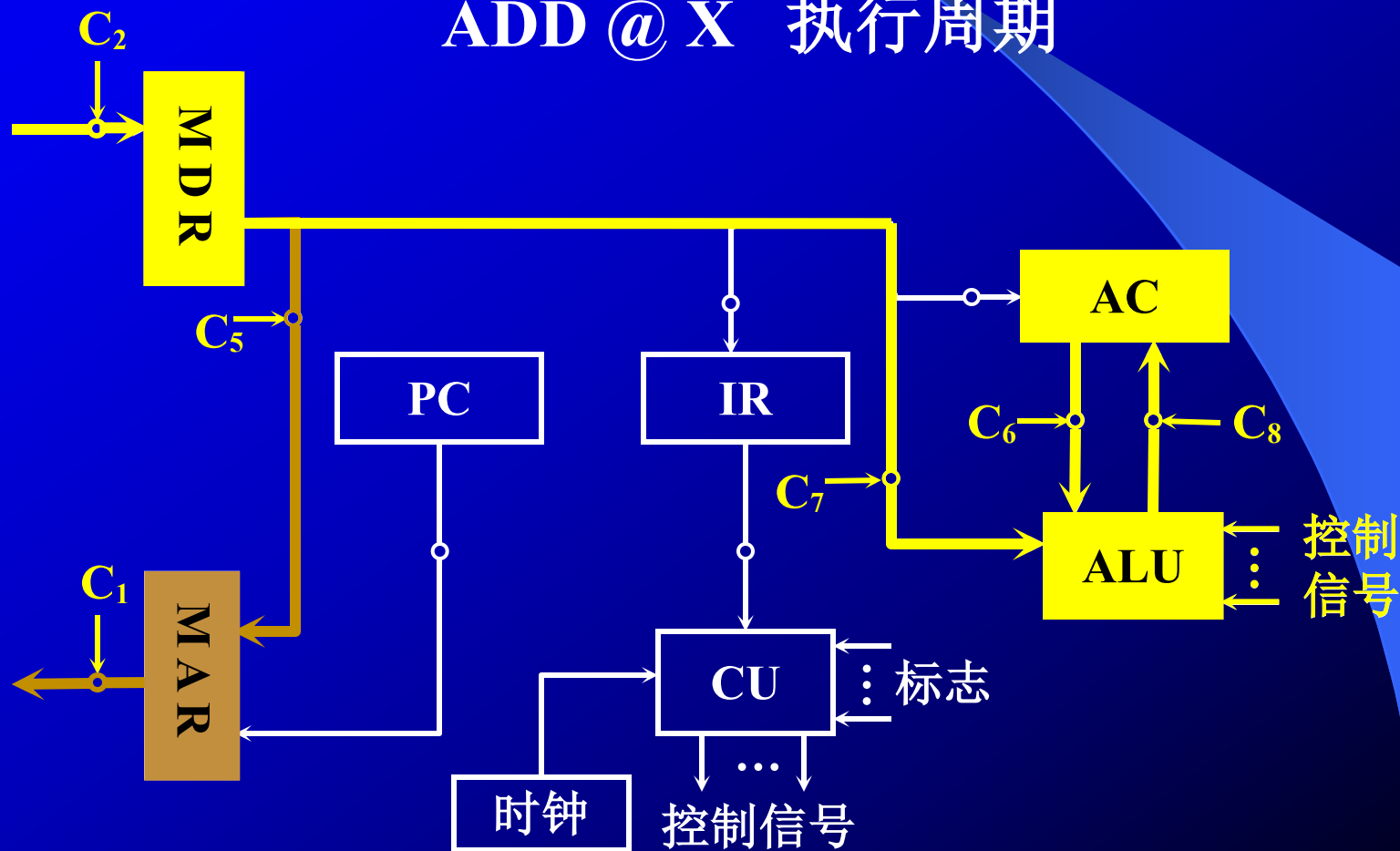


二、控制信号举例

9.2

1. 不采用 CPU 内部总线的方式

ADD @ X 执行周期



2. 采用 CPU 内部总线方式

9.2

(1) ADD @ X 取指周期

- PC $\rightarrow$ MAR $\rightarrow$ 地址线
 PC_0 MAR_i

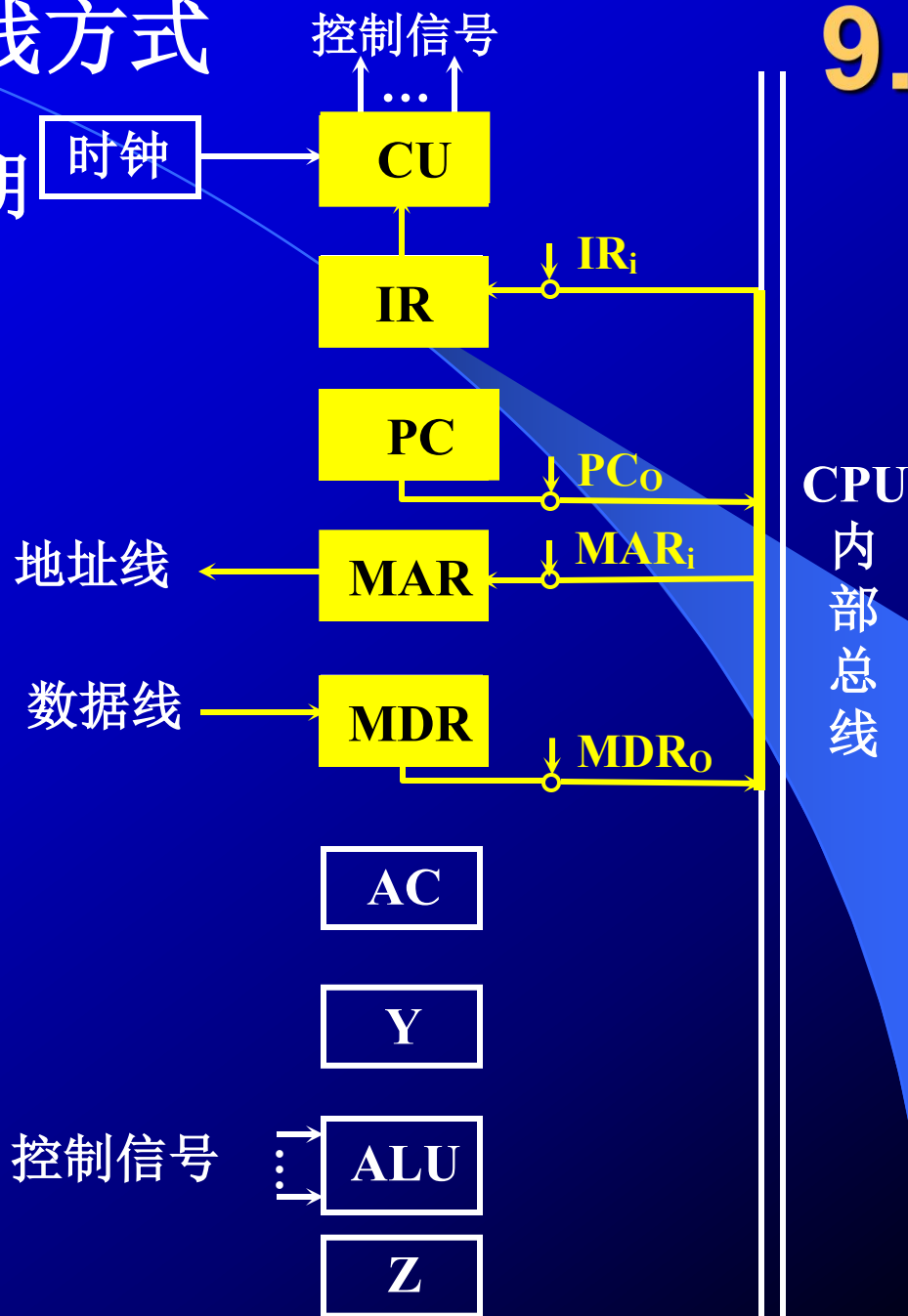
- CU 发读命令 $1 \rightarrow R$

- 数据线 $\rightarrow$ MDR

- MDR $\rightarrow$ IR
 MDR_0 IR_i

- OP (IR) $\rightarrow$ CU

- (PC) + 1 $\rightarrow$ PC



(2) ADD @ X 间址周期

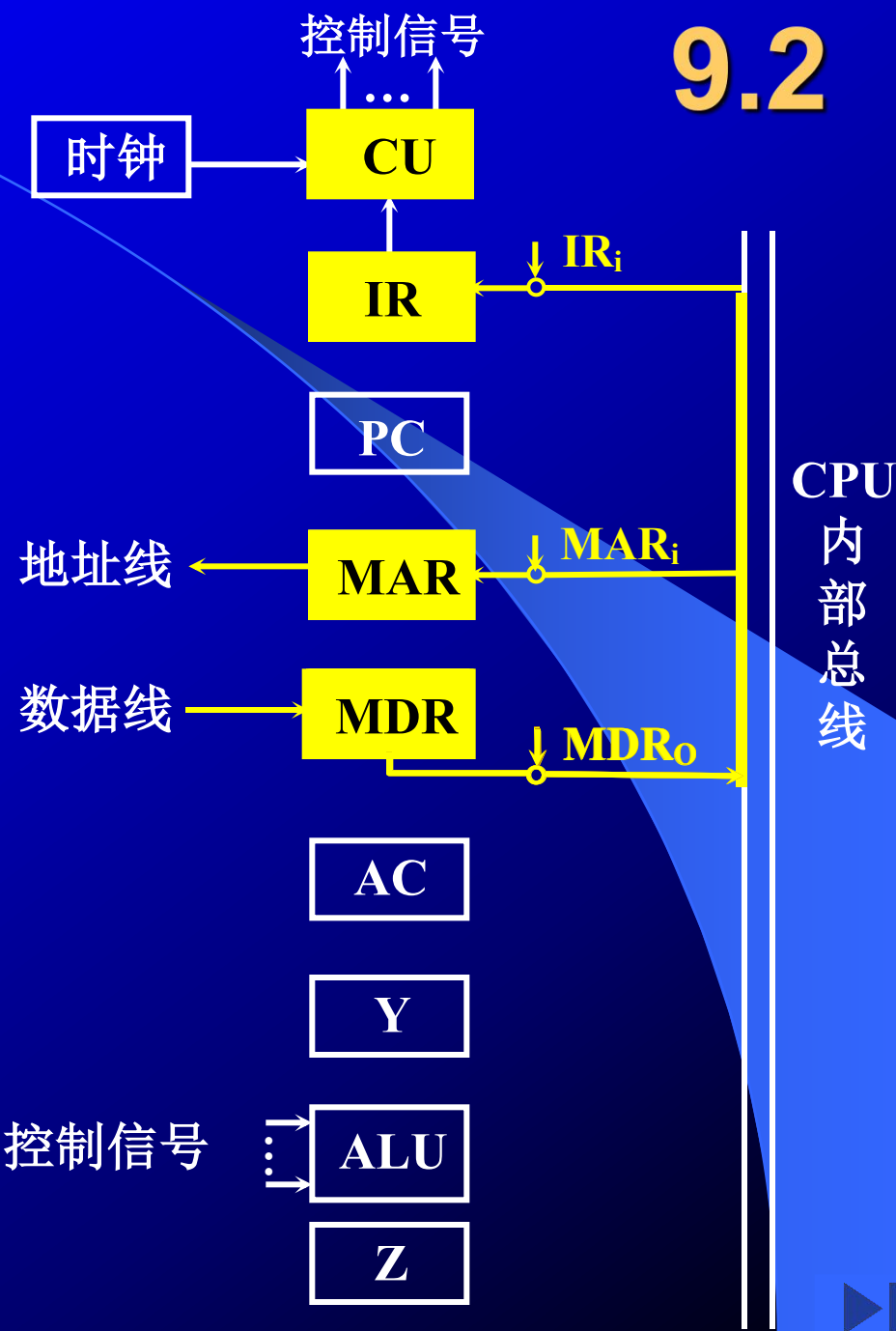
形式地址 $\rightarrow$ MAR

- MDR $\rightarrow$ MAR $\rightarrow$ 地址线
MDR₀ MAR_i

- 1 $\rightarrow$ R

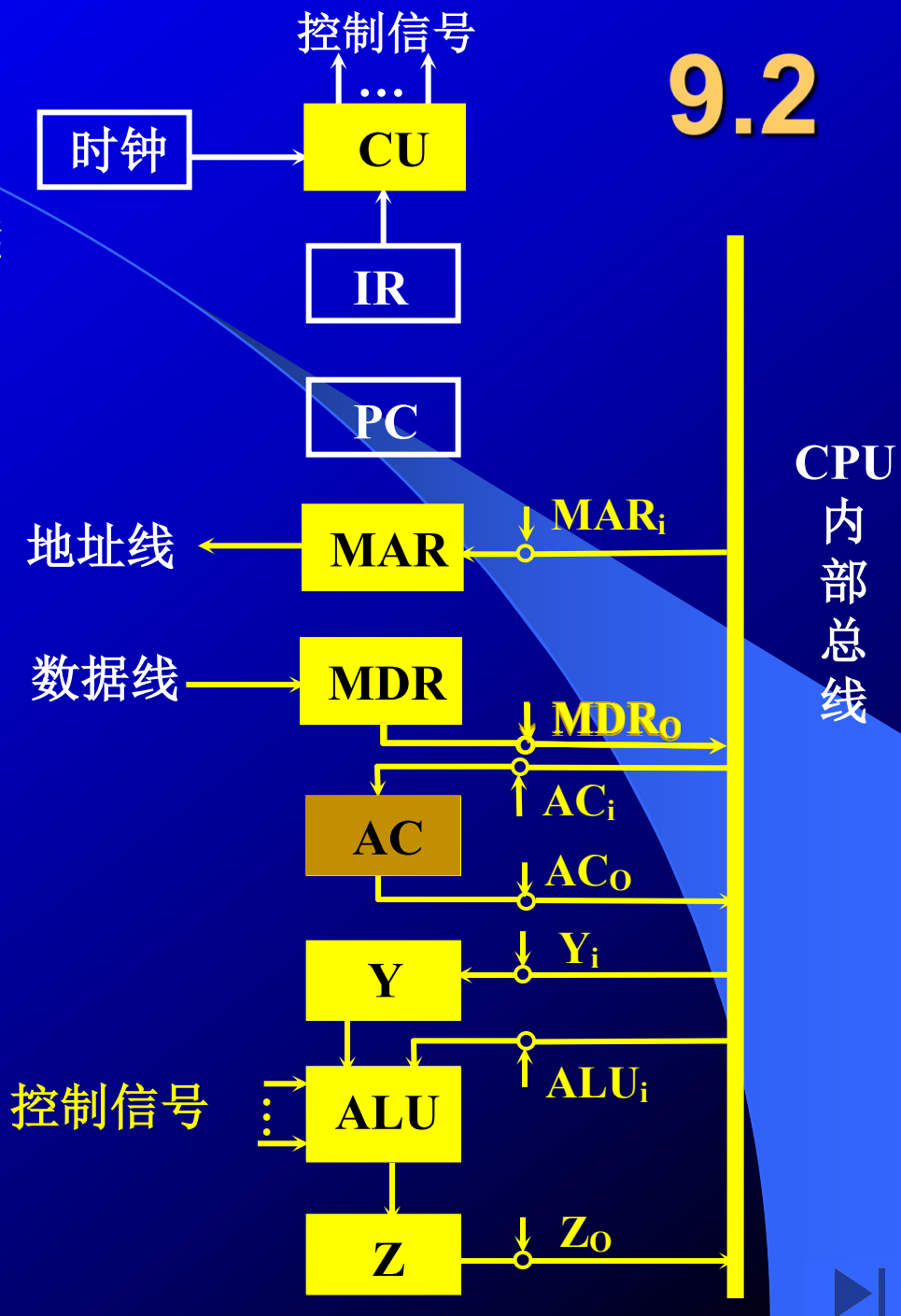
- 数据线 $\rightarrow$ MDR

- MDR $\rightarrow$ IR
MDR₀ IR_i

有效地址 $\rightarrow$ Ad (IR)

(3) ADD @ X 执行周期

- MDR $\longrightarrow$ MAR $\longrightarrow$ 地址线
MDR₀ MAR_i
- 1 $\longrightarrow$ R
- 数据线 $\longrightarrow$ MDR
- MDR $\longrightarrow$ Y $\longrightarrow$ ALU
MDR₀ Y_i
- AC $\longrightarrow$ ALU
AC₀ ALU_i
- (AC) + (Y) $\longrightarrow$ Z
- Z $\longrightarrow$ AC
Z₀ AC_i



三、多级时序系统

9.2

1. 机器周期

(1) 机器周期的概念

所有指令执行过程中的一个基准时间

(2) 确定机器周期需考虑的因素

每条指令的执行 步骤

每一步骤 所需的 时间

(3) 基准时间的确定

- 以完成 最复杂 指令功能的时间 为准
- 以 访问一次存储器 的时间 为基准

若指令字长 = 存储字长 取指周期 = 机器周期



2. 时钟周期（节拍、状态）

9.2

一个机器周期内可完成若干个微操作

每个微操作需一定的时间

将一个机器周期分成若干个时间相等的时间段（节拍、状态、时钟周期）

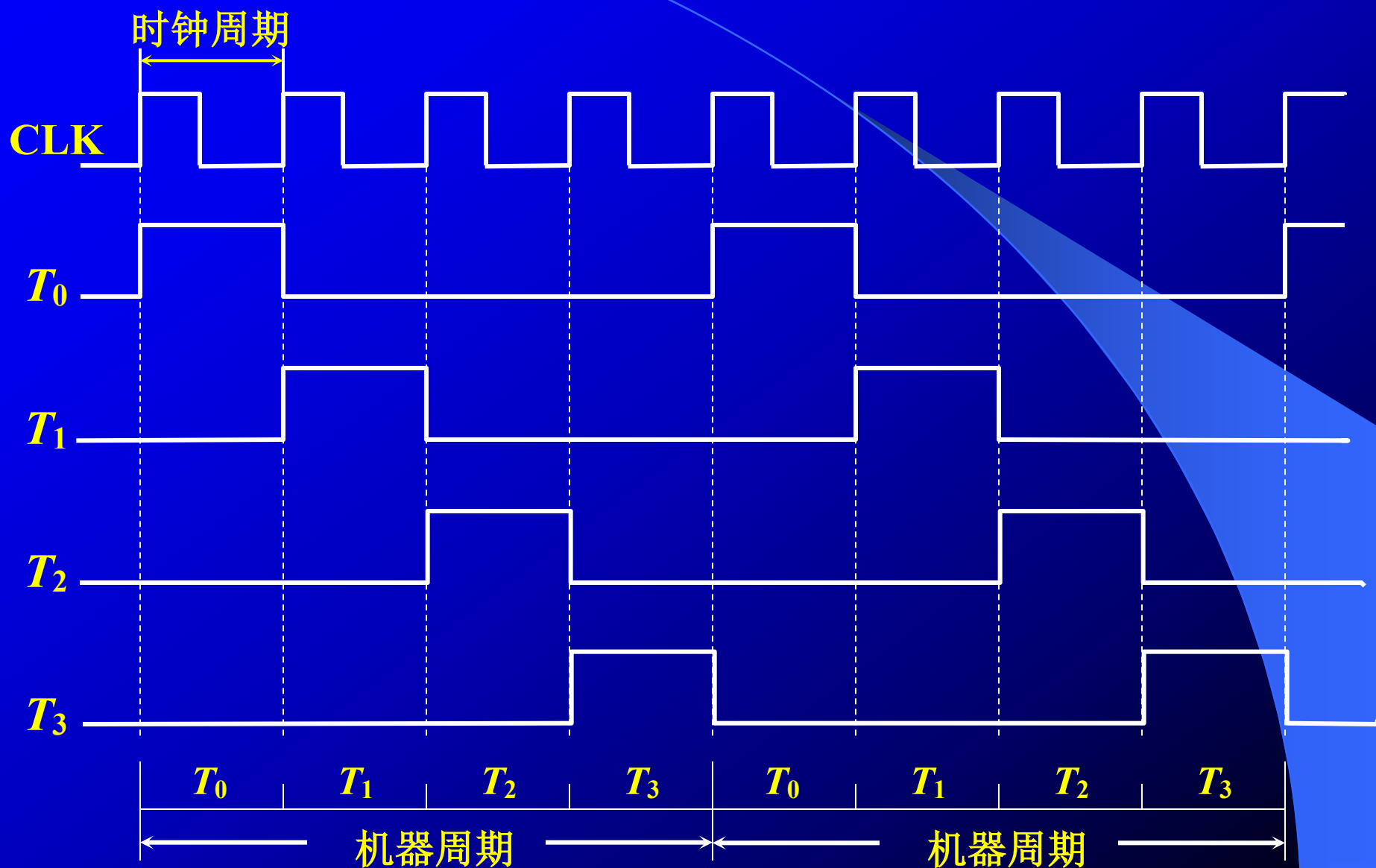
时钟周期是控制计算机操作的最小单位时间

用时钟周期控制产生一个或几个微操作命令



2. 时钟周期（节拍、状态）

9.2



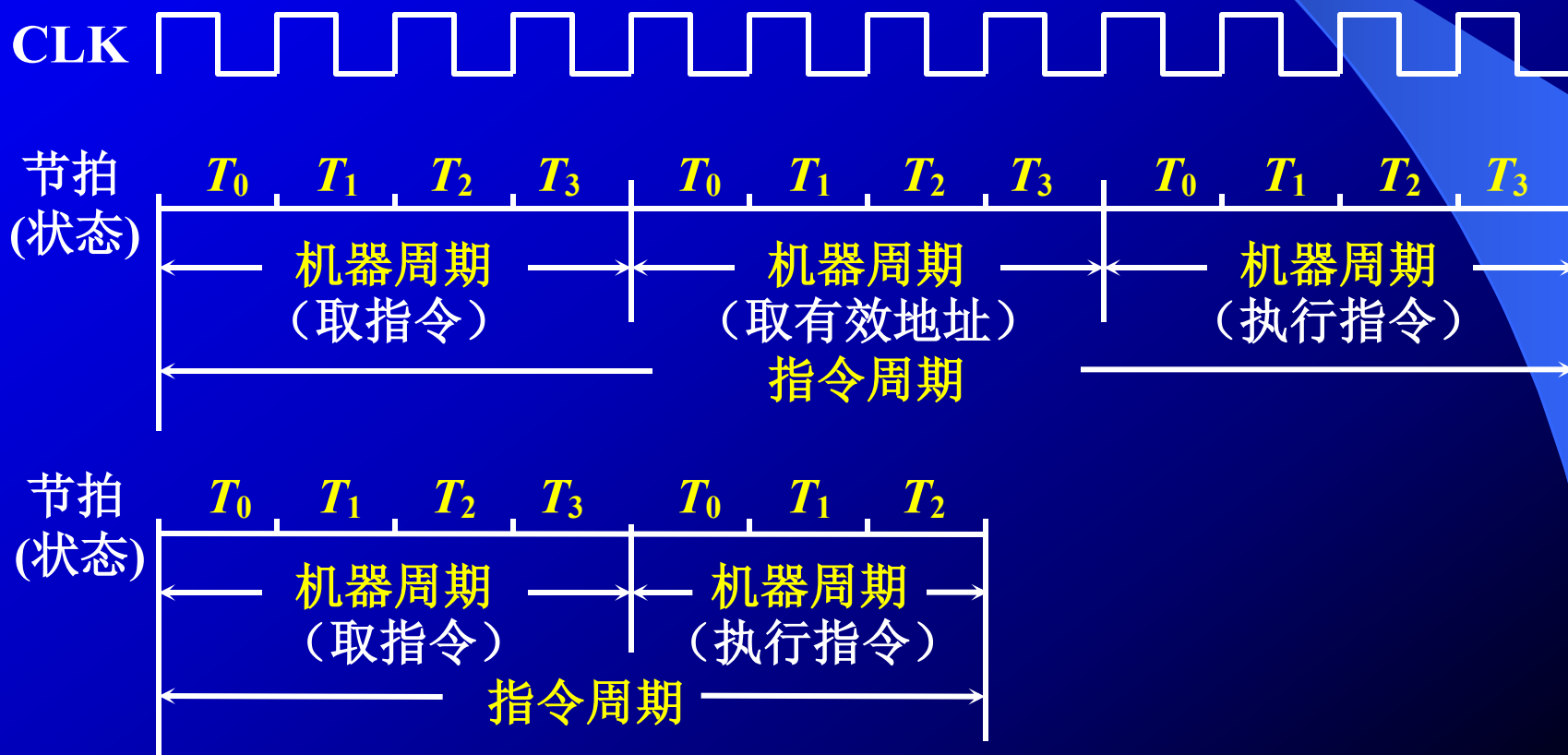
3. 多级时序系统

9.2

机器周期、节拍（状态）组成多级时序系统

一个指令周期包含若干个机器周期

一个机器周期包含若干个时钟周期



4. 机器速度与机器主频的关系

9.2

机器的 主频 f 越快 机器的 速度也越快

在机器周期所含时钟周期数 相同 的前提下，
两机 平均指令执行速度之比 等于 两机主频之比

$$\frac{\text{MIPS}_1}{\text{MIPS}_2} = \frac{f_1}{f_2}$$

机器速度 不仅与 主频有关，还与机器周期中所含
时钟周期（主频的倒数）数 以及指令周期中所含
的 机器周期数有关



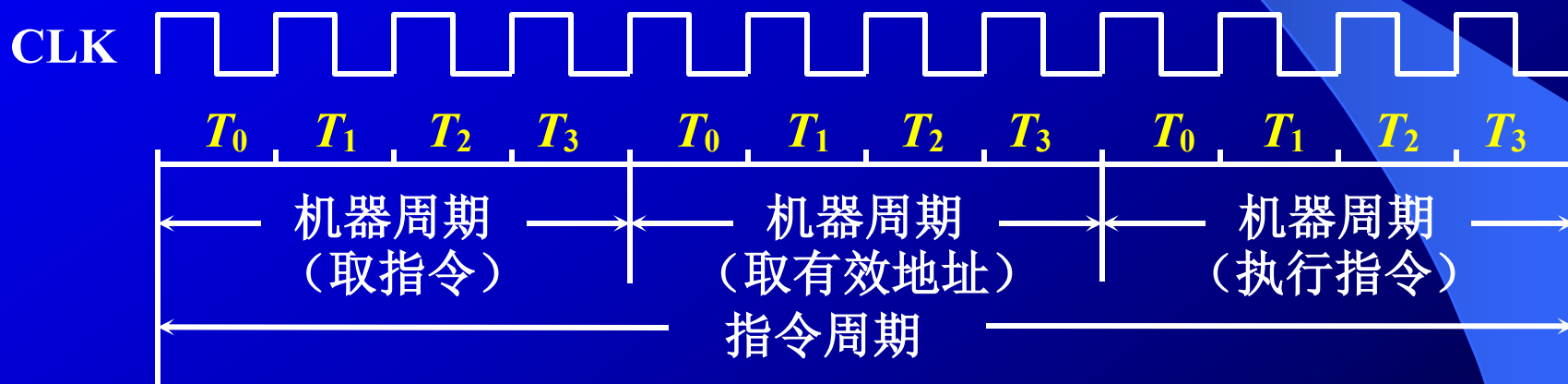
四、控制方式

9.2

产生不同微操作命令序列所用的时序控制方式

1. 同步控制方式

任一微操作均由 **统一基准时标** 的时序信号控制

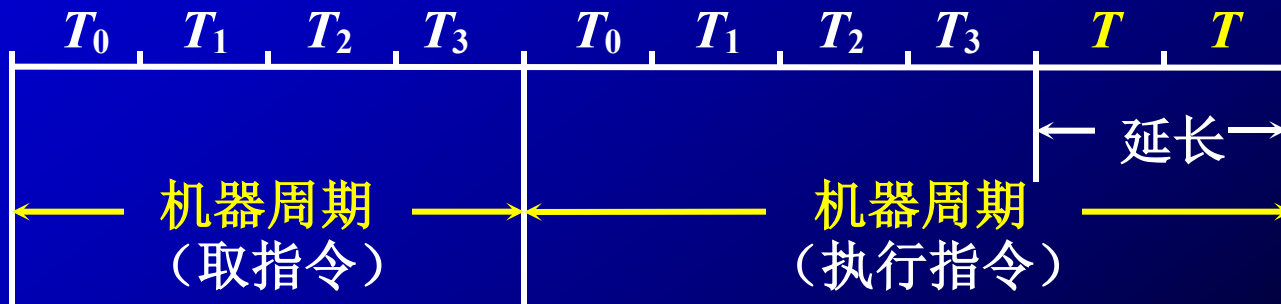
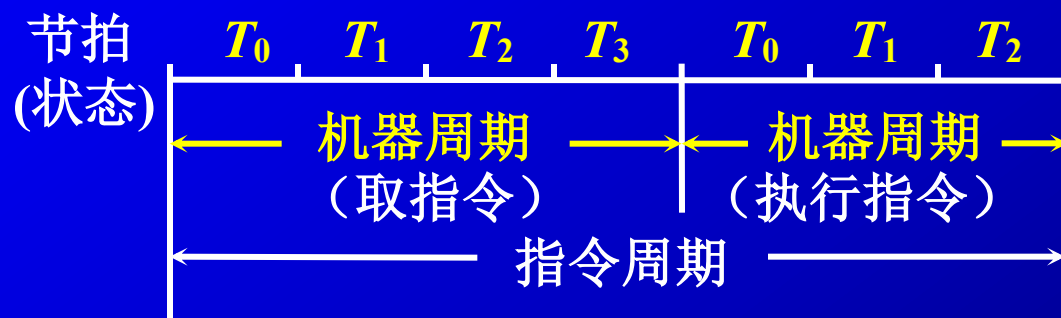


(1) 采用 **定长** 的机器周期

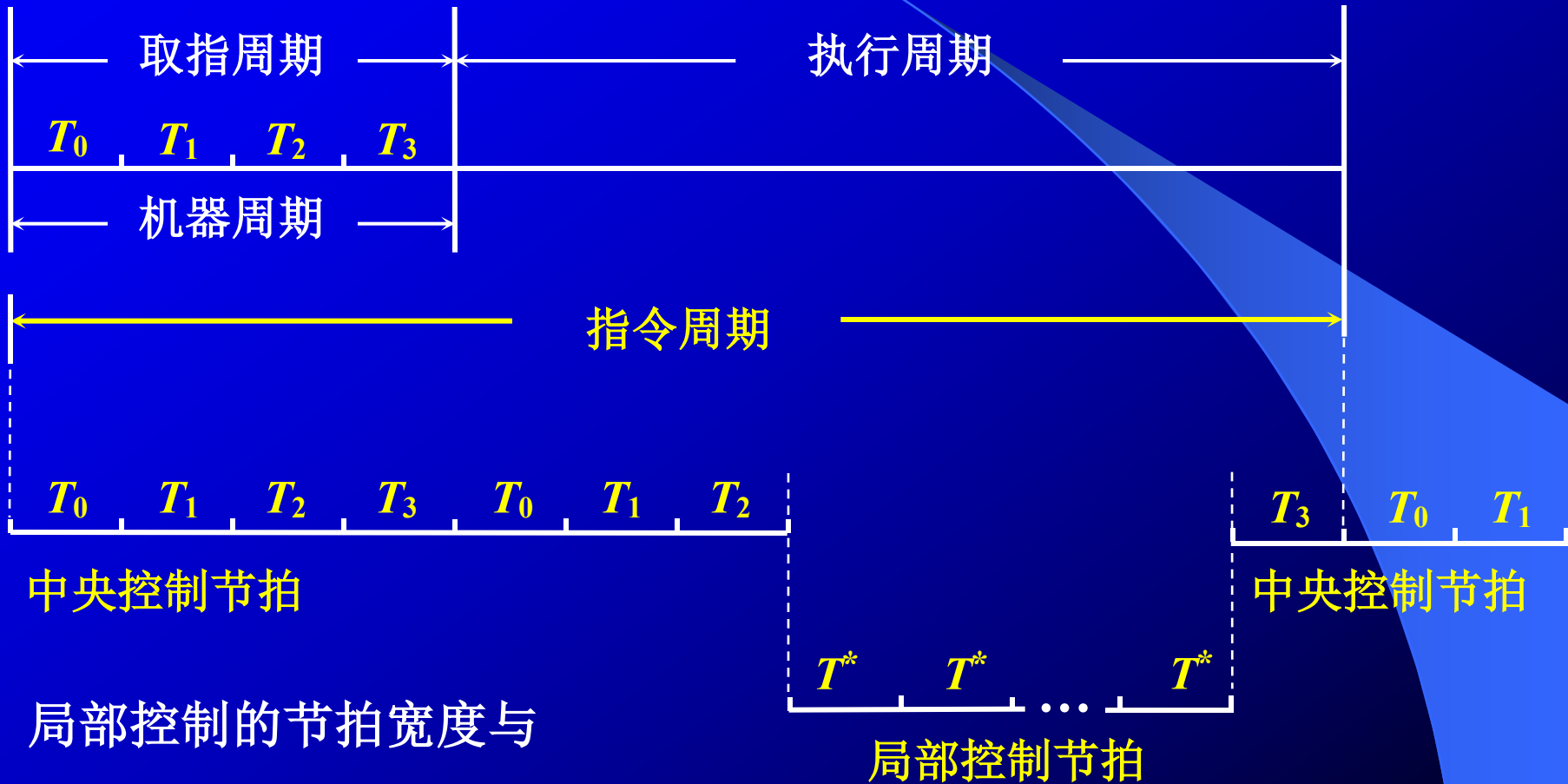
以 **最长** 的 **微操作序列** 和 **最繁** 的微操作作为 **标准**
机器周期内 **节拍数相同**

(2) 采用不定长的机器周期

机器周期内 节拍数不等



(3) 采用中央控制和局部控制相结合的方法 9.2



局部控制的节拍宽度与
中央控制的节拍宽度一致

2. 异步控制方式

无基准时钟信号

无固定的周期节拍和严格的时钟同步

采用 应答方式

3. 联合控制方式

同步与异步相结合

4. 人工控制方式

(1) Reset

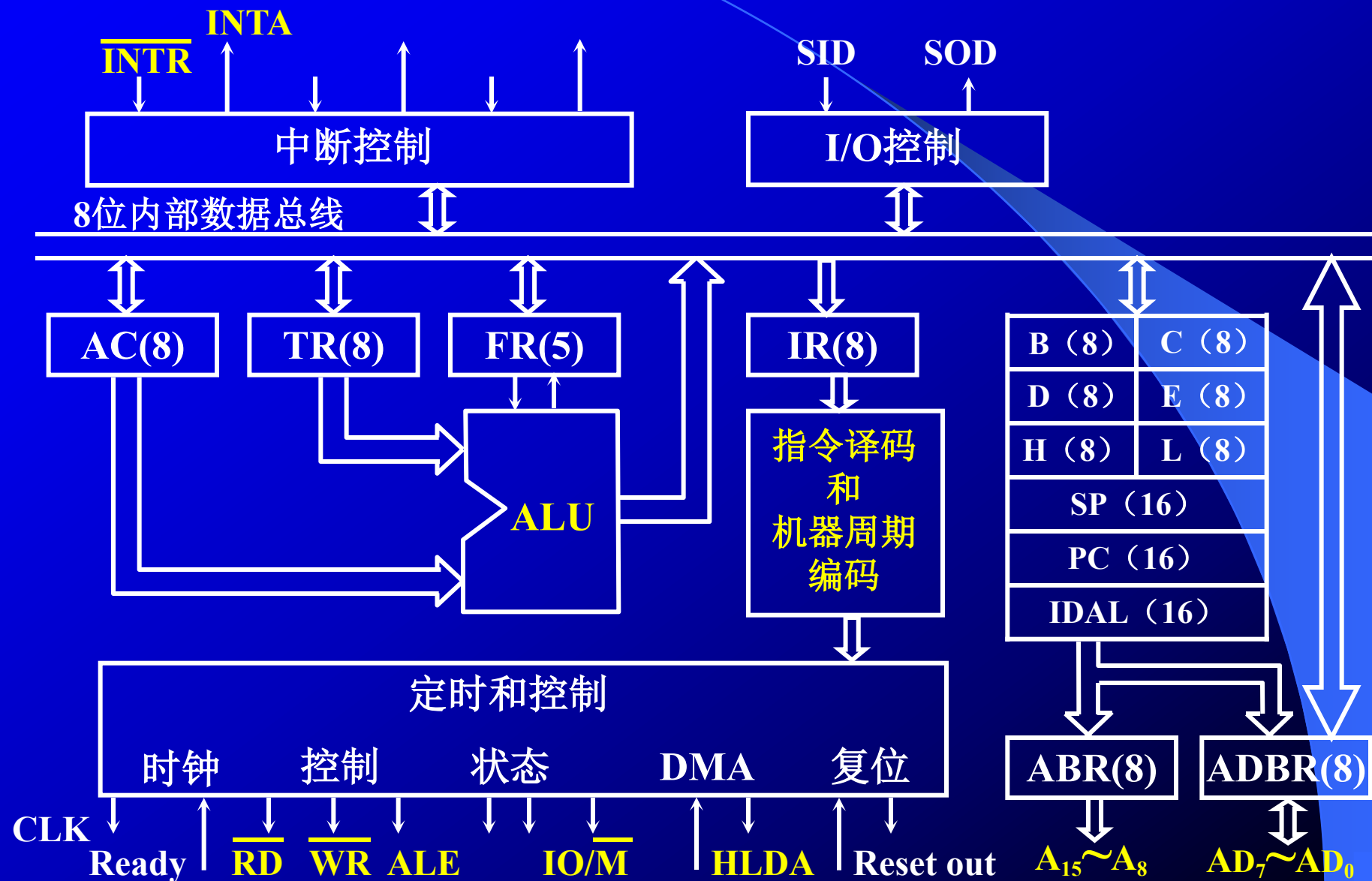
(2) 连续 和 单条 指令执行转换开关

(3) 符合停机开关

五、多级时序系统实例分析

9.2

1. 8085 的组成



2. 8085 的外部引脚

9.2

(1) 地址和数据信号

$A_{15} \sim A_8$ $AD_7 \sim AD_0$

SID SOD

(2) 定时和控制信号

入 X_1 X_2

出 CLK ALE S_0 S_1
 $IO/\overline{M}$ $\overline{RD}$ $\overline{WR}$

(3) 存储器和 I/O 初始化

入 $HOLD$ Ready

出 $HLDA$

X_1	□	1	40	□	V_{CC}
X_2	□	2	39	□	HOLD
Reset out	□	3	38	□	HLDA
SOD	□	4	37	□	CLK(out)
SID	□	5	36	□	$\overline{Rstet\ in}$
Trap	□	6	35	□	Ready
RST7.5	□	7	34	□	$IO/\overline{M}$
RST6.5	□	8	33	□	S_1
RST5.5	□	9	32	□	$\overline{RD}$
$\overline{INTR}$	□	10	31	□	$\overline{WR}$
INTA	□	11	30	□	ALE
AD_0	□	12	29	□	S_0
AD_1	□	13	28	□	A_{15}
AD_2	□	14	27	□	A_{14}
AD_3	□	15	26	□	A_{13}
AD_4	□	16	25	□	A_{12}
AD_5	□	17	24	□	A_{11}
AD_6	□	18	23	□	A_{10}
AD_7	□	19	22	□	A_9
V_{SS}	□	20	21	□	A_8

(4) 与中断有关的信号

入 $\overline{\text{INTR}}$ 出 INTA

Trap 重新启动中断

(5) CPU 初始化

入 $\overline{\text{Reset in}}$ 出 Reset out

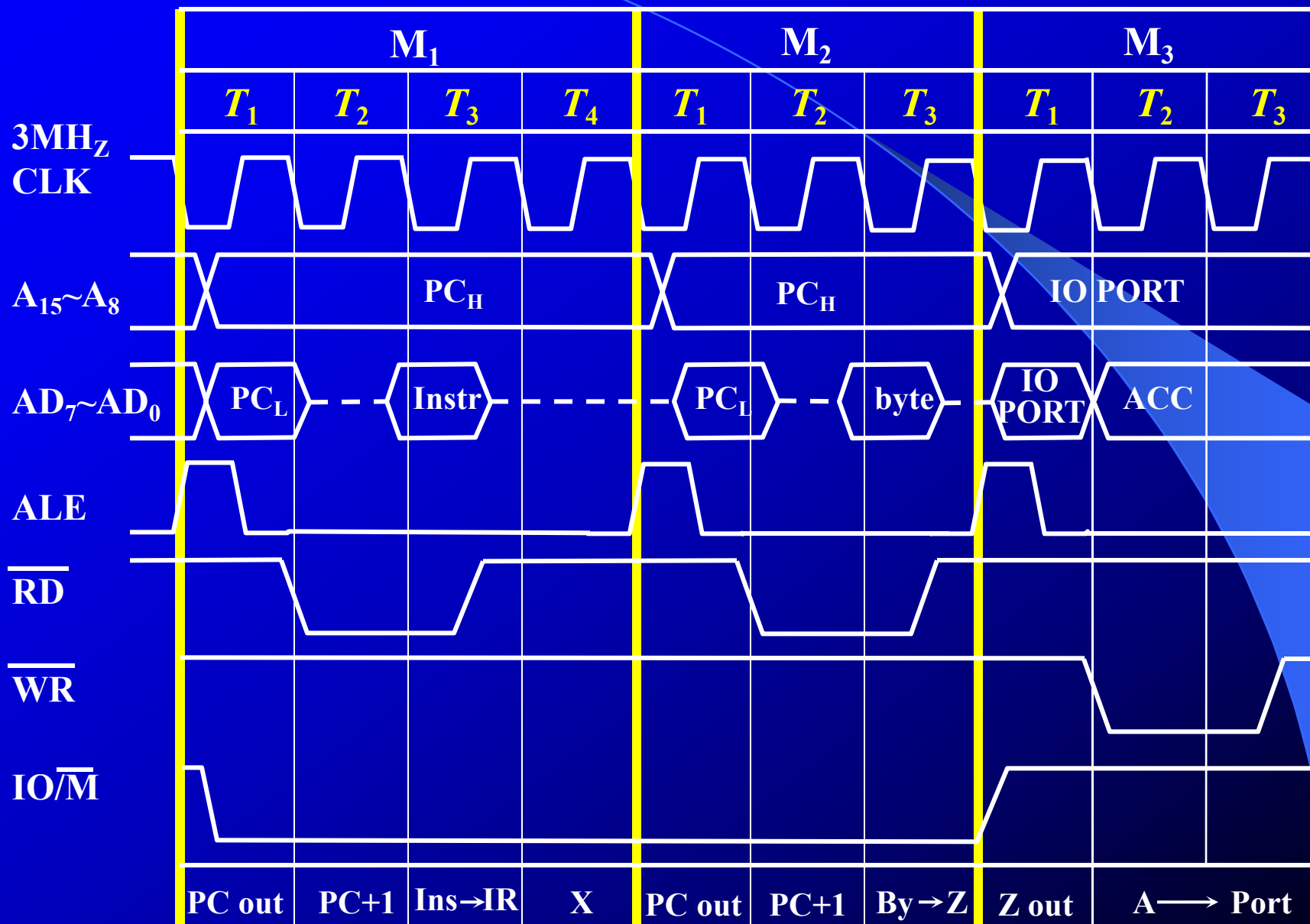
(6) 电源和地

 V_{CC} +5 V V_{SS} 地

X_1	□	1	40	□	V_{CC}
X_2	□	2	39	□	HOLD
Reset out	□	3	38	□	HLDA
SOD	□	4	37	□	$\overline{\text{CLK(out)}}$
SID	□	5	36	□	$\overline{\text{Rreset in}}$
Trap	□	6	35	□	$\overline{\text{Ready}}$
RST7.5	□	7	34	□	IO/M
RST6.5	□	8	33	□	S_1
RST5.5	□	9	32	□	$\overline{\text{RD}}$
$\overline{\text{INTR}}$	□	10	31	□	$\overline{\text{WR}}$
INTA	□	11	30	□	ALE
AD_0	□	12	29	□	S_0
AD_1	□	13	28	□	A_{15}
AD_2	□	14	27	□	A_{14}
AD_3	□	15	26	□	A_{13}
AD_4	□	16	25	□	A_{12}
AD_5	□	17	24	□	A_{11}
AD_6	□	18	23	□	A_{10}
AD_7	□	19	22	□	A_9
V_{SS}	□	20	21	□	A_8

3. 机器周期和节拍（状态）与控制信号的关系

9.2



以一条输出指令（I/O 写）为例

机器周期 M_1 取指令操作码

机器周期 M_2 取设备地址

机器周期 M_3 执行 ACC 的内容写入设备

每个 控制 信号在 指定机器周期 的
指定节拍 T 时刻 发出



第10章 控制单元的设计

10.1 组合逻辑设计

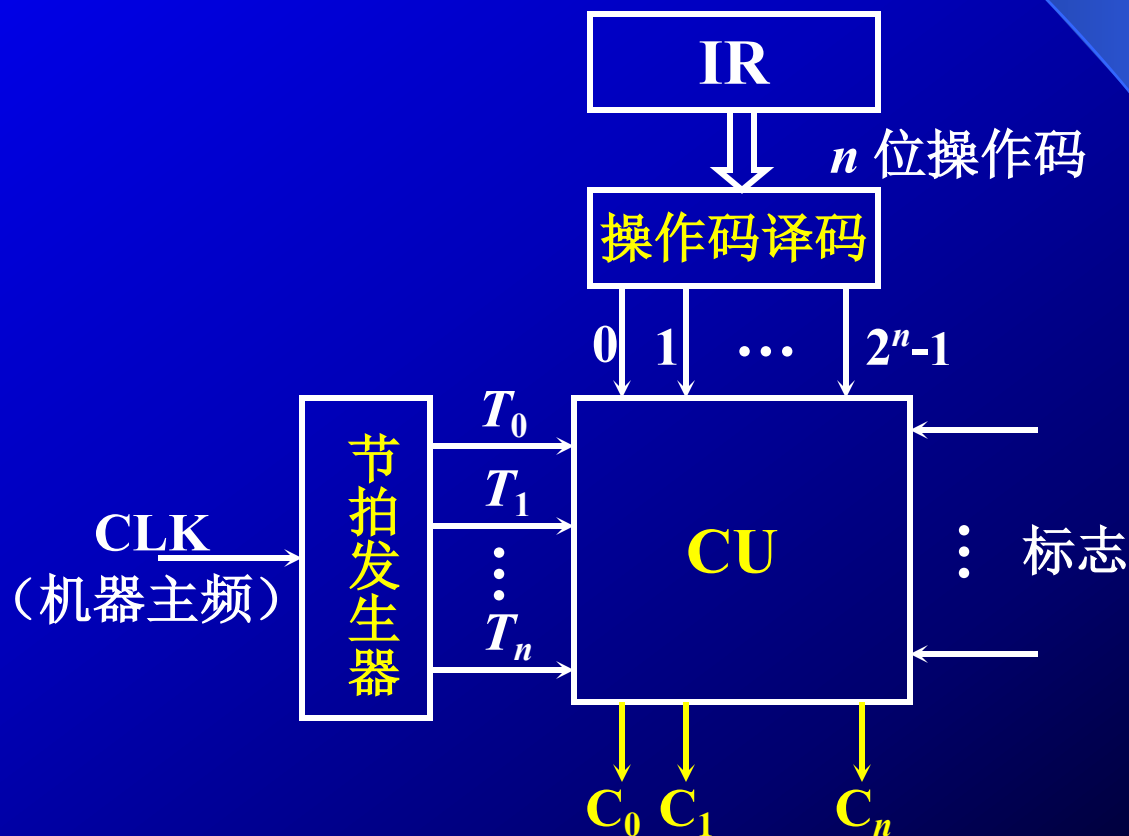
10.2 微程序设计



10.1 组合逻辑设计

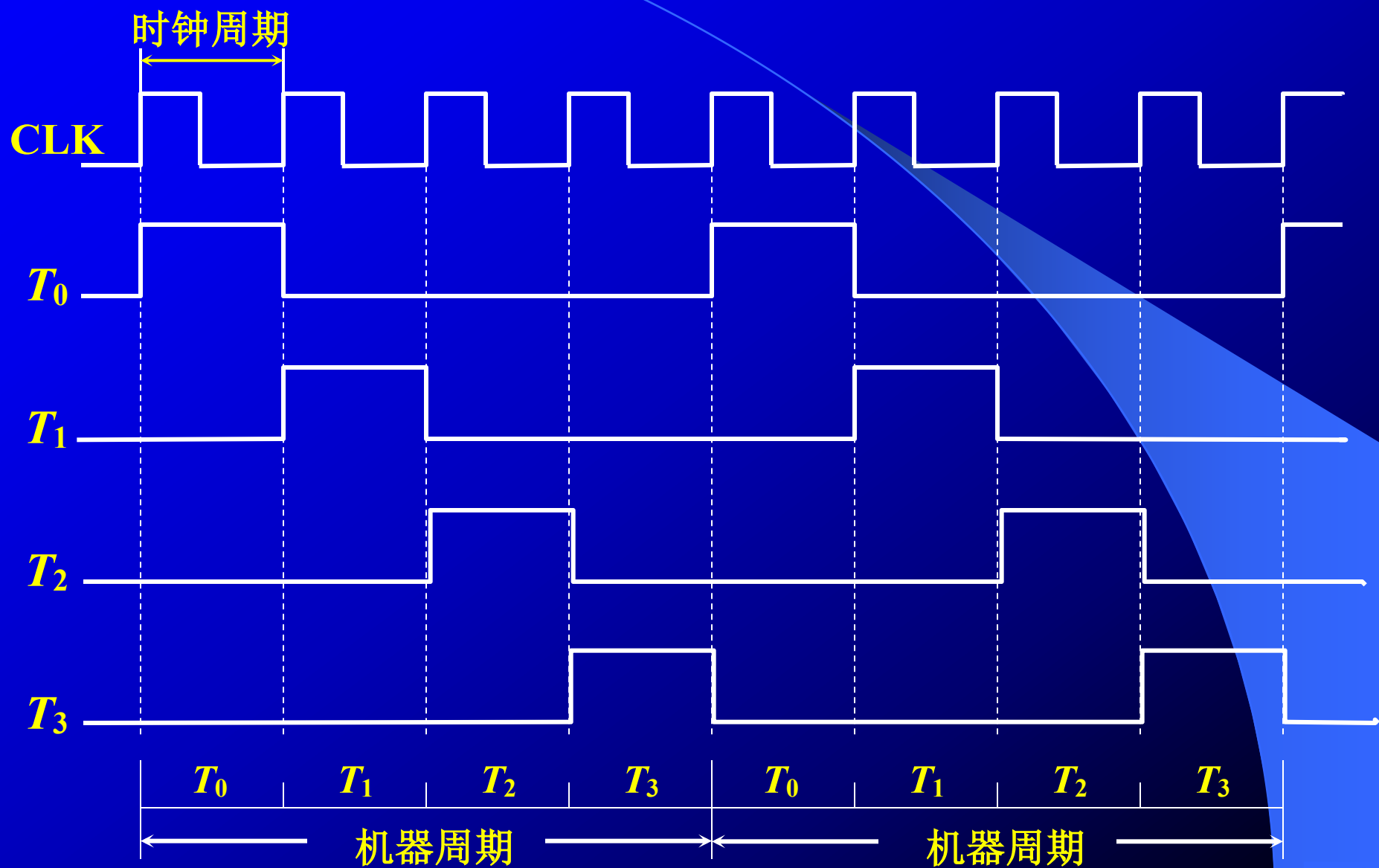
一、组合逻辑控制单元框图

1. CU 外特性



2. 节拍信号

10.1



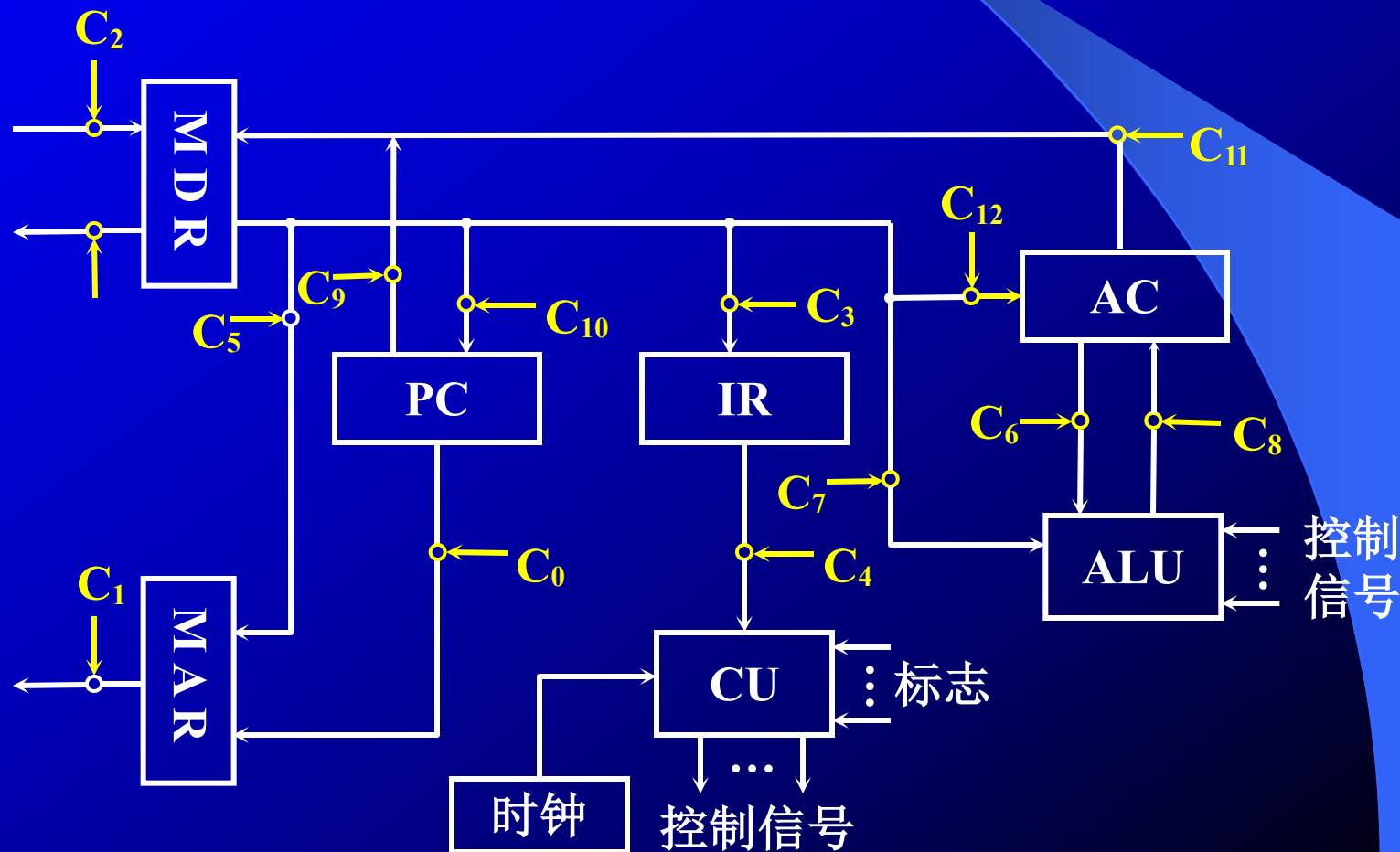
二、微操作的节拍安排

10.1

采用 同步控制方式

一个 机器周期 内有 3个节拍（时钟周期）

CPU 内部结构采用非总线方式



1. 安排微操作时序的原则

原则一 微操作的 先后顺序不得 随意 更改

原则二 被控对象不同 的微操作

尽量安排在一个节拍 内完成

原则三 占用 时间较短 的微操作

尽量 安排在一个节拍 内完成

并允许有先后顺序



2. 取指周期 微操作的 节拍安排

T_0 PC $\longrightarrow$ MAR

1 $\longrightarrow$ R

原则二

T_1 M (MAR) $\longrightarrow$ MDR

(PC) + 1 $\longrightarrow$ PC

原则二

T_2 MDR $\longrightarrow$ IR

OP (IR) $\longrightarrow$ ID

原则三

3. 间址周期 微操作的 节拍安排

T_0 Ad (IR) $\longrightarrow$ MAR

1 $\longrightarrow$ R

T_1 M (MAR) $\longrightarrow$ MDR

T_2 MDR $\longrightarrow$ Ad (IR)



4. 执行周期 微操作的 节拍安排

10.1

① CLA T_0

T_1

T_2 $0 \longrightarrow AC$

② COM T_0

T_1

T_2 $\overline{AC} \longrightarrow AC$

③ SHR T_0

T_1

T_2 $L(AC) \longrightarrow R(AC)$
 $AC_0 \longrightarrow AC_0$



④ CSL T_0
 T_1
 T_2 $R(AC) \longrightarrow L(AC)$ $AC_0 \longrightarrow AC_n$

⑤ STP T_0
 T_1
 T_2 $0 \longrightarrow G$

⑥ ADD X T_0 $Ad(IR) \longrightarrow MAR$ $1 \longrightarrow R$
 T_1 $M(MAR) \longrightarrow MDR$
 T_2 $(AC) + (MDR) \longrightarrow AC$

⑦ STA X T_0 $Ad(IR) \longrightarrow MAR$ $1 \longrightarrow W$
 T_1 $AC \longrightarrow MDR$
 T_2 $MDR \longrightarrow M(MAR)$

⑧ LDA X T_0 $\text{Ad}(\text{IR}) \longrightarrow \text{MAR} \quad 1 \longrightarrow \text{R}$

T_1 $\text{M}(\text{MAR}) \longrightarrow \text{MDR}$

T_2 $\text{MDR} \longrightarrow \text{AC}$

⑨ JMP X T_0

T_1

T_2 $\text{Ad}(\text{IR}) \longrightarrow \text{PC}$

⑩ BAN X T_0

T_1

T_2 $\text{A}_0 \cdot \text{Ad}(\text{IR}) + \bar{\text{A}}_0 \cdot \text{PC} \longrightarrow \text{PC}$

5. 中断周期 微操作的 节拍安排

10.1

T_0 $0 \longrightarrow \text{MAR}$ $1 \longrightarrow \text{W}$ 硬件关中断

T_1 $\text{PC} \longrightarrow \text{MDR}$

T_2 $\text{MDR} \longrightarrow \text{M}(\text{MAR})$ 向量地址 $\longrightarrow \text{PC}$

中断隐指令完成



三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		$PC \rightarrow MAR$						
			$1 \rightarrow R$						
	T_1		$M(MAR) \rightarrow MDR$						
			$(PC) + 1 \rightarrow PC$						
	T_2		$MDR \rightarrow IR$						
			$OP(IR) \rightarrow ID$						
		I	$1 \rightarrow IND$						
		$\bar{I}$	$1 \rightarrow EX$						

间址特征



三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) $\rightarrow$ MAR						
			$1 \rightarrow R$						
	T_1		M(MAR) $\rightarrow$ MDR						
	T_2		MDR $\rightarrow$ Ad (IR)						
		$\overline{IND}$	$1 \rightarrow EX$						

间址周期标志



三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		$Ad(IR) \rightarrow MAR$						
			$1 \rightarrow R$						
			$1 \rightarrow W$						
	T_1		$M(MAR) \rightarrow MDR$						
			$AC \rightarrow MDR$						
	T_2		$(AC) + (MDR) \rightarrow AC$						
			$MDR \rightarrow M(MAR)$						
			$MDR \rightarrow AC$						
			$0 \rightarrow AC$						



三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
FE 取指	T_0		$PC \rightarrow MAR$	1	1	1	1	1	1
			$1 \rightarrow R$	1	1	1	1	1	1
	T_1		$M(MAR) \rightarrow MDR$	1	1	1	1	1	1
			$(PC) + 1 \rightarrow PC$	1	1	1	1	1	1
	T_2		$MDR \rightarrow IR$	1	1	1	1	1	1
			$OP(IR) \rightarrow ID$	1	1	1	1	1	1
		I	$1 \rightarrow IND$			1	1	1	1
		$\bar{I}$	$1 \rightarrow EX$	1	1	1	1	1	1



三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
IND 间址	T_0		Ad (IR) $\rightarrow$ MAR			1	1	1	1
			$1 \rightarrow R$			1	1	1	1
	T_1		M(MAR) $\rightarrow$ MDR			1	1	1	1
	T_2		MDR $\rightarrow$ Ad (IR)			1	1	1	1
		$\overline{\text{IND}}$	$1 \rightarrow \text{EX}$			1	1	1	1



三、组合逻辑设计步骤

10.1

1. 列出操作时间表

工作周期标记	节拍	状态条件	微操作命令信号	CLA	COM	ADD	STA	LDA	JMP
EX 执行	T_0		Ad (IR) $\rightarrow$ MAR			1	1	1	
			1 $\rightarrow$ R			1		1	
			1 $\rightarrow$ W				1		
	T_1		M(MAR) $\rightarrow$ MDR			1		1	
			AC $\rightarrow$ MDR				1		
	T_2		(AC)+(MDR) $\rightarrow$ AC			1			
			MDR $\rightarrow$ M(MAR)				1		
			MDR $\rightarrow$ AC					1	
			0 $\rightarrow$ AC	1					



2. 写出微操作命令的最简表达式

10.1

$M(MAR) \longrightarrow MDR$

$$= FE \cdot T_1 + IND \cdot T_1 (ADD + STA + LDA + JMP + BAN) \\ + EX \cdot T_1 (ADD + LDA)$$

$$= T_1 \{ FE + IND (ADD + STA + LDA + JMP + BAN) \\ + EX (ADD + LDA) \}$$



10.1

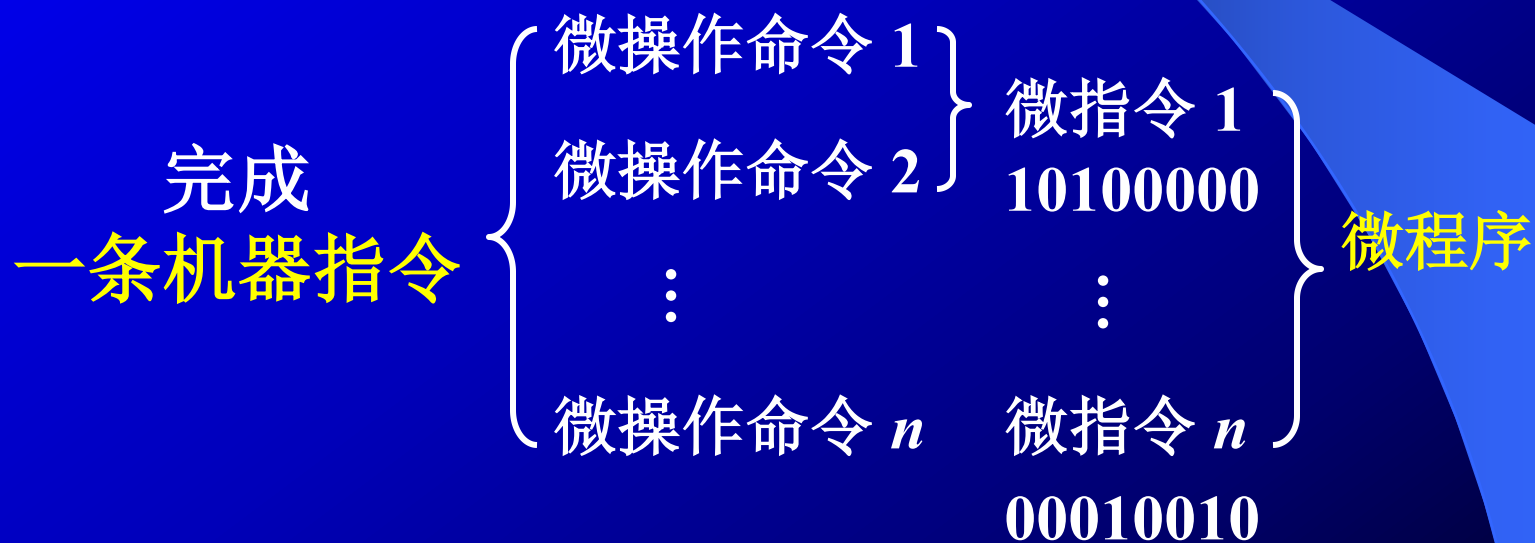


- 思路清晰，简单明了
- 庞杂，调试困难，修改困难
- 速度快 （RISC）

10.2 微程序设计

一、微程序设计思想的产生

1951 英国剑桥大学教授 Wilkes



一条机器指令对应一个微程序

存入 **ROM**

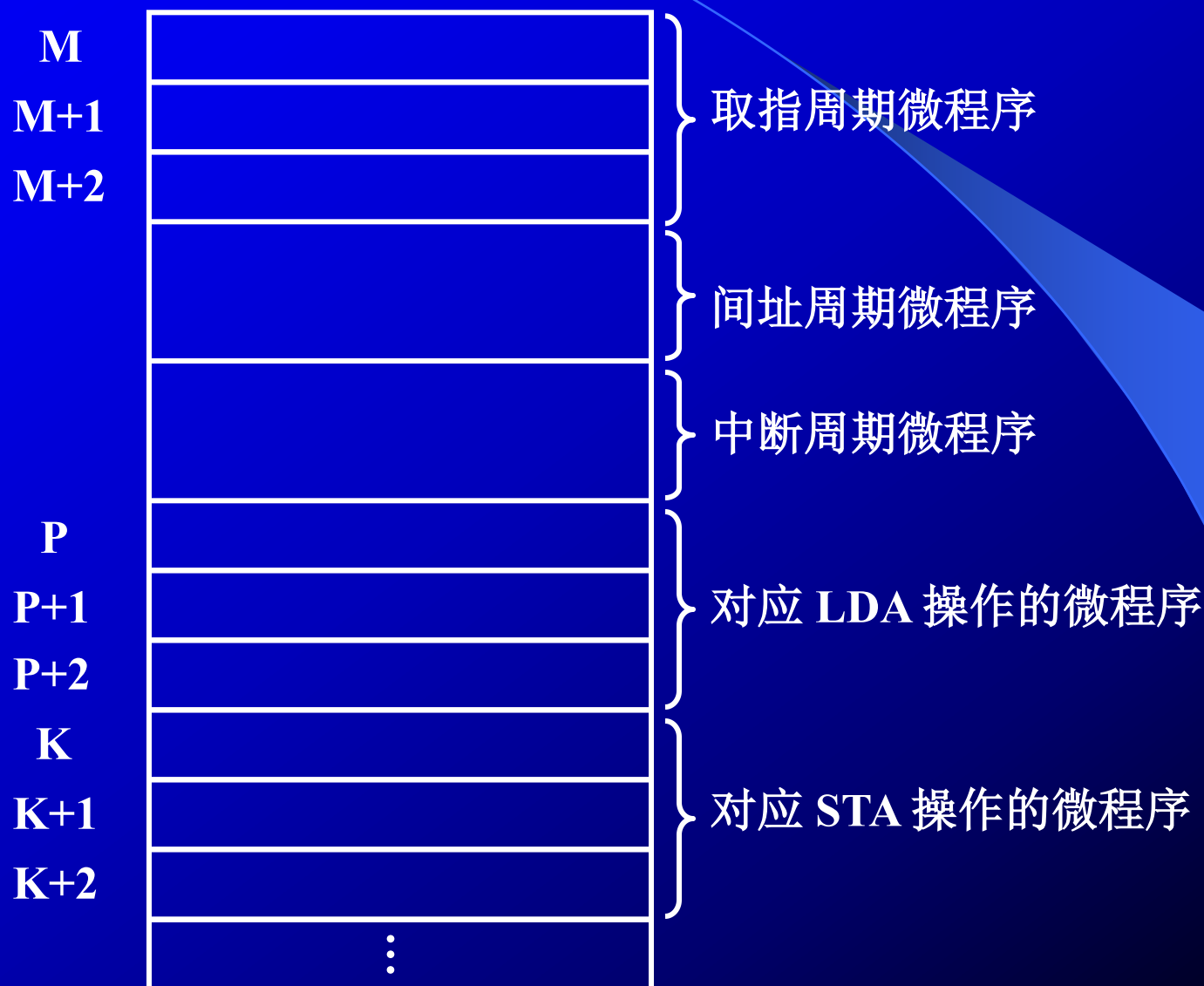
存储逻辑



二、微程序控制单元框图及工作原理

10.2

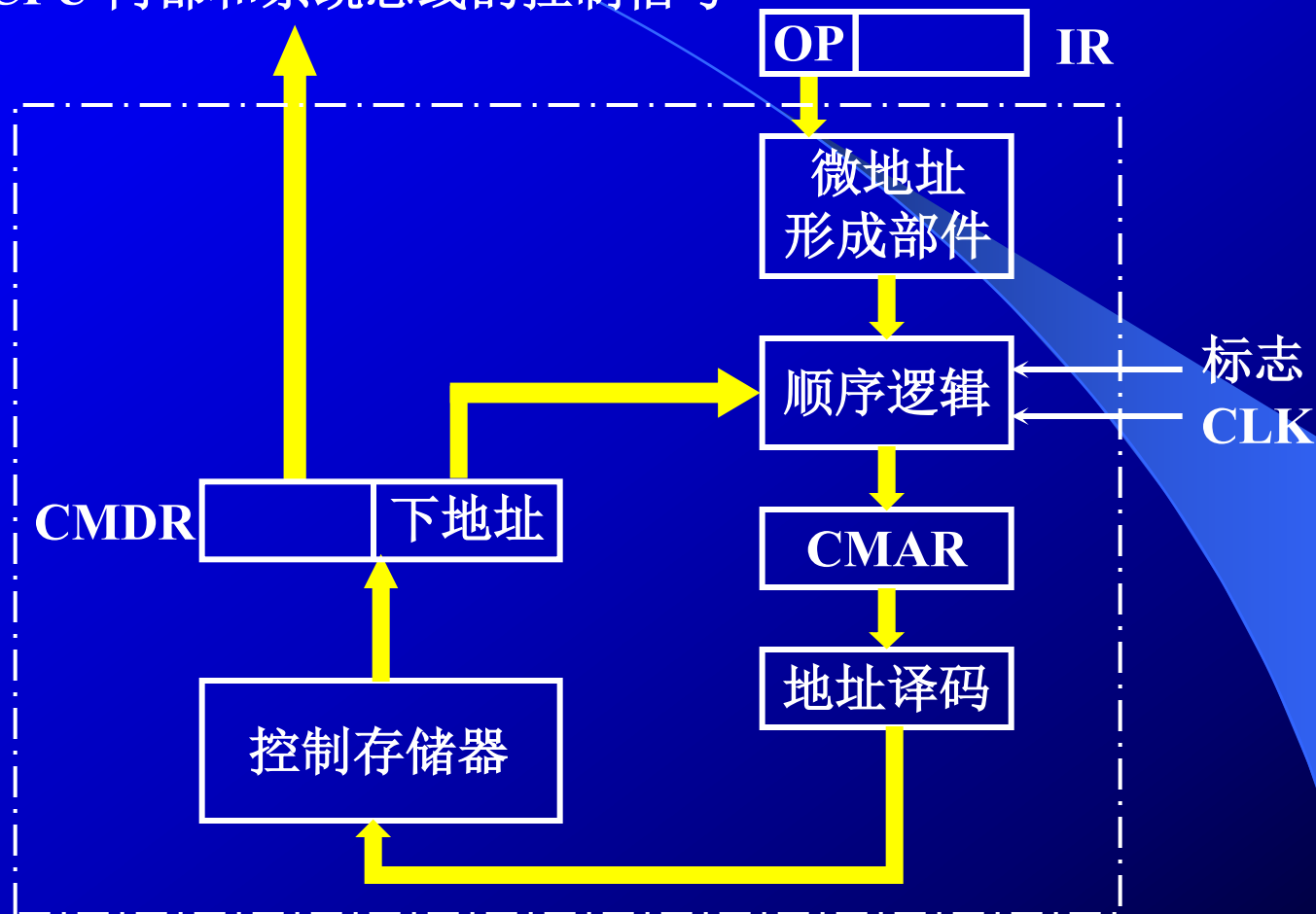
1. 机器指令对应的微程序



2. 微程序控制单元的基本框图

10.2

至 CPU 内部和系统总线的控制信号



微指令基本格式



二、微程序控制单元框图及工作原理

10.2



3. 工作原理

10.2



3. 工作原理

10.2

(1) 取指阶段 执行取指微程序

$M \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

形成下条微指令地址 $M + 1$

$Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

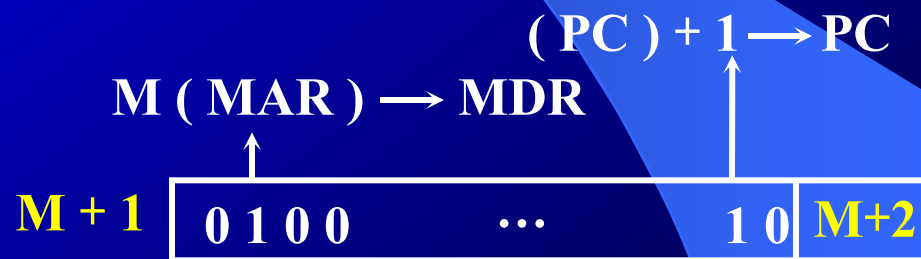
由 CMDR 发命令

形成下条微指令地址 $M + 2$

$Ad(CMDR) \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令



(2) 执行阶段 执行 LDA 微程序

10.2

OP (IR) $\longrightarrow$ 微地址形成部件 $\longrightarrow$ **CMAR** (**P** $\longrightarrow$ **CMAR**)

CM (CMAR) $\longrightarrow$ **CMDR**

由 **CMDR** 发命令

形成下条微指令地址 **Ad (CMDR)** $\longrightarrow$ **CMAR**

CM (CMAR) $\longrightarrow$ **CMDR**

由 **CMDR** 发命令

形成下条微指令地址 **Ad (CMDR)** $\longrightarrow$ **CMAR**

CM (CMAR) $\longrightarrow$ **CMDR**

由 **CMDR** 发命令

形成下条微指令地址 **Ad (CMDR)** $\longrightarrow$ **CMAR**

Ad (IR) $\longrightarrow$ **MAR**

1 $\longrightarrow$ **R**



M (MAR) $\longrightarrow$ **MDR**



MDR $\longrightarrow$ **AC**



(**M** $\longrightarrow$ **CMAR**)



(3) 取指阶段 执行取指微程序

$M \rightarrow CMAR$

$CM(CMAR) \rightarrow CMDR$

由 CMDR 发命令

⋮



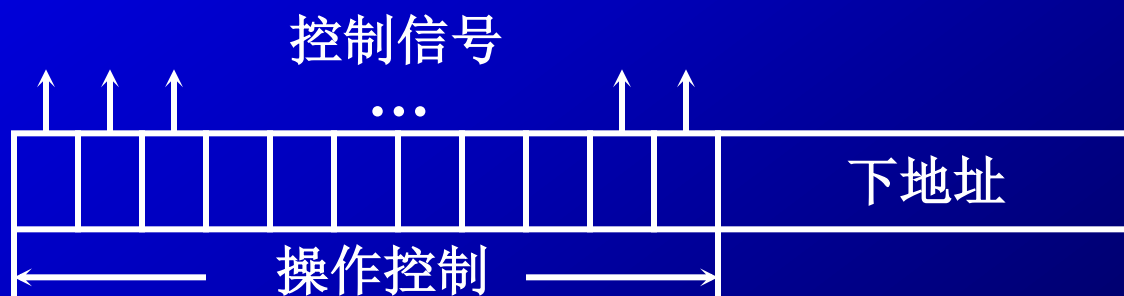
全部微指令存在 CM 中，程序执行过程中 只需读出

- 关键
- 微指令的 操作控制字段如何形成微操作命令
 - 微指令的 后续地址如何形成

三、微指令的编码方式（控制方式）

1. 直接编码（直接控制）方式

在微指令的操作控制字段中，
每一位代表一个微操作命令

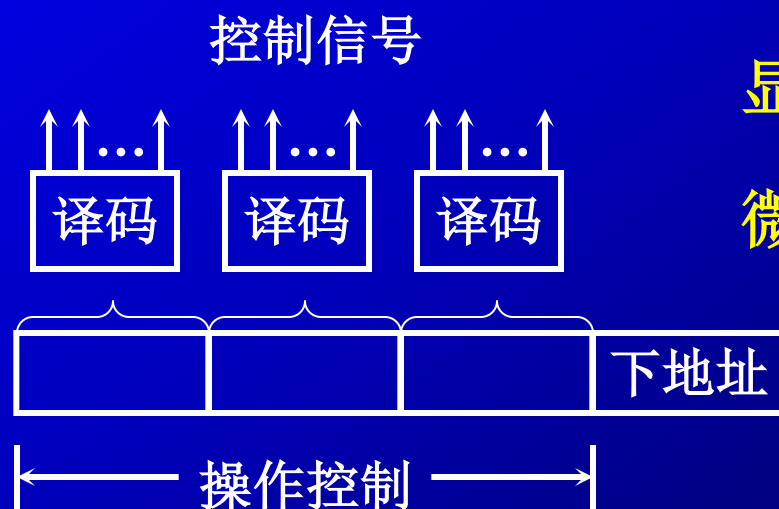


速度最快

某位为 “1” 表示该控制信号有效

2. 字段直接编码方式

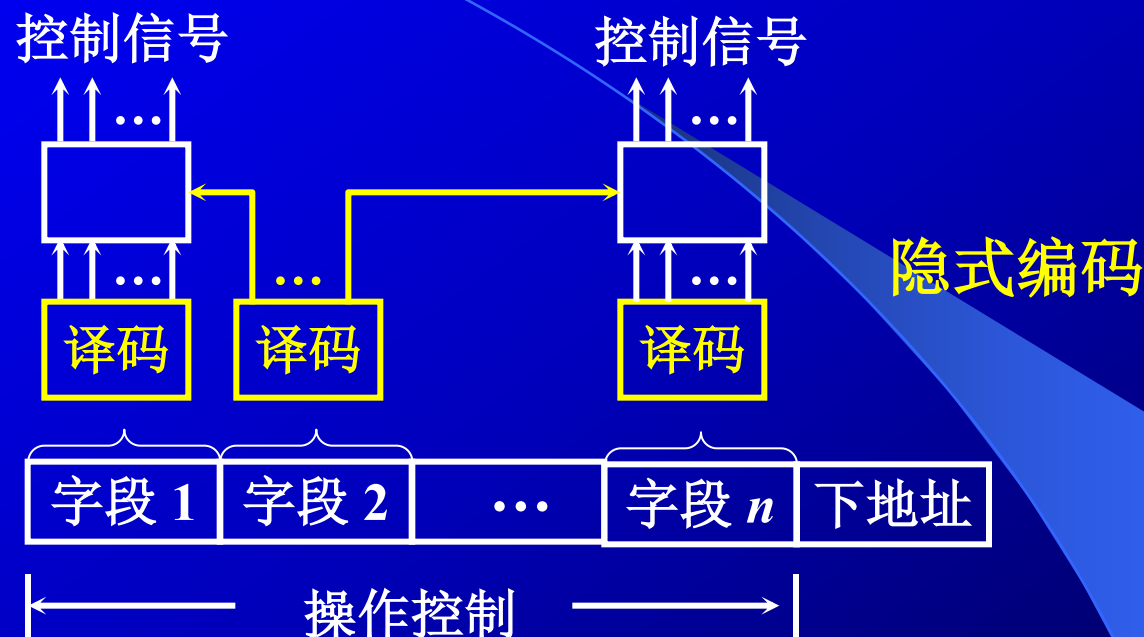
将微指令的控制字段分成若干“段”，
每段经译码后发出控制信号



每个字段中的命令是 **互斥** 的

缩短 了微指令 **字长**，**增加** 了译码 **时间**

3. 字段间接编码方式



4. 混合编码

直接编码和字段编码（直接和间接）混合使用

5. 其他

四、微指令序列地址的形成

10.2

1. 微指令的 **下地址字段** 指出
2. 根据机器指令的 **操作码** 形成
3. **增量计数器**

$$(CMAR) + 1 \rightarrow CMAR$$

4. 分支转移

操作控制字段	转移方式	转移地址
--------	------	------

转移方式

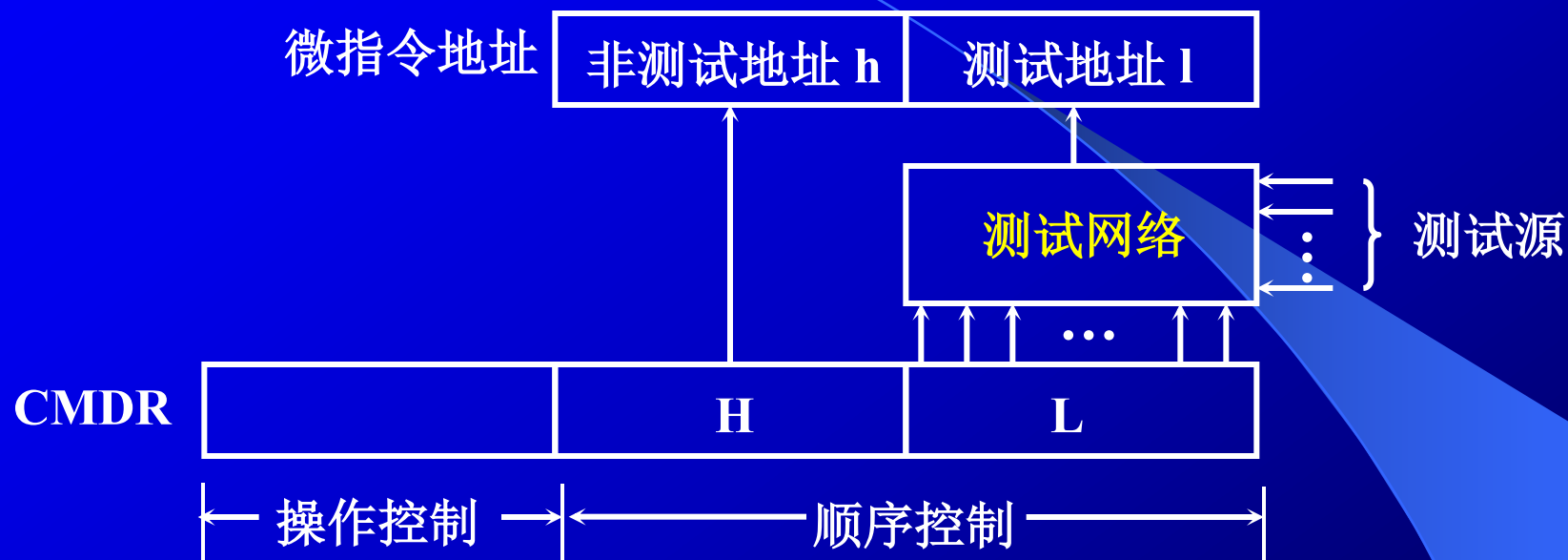
指明判别条件

转移地址

指明转移成功后的去向



5. 通过测试网络



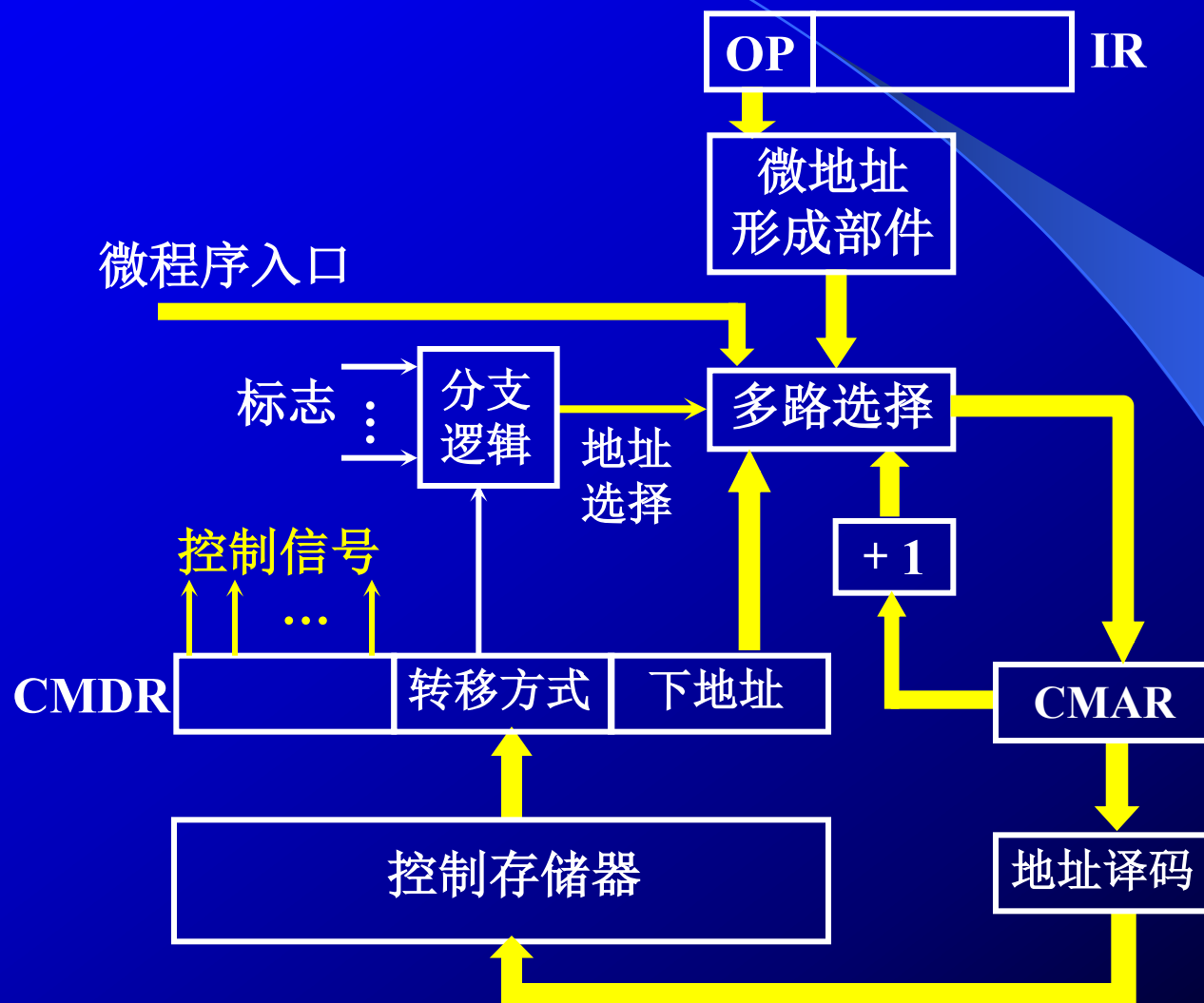
6. 由硬件产生微程序入口地址

第一条微指令地址 由专门 **硬件** 产生

中断周期 由 **硬件** 产生 **中断周期微程序首地址**

7. 后续微指令地址形成方式原理图

10.2



五、微指令格式

1. 水平型微指令

一次能定义并执行多个并行操作

如 直接编码、字段直接编码、字段间接编码、
直接和字段混合编码

2. 垂直型微指令

类似机器指令操作码 的方式

由微操作码字段规定微指令的功能



3. 两种微指令格式的比较

- (1) 水平型微指令比垂直型微指令 并行操作能力强，
灵活性强
- (2) 水平型微指令执行一条机器指令所要的
微指令 数目少，速度快
- (3) 水平型微指令 用较短的微程序结构换取较长的
微指令结构
- (4) 水平型微指令与机器指令 差别大



六、静态微程序设计和动态微程序设计

10.2

静态 微程序无须改变, 采用 **ROM**

动态 通过 **改变微指令** 和 **微程序** 改变机器指令,
有利于仿真, 采用 **EPROM**

七、毫微程序设计

1. 毫微程序设计的基本概念

微程序设计 用 **微程序** 解释机器指令

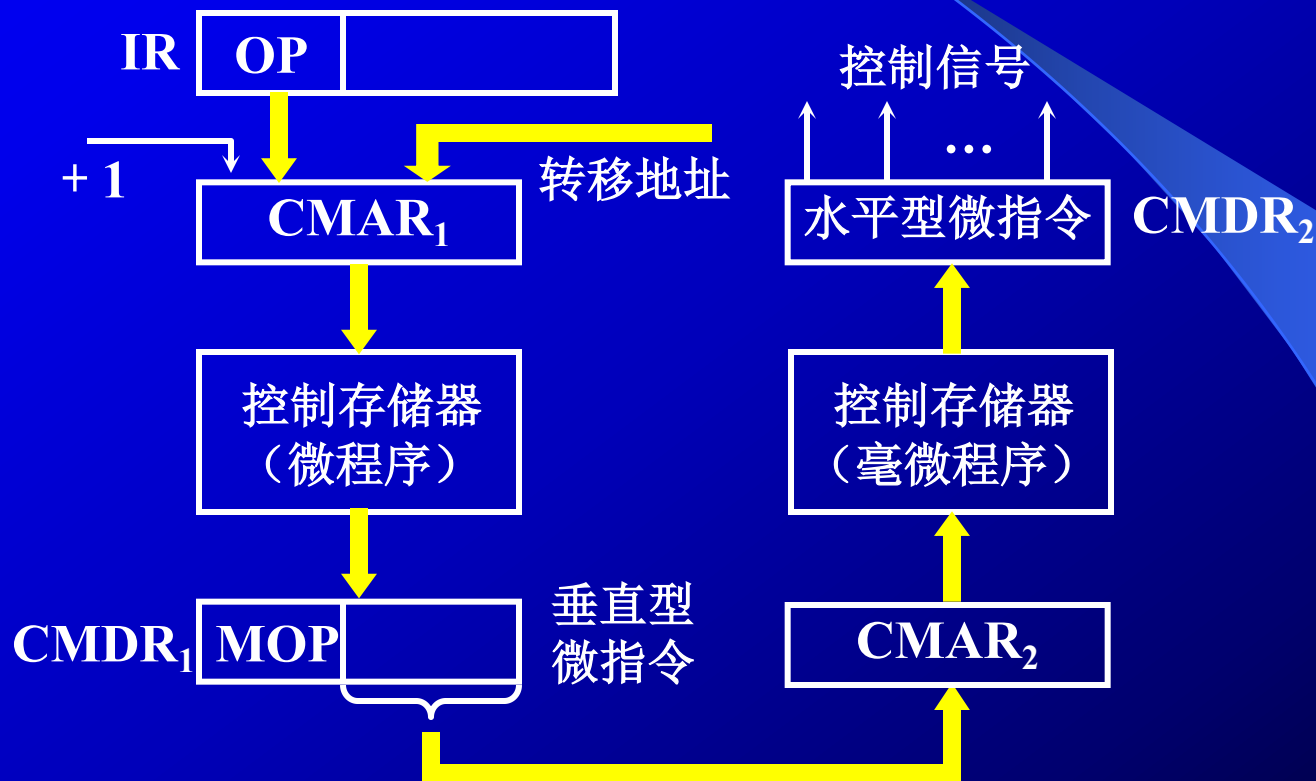
毫微程序设计 用 **毫微程序** 解释微程序

毫微指令与微指令 的关系好比 **微指令与机器指令** 的关系



2. 毫微程序控制存储器的基本组成

10.2



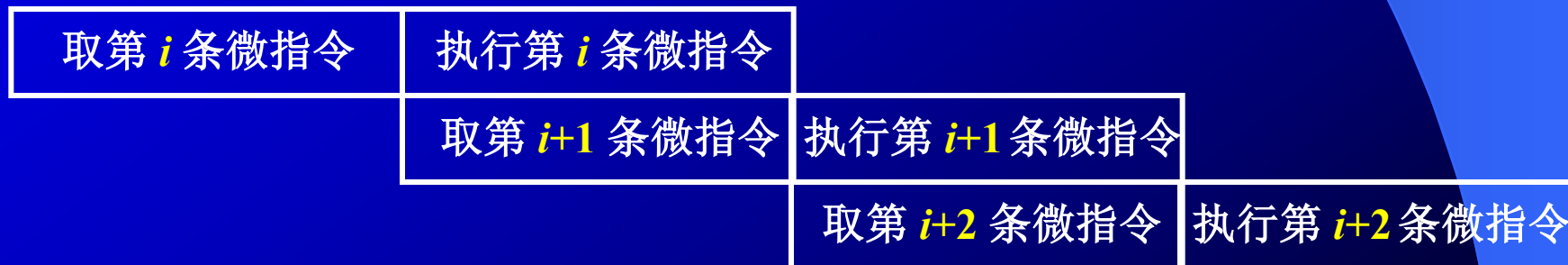
八、串行微程序控制和并行微程序控制

10.2

串行 微程序控制



并行 微程序控制



九、微程序设计举例

10.2

1. 写出对应机器指令的微操作及节拍安排

假设 CPU 结构与组合逻辑相同

(1) 取指阶段微操作分析

3 条微指令

T_0 $PC \rightarrow MAR$ $1 \rightarrow R$

T_1 $M(MAR) \rightarrow MDR$ $(PC) + 1 \rightarrow PC$

T_2 $MDR \rightarrow IR$ $OP(IR) \rightarrow$ 微地址形成部件

需考虑如何安排这条微指令？

则取指操作需 3 条微指令

$Ad(CMDR) \rightarrow CMAR$

$OP(IR) \rightarrow$ 微地址形成部件 $\rightarrow CMAR$



(2) 取指阶段的微操作及节拍安排

考虑到需要 形成后续微指令的地址

T_0 $PC \longrightarrow MAR$ $1 \longrightarrow R$

T_1 $Ad (CMDR) \longrightarrow CMAR$

T_2 $M (MAR) \longrightarrow MDR$ $(PC)+1 \longrightarrow PC$

T_3 $Ad (CMDR) \longrightarrow CMAR$

T_4 $MDR \longrightarrow IR$ $OP (IR) \longrightarrow$ 微地址形成部件

T_5 $OP (IR) \longrightarrow$ 微地址形成部件 $\longrightarrow CMAR$

(3) 执行阶段的微操作及节拍安排

10.2

考虑到需形成后续微指令的地址

取指微程序的入口地址 M
由微指令下地址字段指出

- 非访存指令

- ① CLA 指令

$T_0 \quad 0 \longrightarrow AC$

$T_1 \quad \text{Ad (CMDR)} \longrightarrow \text{CMAR}$

- ② COM 指令

$T_0 \quad \overline{AC} \longrightarrow AC$

$T_1 \quad \text{Ad (CMDR)} \longrightarrow \text{CMAR}$



③ SHR 指令

$$T_0 \quad L(AC) \longrightarrow R(AC) \quad AC_0 \longrightarrow AC_0$$
$$T_1 \quad \mathbf{Ad(CMDR) \longrightarrow CMAR}$$

④ CSL 指令

$$T_0 \quad R(AC) \longrightarrow L(AC) \quad AC_0 \longrightarrow AC_n$$
$$T_1 \quad \mathbf{Ad(CMDR) \longrightarrow CMAR}$$

⑤ STP 指令

$$T_0 \quad 0 \longrightarrow G$$
$$T_1 \quad \mathbf{Ad(CMDR) \longrightarrow CMAR}$$

• 访存指令

10.2

⑥ ADD 指令

T_0 $\text{Ad}(\text{IR}) \longrightarrow \text{MAR}$ $1 \longrightarrow \text{R}$

T_1 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

T_2 $\text{M}(\text{MAR}) \longrightarrow \text{MDR}$

T_3 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

T_4 $(\text{AC}) + (\text{MDR}) \longrightarrow \text{AC}$

T_5 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

⑦ STA 指令

T_0 $\text{Ad}(\text{IR}) \longrightarrow \text{MAR}$ $1 \longrightarrow \text{W}$

T_1 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

T_2 $\text{AC} \longrightarrow \text{MDR}$

T_3 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

T_4 $\text{MDR} \longrightarrow \text{M}(\text{MAR})$

T_5 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$



⑧ LDA 指令

10.2

T_0 $\text{Ad (IR)} \longrightarrow \text{MAR}$ $1 \longrightarrow \text{R}$

T_1 **$\text{Ad (CMDR)} \longrightarrow \text{CMAR}$**

T_2 $\text{M (MAR)} \longrightarrow \text{MDR}$

T_3 **$\text{Ad (CMDR)} \longrightarrow \text{CMAR}$**

T_4 $\text{MDR} \longrightarrow \text{AC}$

T_5 **$\text{Ad (CMDR)} \longrightarrow \text{CMAR}$**



• 转移类指令

⑨ JMP 指令

T_0 $\text{Ad}(\text{IR}) \longrightarrow \text{PC}$

T_1 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

⑩ BAN 指令

T_0 $A_0 \cdot \text{Ad}(\text{IR}) + \bar{A}_0 \cdot (\text{PC}) \longrightarrow \text{PC}$

T_1 $\text{Ad}(\text{CMDR}) \longrightarrow \text{CMAR}$

全部微操作 20个

微指令 38条



2. 确定微指令格式

(1) 微指令的编码方式

采用直接控制

(2) 后续微指令的地址形成方式

由机器指令的操作码通过微地址形成部件形成

由微指令的下地址字段直接给出

(3) 微指令字长

由 20 个微操作

确定 操作控制字段 最少 20 位

由 38 条微指令

确定微指令的 下地址字段 为 6 位

微指令字长 可取 $20 + 6 = 26$ 位



(4) 微指令字长的确定

10.2

38 条微指令中有 19 条
是关于后续微指令地址 $\longrightarrow$ CMAR

其中 $\left\{ \begin{array}{ll} 1 \text{ 条} & \text{OP (IR)} \longrightarrow \text{微地址形成部件} \longrightarrow \text{CMAR} \\ 18 \text{ 条} & \text{Ad (CMDR)} \longrightarrow \text{CMAR} \end{array} \right.$

若用 Ad (CMDR) 直接送控存地址线

则省去了输至 CMAR 的时间, 省去了 CMAR

同理 $\text{OP (IR)} \longrightarrow \text{微地址形成部件} \longrightarrow \text{控存地址线}$

可省去 19 条微指令, 2 个微操作

$$38 - 19 = 19$$

下地址字段最少取 5 位

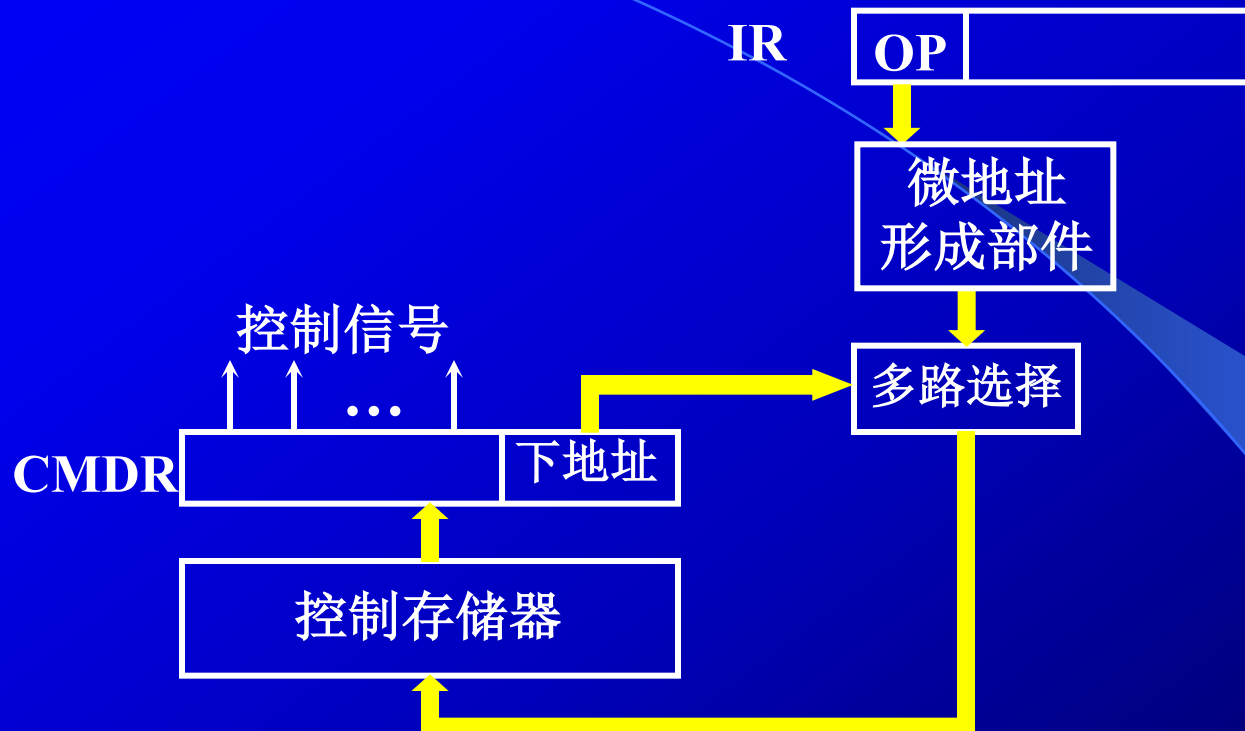
$$20 - 2 = 18$$

操作控制字段最少取 18 位



(5) 省去了 CMAR 的控制存储器

10.2



考虑留有一定的余量

取操作控制字段
下地址字段

18 位 → 24 位
5 位 → 6 位 } 共 30 位

(6) 定义微指令操作控制字段每一位的微操作



3. 编写微指令码点

10.2

微程序 名称	微指令 地址 (八进制)	微指令（二进制代码）														
		操作控制字段									下地址字段					
取指		0	1	2	3	4	...	10	...	23	24	25	26	27	28	29
	00	1	1								0	0	0	0	0	1
	01			1	1						0	0	0	0	1	0
	02					1					×	×	×	×	×	×
CLA	03										0	0	0	0	0	0
COM	04										0	0	0	0	0	0
ADD	10		1					1			0	0	1	0	0	1
	11			1							0	0	1	0	1	0
	12										0	0	0	0	0	0
LDA	16		1					1			0	0	1	1	1	1
	17			1							0	1	0	0	0	0
	20										0	0	0	0	0	0